Multi-Rail Isolated Power Supply Monitoring System

Capstone Design Documentation

Michael Capotosto

November 3, 2026

Rev. ET450-W8

Contents

1 Introduction **1**

1.1 Project Overview 1

1.2 Project Motivation 2

1.3 System Architecture 2

1.4 Design Objectives 2

1.5 Project Scope 3

1.6 Report Organization 4

2 Problem Background and Definition **5**

2.1 Purpose and Scope 5

2.2 Problem Definition 5

2.3 Proposed Solution 6

3 Technology Solution **7**

3.1 Standards and Design Considerations 7

3.2 Performance Summary 7

4 Chosen Conceptual Design **9**

4.0.1 Functional Block Diagram 10

5 Potential Social, Ethical, Legal, and Environmental Impacts of the Project **11**

5.1 Societal Impact 11

5.2 Global Impact 11

5.3 Legal and Regulatory Impact 12

5.4 Ethical Impact 12

5.5 Environmental Impact 13

5.6 Relationship to the Original Requirements 13

6 Design Methodology **15**

6.1 Engineering Analysis 15

6.2 Design Trade-offs 15

6.3 Broader Considerations 16

7 Simulation Results **17**

8 Final Design **19**

8.1 Hardware Design 19

8.2 Software and Communication 19

8.3 Final Design Documentation 20

8.3.1 Functional Block Diagram 21

8.4 Implementation Modifications 22

9 Solution Implementation **23**

9.1 Testing the LTC2945 23

9.1.1 Calculating Shunt Value for Test 24

9.1.2 Pictures of Evaluation Board, Display, and Phoebus Panels 25

9.2 Bring Up and Testing of the First Spin Board 29

9.3 Assembly Verification/Visual Inspection and Smoke Test 30

9.3.1 Visual Inspection 30

9.3.2 Smoke Test 31

9.4 I^2C SDA Communication Problem 31

9.4.1 Initial Symptoms 31

9.4.2 SDA Signal Troubleshooting 32

9.4.3 Identification of Incorrect Component 33

9.4.4 Inverter Bodge Modification 33

9.4.5 Verification After Modification 34

9.4.6 Further Bodges and Repairs 36

10 Evaluation of Solution's Efficacy **41**

10.1 Automated Functional Tests 41

10.1.1 System Under Test . . . 41
10.1.2 Scope of Testing . . . 42
10.1.3 Test Equipment and Software . . . 43
10.1.4 Test Configuration . . . 44
10.1.5 System Acceptance Criteria . . . 44
10.1.6 System Test Matrix . . . 46
10.1.7 Detailed Test Procedures . . . 48
10.1.8 Automated Test Data Collection . . . 51
10.1.9 Test Execution and Result Classification . . . 52
10.1.10 Suspension and Resumption Criteria . . . 52
10.1.11 Test Responsibilities . . . 53
10.1.12 Testing Risks and Contingencies . . . 53
10.1.13 Test Deliverables . . . 54
10.1.14 Test Completion Criteria . . . 54
10.1.15 Test Reports . . . 55

11 Improvements and Optimizations to Solution 57

12 Conclusions 59

12.0.1 Did the Project Meet the Requirements of the Scope? . . . 59
12.0.2 Was the Project Completed on Time? . . . 59
12.0.3 Was the Project Completed Under Budget? . . . 60
12.0.4 Did the Quality and Performance Meet Stakeholder Expectations? . . . 60
12.0.5 Overall Project Assessment . . . 61

13 References 63

References 63

A Functional Specifications 65

B Design Calculations and Circuit Analysis 67

B.1 Current-Sense Shunt Resistor Sizing . . . 67
B.1.1 Current Measurement Resolution . . . 68

B.1.2 Production Shunt Design Table 68
B.1.3 Evaluation-Board Shunt Calculation 68
B.2 PCB Trace and Copper-Pour Current Capacity 69
B.3 I^2C Pull-Up Resistor Sizing 69
B.4 LTC2945 Voltage Measurement Resolution 69
B.5 Negative-Rail Sign Convention 70
B.6 Alarm-Limit Scaling 70
B.7 Controller-Supply Power Budget 70
B.8 Component Capability Checks 71
B.9 Current-Sense Resistor Power Margin 71
B.10 LTC2945 Input and ADC Range 71
B.11 Microcontroller I/O Compatibility 72
B.12 Digital-Isolation Margin 72
B.13 3.3 V Regulator Loading 72
B.14 Tolerance and Measurement-Uncertainty Considerations 72
B.15 Reference Specifications Used for Capability Checks 73

C Design Documents **75**
C.1 Schematics 75
C.1.1 Sensor Board Schematic 75
C.1.2 Arduino Shield Board Schematic 83
C.2 Software Flow Chart 85
C.2.1 Arduino GIGA Firmware Flowchart 86
C.2.2 PSC and EPICS Communication Flow 88
C.2.3 Software Configuration Management 88
C.3 Source Code 89
C.3.1 Firmware Source Files 89
C.3.2 Firmware Header Files 142
C.3.3 EPICS IOC 150

D System Test Plan **441**
D.1 Introduction 441
D.1.1 System Under Test 441

D.1.2 Scope of Testing . . . 442
D.1.3 Test Equipment and Software . . . 443
D.1.4 Test Configuration . . . 444
D.1.5 System Acceptance Criteria . . . 444
D.1.6 System Test Matrix . . . 446
D.1.7 Detailed Test Procedures . . . 448
D.1.8 Automated Test Data Collection . . . 451
D.1.9 Test Execution and Result Classification . . . 452
D.1.10 Suspension and Resumption Criteria . . . 452
D.1.11 Test Responsibilities . . . 453
D.1.12 Testing Risks and Contingencies . . . 453
D.1.13 Test Deliverables . . . 454
D.1.14 Test Completion Criteria . . . 454
D.1.15 Test Plan and Results . . . 455

E Week 3 Assignment, Part B in re: Child Labor 481

F Design Standards 483
F.1 Standards and Specifications Used in the Design . . . 483
F.1.1 IPC-2152 Printed-Circuit Conductor Sizing . . . 483
F.1.2 I^2C Bus Specification . . . 483
F.1.3 IEEE 802.3 Ethernet . . . 484
F.1.4 Isolation Component Safety Ratings . . . 484
F.2 Societal Impact . . . 484
F.3 Global Impact . . . 485
F.4 Legal and Regulatory Impact . . . 485
F.5 Ethical Impact . . . 486
F.6 Environmental Impact . . . 486
F.7 Relationship to the Original Requirements . . . 487

G Role of Physics and Other Sciences 489

H Employer Letter 491

I Updated Component List **493**

I.1 Bill of Materials 495

I.1.1 Sensor Board Bill of Materials 496

I.1.2 Arduino Shield Board Bill of Materials 500

I.1.3 System Costs 502

J Installing EPICS Base and IOCs **503**

J.1 Introduction 503

J.2 EPICS Installation Overview 503

J.3 Installing EPICS Base and Required Modules 503

J.4 Installing the PSU Monitor IOC 505

K CS-Studio - Phoebus **507**

K.1 Introduction 507

K.2 Installing Phoebus 507

Chapter 1

Introduction

1.1 Project Overview

The Multi-Rail Isolated Power Supply Monitoring System is a hardware and software system developed to provide centralized monitoring of multiple electrically isolated power-supply rails. The system measures both output voltage and load current, provides configurable alarm limits, displays operating status and measurement data via a local display or ethernet for integration with an EPICS-based control system.

The completed system monitors six independent power-supply rails:

- +5VA
- +5VB
- +5V
- -5V
- +15V
- -15V

Because the monitored supplies may be electrically isolated from one another, the monitoring system must preserve that isolation while still allowing all six channels to communicate with a common controller. Each supply rail therefore uses an independent voltage/current monitoring channel and an isolated digital communication path.

The primary controller is an Arduino GIGA R1 WiFi. The controller acquires measurement data from the individual monitor channels, converts the data to engineering units, applies the appropriate rail polarity, evaluates configured alarm limits, and presents the resulting information on a GIGA Display Shield. An Ethernet interface provides network communication between the embedded controller and an EPICS Input/Output Controller (IOC). The EPICS IOC then makes the measurements, alarms, and system status available to a CS-Studio Phoebus operator interface.

1.2 Project Motivation

Power-supply systems containing multiple isolated voltage rails present a monitoring challenge because each circuit associated with one supply can't share a direct electrical reference with another supply. Connecting conventional measurement or communication circuits across these references can break isolation between the rails.

A centralized monitor is desirable because it allows the user to observe all supply voltages and currents from a single interface, identify abnormal operating conditions, detect communication or sensor faults, and integrate the power-supply status into a larger control system.

The design developed for this project addresses these requirements by performing voltage and high-side current measurement locally on each monitored rail and transferring the resulting information across electrically isolated digital interfaces. The controller receives measurement data from all six rails without requiring their power references to be directly connected together.

1.3 System Architecture

The monitoring system is divided into three primary functional areas: the power-supply monitoring hardware, the microcontroller, and the supervisory control interface.

The power supply monitoring hardware contains the current-sense resistors, LTC2945 PMICs, and digital isolation circuitry. Each LTC2945 measures the supply voltage and the differential voltage developed across a high-side current-sense resistor. The resulting digital measurement information is sent over the I^2C bus while maintaining the required electrical isolation.

The Arduino GIGA R1 WiFi provides the mcu processing functions. Its firmware initializes the monitoring channels, acquires voltage and current measurements, detects communication errors, converts raw measurements to engineering units, applies channel-specific polarity and scaling, evaluates alarm limits, and updates the local display. Configuration information, including network parameters and alarm limits, is retained in persistent storage so that required settings can survive a controller reset or power cycle.

Network communication is provided through an Ethernet interface using the PSC protocol. The EPICS IOC communicates with the embedded controller using PSCDriver and converts the transmitted values into EPICS process variables. CS-Studio Phoebus provides the remote operator interface for viewing measurements, alarm conditions, connection status, and system state.

1.4 Design Objectives

The major design objectives for the Multi-Rail Isolated Power Supply Monitoring System are to:

1. Measure the voltage of six independent power-supply rails.
2. Measure the load current of each rail using high-side current sensing.
3. Preserve electrical isolation between independently referenced power-supply rails and the common controller.
4. Convert acquired data into meaningful engineering units and correctly represent both positive and negative voltage rails.
5. Detect I^2C communication failures and prevent invalid sensor data from being presented as valid measurements.
6. Compare voltage and current measurements against configurable upper and lower alarm limits.
7. Provide a local graphical display of voltage, current, alarm status, communication status, and network state.
8. Retain configuration and alarm settings through controller resets and power cycles.
9. Provide Ethernet communication for remote monitoring and configuration.
10. Integrate the monitoring system with an EPICS IOC and CS-Studio Phoebus operator interface.
11. Maintain the hardware, firmware, and control-system software under revision control so that changes made during design, implementation, and testing remain traceable.

1.5 Project Scope

The scope of the project includes the design and implementation of the sensor interface hardware, Arduino GIGA controller hardware, embedded firmware, Ethernet communication interface, EPICS IOC, and operator-interface software. The hardware portion includes the six voltage/current monitoring channels, current-sense resistors, digital isolation circuitry, controller interface, Ethernet circuitry, and supporting power-supply circuitry.

The software portion includes the embedded acquisition and alarm-processing firmware, persistent configuration storage, local display interface, PSC network communication, EPICS IOC configuration and database records, and the CS-Studio Phoebus operator interface.

Design files and software associated with the project are maintained in the project Git repository:

`https://github.com/capotosto/PS_Monitor`

The repository provides revision control for the hardware design, firmware, EPICS IOC files, documentation, and other project artifacts generated during development.

1.6 Report Organization

This report documents the development of the monitoring system from preliminary design through implementation and testing. The preliminary-design chapter defines the major functional blocks, presents the system architecture, documents supporting engineering calculations, includes the hardware schematics and bills of materials, and describes the primary firmware and control-system software flows.

Subsequent chapters document prototyping and development activities, the EPICS IOC implementation, and the CS-Studio Phoebus operator interface. Supporting project-management and design documents developed during the earlier phases of the capstone project are retained as part of the overall project documentation and provide traceability between the original requirements, design decisions, implementation, and final system.

Chapter 2

Problem Background and Definition

2.1 Purpose and Scope

This chapter documents the hardware and software design of the Multi-Rail Isolated Power Supply Monitoring System. The design monitors six power-supply rails, measures rail voltage and load current, preserves electrical isolation between floating supplies and the controller, evaluates measurements against user-configurable alarm limits, presents local status on the controller display, and publishes measurements and status to an EPICS control-system interface.

This design is intended to establish the major analog and digital functional blocks, document supporting engineering calculations, define the hardware schematic baseline, and provide software flowcharts detailed enough to support implementation.

2.2 Problem Definition

The project addresses the need to monitor several low-voltage power-supply rails that do not necessarily share a common reference. A conventional monitoring circuit that directly connects the grounds together would defeat the electrical isolation of the supplies and could create unwanted return paths. However, the user needs a single location from which to observe rail voltage, rail current, alarm state, communication status, and the overall condition of the power system.

The required monitoring system must therefore make accurate measurements on six separate rails while preserving isolation between the monitored domains. It must support both positive and negative rails, detect sensor or communication faults, provide configurable voltage and current limits, retain configuration information after reset, present useful local status, and make the same data available to the facility control system over Ethernet. The monitored rails in the present design are +5 V A, +5 V B, +/-5 V, and +/-15 V.

2.3 Proposed Solution

The proposed solution assigns one LTC2945 voltage/current monitor to each power rail. Load current is converted to a small differential voltage by a precision high-side shunt resistor and the LTC2945 digitizes both the shunt voltage and the rail voltage. Each sensor remains referenced to its own monitored supply domain. The I^2C communication path is then isolated so that digital measurement data can be transferred to a common Arduino GIGA R1 WiFi controller without directly bonding the isolated supply returns together.

The Arduino GIGA performs channel acquisition, engineering-unit conversion, negative-rail sign handling, alarm evaluation, configuration storage, display updates, and network communication. A local GIGA Display Shield provides immediate status at the hardware. Ethernet transports measurement and status data using the PSC protocol to an EPICS IOC, where PSCDriver and EPICS records make the values available to a CS-Studio Phoebus operator interface. This architecture separates the rail-side analog measurement functions from the controller and supervisory-control functions while maintaining a single user interface for the complete system.

Chapter 3

Technology Solution

This implemented solution is a six-channel isolated power supply monitoring system with capabilities of local and remote voltage, current, alarm and communication status monitoring.

Each channel is based on a LTC2945 power monitor IC and current-sense resistor. Optoisolators maintain the isolation of the monitored supplies. An Arduino GIGA R1 WiFi board is used for processing the measurements, alarm limit evaluation, and visualization on the local touch screen display. Ethernet connectivity allows the user to integrate the system with an EPICS IOC and Phoebus operator interface.

3.1 Standards and Design Considerations

Standard I2C communication, Ethernet TCP/IP networking and EPICS Channel Access are incorporated in this design. Electrical design considerations include operational constraints of components, PCB trace sizing, current-sense resistor power consumption and isolation needs. Guidance of IPC-2152 was considered for PCB trace size selection, while the electrical operational constraints come from the component datasheets.

This system is designed for laboratory use. Network interfaces of this prototype do not provide the necessary security features for deployment in untrusted networks. No formal product safety, EMC and regulatory certification was conducted on the prototype.

3.2 Performance Summary

Testing of the system was performed to evaluate the measurement accuracy, alarm operation, configuration retention and communication. Voltage and current measurements were compared with reference instruments according to the criteria provided in Chapter 10.

The prototype implements all the desired monitoring capabilities, but testing revealed measurement errors and some start-up and recovery problems that require further evaluation.

Chapter 4

Chosen Conceptual Design

The major system functions are summarized below.

1. **Power-Supply Interface:** Connect to the six monitored rails and route each rail through the associated voltage/current monitoring channel.
2. **Current Measurement:** Convert load current to a differential voltage across a precision shunt resistor and digitize it with the LTC2945.
3. **Voltage Measurement:** Measure the monitored rail voltage using the LTC2945 supply-voltage ADC channel.
4. **Electrical Isolation:** Maintain galvanic isolation between each floating monitored supply and the common controller-side I^2C bus.
5. **Digital Acquisition:** Poll each monitor, identify communication faults, apply channel-specific sign/scaling information, and calculate engineering-unit values.
6. **Alarm Processing:** Compare voltage and current values against configured upper and lower limits and generate channel alarm states.
7. **Local User Interface:** Present all rail measurements, alarms, network state, and communication status on the Arduino GIGA Display.
8. **Network/EPICS Interface:** Publish measurements, alarm limits, alarm state, and connection status over Ethernet for the EPICS IOC and Phoebus interface.

4.0.1 Functional Block Diagram

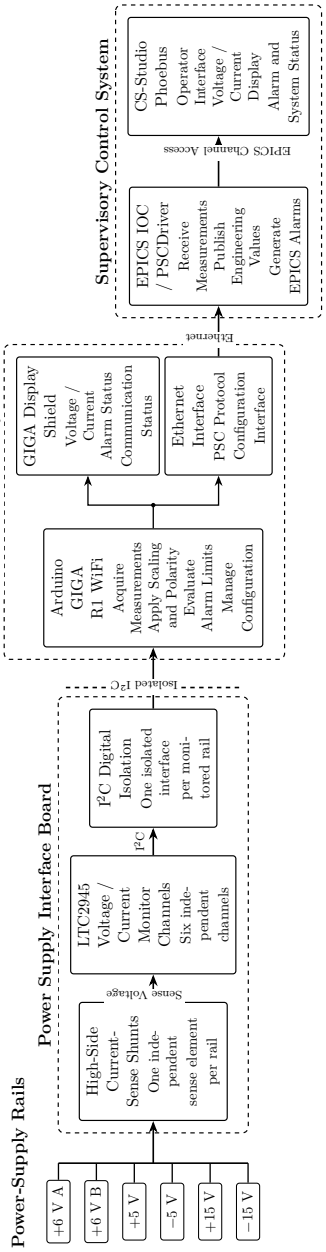


Figure 4.1: Functional block diagram of the multi-rail isolated power supply monitoring system. Six independent power-supply rails are measured using high-side current sensing and LTC2945 voltage/current monitoring channels. Digital isolation preserves electrical independence between the monitored supplies and the common controller. The Arduino GIGA processes the measurements, provides a local operator display, and transmits measurement and status information over Ethernet to the EPICS IOC and CS-Studio Phoebus operator interface.

Chapter 5

Potential Social, Ethical, Legal, and Environmental Impacts of the Project

5.1 Societal Impact

The principal societal impact of the monitoring system is the improved data availability. The six isolated power supplies that would otherwise have to be monitored individually can now be measured from one local display and from the EPICS control system. Abnormal voltage, current, or communication conditions can be detected faster. In a laboratory or industrial setting, this can reduce troubleshooting time and help to avoid operation of the equipment with a power supply failure.

There is also a obligation associated with presenting measurements to an operator. Erroneous data can be more problematic than unavailable data if an operator believes that a failed measurement is a good one. For this reason, communication failure detection was included in the design. The firmware distinguishes an failed or offline sensor from a legitimate zero measurement, and presents the communication state together with the measured values. This was an important design decision since the monitor is meant to present information which an operator can rely on for making maintenance and operational decisions.

5.2 Global Impact

The design is dependent on a global electronics supply chain. ICs, passive components, PCB fabrication, and commercial Arduino modules can be manufactured in several different countries even when they are supplied through United States distributors. This gives access to a wide range of components and lower manufacturing costs, but at the same time creates risks associated with component availability, obsolescence, shipping disruptions, and supplier dependence.

The design mitigates some of these risks through the use of standard component packages, conventional PCB assembly, repetitive circuit blocks, and standard interfaces such as I^2C

and Ethernet. makes qualification of replacement components or alternate manufacturing sources much easier than it would be with highly proprietary hardware.

Global sourcing creates an ethical obligation that is not completely covered by component price and technical performance. For a production version of the monitor, the selection of a supplier should also include consideration of responsible labor practices, material sourcing, regulatory compliance, and traceability of the supplier. These considerations considerations would become increasingly important for large scale production of the design.

5.3 Legal and Regulatory Impact

This project is an engineering prototype and can't be considered to have any system-level regulatory approval solely because individual components carry approvals or compliance markings. If the monitor were converted into a commercial product, the required regulatory evaluation would depend on the installation, enclosure, power source, network interfaces, and markets in which it would be sold.

For example, no attention was paid to RoHS compliance, as the boards are intentionally assembled with leaded solder, so leaded HASL or ENIG were both acceptable ordering options. Use of the monitor with leaded solder however, would be an issue for many markets with RoHS mandates.

EMC, product-safety, and radio-frequency requirements would also have to be identified for the final installation before the production product could be offered for sale. EMC lab compliance testing can be extremely expensive.

The distinction between component compliance and system compliance is therefore important. The design can take advantage of the recognized components and accepted engineering practice without implying any approval not obtained for the finished assembly. This can most easily be related to the idea of "UL Listed" and "UL Registered" (UL vs UR labels) - UL listed components are certified on their own standing, but UR components need to be certified in the installation, and are only a recognized component of a system.

5.4 Ethical Impact

The first ethical obligation associated with the monitor is the accurate and unambiguous presentation of the measured information. The system should not hide sensor failures or substitute erroneous data with apparently valid measurements. The implemented communication failure detection, alarm indication, and separate sensor failure reporting were included partly for this reason.

The next consideration is network security. The current HTTP and PSC interfaces are designed for use in a controlled local network. They do not have security features necessary to be connected to an untrusted network. If the design were deployed in an unprotected network, additional requirements for authentication, access control, encryption, network seg-

mentation, and configuration management should be dfined. Any possibility of unauthorized modification of alarm limits or the network configuration would undermine the reliability of the monitoring system, even if the measurement hardware would continue operating properly.

Ethical considerations also also relevant to procurement. Cost and availability are important engineering considerations, but they should not be the only criteria for selecting a production supplier. Labor practices, material sourcing, counterfeit parts risk, and the ability to provide traceable parts are also appropriate considerations for a product moving from prototype quantities into production.

5.5 Environmental Impact

The main environmental impacts of the monitor come from PCB fabrication, semiconductor manufacturing, raw-materials, transportation, electricity consumption, and eventual electronic waste. It's impossible to make a design with zero environmental impact, but some aspects of this design reduce the quantity of hardware that would need to be disposed of during the useful service life of the product.

The Sensor Board, Arduino Shield, Arduino GIGA, display, and other major assemblies are separable. Therefore, a failed subsystem can therefore be repaired or replaced independent of the complete monitoring system. Standard component packages and commercially available modules were preferred where possible, and this increases the probability of replacement of an individual component. Alarm limits and network configuration are stored in software which allows changing the configuration without producing a new PCB.

For production quantities, RoHS-compliant components should be used where possible. The end-of-life plan should include recovery or recycling of the electronic assemblies. These considerations align with RoHS and WEEE requirements to encourage collection, reuse, recovery, and recycling of electronic equipment.

5.6 Relationship to the Original Requirements

Several of the societal and ethical considerations described above were indirectly represented in the original requirements definition. Electrical isolation was explicitly required to allow the monitoring of one rail would not defeat the isolation of the other. The communication fault detection was required to ensure erroneous data would not be interpreted as a valid electrical measurement. User-configurable alarm limits, local status indication, EPICS monitoring, and persistent configuration storage were also included as functional requirements. These requirements increase the reliability, maintainability, and the quality of the information presented to the operator.

Environmental considerations were introduced later in the design through serviceability, component reuse, and the use of common parts. Regulatory certification, ethical supplier selection, and cybersecurity requirements were not among major items in the original re-

quirements definition. If the prototype were developed into a production product, these aspects should be transformed into explicit requirements with acceptance criteria.

Chapter 6

Design Methodology

The design process involved an iterative engineering method starting with requirements determination and preliminary design and moving on to detailed circuit design, prototyping, implementation, and testing.

6.1 Engineering Analysis

Component selection, schematic drawing, PCB designing, power budgeting, current sense resistor calculation, and electrical operating limitations verification were some of the tasks performed during the preliminary and detailed designs.

The calculations based on Ohm's law, power dissipation, ADC resolution, component tolerance values, and signal isolation techniques were used to determine the circuit requirements. Calculations are provided in Appendix B.

6.2 Design Trade-offs

Among the main trade-offs of the design were resolution, power dissipation, isolation, cost, and development time.

For measuring current, high precision shunt resistors were selected balancing resolution against voltage and power dissipation. Independent isolated channels were selected to maintain the isolation/floating rails of the measured supplies although this entailed extra circuitry.

For speeding up the development process, Arduino GIGA platform with its set of peripherals and display capability was used. Wired Ethernet was chosen for ease of integration into a control system.

6.3 Broader Considerations

In addition to that, safety of electricity use, accuracy and reliability of measurements, maintainability, cybersecurity, and responsible components sourcing were among other considerations of the design which will be addressed in Chapter 5.

Both preliminary and detailed design were then improved through prototyping process discovering some hardware and communication issues which required changes including changes to the I2C circuit.

Chapter 7

Simulation Results

Simulations were performed using LTSpice in a DC Operating Point simulation to verify the calculation for the full-load shunt voltage for each of the voltages. Only the positive polarity was tested in the simulation, as negative polarity will naturally not matter to a a passive element such as the resistor.
The data collected in this simulation matches the data found in Appendix A section B.1, table B.1

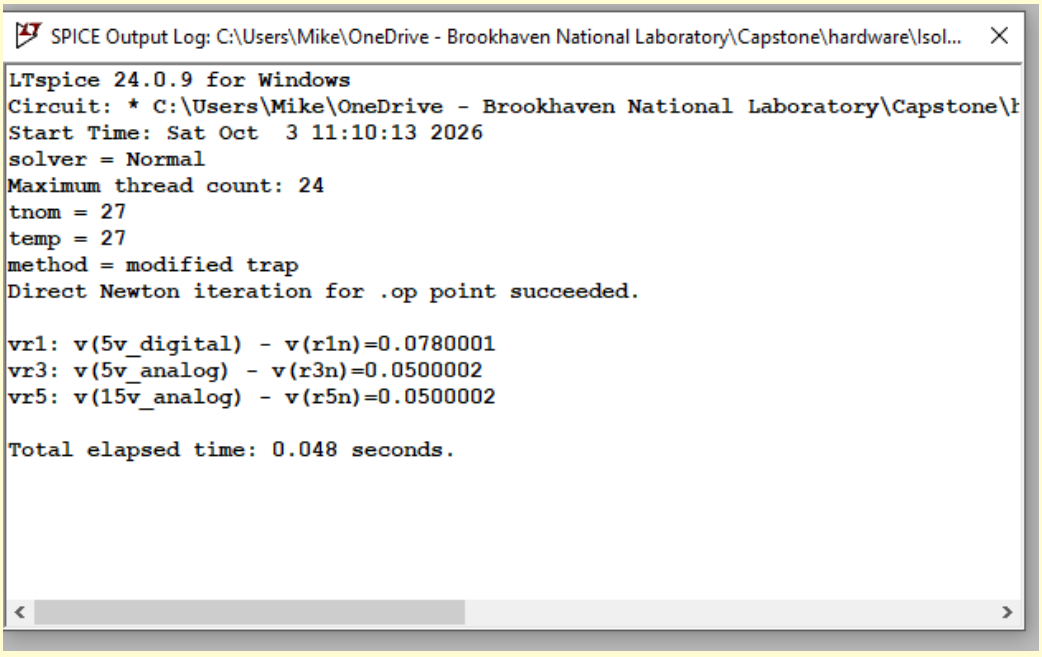


Figure 7.1: Simulation verifying current shunt voltage drops, data log

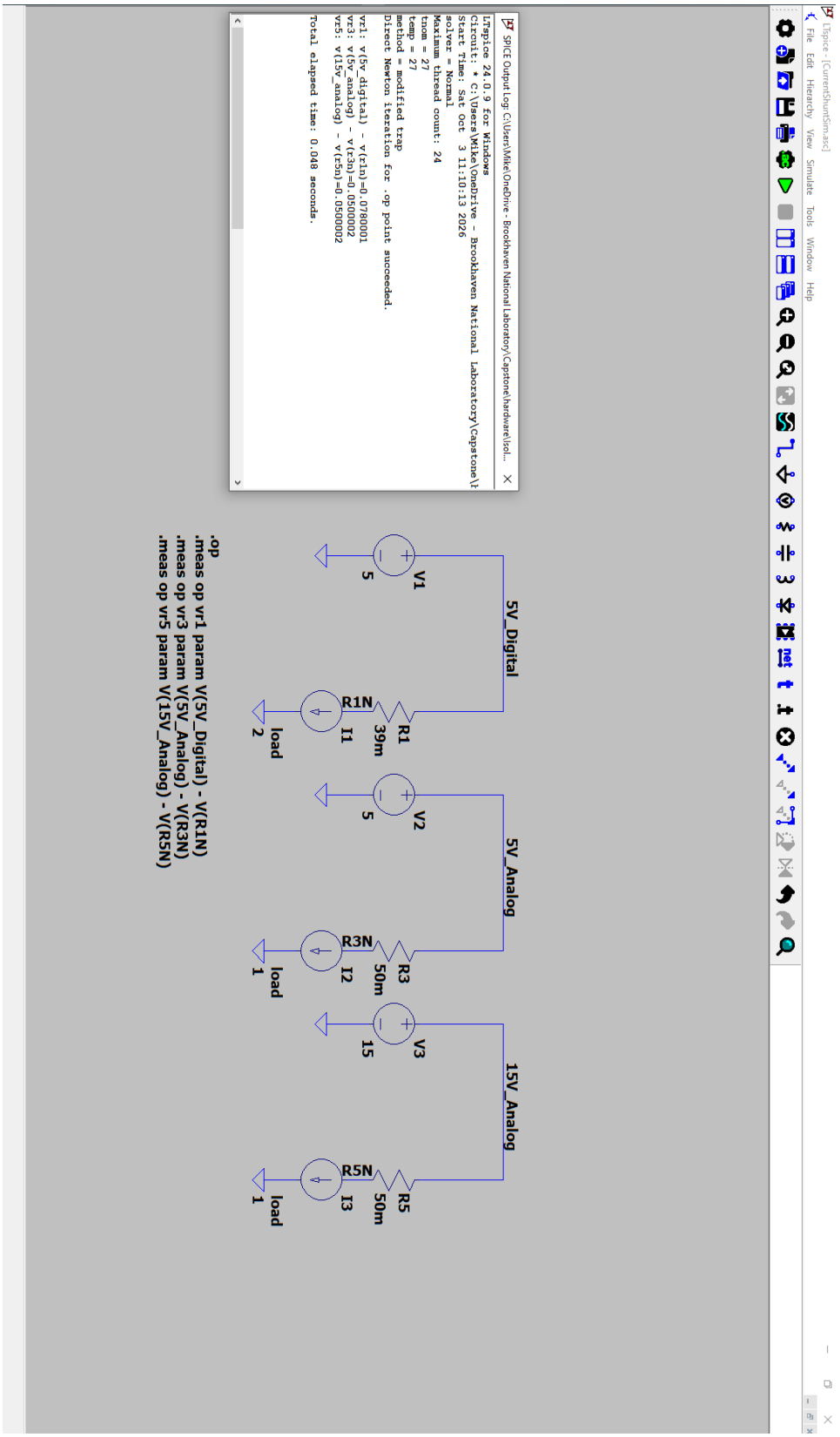


Figure 7.2: Simulation verifying current shunt voltage drops, full simulation screenshot

Chapter 8

Final Design

Final design consists of a six-channel monitoring interface, Arduino GIGA controller, local touchscreen, Ethernet interface, and EPICS control system interface.

8.1 Hardware Design

The system monitors six separate power rail supplies: +5VA, +5VB, +5V, -5V, +15V, and -15V.

Each of the six channels measures the voltage of supply and the voltage on a high-precision current sense resistor using an LTC2945 device. Digital isolation separates each channel of measurements from the I2C bus on the controller side.

Custom-made PCBs house the sensor interfaces, isolation circuits, and Ethernet components. The data acquisition and processing is done by Arduino GIGA R1 WiFi module, while GIGA Display Shield provides the local user interface.

8.2 Software and Communication

The firmware regularly acquires measurements, converts the results to engineering units, applies correct polarity and performs evaluation against upper and lower alarms for voltage and current.

Measurements, alarms and communication status are locally displayed and sent via Ethernet using the PSC protocol. EPICS IOC provides access to the information for the Phoebus operator interface.

Network settings and alarms are stored in non-volatile memory.

8.3 Final Design Documentation

Functional block diagram shows relations between measurement hardware, controller, local display and EPICS interface.

8.3.1 Functional Block Diagram

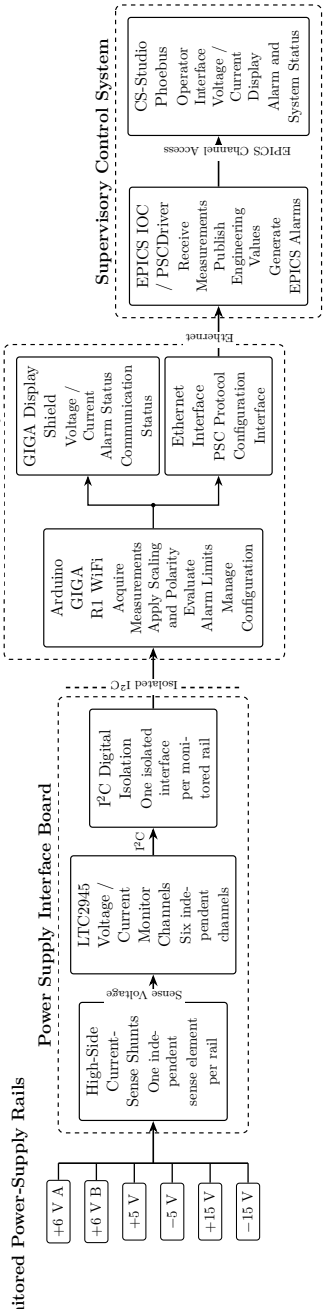


Figure 8.1: Functional block diagram of the multi-rail isolated power supply monitoring system. Six independent power-supply rails are measured using high-side current sensing and LTC2945 voltage/current monitoring channels. Digital isolation preserves electrical independence between the monitored supplies and the common controller. The Arduino GIGA processes the measurements, provides a local operator display, and transmits measurement and status information over Ethernet to the EPICS IOC and CS-Studio Phoebus operator interface.

Full set of schematic diagrams is included in Appendix C. Firmware and EPICS software flowcharts are also provided there.

8.4 Implementation Modifications

Final implementation of the design involved some modifications to the initial design, namely to isolated I2C interface. Logic inversion and pull-up modification have been added to solve the communication issues found during hardware bring-up.

Prototype incorporates all necessary modifications and hardware and software developed during the project.

Chapter 9

Solution Implementation

9.1 Testing the LTC2945

The LTC2945 testing was carried out using a Linear Tech eval board, DEMO CIRCUIT 1697A. Connections and modifications were made as follows:

- VIN: tied to Arduino +5V
- GND: tied to Arduino GND
- ADIN: Tied to GND
- SCL: Tied to Arduino Pin D8 I2C2 SCL
- SDAI/SDAO (JP2 set to TIE): Arduino Pin D9 I2C2 SDA
- VPU: Tied to Arduino +3v3
- VOUT: Float or tied to dummy load (1kΩ to 10kΩ)
- 20mΩ shunt replaced with 17Ω shunt for testing with ∼5kΩ load
- Installed 4.7kΩ pull up on I2C lines via R4, R5 footprints on EVB

9.1.1 Calculating Shunt Value for Test

Using the 5V from the arduino as a source, and a load ranging 1k to 10k ohms, we calculate:

$$R_{L,min} = 1k\Omega$$

$$V_{shunt} = V_S \frac{R_S}{R_L + R_S}$$

$$R_S = \frac{V_{Shunt} R_L}{V_s - V_{Shunt}}$$

$$V_{shunt} = 0.1024V$$

$$R_{S,fullscale} = \frac{(0.1024V)(1k\Omega)}{5V - 0.1024V} = 20.91\Omega$$

Adding some headroom, say, 15%, and going to an E96 resistor value...

$$R_S = 0.85(20.91) = 17.8\Omega$$

Verifying for the high end of the range:

$$I_{min} = \frac{5V}{10k\Omega + 17.8\Omega} = 499.1\mu A$$

$$V_{shunt,min} = (499.1\mu A)(17.8\Omega) = 8.88mV$$

The shunt will range from 8.88mV to 87.44mV, within the ADC's usable range of 0V to 102.4mV.

9.1.2 Pictures of Evaluation Board, Display, and Phoebus Panels

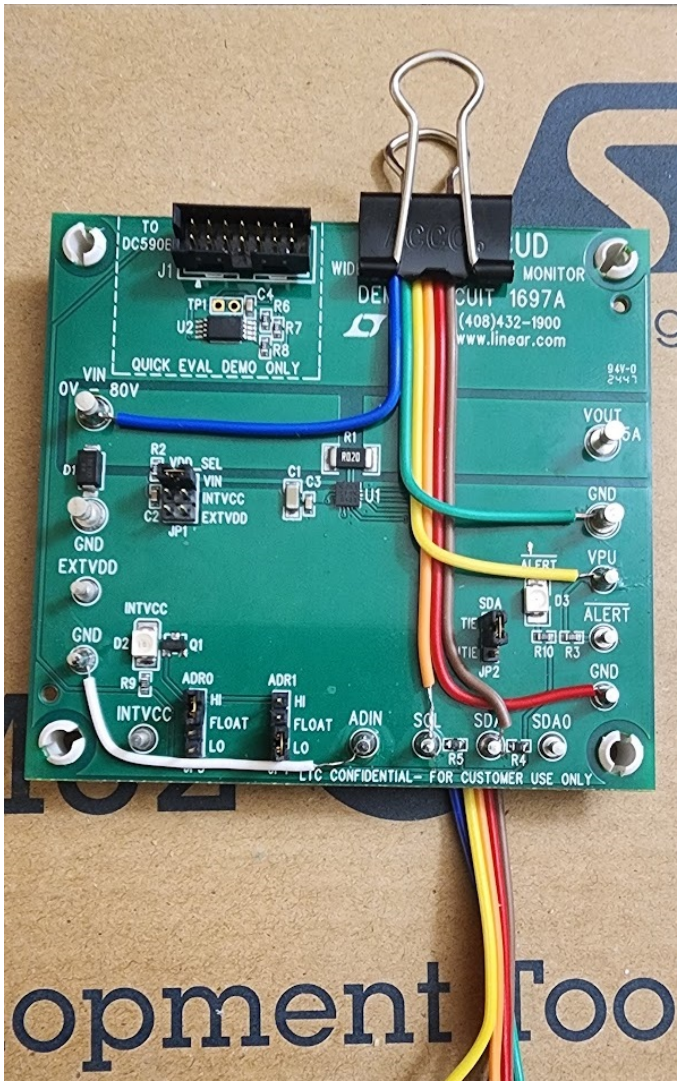


Figure 9.1: DEMO CIRCUIT 1697A Eval Board with Original Shunt, fly leads to connect to Arduino.

Figure 9.2: DEMO CIRCUIT 1697A Eval Board with 17Ω Shunt, fly leads to connect to Arduino, and 4.7kΩ test load.

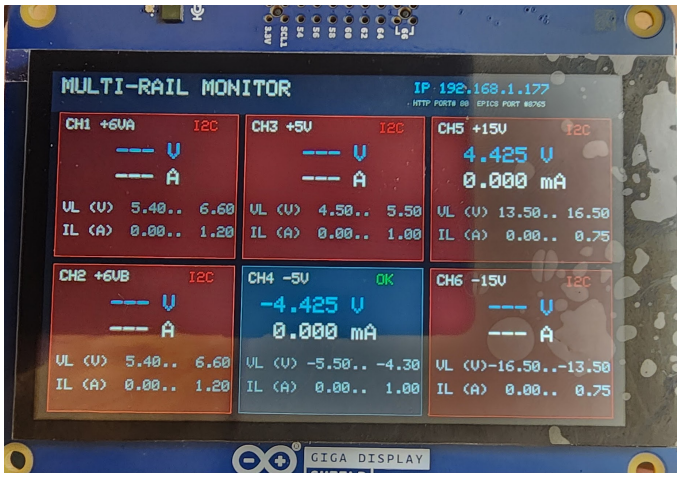


Figure 9.3: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm

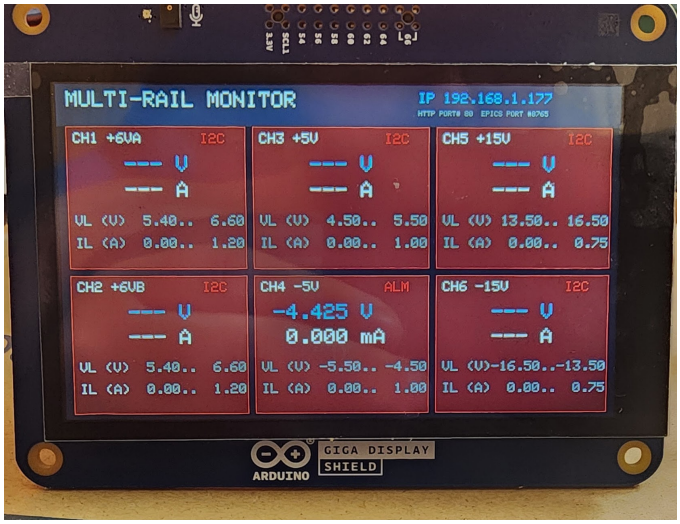


Figure 9.4: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, with UVL alarm, other channels I2C ALARM.

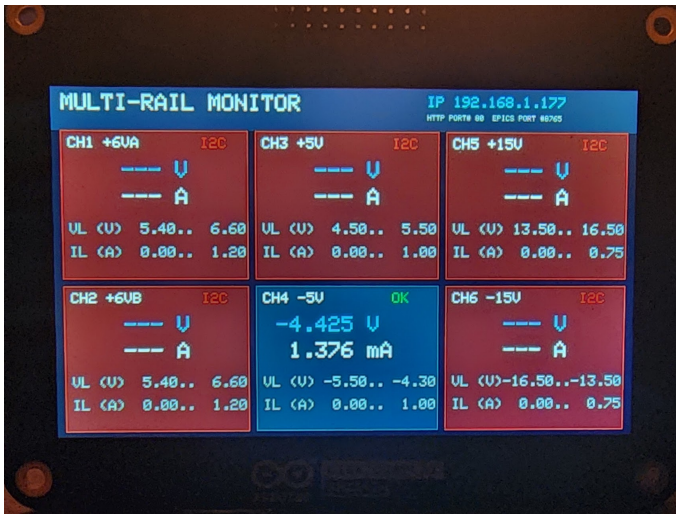


Figure 9.5: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load

Figure 9.6: Arduino Giga with GIGA DISPLAY SHIELD showing dummy -5V rail being monitored, without alarm, 3.3kΩ Load

Figure 9.7: Arduino Giga with GIGA DISPLAY SHIELD connected to an LTC2945 eval board

9.2 Bring Up and Testing of the First Spin Board

After construction of the printed circuit boards was complete, the board bring-up process was carried out prior to system-level testing. The objective of the board bring-up process was to ensure that the electronic operation of the circuitry was functioning, locate assembly

problems and communication issues, and make modifications if required before running the entire system.

In this case, initial testing was not done by powering up and trying to run the entire system, but was carried out incrementally. Each rail, each signal and functional block was tested separately so as to pinpoint failures without adding other variables. Measurements were made with the use of a digital multimeter and oscilloscope, whereas firmware and serial diagnostic output was used to ascertain if the Arduino was communicating with peripheral devices.

Several problems were noted during this phase, but of primary concern was an error in the I^2C Serial Data (SDA) interface between the circuitry of the Arduino side and the isolated circuitry. This problem led to a failure of I^2C communication and troubleshooting had to be done at the signal level with an oscilloscope. The troubleshooting revealed that there was an incorrect selection of a component in the constructed board, and temporary modification of the board was done with the help of an additional logic inverter to fix the problem and allow further development and testing of the system.

The subsequent sections describe the board bring-up process, detected failures, measurements made for diagnostics, performed modifications, and the testing of the fixed board.

9.3 Assembly Verification/Visual Inspection and Smoke Test

9.3.1 Visual Inspection

I hand-assembled all components of the boards, so a thorough visual inspection was carried out. The board was assembled using water-soluble paste flux and organic core water soluble 63/37 Sn/Pb solder wire (0.031" and 0.015"). As such, the boards were washed thoroughly prior to inspection.

Only one notable finding was found, U20 was shifted off the pads, with the exposed pad in the center having 'snapped' to pins 4/5/6. The board was reworked via hot air prior to power being applied to correct this. No other defects were found on visual inspection.

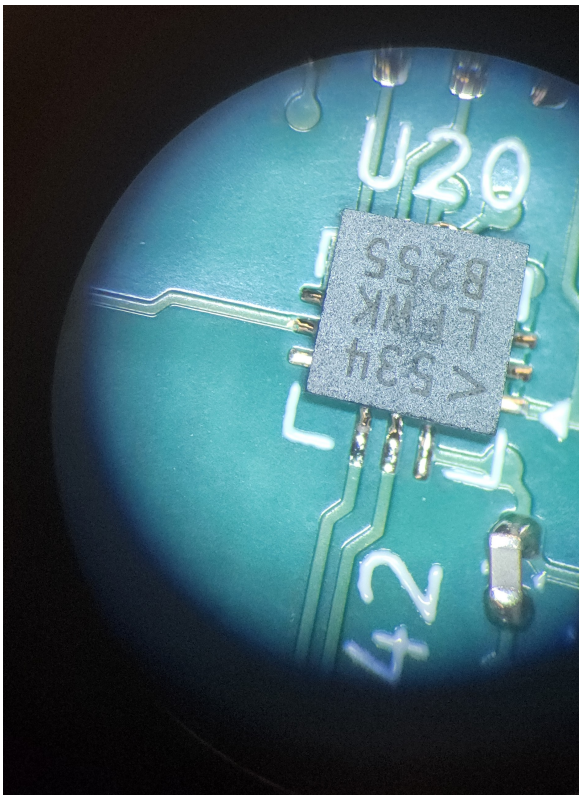


Figure 9.8: The PMIC, U20, was found to have not been properly aligned when soldered initially, requiring rework to re-center the component.

9.3.2 Smoke Test

An open and short test was performed on the input and output side of the 'Sensor' board's regulator. Both the input and output side showed a charging capacitive load on the DMM (increasing resistance), and so it was deemed safe to proceed to power on testing.

A +5V supply was connected to the input, set for a 100mA current limit, and the board came up to approximately 3.6V. The current limit was increased to 500mA, and the bench supply came up to 5V and regulated. The output of the on-board regulator was verified to be putting out 3.3V. The microcontroller was observed to boot properly.

9.4 I^{2}C SDA Communication Problem

9.4.1 Initial Symptoms

One of the first significant problems encountered during board bring-up was a failure of the isolated I^{2}C interface. The Arduino firmware attempted to communicate with the monitoring devices, but the expected responses were not received. Because I^{2}C communication requires

both the Serial Clock (SCL) and Serial Data (SDA) signals to operate correctly, the two signals were examined at multiple locations on the PCB using an oscilloscope.

During the initial measurements, unusual waveform shapes were observed on portions of the isolated interface. These measurements were initially misleading because the oscilloscope reference and the isolated circuit grounds were not connected as intended. As I was taking measurements, working only on the +6VA supply initially to focus testing, I was measuring on both isolated and non-isolated side of the bus, and I was using floating bench supplies. After connecting all the supply commons, the waveforms were reasonable.

The SCL signal was then observed switching as expected, demonstrating that the Arduino was attempting to generate I^2C transactions. The SDA path, however, did not behave correctly. At one point in the signal chain the clock activity was clearly visible, while the corresponding SDA-side output remained at a nearly constant logic level. This indicated that the failure was not caused by the Arduino failing to initiate communication, but was instead located within the SDA interface circuitry.

With the sensor board disconnected, I was able to observe clock and data transmissions, with NACK visible on the 9th clock as below:

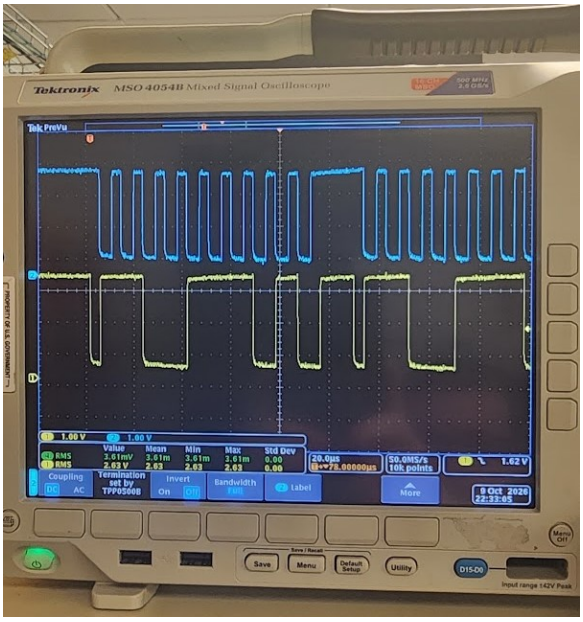


Figure 9.9: I2C messages on Arduino shield cable driver output, with sensor board disconnected, NACK bit on 9th clock pulse

9.4.2 SDA Signal Troubleshooting

Troubleshooting proceeded by following the SDA signal through the circuit one node at a time. The signal was measured on both sides of the isolation circuitry and at the input and output of the associated logic circuitry. This approach made it possible to determine where the expected transitions stopped propagating.

Testing showed that the input side of the isolation circuit was responding to SDA activity. The voltage measured across the input device during a low state was approximately 1.34 V, indicating that the input element was being driven. However, the corresponding output did not reproduce a valid SDA waveform.

Additional testing was performed with the pull-up and drive circuitry connected and disconnected. The current limiting resistor for the optoisolators was dropped from 2k2Ω to 909 Ω resistor to ensure the drive current wasn't marginal, and the observed nodes only fell to approximately 1.8 V during what should have been a logic-low condition. This level was too high to provide a reliable digital low for the following logic stage. It was behaving almost as if two push-pull outputs were fighting each other...

9.4.3 Identification of Incorrect Component

At this stage, I focused my attention around the PMIC's $\overline{SDAO}$ signal...Measuring the input pin of the optocoupler, I observed the line was being held high! This was expected behavior for I2C, except the selected part was designed to have an inverted SDA output!

Review of the schematic, assembled board, and component ordering information revealed that the part populated in the SDA interface was not the intended device configuration. Although the installed part was electrically similar to the device considered during the design, its logic behavior did not match the requirements of the implemented circuit.

I had ordered from Digikey the LTC2945IUD....the part required was the **LTC2945IUD-1**.

As a result, the output signal had the wrong logical relationship to the SDA signal required by the remainder of the interface. The PCB therefore could not be corrected through firmware alone. The hardware signal needed to be inverted before being applied to the following stage.

This problem was not immediately obvious during assembly because the component was physically compatible with the PCB footprint.

9.4.4 Inverter Bodge Modification

To continue board development without waiting on a parts order, a hardware bodge was implemented to correct the SDA signal polarity. A Texas Instruments SN74LVC2G04DCKR dual inverter was used for the modification, as an inverter I had on hand in the shop that was 3.3V compatible. The device contains two independent logic inverters and operates from single supply voltage range compatible with the logic used on the board. The DCK package is an SC-70-6 which is easily installed to the board with epoxy and tied into the circuit with 32awg magnet wire.

For the inverter channel used in the modification, pin 1 is the logic input and pin 6 is the corresponding inverted output. The affected SDA-related signal was therefore rerouted through this inverter before being connected to the following stage. The modification provided the additional inversion required to produce the logic polarity originally expected by the circuit.

The modification can be represented conceptually as

$$V_{\mathrm{OUT}} = \overline{V_{\mathrm{IN}}}, \tag{9.1}$$

where V_{IN} is the signal produced by the existing interface and V_{OUT} is the corrected signal applied to the following FET.

The bodge required cutting or and lifting the trace between the iso and the fet's gate, and adding short jumper wires between the existing circuit, the inverter input, and the fet gate. Power and ground were also provided to the inverter by tying into the optoisolator's 3.3V digital supply (via it's Pin 8, and the ground via pin 4).

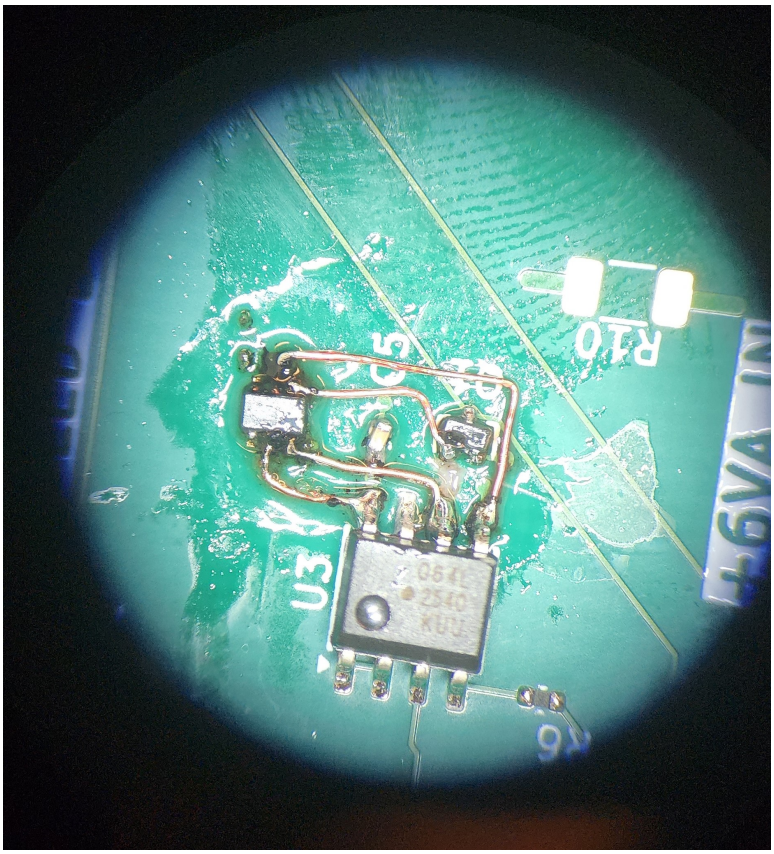


Figure 9.10: SC70-6 package inverter epoxied to the board, and 32awg bodge wires added, with cut trace, between optocoupler output and FET gate

9.4.5 Verification After Modification

After implementing the SDA modification, the SDA signal was inspected once again using an oscilloscope at several points in the communication circuit. The reason for this inspection was not only to check the presence of a waveform, but also to see whether there was proper propagation of logic-high and logic-low signals through the entire interface.

This signal was checked alongside the SCL waveform during the I^2C communication by the Arduino. The focus of observation was on the ninth clock pulse, which is where an I^2C slave should acknowledge the transmission by lowering the SDA signal. This observation was important since it showed that the process was moving further than before modification.

After confirming the waveforms, once again, communication was tested through the firmware. Success in communication after modification proved that the error occurred at the hardware stage of the SDA signal implementation and not within the I^2C software itself.

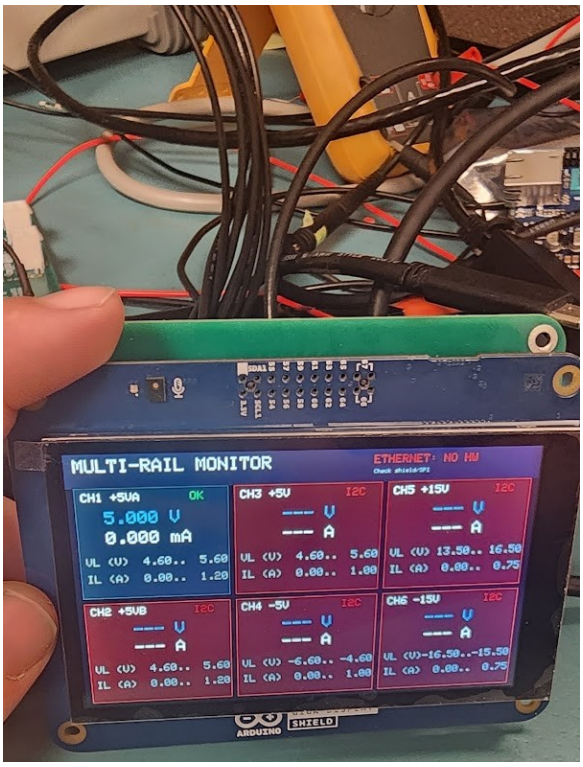


Figure 9.11: Communication is now working for the +5VA channel that has had the I2C connection repaired. The other 5 channels have had the FET gate floated (copper traces cut and lifted) from the opto to release the data bus, but have not yet had their inverters installed.

The troubleshooting process used to locate the problem can be summarized as:

1. Confirm that the required circuit power supplies are present.
2. Establish the correct oscilloscope ground reference for each isolated portion of the circuit.
3. Verify that the Arduino generates I^2C clock activity.
4. Probe SDA sequentially through each stage of the interface.
5. Identify the point where valid SDA transitions are lost.

6. Compare the installed component and its logic behavior with the schematic requirements.
7. Modify the circuit to correct the signal polarity.
8. Re-measure the SDA waveform.
9. Repeat the firmware communication test.

The error was helpful in the bring-up of the board since it showed that embedded system debugging must consider both the hardware and the software at the same time. The error of the firmware showing communication problems via I^2C showed the symptom but using the oscilloscope to measure the values pointed out where the exact problem was located.

9.4.6 Further Bodges and Repairs

Repairs were carried out on the remaining 5 channels. Additionally, it was found that the +5VA and -5V both had the same I2C address configured; this was modified by removing U20, cutting the traces to the address pins, and running 30awg magnet wire to tie Pin 3 to INTVCC (via U21-8) and Pin 2 to -5V via C43-2. Power was restored and then it was observed to now be working on all 6 channels!

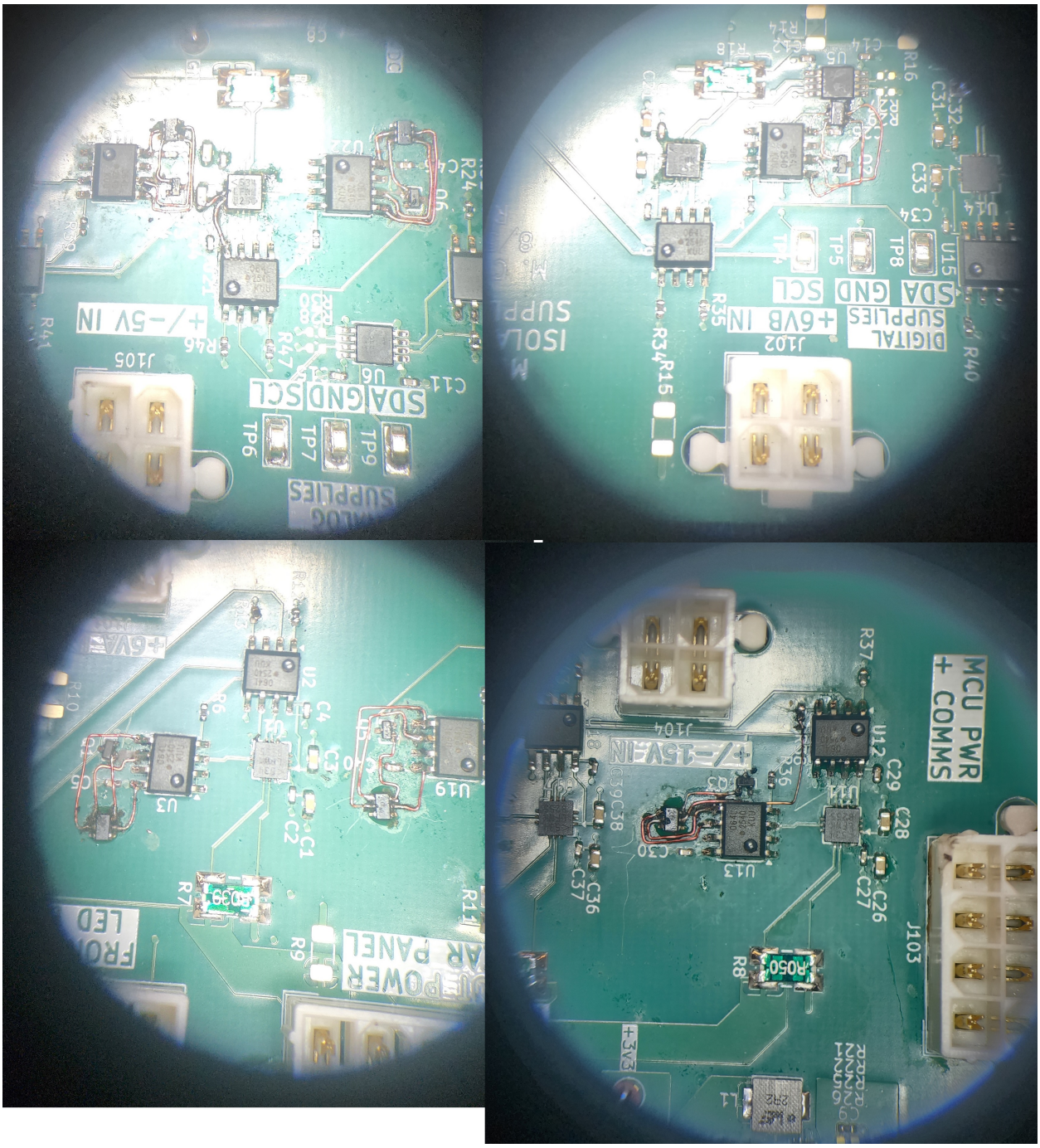


Figure 9.12: Rework is completed for all 6 channels

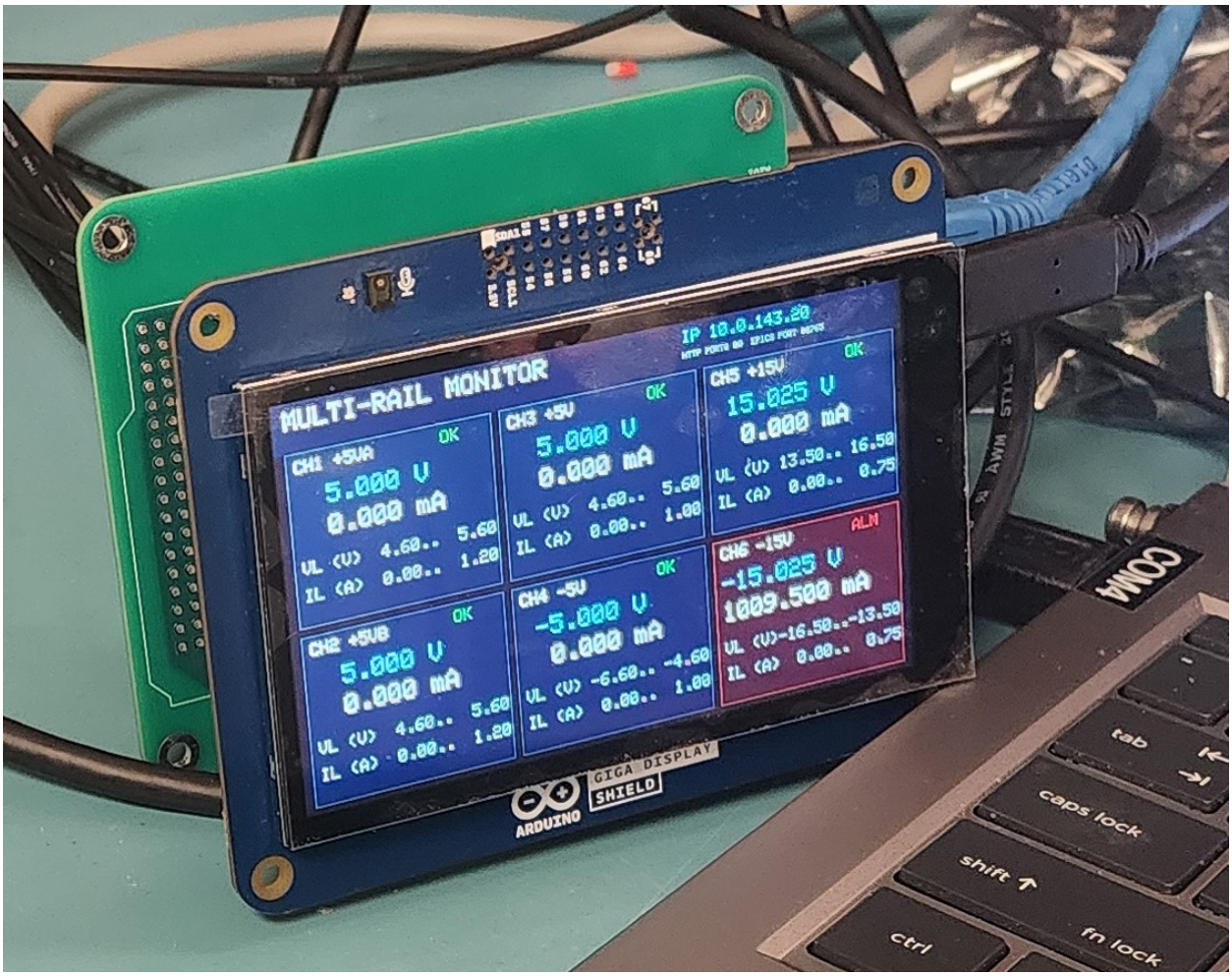


Figure 9.13: All 6 channels now working!

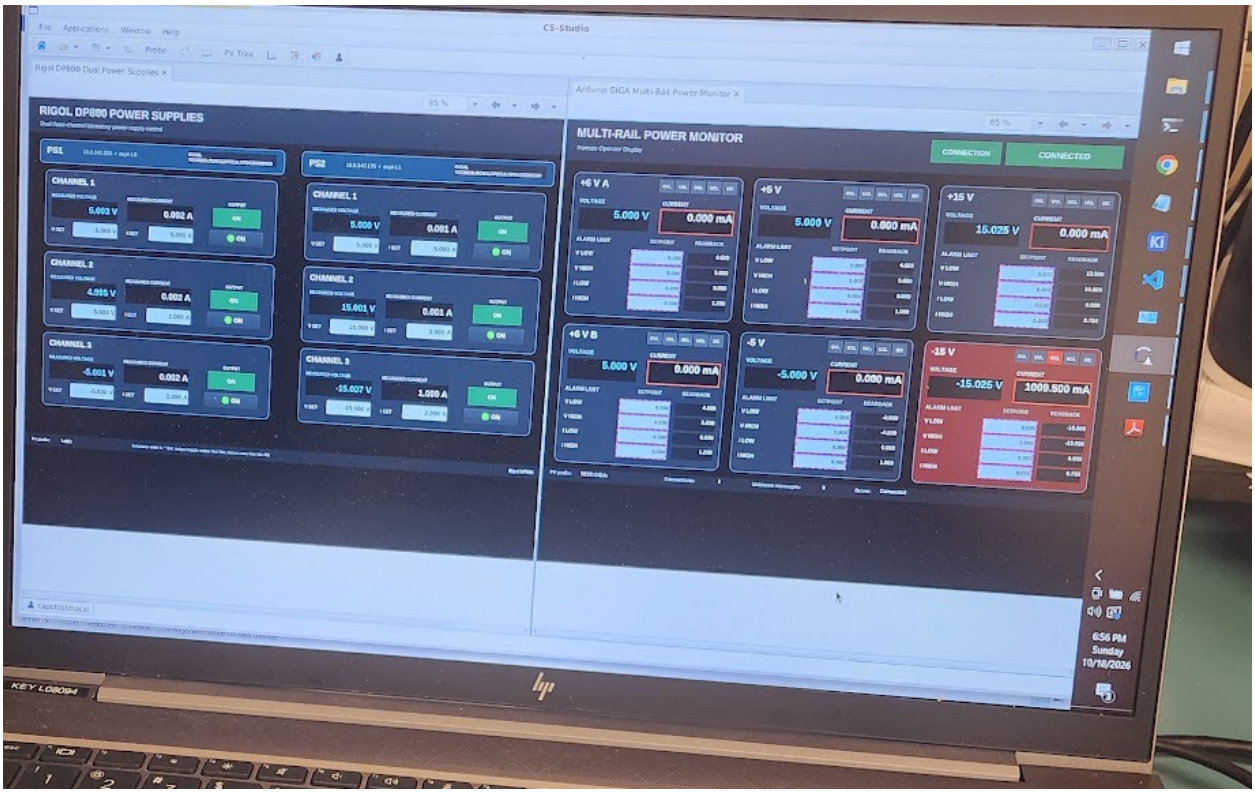


Figure 9.14: Control Panels working as well, showing 1A dummy load on one channel

Chapter 10

Evaluation of Solution's Efficacy

10.1 Automated Functional Tests

10.1.1 System Under Test

The system under test is the completed six-channel power supply monitoring system developed for this capstone project. The system monitors the following power rails:

- $+5\,\mathrm{V}A$
- $+5\,\mathrm{V}B$
- $+5\,\mathrm{V}$
- $-5\,\mathrm{V}$
- $+15\,\mathrm{V}$
- $-15\,\mathrm{V}$

Each rail has its own separate voltage and current monitoring. The measurement circuits are isolated from the communication channel on the Arduino side and are connected to Arduino GIGA R1 WiFi. The Arduino has a local graphical display by using GIGA Display Shield and Ethernet connection by a custom interface shield I've built. The Ethernet connection works with EPICS Input/Output Controller (IOC), making it possible to monitor the rails measurement and status remotely.

The testing of the system checks out the entire system but not each circuit block separately. This test involves the sensor circuits, isolation circuits, Arduino firmware, display, Ethernet connection, EPICS IOC, and related software as one whole system.

10.1.1.1 Test Objectives

The primary objectives of the system test are to verify that:

1. the complete system powers on and initializes without errors;
2. all six power rails are identified and displayed correctly;
3. the voltage measurement for each rail agrees with an external reference measurement within the specified system tolerance;
4. the current measurement for each rail agrees with the applied load within the specified system tolerance;
5. positive and negative supply rails are reported with the correct voltage polarity and current convention;
6. measurements shown on the local display correspond with the values available through EPICS;
7. voltage and current operating limits produce the appropriate status indication;
8. user-configurable settings remain stored following a power cycle;
9. Ethernet or IOC communication can be interrupted and restored without requiring a complete reprogramming of the monitor;
10. measurement data can be recorded and used to generate a permanent test report; and
11. the completed system can successfully perform the functions required for the final capstone demonstration.

10.1.2 Scope of Testing

10.1.2.1 Features to be Tested

System testing will include the following functions:

- system startup and initialization;
- detection and monitoring of all six power rails;
- voltage measurement;
- current measurement;
- measurement response to changes in load;
- correct handling of positive and negative rails;

- local GIGA Display operation;
- Ethernet communication;
- EPICS process-variable updates;
- voltage and current limit/status indications;
- storage and restoration of persistent settings;
- communication-loss recovery;
- repeated measurements to identify intermittent operation; and
- automatic recording and reporting of system test results.

10.1.2.2 Features Not Included in This Test

The system test is intended to verify the functional requirements of the prototype only. The following tests are therefore outside the scope of this system-level test plan:

- EMC testing;
- qualification tests for environmental temperature/humidity;
- qualification tests for vibrations or mechanical shock;
- qualification tests for life cycle/accelerated aging;
- destructive overvoltage tests; and
- destructive overcurrent tests.

10.1.3 Test Equipment and Software

The following equipment and software will be used during system testing:

Equipment/Software	Purpose
Power supply monitor prototype	Complete hardware and firmware system under test.
DC bench supplies	Provide the six input rails required by the monitor.
Rigol DL3021 electronic load	Provide programmable and repeatable loading of individual power rails.
Computer running EPICS IOC	Receive voltage, current, and status information from the Arduino over Ethernet.

Equipment/Software	Purpose
Phoebus/EPICS display	Provide operator-level observation of process variables and system status.
Python scripts	Control the electronic load, acquire EPICS values using PyEPICS, save raw test data, and automate portions of the test sequence.
Ethernet network	Provide communication between the Arduino and EPICS IOC.

10.1.4 Test Configuration

During testing, the assembled power supply monitor will be operated in its normal system configuration. All six monitored supplies will be connected to the appropriate monitoring inputs. The Arduino, GIGA Display Shield, custom monitoring hardware, and Ethernet interface will be powered and operating simultaneously.

For measurements involving the programmable electronic load, only the rail under test will be connected to the electronic load. The remaining rails will remain energized so that the monitor continues operating as a complete system, and may or may not be connected across resistive loads.

The electronic load measurement will provide the reference for both load voltage and current.

10.1.5 System Acceptance Criteria

The test is intended to establish whether the assembled prototype satisfies its functional requirements. The following system-level acceptance criteria will be used.

Parameter	Acceptance Criterion
System startup	System initialization should not involve a firmware error or user action, and all six rails should be shown.
Voltage measurement	The monitor reading should match the external reference voltage within ±2% across the full tested operating range.
Current measurement	Monitor reading should match the reference current within ±5% across the full intended operating range. In addition, the uncertainty and resolution of the reference device should be considered at low currents.
Positive/negative rail determination	Each measurement should use the correct polarity. The current in the negative channel should follow the sign convention used in the firmware.

Parameter	Acceptance Criterion
Display functionality	The display should show the proper identification of the rail, along with the rail voltage, current, and limits.
EPICS operation	All process variables should communicate and report their status without errors in normal operations.
Display and EPICS agreement	Local and remote readings should be identical to the same underlying measurement. Differences from rounding to integer in display are allowable.
Limit status	Any programmed limit should change status as the corresponding measurement crosses the limit and return to the normal status upon the measurement returning to the allowable range.
Persistent configuration	Any stored settings should be preserved after powering down and up the system.
Network recovery	After the restoration of Ethernet communication, EPICS should continue to update without any firmware reprogramming.
Test data	Each automatic test should yield a recorded raw-data file allowing for test results reconstruction.

The voltage and current accuracy requirements above are system-level requirements rather than component-level accuracy specifications. They include the combined effects of the sensor, shunt resistor, analog-to-digital conversion, calibration, wiring, firmware calculations, and reference-instrument uncertainty.

10.1.6 System Test Matrix

The system tests that are planned are listed in Table D.3 below.

Table 10.3: System Level Test Matrix

ID	Test	Method	Pass Criterion	Result
ST-01	Power On and Initialization	Apply power to the entire system and check the operation of Arduino, the local display, the sensor channels and the network interface.	The system initializes properly, all six channels are operational.	☐ Pass ☐ Fail ☐ Retest
ST-02	Rail Identification	Apply power to the six rails and correlate the displayed channel with the corresponding input physical connection.	The correct rail identification with the right polarity occurs.	☐ Pass ☐ Fail ☐ Retest
ST-03	Voltage Accuracy	Measure each rail using a DMM or the E-Load and compare the reference value with the local and EPICS values at different operating points.	The error is within the specified voltage tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-04	Current Accuracy	Apply different loads to each rail using the programmable electronic load and compare the reference current with the monitor measurement.	The error is within the specified current tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-05	Dynamic Load Response	Change the current of the electronic load while monitoring the local display and EPICS values.	The current and voltage readings change accordingly to the load change.	☐ Pass ☐ Fail ☐ Retest
ST-06	Local Display	Monitor voltage, current, rail identification and status of all channels.	All necessary information is visible and correlates with the connected rail.	☐ Pass ☐ Fail ☐ Retest

ID	Test	Method	Pass Criterion	Result
ST-07	EPICS Communications	Run the IOC and check all voltage, current and status process variables.	All necessary process variables are connected and updated properly.	☐ Pass ☐ Fail ☐ Retest
ST-08	Limit / Alarm Operation	Set a limit and change the current of the electronic load or supply until the measured value reaches this limit.	The proper limit indication is created and then is cleared when the value is inside the limits.	☐ Pass ☐ Fail ☐ Retest
ST-09	Persistent Settings	Change some settings, cycle system power and check the value after the power-up.	The stored value is restored properly.	☐ Pass ☐ Fail ☐ Retest
ST-10	Communication Recovery	Disconnect Ethernet while the system is running and reconnect the network and monitor the IOC.	Monitoring on the local machine continues and communication recovers.	☐ Pass ☐ Fail ☐ Retest
ST-11	Repeatability	Make repeated measurements at selected points, including after power cycle.	Measurement remains within the specified acceptance limits with no intermittent failures of the channels.	☐ Pass ☐ Fail ☐ Retest
ST-12	End-To-End Demonstration	Run all parts of the system and demonstrate the operation of local and remote monitoring by changing the load on the selected channel.	All hardware, firmware, display, network, IOC and test software run cooperatively with no system level failure.	☐ Pass ☐ Fail ☐ Retest

10.1.7 Detailed Test Procedures

10.1.7.1 ST-01: Power-On and Initialization

1. Inspect all power, sensor, display, and Ethernet connections.
2. Verify that the electronic load is disabled.
3. Energize the required monitored power supplies.
4. Apply power to the Arduino and monitoring electronics.
5. Observe the Arduino startup sequence.
6. Confirm that the GIGA Display initializes.
7. Confirm that all six expected rail channels are displayed.
8. Start or verify operation of the EPICS IOC.
9. Verify that the IOC establishes communication with the monitor.
10. Record any startup errors or unexpected behavior.

The test passes when the complete system reaches its normal operating state without manual recovery, all six rail channels are present, and network communication can be established.

10.1.7.2 ST-03 and ST-04: Voltage and Current Measurement

Voltage and current accuracy testing will be performed for each of the six rails. Testing will be automated where practical to improve repeatability and preserve the raw test data.

For each selected rail:

1. Select the rail to be tested in the Python test program.
2. Verify the physical connection of the rail and electronic load.
3. Connect the DMM directly across the rail.
4. Set the electronic load to its initial test condition with the input disabled.
5. Enable the load.
6. Allow the readings to stabilize.
7. Record the DMM or E-Load reference voltage.
8. Record the electronic-load reference current.

9. Acquire the corresponding EPICS voltage and current values using PyEPICS.
10. Calculate the voltage and current measurement errors.
11. Repeat the measurement at several loads distributed throughout the intended operating range of the rail.
12. Disable the load before changing connections.
13. Save all raw measurements to the test-data file.

Voltage error will be calculated as

$$\text{Voltage Error (\%)} = \frac{V_{\text{monitor}} - V_{\text{reference}}}{|V_{\text{reference}}|} \times 100. \tag{10.1}$$

Current error will be calculated as

$$\text{Current Error (\%)} = \frac{I_{\text{monitor}} - I_{\text{reference}}}{|I_{\text{reference}}|} \times 100. \tag{10.2}$$

Absolute error will also be retained in the raw data. This is particularly important for low-current measurements where percentage error alone can become misleading.

A rail passes when each required measurement point remains within the specified acceptance criteria. If a point fails, the rail may be retested after the cause of the failure is investigated. Original failed data will not be deleted when a retest is performed.

10.1.7.3 ST-07: EPICS Communication Test

1. Power the complete monitor and connect it to the Ethernet network.
2. Start the EPICS IOC.
3. Verify that the IOC connects to the Arduino TCP server.
4. Observe the voltage and current process variables for all six rails.
5. Compare the EPICS values with the local display.
6. Change the load on a selected rail.
7. Verify that the corresponding EPICS voltage and current values update.
8. Repeat the load change on at least one positive and one negative rail.

The test passes when the required process variables remain connected, values change in response to the physical system, and EPICS and local-display values agree within their displayed resolution.

10.1.7.4 ST-08: Limit and Status Test

1. Select a rail for testing.
2. Record the existing voltage and current limit settings.
3. Configure a test limit that can be crossed safely without exceeding the electrical rating of the hardware.
4. Operate the rail below the limit and verify that normal status is displayed.
5. Increase or decrease the applied load until the selected threshold is crossed.
6. Verify the local status indication.
7. Verify the corresponding EPICS status.
8. Return the measurement to the normal operating range.
9. Verify that the status returns to normal.
10. Restore the original limit setting.

No test limit shall require the monitored supply, sensor, wiring, or electronic load to be operated outside its rated limits.

10.1.7.5 ST-09: Persistent Settings Test

1. Change one or more user-adjustable system settings to known values.
2. Verify that the settings are accepted by the firmware.
3. Remove power from the Arduino and monitor electronics.
4. Wait several seconds.
5. Restore power.
6. Allow the firmware to complete initialization.
7. Read the previously modified settings.
8. Compare the restored values with the values stored before power removal.

The test passes when all settings intended to be persistent are restored without requiring Ethernet communication or manual re-entry.

10.1.7.6 ST-10: Communication-Loss and Recovery Test

1. Establish normal communication between the monitor and IOC.
2. Verify that EPICS values are updating.
3. Disconnect the Ethernet cable.
4. Verify that local monitoring and the GIGA Display continue operating.
5. Observe the IOC response to the communication failure.
6. Reconnect the Ethernet cable.
7. Verify that communication is re-established.
8. Confirm that the process variables resume updating with valid values.

The system passes if the loss of Ethernet communication does not prevent local operation and normal EPICS communications resume after the network connection is restored.

10.1.8 Automated Test Data Collection

A Python test application will be used for measurement testing. The application interfaces with the Rigol DL3021 programmable electronic load and the EPICS IOC. EPICS Channel Access is performed using the PyEPICS library.

The test operator may select an individual power rail instead of being required to test all six rails in a single session. This permits channels to be tested or retested at different times while retaining all results as part of the same system test campaign.

For each test point, the following information will be retained when applicable:

- test date and time;
- rail identification;
- test iteration;
- electronic-load operating mode;
- programmed load value;
- reference voltage;
- reference current;
- EPICS voltage measurement;
- EPICS current measurement;

- absolute voltage error;
- percentage voltage error;
- absolute current error;
- percentage current error;
- applicable limits;
- test result; and
- operator notes.

Raw data will be preserved independently from the final report so that the report can be regenerated without repeating the physical test. After the required tests have been completed, the saved data will be processed using ReportLab to produce a final test report containing the recorded values and pass/fail results.

10.1.9 Test Execution and Result Classification

Each test will be assigned one of the following results:

PASS: The test was completed and all applicable acceptance criteria were met.

FAIL: The test was completed but one or more acceptance criteria were not met.

INCOMPLETE: The test could not be completed because of an unrelated equipment, software, or setup problem.

RETEST: A previously failed or incomplete test was repeated following a corrective action.

A failed test will not be deleted or overwritten following corrective action. The original result and subsequent retest will both be retained to provide traceability of the development process.

10.1.10 Suspension and Resumption Criteria

Testing will be immediately suspended if any of the following conditions occur:

- unexpected overheating of a component or conductor;
- evidence of a short circuit or incorrect wiring;
- operation outside the rated voltage or current of the test equipment;

- unstable or uncontrolled power-supply output;
- a firmware condition that could apply an unintended load;
- loss of isolation where isolation is required; or
- any condition that could damage the prototype or test equipment.

Testing may resume after the cause has been identified, corrective action has been completed, and the affected portion of the system has been inspected or verified before power is reapplied.

10.1.11 Test Responsibilities

As the designer and developer of the capstone project, Mike Capotosto will serve as the primary test operator. Responsibilities include:

- preparing and inspecting the test setup;
- configuring the test equipment;
- executing each test procedure;
- recording results;
- identifying and documenting failures;
- making required corrections;
- performing retests;
- maintaining the raw test data; and
- preparing the final test report and demonstration.

Where practical, automated data acquisition will be used to reduce transcription errors and improve repeatability.

10.1.12 Testing Risks and Contingencies

Several risks may affect system testing. Improper electronic-load connections, particularly when testing the negative supply rails, could damage the electronic load or power supply. Connections will therefore be verified before the load input is enabled.

Network problems could also produce an apparent measurement failure when the actual sensor hardware is operating correctly. Local display values and direct instrument measurements will be used to distinguish a network or IOC problem from a hardware measurement problem.

A failed sensor channel could prevent the complete six-channel system test from being performed. Because the test software allows individual rail selection, completed channel data can be retained while the affected channel is repaired and subsequently retested.

Finally, automatic test software itself could introduce an error in the recorded results. Critical results will therefore be compared with the front-panel readings of the electronic load, local display, and EPICS interface during initial testing to confirm correct operation of the automation.

10.1.13 Test Deliverables

The completed system test will produce the following deliverables:

- this system test plan;
- saved raw test data;
- automatically generated test report;
- documentation of failed tests and corrective actions;
- results of any required retests;
- photographs of the final test configuration; and
- video footage demonstrating selected system-level tests.

10.1.14 Test Completion Criteria

System-level testing will be considered complete when:

1. every test identified in Table D.3 has been executed;
2. each of the six rail channels has completed voltage and current verification;
3. all critical functional tests have received a PASS result;
4. any failed tests have been documented and either corrected and retested or identified as known limitations;
5. raw test data has been preserved;
6. the final test report has been generated

Successful completion of this test plan will demonstrate that the individual hardware and software portions developed throughout the project operate together as a complete power supply monitoring system.

10.1.15 Test Reports

Refer to the Appendix D test plan for an example test report.

Chapter 11

Improvements and Optimizations to Solution

Despite the fact that the completed system satisfied the acceptance criteria, some improvements are recommended for the future PCB revision.

The most important hardware improvement is the incorporation of the SDA polarity fix directly in the PCB design. The prototype required the SN74LVC2G04 inverter to correct the SDA signal polarity after the originally installed LTC2945 device variant turned out to be incompatible with the intended isolation interface. The future revision should use the correct LTC2945 variant throughout the design or incorporate the required inversion directly in the schematic and PCB.

All required I^2C pull-up resistors and defined logic states should also be incorporated directly in the next PCB revision. It will eliminate the prototype modification and improve the repeatability of assembly and startup.

The firmware initialization can also be made more tolerant of arbitrary power-up sequencing. During the development, the communication could be failed when the controller tried to initialize the sensor before the corresponding isolated rail circuitry was powered. The future firmware revision could periodically retry the unavailable sensors so that the communication is established automatically when the monitored rail is powered.

The per-channel measurement calibration is another potential improvement. The present system calculates the current using the configured current sense resistance. The measurement of the actual resistance or the determination of the calibration factor against the external reference will allow the individual gain correction to be stored for each channel. It will help to reduce the systematic errors caused by the shunt tolerance and allow tighter acceptance limits if the improved measurement accuracy is required.

The additional environmental characterization is also useful if the design is going to be developed beyond the capstone prototype. The testing at multiple ambient temperatures can be used to determine whether the temperature affects the measurement accuracy. EMC testing can determine the susceptibility to and generation of the electromagnetic interference.

The long duration testing can be used to detect the intermittent communication, thermal, network or firmware problems that might not occur during the shorter acceptance test.

The automated test system should be retained for the future revisions. Since the raw measurements are saved separately from the report generation, the future hardware or firmware revisions can be tested using the same acceptance procedure and direct comparison between the revisions is possible.

These recommendations represent the refinements in reliability, manufacturability, calibration and qualification and not the changes to the basic architecture. The successful final test proves that the overall system concept is suitable for the further development.

Chapter 12

Conclusions

12.0.1 Did the Project Meet the Requirements of the Scope?

The project fulfilled the major requirements of the original project scope.

The completed system provides six independent voltage and current monitoring channels corresponding to +5VA, +5VB, +5 V, −5 V, +15 V, and −15 V. The high side current sensing is used for each channel while the electrical isolation is preserved between independently referenced monitored supplies and the common controller.

The completed hardware includes the sensor interface, current sense resistors, isolation circuitry, Arduino GIGA controller interface, local display, controller power circuitry and Ethernet interface. The software implementation includes the embedded measurement acquisition, conversion to engineering units, configurable limits, alarm processing, persistent configuration storage, local graphical display, network communication, the EPICS IOC, process variable database records and the Phoebus operator interface.

The final system testing confirmed the correct operation of all six measurement channels, the correct positive and negative voltage representation, programmable limit operation, persistent settings, local display operation, Ethernet communication, EPICS monitoring, communication recovery, repeatability, and end-to-end operation.

The hardware, firmware, IOC files, operator interface files, documentation and other project artifacts were also maintained under revision control as required by the project scope.

Based on the completed implementation and the final system test results, the capstone project is therefore considered to have fulfilled its technical scope.

12.0.2 Was the Project Completed on Time?

The project was completed within the planned capstone schedule.

The original project plan distributed the development across the eight week ET410 planning phase and the following eight week ET450 implementation phase, with the completed system

expected to be ready for demonstration by Week 7 of ET450. The final automated system test was completed on October 18, 2026, giving the completed and verified system before the planned final demonstration period.

The project experienced the schedule risk during the hardware bring-up. Troubleshooting of the isolated I^2C interface required additional effort after the SDA polarity problem and other communication problems were discovered. The startup sequencing and Ethernet integration also required the troubleshooting that was not anticipated in the initial plan.

These problems consumed the schedule margin, but they were fixed without preventing the completion of the prototype and final system test within the capstone schedule. The project is therefore considered to have fulfilled the schedule requirement.

12.0.3 Was the Project Completed Under Budget?

The project was not completed within the original preliminary hardware budget.

The initial project proposal assumed the material and contingency budget of $660. This estimate was based on the incomplete information about the detailed component selection, custom PCB layout and final vendor pricing.

After completion of the detailed design and Bill of Materials, the estimated one system hardware purchase cost became $1,019.03. It includes $242.36 in the Sensor Board components, $473.57 for the minimum Sensor Board PCB order, $180.90 in the Arduino Shield components, and $122.19 for the minimum Arduino Shield PCB order.

Compared to the original $660 estimate, the final documented hardware purchase estimate is $359.03 higher than the preliminary budget. The additional cost is due to a better definition of the component as well as the minimum order cost related to the fabrication of the custom PCBs.

Thus, the cost estimate was good to use as a preliminary cost estimate value but did not provide enough cost to build a complete prototype of a custom PCB. The project should be considered above the original hardware budget.

This cost comparison concerns only the documented prototype hardware cost. Laboratory test equipment, software, labor, shipping, taxes, enclosure hardware, cabling and any other items not included into the project cost analysis are not included into this cost comparison.

12.0.4 Did the Quality and Performance Meet Stakeholder Expectations?

The quality and performance of the completed prototype met the expectations set for the capstone project.

The main project stakeholders are the project developer, the course instructor and the potential technical users of the system. The best evidence of system quality is the completed acceptance test. All twelve expected system tests were completed successfully, all six mon-

itored rails passed the voltage and current check, and the complete system demonstration was successful without any system failures.

The prototype showed the capability to collect isolated voltage and current measurements, display the data locally, compare the data with configurable limits, keep the configuration after the restart, send the data via Ethernet, support the EPICS integration, present the data using Phoebus and to restore the operation after the network connection interruption.

The project provided the possibility to develop the repeatable automated test procedure instead of using the subjective one. The raw data were saved and used for the creation of a test report including the measurement data, pass/fail result and the test evidence. It improves the confidence in the final result and allows assessing future revisions of the hardware.

The prototype still includes some features which should be improved in the future production revision of the design, such as SDA inverter modification, modified implementation of the I^2C pull-up, correct startup sequence and the possible implementation of the per-channel calibration. They do not prevent the prototype from meeting the acceptance criteria, but may influence the manufacturability and further improvements of the design.

Based on the final test results, the completed project can be stated to meet the stakeholder expectations in terms of functionality and performance.

12.0.5 Overall Project Assessment

The Multi-Rail Isolated Power Supply Monitoring System successfully passed the full project cycle including the project definition and preliminary design, custom hardware development, firmware implementation, network and EPICS integration, board bring-up, troubleshooting and system verification.

The biggest challenges of the project were related to the hardware integration instead of development of the individual functions of the system. The troubleshooting of the isolated I^2C interface showed the necessity to check the component version, polarity, pull-up requirement, signal states and startup behavior in the physical implementation. These issues were found by the measurements, fixed and retested in the final system tests.

The completed prototype met the defined project scope, completed in the required time and met the final technical acceptance criteria. Although the project cost exceeds the original preliminary hardware budget, the detailed cost estimation provides more realistic basis for the estimation of the cost of prototype and future production revision.

Finally, the successful completion of the system test showed that the hardware, embedded firmware, local user interface, Ethernet communication, EPICS IOC, Phoebus interface and the automated test software work as the integrated system. Thus, the project successfully reached the main goal of development and demonstration of the functioning six-channel isolated power supply monitoring system.

Chapter 13

References

References

Analog Devices. (n.d.). *LTC2945: Wide range I^2C power monitor.* https://www.analog.com/en/products/ltc2945.html

Arduino. (n.d.). *GIGA R1 WiFi.* https://store.arduino.cc/products/giga-r1-wifi

Broadcom. (n.d.). *ACPL-064L-000E.* https://www.broadcom.com/products/optocouplers/industrial-plastic/digital-optocouplers/10mbd/acpl-064l-000e

International Labour Organization. (1973). *Minimum Age Convention, 1973 (No. 138).* https://www.ilo.org/dyn/normlex/en/f?p=NORMLEXPUB:12100:0::NO:12100:P12100_INSTRUMENT_ID:312283

IPC. (2009). *IPC-2152: Standard for determining current carrying capacity in printed board design.* https://www.ipc.org/TOC/IPC-2152.pdf

KiCad. (n.d.). *Track width versus current formula.* https://docs.kicad.org/doxygen/tracks__width__versus__current__formula_8h.html

Ohmite Manufacturing Company. (n.d.). *LVK series current-sense resistors.*

https://www.ohmite.com/assets/files/Datasheets/lvk-series.pdf

OpenAI. (2026). *ChatGPT* (Apr 2026 version) [Large language model].

https://chatgpt.com

Texas Instruments. (n.d.-a). *TCA9803 level-translating I^2C bus buffer/repeater*.

https://www.ti.com/product/TCA9803

Texas Instruments. (n.d.-b). *TPS62912 low-noise, low-ripple buck converter*.

https://www.ti.com/product/TPS62912

Appendix A

Functional Specifications

The functional specifications used for the design are summarized in Table A.1. These values define the intended operating envelope of the prototype and establish the quantities that must be checked during circuit analysis and later verification testing.

Table A.1: Functional specifications.

Function	**Specification**
Monitored rails	+5 V A, +5 V B, +5 V, -5 V, +15 V, and -15 V
Number of channels	Six independent voltage/current measurement channels
+5 V A/B current range	0 to 2 A design range per rail
+5 V, -5 V, +15 V, -15 V current range	0 to 1 A design range per rail
Voltage measurement resolution	25 mV/LSB nominal using the LTC2945 rail-voltage ADC
Current-sense resolution	25 μV/LSB across the selected shunt resistor; approximately 0.641 mA/LSB with 39 mΩ and 0.500 mA/LSB with 50 mΩ
Isolation	Independent digital isolation for every floating monitor channel
Alarm processing	User-configurable upper/lower voltage and current limits with local and EPICS status indication
Fault handling	Detect failed I^2C transactions and prevent invalid data from being treated as a valid measurement
Local interface	GIGA Display Shield showing voltage, current, alarm state, communication state, and configured limits
Network interface	Ethernet PSC server for EPICS IOC communication plus browser-based configuration/monitoring services
Configuration retention	Network parameters and alarm limits stored in nonvolatile memory
Controller logic level	3.3 V; external interfaces must not exceed the GIGA I/O limits

Appendix B

Design Calculations and Circuit Analysis

This appendix contains the electrical calculations used to select and verify the preliminary hardware. In addition to nominal values, voltage, current, power, logic-level compatibility, and component capability are considered so that the design does not intentionally operate a component beyond its specified limits.

B.1 Current-Sense Shunt Resistor Sizing

The LTC2945 measures the differential voltage between SENSE+ and SENSE– with a full-scale range of 102.4 mV. The ideal shunt resistance for a specified maximum measurable current is therefore

$$R_{SHUNT,ideal} = \frac{V_{SENSE,FS}}{I_{MAX}} = \frac{102.4\,\text{mV}}{I_{MAX}}. \tag{B.1}$$

To prevent saturation, a scaling factor, K_H, is applied to leave some headroom:

$$R_{SHUNT} = K_H \frac{102.4\,\text{mV}}{I_{MAX}}, \qquad 0 < K_H < 1. \tag{B.2}$$

For example, $K_H = 0.80$ reserves 20% headroom for overload, tolerance, and transient conditions.

The shunt power at maximum continuous current is

$$P_{SHUNT} = I_{MAX}^2 R_{SHUNT}. \tag{B.3}$$

The installed resistor power rating should exceed this value.

B.1.1 Current Measurement Resolution

The LTC2945 current-sense ADC has a nominal sense-voltage resolution of 25 µV per LSB. The current represented by one ADC count is

$$I_{LSB} = \frac{25\,\mu V}{R_{SHUNT}}. \tag{B.4}$$

This relationship provides an important design tradeoff: a smaller shunt lowers power dissipation but also increases the minimum current increment represented by one ADC count.

B.1.2 Production Shunt Design Table

Table B.1: Production current-sense resistor calculations.

Rail	I_{MAX} (A)	R_{ideal} (Ω)	Selected R (Ω)	V_{SHUNT} (mV)	I_{LSB} (mA/LSB)	P_{MAX} (W)
+5VA	2.00	0.0512	0.039	78.0	0.641	0.156
+5VB	2.00	0.0512	0.039	78.0	0.641	0.156
+5V	1.00	0.1024	0.050	50.0	0.500	0.050
-5V	1.00	0.1024	0.050	50.0	0.500	0.050
+15V	1.00	0.1024	0.050	50.0	0.500	0.050
-15V	1.00	0.1024	0.050	50.0	0.500	0.050

B.1.3 Evaluation-Board Shunt Calculation

The prototype LTC2945 evaluation board was tested from a 5 V source with a load range of approximately 1 kΩ to 10 kΩ. This is a low-current bench-test condition and is separate from the production shunt design.

At the minimum load resistance, the shunt required to place the current-sense input at full scale is

$$R_{S,FS} = \frac{V_{SHUNT} R_L}{V_S - V_{SHUNT}} = \frac{(0.1024\,V)(1\,k\Omega)}{5\,V - 0.1024\,V} = 20.91\,\Omega. \tag{B.5}$$

Applying approximately 15% headroom gives

$$R_S = (0.85)(20.91\,\Omega) \approx 17.8\,\Omega. \tag{B.6}$$

At a 10 kΩ load,

$$I_{MIN} = \frac{5\,V}{10\,k\Omega + 17.8\,\Omega} \approx 499.1\,\mu A, \tag{B.7}$$

and therefore

$$V_{SHUNT,MIN} = I_{MIN} R_S \approx (499.1\,\mu\text{A})(17.8\,\Omega) \approx 8.88\,\text{mV}. \tag{B.8}$$

The prototype sense voltage therefore remains inside the LTC2945 102.4 mV full-scale range over the intended test loads.

B.2 PCB Trace and Copper-Pour Current Capacity

Power-path copper must carry the maximum rail current without excessive temperature rise or voltage drop. KiCad's track-width calculator uses a closed-form fit to IPC-2152 data. For an external trace it can be expressed as

$$\Delta T = K I^a W^b T_h^c, \tag{B.9}$$

where I is current in amperes, W is trace width in mils, T_h is copper thickness in mils, and for external tracks the KiCad coefficients are

$$K = 215.3, \qquad a = 2, \qquad b = -1.15, \qquad c = -1.0. \tag{B.10}$$

Solving for minimum width,

$$W_{MIN} = \left(\frac{\Delta T}{K I^a T_h^c} \right)^{1/b}. \tag{B.11}$$

For 1 oz copper ($T_h \approx 1.378$ mil) and a preliminary 10 °C allowed rise, the calculated external-trace widths are approximately 3.27 mil at 0.5 A, 10.9 mil at 1 A, and 36.4 mil at 2 A. These are thermal minimums only; the actual layout should use wider traces or copper pours where space permits.

B.3 I²C Pull-Up Resistor Sizing

The I2C bus buffer and I2C line driver both have internal active current sources that eliminate the need for external pullups. Footprints were placed for 4.7k pullups but are set as Do Not Populate.

B.4 LTC2945 Voltage Measurement Resolution

The LTC2945 rail-voltage channel has a nominal full-scale range of 102.4 V and a nominal resolution of 25 mV per LSB. The quantization-only percentage of a measured rail voltage is therefore

$$\epsilon_Q = \frac{25\,\text{mV}}{V_{RAIL}} \times 100\%. \tag{B.12}$$

For example, one count corresponds to approximately 0.50% of a 5 V rail and 0.17% of a 15 V rail. This is only ADC resolution and not total measurement accuracy.

B.5 Negative-Rail Sign Convention

The -5 V and -15 V supplies are electrically isolated from the controller. The monitor can therefore be placed in the high side of each isolated rail and read a positive local differential quantity even when the system-level rail is labeled negative. Firmware applies the configured sign after acquisition:

$$V_{DISPLAY} = S_V V_{MEASURED}, \qquad S_V \in \{+1, -1\}. \tag{B.13}$$

The same channel metadata is used by the local display and network interface so that a negative rail is represented consistently throughout the system.

B.6 Alarm-Limit Scaling

User alarm thresholds are stored and compared in engineering units. For a lower voltage limit V_L and upper voltage limit V_H, the normal voltage state is

$$V_L \le V_{MEAS} \le V_H. \tag{B.14}$$

Similarly, for lower and upper current limits,

$$I_L \le I_{MEAS} \le I_H. \tag{B.15}$$

If raw ADC thresholds are ever written directly to the LTC2945, the corresponding register value must be calculated from the ADC LSB size. For the current-sense channel,

$$N_I = \frac{I_{LIMIT} R_{SHUNT}}{25\,\mu\text{V/LSB}}, \tag{B.16}$$

and for the rail-voltage channel,

$$N_V = \frac{V_{LIMIT}}{25\,\text{mV/LSB}}. \tag{B.17}$$

B.7 Controller-Supply Power Budget

A preliminary controller power budget was developed using datasheet maximum values where available and conservative design-current allowances for assemblies where a complete board-level maximum current is not specified. A 25% design margin is applied to the calculated load.

Table B.2: Preliminary controller-side power budget.

Load	Supply (V)	Design Current (A)	Power (W)
Arduino GIGA R1 WiFi	5.0	0.500	2.500
GIGA Display Shield	3.3	0.250	0.825
W6100 Ethernet interface	3.3	0.265	0.875
I^2C isolation/controller circuitry	3.3	0.065	0.215
Subtotal	–	–	4.415
25% design margin	–	–	1.104
Total	–	–	**5.519**

B.8 Component Capability Checks

The calculations above establish the required electrical values. The following checks compare those values with the capability of the selected components.

B.9 Current-Sense Resistor Power Margin

The selected LVK-series 39 mΩ and 50 mΩ four-terminal shunts are rated at 0.75 W. At the maximum design current of 2 A, the 39 mΩ resistor dissipates

$$P = (2\,\mathrm{A})^2(0.039\,\Omega) = 0.156\,\mathrm{W}. \tag{B.18}$$

This is approximately 20.8% of the 0.75 W rating, corresponding to about a 4.8:1 power-rating margin before considering PCB temperature and derating. At 1 A, a 50 mΩ shunt dissipates only 0.050 W, approximately 6.7% of a 0.75 W rating.

B.10 LTC2945 Input and ADC Range

The highest monitored nominal rail is 15 V, far below the LTC2945 0–80 V rail monitoring range. The selected production shunts produce 78 mV at 2 A on the 39 mΩ channels and 50 mV at 1 A on the 50 mΩ channels. Both values are below the 102.4 mV current-sense full scale, leaving approximately 24.4 mV and 52.4 mV of ADC headroom respectively. The LTC2945 can operate from an external 2.7–5.9 V supply, so a 3.3 V sensor-side logic supply is within its specified operating range.

B.11 Microcontroller I/O Compatibility

The Arduino GIGA R1 WiFi operates with 3.3 V logic and specifies 8 mA maximum DC current per I/O pin. The controller-side I^2C interface is a logic interface, not a load-power interface. The TCA9803 used in the design supports a 0.8–3.6 V A-side supply and a 1.65–3.6 V B-side supply and provides an internal B-side current source of up to 3 mA. A 3 mA bus-low current is below the 8 mA GIGA per-pin limit, providing more than a factor of two current margin. The GIGA pins therefore do not directly source the optocoupler LED or rail-load current; the external interface circuitry performs the required buffering and isolation.

B.12 Digital-Isolation Margin

The ACPL-064L digital optocoupler is specified for 3.3 V/5 V operation and is UL 1577 recognized for 3750 V_{RMS} isolation for one minute. The largest normal potential difference between the nominal monitored rails is only 30 V from -15 V to +15 V. The optocoupler rating therefore greatly exceeds the functional isolation voltage required by this low-voltage monitoring system. This comparison verifies component capability; it is not intended to redefine the safety rating of the complete assembled product, which also depends on PCB creepage, clearance, connectors, wiring, and enclosure construction.

B.13 3.3 V Regulator Loading

The TPS62912 is rated for up to 2 A output. Using the Week 1 design-current allowances for the display, Ethernet interface, and controller-side isolation circuitry gives an estimated 3.3 V load of

$$I_{3.3} = 0.250 + 0.265 + 0.065 = 0.580\,\text{A}. \tag{B.19}$$

This is approximately 29% of the regulator's 2 A output rating and leaves substantial current margin for transient and implementation variation.

B.14 Tolerance and Measurement-Uncertainty Considerations

The production current-sense resistors have a 1% resistance tolerance. Because current is calculated from $I = V_{SHUNT}/R_{SHUNT}$, an uncorrected 1% shunt error produces approximately a 1% gain error in calculated current. On a 2 A range, this corresponds to approximately 20 mA, which is much larger than the 0.641 mA quantization step of the 39 mΩ channel. The LTC2945 also has its own conversion error; therefore component tolerance rather than ADC quantization is an important contributor to absolute current accuracy. If higher accuracy is

required, the actual shunt resistance can be measured and a per-channel calibration factor stored in firmware.

A conservative worst-case linear addition of a 1% shunt tolerance and the LTC2945's less-than-±0.75% total unadjusted conversion error gives an upper-bound estimate of approximately ±1.75% before wiring, thermal, and calibration effects. This estimate is used for design margin rather than as a claim of final calibrated system accuracy.

B.15 Reference Specifications Used for Capability Checks

- Analog Devices, LTC2945 data sheet and product information: `https://www.analog.com/en/products/ltc2945.html`
- Arduino, GIGA R1 WiFi technical specifications: `https://store.arduino.cc/products/giga-r1-wifi`
- Broadcom, ACPL-064L low-power digital optocoupler data sheet/product page: `https://www.broadcom.com/products/optocouplers/industrial-plastic/digital-optocouplers/10mbd/acpl-064l-000e`
- Texas Instruments, TCA9803 I^2C buffer data sheet: `https://www.ti.com/product/TCA9803`
- Texas Instruments, TPS62912 buck converter data sheet: `https://www.ti.com/product/TPS62912`
- Ohmite LVK-series current-sense resistor specifications: `https://www.ohmite.com/assets/files/Datasheets/lvk-series.pdf`

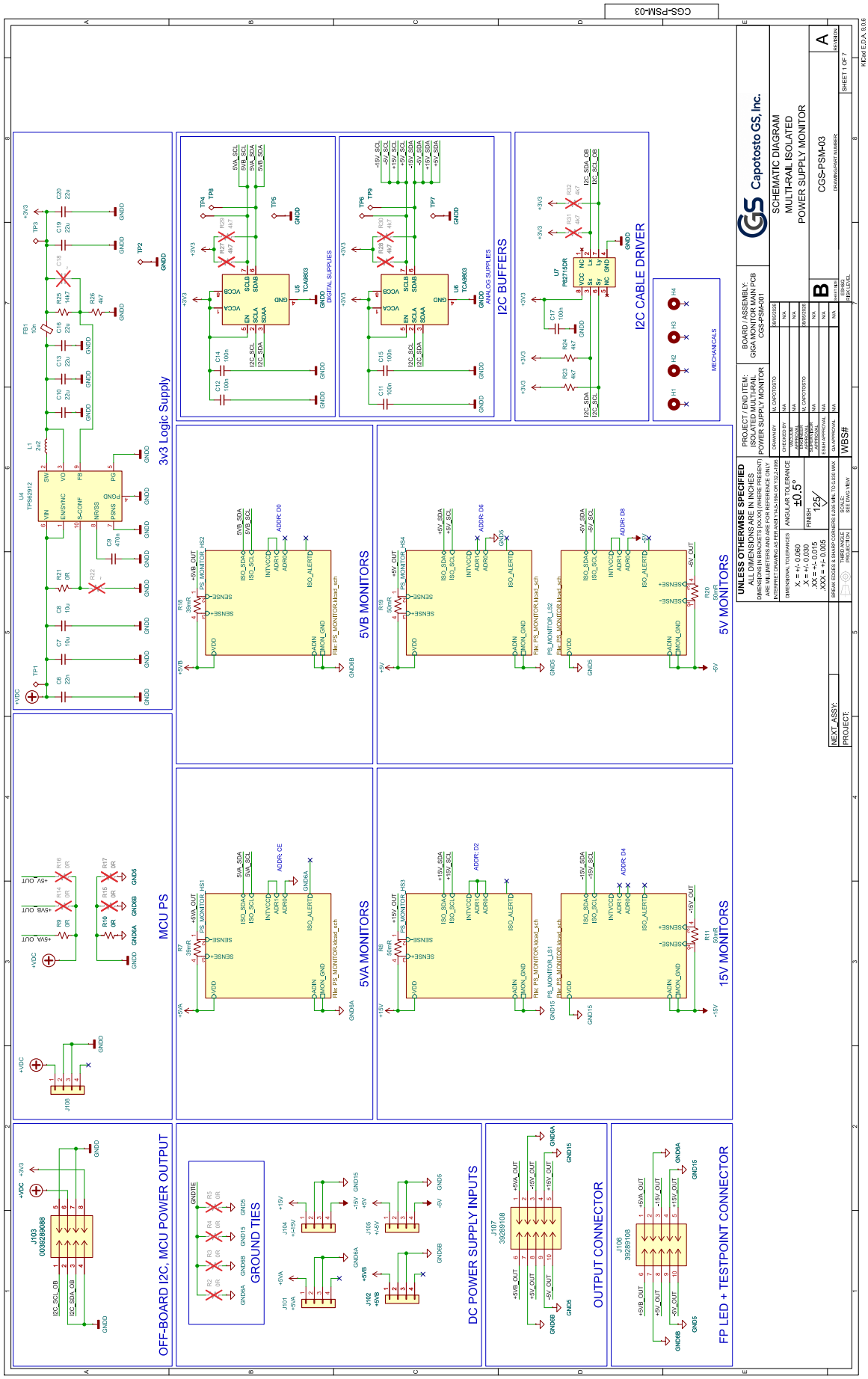
Appendix C

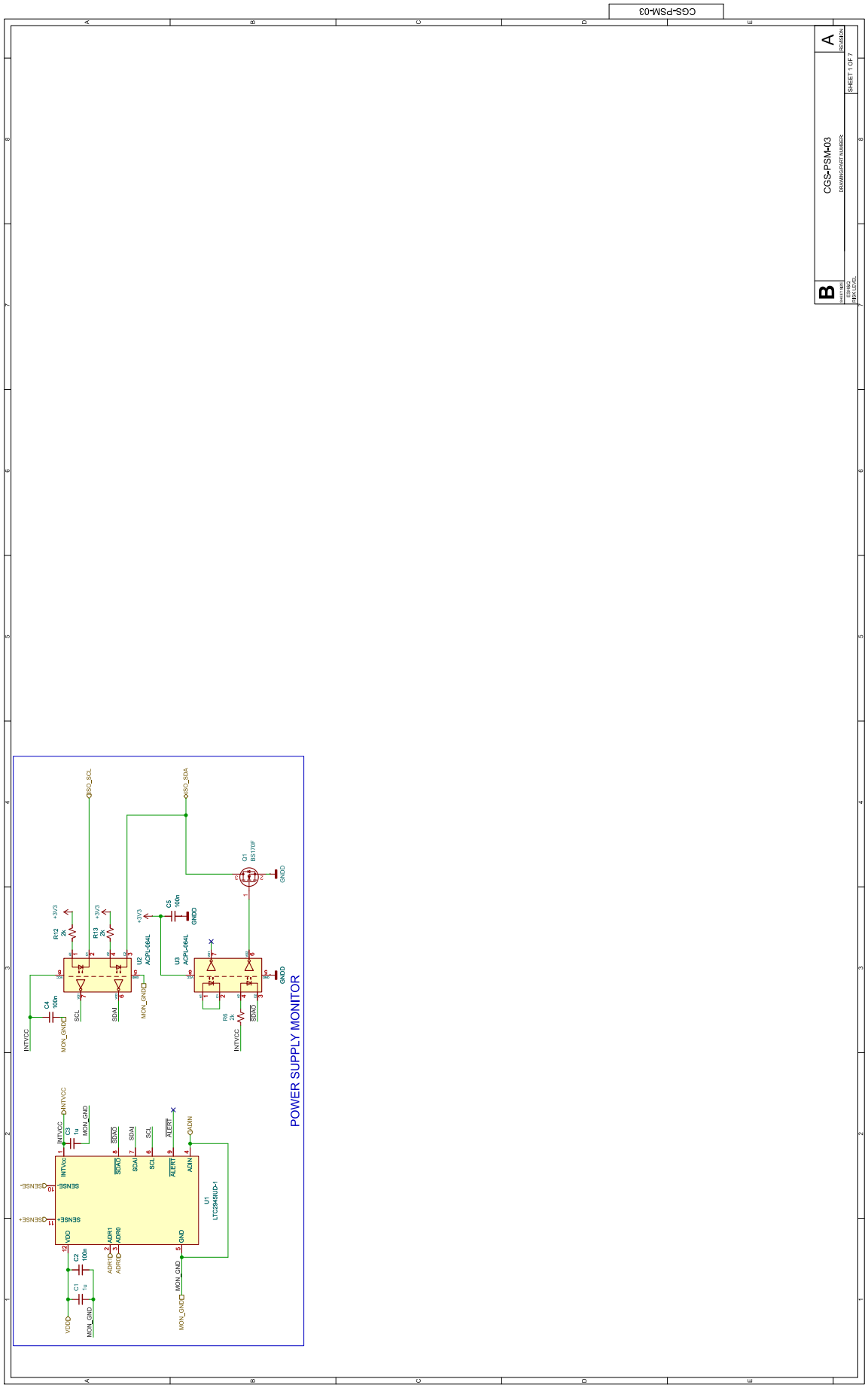
Design Documents

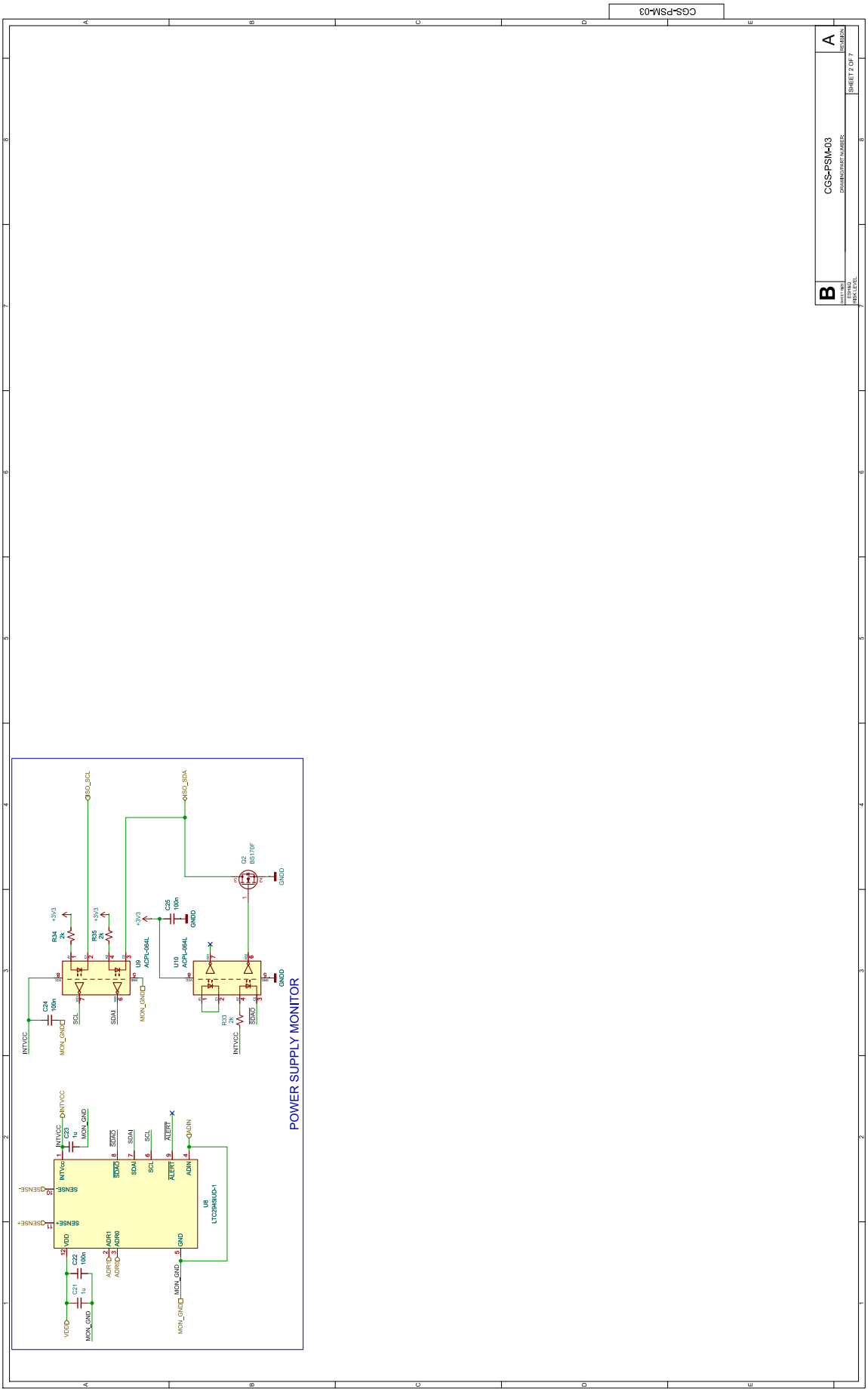
C.1 Schematics

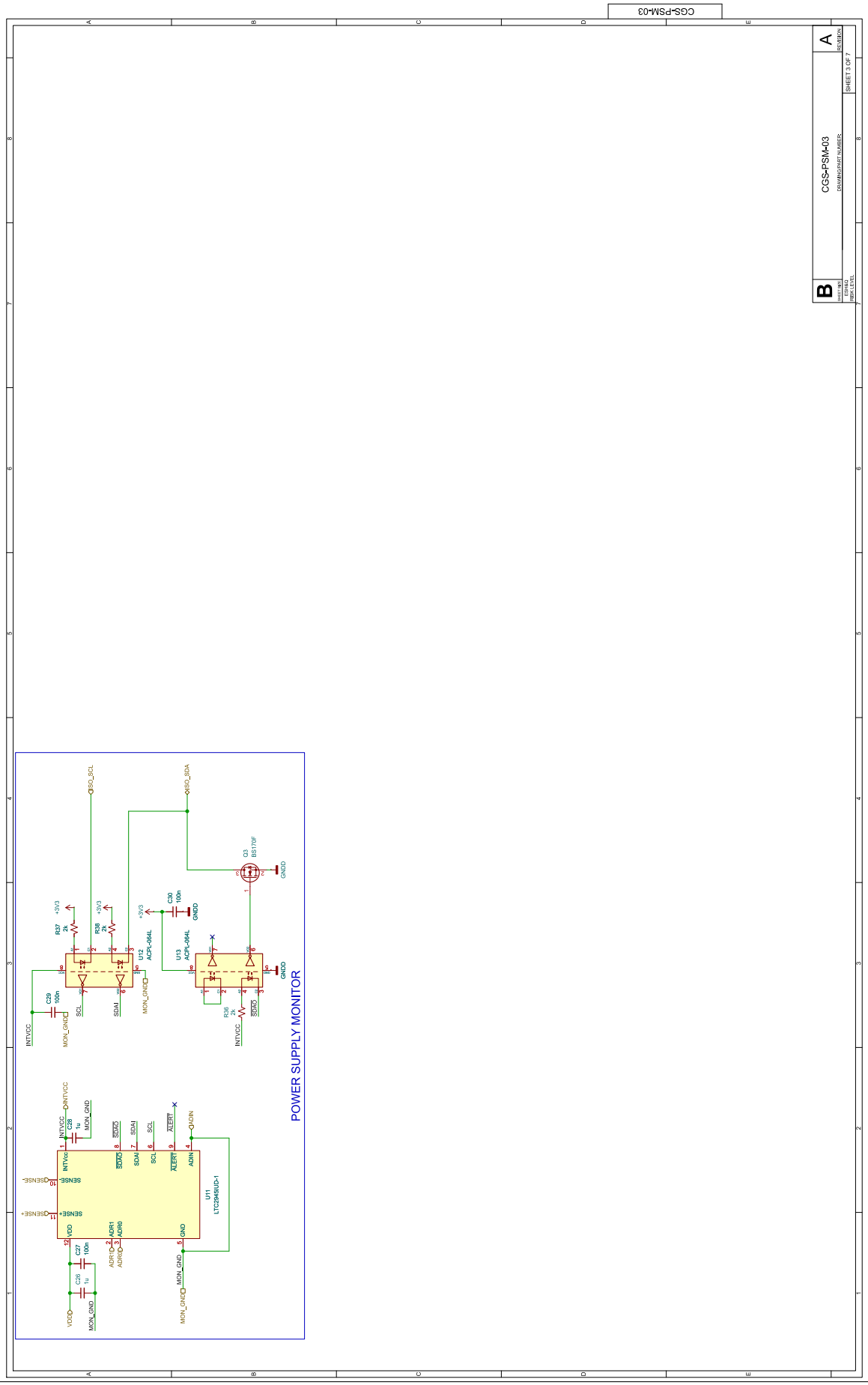
The hardware schematics are maintained in KiCad and are included below directly from the plotted PDF files in the project hardware directory.

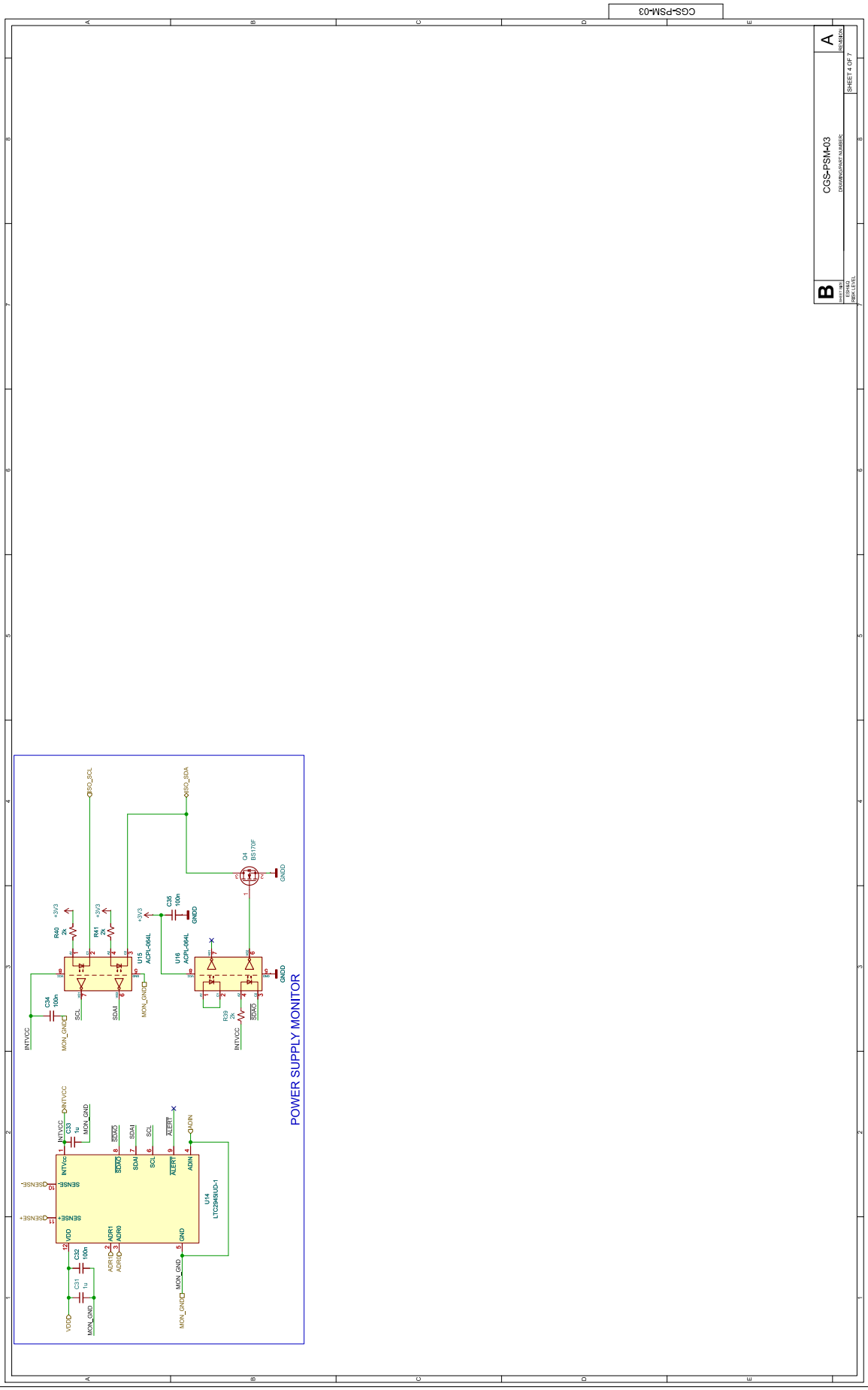
C.1.1 Sensor Board Schematic

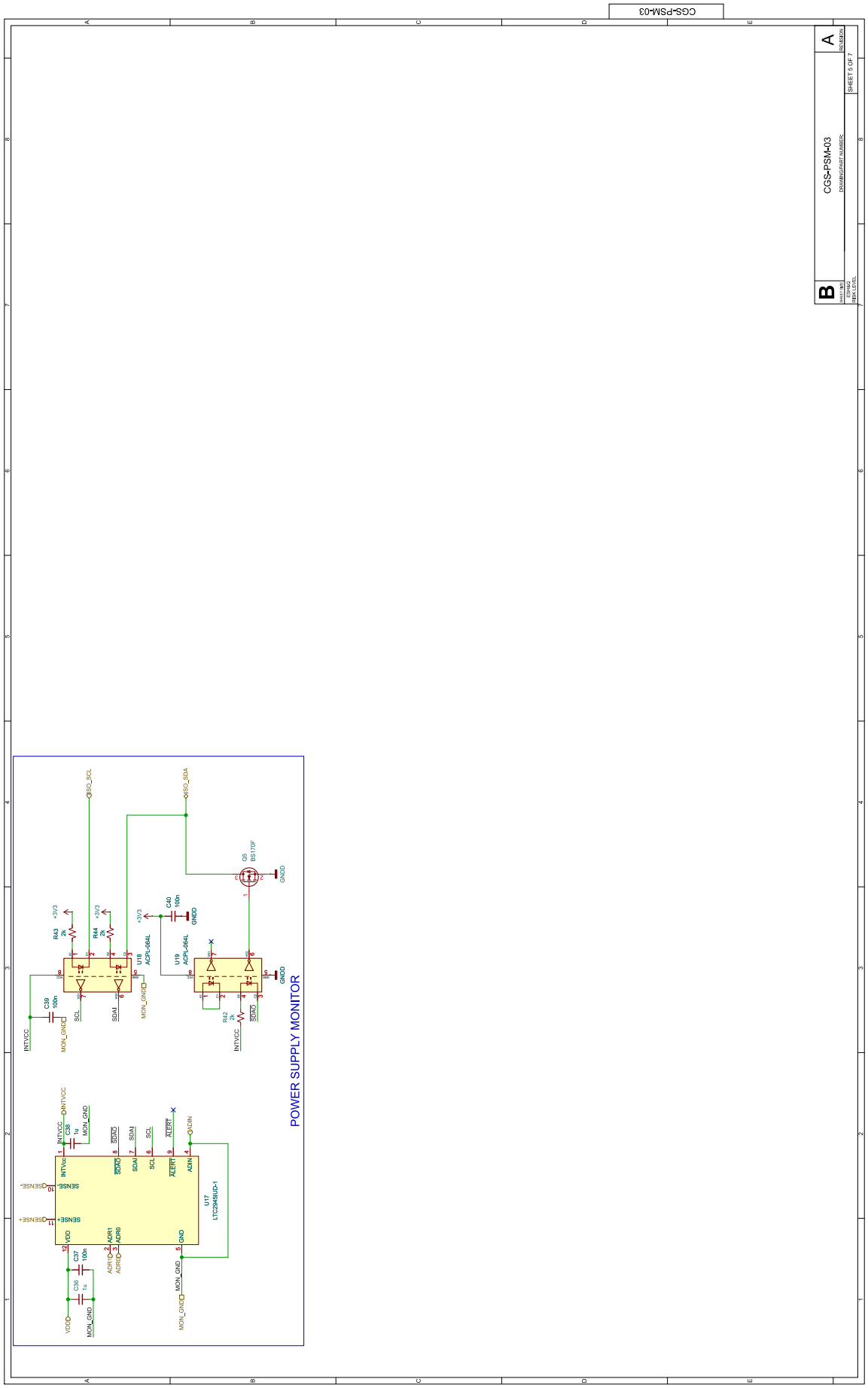

CGS-PSM-03
3v3 Logic Supply
MCU PS
5VB MONITORS
5VA MONITORS
15V MONITORS
5V MONITORS
I2C BUFFERS
I2C CABLE DRIVER
MECHANICALS
OFF-BOARD I2C, MCU POWER OUTPUT
GROUND TIES
DC POWER SUPPLY INPUTS
OUTPUT CONNECTOR
FP LED + TESTPOINT CONNECTOR
Capotosto GS, Inc.
SCHEMATIC DIAGRAM
MULTI-RAIL ISOLATED
POWER SUPPLY MONITOR
CGS-PSM-03
A
B
SHEET 1 OF 7

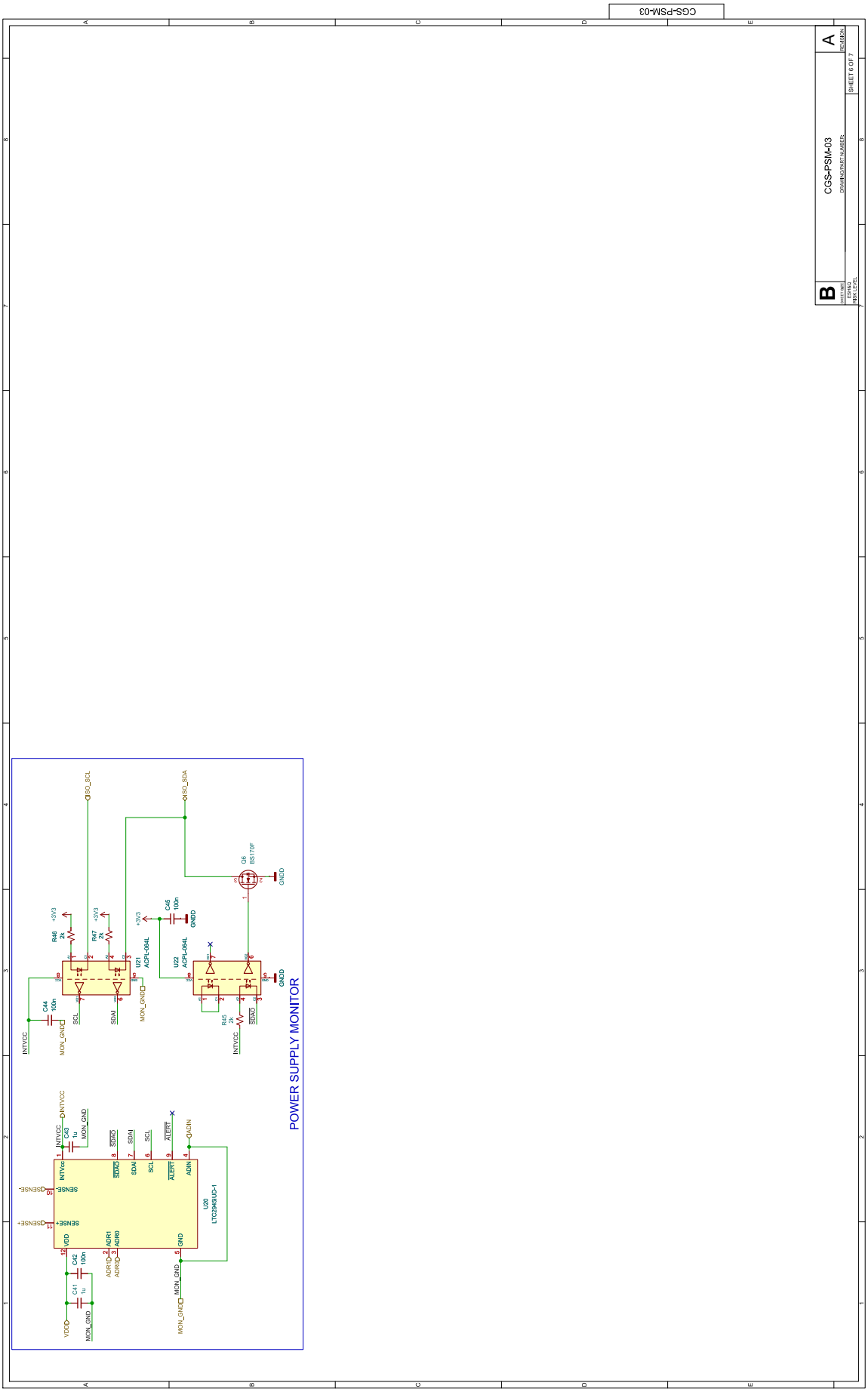

CGS-PSM-03
CGS-PSM-03
A
SHEET 1 OF 7
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
INTVCC
MON_GND
GNDD
+3V3
SCL
SDAI
SDAO
ALERT
ADIN
SENSE-
SENSE+
VDD
ADR1
ADR0
U1
LTC2945IUD-1
U2
ACPL-064L
U3
ACPL-064L
Q1
BS170F


CGS-PSM-03
CGS-PSM-03
A
SHEET 2 OF 7
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
+3V3
INTVCC
MON_GND
GNDD
VDDD
U8
LTC2945IUD-1
U9
ACPL-064L
U10
ACPL-064L
Q2
BS170F
C21
C22
C23
C24
C25
R33
R34
R35

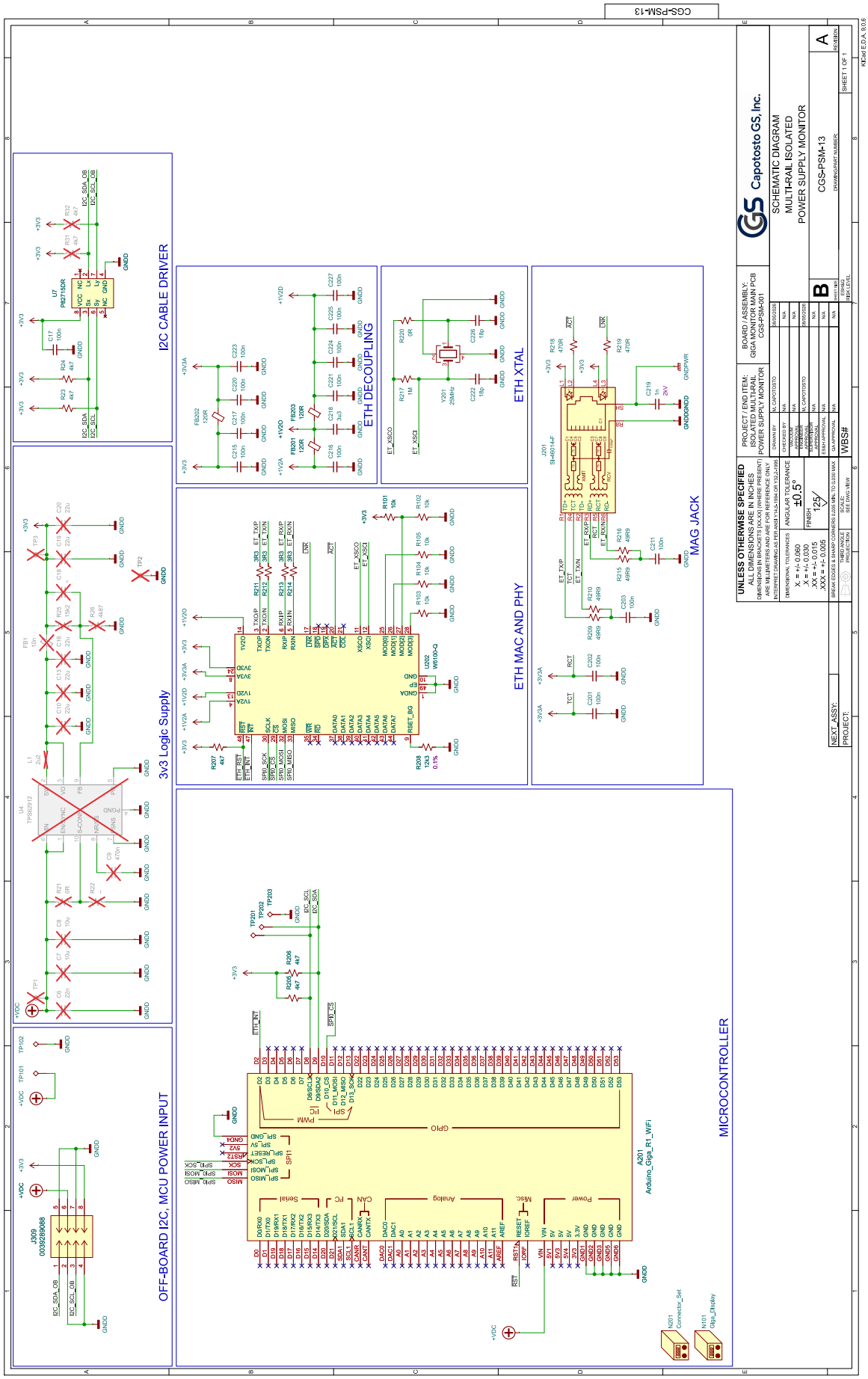

CGS-PSM-03
CGS-PSM-03
A
SHEET 3 OF 7
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
INTVCC
MON_GND
GNDD
+3V3
SCL
SDAI
SDAO
ALERT
ADIN
VDDD
SENSE-
SENSE+
VDD
ADR1
ADR0
GND
U11
LTC2945IUD-1
U12
ACPL-064L
U13
ACPL-064L
Q3
BS170F
R37
2k
R38
2k
R36
2k
C26
1u
C27
100n
C28
1u
C29
100n
C30
100n


CGS-PSM-03
CGS-PSM-03
A
SHEET 4 OF 7
B
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
+3V3
INTVCC
MON_GND
GNDD
VDDD
SCL
SDAI
SDAO
ALERT
ADIN
SENSE-
SENSE+
VDD
ADR1
ADR0
GND
C4
BS170F
C5
100n
C32
100n
C33
1u
C34
100n
C31
1u
R40
2k
R41
2k
R39
2k
U14
LTC2945IUD-1
U15
ACPL-064L
U16
ACPL-064L


CGS-PSM-03
A
CGS-PSM-03
SHEET 5 OF 7
B
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
Q5
BS170F
GNDD
+3V3
R43
2k
R44
2k
C40
100n
U18
ACPL-064L
U19
ACPL-064L
C39
100n
MON_GNDD
SCL
SDAI
SDAO
INTVCC
R42
2k
C38
1u
ALERT
ADIN
U17
LTC2945IUD-1
SENSE-
SENSE+
VDD
ADR1
ADR0
GND
C37
100n
C36
1u
VDDD


CGS-PSM-03
CGS-PSM-03
A
SHEET 6 OF 7
POWER SUPPLY MONITOR
ISO_SCL
ISO_SDA
Q5
BS170F
GNDD
+3V3
R46
2k
R47
2k
C45
100n
U21
ACPL-064L
U22
ACPL-064L
C44
100n
SCL
SDA1
MON_GND
R45
2k
SDAO
INTVCC
C43
1u
ALERT
ADIN
U20
LTC2945IUD-1
SENSE-
SENSE+
VDD
ADR1
ADR0
GND
C42
100n
C41
1u
VDDD

C.1.2 Arduino Shield Board Schematic


OFF-BOARD I2C, MCU POWER INPUT
3v3 Logic Supply
I2C CABLE DRIVER
ETH DECOUPLING
ETH XTAL
ETH MAC AND PHY
MAG JACK
MICROCONTROLLER
Capotosto GS, Inc.
SCHEMATIC DIAGRAM
MULTI-RAIL ISOLATED
POWER SUPPLY MONITOR
CGS-PSM-13
A
B
UNLESS OTHERWISE SPECIFIED
ALL DIMENSIONS ARE IN INCHES
PROJECT / END ITEM:
GIGA KNOB SHIELD MAIN PCB
CGS-PSM-001
BOARD / ASSEMBLY:
GIGA KNOB SHIELD MAIN PCB
CGS-PSM-A001
NEXT ASSY:
PROJECT:
WBS#
SHEET 1 OF 1

C.2 Software Flow Chart

This appendix provides the detailed software flow requested for the Week 2 preliminary-design package. It shows the firmware acquisition/alarm loop and the PSC-to-EPICS data path used by the implemented software.

The software for the Multi-Rail Isolated Power Supply Monitoring System is maintained with the hardware design files in the project Git repository:

`https://github.com/capotosto/PS_Monitor`

The repository provides version control for the Arduino GIGA firmware, hardware-design files, and EPICS IOC software used by the monitoring system. The embedded firmware is maintained under the `firmware` portion of the repository, while the EPICS IOC and associated database/configuration files are maintained under `epics-ioc`. The flowcharts in this section describe the major software operations implemented by these portions of the project.

The Arduino GIGA firmware performs system initialization, acquires voltage and current measurements from the six LTC2945 monitor channels, evaluates alarm conditions, updates the local display, and publishes data through the Ethernet interface. The primary firmware execution sequence is:

1. Initialize the GIGA Display, persistent settings, Ethernet interface, I^2C interface, and channel configuration.
2. Load stored network parameters and voltage/current alarm limits from persistent storage.
3. Initialize communication with each of the six LTC2945 monitoring channels.
4. Enter the continuous acquisition and communication loop.
5. Poll each LTC2945 channel for voltage and current measurements.
6. Verify I^2C communication status for each channel.
7. If communication fails, mark the associated channel with an I^2C error and prevent invalid measurement data from being used.
8. Convert valid raw measurement data to engineering units and apply the appropriate channel polarity and calibration/scaling information.
9. Compare each voltage and current measurement with its configured upper and lower alarm limits.
10. Update the GIGA Display when measurement values, alarm states, or communication states change.

11. Publish the current measurements and status information through the Ethernet PSC protocol interface.
12. Service incoming configuration requests and store modified settings when required.
13. Repeat the acquisition and communication sequence continuously.

C.2.1 Arduino GIGA Firmware Flowchart

Figure C.1 shows the top-level execution flow of the Arduino GIGA firmware. Following initialization, the controller continuously acquires data from each monitoring channel, verifies communication, converts valid measurements to engineering units, evaluates alarm limits, updates the local display, and services the Ethernet/PSC interface.

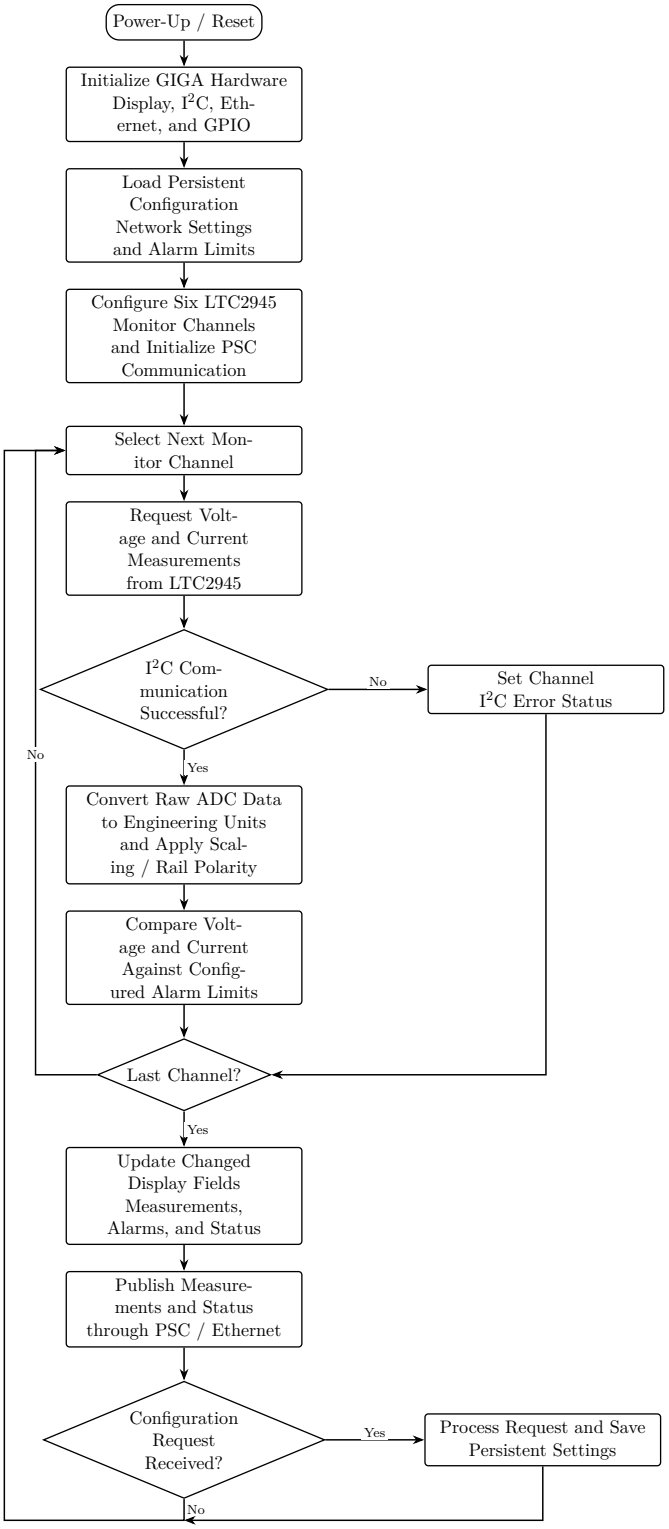


Figure C.1: Top-level Arduino GIGA firmware flowchart. Following initialization, the controller continuously acquires measurements from the six LTC2945 monitoring channels, validates I^2C communication, converts valid data to engineering units, evaluates configured alarm limits, updates the local display, publishes measurements through the PSC/Ethernet interface, and services configuration requests.

C.2.2 PSC and EPICS Communication Flow

The Arduino GIGA does not directly implement EPICS Channel Access. Instead, measurement and status information is transmitted through the Ethernet interface using the PSC protocol. The EPICS IOC uses PSCDriver to communicate with the GIGA, maps the received values into EPICS process variables, and makes those process variables available to the CS-Studio Phoebus operator interface.

This separation allows the embedded controller to remain responsible for hardware acquisition and local alarm evaluation while the IOC provides the interface between the embedded hardware and the facility control system. Figure C.2 summarizes this software data path.

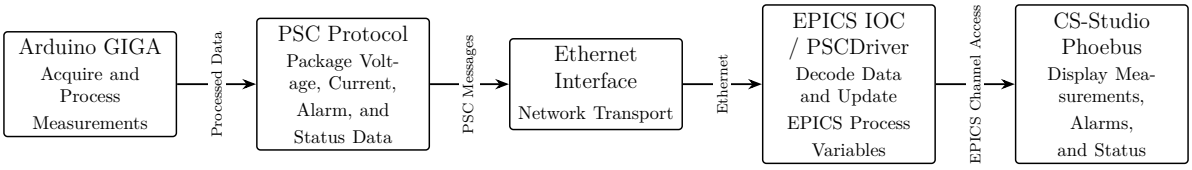


Figure C.2: Software communication path from the Arduino GIGA firmware through PSCDriver and the EPICS IOC to the CS-Studio Phoebus operator interface.

C.2.3 Software Configuration Management

Firmware, IOC configuration files, database definitions, and operator-interface files are maintained under Git version control so that hardware and software revisions can be associated with a defined project baseline. The current software implementation and revision history are available in the `PS_Monitor` repository at:

`https://github.com/capotosto/PS_Monitor`

Changes made during implementation and testing are committed to the repository, providing traceability between the preliminary design presented in this report and the final implemented system. The repository therefore serves as the controlled source for the firmware and control-system software associated with the project.

C.3 Source Code

The following listings provide the source code for the Arduino GIGA firmware and the configuration files used by the EPICS IOC.

The original source files and revision history are maintained at:

`https://github.com/capotosto/PS_Monitor`

The embedded firmware, IOCs, and supporting scripts were developed through an iterative workflow combining self-authored code and AI-assisted optimizations, refactoring, expanding of features, etc. (OpenAI, 2026).

C.3.1 Firmware Source Files

```
/*
  Arduino GIGA R1 + GIGA Display Shield + Ethernet Shield 2
  Modular six-channel power monitor.

  Services:
    HTTP dashboard: TCP port 80
    EPICS PSCDriver: TCP port 8765
*/

#include <Arduino.h>
#include "AppConfig.h"
#include "AppState.h"
#include "DisplayUI.h"
#include "EthernetServices.h"
#include "HttpDashboard.h"
#include "PersistentStorage.h"
#include "PscProtocol.h"
#include "SensorConfig.h"
#include "SensorManager.h"

namespace {
uint32_t lastSensorPollMs = 0;
uint32_t lastDisplayMs = 0;
}

void setup() {
  Serial.begin(115200);

  Serial.begin(115200);

  delay(10000);

  const uint32_t serialWaitStart = millis();
  while (!Serial && millis() - serialWaitStart < 1500) {
    delay(10);
  }

  initializeSettingsStorage();  //Initialize QSPI configuration
  initializeSensors();  //Initialize I2C PMICs
  initializeDisplay();  //Initialize display shield
  initializeEthernet();  //Initialize ethernet shield
  redrawDisplay();  //Redraw display
}

```

```
void loop() {
  const uint32_t now = millis();

  if (now - lastSensorPollMs >= SENSOR_POLL_PERIOD_MS) {
    lastSensorPollMs = now;
    updateSensorMeasurements();  //Update the sensor readouts
  }

  if (now - lastDisplayMs >= DISPLAY_REFRESH_PERIOD_MS) {
    lastDisplayMs = now;
    redrawDisplay();  //Redraw display with new values
  }

  /*
    PSC is serviced both before and after HTTP so a browser request cannot
    unnecessarily delay the persistent IOC connection.
  */
  servicePsc();  //R/W to the IOC
  serviceHttp();  //R/W to the webpage
  servicePsc();  //R/W to the IOC
}
```

Listing C.1: Firmware: src/main.cpp

```
/*
  Global application state for the multi-rail power supply monitor.

  This module defines:
    - compiled/default Ethernet configuration
    - runtime network configuration
    - configuration and live state for all six monitor channels
    - system-wide storage/network status flags
    - common alarm-state evaluation functions

  The channel array is the shared source of live measurement, limit, alarm,
  and sensor-status information used by the display, HTTP interface, PSC
  protocol, and persistent-storage modules.
*/

#include "AppState.h"

#include <Arduino.h>

//Default static Ethernet configuration.
const NetworkSettings COMPILED_NETWORK_SETTINGS = {
  {192, 168, 1, 177},  // IP address
  {192, 168, 1, 1},    // DNS server
  {192, 168, 1, 1},    // Gateway
  {255, 255, 255, 0}   // Subnet mask
};

NetworkSettings networkSettings = COMPILED_NETWORK_SETTINGS;

/*Initial state for each monitored rail.

  MonitorChannel field order:
    name
    nominal voltage
    nominal current
    measured voltage
    measured current
    alarm limits {V low, V high, I low, I high}
    voltage-low alarm
    voltage-high alarm
    current-low alarm
    current-high alarm
    reading valid
```

```
    sensor online
    sensor/I2C error code
    consecutive sensor failures
    timestamp of last valid reading

  Measured values and status fields are initialized to zero/false and are
  populated by SensorManager after the LTC2945 devices are initialized*/
MonitorChannel channels[CHANNEL_COUNT] = {
  // name, nominal V/I, measured V/I, limits, four alarms,
  // reading valid, sensor online, error, failures, last-valid time
  {"CH1 +5VA",   5.00f, 0.60f, 0.0f, 0.0f,
   { 4.40f,  5.60f, 0.00f, 1.20f},
   false, false, false, false, false, false, 0, 0, 0},

  {"CH2 +5VB",   6.00f, 0.60f, 0.0f, 0.0f,
   { 4.40f,  5.60f, 0.00f, 1.20f},
   false, false, false, false, false, false, 0, 0, 0},

  {"CH3 +5V",    5.00f, 0.50f, 0.0f, 0.0f,
   { 4.50f,  5.50f, 0.00f, 1.00f},
   false, false, false, false, false, false, 0, 0, 0},

  {"CH4 -5V",   -5.00f, 0.50f, 0.0f, 0.0f,
   {-5.50f, -4.50f, 0.00f, 1.00f},
   false, false, false, false, false, false, 0, 0, 0},

  {"CH5 +15V",  15.00f, 0.35f, 0.0f, 0.0f,
   {13.50f, 16.50f, 0.00f, 0.75f},
   false, false, false, false, false, false, 0, 0, 0},

  {"CH6 -15V", -15.00f, 0.35f, 0.0f, 0.0f,
   {-16.50f, -13.50f, 0.00f, 0.75f},
   false, false, false, false, false, false, 0, 0, 0}
};

/*System status flags shared between application modules.

  settingsStorageReady   - QSPI settings filesystem mounted and usable
  networkRestartRequired - saved network settings differ from active settings
  ethernetHardwarePresent- Ethernet controller was detected successfully*/
bool settingsStorageReady = false;
bool networkRestartRequired = false;
bool ethernetHardwarePresent = false;

//Return the overall fault/alarm state for a channel
bool channelIsInAlarm(const MonitorChannel &channel) {
  return !channel.sensorOnline ||
         channel.voltageLowAlarm ||
         channel.voltageHighAlarm ||
         channel.currentLowAlarm ||
         channel.currentHighAlarm;
}

/*Recalculate the four electrical alarm flags from the latest measurement
  and the configured alarm limits.

    UVL - undervoltage / insufficient voltage magnitude
    OVL - overvoltage / excessive voltage magnitude
    UCL - current below lower limit
    OCL - current above upper limit

  Positive and negative voltage rails require different numeric comparisons
  for UVL/OVL. For example, on a -5 V rail:

    -4.3 V is insufficient magnitude -> UVL
    -5.7 V is excessive magnitude    -> OVL

  Current is represented as positive load-current magnitude, so current
```

```
  alarm comparisons use the same direction for positive and negative rails.*/
void updateAlarmState(MonitorChannel &channel) {
  /*
    These booleans are semantic:
      voltageLowAlarm  = UVL
      voltageHighAlarm = OVL

    For a negative rail:
      -5.7 V is excessive magnitude and must be OVL.
      -4.3 V is insufficient magnitude and must be UVL.
  */
  if (channel.nominalVoltage < 0.0f) {
    channel.voltageLowAlarm =
      channel.voltage > channel.limits.voltageHigh;  // UVL

    channel.voltageHighAlarm =
      channel.voltage < channel.limits.voltageLow;   // OVL
  } else {
    channel.voltageLowAlarm =
      channel.voltage < channel.limits.voltageLow;   // UVL

    channel.voltageHighAlarm =
      channel.voltage > channel.limits.voltageHigh;  // OVL
  }

  channel.currentLowAlarm =
    channel.current < channel.limits.currentLow;      // UCL

  channel.currentHighAlarm =
    channel.current > channel.limits.currentHigh;     // OCL
}
```

Listing C.2: Firmware: src/AppState.cpp

```
/*
  Local GIGA Display UI

  The display presents all six monitored rails with Ethernet/network status.

  To minimize display flicker and unnecessary re-drawing, the UI
  maintains a cached snapshot of the values currently represented on screen.
  During each refresh, the current application state is compared with the
  cached state and only fields that have changed are redrawn.

  A complete channel card is redrawn when its overall alarm state changes
  because the card background/border color also changes.
  */
#include "DisplayUI.h"

#include <Arduino.h>
#include <Arduino_GigaDisplay_GFX.h>
#include <Ethernet.h>

#include <math.h>
#include <stdio.h>
#include <string.h>

#include "AppConfig.h"
#include "AppState.h"

namespace {

GigaDisplay_GFX display;

uint16_t COLOR_BLACK;
uint16_t COLOR_WHITE;
uint16_t COLOR_BACKGROUND;
```

```
uint16_t COLOR_HEADER;
uint16_t COLOR_CARD;
uint16_t COLOR_BORDER;
uint16_t COLOR_OK;
uint16_t COLOR_ALARM;
uint16_t COLOR_WARNING;
uint16_t COLOR_CYAN;
uint16_t COLOR_MUTED;

constexpr int SCREEN_WIDTH = 800;
constexpr int SCREEN_HEIGHT = 480;
constexpr int HEADER_HEIGHT = 58;
constexpr int MARGIN_X = 12;
constexpr int GAP_X = 10;
constexpr int GAP_Y = 10;
constexpr int CARD_WIDTH = 252;
constexpr int CARD_HEIGHT = 202;

/*Cached representation of the information currently drawn for one channel.

  Floating-point measurements and limits are converted to integer milli-units
  before comparison.
  AlarmMask captures the individual alarm/fault states so the status field
  can be refreshed when any condition changes.*/
struct ChannelDisplaySnapshot {
  bool initialized = false;

  int32_t voltageMilli = 0;
  int32_t currentMilli = 0;

  int32_t voltageLowMilli = 0;
  int32_t voltageHighMilli = 0;
  int32_t currentLowMilli = 0;
  int32_t currentHighMilli = 0;

  uint8_t alarmMask = 0;
  bool readingValid = false;
  bool sensorOnline = false;
  bool overallAlarm = false;
};

struct HeaderDisplaySnapshot {
  bool initialized = false;
  bool hardwarePresent = false;
  int linkStatus = -1;
  uint32_t packedIp = 0;
};

ChannelDisplaySnapshot channelDisplayCache[CHANNEL_COUNT];
HeaderDisplaySnapshot headerDisplayCache;

/*
  Screen arrangement:

    +5VA     +5V      +15V
    +5VB     -5V      -15V

  Values are indexes into channels[].
*/
constexpr uint8_t DISPLAY_ORDER[CHANNEL_COUNT] = {
  0, 2, 4,
  1, 3, 5
};

/*Convert floats to 0.001 base-unit resolution for display
  change detection. For voltage this corresponds to 1 mV; for current stored
  in amperes it corresponds to 1 mA.*/
int32_t toDisplayedMilli(float value) {
```

```
  return static_cast<int32_t>(lroundf(value * 1000.0f));
}

uint32_t packIp(const IPAddress &ip) {
  return
    (static_cast<uint32_t>(ip[0]) << 24) |
    (static_cast<uint32_t>(ip[1]) << 16) |
    (static_cast<uint32_t>(ip[2]) << 8) |
    static_cast<uint32_t>(ip[3]);
}

/*Encode the alarm/fault conditions that affect the channel status display:

    bit 0 - voltage below lower limit
    bit 1 - voltage above upper limit
    bit 2 - current below lower limit
    bit 3 - current above upper limit
    bit 4 - LTC2945/I2C communication fault

  The mask is cached so the header status is redrawn only when one of these
  conditions changes.*/
uint8_t makeDisplayAlarmMask(const MonitorChannel &channel) {
  uint8_t mask = 0;

  if (channel.voltageLowAlarm) {
    mask |= 1U << 0;  // UVL
  }
  if (channel.voltageHighAlarm) {
    mask |= 1U << 1;  // OVL
  }
  if (channel.currentLowAlarm) {
    mask |= 1U << 2;  // UCL
  }
  if (channel.currentHighAlarm) {
    mask |= 1U << 3;  // OCL
  }
  if (!channel.sensorOnline) {
    mask |= 1U << 4;  // I2C
  }

  return mask;
}

/*Capture the current channel state*/
ChannelDisplaySnapshot captureChannel(
  const MonitorChannel &channel
) {
  ChannelDisplaySnapshot snapshot;

  snapshot.initialized = true;

  snapshot.voltageMilli = toDisplayedMilli(channel.voltage);
  snapshot.currentMilli = toDisplayedMilli(channel.current);

  snapshot.voltageLowMilli =
    toDisplayedMilli(channel.limits.voltageLow);

  snapshot.voltageHighMilli =
    toDisplayedMilli(channel.limits.voltageHigh);

  snapshot.currentLowMilli =
    toDisplayedMilli(channel.limits.currentLow);

  snapshot.currentHighMilli =
    toDisplayedMilli(channel.limits.currentHigh);

  snapshot.alarmMask = makeDisplayAlarmMask(channel);
  snapshot.readingValid = channel.readingValid;
```

```
  snapshot.sensorOnline = channel.sensorOnline;
  snapshot.overallAlarm = channelIsInAlarm(channel);

  return snapshot;
}

HeaderDisplaySnapshot captureHeader() {
  HeaderDisplaySnapshot snapshot;

  snapshot.initialized = true;
  snapshot.hardwarePresent = ethernetHardwarePresent;

  if (!ethernetHardwarePresent) {
    snapshot.linkStatus = -1;
    snapshot.packedIp = 0;
    return snapshot;
  }

  snapshot.linkStatus =
    static_cast<int>(Ethernet.linkStatus());

  snapshot.packedIp = packIp(Ethernet.localIP());
  return snapshot;
}

bool headerChanged(
  const HeaderDisplaySnapshot &previous,
  const HeaderDisplaySnapshot &current
) {
  return
    !previous.initialized ||
    previous.hardwarePresent != current.hardwarePresent ||
    previous.linkStatus != current.linkStatus ||
    previous.packedIp != current.packedIp;
}

/*Convert a display slot number (0-5) into the upper-left pixel coordinates
  of its channel card within the 3-column by 2-row layout.*/
void cardPosition(
  uint8_t displaySlot,
  int &x,
  int &y
) {
  const uint8_t column = displaySlot % 3;
  const uint8_t row = displaySlot / 3;

  x = MARGIN_X + column * (CARD_WIDTH + GAP_X);
  y = HEADER_HEIGHT + row * (CARD_HEIGHT + GAP_Y);
}

uint16_t channelCardColor(const MonitorChannel &channel) {
  return channelIsInAlarm(channel)
    ? display.color565(72, 23, 29)
    : COLOR_CARD;
}

uint16_t channelStatusColor(const MonitorChannel &channel) {
  return channelIsInAlarm(channel)
    ? COLOR_ALARM
    : COLOR_OK;
}

void printAt(
  int x,
  int y,
  uint8_t size,
  uint16_t color,
  const char *text
```

```
) {
  display.setTextSize(size);
  display.setTextColor(color);
  display.setCursor(x, y);
  display.print(text);
}

void clearField(
  int x,
  int y,
  int width,
  int height,
  uint16_t background
) {
  display.fillRect(x, y, width, height, background);
}

//Draw screen elements that do not normally change during operation
void drawStaticLayout() {
  display.fillScreen(COLOR_BACKGROUND);
  display.fillRect(
    0,
    0,
    SCREEN_WIDTH,
    HEADER_HEIGHT,
    COLOR_HEADER
  );

  printAt(14, 9, 3, COLOR_WHITE, "MULTI-RAIL MONITOR");
}

/*Draw the Ethernet status area in the screen header.
  Display states:
    - no Ethernet hardware detected
    - Ethernet hardware present but physical link is down
    - active Ethernet connection showing the current IP address and the
      HTTP/PSC service ports*/
void drawHeaderNetworkStatus() {
  display.fillRect(
    500,
    0,
    300,
    HEADER_HEIGHT,
    COLOR_HEADER
  );

  char line1[32];
  char line2[32];

  if (!ethernetHardwarePresent) {
    snprintf(line1, sizeof(line1), "ETHERNET: NO HW");
    snprintf(line2, sizeof(line2), "Check shield/SPI");

    printAt(510, 10, 2, COLOR_ALARM, line1);
    printAt(510, 35, 1, COLOR_MUTED, line2);
    return;
  }

  const EthernetLinkStatus link = Ethernet.linkStatus();

  if (link == LinkOFF) {
    snprintf(line1, sizeof(line1), "ETHERNET: LINK OFF");

    const IPAddress ip = Ethernet.localIP();

    snprintf(
      line2,
      sizeof(line2),
```

```
      "IP %u.%u.%u.%u",
      ip[0],
      ip[1],
      ip[2],
      ip[3]
    );

    printAt(510, 10, 2, COLOR_WARNING, line1);
    printAt(510, 35, 1, COLOR_MUTED, line2);
    return;
  }

  const IPAddress ip = Ethernet.localIP();

  snprintf(
    line1,
    sizeof(line1),
    "IP %u.%u.%u.%u",
    ip[0],
    ip[1],
    ip[2],
    ip[3]
  );

  snprintf(
    line2,
    sizeof(line2),
    "HTTP PORT# %u  EPICS PORT #%u",
    HTTP_PORT,
    PSC_PORT
  );

  printAt(510, 10, 2, COLOR_CYAN, line1);
  printAt(510, 35, 1, COLOR_MUTED, line2);
}

/*Update the status indicator in a channel card:

    I2C - sensor communication unavailable
    ALM - one or more configured alarm conditions active
    OK  - sensor online and no alarm conditions active
  Communication failure takes priority over normal alarm indication.*/
void drawChannelHeaderState(
  uint8_t channelIndex,
  uint8_t displaySlot
) {
  int x;
  int y;
  cardPosition(displaySlot, x, y);

  const MonitorChannel &channel = channels[channelIndex];
  const uint16_t background = channelCardColor(channel);
  const uint16_t statusColor = channelStatusColor(channel);

  const char *headerState =
    !channel.sensorOnline ? "I2C" :
    channelIsInAlarm(channel) ? "ALM" :
    "OK";

  clearField(
    x + 178,
    y + 6,
    66,
    25,
    background
  );

  printAt(
```

```
    x + 188,
    y + 9,
    2,
    statusColor,
    headerState
  );
}

//Redraw only the voltage field. Invalid measurements are displayed as "---"
void drawChannelVoltage(
  uint8_t channelIndex,
  uint8_t displaySlot
) {
  int x;
  int y;
  cardPosition(displaySlot, x, y);

  const MonitorChannel &channel = channels[channelIndex];
  const uint16_t background = channelCardColor(channel);

  char buffer[32];

  if (channel.readingValid) {
    snprintf(
      buffer,
      sizeof(buffer),
      "%7.3f V",
      channel.voltage
    );
  } else {
    snprintf(buffer, sizeof(buffer), "    --- V");
  }

  clearField(
    x + 8,
    y + 38,
    226,
    35,
    background
  );

  printAt(
    x + 10,
    y + 42,
    3,
    COLOR_CYAN,
    buffer
  );
}

//Redraw only the current field. Current is stored internally in amps
void drawChannelCurrent(
  uint8_t channelIndex,
  uint8_t displaySlot
) {
  int x;
  int y;
  cardPosition(displaySlot, x, y);

  const MonitorChannel &channel = channels[channelIndex];
  const uint16_t background = channelCardColor(channel);

  char buffer[32];

  if (channel.readingValid) {
    snprintf(
      buffer,
      sizeof(buffer),
```

```
      "%7.3f mA",
      channel.current * 1000.0f
    );
  } else {
    snprintf(buffer, sizeof(buffer), "    --- A");
  }

  clearField(
    x + 8,
    y + 74,
    226,
    35,
    background
  );

  printAt(
    x + 10,
    y + 78,
    3,
    COLOR_WHITE,
    buffer
  );
}

void drawChannelLimits(
  uint8_t channelIndex,
  uint8_t displaySlot
) {
  int x;
  int y;
  cardPosition(displaySlot, x, y);

  const MonitorChannel &channel = channels[channelIndex];
  const uint16_t background = channelCardColor(channel);

  char buffer[32];

  clearField(
    x + 8,
    y + 114,
    236,
    80,
    background
  );

  snprintf(
    buffer,
    sizeof(buffer),
    "VL (V)%6.2f..%6.2f",
    channel.limits.voltageLow,
    channel.limits.voltageHigh
  );

  printAt(
    x + 10,
    y + 126,
    2,
    COLOR_MUTED,
    buffer
  );

  snprintf(
    buffer,
    sizeof(buffer),
    "IL (A)%6.2f..%6.2f",
    channel.limits.currentLow,
    channel.limits.currentHigh
  );
```

```
510
511   printAt(
512     x + 10,
513     y + 158,
514     2,
515     COLOR_MUTED,
516     buffer
517   );
518 }
519
520 //Redraw an entire channel card.
521 void drawCompleteChannelCard(
522   uint8_t channelIndex,
523   uint8_t displaySlot
524 ) {
525   int x;
526   int y;
527   cardPosition(displaySlot, x, y);
528
529   const MonitorChannel &channel = channels[channelIndex];
530   const bool alarm = channelIsInAlarm(channel);
531   const uint16_t cardColor = channelCardColor(channel);
532
533   display.fillRect(
534     x,
535     y,
536     CARD_WIDTH,
537     CARD_HEIGHT,
538     cardColor
539   );
540
541   display.drawRect(
542     x,
543     y,
544     CARD_WIDTH,
545     CARD_HEIGHT,
546     alarm ? COLOR_ALARM : COLOR_BORDER
547   );
548
549   printAt(
550     x + 10,
551     y + 9,
552     2,
553     COLOR_WHITE,
554     channel.name
555   );
556
557   drawChannelHeaderState(channelIndex, displaySlot);
558   drawChannelVoltage(channelIndex, displaySlot);
559   drawChannelCurrent(channelIndex, displaySlot);
560   drawChannelLimits(channelIndex, displaySlot);
561 }
562
563 /*Refresh one channel using change detection.
564
565   The current channel state is compared with the cached state already shown
566   on screen.
567     - first update or overall alarm change -> redraw complete card
568     - voltage/validity change              -> redraw voltage only
569     - current/validity change              -> redraw current only
570     - alarm-limit change                   -> redraw limits only
571     - sensor/alarm-bit change              -> redraw status only*/
572 void refreshChannelCard(
573   uint8_t channelIndex,
574   uint8_t displaySlot
575 ) {
576   const MonitorChannel &channel = channels[channelIndex];
577
```

```
  ChannelDisplaySnapshot &previous =
    channelDisplayCache[channelIndex];

  const ChannelDisplaySnapshot current =
    captureChannel(channel);

  if (
    !previous.initialized ||
    previous.overallAlarm != current.overallAlarm
  ) {
    drawCompleteChannelCard(
      channelIndex,
      displaySlot
    );

    previous = current;
    return;
  }

  if (
    previous.readingValid != current.readingValid ||
    previous.voltageMilli != current.voltageMilli
  ) {
    drawChannelVoltage(
      channelIndex,
      displaySlot
    );
  }

  if (
    previous.readingValid != current.readingValid ||
    previous.currentMilli != current.currentMilli
  ) {
    drawChannelCurrent(
      channelIndex,
      displaySlot
    );
  }

  if (
    previous.voltageLowMilli != current.voltageLowMilli ||
    previous.voltageHighMilli != current.voltageHighMilli ||
    previous.currentLowMilli != current.currentLowMilli ||
    previous.currentHighMilli != current.currentHighMilli
  ) {
    drawChannelLimits(
      channelIndex,
      displaySlot
    );
  }

  if (
    previous.sensorOnline != current.sensorOnline ||
    previous.alarmMask != current.alarmMask
  ) {
    drawChannelHeaderState(
      channelIndex,
      displaySlot
    );
  }

  previous = current;
}

}

void initializeColors() {
  COLOR_BLACK      = display.color565(0, 0, 0);
```

```
  COLOR_WHITE      = display.color565(255, 255, 255);
  COLOR_BACKGROUND = display.color565(12, 18, 25);
  COLOR_HEADER     = display.color565(24, 35, 48);
  COLOR_CARD       = display.color565(31, 43, 56);
  COLOR_BORDER     = display.color565(77, 96, 115);
  COLOR_OK         = display.color565(34, 177, 76);
  COLOR_ALARM      = display.color565(185, 36, 45);
  COLOR_WARNING    = display.color565(235, 170, 35);
  COLOR_CYAN       = display.color565(64, 196, 255);
  COLOR_MUTED      = display.color565(175, 190, 205);
}

void redrawDisplay() {
  const HeaderDisplaySnapshot currentHeader =
    captureHeader();

  if (
    headerChanged(
      headerDisplayCache,
      currentHeader
    )
  ) {
    drawHeaderNetworkStatus();
    headerDisplayCache = currentHeader;
  }

  for (
    uint8_t displaySlot = 0;
    displaySlot < CHANNEL_COUNT;
    ++displaySlot
  ) {
    refreshChannelCard(
      DISPLAY_ORDER[displaySlot],
      displaySlot
    );
  }
}

void initializeDisplay() {
  display.begin();
  display.setRotation(1);
  display.setTextWrap(false);

  initializeColors();
  drawStaticLayout();

  redrawDisplay();
}
```

Listing C.3: Firmware: src/DisplayUI.cpp

```
/*Ethernet initialization for the multi-rail power supply monitor.

  This module:
    - configures the Ethernet Shield SPI chip-select pins
    - applies the current static IPv4 configuration
    - initializes the Arduino Ethernet interface
    - verifies that Ethernet hardware is present
    - starts the HTTP dashboard and PSC/EPICS servers

  Network settings are loaded into networkSettings before this function is
  called, either from persistent QSPI storage or from compiled defaults.*/
#include "EthernetServices.h"

#include <Arduino.h>
#include <Ethernet.h>
#include <SPI.h>
```

```
17
18 #include "AppConfig.h"
19 #include "AppState.h"
20 #include "HttpDashboard.h"
21 #include "PscProtocol.h"
22
23
24 //Sets the MAC address used by the Ethernet interface.
25 namespace {
26 byte macAddress[] = {0xDE, 0xAD, 0xBE, 0xEF, 0xDE, 0xAD};
27 }
28
29 /*Initialize the Ethernet hardware and network services.
30
31   Startup sequence:
32     1. Select the Ethernet controller SPI chip-select pin.
33     2. Disable the SD-card SPI device so it cannot interfere with Ethernet.
34     3. Convert the stored byte-array network configuration to IPAddress
35        objects required by the Arduino Ethernet library.
36     4. Start Ethernet using the configured static IP parameters.
37     5. Verify that the Ethernet controller is physically detected.
38     6. Start the HTTP dashboard and PSC server only when hardware is present.*/
39 void initializeEthernet() {
40   Ethernet.init(ETHERNET_CS_PIN);
41   pinMode(SD_CS_PIN, OUTPUT);
42   digitalWrite(SD_CS_PIN, HIGH);
43
44   const IPAddress localIp(
45     networkSettings.ip[0],
46     networkSettings.ip[1],
47     networkSettings.ip[2],
48     networkSettings.ip[3]
49   );
50   const IPAddress dnsServer(
51     networkSettings.dns[0],
52     networkSettings.dns[1],
53     networkSettings.dns[2],
54     networkSettings.dns[3]
55   );
56   const IPAddress gateway(
57     networkSettings.gateway[0],
58     networkSettings.gateway[1],
59     networkSettings.gateway[2],
60     networkSettings.gateway[3]
61   );
62   const IPAddress subnetMask(
63     networkSettings.subnet[0],
64     networkSettings.subnet[1],
65     networkSettings.subnet[2],
66     networkSettings.subnet[3]
67   );
68
69
70   // Configure the interface using the saved/static network configuration
71   Ethernet.begin(macAddress, localIp, dnsServer, gateway, subnetMask);
72   delay(250);
73
74   ethernetHardwarePresent =
75     Ethernet.hardwareStatus() != EthernetNoHardware;
76
77   if (ethernetHardwarePresent) {
78     initializeHttpServer();
79     initializePscServer();
80   }
81
82   Serial.print("Ethernet hardware: ");
83   Serial.println(ethernetHardwarePresent ? "present" : "not detected");
84
```

```
  //If hardware is absent, leave the services stopped
  if (ethernetHardwarePresent) {
    Serial.print("HTTP server: http://");
    Serial.println(Ethernet.localIP());
    Serial.print("PSC server: ");
    Serial.print(Ethernet.localIP());
    Serial.print(":");
    Serial.println(PSC_PORT);
  }
}
```

Listing C.4: Firmware: src/EthernetServices.cpp

```
/*Embedded HTTP configuration and monitoring interface.

  This module provides a webpage served directly by the
  Arduino GIGA over Ethernet.

  Supported functions:
    - display live voltage/current measurements for all channels
    - show alarm and sensor communication status
    - modify voltage/current alarm limits
    - modify static network configuration
    - persist accepted configuration changes to internal QSPI flash
    - synchronize changed alarm limits with the PSC/EPICS interface*/
#include "HttpDashboard.h"

#include <Arduino.h>
#include <Ethernet.h>

#include <math.h>
#include <stdio.h>
#include <stdlib.h>

#include "AppConfig.h"
#include "AppState.h"
#include "PersistentStorage.h"
#include "PscProtocol.h"

namespace {
EthernetServer httpServer(HTTP_PORT);
}

void initializeHttpServer() {
  httpServer.begin();
}

/*Read one line from the client.

  Reading stops when:
    - a newline is received,
    - the client disconnects, or
    - timeoutMs expires.

  Carriage returns are discarded and input is capped at maxLength to prevent
  an unexpectedly long request line or header from consuming excessive RAM.*/
String readHttpLine(EthernetClient &client, uint32_t timeoutMs, size_t maxLength) {
  String line;
  line.reserve(maxLength);

  const uint32_t start = millis();

  while (client.connected() && (millis() - start < timeoutMs)) {
    while (client.available()) {
      const char c = static_cast<char>(client.read());

      if (c == '\n') {
```

```
        return line;
      }

      if (c != '\r' && line.length() < maxLength) {
        line += c;
      }
    }
  }

  return line;
}

/*Ignore HTTP request headers until the required blank line is
  reached. The 30-line limit prevents a malformed client from keeping the
  embedded server occupied indefinitely.*/
void discardHttpHeaders(EthernetClient &client) {
  for (uint8_t i = 0; i < 30; ++i) {
    String line = readHttpLine(client, 300, 255);

    if (line.length() == 0) {
      break;
    }
  }
}


/*
  Minimal HTTP response helpers used by the dashboard endpoints.

  Responses disable caching so measurements/configuration shown by the
  browser reflect the current controller state.
*/

void sendHttpHeader(
  EthernetClient &client,
  const char *status,
  const char *contentType
) {
  client.print(F("HTTP/1.1 "));
  client.println(status);
  client.print(F("Content-Type: "));
  client.println(contentType);
  client.println(F("Cache-Control: no-store"));
  client.println(F("Access-Control-Allow-Origin: *"));
  client.println(F("Connection: close"));
  client.println();
}

void sendRedirectToDashboard(EthernetClient &client) {
  client.println(F("HTTP/1.1 303 See Other"));
  client.println(F("Location: /"));
  client.println(F("Cache-Control: no-store"));
  client.println(F("Connection: close"));
  client.println();
}

void sendBadRequest(EthernetClient &client, const char *message) {
  sendHttpHeader(client, "400 Bad Request", "text/plain; charset=utf-8");
  client.println(message);
}

void sendNotFound(EthernetClient &client) {
  sendHttpHeader(client, "404 Not Found", "text/plain; charset=utf-8");
  client.println(F("Not found"));
}

/*Locate a query-string parameter in the request target.
  Example:
```

```
    /api/limits?ch=1&vlow=4.5

    key "vlow" returns "4.5"*/
bool getQueryParameter(
  const String &target,
  const char *key,
  String &value
) {
  const int questionMark = target.indexOf('?');

  if (questionMark < 0) {
    return false;
  }

  int start = questionMark + 1;

  while (start < static_cast<int>(target.length())) {
    int end = target.indexOf('&', start);

    if (end < 0) {
      end = target.length();
    }

    const String pair = target.substring(start, end);
    const int equals = pair.indexOf('=');

    if (equals > 0 && pair.substring(0, equals) == key) {
      value = pair.substring(equals + 1);
      return true;
    }

    start = end + 1;
  }

  return false;
}

bool parseStrictFloat(const String &text, float &value) {
  if (text.length() == 0) {
    return false;
  }

  char *endPointer = nullptr;
  value = strtof(text.c_str(), &endPointer);

  return endPointer != text.c_str() &&
         *endPointer == '\0' &&
         isfinite(value);
}

bool parseStrictInt(const String &text, int &value) {
  if (text.length() == 0) {
    return false;
  }

  char *endPointer = nullptr;
  const long parsed = strtol(text.c_str(), &endPointer, 10);

  if (endPointer == text.c_str() || *endPointer != '\0') {
    return false;
  }

  value = static_cast<int>(parsed);
  return true;
}

bool parseIpv4Address(
  const String &text,
```

```
  uint8_t address[4]
) {
  int values[4];
  char trailingCharacter = '\0';

  const int fields = sscanf(
    text.c_str(),
    "%d.%d.%d.%d%c",
    &values[0],
    &values[1],
    &values[2],
    &values[3],
    &trailingCharacter
  );

  if (fields != 4) {
    return false;
  }

  for (uint8_t i = 0; i < 4; ++i) {
    if (values[i] < 0 || values[i] > 255) {
      return false;
    }

    address[i] = static_cast<uint8_t>(values[i]);
  }

  return true;
}

void formatIpv4Address(
  const uint8_t address[4],
  char *buffer,
  size_t bufferSize
) {
  snprintf(
    buffer,
    bufferSize,
    "%u.%u.%u.%u",
    address[0],
    address[1],
    address[2],
    address[3]
  );
}

/*Validate and store a network-configuration request from the web interface.

  Required query parameters:
    ip      - device IPv4 address
    dns     - DNS server; 0.0.0.0 is allowed
    gateway - default gateway; 0.0.0.0 is allowed
    subnet  - contiguous IPv4 subnet mask

  The requested values are parsed and validated before modifying the live
  configuration structure.

  The previous configuration is retained until the new values are
  successfully written to flash. If the flash write fails,
  the in-memory values are rolled back.

  A successful change sets networkRestartRequired because the Ethernet
  interface continues using its current address until reset/power-cycle.*/
bool updateNetworkFromRequest(
  const String &target,
  char *errorMessage,
  size_t errorMessageSize
) {
```

```
  String ipText;
  String dnsText;
  String gatewayText;
  String subnetText;

  if (!getQueryParameter(target, "ip", ipText) ||
      !getQueryParameter(target, "dns", dnsText) ||
      !getQueryParameter(target, "gateway", gatewayText) ||
      !getQueryParameter(target, "subnet", subnetText)) {
    snprintf(
      errorMessage,
      errorMessageSize,
      "Required parameters: ip, dns, gateway, subnet"
    );
    return false;
  }

  NetworkSettings requestedSettings;

  if (!parseIpv4Address(ipText, requestedSettings.ip) ||
      !parseIpv4Address(dnsText, requestedSettings.dns) ||
      !parseIpv4Address(gatewayText, requestedSettings.gateway) ||
      !parseIpv4Address(subnetText, requestedSettings.subnet)) {
    snprintf(
      errorMessage,
      errorMessageSize,
      "Enter each address in dotted-decimal form"
    );
    return false;
  }

  if (!hostAddressIsValid(requestedSettings.ip)) {
    snprintf(errorMessage, errorMessageSize, "Invalid device IP address");
    return false;
  }

  if (!optionalHostAddressIsValid(requestedSettings.dns)) {
    snprintf(errorMessage, errorMessageSize, "Invalid DNS address");
    return false;
  }

  if (!optionalHostAddressIsValid(requestedSettings.gateway)) {
    snprintf(errorMessage, errorMessageSize, "Invalid gateway address");
    return false;
  }

  if (!subnetMaskIsValid(requestedSettings.subnet)) {
    snprintf(errorMessage, errorMessageSize, "Invalid subnet mask");
    return false;
  }

  const NetworkSettings previousSettings = networkSettings;
  networkSettings = requestedSettings;

  if (!savePersistentSettings()) {
    networkSettings = previousSettings;

    snprintf(
      errorMessage,
      errorMessageSize,
      "Persistent storage write failed; network was not changed"
    );
    return false;
  }

  networkRestartRequired = true;
  return true;
}
```

```
327
328 /*Validate and apply alarm-limit update from the dashboard.
329
330   Expected parameters:
331     ch    - channel number, 1 through CHANNEL_COUNT
332     vlow  - lower voltage alarm threshold
333     vhigh - upper voltage alarm threshold
334     ilow  - lower current alarm threshold
335     ihigh - upper current alarm threshold
336
337   Lower limits must remain below their corresponding upper limits.
338
339   The previous limits are saved before modification. If flash write
340   fails, the old values and alarm state are restored.
341
342   After a successful update, the new limits and alarm state are immediately
343   published to the connected PSC/EPICS client so the browser, controller,
344   and IOC remain synchronized.*/
345 bool updateLimitsFromRequest(
346   const String &target,
347   char *errorMessage,
348   size_t errorMessageSize
349 ) {
350   String channelText;
351   String voltageLowText;
352   String voltageHighText;
353   String currentLowText;
354   String currentHighText;
355
356   if (!getQueryParameter(target, "ch", channelText) ||
357       !getQueryParameter(target, "vlow", voltageLowText) ||
358       !getQueryParameter(target, "vhigh", voltageHighText) ||
359       !getQueryParameter(target, "ilow", currentLowText) ||
360       !getQueryParameter(target, "ihigh", currentHighText)) {
361     snprintf(
362       errorMessage,
363       errorMessageSize,
364       "Required parameters: ch, vlow, vhigh, ilow, ihigh"
365     );
366     return false;
367   }
368
369   int channelNumber = 0;
370   float voltageLow = 0.0f;
371   float voltageHigh = 0.0f;
372   float currentLow = 0.0f;
373   float currentHigh = 0.0f;
374
375   if (!parseStrictInt(channelText, channelNumber) ||
376       !parseStrictFloat(voltageLowText, voltageLow) ||
377       !parseStrictFloat(voltageHighText, voltageHigh) ||
378       !parseStrictFloat(currentLowText, currentLow) ||
379       !parseStrictFloat(currentHighText, currentHigh)) {
380     snprintf(errorMessage, errorMessageSize, "One or more values are invalid");
381     return false;
382   }
383
384   if (channelNumber < 1 || channelNumber > CHANNEL_COUNT) {
385     snprintf(errorMessage, errorMessageSize, "Channel must be 1 through 6");
386     return false;
387   }
388
389   if (!(voltageLow < voltageHigh)) {
390     snprintf(errorMessage, errorMessageSize, "vlow must be less than vhigh");
391     return false;
392   }
393
394   if (!(currentLow < currentHigh)) {
```

```
    snprintf(errorMessage, errorMessageSize, "ilow must be less than ihigh");
    return false;
  }

  MonitorChannel &channel = channels[channelNumber - 1];
  const AlarmLimits previousLimits = channel.limits;

  channel.limits.voltageLow = voltageLow;
  channel.limits.voltageHigh = voltageHigh;
  channel.limits.currentLow = currentLow;
  channel.limits.currentHigh = currentHigh;

  updateAlarmState(channel);

  if (!savePersistentSettings()) {
    channel.limits = previousLimits;
    updateAlarmState(channel);

    snprintf(
      errorMessage,
      errorMessageSize,
      "Persistent storage write failed; limits were not changed"
    );
    return false;
  }

  sendPscLimits();
  sendPscAlarms();
  sendPscChannelSetpointEchoes(channelNumber - 1);
  return true;
}

void printHtmlFloatInput(
  EthernetClient &client,
  const char *formId,
  const char *name,
  float value
) {
  client.print(F("<input type='number' step='0.001' form='"));
  client.print(formId);
  client.print(F("' name='"));
  client.print(name);
  client.print(F("' value='"));
  client.print(value, 3);
  client.print(F("' required>"));
}

void printHtmlIpInput(
  EthernetClient &client,
  const char *name,
  const uint8_t address[4]
) {
  char addressText[20];
  formatIpv4Address(address, addressText, sizeof(addressText));

  client.print(F("<input class='ipinput' type='text' name='"));
  client.print(name);
  client.print(F("' value='"));
  client.print(addressText);
  client.print(
    F("' inputmode='decimal' pattern='[0-9]{1,3}(\\.[0-9]{1,3}){3}' required>")
  );
}

/*Display confirmation after network settings have been stored.

  The page explicitly tells the user to reset or power-cycle the controller before
  reconnecting at the new address.*/
```

```
463 void sendNetworkSavedPage(EthernetClient &client) {
464   char newIp[20];
465   formatIpv4Address(networkSettings.ip, newIp, sizeof(newIp));
466
467   sendHttpHeader(client, "200 OK", "text/html; charset=utf-8");
468
469   client.println(F(
470     "<!doctype html><html><head><meta charset='utf-8'>"
471     "<meta name='viewport' content='width=device-width,initial-scale=1'>"
472     "<title>Network settings saved</title>"
473     "<style>"
474     "body{font-family:Arial,sans-serif;background:#101820;color:#eef4f8;margin:32px}"
475     "a{color:#67c7ff}code{background:#243342;padding:4px 6px}"
476     "</style></head><body>"
477     "<h1>Network settings saved</h1>"
478     "<p>The GIGA is still using its old address for this session.</p>"
479     "<p>Reset or power-cycle the board, then open:</p><p><code>http://"
480   ));
481   client.print(newIp);
482   client.println(F(
483     "/</code></p>"
484     "<p>Use the reset button or cycle power after this page finishes loading.</p>"
485     "<p><a href='/'>Return to the current dashboard</a></p>"
486     "</body></html>"
487   ));
488 }
489
490 /*Generate the complete monitoring/configuration dashboard.
491
492   The page displays:
493     - live voltage and current for every rail
494     - OK/alarm/I2C-error state
495     - editable voltage and current alarm limits
496     - current PSC/EPICS connection state
497     - current static network configuration
498     - persistent-storage status
499     - reset-required warning after a network change
500
501   Each channel uses its own form so a single set of limits can be submitted
502   without modifying the other channels.
503
504   Invalid sensor readings are displayed as "---" rather than zero so a
505   communication failure cannot be mistaken for a real electrical value.*/
506 void sendDashboard(EthernetClient &client) {
507   sendHttpHeader(client, "200 OK", "text/html; charset=utf-8");
508
509   client.println(F(
510     "<!doctype html><html><head>"
511     "<meta charset='utf-8'>"
512     "<meta name='viewport' content='width=device-width,initial-scale=1'>"
513     "<title>GIGA Multi-Rail Monitor</title>"
514     "<style>"
515     "body{font-family:Arial,sans-serif;background:#101820;color:#eef4f8;margin:24px}"
516     "h1{margin-bottom:4px}"
517     "a{color:#67c7ff}"
518     "table{border-collapse:collapse;width:100%;max-width:1200px;background:#1e2b38}"
519     "th,td{border:1px solid #526579;padding:8px;text-align:center}"
520     "th{background:#26394b}"
521     "tr.alarm{background:#551f26}"
522     "tr.ok{background:#20372d}"
523     "input{width:80px;padding:5px;background:#f7fafc;border:1px solid #9aabba}""input.ipinput{width:132px}"".
        network{max-width:900px;background:#1e2b38;border:1px solid #526579;padding:16px;margin:18px 0}"".network
         label{display:inline-block;margin:6px 12px 6px 0}"
524     "button{padding:7px 12px;font-weight:bold}"
525     "code{background:#243342;padding:3px 5px}"
526     ".small{color:#b5c5d4;font-size:.9rem}"
527     "</style></head><body>"
528   ));
```

```
529
530   client.println(F("<h1>Arduino GIGA Multi-Rail Monitor</h1>"));
531   client.print(F("<p class='small'>Live LTC2945 measurements. Alarm limits are "));
532   client.print(
533     settingsStorageReady
534       ? F("saved in internal QSPI flash.")
535       : F("not persistent because storage is unavailable.")
536   );
537   client.println(F("</p>"));
538   client.print(F("<p class='small'>PSCDriver endpoint: <code>"));
539   client.print(Ethernet.localIP());
540   client.print(F(":"));
541   client.print(PSC_PORT);
542   client.print(F("</code> &mdash; "));
543   client.print(pscIsConnected() ? F("IOC connected") : F("waiting for IOC"));
544   client.println(F("</p>"));
545
546   for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
547     client.print(F("<form id='limitForm"));
548     client.print(i + 1);
549     client.println(F("' method='get' action='/api/limits'></form>"));
550   }
551
552   client.println(F(
553     "<table><thead><tr>"
554     "<th>Channel</th><th>Voltage</th><th>Current</th><th>State</th>"
555     "<th>V low</th><th>V high</th><th>I low</th><th>I high</th><th>Apply</th>"
556     "</tr></thead><tbody>"
557   ));
558
559   for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
560     const MonitorChannel &channel = channels[i];
561     char formId[20];
562     snprintf(formId, sizeof(formId), "limitForm%u", i + 1);
563
564     client.print(F("<tr class='"));
565     client.print(channelIsInAlarm(channel) ? F("alarm") : F("ok"));
566     client.print(F("'>"));
567
568     client.print(F("<td><strong>"));
569     client.print(channel.name);
570     client.print(F("</strong><input type='hidden' form='"));
571     client.print(formId);
572     client.print(F("' name='ch' value='"));
573     client.print(i + 1);
574     client.println(F("'></td>"));
575
576     client.print(F("<td>"));
577     if (channel.readingValid) {
578       client.print(channel.voltage, 3);
579       client.print(F(" V"));
580     } else {
581       client.print(F("---"));
582     }
583     client.println(F("</td>"));
584
585     client.print(F("<td>"));
586     if (channel.readingValid) {
587       client.print(channel.current, 3);
588       client.print(F(" A"));
589     } else {
590       client.print(F("---"));
591     }
592     client.println(F("</td>"));
593
594     client.print(F("<td>"));
595     if (!channel.sensorOnline) {
596       client.print(F("I2C ERROR 0x"));
```

```
      client.print(channel.sensorError, HEX);
    } else {
      client.print(channelIsInAlarm(channel) ? F("ALARM") : F("OK"));
    }
    client.println(F("</td>"));

    client.print(F("<td>"));
    printHtmlFloatInput(client, formId, "vlow", channel.limits.voltageLow);
    client.println(F("</td>"));

    client.print(F("<td>"));
    printHtmlFloatInput(client, formId, "vhigh", channel.limits.voltageHigh);
    client.println(F("</td>"));

    client.print(F("<td>"));
    printHtmlFloatInput(client, formId, "ilow", channel.limits.currentLow);
    client.println(F("</td>"));

    client.print(F("<td>"));
    printHtmlFloatInput(client, formId, "ihigh", channel.limits.currentHigh);
    client.println(F("</td>"));

    client.print(F("<td><button type='submit' form='"));
    client.print(formId);
    client.println(F("'>Set</button></td>"));
    client.println(F("</tr>"));
  }

  client.println(F("</tbody></table>"));

  client.println(F(
    "<div class='network'><h2>Network settings</h2>"
    "<p class='small'>Changes are stored immediately and applied after reset.</p>"
    "<form method='get' action='/api/network'>"
  ));

  client.print(F("<label>IP address<br>"));
  printHtmlIpInput(client, "ip", networkSettings.ip);
  client.println(F("</label>"));

  client.print(F("<label>Gateway<br>"));
  printHtmlIpInput(client, "gateway", networkSettings.gateway);
  client.println(F("</label>"));

  client.print(F("<label>Subnet mask<br>"));
  printHtmlIpInput(client, "subnet", networkSettings.subnet);
  client.println(F("</label>"));

  client.print(F("<label>DNS server<br>"));
  printHtmlIpInput(client, "dns", networkSettings.dns);
  client.println(F("</label>"));

  client.println(F(
    "<br><button type='submit'>Save network settings</button>"
    "</form>"
  ));

  if (networkRestartRequired) {
    client.println(F(
      "<p><strong>Reset required:</strong> saved network settings differ "
      "from the address currently in use.</p>"
    ));
  }

  client.println(F(
    "<p class='small'>A wrong but valid address can make this page unreachable. "
    "Recovery: set <code>FORCE_COMPILED_NETWORK_ON_BOOT</code> to "
    "<code>true</code> and upload over USB.</p></div>"
```

```
  ));

  client.println(F("</body></html>"));
}

/*Parse and dispatch one HTTP request.

  Supported routes:
    GET /                  - monitoring/configuration dashboard
    GET /api/network?...   - validate and store network settings
    GET /api/limits?...    - validate and store one channel's alarm limits

  Any non-GET method returns HTTP 405.
  Unknown paths return HTTP 404.
  Invalid submitted values return HTTP 400 with a diagnostic message.*/
void handleHttpRequest(EthernetClient &client, const String &requestLine) {
  const int firstSpace = requestLine.indexOf(' ');
  const int secondSpace = requestLine.indexOf(' ', firstSpace + 1);

  if (firstSpace < 0 || secondSpace < 0) {
    sendBadRequest(client, "Malformed HTTP request line");
    return;
  }

  const String method = requestLine.substring(0, firstSpace);
  const String target = requestLine.substring(firstSpace + 1, secondSpace);

  if (method != "GET") {
    sendHttpHeader(client, "405 Method Not Allowed", "text/plain; charset=utf-8");
    client.println(F("Only GET is supported"));
    return;
  }

  if (target == "/" || target.startsWith("/?")) {
    sendDashboard(client);
    return;
  }

  if (target.startsWith("/api/network?")) {
    char errorMessage[128];

    if (!updateNetworkFromRequest(
          target,
          errorMessage,
          sizeof(errorMessage)
        )) {
      sendBadRequest(client, errorMessage);
      return;
    }

    sendNetworkSavedPage(client);
    return;
  }

  if (target.startsWith("/api/limits?")) {
    char errorMessage[96];

    if (!updateLimitsFromRequest(target, errorMessage, sizeof(errorMessage))) {
      sendBadRequest(client, errorMessage);
      return;
    }

    sendRedirectToDashboard(client);
    return;
  }

  sendNotFound(client);
}
```

```
733
734 /*Service one pending HTTP connection.
735
736   This routine is called repeatedly from the main application loop. It
737   accepts a client, reads the request line and headers, dispatches the
738   request, then closes the connection.
739
740   The implementation intentionally uses short-lived HTTP/1.1 connections
741   rather than keeping browser connections open, which keeps the embedded
742   server simple and prevents it from monopolizing the main control loop.*/
743 void serviceHttp() {
744   if (!ethernetHardwarePresent) {
745     return;
746   }
747
748   EthernetClient client = httpServer.available();
749
750   if (!client) {
751     return;
752   }
753
754   const String requestLine = readHttpLine(client, 500, 300);
755   discardHttpHeaders(client);
756
757   if (requestLine.length() > 0) {
758     Serial.println(requestLine);
759     handleHttpRequest(client, requestLine);
760   }
761
762   delay(1);
763   client.stop();
764 }
```

Listing C.5: Firmware: src/HttpDashboard.cpp

```
1  /*
2    LTC2945 voltage/current monitor driver.
3
4    Each driver instance represents one LTC2945 connected to an isolated
5    measurement channel.
6
7    The driver handles:
8
9      - I2C device validation and register access
10     - explicit CONTROL-register configuration
11     - automatic recovery after LTC2945 power cycling
12     - selectable VIN source (SENSE+ or VDD)
13     - CONTROL-register readback verification
14     - 12-bit ADC register decoding
15     - conversion of raw values to engineering units
16     - channel-specific voltage/current polarity
17     - current calculation from configured shunt resistance
18     - I2C and data-validity error reporting
19
20   Positive rails use SENSE+ as the VIN measurement source.
21
22   Negative rails using low-side current sensing use VDD as the VIN
23   measurement source. The LTC2945 sees this as a positive local voltage
24   magnitude. voltageSign_ converts that magnitude to the negative
25   system-level rail voltage.
26
27   The CONTROL register is checked before each measurement. This is
28   important because an LTC2945 that loses power while the Arduino remains
29   running will return to its power-up CONTROL value. The driver detects
30   that condition and restores the channel configuration automatically.
31 */
32
```

```cpp
#include "LTC2945.h"

#include <math.h>


/*
  Construct one LTC2945 channel.

  Params:
    wire          - I2C bus used by this device
    address       - LTC2945 7-bit I2C address
    shuntOhms     - installed current-sense resistance in ohms
    voltageSign   - +1 or -1 multiplier applied to reported VIN
    currentSign   - +1 or -1 multiplier applied to reported current
    voltageSource - VIN ADC source
*/
LTC2945::LTC2945(
  TwoWire &wire,
  uint8_t address,
  float shuntOhms,
  float voltageSign,
  float currentSign,
  LTC2945::VoltageSource voltageSource
)
  : wire_(wire),
    address_(address),
    shuntOhms_(shuntOhms),
    voltageSign_(voltageSign),
    currentSign_(currentSign),
    voltageSource_(voltageSource) {}


/*
  Return the CONTROL-register value required for this channel.
*/
uint8_t LTC2945::expectedControlValue() const {
  if (voltageSource_ == VoltageSource::Vdd) {
    return CONTROL_CONTINUOUS_VDD;
  }

  return CONTROL_CONTINUOUS_SENSE_PLUS;
}


/*
  Configure the LTC2945 CONTROL register and verify that the value
  was accepted.

  Software error:
    0xF7 - CONTROL register readback did not match expected value
*/
bool LTC2945::configure() {
  const uint8_t expected = expectedControlValue();

  if (!writeRegister8(REG_CONTROL, expected)) {
    return false;
  }

  uint8_t readback = 0;

  if (!readRegister8(REG_CONTROL, readback)) {
    return false;
  }

  if (readback != expected) {
    lastError_ = 0xF7;
    return false;
  }
```

```
101
102   lastError_ = 0;
103   return true;
104 }
105
106
107 /*
108   Confirm that the device still has the correct CONTROL configuration.
109
110   This provides automatic recovery when an individual monitored rail is
111   power-cycled while the Arduino remains powered.
112
113   Example for -5V / -15V:
114
115     normal configured state:
116       CONTROL = 0x01
117
118     rail power cycled:
119       LTC2945 resets
120       CONTROL returns to 0x05
121
122     next measurement:
123       driver detects 0x05
124       driver rewrites 0x01
125       measurement proceeds normally
126 */
127 bool LTC2945::ensureConfigured() {
128   const uint8_t expected = expectedControlValue();
129
130   uint8_t current = 0;
131
132   if (!readRegister8(REG_CONTROL, current)) {
133     return false;
134   }
135
136   if (current == expected) {
137     lastError_ = 0;
138     return true;
139   }
140
141   return configure();
142 }
143
144
145 /*
146   Validate the configured device parameters, confirm that the LTC2945
147   acknowledges, configure its CONTROL register, and verify the
148   configuration.
149
150   Software errors:
151     0xF1 - configured I2C address outside LTC2945 range
152     0xF2 - invalid shunt resistance
153     0xF7 - CONTROL register verification failed
154
155   Wire/I2C errors are stored directly in lastError_.
156 */
157 bool LTC2945::begin() {
158   if (address_ < 0x67 || address_ > 0x6F) {
159     lastError_ = 0xF1;
160     return false;
161   }
162
163   if (!isfinite(shuntOhms_) || shuntOhms_ <= 0.0f) {
164     lastError_ = 0xF2;
165     return false;
166   }
167
168   if (!ping()) {
```

```
    return false;
  }

  return configure();
}


/*
  Probe the configured I2C address without transferring register data.
*/
bool LTC2945::ping() {
  wire_.beginTransmission(address_);

  lastError_ = wire_.endTransmission();

  return lastError_ == 0;
}


/*
  Write one 8-bit LTC2945 register.
*/
bool LTC2945::writeRegister8(
  uint8_t reg,
  uint8_t value
) {
  wire_.beginTransmission(address_);

  wire_.write(reg);
  wire_.write(value);

  lastError_ = wire_.endTransmission();

  return lastError_ == 0;
}


/*
  Read one 8-bit LTC2945 register.
*/
bool LTC2945::readRegister8(
  uint8_t reg,
  uint8_t &value
) {
  return readRegisters(
    reg,
    &value,
    1
  );
}


/*
  Read a contiguous block of LTC2945 registers.

  The register pointer is written first, followed by a repeated START
  so the bus remains under control while switching from write to read.

  The Wire receive buffer is limited, so reads larger than 32 bytes
  are rejected.

  Software errors:
    0xF3 - invalid destination buffer or requested length
    0xF4 - fewer bytes returned than requested
    0xF5 - receive buffer became empty before all bytes were copied
*/
bool LTC2945::readRegisters(
  uint8_t startReg,
```

```
  uint8_t *buffer,
  size_t length
) {
  if (
    buffer == nullptr ||
    length == 0 ||
    length > 32
  ) {
    lastError_ = 0xF3;
    return false;
  }

  wire_.beginTransmission(address_);
  wire_.write(startReg);

  /*
    Repeated START:
    keep control of the bus while switching from write to read.
  */
  lastError_ = wire_.endTransmission(false);

  if (lastError_ != 0) {
    return false;
  }

  const size_t received = wire_.requestFrom(
    static_cast<int>(address_),
    static_cast<int>(length)
  );

  if (received != length) {
    lastError_ = 0xF4;

    while (wire_.available()) {
      wire_.read();
    }

    return false;
  }

  for (size_t i = 0; i < length; ++i) {
    if (!wire_.available()) {
      lastError_ = 0xF5;
      return false;
    }

    buffer[i] =
      static_cast<uint8_t>(wire_.read());
  }

  lastError_ = 0;
  return true;
}


/*
  Read one 12-bit LTC2945 ADC result from its two-register
  representation.
*/
bool LTC2945::read12Bit(
  uint8_t msbRegister,
  uint16_t &code
) {
  uint8_t bytes[2];

  if (!readRegisters(
        msbRegister,
        bytes,
```

```
        sizeof(bytes)
      )) {
    return false;
  }

  /*
    ADC results are left-justified:

      first byte:
        bits 11..4

      second byte:
        bits 3..0 in the upper nibble
  */
  const uint16_t packed =
    (static_cast<uint16_t>(bytes[0]) << 8) |
    static_cast<uint16_t>(bytes[1]);

  code = packed >> 4;

  return true;
}


/*
  Read voltage and current measurements.

  Before accepting a measurement, the CONTROL register is checked.

  This means an individual LTC2945 can lose power and restart while
  the Arduino remains powered. When the device returns, its CONTROL
  register is automatically restored before measurement data is used.
*/
bool LTC2945::read(
  Reading &reading
) {
  reading = Reading{};

  /*
    Verify the channel configuration before reading ADC results.

    This is especially important for the negative rails. Their
    LTC2945s must use:

      CONTROL = 0x01
      VIN source = VDD

    rather than the power-up default:

      CONTROL = 0x05
      VIN source = SENSE+
  */
  if (!ensureConfigured()) {
    reading.i2cError = lastError_;
    return false;
  }

  if (!read12Bit(
        REG_VIN_MSB,
        reading.voltageCode
      )) {
    reading.i2cError = lastError_;
    return false;
  }

  if (!read12Bit(
        REG_DELTA_SENSE_MSB,
        reading.currentCode
```

```
    )) {
    reading.i2cError = lastError_;
    return false;
  }

  /*
    VIN ADC:
      25 mV / LSB

    Positive rails:
      source = SENSE+

    Negative rails:
      source = VDD

    voltageSign_ converts the positive local VDD magnitude of a
    negative rail to its negative system-level value.
  */
  reading.voltage =
    voltageSign_ *
    static_cast<float>(
      reading.voltageCode
    ) *
    VIN_LSB_VOLTS;

  /*
    Delta-sense ADC:
      25 uV / LSB

    Current:
      I = Vsense / Rshunt
  */
  const float senseVoltage =
    static_cast<float>(
      reading.currentCode
    ) *
    DELTA_SENSE_LSB_VOLTS;

  reading.current =
    currentSign_ *
    senseVoltage /
    shuntOhms_;

  /*
    Calculate signed electrical power from the engineering-unit
    voltage and current values.
  */
  reading.power =
    reading.voltage *
    reading.current;

  reading.valid =
    isfinite(reading.voltage) &&
    isfinite(reading.current) &&
    isfinite(reading.power);

  if (!reading.valid) {
    reading.i2cError = 0xF6;
    lastError_ = 0xF6;

    return false;
  }

  reading.i2cError = 0;
  lastError_ = 0;

  return true;
}
```

```

uint8_t LTC2945::address() const {
  return address_;
}


uint8_t LTC2945::lastError() const {
  return lastError_;
}
```

Listing C.6: Firmware: src/LTC2945.cpp

```
/*
  Persistent configuration storage for the PSU monitor.

  Alarm limits and network settings are stored in the Arduino GIGA's internal
  QSPI

  Each stored settings image contains:
    - a magic value identifying the file format
    - a format/version number
    - the expected channel count
    - alarm limits for every monitored channel
    - Ethernet/network configuration
    - an FNV-1a checksum

  Saved data is validated before it is applied. Invalid, incompatible, or
  corrupted settings are rejected and compiled defaults are used instead.

  Settings are first written to a temporary file and then renamed into place
  to reduce the chance of losing configuration if power is removed during a
  write operation.
*/

#include "PersistentStorage.h"

#include <Arduino.h>

#include "BlockDevice.h"
#include "MBRBlockDevice.h"
#include "LittleFileSystem.h"

#include <math.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>

#include "AppConfig.h"
#include "AppState.h"

/*The GIGA's default QSPI block device contains a few MBR partitions.
  Partition 4 is reserved for application/user storage.*/
using namespace mbed;

namespace {
BlockDevice *qspiRoot = BlockDevice::get_default_instance();
MBRBlockDevice qspiUserPartition(qspiRoot, 4);
LittleFileSystem qspiUserFileSystem("user");
}

/*Calculate a 32-bit checksum.

  This is used to detect accidental corruption of the settings structure in
  flash.*/
uint32_t calculateChecksum(
  const uint8_t *bytes,
```

```
  size_t byteCount
) {
  // FNV-1a checksum.
  uint32_t hash = 2166136261UL;

  for (size_t i = 0; i < byteCount; ++i) {
    hash ^= bytes[i];
    hash *= 16777619UL;
  }

  return hash;
}

/*Calculate the checksum over every byte preceding the checksum field.

  PersistentSettings is stored directly so changes to its layout must be
  accompanied by a SETTINGS_VERSION change to prevent an older stored image
  from being interpreted using a newer structure layout.*/
uint32_t calculateSettingsChecksum(
  const PersistentSettings &settings
) {
  return calculateChecksum(
    reinterpret_cast<const uint8_t *>(&settings),
    offsetof(PersistentSettings, checksum)
  );
}

bool alarmLimitsAreValid(const AlarmLimits &limits) {
  constexpr float ABSOLUTE_LIMIT = 10000.0f;

  return isfinite(limits.voltageLow) &&
         isfinite(limits.voltageHigh) &&
         isfinite(limits.currentLow) &&
         isfinite(limits.currentHigh) &&
         limits.voltageLow < limits.voltageHigh &&
         limits.currentLow < limits.currentHigh &&
         fabsf(limits.voltageLow) <= ABSOLUTE_LIMIT &&
         fabsf(limits.voltageHigh) <= ABSOLUTE_LIMIT &&
         fabsf(limits.currentLow) <= ABSOLUTE_LIMIT &&
         fabsf(limits.currentHigh) <= ABSOLUTE_LIMIT;
}

bool addressIsAll(
  const uint8_t address[4],
  uint8_t value
) {
  return address[0] == value &&
         address[1] == value &&
         address[2] == value &&
         address[3] == value;
}

/*Validate an IPv4 host address.
  Reject:
    - 0.0.0.0
    - 255.255.255.255
    - loopback addresses (127.x.x.x)
    - multicast/experimental address space above 223.x.x.x*/
bool hostAddressIsValid(const uint8_t address[4]) {
  if (addressIsAll(address, 0) ||
      addressIsAll(address, 255)) {
    return false;
  }

  // Reject multicast, experimental, and loopback ranges.
  return address[0] >= 1 &&
         address[0] <= 223 &&
         address[0] != 127;
```

```
}

bool optionalHostAddressIsValid(const uint8_t address[4]) {
  // A zero gateway or DNS address is allowed.
  return addressIsAll(address, 0) ||
         hostAddressIsValid(address);
}

/*Test no 1 bit follows a 0 bit in the netmask*/
bool subnetMaskIsValid(const uint8_t subnet[4]) {
  const uint32_t mask =
    (static_cast<uint32_t>(subnet[0]) << 24) |
    (static_cast<uint32_t>(subnet[1]) << 16) |
    (static_cast<uint32_t>(subnet[2]) << 8) |
    static_cast<uint32_t>(subnet[3]);

  if (mask == 0) {
    return false;
  }

  bool zeroBitSeen = false;

  for (int bit = 31; bit >= 0; --bit) {
    const bool one = (mask & (1UL << bit)) != 0;

    if (!one) {
      zeroBitSeen = true;
    } else if (zeroBitSeen) {
      // A valid mask cannot contain a 1 after its first 0.
      return false;
    }
  }

  return true;
}

bool networkSettingsAreValid(
  const NetworkSettings &settings
) {
  return hostAddressIsValid(settings.ip) &&
         optionalHostAddressIsValid(settings.dns) &&
         optionalHostAddressIsValid(settings.gateway) &&
         subnetMaskIsValid(settings.subnet);
}

/*Build a complete persistent-settings image*/
void buildPersistentSettings(PersistentSettings &settings) {
  memset(&settings, 0, sizeof(settings));

  settings.magic = SETTINGS_MAGIC;
  settings.version = SETTINGS_VERSION;
  settings.channelCount = CHANNEL_COUNT;

  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    settings.limits[i] = channels[i].limits;
  }

  settings.network = networkSettings;
  settings.checksum = calculateSettingsChecksum(settings);
}

/*Validate a settings image before allowing it to affect the running system.
    - expected magic number
    - compatible structure version
    - expected number of channels
    - checksum integrity
    - valid network configuration
    - valid alarm limits for every channel
```

```
*/
bool persistentSettingsAreValid(
  const PersistentSettings &settings
) {
  if (settings.magic != SETTINGS_MAGIC ||
      settings.version != SETTINGS_VERSION ||
      settings.channelCount != CHANNEL_COUNT ||
      settings.checksum != calculateSettingsChecksum(settings) ||
      !networkSettingsAreValid(settings.network)) {
    return false;
  }

  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    if (!alarmLimitsAreValid(settings.limits[i])) {
      return false;
    }
  }

  return true;
}

/*Read PersistentSettings structure from QSPI and validate it.*/
bool readPersistentSettingsFile(
  const char* path,
  PersistentSettings& settings
) {
  FILE* file = fopen(path, "rb");

  if (file == nullptr) {
    return false;
  }

  const size_t bytesRead =
    fread(&settings, 1, sizeof(settings), file);

  fclose(file);

  return bytesRead == sizeof(settings) &&
         persistentSettingsAreValid(settings);
}

/*
  Apply persistent settings to the live application state.

  Alarm states are recalculated immediately after restoring each channel's
  limits so the displayed/network alarm state remains consistent.

  When FORCE_COMPILED_NETWORK_ON_BOOT is enabled, saved network parameters
  are deliberately ignored while saved alarm limits are still restored.
*/
void applyPersistentSettings(
  const PersistentSettings &settings
) {
  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    channels[i].limits = settings.limits[i];
    updateAlarmState(channels[i]);
  }

  if (!FORCE_COMPILED_NETWORK_ON_BOOT) {
    networkSettings = settings.network;
  } else {
    networkSettings = COMPILED_NETWORK_SETTINGS;
  }
}

/*Save the current configuration to internal QSPI flash.

  A temporary-file replacement scheme is used:
```

```
259
260     1. Build and checksum the new settings image.
261     2. Write it completely to SETTINGS_TEMP_FILE.
262     3. Flush and close the temporary file.
263     4. Remove the previous SETTINGS_FILE.
264     5. Rename the verified temporary file to SETTINGS_FILE.
265   Returns false if storage is unavailable or any file operation fails.*/
266 bool savePersistentSettings() {
267   if (!settingsStorageReady) {
268     return false;
269   }
270
271   PersistentSettings settings;
272   buildPersistentSettings(settings);
273
274   FILE *file = fopen(SETTINGS_TEMP_FILE, "wb");
275
276   if (file == nullptr) {
277     Serial.println("Could not open temporary settings file.");
278     return false;
279   }
280
281   const size_t bytesWritten =
282     fwrite(&settings, 1, sizeof(settings), file);
283
284   const int flushResult = fflush(file);
285   const int closeResult = fclose(file);
286
287   if (bytesWritten != sizeof(settings) ||
288       flushResult != 0 ||
289       closeResult != 0) {
290     remove(SETTINGS_TEMP_FILE);
291     Serial.println("Persistent-settings write failed.");
292     return false;
293   }
294
295   /*
296     Keep the temporary file valid until the final rename. On startup, the
297     loader checks both files, so a reset during this short replacement window
298     can still recover the new settings.
299   */
300   remove(SETTINGS_FILE);
301
302   if (rename(SETTINGS_TEMP_FILE, SETTINGS_FILE) != 0) {
303     Serial.println("Could not promote temporary settings file.");
304     return false;
305   }
306
307   Serial.println("Alarm and network settings saved to internal QSPI flash.");
308   return true;
309 }
310
311 /*Restore persistent configuration from QSPI flash.
312
313   The normal settings file is tried first. If it is missing or invalid, the
314   temporary file is checked as a recovery path for an interrupted save.
315
316   No settings are applied unless the complete stored image passes all
317   integrity and range checks*/
318 bool loadPersistentSettings() {
319   if (!settingsStorageReady) {
320     return false;
321   }
322
323   PersistentSettings settings;
324
325   if (readPersistentSettingsFile(
326         SETTINGS_FILE,
```

```
      settings
    )) {
  applyPersistentSettings(settings);

  Serial.println(
    "Alarm and network settings loaded from internal QSPI flash."
  );

  return true;
 }

 /*
  Recover a valid temporary file if power was removed between replacing
  the old settings file and renaming the new one.
 */
 if (readPersistentSettingsFile(
      SETTINGS_TEMP_FILE,
      settings
    )) {
  applyPersistentSettings(settings);

  remove(SETTINGS_FILE);
  rename(SETTINGS_TEMP_FILE, SETTINGS_FILE);

  Serial.println(
    "Recovered settings from temporary settings file."
  );

  return true;
 }

 return false;
}

/*Initialize the user-settings filesystem in internal QSPI flash.
   1. Mount LittleFS on MBR part. 4.
   2. If mounting fails and formatting is permitted, reformat only the
      application/user partition.
   3. Attempt to load previously saved settings.
   4. If no valid settings exist, restore compiled defaults and save them
      as the initial persistent configuration.*/
bool initializeSettingsStorage() {
 int result = qspiUserFileSystem.mount(&qspiUserPartition);

 if (result != 0 && FORMAT_USER_PARTITION_IF_NEEDED) {
  Serial.println(
    "QSPI user partition is not mountable; formatting partition 4."
  );

  /*
    reformat() formats and mounts only partition 4. It does not format the
    Wi-Fi firmware, OTA, or provisioning partitions.
  */
  result = qspiUserFileSystem.reformat(&qspiUserPartition);
 }

 if (result != 0) {
  Serial.print("QSPI user storage unavailable; error ");
  Serial.println(result);
  settingsStorageReady = false;
  return false;
 }

 settingsStorageReady = true;

 if (!loadPersistentSettings()) {
  Serial.println(
    "No valid saved settings found; storing compiled defaults."
```

```
    );

    networkSettings = COMPILED_NETWORK_SETTINGS;

    if (!savePersistentSettings()) {
      Serial.println("Could not store compiled defaults.");
      return false;
    }
  }

  return true;
}
```

Listing C.7: Firmware: src/PersistentStorage.cpp

```
/*
  PSC protocol server for communication between the Arduino GIGA and
  the EPICS PSCDriver IOC.

  Packet format:
    Bytes 0-1 : ASCII magic "PS"
    Bytes 2-3 : 16-bit message ID, big-endian
    Bytes 4-7 : 32-bit body length, big-endian
    Bytes 8+  : message body

  The server:
    - accepts one PSC client connection at a time
    - periodically publishes measurements and alarm states
    - publishes a complete state snapshot when a client connects
    - accepts alarm-limit setpoints from the IOC
    - validates and stores accepted limits in persistent storage
    - echoes the actual stored value so EPICS AO records can resynchronize

  All multibyte numeric values are serialized in big-endian order.
*/

#include "PscProtocol.h"

#include <Arduino.h>
#include <Ethernet.h>

#include <math.h>
#include <string.h>

#include "AppConfig.h"
#include "AppState.h"
#include "PersistentStorage.h"

namespace {
EthernetServer pscServer(PSC_PORT);
EthernetClient pscClient;

/* Incremental receive-parser state. Ethernet data may arrive in partial
packets, so the header and body are assembled across servicePsc() calls.*/
uint32_t lastPscPublishMs = 0;
uint8_t pscHeaderBuffer[8];
uint8_t pscBodyBuffer[PSC_MAX_BODY_LENGTH];
uint8_t pscHeaderIndex = 0;
uint32_t pscBodyIndex = 0;
uint16_t pscIncomingMessageId = 0;
uint32_t pscIncomingBodyLength = 0;
bool pscReadingBody = false;
}

void initializePscServer() {
  pscServer.begin();
}
```

```
53
54  bool pscIsConnected() {
55    return static_cast<bool>(pscClient) && pscClient.connected();
56  }
57
58  void resetPscReceiveState() {
59    pscHeaderIndex = 0;
60    pscBodyIndex = 0;
61    pscIncomingMessageId = 0;
62    pscIncomingBodyLength = 0;
63    pscReadingBody = false;
64  }
65
66  /*PSC messages use big-endian. Helpers convert ints and floats between
67  wire represnetation and Giga native.*/
68  uint16_t readU16BigEndian(const uint8_t *source) {
69    return
70      (static_cast<uint16_t>(source[0]) << 8) |
71      static_cast<uint16_t>(source[1]);
72  }
73
74  uint32_t readU32BigEndian(const uint8_t *source) {
75    return
76      (static_cast<uint32_t>(source[0]) << 24) |
77      (static_cast<uint32_t>(source[1]) << 16) |
78      (static_cast<uint32_t>(source[2]) << 8) |
79      static_cast<uint32_t>(source[3]);
80  }
81
82  void writeU16BigEndian(uint8_t *destination, uint16_t value) {
83    destination[0] = static_cast<uint8_t>((value >> 8) & 0xFF);
84    destination[1] = static_cast<uint8_t>(value & 0xFF);
85  }
86
87  void writeU32BigEndian(uint8_t *destination, uint32_t value) {
88    destination[0] = static_cast<uint8_t>((value >> 24) & 0xFF);
89    destination[1] = static_cast<uint8_t>((value >> 16) & 0xFF);
90    destination[2] = static_cast<uint8_t>((value >> 8) & 0xFF);
91    destination[3] = static_cast<uint8_t>(value & 0xFF);
92  }
93
94  float readFloatBigEndian(const uint8_t *source) {
95    const uint32_t bits = readU32BigEndian(source);
96    float value = 0.0f;
97    memcpy(&value, &bits, sizeof(value));
98    return value;
99  }
100
101 void writeFloatBigEndian(uint8_t *destination, float value) {
102   uint32_t bits = 0;
103   memcpy(&bits, &value, sizeof(bits));
104   writeU32BigEndian(destination, bits);
105 }
106
107 /*Writes a PSC packet segment to the connected TCP client.
108
109   EthernetClient::write() is not assumed to transmit the complete buffer
110   in one call. Continue writing until all bytes are sent, the connection
111   closes, or PSC_WRITE_TIMEOUT_MS expires.*/
112
113 bool writeAllPscBytes(const uint8_t *data, size_t length) {
114   if (!pscIsConnected()) {
115     return false;
116   }
117
118   size_t offset = 0;
119   const uint32_t start = millis();
120
```

```
  while (offset < length && pscClient.connected()) {
    const size_t written = pscClient.write(data + offset, length - offset);

    if (written > 0) {
      offset += written;
      continue;
    }

    if (millis() - start >= PSC_WRITE_TIMEOUT_MS) {
      break;
    }

    delay(1);
  }

  return offset == length;
}

/*Builds and transmits one PSC message.
  Header:
    "PS" + uint16 message ID + uint32 payload length
*/
bool sendPscMessage(
  uint16_t messageId,
  const uint8_t *body,
  uint32_t bodyLength
) {
  if (!pscIsConnected()) {
    return false;
  }

  uint8_t header[8];
  header[0] = 'P';
  header[1] = 'S';
  writeU16BigEndian(&header[2], messageId);
  writeU32BigEndian(&header[4], bodyLength);

  if (!writeAllPscBytes(header, sizeof(header))) {
    return false;
  }

  return bodyLength == 0 || writeAllPscBytes(body, bodyLength);
}

/*
  Alarm/status bit definitions sent in PSC_MSG_ALARMS:

    bit 0 - voltage low alarm
    bit 1 - voltage high alarm
    bit 2 - current low alarm
    bit 3 - current high alarm
    bit 4 - sensor offline / communication fault

  Remaining bits are reserved.
*/
uint32_t alarmMaskForChannel(const MonitorChannel &channel) {
  uint32_t mask = 0;

  if (channel.voltageLowAlarm) {
    mask |= 1UL << 0;
  }
  if (channel.voltageHighAlarm) {
    mask |= 1UL << 1;
  }
  if (channel.currentLowAlarm) {
    mask |= 1UL << 2;
  }
  if (channel.currentHighAlarm) {
```

```
    mask |= 1UL << 3;
  }
  if (!channel.sensorOnline) {
    mask |= 1UL << 4;
  }

  return mask;
}

/*Write out voltage/current pairs for all channels.

  Invalid channels are written as NaN rather than 0.0 so an
  I2C or sensor failure cannot be mistaken for real readouts.*/
bool sendPscMeasurements() {
  uint8_t body[CHANNEL_COUNT * 2 * sizeof(float)];
  size_t offset = 0;

  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    const float voltage =
      channels[i].readingValid ? channels[i].voltage : NAN;

    const float current =
      channels[i].readingValid ? channels[i].current : NAN;

    writeFloatBigEndian(&body[offset], voltage);
    offset += sizeof(float);
    writeFloatBigEndian(&body[offset], current);
    offset += sizeof(float);
  }

  return sendPscMessage(
    PSC_MSG_MEASUREMENTS,
    body,
    sizeof(body)
  );
}

bool sendPscLimits() {
  uint8_t body[CHANNEL_COUNT * 4 * sizeof(float)];
  size_t offset = 0;

  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    writeFloatBigEndian(&body[offset], channels[i].limits.voltageLow);
    offset += sizeof(float);
    writeFloatBigEndian(&body[offset], channels[i].limits.voltageHigh);
    offset += sizeof(float);
    writeFloatBigEndian(&body[offset], channels[i].limits.currentLow);
    offset += sizeof(float);
    writeFloatBigEndian(&body[offset], channels[i].limits.currentHigh);
    offset += sizeof(float);
  }

  return sendPscMessage(PSC_MSG_LIMITS, body, sizeof(body));
}

bool sendPscAlarms() {
  uint8_t body[CHANNEL_COUNT * sizeof(uint32_t)];
  size_t offset = 0;

  for (uint8_t i = 0; i < CHANNEL_COUNT; ++i) {
    writeU32BigEndian(&body[offset], alarmMaskForChannel(channels[i]));
    offset += sizeof(uint32_t);
  }

  return sendPscMessage(PSC_MSG_ALARMS, body, sizeof(body));
}

/*PSC setpoint address map.
```

```
257
258   Each channel holds four consecutive addresses:
259
260     channel * 4 + 0 : voltage low limit
261     channel * 4 + 1 : voltage high limit
262     channel * 4 + 2 : current low limit
263     channel * 4 + 3 : current high limit
264
265   Example:
266     addresses 0-3   -> channel 0
267     addresses 4-7   -> channel 1
268     ...
269 */
270 float limitValueByAddress(uint32_t address) {
271   const uint8_t channelIndex = address / 4;
272   const uint8_t fieldIndex = address % 4;
273   const AlarmLimits &limits = channels[channelIndex].limits;
274
275   switch (fieldIndex) {
276     case 0:
277       return limits.voltageLow;
278     case 1:
279       return limits.voltageHigh;
280     case 2:
281       return limits.currentLow;
282     default:
283       return limits.currentHigh;
284   }
285 }
286
287 bool sendPscSetpointEcho(uint32_t address, float value) {
288   uint8_t body[8];
289   writeU32BigEndian(&body[0], address);
290   writeFloatBigEndian(&body[4], value);
291   return sendPscMessage(PSC_MSG_SETPOINT, body, sizeof(body));
292 }
293
294 void sendPscChannelSetpointEchoes(uint8_t channelIndex) {
295   if (!pscIsConnected() || channelIndex >= CHANNEL_COUNT) {
296     return;
297   }
298
299   const uint32_t firstAddress = static_cast<uint32_t>(channelIndex) * 4;
300
301   for (uint32_t address = firstAddress;
302        address < firstAddress + 4;
303        ++address) {
304     sendPscSetpointEcho(address, limitValueByAddress(address));
305   }
306 }
307
308 void sendAllPscSetpointEchoes() {
309   if (!pscIsConnected()) {
310     return;
311   }
312
313   for (uint32_t address = 0; address < CHANNEL_COUNT * 4; ++address) {
314     sendPscSetpointEcho(address, limitValueByAddress(address));
315   }
316 }
317
318 /*Send all states required to synchronize the IOC.
319   This prevents the IOC from waiting for multiple periodic publish cycles
320   before measurements, limits, alarms, and AO setpoint readbacks are valid.*/
321 void sendCompletePscSnapshot() {
322   if (!pscIsConnected()) {
323     return;
324   }
```

```
325
326   sendPscMeasurements();
327   sendPscLimits();
328   sendPscAlarms();
329   sendAllPscSetpointEchoes();
330   lastPscPublishMs = millis();
331 }
332
333 /*Apply one alarm-limit update received from EPICS.
334
335   The requested value is accepted only if:
336     1. the address maps to a valid channel/limit field,
337     2. the value is finite,
338     3. the complete set of limits remains logically valid, and
339     4. the updated configuration can be saved to persistent storage.
340
341   If validation or persistent storage fails, the previous limits are
342   restored so RAM and NVM configuration remain consistent.*/
343 bool applyPscSetpoint(uint32_t address, float requestedValue) {
344   if (address >= CHANNEL_COUNT * 4 || !isfinite(requestedValue)) {
345     return false;
346   }
347
348   const uint8_t channelIndex = address / 4;
349   const uint8_t fieldIndex = address % 4;
350   MonitorChannel &channel = channels[channelIndex];
351   const AlarmLimits previousLimits = channel.limits;
352
353   switch (fieldIndex) {
354     case 0:
355       channel.limits.voltageLow = requestedValue;
356       break;
357     case 1:
358       channel.limits.voltageHigh = requestedValue;
359       break;
360     case 2:
361       channel.limits.currentLow = requestedValue;
362       break;
363     case 3:
364       channel.limits.currentHigh = requestedValue;
365       break;
366     default:
367       return false;
368   }
369
370   if (!alarmLimitsAreValid(channel.limits)) {
371     channel.limits = previousLimits;
372     return false;
373   }
374
375   updateAlarmState(channel);
376
377   if (!savePersistentSettings()) {
378     channel.limits = previousLimits;
379     updateAlarmState(channel);
380     return false;
381   }
382
383   return true;
384 }
385
386 /*Process a complete received PSC message. The embedded controller currently
387 accepts only PSC_MSG_SETPOINT messagesfrom the IOC. All other message types
388 are ignored and reported over the serial console.*/
389
390 void handlePscMessage(
391   uint16_t messageId,
392   const uint8_t *body,
```

```
  uint32_t bodyLength
) {
  if (messageId != PSC_MSG_SETPOINT || bodyLength != 8) {
    Serial.print("Ignoring PSC message ID ");
    Serial.print(messageId);
    Serial.print(" with body length ");
    Serial.println(bodyLength);
    return;
  }

  const uint32_t address = readU32BigEndian(&body[0]);
  const float requestedValue = readFloatBigEndian(&body[4]);

  if (address >= CHANNEL_COUNT * 4) {
    Serial.print("Rejected PSC setpoint address ");
    Serial.println(address);
    return;
  }

  const bool accepted = applyPscSetpoint(address, requestedValue);
  const float actualValue = limitValueByAddress(address);

  Serial.print("PSC setpoint address ");
  Serial.print(address);
  Serial.print(accepted ? " accepted: " : " rejected; actual: ");
  Serial.println(actualValue, 4);

  // Echo the actual device value so info(SYNC, "SAME") AO records resync.
  sendPscSetpointEcho(address, actualValue);
  sendPscLimits();
  sendPscAlarms();
}

/*
  Incremental PSC packet parser.

  TCP is a byte stream, so a complete PSC packet may not arrive in a single
  read. This state machine:

    1. searches for the "PS" synchronization bytes,
    2. collects the remaining 8-byte header,
    3. extracts message ID and body length,
    4. validates the body length,
    5. collects the body,
    6. dispatches the complete message.

  If synchronization is lost, incoming bytes are discarded until another
  valid "PS" header prefix is found.
*/
void consumePscByte(uint8_t value) {
  if (pscReadingBody) {
    pscBodyBuffer[pscBodyIndex++] = value;

    if (pscBodyIndex >= pscIncomingBodyLength) {
      handlePscMessage(
        pscIncomingMessageId,
        pscBodyBuffer,
        pscIncomingBodyLength
      );
      resetPscReceiveState();
    }

    return;
  }

  if (pscHeaderIndex == 0) {
    if (value == 'P') {
      pscHeaderBuffer[0] = value;
```

```
      pscHeaderIndex = 1;
    }
    return;
  }

  if (pscHeaderIndex == 1) {
    if (value == 'S') {
      pscHeaderBuffer[1] = value;
      pscHeaderIndex = 2;
    } else if (value == 'P') {
      pscHeaderBuffer[0] = value;
      pscHeaderIndex = 1;
    } else {
      pscHeaderIndex = 0;
    }
    return;
  }

  pscHeaderBuffer[pscHeaderIndex++] = value;

  if (pscHeaderIndex < sizeof(pscHeaderBuffer)) {
    return;
  }

  pscIncomingMessageId = readU16BigEndian(&pscHeaderBuffer[2]);
  pscIncomingBodyLength = readU32BigEndian(&pscHeaderBuffer[4]);
  pscBodyIndex = 0;

  if (pscIncomingBodyLength > PSC_MAX_BODY_LENGTH) {
    Serial.print("PSC body too large: ");
    Serial.println(pscIncomingBodyLength);
    pscClient.stop();
    resetPscReceiveState();
    return;
  }

  if (pscIncomingBodyLength == 0) {
    handlePscMessage(pscIncomingMessageId, nullptr, 0);
    resetPscReceiveState();
    return;
  }

  pscReadingBody = true;
}

void acceptPscClient() {
  EthernetClient incoming = pscServer.accept();

  if (!incoming) {
    return;
  }

  if (pscIsConnected()) {
    // PSCDriver normally creates only one connection. Reject extras cleanly.
    incoming.stop();
    return;
  }

  pscClient.stop();
  pscClient = incoming;
  resetPscReceiveState();

  Serial.print("PSC client connected from ");
  Serial.println(pscClient.remoteIP());
  sendCompletePscSnapshot();
}

/*
```

```
  Non-blocking PSC network service routine.

  Called repeatedly from the main application loop to:
    - detect disconnected clients,
    - accept a new IOC connection,
    - consume all currently available receive bytes,
    - periodically publish measurements and alarm state.

  Limits are not sent every cycle because they change only when
  configuration is modified; a complete copy is sent on initial connection
  and after a setpoint transaction.
*/
void servicePsc() {
  if (!ethernetHardwarePresent) {
    return;
  }

  if (pscClient && !pscClient.connected()) {
    Serial.println("PSC client disconnected.");
    pscClient.stop();
    resetPscReceiveState();
  }

  acceptPscClient();

  if (!pscIsConnected()) {
    return;
  }

  while (pscClient.available() > 0) {
    const int value = pscClient.read();

    if (value < 0) {
      break;
    }

    consumePscByte(static_cast<uint8_t>(value));
  }

  const uint32_t now = millis();

  if (now - lastPscPublishMs >= PSC_PUBLISH_PERIOD_MS) {
    lastPscPublishMs = now;
    sendPscMeasurements();
    sendPscAlarms();
  }
}
```

Listing C.8: Firmware: src/PscProtocol.cpp

```
#include "SensorManager.h"

#include <Arduino.h>
#include <Wire.h>

#include "AppConfig.h"
#include "AppState.h"
#include "LTC2945.h"
#include "SensorConfig.h"

namespace {

// All six isolator outputs share the same GIGA-side bus.
TwoWire &sensorBus = Wire2;

/*
  Array order must match channels[] in AppState.cpp:
```

```
18
19      0 +6VA
20      1 +6VB
21      2 +5V
22      3 -5V
23      4 +15V
24      5 -15V
25
26    Positive rails use SENSE+ as the LTC2945 VIN source.
27
28    The -5V and -15V channels use low-side current sensing. Their
29    LTC2945 VIN ADC therefore measures VDD instead of SENSE+.
30
31    For negative rails the local VDD measurement is a positive
32    magnitude. The -1.0 voltage polarity multiplier converts that
33    magnitude to the corresponding negative system-level voltage.
34  */
35
36
37  // Constructor arguments:
38  //
39  //   I2C bus
40  //   device address
41  //   shunt resistance
42  //   voltage polarity multiplier
43  //   current polarity multiplier
44  //   VIN measurement source
45  //
46  LTC2945 sensors[CHANNEL_COUNT] = {
47    {
48      sensorBus,
49      LTC_ADDRESS_6VA,
50      SHUNT_6VA_OHMS,
51      +1.0f,
52      +1.0f,
53      LTC2945::VoltageSource::SensePlus
54    },
55
56    {
57      sensorBus,
58      LTC_ADDRESS_6VB,
59      SHUNT_6VB_OHMS,
60      +1.0f,
61      +1.0f,
62      LTC2945::VoltageSource::SensePlus
63    },
64
65    {
66      sensorBus,
67      LTC_ADDRESS_P5V,
68      SHUNT_P5V_OHMS,
69      +1.0f,
70      +1.0f,
71      LTC2945::VoltageSource::SensePlus
72    },
73
74    {
75      sensorBus,
76      LTC_ADDRESS_M5V,
77      SHUNT_M5V_OHMS,
78      -1.0f,
79      +1.0f,
80      LTC2945::VoltageSource::Vdd
81    },
82
83    {
84      sensorBus,
85      LTC_ADDRESS_P15V,
```

```
    SHUNT_P15V_OHMS,
    +1.0f,
    +1.0f,
    LTC2945::VoltageSource::SensePlus
  },

  {
    sensorBus,
    LTC_ADDRESS_M15V,
    SHUNT_M15V_OHMS,
    -1.0f,
    +1.0f,
    LTC2945::VoltageSource::Vdd
  }
};


void printAddress(
  uint8_t address
) {
  Serial.print("0x");

  if (address < 0x10) {
    Serial.print('0');
  }

  Serial.print(
    address,
    HEX
  );
}


void reportSensorTransition(
  uint8_t index,
  bool previousOnline,
  bool currentOnline
) {
  if (
    previousOnline ==
    currentOnline
  ) {
    return;
  }

  Serial.print(
    channels[index].name
  );

  Serial.print(" ");

  Serial.println(
    currentOnline
      ? "ONLINE"
      : "OFFLINE"
  );
}

}


/*
  Print channel names and configured I2C addresses.
*/
void printSensorConfiguration() {
  Serial.println();

  Serial.println(
```

```
    "LTC2945 sensor configuration:"
  );

  for (
    uint8_t i = 0;
    i < CHANNEL_COUNT;
    ++i
  ) {
    Serial.print("  ");

    Serial.print(
      channels[i].name
    );

    Serial.print(
      "  address "
    );

    printAddress(
      sensors[i].address()
    );

    Serial.println();
  }

  Serial.println();
}


/*
  Initialize the I2C bus and each LTC2945 monitor.

  Each channel begins in an invalid/offline state.

  LTC2945::begin():

    - verifies the device responds
    - writes the appropriate CONTROL value
    - reads the CONTROL register back
    - verifies the configuration

  Positive rails:
    CONTROL = 0x05
    VIN = SENSE+

  Negative rails:
    CONTROL = 0x01
    VIN = VDD

  Failed initialization records the communication/configuration error
  and immediately sets the failure count to the offline threshold.

  A complete measurement pass is performed before the display and PSC
  interfaces begin.
*/
void initializeSensors() {
  sensorBus.begin();

  sensorBus.setClock(
    SENSOR_I2C_CLOCK_HZ
  );

  printSensorConfiguration();

  for (
    uint8_t i = 0;
    i < CHANNEL_COUNT;
    ++i
```

```
  ) {
    MonitorChannel &channel =
      channels[i];

    channel.readingValid = false;
    channel.sensorOnline = false;
    channel.sensorError = 0;
    channel.consecutiveSensorFailures = 0;
    channel.lastValidReadingMs = 0;

    const bool initialized =
      sensors[i].begin();

    Serial.print(
      channel.name
    );

    Serial.print(
      " at "
    );

    printAddress(
      sensors[i].address()
    );

    if (initialized) {
      Serial.println(
        " acknowledged and configured"
      );
    } else {
      channel.sensorError =
        sensors[i].lastError();

      channel.consecutiveSensorFailures =
        SENSOR_FAILURES_TO_OFFLINE;

      Serial.print(
        " failed, error 0x"
      );

      Serial.println(
        channel.sensorError,
        HEX
      );
    }
  }

  /*
    Populate the first complete snapshot before the display and
    PSC services begin.
  */
  updateSensorMeasurements();
}


/*
  Poll all six LTC2945 channels and update MonitorChannel state.

  LTC2945::read() verifies the CONTROL register before every
  measurement.

  This makes recovery automatic if an individual monitored supply is
  switched off and back on while the Arduino remains powered.

  Example:

    -5V monitor running:
      CONTROL = 0x01
```

```
290
291     -5V supply turned off:
292       LTC2945 loses power
293
294     -5V supply turned back on:
295       LTC2945 starts with CONTROL = 0x05
296
297     next Arduino poll:
298       read() detects incorrect CONTROL value
299       CONTROL is rewritten to 0x01
300       readback is verified
301       voltage/current measurements are then acquired
302
303   Successful reads:
304
305     - replace voltage/current with newest measurement
306     - mark reading valid
307     - mark sensor online
308     - clear communication error
309     - clear failure counter
310     - update timestamp
311     - update alarm state
312
313   Failed reads:
314
315     - preserve last valid measurement values
316     - record LTC2945/I2C error
317     - increment failure counter
318     - mark sensor offline after the configured failure threshold
319 */
320 void updateSensorMeasurements() {
321   for (
322     uint8_t i = 0;
323     i < CHANNEL_COUNT;
324     ++i
325   ) {
326     MonitorChannel &channel =
327       channels[i];
328
329     const bool previousOnline =
330       channel.sensorOnline;
331
332     LTC2945::Reading reading;
333
334     if (
335       sensors[i].read(reading)
336     ) {
337       channel.voltage =
338         reading.voltage;
339
340       channel.current =
341         reading.current;
342
343       channel.readingValid = true;
344       channel.sensorOnline = true;
345       channel.sensorError = 0;
346       channel.consecutiveSensorFailures = 0;
347       channel.lastValidReadingMs = millis();
348
349       updateAlarmState(
350         channel
351       );
352     } else {
353       channel.sensorError =
354         sensors[i].lastError();
355
356       if (
357         channel.consecutiveSensorFailures <
```

```
        255
      ) {
        ++channel.consecutiveSensorFailures;
      }

      if (
        channel.consecutiveSensorFailures >=
        SENSOR_FAILURES_TO_OFFLINE
      ) {
        channel.sensorOnline = false;
      }

      /*
        Keep the last valid voltage and current.

        Replacing a failed read with zero could make an I2C or
        configuration fault appear to be a genuine rail
        undervoltage condition.
      */
    }

    /*
      Log only actual ONLINE/OFFLINE transitions.
    */
    reportSensorTransition(
      i,
      previousOnline,
      channel.sensorOnline
    );
  }
}
```

Listing C.9: Firmware: src/SensorManager.cpp

C.3.2 Firmware Header Files

```
#pragma once

#include <Arduino.h>

// ---------------------------------------------------------------------------
// Application size and scheduling
// ---------------------------------------------------------------------------
constexpr uint8_t CHANNEL_COUNT = 6;
constexpr uint32_t DISPLAY_REFRESH_PERIOD_MS = 500;

// ---------------------------------------------------------------------------
// Ethernet Shield 2
// ---------------------------------------------------------------------------
constexpr uint8_t ETHERNET_CS_PIN = 10;
constexpr uint8_t SD_CS_PIN = 4;
constexpr uint16_t HTTP_PORT = 80;
constexpr uint16_t PSC_PORT = 8765;

// ---------------------------------------------------------------------------
// PSCDriver protocol
// ---------------------------------------------------------------------------
constexpr uint32_t PSC_PUBLISH_PERIOD_MS = 500;
constexpr uint32_t PSC_WRITE_TIMEOUT_MS = 250;
constexpr uint32_t PSC_MAX_BODY_LENGTH = 256;

constexpr uint16_t PSC_MSG_MEASUREMENTS = 1;
constexpr uint16_t PSC_MSG_LIMITS = 2;
constexpr uint16_t PSC_MSG_ALARMS = 3;
```

```
constexpr uint16_t PSC_MSG_SETPOINT = 10;

// ----------------------------------------------------------------------------
// Persistent QSPI settings
// ----------------------------------------------------------------------------
constexpr bool FORMAT_USER_PARTITION_IF_NEEDED = true;

constexpr uint32_t SETTINGS_MAGIC = 0x4D524C4DUL;  // "MRLM"
constexpr uint16_t SETTINGS_VERSION = 2;

constexpr const char *SETTINGS_FILE = "/user/rail_limits.bin";
constexpr const char *SETTINGS_TEMP_FILE = "/user/rail_limits.tmp";

/*
  Recovery mode:
  Set true and upload over USB if a saved IP address makes the web interface
  unreachable. The compiled network defaults will be used for that boot.
*/
constexpr bool FORCE_COMPILED_NETWORK_ON_BOOT = false;
```

Listing C.10: Firmware: include/AppConfig.h

```
#pragma once

#include "DataModel.h"

// Compiled defaults and current runtime settings.
extern const NetworkSettings COMPILED_NETWORK_SETTINGS;
extern NetworkSettings networkSettings;

extern MonitorChannel channels[CHANNEL_COUNT];

extern bool settingsStorageReady;
extern bool networkRestartRequired;
extern bool ethernetHardwarePresent;

bool channelIsInAlarm(const MonitorChannel &channel);
void updateAlarmState(MonitorChannel &channel);
```

Listing C.11: Firmware: include/AppState.h

```
#pragma once

#include <Arduino.h>

#include "AppConfig.h"

struct NetworkSettings {
  uint8_t ip[4];
  uint8_t dns[4];
  uint8_t gateway[4];
  uint8_t subnet[4];
};

struct AlarmLimits {
  float voltageLow;
  float voltageHigh;
  float currentLow;
  float currentHigh;
};

struct PersistentSettings {
  uint32_t magic;
  uint16_t version;
  uint16_t channelCount;
  AlarmLimits limits[CHANNEL_COUNT];
```

```
  NetworkSettings network;
  uint32_t checksum;
};

struct MonitorChannel {
  const char *name;
  float nominalVoltage;
  float nominalCurrent;
  float voltage;
  float current;
  AlarmLimits limits;
  bool voltageLowAlarm;
  bool voltageHighAlarm;
  bool currentLowAlarm;
  bool currentHighAlarm;

  bool readingValid;
  bool sensorOnline;
  uint8_t sensorError;
  uint8_t consecutiveSensorFailures;
  uint32_t lastValidReadingMs;
};
```

Listing C.12: Firmware: include/DataModel.h

```
#pragma once

void initializeDisplay();
void redrawDisplay();
```

Listing C.13: Firmware: include/DisplayUI.h

```
#pragma once

void initializeEthernet();
```

Listing C.14: Firmware: include/EthernetServices.h

```
#pragma once

void initializeHttpServer();
void serviceHttp();
```

Listing C.15: Firmware: include/HttpDashboard.h

```
#pragma once

#include <Arduino.h>
#include <Wire.h>

class LTC2945 {
 public:
  enum class VoltageSource : uint8_t {
    SensePlus,
    Vdd
  };

  struct Reading {
    float voltage = NAN;
    float current = NAN;
    float power = NAN;
    uint16_t voltageCode = 0;
    uint16_t currentCode = 0;
    bool valid = false;
    uint8_t i2cError = 0;
```

```
  };

  /*
    voltageSign:
      +1.0 for +6VA, +6VB, +5V, +15V
      -1.0 for -5V and -15V

    currentSign:
      +1.0 when normal load current produces SENSE+ > SENSE-

    voltageSource:
      SensePlus for positive rails
      Vdd for negative rails using low-side current sensing
  */
  LTC2945(
    TwoWire &wire,
    uint8_t address,
    float shuntOhms,
    float voltageSign = 1.0f,
    float currentSign = 1.0f,
    VoltageSource voltageSource = VoltageSource::SensePlus
  );

  bool begin();
  bool ping();
  bool read(Reading &reading);

  uint8_t address() const;
  uint8_t lastError() const;

private:
  static constexpr uint8_t REG_CONTROL = 0x00;
  static constexpr uint8_t REG_DELTA_SENSE_MSB = 0x14;
  static constexpr uint8_t REG_VIN_MSB = 0x1E;

  /*
    CONTROL register values:

      0x05:
        continuous conversion
        VIN source = SENSE+

      0x01:
        continuous conversion
        VIN source = VDD
  */
  static constexpr uint8_t CONTROL_CONTINUOUS_SENSE_PLUS = 0x05;
  static constexpr uint8_t CONTROL_CONTINUOUS_VDD = 0x01;

  static constexpr float VIN_LSB_VOLTS = 0.025f;
  static constexpr float DELTA_SENSE_LSB_VOLTS = 25.0e-6f;

  TwoWire &wire_;
  uint8_t address_;
  float shuntOhms_;
  float voltageSign_;
  float currentSign_;
  VoltageSource voltageSource_;

  uint8_t lastError_ = 0;

  uint8_t expectedControlValue() const;

  bool configure();
  bool ensureConfigured();

  bool writeRegister8(uint8_t reg, uint8_t value);
  bool readRegister8(uint8_t reg, uint8_t &value);
```

```
89
90    bool readRegisters(
91      uint8_t startReg,
92      uint8_t *buffer,
93      size_t length
94    );
95
96    bool read12Bit(
97      uint8_t msbRegister,
98      uint16_t &code
99    );
100 };
```

Listing C.16: Firmware: include/LTC2945.h

```
1  #pragma once
2
3  #include "DataModel.h"
4
5  bool initializeSettingsStorage();
6  bool loadPersistentSettings();
7  bool savePersistentSettings();
8
9  bool alarmLimitsAreValid(const AlarmLimits &limits);
10 bool hostAddressIsValid(const uint8_t address[4]);
11 bool optionalHostAddressIsValid(const uint8_t address[4]);
12 bool subnetMaskIsValid(const uint8_t subnet[4]);
13 bool networkSettingsAreValid(const NetworkSettings &settings);
```

Listing C.17: Firmware: include/PersistentStorage.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  void initializePscServer();
6  void servicePsc();
7
8  bool pscIsConnected();
9
10 bool sendPscMeasurements();
11 bool sendPscLimits();
12 bool sendPscAlarms();
13 void sendPscChannelSetpointEchoes(uint8_t channelIndex);
14 void sendAllPscSetpointEchoes();
15 void sendCompletePscSnapshot();
```

Listing C.18: Firmware: include/PscProtocol.h

```
1  #pragma once
2
3  #include <Arduino.h>
4
5  constexpr uint32_t SENSOR_I2C_CLOCK_HZ = 100000;
6  constexpr uint32_t SENSOR_POLL_PERIOD_MS = 200;
7  constexpr uint8_t SENSOR_FAILURES_TO_OFFLINE = 3;
8
9  /*
10   Arduino Wire uses 7-bit addresses.
11
12   Schematic device-address byte -> Arduino address:
13     CE       -> 0x67
14     D0/D1    -> 0x68
15     D2/D3    -> 0x69
16     D4/D5    -> 0x6A
17     D6/D7    -> 0x6B
```

```
    D8/D9    -> 0x6C
*/
constexpr uint8_t LTC_ADDRESS_6VA  = 0x67;
constexpr uint8_t LTC_ADDRESS_6VB  = 0x6D;
constexpr uint8_t LTC_ADDRESS_P15V = 0x69;
constexpr uint8_t LTC_ADDRESS_M15V = 0x6A;
constexpr uint8_t LTC_ADDRESS_P5V  = 0x6B;
constexpr uint8_t LTC_ADDRESS_M5V  = 0x6C;

constexpr float SHUNT_6VA_OHMS  = 0.039f;
constexpr float SHUNT_6VB_OHMS  = 0.039f;
constexpr float SHUNT_P5V_OHMS  = 0.050f;
constexpr float SHUNT_M5V_OHMS  = 0.050f;
constexpr float SHUNT_P15V_OHMS = 0.050f;
constexpr float SHUNT_M15V_OHMS = 0.050f;
```

Listing C.19: Firmware: include/SensorConfig.h

```
#pragma once

void initializeSensors();
void updateSensorMeasurements();
void printSensorConfiguration();
```

Listing C.20: Firmware: include/SensorManager.h

```
/**
 * @file lvgl.h
 * Include all LVGL related headers
 */

#ifndef LVGL_H
#define LVGL_H

#ifdef __cplusplus
extern "C" {
#endif

/*************************
 * CURRENT VERSION OF LVGL
 *************************/
#include "lv_version.h"

/*********************
 *      INCLUDES
 *********************/
#include "src/lv_init.h"

#include "src/stdlib/lv_mem.h"
#include "src/stdlib/lv_string.h"
#include "src/stdlib/lv_sprintf.h"

#include "src/misc/lv_log.h"
#include "src/misc/lv_timer.h"
#include "src/misc/lv_math.h"
#include "src/misc/lv_array.h"
#include "src/misc/lv_async.h"
#include "src/misc/lv_anim_timeline.h"
#include "src/misc/lv_profiler_builtin.h"
#include "src/misc/lv_rb.h"
#include "src/misc/lv_utils.h"
#include "src/misc/lv_iter.h"
#include "src/misc/lv_circle_buf.h"
#include "src/misc/lv_tree.h"

#include "src/osal/lv_os.h"
```

```
41
42  #include "src/tick/lv_tick.h"
43
44  #include "src/core/lv_obj.h"
45  #include "src/core/lv_group.h"
46  #include "src/core/lv_refr.h"
47  #include "src/core/lv_observer.h"
48  #include "src/indev/lv_indev.h"
49  #include "src/indev/lv_indev_gesture.h"
50  #include "src/indev/lv_gridnav.h"
51  #include "src/display/lv_display.h"
52
53  #include "src/font/lv_font.h"
54  #include "src/font/binfont_loader/lv_binfont_loader.h"
55  #include "src/font/fmt_txt/lv_font_fmt_txt.h"
56  #include "src/font/imgfont/lv_imgfont.h"
57  #include "src/font/font_manager/lv_font_manager.h"
58
59  #include "src/widgets/animimage/lv_animimage.h"
60  #include "src/widgets/arc/lv_arc.h"
61  #include "src/widgets/arclabel/lv_arclabel.h"
62  #include "src/widgets/bar/lv_bar.h"
63  #include "src/widgets/button/lv_button.h"
64  #include "src/widgets/buttonmatrix/lv_buttonmatrix.h"
65  #include "src/widgets/calendar/lv_calendar.h"
66  #include "src/widgets/canvas/lv_canvas.h"
67  #include "src/widgets/chart/lv_chart.h"
68  #include "src/widgets/checkbox/lv_checkbox.h"
69  #include "src/widgets/dropdown/lv_dropdown.h"
70  #include "src/widgets/gif/lv_gif.h"
71  #include "src/widgets/image/lv_image.h"
72  #include "src/widgets/imagebutton/lv_imagebutton.h"
73  #include "src/widgets/keyboard/lv_keyboard.h"
74  #include "src/widgets/label/lv_label.h"
75  #include "src/widgets/led/lv_led.h"
76  #include "src/widgets/line/lv_line.h"
77  #include "src/widgets/list/lv_list.h"
78  #include "src/widgets/lottie/lv_lottie.h"
79  #include "src/widgets/menu/lv_menu.h"
80  #include "src/widgets/msgbox/lv_msgbox.h"
81  #include "src/widgets/roller/lv_roller.h"
82  #include "src/widgets/scale/lv_scale.h"
83  #include "src/widgets/slider/lv_slider.h"
84  #include "src/widgets/span/lv_span.h"
85  #include "src/widgets/spinbox/lv_spinbox.h"
86  #include "src/widgets/spinner/lv_spinner.h"
87  #include "src/widgets/switch/lv_switch.h"
88  #include "src/widgets/table/lv_table.h"
89  #include "src/widgets/tabview/lv_tabview.h"
90  #include "src/widgets/textarea/lv_textarea.h"
91  #include "src/widgets/tileview/lv_tileview.h"
92  #include "src/widgets/win/lv_win.h"
93  #include "src/widgets/3dtexture/lv_3dtexture.h"
94  #include "src/widgets/ime/lv_ime_pinyin.h"
95
96  #include "src/debugging/sysmon/lv_sysmon.h"
97  #include "src/debugging/monkey/lv_monkey.h"
98  #include "src/debugging/test/lv_test.h"
99
100 #include "src/others/fragment/lv_fragment.h"
101 #include "src/others/file_explorer/lv_file_explorer.h"
102 #include "src/others/translation/lv_translation.h"
103
104 #include "src/libs/barcode/lv_barcode.h"
105 #include "src/libs/bin_decoder/lv_bin_decoder.h"
106 #include "src/libs/bmp/lv_bmp.h"
107 #include "src/libs/rle/lv_rle.h"
108 #include "src/libs/fsdrv/lv_fsdrv.h"
```

```
#include "src/libs/lodepng/lv_lodepng.h"
#include "src/libs/libpng/lv_libpng.h"
#include "src/libs/libwebp/lv_libwebp.h"
#include "src/libs/gltf/gltf_data/lv_gltf_model.h"
#include "src/libs/gltf/gltf_view/lv_gltf.h"
#include "src/libs/gstreamer/lv_gstreamer.h"
#include "src/libs/qrcode/lv_qrcode.h"
#include "src/libs/tjpgd/lv_tjpgd.h"
#include "src/libs/libjpeg_turbo/lv_libjpeg_turbo.h"
#include "src/libs/freetype/lv_freetype.h"
#include "src/libs/rlottie/lv_rlottie.h"
#include "src/libs/ffmpeg/lv_ffmpeg.h"
#include "src/libs/tiny_ttf/lv_tiny_ttf.h"
#include "src/libs/svg/lv_svg.h"
#include "src/libs/svg/lv_svg_render.h"

#include "src/layouts/lv_layout.h"

#include "src/draw/lv_draw_buf.h"
#include "src/draw/lv_draw_vector.h"
#include "src/draw/sw/lv_draw_sw_utils.h"
#include "src/draw/eve/lv_draw_eve_target.h"
#include "src/draw/snapshot/lv_snapshot.h"

#include "src/themes/lv_theme.h"

#include "src/drivers/lv_drivers.h"

/* Define LV_DISABLE_API_MAPPING using a compiler option
 * to make sure your application is not using deprecated names */
#ifndef LV_DISABLE_API_MAPPING
#include "src/lv_api_map_v8.h"
#include "src/lv_api_map_v9_0.h"
#include "src/lv_api_map_v9_1.h"
#include "src/lv_api_map_v9_2.h"
#include "src/lv_api_map_v9_3.h"
#include "src/lv_api_map_v9_4.h"
#endif /*LV_DISABLE_API_MAPPING*/

#if LV_USE_PRIVATE_API
#include "src/lvgl_private.h"
#endif


/*********************
 *      DEFINES
 *********************/

/**********************
 *      TYPEDEFS
 **********************/

/**********************
 * GLOBAL PROTOTYPES
 **********************/

/**********************
 *      MACROS
 **********************/

/** Gives 1 if the x.y.z version is supported in the current version
 * Usage:
 *
 * - Require v6
 * #if LV_VERSION_CHECK(6,0,0)
 *   new_func_in_v6();
 * #endif
 *
```

```
 *
 * - Require at least v5.3
 * #if LV_VERSION_CHECK(5,3,0)
 *   new_feature_from_v5_3();
 * #endif
 *
 *
 * - Require v5.3.2 bugfixes
 * #if LV_VERSION_CHECK(5,3,2)
 *   bugfix_in_v5_3_2();
 * #endif
 *
 */
#define LV_VERSION_CHECK(x,y,z) (x == LVGL_VERSION_MAJOR && (y < LVGL_VERSION_MINOR || (y ==
     LVGL_VERSION_MINOR && z <= LVGL_VERSION_PATCH)))

/**
 * Wrapper functions for VERSION macros
 */

static inline int lv_version_major(void)
{
    return LVGL_VERSION_MAJOR;
}

static inline int lv_version_minor(void)
{
    return LVGL_VERSION_MINOR;
}

static inline int lv_version_patch(void)
{
    return LVGL_VERSION_PATCH;
}

static inline const char * lv_version_info(void)
{
    return LVGL_VERSION_INFO;
}

#ifdef __cplusplus
} /*extern "C"*/
#endif

#endif /*LVGL_H*/
```

Listing C.21: Firmware: include/lvgl.h

C.3.3 EPICS IOC

```
file "db/gigaPscChannel.template" {
pattern {
  CHAN, DESC, VOFF, IOFF, VLOWOFF, VHIGHOFF, ILOWOFF, IHIGHOFF, SOFF,
  VLOWADDR, VHIGHADDR, ILOWADDR, IHIGHADDR, VLO, VHI, ILO_MA, IHI_MA
}
{
  "6VA",  "+6 V A",  0,  4,  0,  4,  8, 12,  0,  0,  1,  2,  3,   5.40,   6.60, 0.00, 1200
}
{
  "6VB",  "+6 V B",  8, 12, 16, 20, 24, 28,  4,  4,  5,  6,  7,   5.40,   6.60, 0.00, 1200
}
{
  "P5V",  "+5 V",   16, 20, 32, 36, 40, 44,  8,  8,  9, 10, 11,   4.50,   5.50, 0.00, 1000
}
```

```
{
  "M5V",  "-5 V",   24, 28, 48, 52, 56, 60, 12, 12, 13, 14, 15,  -5.50,  -4.50, 0.00, 1000
}
{
  "P15V", "+15 V",  32, 36, 64, 68, 72, 76, 16, 16, 17, 18, 19,  13.50,  16.50, 0.00, 750
}
{
  "M15V", "-15 V",  40, 44, 80, 84, 88, 92, 20, 20, 21, 22, 23, -16.50, -13.50, 0.00, 750
}
}
```

Listing C.22: EPICS IOC: db/gigaPsc.substitutions

```
record(bi, "$(P)$(R):CONNECTED") {
    field(DESC, "GIGA PSC TCP connection")
    field(DTYP, "PSC Ctrl Connected")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "I/O Intr")
    field(ZNAM, "Disconnected")
    field(ONAM, "Connected")
    field(ZSV,  "MAJOR")
}

record(stringin, "$(P)$(R):DRIVER_MESSAGE") {
    field(DESC, "Last PSCDriver message")
    field(DTYP, "PSC Ctrl Message")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "I/O Intr")
}

record(longin, "$(P)$(R):CONNECTION_COUNT") {
    field(DESC, "PSC connection count")
    field(DTYP, "PSC Conn Count")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "1 second")
}

record(longin, "$(P)$(R):UNKNOWN_MESSAGE_COUNT") {
    field(DESC, "Unknown PSC message count")
    field(DTYP, "PSC Unknown Msg Count")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "1 second")
}

# PSC Single F32 records queue one message. This flushes queued messages to TCP.
record(bo, "$(P)$(R):SEND_ALL") {
    field(DESC, "Flush PSC transmit queue")
    field(DTYP, "PSC Ctrl Send All")
    field(OUT,  "@$(DEVICE)")
    field(ZNAM, "Send")
    field(ONAM, "Send")
}
```

Listing C.23: EPICS IOC: db/gigaPscBase.db

```
# Direct PSC receive records. These are intentionally RAW/internal PVs.

record(ai, "$(P)$(R):$(CHAN):VOLT_RAW") {
field(DESC, "$(DESC) voltage from PSC")
field(DTYP, "PSC Reg F32")
field(INP,  "@$(DEVICE) 1 $(VOFF)")
field(SCAN, "I/O Intr")
field(TSE,  "-2")
field(PREC, "4")
field(EGU,  "V")
}
```

```
12
13 record(ai, "$(P)$(R):$(CHAN):CURR_RAW") {
14 field(DESC, "$(DESC) current from PSC")
15 field(DTYP, "PSC Reg F32")
16 field(INP,  "@$(DEVICE) 1 $(IOFF)")
17 field(SCAN, "I/O Intr")
18 field(TSE,  "-2")
19 field(PREC, "4")
20 field(EGU,  "A")
21 }
22
23 # Public calc records copy and scale the direct values.
24
25 # The CONNECTED CP/MS link causes immediate processing and propagates
26
27 # a MAJOR alarm when the TCP connection drops.
28
29 record(calc, "$(P)$(R):$(CHAN):VOLT") {
30 field(DESC, "$(DESC) voltage")
31 field(INPA, "$(P)$(R):$(CHAN):VOLT_RAW CP NMS")
32 field(INPB, "$(P)$(R):CONNECTED CP MS")
33 field(CALC, "A")
34 field(PREC, "4")
35 field(EGU,  "V")
36 field(LOW,  "$(VLO)")
37 field(LSV,  "MAJOR")
38 field(HIGH, "$(VHI)")
39 field(HSV,  "MAJOR")
40 }
41
42 record(calc, "$(P)$(R):$(CHAN):CURR") {
43 field(DESC, "$(DESC) current")
44 field(INPA, "$(P)$(R):$(CHAN):CURR_RAW CP NMS")
45 field(INPB, "$(P)$(R):CONNECTED CP MS")
46
47
48 # Convert amperes received from the Arduino to milliamperes.
49 field(CALC, "A*1000")
50
51 field(PREC, "3")
52 field(EGU,  "mA")
53
54 # The public current alarm limits are expressed in milliamperes.
55 field(LOW,  "$(ILO_MA)")
56 field(LSV,  "MAJOR")
57 field(HIGH, "$(IHI_MA)")
58 field(HSV,  "MAJOR")
59
60
61 }
62
63 # Device-confirmed alarm-limit readbacks from PSC message 2.
64
65 # Current readbacks remain in amperes because they come directly
66
67 # from the Arduino/PSC protocol.
68
69 record(ai, "$(P)$(R):$(CHAN):VLOW_RBV") {
70 field(DTYP, "PSC Reg F32")
71 field(INP,  "@$(DEVICE) 2 $(VLOWOFF)")
72 field(SCAN, "I/O Intr")
73 field(PREC, "4")
74 field(EGU,  "V")
75 field(FLNK, "$(P)$(R):$(CHAN):SYNC_VLOW")
76 }
77
78 record(ai, "$(P)$(R):$(CHAN):VHIGH_RBV") {
79 field(DTYP, "PSC Reg F32")
```

```
field(INP,  "@$(DEVICE) 2 $(VHIGHOFF)")
field(SCAN, "I/O Intr")
field(PREC, "4")
field(EGU,  "V")
field(FLNK, "$(P)$(R):$(CHAN):SYNC_VHIGH")
}

record(ai, "$(P)$(R):$(CHAN):ILOW_RBV") {
field(DTYP, "PSC Reg F32")
field(INP,  "@$(DEVICE) 2 $(ILOWOFF)")
field(SCAN, "I/O Intr")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):$(CHAN):SYNC_ILOW")
}

record(ai, "$(P)$(R):$(CHAN):IHIGH_RBV") {
field(DTYP, "PSC Reg F32")
field(INP,  "@$(DEVICE) 2 $(IHIGHOFF)")
field(SCAN, "I/O Intr")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):$(CHAN):SYNC_IHIGH")
}

# Keep the public measurement alarm fields synchronized

# with the device-confirmed alarm limits.

record(calcout, "$(P)$(R):$(CHAN):SYNC_VLOW") {
field(INPA, "$(P)$(R):$(CHAN):VLOW_RBV NPP NMS")
field(CALC, "A")
field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):VOLT.LOW NPP")
}

record(calcout, "$(P)$(R):$(CHAN):SYNC_VHIGH") {
field(INPA, "$(P)$(R):$(CHAN):VHIGH_RBV NPP NMS")
field(CALC, "A")
field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):VOLT.HIGH NPP")
}

record(calcout, "$(P)$(R):$(CHAN):SYNC_ILOW") {
field(INPA, "$(P)$(R):$(CHAN):ILOW_RBV NPP NMS")


# Convert the device limit from amperes to milliamperes before
# writing it into the CURR record's LOW alarm field.
field(CALC, "A*1000")

field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):CURR.LOW NPP")


}

record(calcout, "$(P)$(R):$(CHAN):SYNC_IHIGH") {
field(INPA, "$(P)$(R):$(CHAN):IHIGH_RBV NPP NMS")


# Convert the device limit from amperes to milliamperes before
# writing it into the CURR record's HIGH alarm field.
field(CALC, "A*1000")

```

```
field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):CURR.HIGH NPP")


}

# Device-side alarm mask:

# Bit 0: VLOW

# Bit 1: VHIGH

# Bit 2: ILOW

# Bit 3: IHIGH

record(longin, "$(P)$(R):$(CHAN):ALARM_MASK") {
field(DESC, "$(DESC) device alarm mask")
field(DTYP, "PSC Reg")
field(INP,  "@$(DEVICE) 3 $(SOFF)")
field(SCAN, "I/O Intr")
}

record(calc, "$(P)$(R):$(CHAN):ANY_ALARM") {
field(DESC, "$(DESC) any device alarm")
field(INPA, "$(P)$(R):$(CHAN):ALARM_MASK CP NMS")
field(INPB, "$(P)$(R):CONNECTED CP MS")
field(CALC, "A!=0")
field(HIGH, "0.5")
field(HSV,  "MAJOR")
}

# Immediate persistent writes through PSC Single F32 message ID 10.

# info(SYNC, "SAME") lets the Arduino echo the actual accepted value back.

record(ao, "$(P)$(R):$(CHAN):VLOW_SP") {
field(DESC, "$(DESC) voltage low setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(VLOWADDR)")
field(PREC, "4")
field(EGU,  "V")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):VHIGH_SP") {
field(DESC, "$(DESC) voltage high setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(VHIGHADDR)")
field(PREC, "4")
field(EGU,  "V")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):ILOW_SP") {
field(DESC, "$(DESC) current low setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(ILOWADDR)")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):IHIGH_SP") {
```

```
field(DESC, "$(DESC) current high setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(IHIGHADDR)")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}
```

Listing C.24: EPICS IOC: db/gigaPscChannel.template

```
menu(aSubEFLG) {
    choice(aSubEFLG_NEVER, "NEVER")
    choice(aSubEFLG_ON_CHANGE, "ON CHANGE")
    choice(aSubEFLG_ALWAYS, "ALWAYS")
}
menu(aSubLFLG) {
    choice(aSubLFLG_IGNORE, "IGNORE")
    choice(aSubLFLG_READ, "READ")
}
menu(aaiPOST) {
    choice(aaiPOST_Always, "Always")
    choice(aaiPOST_OnChange, "On Change")
}
menu(aaoPOST) {
    choice(aaoPOST_Always, "Always")
    choice(aaoPOST_OnChange, "On Change")
}
menu(aoOIF) {
    choice(aoOIF_Full, "Full")
    choice(aoOIF_Incremental, "Incremental")
}
menu(bufferingALG) {
    choice(bufferingALG_FIFO, "FIFO Buffer")
    choice(bufferingALG_LIFO, "LIFO Buffer")
}
menu(calcoutDOPT) {
    choice(calcoutDOPT_Use_VAL, "Use CALC")
    choice(calcoutDOPT_Use_OVAL, "Use OCAL")
}
menu(calcoutINAV) {
    choice(calcoutINAV_EXT_NC, "Ext PV NC")
    choice(calcoutINAV_EXT, "Ext PV OK")
    choice(calcoutINAV_LOC, "Local PV")
    choice(calcoutINAV_CON, "Constant")
}
menu(calcoutOOPT) {
    choice(calcoutOOPT_Every_Time, "Every Time")
    choice(calcoutOOPT_On_Change, "On Change")
    choice(calcoutOOPT_When_Zero, "When Zero")
    choice(calcoutOOPT_When_Non_zero, "When Non-zero")
    choice(calcoutOOPT_Transition_To_Zero, "Transition To Zero")
    choice(calcoutOOPT_Transition_To_Non_zero, "Transition To Non-zero")
}
menu(compressALG) {
    choice(compressALG_N_to_1_Low_Value, "N to 1 Low Value")
    choice(compressALG_N_to_1_High_Value, "N to 1 High Value")
    choice(compressALG_N_to_1_Average, "N to 1 Average")
    choice(compressALG_Average, "Average")
    choice(compressALG_Circular_Buffer, "Circular Buffer")
    choice(compressALG_N_to_1_Median, "N to 1 Median")
}
menu(dfanoutSELM) {
    choice(dfanoutSELM_All, "All")
    choice(dfanoutSELM_Specified, "Specified")
    choice(dfanoutSELM_Mask, "Mask")
}
```

```
menu(fanoutSELM) {
    choice(fanoutSELM_All, "All")
    choice(fanoutSELM_Specified, "Specified")
    choice(fanoutSELM_Mask, "Mask")
}
menu(histogramCMD) {
    choice(histogramCMD_Read, "Read")
    choice(histogramCMD_Clear, "Clear")
    choice(histogramCMD_Start, "Start")
    choice(histogramCMD_Stop, "Stop")
}
menu(longoutOOPT) {
    choice(longoutOOPT_Every_Time, "Every Time")
    choice(longoutOOPT_On_Change, "On Change")
    choice(longoutOOPT_When_Zero, "When Zero")
    choice(longoutOOPT_When_Non_zero, "When Non-zero")
    choice(longoutOOPT_Transition_To_Zero, "Transition To Zero")
    choice(longoutOOPT_Transition_To_Non_zero, "Transition To Non-zero")
}
menu(menuAlarmSevr) {
    choice(menuAlarmSevrNO_ALARM, "NO_ALARM")
    choice(menuAlarmSevrMINOR, "MINOR")
    choice(menuAlarmSevrMAJOR, "MAJOR")
    choice(menuAlarmSevrINVALID, "INVALID")
}
menu(menuAlarmStat) {
    choice(menuAlarmStatNO_ALARM, "NO_ALARM")
    choice(menuAlarmStatREAD, "READ")
    choice(menuAlarmStatWRITE, "WRITE")
    choice(menuAlarmStatHIHI, "HIHI")
    choice(menuAlarmStatHIGH, "HIGH")
    choice(menuAlarmStatLOLO, "LOLO")
    choice(menuAlarmStatLOW, "LOW")
    choice(menuAlarmStatSTATE, "STATE")
    choice(menuAlarmStatCOS, "COS")
    choice(menuAlarmStatCOMM, "COMM")
    choice(menuAlarmStatTIMEOUT, "TIMEOUT")
    choice(menuAlarmStatHWLIMIT, "HWLIMIT")
    choice(menuAlarmStatCALC, "CALC")
    choice(menuAlarmStatSCAN, "SCAN")
    choice(menuAlarmStatLINK, "LINK")
    choice(menuAlarmStatSOFT, "SOFT")
    choice(menuAlarmStatBAD_SUB, "BAD_SUB")
    choice(menuAlarmStatUDF, "UDF")
    choice(menuAlarmStatDISABLE, "DISABLE")
    choice(menuAlarmStatSIMM, "SIMM")
    choice(menuAlarmStatREAD_ACCESS, "READ_ACCESS")
    choice(menuAlarmStatWRITE_ACCESS, "WRITE_ACCESS")
}
menu(menuConvert) {
    choice(menuConvertNO_CONVERSION, "NO CONVERSION")
    choice(menuConvertSLOPE, "SLOPE")
    choice(menuConvertLINEAR, "LINEAR")
    choice(menuConverttypeKdegF, "typeKdegF")
    choice(menuConverttypeKdegC, "typeKdegC")
    choice(menuConverttypeJdegF, "typeJdegF")
    choice(menuConverttypeJdegC, "typeJdegC")
    choice(menuConverttypeEdegF, "typeEdegF(ixe only)")
    choice(menuConverttypeEdegC, "typeEdegC(ixe only)")
    choice(menuConverttypeTdegF, "typeTdegF")
    choice(menuConverttypeTdegC, "typeTdegC")
    choice(menuConverttypeRdegF, "typeRdegF")
    choice(menuConverttypeRdegC, "typeRdegC")
    choice(menuConverttypeSdegF, "typeSdegF")
    choice(menuConverttypeSdegC, "typeSdegC")
}
menu(menuFtype) {
    choice(menuFtypeSTRING, "STRING")
```

```
    choice(menuFtypeCHAR, "CHAR")
    choice(menuFtypeUCHAR, "UCHAR")
    choice(menuFtypeSHORT, "SHORT")
    choice(menuFtypeUSHORT, "USHORT")
    choice(menuFtypeLONG, "LONG")
    choice(menuFtypeULONG, "ULONG")
    choice(menuFtypeINT64, "INT64")
    choice(menuFtypeUINT64, "UINT64")
    choice(menuFtypeFLOAT, "FLOAT")
    choice(menuFtypeDOUBLE, "DOUBLE")
    choice(menuFtypeENUM, "ENUM")
}
menu(menuIvoa) {
    choice(menuIvoaContinue_normally, "Continue normally")
    choice(menuIvoaDon_t_drive_outputs, "Don't drive outputs")
    choice(menuIvoaSet_output_to_IVOV, "Set output to IVOV")
}
menu(menuOmsl) {
    choice(menuOmslsupervisory, "supervisory")
    choice(menuOmslclosed_loop, "closed_loop")
}
menu(menuPini) {
    choice(menuPiniNO, "NO")
    choice(menuPiniYES, "YES")
    choice(menuPiniRUN, "RUN")
    choice(menuPiniRUNNING, "RUNNING")
    choice(menuPiniPAUSE, "PAUSE")
    choice(menuPiniPAUSED, "PAUSED")
}
menu(menuPost) {
    choice(menuPost_OnChange, "On Change")
    choice(menuPost_Always, "Always")
}
menu(menuPriority) {
    choice(menuPriorityLOW, "LOW")
    choice(menuPriorityMEDIUM, "MEDIUM")
    choice(menuPriorityHIGH, "HIGH")
}
menu(menuScan) {
    choice(menuScanPassive, "Passive")
    choice(menuScanEvent, "Event")
    choice(menuScanI_O_Intr, "I/O Intr")
    choice(menuScan10_second, "10 second")
    choice(menuScan5_second, "5 second")
    choice(menuScan2_second, "2 second")
    choice(menuScan1_second, "1 second")
    choice(menuScan_5_second, ".5 second")
    choice(menuScan_2_second, ".2 second")
    choice(menuScan_1_second, ".1 second")
}
menu(menuSimm) {
    choice(menuSimmNO, "NO")
    choice(menuSimmYES, "YES")
    choice(menuSimmRAW, "RAW")
}
menu(menuYesNo) {
    choice(menuYesNoNO, "NO")
    choice(menuYesNoYES, "YES")
}
menu(selSELM) {
    choice(selSELM_Specified, "Specified")
    choice(selSELM_High_Signal, "High Signal")
    choice(selSELM_Low_Signal, "Low Signal")
    choice(selSELM_Median_Signal, "Median Signal")
}
menu(seqSELM) {
    choice(seqSELM_All, "All")
    choice(seqSELM_Specified, "Specified")
```

```
193     choice(seqSELM_Mask, "Mask")
194 }
195 menu(stringinPOST) {
196     choice(stringinPOST_OnChange, "On Change")
197     choice(stringinPOST_Always, "Always")
198 }
199 menu(stringoutPOST) {
200     choice(stringoutPOST_OnChange, "On Change")
201     choice(stringoutPOST_Always, "Always")
202 }
203 menu(waveformPOST) {
204     choice(waveformPOST_Always, "Always")
205     choice(waveformPOST_OnChange, "On Change")
206 }
207 recordtype(aSub) {
208     %#include "epicsTypes.h"
209     %#include "link.h"
210     %#include "epicsMutex.h"
211     %#include "ellLib.h"
212     %#include "devSup.h"
213     %#include "epicsTime.h"
214     %struct aSubRecord;
215     field(NAME, DBF_STRING) {
216         size(61)
217         special(SPC_NOMOD)
218         prompt("Record Name")
219     }
220     field(DESC, DBF_STRING) {
221         size(41)
222         promptgroup("10 - Common")
223         prompt("Descriptor")
224     }
225     field(ASG, DBF_STRING) {
226         size(29)
227         promptgroup("10 - Common")
228         special(SPC_AS)
229         prompt("Access Security Group")
230     }
231     field(SCAN, DBF_MENU) {
232         promptgroup("20 - Scan")
233         interest(1)
234         menu(menuScan)
235         special(SPC_SCAN)
236         prompt("Scan Mechanism")
237     }
238     field(PINI, DBF_MENU) {
239         promptgroup("20 - Scan")
240         interest(1)
241         menu(menuPini)
242         prompt("Process at iocInit")
243     }
244     field(PHAS, DBF_SHORT) {
245         promptgroup("20 - Scan")
246         interest(1)
247         special(SPC_SCAN)
248         prompt("Scan Phase")
249     }
250     field(EVNT, DBF_STRING) {
251         size(40)
252         promptgroup("20 - Scan")
253         interest(1)
254         special(SPC_SCAN)
255         prompt("Event Name")
256     }
257     field(TSE, DBF_SHORT) {
258         promptgroup("20 - Scan")
259         interest(1)
260         prompt("Time Stamp Event")
```

```
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
```

```
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Reprocess ")
    }
    field(ASP, DBF_NOACCESS) {
        interest(4)
        extra("struct asgMember    *asp")
        special(SPC_NOMOD)
        prompt("Access Security Pvt")
    }
    field(PPN, DBF_NOACCESS) {
        interest(4)
        extra("struct processNotify *ppn")
        special(SPC_NOMOD)
        prompt("pprocessNotify")
```

```
397     }
398     field(PPNR, DBF_NOACCESS) {
399         interest(4)
400         extra("struct processNotifyRecord *ppnr")
401         special(SPC_NOMOD)
402         prompt("pprocessNotifyRecord")
403     }
404     field(SPVT, DBF_NOACCESS) {
405         interest(4)
406         extra("struct scan_element *spvt")
407         special(SPC_NOMOD)
408         prompt("Scan Private")
409     }
410     field(RSET, DBF_NOACCESS) {
411         interest(4)
412         extra("struct typed_rset   *rset")
413         special(SPC_NOMOD)
414         prompt("Address of RSET")
415     }
416     field(DSET, DBF_NOACCESS) {
417         interest(4)
418         extra("unambiguous_dset    *dset")
419         special(SPC_NOMOD)
420         prompt("DSET address")
421     }
422     field(DPVT, DBF_NOACCESS) {
423         interest(4)
424         extra("void                *dpvt")
425         special(SPC_NOMOD)
426         prompt("Device Private")
427     }
428     field(RDES, DBF_NOACCESS) {
429         interest(4)
430         extra("struct dbRecordType *rdes")
431         special(SPC_NOMOD)
432         prompt("Address of dbRecordType")
433     }
434     field(LSET, DBF_NOACCESS) {
435         interest(4)
436         extra("struct lockRecord   *lset")
437         special(SPC_NOMOD)
438         prompt("Lock Set")
439     }
440     field(PRIO, DBF_MENU) {
441         promptgroup("20 - Scan")
442         interest(1)
443         menu(menuPriority)
444         special(SPC_SCAN)
445         prompt("Scheduling Priority")
446     }
447     field(TPRO, DBF_UCHAR) {
448         prompt("Trace Processing")
449     }
450     field(BKPT, DBF_NOACCESS) {
451         interest(1)
452         extra("epicsUInt8          bkpt")
453         special(SPC_NOMOD)
454         prompt("Break Point")
455     }
456     field(UDF, DBF_UCHAR) {
457         promptgroup("10 - Common")
458         interest(1)
459         initial("1")
460         pp(TRUE)
461         prompt("Undefined")
462     }
463     field(UDFS, DBF_MENU) {
464         promptgroup("70 - Alarm")
```

```
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_LONG) {
        asl(ASL0)
        prompt("Subr. return value")
    }
    field(OVAL, DBF_LONG) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Old return value")
    }
    field(INAM, DBF_STRING) {
        size(41)
        promptgroup("30 - Action")
        interest(1)
        special(SPC_NOMOD)
        prompt("Initialize Subr. Name")
    }
    field(LFLG, DBF_MENU) {
        promptgroup("30 - Action")
        interest(1)
        menu(aSubLFLG)
        prompt("Subr. Input Enable")
    }
    field(SUBL, DBF_INLINK) {
        promptgroup("30 - Action")
        interest(1)
        special(SPC_NOMOD)
        prompt("Subroutine Name Link")
    }
    field(SNAM, DBF_STRING) {
        size(41)
        promptgroup("30 - Action")
        interest(1)
        special(SPC_MOD)
        prompt("Process Subr. Name")
    }
    field(ONAM, DBF_STRING) {
        size(41)
        promptgroup("30 - Action")
        interest(3)
        special(SPC_NOMOD)
        prompt("Old Subr. Name")
    }
    field(SADR, DBF_NOACCESS) {
        interest(2)
        extra("long (*sadr)(struct aSubRecord *)")
        special(SPC_NOMOD)
        prompt("Subroutine Address")
```

```
533     }
534     field(CADR, DBF_NOACCESS) {
535         interest(2)
536         extra("void (*cadr)(struct aSubRecord *)")
537         special(SPC_NOMOD)
538         prompt("Subroutine Cleanup Address")
539     }
540     field(BRSV, DBF_MENU) {
541         promptgroup("70 - Alarm")
542         interest(1)
543         menu(menuAlarmSevr)
544         pp(TRUE)
545         prompt("Bad Return Severity")
546     }
547     field(PREC, DBF_SHORT) {
548         promptgroup("80 - Display")
549         interest(1)
550         prop(YES)
551         prompt("Display Precision")
552     }
553     field(EFLG, DBF_MENU) {
554         promptgroup("50 - Output")
555         interest(1)
556         initial("1")
557         menu(aSubEFLG)
558         prompt("Output Event Flag")
559     }
560     field(INPA, DBF_INLINK) {
561         promptgroup("41 - Input A-G")
562         interest(1)
563         prompt("Input Link A")
564     }
565     field(INPB, DBF_INLINK) {
566         promptgroup("41 - Input A-G")
567         interest(1)
568         prompt("Input Link B")
569     }
570     field(INPC, DBF_INLINK) {
571         promptgroup("41 - Input A-G")
572         interest(1)
573         prompt("Input Link C")
574     }
575     field(INPD, DBF_INLINK) {
576         promptgroup("41 - Input A-G")
577         interest(1)
578         prompt("Input Link D")
579     }
580     field(INPE, DBF_INLINK) {
581         promptgroup("41 - Input A-G")
582         interest(1)
583         prompt("Input Link E")
584     }
585     field(INPF, DBF_INLINK) {
586         promptgroup("41 - Input A-G")
587         interest(1)
588         prompt("Input Link F")
589     }
590     field(INPG, DBF_INLINK) {
591         promptgroup("41 - Input A-G")
592         interest(1)
593         prompt("Input Link G")
594     }
595     field(INPH, DBF_INLINK) {
596         promptgroup("42 - Input H-N")
597         interest(1)
598         prompt("Input Link H")
599     }
600     field(INPI, DBF_INLINK) {
```

```
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link I")
    }
    field(INPJ, DBF_INLINK) {
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link J")
    }
    field(INPK, DBF_INLINK) {
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link K")
    }
    field(INPL, DBF_INLINK) {
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link L")
    }
    field(INPM, DBF_INLINK) {
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link M")
    }
    field(INPN, DBF_INLINK) {
        promptgroup("42 - Input H-N")
        interest(1)
        prompt("Input Link N")
    }
    field(INPO, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link O")
    }
    field(INPP, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link P")
    }
    field(INPQ, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link Q")
    }
    field(INPR, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link R")
    }
    field(INPS, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link S")
    }
    field(INPT, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link T")
    }
    field(INPU, DBF_INLINK) {
        promptgroup("43 - Input O-U")
        interest(1)
        prompt("Input Link U")
    }
    field(A, DBF_NOACCESS) {
        interest(2)
        extra("void *a")
        asl(ASL0)
```

```
669         special(SPC_DBADDR)
670         prompt("Input value A")
671     }
672     field(B, DBF_NOACCESS) {
673         interest(2)
674         extra("void *b")
675         asl(ASL0)
676         special(SPC_DBADDR)
677         prompt("Input value B")
678     }
679     field(C, DBF_NOACCESS) {
680         interest(2)
681         extra("void *c")
682         asl(ASL0)
683         special(SPC_DBADDR)
684         prompt("Input value C")
685     }
686     field(D, DBF_NOACCESS) {
687         interest(2)
688         extra("void *d")
689         asl(ASL0)
690         special(SPC_DBADDR)
691         prompt("Input value D")
692     }
693     field(E, DBF_NOACCESS) {
694         interest(2)
695         extra("void *e")
696         asl(ASL0)
697         special(SPC_DBADDR)
698         prompt("Input value E")
699     }
700     field(F, DBF_NOACCESS) {
701         interest(2)
702         extra("void *f")
703         asl(ASL0)
704         special(SPC_DBADDR)
705         prompt("Input value F")
706     }
707     field(G, DBF_NOACCESS) {
708         interest(2)
709         extra("void *g")
710         asl(ASL0)
711         special(SPC_DBADDR)
712         prompt("Input value G")
713     }
714     field(H, DBF_NOACCESS) {
715         interest(2)
716         extra("void *h")
717         asl(ASL0)
718         special(SPC_DBADDR)
719         prompt("Input value H")
720     }
721     field(I, DBF_NOACCESS) {
722         interest(2)
723         extra("void *i")
724         asl(ASL0)
725         special(SPC_DBADDR)
726         prompt("Input value I")
727     }
728     field(J, DBF_NOACCESS) {
729         interest(2)
730         extra("void *j")
731         asl(ASL0)
732         special(SPC_DBADDR)
733         prompt("Input value J")
734     }
735     field(K, DBF_NOACCESS) {
736         interest(2)
```

```
        extra("void *k")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value K")
    }
    field(L, DBF_NOACCESS) {
        interest(2)
        extra("void *l")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value L")
    }
    field(M, DBF_NOACCESS) {
        interest(2)
        extra("void *m")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value M")
    }
    field(N, DBF_NOACCESS) {
        interest(2)
        extra("void *n")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value N")
    }
    field(O, DBF_NOACCESS) {
        interest(2)
        extra("void *o")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value O")
    }
    field(P, DBF_NOACCESS) {
        interest(2)
        extra("void *p")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value P")
    }
    field(Q, DBF_NOACCESS) {
        interest(2)
        extra("void *q")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value Q")
    }
    field(R, DBF_NOACCESS) {
        interest(2)
        extra("void *r")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value R")
    }
    field(S, DBF_NOACCESS) {
        interest(2)
        extra("void *s")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value S")
    }
    field(T, DBF_NOACCESS) {
        interest(2)
        extra("void *t")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value T")
    }
```

```
    field(U, DBF_NOACCESS) {
        interest(2)
        extra("void *u")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Input value U")
    }
    field(FTA, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of A")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTB, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of B")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTC, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of C")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTD, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of D")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTE, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of E")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTF, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of F")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTG, DBF_MENU) {
        promptgroup("41 - Input A-G")
        menu(menuFtype)
        prompt("Type of G")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTH, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of H")
        interest(1)
```

```
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTI, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of I")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTJ, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of J")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTK, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of K")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTL, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of L")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTM, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of M")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTN, DBF_MENU) {
        promptgroup("42 - Input H-N")
        menu(menuFtype)
        prompt("Type of N")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTO, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of O")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTP, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of P")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTQ, DBF_MENU) {
```

```
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of Q")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTR, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of R")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTS, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of S")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTT, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of T")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTU, DBF_MENU) {
        promptgroup("43 - Input O-U")
        menu(menuFtype)
        prompt("Type of U")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(NOA, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in A")
    }
    field(NOB, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in B")
    }
    field(NOC, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in C")
    }
    field(NOD, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in D")
    }
    field(NOE, DBF_ULONG) {
```

```
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in E")
    }
    field(NOF, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in F")
    }
    field(NOG, DBF_ULONG) {
        promptgroup("41 - Input A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in G")
    }
    field(NOH, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in H")
    }
    field(NOI, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in I")
    }
    field(NOJ, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in J")
    }
    field(NOK, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in K")
    }
    field(NOL, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in L")
    }
    field(NOM, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in M")
    }
    field(NON, DBF_ULONG) {
        promptgroup("42 - Input H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in N")
```

```
    }
    field(NOO, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in O")
    }
    field(NOP, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in P")
    }
    field(NOQ, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in Q")
    }
    field(NOR, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in R")
    }
    field(NOS, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in S")
    }
    field(NOT, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in T")
    }
    field(NOU, DBF_ULONG) {
        promptgroup("43 - Input O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in U")
    }
    field(NEA, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in A")
    }
    field(NEB, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in B")
    }
    field(NEC, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in C")
    }
```

```
    field(NED, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in D")
    }
    field(NEE, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in E")
    }
    field(NEF, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in F")
    }
    field(NEG, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in G")
    }
    field(NEH, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in H")
    }
    field(NEI, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in I")
    }
    field(NEJ, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in J")
    }
    field(NEK, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in K")
    }
    field(NEL, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in L")
    }
    field(NEM, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in M")
    }
    field(NEN, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in N")
    }
    field(NEO, DBF_ULONG) {
        interest(3)
```

```
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in O")
    }
    field(NEP, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in P")
    }
    field(NEQ, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in Q")
    }
    field(NER, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in R")
    }
    field(NES, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in S")
    }
    field(NET, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in T")
    }
    field(NEU, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in U")
    }
    field(OUTA, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
        prompt("Output Link A")
    }
    field(OUTB, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
        prompt("Output Link B")
    }
    field(OUTC, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
        prompt("Output Link C")
    }
    field(OUTD, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
        prompt("Output Link D")
    }
    field(OUTE, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
        prompt("Output Link E")
    }
    field(OUTF, DBF_OUTLINK) {
        promptgroup("51 - Output A-G")
        interest(1)
```

```
1281        prompt("Output Link F")
1282    }
1283    field(OUTG, DBF_OUTLINK) {
1284        promptgroup("51 - Output A-G")
1285        interest(1)
1286        prompt("Output Link G")
1287    }
1288    field(OUTH, DBF_OUTLINK) {
1289        promptgroup("52 - Output H-N")
1290        interest(1)
1291        prompt("Output Link H")
1292    }
1293    field(OUTI, DBF_OUTLINK) {
1294        promptgroup("52 - Output H-N")
1295        interest(1)
1296        prompt("Output Link I")
1297    }
1298    field(OUTJ, DBF_OUTLINK) {
1299        promptgroup("52 - Output H-N")
1300        interest(1)
1301        prompt("Output Link J")
1302    }
1303    field(OUTK, DBF_OUTLINK) {
1304        promptgroup("52 - Output H-N")
1305        interest(1)
1306        prompt("Output Link K")
1307    }
1308    field(OUTL, DBF_OUTLINK) {
1309        promptgroup("52 - Output H-N")
1310        interest(1)
1311        prompt("Output Link L")
1312    }
1313    field(OUTM, DBF_OUTLINK) {
1314        promptgroup("52 - Output H-N")
1315        interest(1)
1316        prompt("Output Link M")
1317    }
1318    field(OUTN, DBF_OUTLINK) {
1319        promptgroup("52 - Output H-N")
1320        interest(1)
1321        prompt("Output Link N")
1322    }
1323    field(OUTO, DBF_OUTLINK) {
1324        promptgroup("53 - Output O-U")
1325        interest(1)
1326        prompt("Output Link O")
1327    }
1328    field(OUTP, DBF_OUTLINK) {
1329        promptgroup("53 - Output O-U")
1330        interest(1)
1331        prompt("Output Link P")
1332    }
1333    field(OUTQ, DBF_OUTLINK) {
1334        promptgroup("53 - Output O-U")
1335        interest(1)
1336        prompt("Output Link Q")
1337    }
1338    field(OUTR, DBF_OUTLINK) {
1339        promptgroup("53 - Output O-U")
1340        interest(1)
1341        prompt("Output Link R")
1342    }
1343    field(OUTS, DBF_OUTLINK) {
1344        promptgroup("53 - Output O-U")
1345        interest(1)
1346        prompt("Output Link S")
1347    }
1348    field(OUTT, DBF_OUTLINK) {
```

```
1349        promptgroup("53 - Output O-U")
1350        interest(1)
1351        prompt("Output Link T")
1352    }
1353    field(OUTU, DBF_OUTLINK) {
1354        promptgroup("53 - Output O-U")
1355        interest(1)
1356        prompt("Output Link U")
1357    }
1358    field(VALA, DBF_NOACCESS) {
1359        interest(2)
1360        extra("void *vala")
1361        asl(ASL0)
1362        special(SPC_DBADDR)
1363        prompt("Output value A")
1364    }
1365    field(VALB, DBF_NOACCESS) {
1366        interest(2)
1367        extra("void *valb")
1368        asl(ASL0)
1369        special(SPC_DBADDR)
1370        prompt("Output value B")
1371    }
1372    field(VALC, DBF_NOACCESS) {
1373        interest(2)
1374        extra("void *valc")
1375        asl(ASL0)
1376        special(SPC_DBADDR)
1377        prompt("Output value C")
1378    }
1379    field(VALD, DBF_NOACCESS) {
1380        interest(2)
1381        extra("void *vald")
1382        asl(ASL0)
1383        special(SPC_DBADDR)
1384        prompt("Output value D")
1385    }
1386    field(VALE, DBF_NOACCESS) {
1387        interest(2)
1388        extra("void *vale")
1389        asl(ASL0)
1390        special(SPC_DBADDR)
1391        prompt("Output value E")
1392    }
1393    field(VALF, DBF_NOACCESS) {
1394        interest(2)
1395        extra("void *valf")
1396        asl(ASL0)
1397        special(SPC_DBADDR)
1398        prompt("Output value F")
1399    }
1400    field(VALG, DBF_NOACCESS) {
1401        interest(2)
1402        extra("void *valg")
1403        asl(ASL0)
1404        special(SPC_DBADDR)
1405        prompt("Output value G")
1406    }
1407    field(VALH, DBF_NOACCESS) {
1408        interest(2)
1409        extra("void *valh")
1410        asl(ASL0)
1411        special(SPC_DBADDR)
1412        prompt("Output value H")
1413    }
1414    field(VALI, DBF_NOACCESS) {
1415        interest(2)
1416        extra("void *vali")
```

```
1417        asl(ASL0)
1418        special(SPC_DBADDR)
1419        prompt("Output value I")
1420    }
1421    field(VALJ, DBF_NOACCESS) {
1422        interest(2)
1423        extra("void *valj")
1424        asl(ASL0)
1425        special(SPC_DBADDR)
1426        prompt("Output value J")
1427    }
1428    field(VALK, DBF_NOACCESS) {
1429        interest(2)
1430        extra("void *valk")
1431        asl(ASL0)
1432        special(SPC_DBADDR)
1433        prompt("Output value K")
1434    }
1435    field(VALL, DBF_NOACCESS) {
1436        interest(2)
1437        extra("void *vall")
1438        asl(ASL0)
1439        special(SPC_DBADDR)
1440        prompt("Output value L")
1441    }
1442    field(VALM, DBF_NOACCESS) {
1443        interest(2)
1444        extra("void *valm")
1445        asl(ASL0)
1446        special(SPC_DBADDR)
1447        prompt("Output value M")
1448    }
1449    field(VALN, DBF_NOACCESS) {
1450        interest(2)
1451        extra("void *valn")
1452        asl(ASL0)
1453        special(SPC_DBADDR)
1454        prompt("Output value N")
1455    }
1456    field(VALO, DBF_NOACCESS) {
1457        interest(2)
1458        extra("void *valo")
1459        asl(ASL0)
1460        special(SPC_DBADDR)
1461        prompt("Output value O")
1462    }
1463    field(VALP, DBF_NOACCESS) {
1464        interest(2)
1465        extra("void *valp")
1466        asl(ASL0)
1467        special(SPC_DBADDR)
1468        prompt("Output value P")
1469    }
1470    field(VALQ, DBF_NOACCESS) {
1471        interest(2)
1472        extra("void *valq")
1473        asl(ASL0)
1474        special(SPC_DBADDR)
1475        prompt("Output value Q")
1476    }
1477    field(VALR, DBF_NOACCESS) {
1478        interest(2)
1479        extra("void *valr")
1480        asl(ASL0)
1481        special(SPC_DBADDR)
1482        prompt("Output value R")
1483    }
1484    field(VALS, DBF_NOACCESS) {
```

```
        interest(2)
        extra("void *vals")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Output value S")
    }
    field(VALT, DBF_NOACCESS) {
        interest(2)
        extra("void *valt")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Output value T")
    }
    field(VALU, DBF_NOACCESS) {
        interest(2)
        extra("void *valu")
        asl(ASL0)
        special(SPC_DBADDR)
        prompt("Output value U")
    }
    field(OVLA, DBF_NOACCESS) {
        interest(4)
        extra("void *ovla")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output A")
    }
    field(OVLB, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlb")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output B")
    }
    field(OVLC, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlc")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output C")
    }
    field(OVLD, DBF_NOACCESS) {
        interest(4)
        extra("void *ovld")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output D")
    }
    field(OVLE, DBF_NOACCESS) {
        interest(4)
        extra("void *ovle")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output E")
    }
    field(OVLF, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlf")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output F")
    }
    field(OVLG, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlg")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output G")
```

```
1553        }
1554        field(OVLH, DBF_NOACCESS) {
1555            interest(4)
1556            extra("void *ovlh")
1557            asl(ASL0)
1558            special(SPC_NOMOD)
1559            prompt("Old Output H")
1560        }
1561        field(OVLI, DBF_NOACCESS) {
1562            interest(4)
1563            extra("void *ovli")
1564            asl(ASL0)
1565            special(SPC_NOMOD)
1566            prompt("Old Output I")
1567        }
1568        field(OVLJ, DBF_NOACCESS) {
1569            interest(4)
1570            extra("void *ovlj")
1571            asl(ASL0)
1572            special(SPC_NOMOD)
1573            prompt("Old Output J")
1574        }
1575        field(OVLK, DBF_NOACCESS) {
1576            interest(4)
1577            extra("void *ovlk")
1578            asl(ASL0)
1579            special(SPC_NOMOD)
1580            prompt("Old Output K")
1581        }
1582        field(OVLL, DBF_NOACCESS) {
1583            interest(4)
1584            extra("void *ovll")
1585            asl(ASL0)
1586            special(SPC_NOMOD)
1587            prompt("Old Output L")
1588        }
1589        field(OVLM, DBF_NOACCESS) {
1590            interest(4)
1591            extra("void *ovlm")
1592            asl(ASL0)
1593            special(SPC_NOMOD)
1594            prompt("Old Output M")
1595        }
1596        field(OVLN, DBF_NOACCESS) {
1597            interest(4)
1598            extra("void *ovln")
1599            asl(ASL0)
1600            special(SPC_NOMOD)
1601            prompt("Old Output N")
1602        }
1603        field(OVLO, DBF_NOACCESS) {
1604            interest(4)
1605            extra("void *ovlo")
1606            asl(ASL0)
1607            special(SPC_NOMOD)
1608            prompt("Old Output O")
1609        }
1610        field(OVLP, DBF_NOACCESS) {
1611            interest(4)
1612            extra("void *ovlp")
1613            asl(ASL0)
1614            special(SPC_NOMOD)
1615            prompt("Old Output P")
1616        }
1617        field(OVLQ, DBF_NOACCESS) {
1618            interest(4)
1619            extra("void *ovlq")
1620            asl(ASL0)
```

```
        special(SPC_NOMOD)
        prompt("Old Output Q")
    }
    field(OVLR, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlr")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output R")
    }
    field(OVLS, DBF_NOACCESS) {
        interest(4)
        extra("void *ovls")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output S")
    }
    field(OVLT, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlt")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output T")
    }
    field(OVLU, DBF_NOACCESS) {
        interest(4)
        extra("void *ovlu")
        asl(ASL0)
        special(SPC_NOMOD)
        prompt("Old Output U")
    }
    field(FTVA, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALA")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVB, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALB")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVC, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALC")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVD, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALD")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVE, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALE")
        interest(1)
```

```
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVF, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALF")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVG, DBF_MENU) {
        promptgroup("51 - Output A-G")
        menu(menuFtype)
        prompt("Type of VALG")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVH, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALH")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVI, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALI")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVJ, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALJ")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVK, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALK")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVL, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALL")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVM, DBF_MENU) {
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALM")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVN, DBF_MENU) {
```

```
        promptgroup("52 - Output H-N")
        menu(menuFtype)
        prompt("Type of VALN")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVO, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALO")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVP, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALP")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVQ, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALQ")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVR, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALR")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVS, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALS")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVT, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALT")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(FTVU, DBF_MENU) {
        promptgroup("53 - Output O-U")
        menu(menuFtype)
        prompt("Type of VALU")
        interest(1)
        initial("DOUBLE")
        special(SPC_NOMOD)
    }
    field(NOVA, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
```

```
        prompt("Max. elements in VALA")
    }
    field(NOVB, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALB")
    }
    field(NOVC, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALC")
    }
    field(NOVD, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALD")
    }
    field(NOVE, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALE")
    }
    field(NOVF, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALF")
    }
    field(NOVG, DBF_ULONG) {
        promptgroup("51 - Output A-G")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALG")
    }
    field(NOVH, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VAlH")
    }
    field(NOVI, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALI")
    }
    field(NOVJ, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALJ")
    }
    field(NOVK, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
```

```
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALK")
    }
    field(NOVL, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALL")
    }
    field(NOVM, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALM")
    }
    field(NOVN, DBF_ULONG) {
        promptgroup("52 - Output H-N")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALN")
    }
    field(NOVO, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALO")
    }
    field(NOVP, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALP")
    }
    field(NOVQ, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALQ")
    }
    field(NOVR, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALR")
    }
    field(NOVS, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALS")
    }
    field(NOVT, DBF_ULONG) {
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALT")
    }
    field(NOVU, DBF_ULONG) {
```

```
        promptgroup("53 - Output O-U")
        interest(1)
        initial("1")
        special(SPC_NOMOD)
        prompt("Max. elements in VALU")
    }
    field(NEVA, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALA")
    }
    field(NEVB, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALB")
    }
    field(NEVC, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALC")
    }
    field(NEVD, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALD")
    }
    field(NEVE, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALE")
    }
    field(NEVF, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALF")
    }
    field(NEVG, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALG")
    }
    field(NEVH, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VAlH")
    }
    field(NEVI, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALI")
    }
    field(NEVJ, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALJ")
    }
    field(NEVK, DBF_ULONG) {
        interest(3)
```

```
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALK")
    }
    field(NEVL, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALL")
    }
    field(NEVM, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALM")
    }
    field(NEVN, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALN")
    }
    field(NEVO, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALO")
    }
    field(NEVP, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALP")
    }
    field(NEVQ, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALQ")
    }
    field(NEVR, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALR")
    }
    field(NEVS, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALS")
    }
    field(NEVT, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALT")
    }
    field(NEVU, DBF_ULONG) {
        interest(3)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VALU")
    }
    field(ONVA, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
```

```
        prompt("Num. elements in OVLA")
    }
    field(ONVB, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLB")
    }
    field(ONVC, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLC")
    }
    field(ONVD, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLD")
    }
    field(ONVE, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLE")
    }
    field(ONVF, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLF")
    }
    field(ONVG, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLG")
    }
    field(ONVH, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in VAlH")
    }
    field(ONVI, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLI")
    }
    field(ONVJ, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLJ")
    }
    field(ONVK, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLK")
    }
    field(ONVL, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLL")
    }
```

```
    field(ONVM, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLM")
    }
    field(ONVN, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLN")
    }
    field(ONVO, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLO")
    }
    field(ONVP, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLP")
    }
    field(ONVQ, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLQ")
    }
    field(ONVR, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLR")
    }
    field(ONVS, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLS")
    }
    field(ONVT, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLT")
    }
    field(ONVU, DBF_ULONG) {
        interest(4)
        initial("1")
        special(SPC_NOMOD)
        prompt("Num. elements in OVLU")
    }
}
recordtype(aai) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct aaiRecord;
    %typedef struct aaidset {
    %    dset common; /*init_record returns: (-1,0,AAI_DEVINIT_PASS1)=>(failure,success,callback)*/
    %    long (*read_aai)(struct aaiRecord *prec); /*returns: (-1,0)=>(failure,success)*/
```

```
2233    %} aaidset;
2234    %#define HAS_aaidset
2235    %#define AAI_POSTS_NORD
2236    %#define AAI_DEVINIT_PASS1 2
2237    %
2238    %#include "callback.h"
2239    field(NAME, DBF_STRING) {
2240        size(61)
2241        special(SPC_NOMOD)
2242        prompt("Record Name")
2243    }
2244    field(DESC, DBF_STRING) {
2245        size(41)
2246        promptgroup("10 - Common")
2247        prompt("Descriptor")
2248    }
2249    field(ASG, DBF_STRING) {
2250        size(29)
2251        promptgroup("10 - Common")
2252        special(SPC_AS)
2253        prompt("Access Security Group")
2254    }
2255    field(SCAN, DBF_MENU) {
2256        promptgroup("20 - Scan")
2257        interest(1)
2258        menu(menuScan)
2259        special(SPC_SCAN)
2260        prompt("Scan Mechanism")
2261    }
2262    field(PINI, DBF_MENU) {
2263        promptgroup("20 - Scan")
2264        interest(1)
2265        menu(menuPini)
2266        prompt("Process at iocInit")
2267    }
2268    field(PHAS, DBF_SHORT) {
2269        promptgroup("20 - Scan")
2270        interest(1)
2271        special(SPC_SCAN)
2272        prompt("Scan Phase")
2273    }
2274    field(EVNT, DBF_STRING) {
2275        size(40)
2276        promptgroup("20 - Scan")
2277        interest(1)
2278        special(SPC_SCAN)
2279        prompt("Event Name")
2280    }
2281    field(TSE, DBF_SHORT) {
2282        promptgroup("20 - Scan")
2283        interest(1)
2284        prompt("Time Stamp Event")
2285    }
2286    field(TSEL, DBF_INLINK) {
2287        promptgroup("20 - Scan")
2288        interest(1)
2289        prompt("Time Stamp Link")
2290    }
2291    field(DTYP, DBF_DEVICE) {
2292        promptgroup("10 - Common")
2293        interest(1)
2294        prompt("Device Type")
2295    }
2296    field(DISV, DBF_SHORT) {
2297        promptgroup("20 - Scan")
2298        initial("1")
2299        prompt("Disable Value")
2300    }
```

```
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
```

```
2369        }
2370        field(ACKS, DBF_MENU) {
2371            interest(2)
2372            menu(menuAlarmSevr)
2373            special(SPC_NOMOD)
2374            prompt("Alarm Ack Severity")
2375        }
2376        field(ACKT, DBF_MENU) {
2377            promptgroup("70 - Alarm")
2378            interest(2)
2379            initial("YES")
2380            menu(menuYesNo)
2381            special(SPC_NOMOD)
2382            prompt("Alarm Ack Transient")
2383        }
2384        field(DISS, DBF_MENU) {
2385            promptgroup("70 - Alarm")
2386            interest(1)
2387            menu(menuAlarmSevr)
2388            prompt("Disable Alarm Sevrty")
2389        }
2390        field(LCNT, DBF_UCHAR) {
2391            interest(2)
2392            special(SPC_NOMOD)
2393            prompt("Lock Count")
2394        }
2395        field(PACT, DBF_UCHAR) {
2396            interest(1)
2397            special(SPC_NOMOD)
2398            prompt("Record active")
2399        }
2400        field(PUTF, DBF_UCHAR) {
2401            interest(1)
2402            special(SPC_NOMOD)
2403            prompt("dbPutField process")
2404        }
2405        field(RPRO, DBF_UCHAR) {
2406            interest(1)
2407            special(SPC_NOMOD)
2408            prompt("Reprocess ")
2409        }
2410        field(ASP, DBF_NOACCESS) {
2411            interest(4)
2412            extra("struct asgMember    *asp")
2413            special(SPC_NOMOD)
2414            prompt("Access Security Pvt")
2415        }
2416        field(PPN, DBF_NOACCESS) {
2417            interest(4)
2418            extra("struct processNotify *ppn")
2419            special(SPC_NOMOD)
2420            prompt("pprocessNotify")
2421        }
2422        field(PPNR, DBF_NOACCESS) {
2423            interest(4)
2424            extra("struct processNotifyRecord *ppnr")
2425            special(SPC_NOMOD)
2426            prompt("pprocessNotifyRecord")
2427        }
2428        field(SPVT, DBF_NOACCESS) {
2429            interest(4)
2430            extra("struct scan_element *spvt")
2431            special(SPC_NOMOD)
2432            prompt("Scan Private")
2433        }
2434        field(RSET, DBF_NOACCESS) {
2435            interest(4)
2436            extra("struct typed_rset   *rset")
```

```
2437        special(SPC_NOMOD)
2438        prompt("Address of RSET")
2439    }
2440    field(DSET, DBF_NOACCESS) {
2441        interest(4)
2442        extra("unambiguous_dset    *dset")
2443        special(SPC_NOMOD)
2444        prompt("DSET address")
2445    }
2446    field(DPVT, DBF_NOACCESS) {
2447        interest(4)
2448        extra("void                *dpvt")
2449        special(SPC_NOMOD)
2450        prompt("Device Private")
2451    }
2452    field(RDES, DBF_NOACCESS) {
2453        interest(4)
2454        extra("struct dbRecordType *rdes")
2455        special(SPC_NOMOD)
2456        prompt("Address of dbRecordType")
2457    }
2458    field(LSET, DBF_NOACCESS) {
2459        interest(4)
2460        extra("struct lockRecord   *lset")
2461        special(SPC_NOMOD)
2462        prompt("Lock Set")
2463    }
2464    field(PRIO, DBF_MENU) {
2465        promptgroup("20 - Scan")
2466        interest(1)
2467        menu(menuPriority)
2468        special(SPC_SCAN)
2469        prompt("Scheduling Priority")
2470    }
2471    field(TPRO, DBF_UCHAR) {
2472        prompt("Trace Processing")
2473    }
2474    field(BKPT, DBF_NOACCESS) {
2475        interest(1)
2476        extra("epicsUInt8          bkpt")
2477        special(SPC_NOMOD)
2478        prompt("Break Point")
2479    }
2480    field(UDF, DBF_UCHAR) {
2481        promptgroup("10 - Common")
2482        interest(1)
2483        initial("1")
2484        pp(TRUE)
2485        prompt("Undefined")
2486    }
2487    field(UDFS, DBF_MENU) {
2488        promptgroup("70 - Alarm")
2489        interest(1)
2490        initial("INVALID")
2491        menu(menuAlarmSevr)
2492        prompt("Undefined Alarm Sevrty")
2493    }
2494    field(TIME, DBF_NOACCESS) {
2495        interest(2)
2496        extra("epicsTimeStamp      time")
2497        special(SPC_NOMOD)
2498        prompt("Time")
2499    }
2500    field(UTAG, DBF_UINT64) {
2501        interest(3)
2502        special(SPC_NOMOD)
2503        prompt("Time Tag")
2504    }
```

```
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_NOACCESS) {
        extra("void *       val")
        asl(ASL0)
        pp(TRUE)
        special(SPC_DBADDR)
        prompt("Value")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(INP, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input Specification")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(NELM, DBF_ULONG) {
        promptgroup("30 - Action")
        prompt("Number of Elements")
        prop(YES)
        interest(1)
        initial("1")
        special(SPC_NOMOD)
    }
    field(FTVL, DBF_MENU) {
        promptgroup("30 - Action")
        interest(1)
        menu(menuFtype)
        special(SPC_NOMOD)
        prompt("Field Type of Value")
    }
    field(NORD, DBF_ULONG) {
        special(SPC_NOMOD)
        prompt("Number elements read")
    }
    field(BPTR, DBF_NOACCESS) {
        interest(4)
        extra("void *       bptr")
        special(SPC_NOMOD)
        prompt("Buffer Pointer")
    }
    field(SIML, DBF_INLINK) {
```

```
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Mode Link")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuYesNo)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Input Link")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(MPST, DBF_MENU) {
        promptgroup("80 - Display")
        interest(1)
        menu(aaiPOST)
        prompt("Post Value Monitors")
    }
    field(APST, DBF_MENU) {
        promptgroup("80 - Display")
        interest(1)
        menu(aaiPOST)
        prompt("Post Archive Monitors")
    }
    field(HASH, DBF_ULONG) {
        interest(3)
        prompt("Hash of OnChange data.")
    }
}
device(aai, CONSTANT, devAaiSoft, "Soft Channel")
recordtype(aao) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
```

```
2641    %#include "ellLib.h"
2642    %#include "devSup.h"
2643    %#include "epicsTime.h"
2644    %
2645    %/* Declare Device Support Entry Table */
2646    %struct aaoRecord;
2647    %typedef struct aaodset {
2648    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
2649    %    long (*write_aao)(struct aaoRecord *prec); /*returns: (-1,0)=>(failure,success)*/
2650    %} aaodset;
2651    %#define HAS_aaodset
2652    %#define AAO_POSTS_NORD
2653    %
2654    %#include "callback.h"
2655    field(NAME, DBF_STRING) {
2656        size(61)
2657        special(SPC_NOMOD)
2658        prompt("Record Name")
2659    }
2660    field(DESC, DBF_STRING) {
2661        size(41)
2662        promptgroup("10 - Common")
2663        prompt("Descriptor")
2664    }
2665    field(ASG, DBF_STRING) {
2666        size(29)
2667        promptgroup("10 - Common")
2668        special(SPC_AS)
2669        prompt("Access Security Group")
2670    }
2671    field(SCAN, DBF_MENU) {
2672        promptgroup("20 - Scan")
2673        interest(1)
2674        menu(menuScan)
2675        special(SPC_SCAN)
2676        prompt("Scan Mechanism")
2677    }
2678    field(PINI, DBF_MENU) {
2679        promptgroup("20 - Scan")
2680        interest(1)
2681        menu(menuPini)
2682        prompt("Process at iocInit")
2683    }
2684    field(PHAS, DBF_SHORT) {
2685        promptgroup("20 - Scan")
2686        interest(1)
2687        special(SPC_SCAN)
2688        prompt("Scan Phase")
2689    }
2690    field(EVNT, DBF_STRING) {
2691        size(40)
2692        promptgroup("20 - Scan")
2693        interest(1)
2694        special(SPC_SCAN)
2695        prompt("Event Name")
2696    }
2697    field(TSE, DBF_SHORT) {
2698        promptgroup("20 - Scan")
2699        interest(1)
2700        prompt("Time Stamp Event")
2701    }
2702    field(TSEL, DBF_INLINK) {
2703        promptgroup("20 - Scan")
2704        interest(1)
2705        prompt("Time Stamp Link")
2706    }
2707    field(DTYP, DBF_DEVICE) {
2708        promptgroup("10 - Common")
```

```
2709        interest(1)
2710        prompt("Device Type")
2711    }
2712    field(DISV, DBF_SHORT) {
2713        promptgroup("20 - Scan")
2714        initial("1")
2715        prompt("Disable Value")
2716    }
2717    field(DISA, DBF_SHORT) {
2718        prompt("Disable")
2719    }
2720    field(SDIS, DBF_INLINK) {
2721        promptgroup("20 - Scan")
2722        interest(1)
2723        prompt("Scanning Disable")
2724    }
2725    field(MLOK, DBF_NOACCESS) {
2726        interest(4)
2727        extra("epicsMutexId        mlok")
2728        special(SPC_NOMOD)
2729        prompt("Monitor lock")
2730    }
2731    field(MLIS, DBF_NOACCESS) {
2732        interest(4)
2733        extra("ELLLIST             mlis")
2734        special(SPC_NOMOD)
2735        prompt("Monitor List")
2736    }
2737    field(BKLNK, DBF_NOACCESS) {
2738        interest(4)
2739        extra("ELLLIST             bklnk")
2740        special(SPC_NOMOD)
2741        prompt("Backwards link tracking")
2742    }
2743    field(DISP, DBF_UCHAR) {
2744        promptgroup("10 - Common")
2745        interest(1)
2746        prompt("Disable putField")
2747    }
2748    field(PROC, DBF_UCHAR) {
2749        interest(3)
2750        pp(TRUE)
2751        prompt("Force Processing")
2752    }
2753    field(STAT, DBF_MENU) {
2754        initial("UDF")
2755        menu(menuAlarmStat)
2756        special(SPC_NOMOD)
2757        prompt("Alarm Status")
2758    }
2759    field(SEVR, DBF_MENU) {
2760        menu(menuAlarmSevr)
2761        special(SPC_NOMOD)
2762        prompt("Alarm Severity")
2763    }
2764    field(AMSG, DBF_STRING) {
2765        size(40)
2766        special(SPC_NOMOD)
2767        prompt("Alarm Message")
2768    }
2769    field(NSTA, DBF_MENU) {
2770        interest(2)
2771        menu(menuAlarmStat)
2772        special(SPC_NOMOD)
2773        prompt("New Alarm Status")
2774    }
2775    field(NSEV, DBF_MENU) {
2776        interest(2)
```

```
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Reprocess ")
    }
    field(ASP, DBF_NOACCESS) {
        interest(4)
        extra("struct asgMember    *asp")
        special(SPC_NOMOD)
        prompt("Access Security Pvt")
    }
    field(PPN, DBF_NOACCESS) {
        interest(4)
        extra("struct processNotify *ppn")
        special(SPC_NOMOD)
        prompt("pprocessNotify")
    }
    field(PPNR, DBF_NOACCESS) {
        interest(4)
        extra("struct processNotifyRecord *ppnr")
        special(SPC_NOMOD)
        prompt("pprocessNotifyRecord")
    }
    field(SPVT, DBF_NOACCESS) {
```

```
2845        interest(4)
2846        extra("struct scan_element *spvt")
2847        special(SPC_NOMOD)
2848        prompt("Scan Private")
2849    }
2850    field(RSET, DBF_NOACCESS) {
2851        interest(4)
2852        extra("struct typed_rset   *rset")
2853        special(SPC_NOMOD)
2854        prompt("Address of RSET")
2855    }
2856    field(DSET, DBF_NOACCESS) {
2857        interest(4)
2858        extra("unambiguous_dset    *dset")
2859        special(SPC_NOMOD)
2860        prompt("DSET address")
2861    }
2862    field(DPVT, DBF_NOACCESS) {
2863        interest(4)
2864        extra("void                *dpvt")
2865        special(SPC_NOMOD)
2866        prompt("Device Private")
2867    }
2868    field(RDES, DBF_NOACCESS) {
2869        interest(4)
2870        extra("struct dbRecordType *rdes")
2871        special(SPC_NOMOD)
2872        prompt("Address of dbRecordType")
2873    }
2874    field(LSET, DBF_NOACCESS) {
2875        interest(4)
2876        extra("struct lockRecord   *lset")
2877        special(SPC_NOMOD)
2878        prompt("Lock Set")
2879    }
2880    field(PRIO, DBF_MENU) {
2881        promptgroup("20 - Scan")
2882        interest(1)
2883        menu(menuPriority)
2884        special(SPC_SCAN)
2885        prompt("Scheduling Priority")
2886    }
2887    field(TPRO, DBF_UCHAR) {
2888        prompt("Trace Processing")
2889    }
2890    field(BKPT, DBF_NOACCESS) {
2891        interest(1)
2892        extra("epicsUInt8          bkpt")
2893        special(SPC_NOMOD)
2894        prompt("Break Point")
2895    }
2896    field(UDF, DBF_UCHAR) {
2897        promptgroup("10 - Common")
2898        interest(1)
2899        initial("1")
2900        pp(TRUE)
2901        prompt("Undefined")
2902    }
2903    field(UDFS, DBF_MENU) {
2904        promptgroup("70 - Alarm")
2905        interest(1)
2906        initial("INVALID")
2907        menu(menuAlarmSevr)
2908        prompt("Undefined Alarm Sevrty")
2909    }
2910    field(TIME, DBF_NOACCESS) {
2911        interest(2)
2912        extra("epicsTimeStamp      time")
```

```
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_NOACCESS) {
        extra("void *       val")
        asl(ASL0)
        pp(TRUE)
        special(SPC_DBADDR)
        prompt("Value")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(OUT, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Specification")
    }
    field(DOL, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Desired Output Link")
    }
    field(OMSL, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        menu(menuOmsl)
        prompt("Output Mode Select")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(NELM, DBF_ULONG) {
        promptgroup("30 - Action")
        prompt("Number of Elements")
        prop(YES)
        interest(1)
        initial("1")
        special(SPC_NOMOD)
```

```
2981    }
2982    field(FTVL, DBF_MENU) {
2983        promptgroup("30 - Action")
2984        interest(1)
2985        menu(menuFtype)
2986        special(SPC_NOMOD)
2987        prompt("Field Type of Value")
2988    }
2989    field(NORD, DBF_ULONG) {
2990        special(SPC_NOMOD)
2991        prompt("Number elements read")
2992    }
2993    field(BPTR, DBF_NOACCESS) {
2994        interest(4)
2995        extra("void *       bptr")
2996        special(SPC_NOMOD)
2997        prompt("Buffer Pointer")
2998    }
2999    field(SIML, DBF_INLINK) {
3000        promptgroup("90 - Simulate")
3001        interest(1)
3002        prompt("Simulation Mode Link")
3003    }
3004    field(SIMM, DBF_MENU) {
3005        interest(1)
3006        menu(menuYesNo)
3007        special(SPC_MOD)
3008        prompt("Simulation Mode")
3009    }
3010    field(SIMS, DBF_MENU) {
3011        promptgroup("90 - Simulate")
3012        interest(2)
3013        menu(menuAlarmSevr)
3014        prompt("Simulation Mode Severity")
3015    }
3016    field(SIOL, DBF_OUTLINK) {
3017        promptgroup("90 - Simulate")
3018        interest(1)
3019        prompt("Simulation Output Link")
3020    }
3021    field(OLDSIMM, DBF_MENU) {
3022        interest(4)
3023        menu(menuSimm)
3024        special(SPC_NOMOD)
3025        prompt("Prev. Simulation Mode")
3026    }
3027    field(SSCN, DBF_MENU) {
3028        promptgroup("90 - Simulate")
3029        interest(1)
3030        initial("65535")
3031        menu(menuScan)
3032        prompt("Sim. Mode Scan")
3033    }
3034    field(SDLY, DBF_DOUBLE) {
3035        promptgroup("90 - Simulate")
3036        interest(2)
3037        initial("-1.0")
3038        prompt("Sim. Mode Async Delay")
3039    }
3040    field(SIMPVT, DBF_NOACCESS) {
3041        interest(4)
3042        extra("epicsCallback            *simpvt")
3043        special(SPC_NOMOD)
3044        prompt("Sim. Mode Private")
3045    }
3046    field(MPST, DBF_MENU) {
3047        promptgroup("80 - Display")
3048        interest(1)
```

```
        menu(aaoPOST)
        prompt("Post Value Monitors")
    }
    field(APST, DBF_MENU) {
        promptgroup("80 - Display")
        interest(1)
        menu(aaoPOST)
        prompt("Post Archive Monitors")
    }
    field(HASH, DBF_ULONG) {
        interest(3)
        prompt("Hash of OnChange data.")
    }
}
device(aao, CONSTANT, devAaoSoft, "Soft Channel")
recordtype(ai) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct aiRecord;
    %typedef struct aidset {
    %    dset common;
    %    long (*read_ai)(struct aiRecord *prec);
    %    long (*special_linconv)(struct aiRecord *prec, int after);
    %} aidset;
    %#define HAS_aidset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
```

```
3117    field(EVNT, DBF_STRING) {
3118        size(40)
3119        promptgroup("20 - Scan")
3120        interest(1)
3121        special(SPC_SCAN)
3122        prompt("Event Name")
3123    }
3124    field(TSE, DBF_SHORT) {
3125        promptgroup("20 - Scan")
3126        interest(1)
3127        prompt("Time Stamp Event")
3128    }
3129    field(TSEL, DBF_INLINK) {
3130        promptgroup("20 - Scan")
3131        interest(1)
3132        prompt("Time Stamp Link")
3133    }
3134    field(DTYP, DBF_DEVICE) {
3135        promptgroup("10 - Common")
3136        interest(1)
3137        prompt("Device Type")
3138    }
3139    field(DISV, DBF_SHORT) {
3140        promptgroup("20 - Scan")
3141        initial("1")
3142        prompt("Disable Value")
3143    }
3144    field(DISA, DBF_SHORT) {
3145        prompt("Disable")
3146    }
3147    field(SDIS, DBF_INLINK) {
3148        promptgroup("20 - Scan")
3149        interest(1)
3150        prompt("Scanning Disable")
3151    }
3152    field(MLOK, DBF_NOACCESS) {
3153        interest(4)
3154        extra("epicsMutexId        mlok")
3155        special(SPC_NOMOD)
3156        prompt("Monitor lock")
3157    }
3158    field(MLIS, DBF_NOACCESS) {
3159        interest(4)
3160        extra("ELLLIST             mlis")
3161        special(SPC_NOMOD)
3162        prompt("Monitor List")
3163    }
3164    field(BKLNK, DBF_NOACCESS) {
3165        interest(4)
3166        extra("ELLLIST             bklnk")
3167        special(SPC_NOMOD)
3168        prompt("Backwards link tracking")
3169    }
3170    field(DISP, DBF_UCHAR) {
3171        promptgroup("10 - Common")
3172        interest(1)
3173        prompt("Disable putField")
3174    }
3175    field(PROC, DBF_UCHAR) {
3176        interest(3)
3177        pp(TRUE)
3178        prompt("Force Processing")
3179    }
3180    field(STAT, DBF_MENU) {
3181        initial("UDF")
3182        menu(menuAlarmStat)
3183        special(SPC_NOMOD)
3184        prompt("Alarm Status")
```

```
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Reprocess ")
    }
```

```
3253    field(ASP, DBF_NOACCESS) {
3254        interest(4)
3255        extra("struct asgMember    *asp")
3256        special(SPC_NOMOD)
3257        prompt("Access Security Pvt")
3258    }
3259    field(PPN, DBF_NOACCESS) {
3260        interest(4)
3261        extra("struct processNotify *ppn")
3262        special(SPC_NOMOD)
3263        prompt("pprocessNotify")
3264    }
3265    field(PPNR, DBF_NOACCESS) {
3266        interest(4)
3267        extra("struct processNotifyRecord *ppnr")
3268        special(SPC_NOMOD)
3269        prompt("pprocessNotifyRecord")
3270    }
3271    field(SPVT, DBF_NOACCESS) {
3272        interest(4)
3273        extra("struct scan_element *spvt")
3274        special(SPC_NOMOD)
3275        prompt("Scan Private")
3276    }
3277    field(RSET, DBF_NOACCESS) {
3278        interest(4)
3279        extra("struct typed_rset   *rset")
3280        special(SPC_NOMOD)
3281        prompt("Address of RSET")
3282    }
3283    field(DSET, DBF_NOACCESS) {
3284        interest(4)
3285        extra("unambiguous_dset    *dset")
3286        special(SPC_NOMOD)
3287        prompt("DSET address")
3288    }
3289    field(DPVT, DBF_NOACCESS) {
3290        interest(4)
3291        extra("void                *dpvt")
3292        special(SPC_NOMOD)
3293        prompt("Device Private")
3294    }
3295    field(RDES, DBF_NOACCESS) {
3296        interest(4)
3297        extra("struct dbRecordType *rdes")
3298        special(SPC_NOMOD)
3299        prompt("Address of dbRecordType")
3300    }
3301    field(LSET, DBF_NOACCESS) {
3302        interest(4)
3303        extra("struct lockRecord   *lset")
3304        special(SPC_NOMOD)
3305        prompt("Lock Set")
3306    }
3307    field(PRIO, DBF_MENU) {
3308        promptgroup("20 - Scan")
3309        interest(1)
3310        menu(menuPriority)
3311        special(SPC_SCAN)
3312        prompt("Scheduling Priority")
3313    }
3314    field(TPRO, DBF_UCHAR) {
3315        prompt("Trace Processing")
3316    }
3317    field(BKPT, DBF_NOACCESS) {
3318        interest(1)
3319        extra("epicsUInt8          bkpt")
3320        special(SPC_NOMOD)
```

```
3321        prompt("Break Point")
3322    }
3323    field(UDF, DBF_UCHAR) {
3324        promptgroup("10 - Common")
3325        interest(1)
3326        initial("1")
3327        pp(TRUE)
3328        prompt("Undefined")
3329    }
3330    field(UDFS, DBF_MENU) {
3331        promptgroup("70 - Alarm")
3332        interest(1)
3333        initial("INVALID")
3334        menu(menuAlarmSevr)
3335        prompt("Undefined Alarm Sevrty")
3336    }
3337    field(TIME, DBF_NOACCESS) {
3338        interest(2)
3339        extra("epicsTimeStamp      time")
3340        special(SPC_NOMOD)
3341        prompt("Time")
3342    }
3343    field(UTAG, DBF_UINT64) {
3344        interest(3)
3345        special(SPC_NOMOD)
3346        prompt("Time Tag")
3347    }
3348    field(FLNK, DBF_FWDLINK) {
3349        promptgroup("20 - Scan")
3350        interest(1)
3351        prompt("Forward Process Link")
3352    }
3353    field(VAL, DBF_DOUBLE) {
3354        promptgroup("40 - Input")
3355        asl(ASL0)
3356        pp(TRUE)
3357        prompt("Current EGU Value")
3358    }
3359    field(INP, DBF_INLINK) {
3360        promptgroup("40 - Input")
3361        interest(1)
3362        prompt("Input Specification")
3363    }
3364    field(PREC, DBF_SHORT) {
3365        promptgroup("80 - Display")
3366        interest(1)
3367        prop(YES)
3368        prompt("Display Precision")
3369    }
3370    field(LINR, DBF_MENU) {
3371        promptgroup("60 - Convert")
3372        menu(menuConvert)
3373        prompt("Linearization")
3374        interest(1)
3375        pp(TRUE)
3376        special(SPC_LINCONV)
3377    }
3378    field(EGUF, DBF_DOUBLE) {
3379        promptgroup("60 - Convert")
3380        interest(1)
3381        pp(TRUE)
3382        special(SPC_LINCONV)
3383        prompt("Engineer Units Full")
3384    }
3385    field(EGUL, DBF_DOUBLE) {
3386        promptgroup("60 - Convert")
3387        interest(1)
3388        pp(TRUE)
```

```
        special(SPC_LINCONV)
        prompt("Engineer Units Low")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(AOFF, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(1)
        pp(TRUE)
        prompt("Adjustment Offset")
    }
    field(ASLO, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Adjustment Slope")
    }
    field(SMOO, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(1)
        prompt("Smoothing")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
```

```
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HYST, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(AFTC, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(ADEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(LALM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(AFVL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Alarm Filter Value")
    }
    field(ALST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_DOUBLE) {
```

```
3525        interest(3)
3526        special(SPC_NOMOD)
3527        prompt("Last Val Monitored")
3528    }
3529    field(ESLO, DBF_DOUBLE) {
3530        promptgroup("60 - Convert")
3531        interest(2)
3532        initial("1")
3533        pp(TRUE)
3534        prompt("Raw to EGU Slope")
3535    }
3536    field(EOFF, DBF_DOUBLE) {
3537        promptgroup("60 - Convert")
3538        interest(2)
3539        pp(TRUE)
3540        prompt("Raw to EGU Offset")
3541    }
3542    field(ROFF, DBF_ULONG) {
3543        interest(2)
3544        pp(TRUE)
3545        prompt("Raw Offset")
3546    }
3547    field(PBRK, DBF_NOACCESS) {
3548        interest(4)
3549        extra("void *   pbrk")
3550        special(SPC_NOMOD)
3551        prompt("Ptrto brkTable")
3552    }
3553    field(INIT, DBF_SHORT) {
3554        interest(3)
3555        special(SPC_NOMOD)
3556        prompt("Initialized?")
3557    }
3558    field(LBRK, DBF_SHORT) {
3559        interest(3)
3560        special(SPC_NOMOD)
3561        prompt("LastBreak Point")
3562    }
3563    field(RVAL, DBF_LONG) {
3564        pp(TRUE)
3565        prompt("Current Raw Value")
3566    }
3567    field(ORAW, DBF_LONG) {
3568        interest(3)
3569        special(SPC_NOMOD)
3570        prompt("Previous Raw Value")
3571    }
3572    field(SIOL, DBF_INLINK) {
3573        promptgroup("90 - Simulate")
3574        interest(1)
3575        prompt("Simulation Input Link")
3576    }
3577    field(SVAL, DBF_DOUBLE) {
3578        prompt("Simulation Value")
3579    }
3580    field(SIML, DBF_INLINK) {
3581        promptgroup("90 - Simulate")
3582        interest(1)
3583        prompt("Simulation Mode Link")
3584    }
3585    field(SIMM, DBF_MENU) {
3586        interest(1)
3587        menu(menuSimm)
3588        special(SPC_MOD)
3589        prompt("Simulation Mode")
3590    }
3591    field(SIMS, DBF_MENU) {
3592        promptgroup("90 - Simulate")
```

```
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(ai, CONSTANT, devAiSoft, "Soft Channel")
device(ai, CONSTANT, devAiSoftRaw, "Raw Soft Channel")
device(ai, CONSTANT, devAiSoftCallback, "Async Soft Channel")
device(ai, INST_IO, devTimestampAI, "Soft Timestamp")
device(ai, INST_IO, devAiGeneralTime, "General Time")
device(ai, INST_IO, devPSCRegAi, "PSC Reg")
device(ai, INST_IO, devPSCRegF32Ai, "PSC Reg F32")
device(ai, INST_IO, devPSCRegF64Ai, "PSC Reg F64")
device(ai, INST_IO, devPSCRegRBAi, "PSC RB Reg")
device(ai, INST_IO, devPSCRegRBF32Ai, "PSC RB Reg F32")
device(ai, INST_IO, devPSCRegRBF64Ai, "PSC RB Reg F64")
recordtype(ao) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct aoRecord;
    %typedef struct aodset {
    %    dset common; /*init_record returns: (0,2)=>(success,success no convert)*/
    %    long (*write_ao)(struct aoRecord *prec); /*(0)=>(success ) */
    %    long (*special_linconv)(struct aoRecord *prec, int after);
    %} aodset;
    %#define HAS_aodset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
```

```
3661    }
3662    field(ASG, DBF_STRING) {
3663        size(29)
3664        promptgroup("10 - Common")
3665        special(SPC_AS)
3666        prompt("Access Security Group")
3667    }
3668    field(SCAN, DBF_MENU) {
3669        promptgroup("20 - Scan")
3670        interest(1)
3671        menu(menuScan)
3672        special(SPC_SCAN)
3673        prompt("Scan Mechanism")
3674    }
3675    field(PINI, DBF_MENU) {
3676        promptgroup("20 - Scan")
3677        interest(1)
3678        menu(menuPini)
3679        prompt("Process at iocInit")
3680    }
3681    field(PHAS, DBF_SHORT) {
3682        promptgroup("20 - Scan")
3683        interest(1)
3684        special(SPC_SCAN)
3685        prompt("Scan Phase")
3686    }
3687    field(EVNT, DBF_STRING) {
3688        size(40)
3689        promptgroup("20 - Scan")
3690        interest(1)
3691        special(SPC_SCAN)
3692        prompt("Event Name")
3693    }
3694    field(TSE, DBF_SHORT) {
3695        promptgroup("20 - Scan")
3696        interest(1)
3697        prompt("Time Stamp Event")
3698    }
3699    field(TSEL, DBF_INLINK) {
3700        promptgroup("20 - Scan")
3701        interest(1)
3702        prompt("Time Stamp Link")
3703    }
3704    field(DTYP, DBF_DEVICE) {
3705        promptgroup("10 - Common")
3706        interest(1)
3707        prompt("Device Type")
3708    }
3709    field(DISV, DBF_SHORT) {
3710        promptgroup("20 - Scan")
3711        initial("1")
3712        prompt("Disable Value")
3713    }
3714    field(DISA, DBF_SHORT) {
3715        prompt("Disable")
3716    }
3717    field(SDIS, DBF_INLINK) {
3718        promptgroup("20 - Scan")
3719        interest(1)
3720        prompt("Scanning Disable")
3721    }
3722    field(MLOK, DBF_NOACCESS) {
3723        interest(4)
3724        extra("epicsMutexId        mlok")
3725        special(SPC_NOMOD)
3726        prompt("Monitor lock")
3727    }
3728    field(MLIS, DBF_NOACCESS) {
```

```
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
```

```
3797    field(DISS, DBF_MENU) {
3798        promptgroup("70 - Alarm")
3799        interest(1)
3800        menu(menuAlarmSevr)
3801        prompt("Disable Alarm Sevrty")
3802    }
3803    field(LCNT, DBF_UCHAR) {
3804        interest(2)
3805        special(SPC_NOMOD)
3806        prompt("Lock Count")
3807    }
3808    field(PACT, DBF_UCHAR) {
3809        interest(1)
3810        special(SPC_NOMOD)
3811        prompt("Record active")
3812    }
3813    field(PUTF, DBF_UCHAR) {
3814        interest(1)
3815        special(SPC_NOMOD)
3816        prompt("dbPutField process")
3817    }
3818    field(RPRO, DBF_UCHAR) {
3819        interest(1)
3820        special(SPC_NOMOD)
3821        prompt("Reprocess ")
3822    }
3823    field(ASP, DBF_NOACCESS) {
3824        interest(4)
3825        extra("struct asgMember    *asp")
3826        special(SPC_NOMOD)
3827        prompt("Access Security Pvt")
3828    }
3829    field(PPN, DBF_NOACCESS) {
3830        interest(4)
3831        extra("struct processNotify *ppn")
3832        special(SPC_NOMOD)
3833        prompt("pprocessNotify")
3834    }
3835    field(PPNR, DBF_NOACCESS) {
3836        interest(4)
3837        extra("struct processNotifyRecord *ppnr")
3838        special(SPC_NOMOD)
3839        prompt("pprocessNotifyRecord")
3840    }
3841    field(SPVT, DBF_NOACCESS) {
3842        interest(4)
3843        extra("struct scan_element *spvt")
3844        special(SPC_NOMOD)
3845        prompt("Scan Private")
3846    }
3847    field(RSET, DBF_NOACCESS) {
3848        interest(4)
3849        extra("struct typed_rset   *rset")
3850        special(SPC_NOMOD)
3851        prompt("Address of RSET")
3852    }
3853    field(DSET, DBF_NOACCESS) {
3854        interest(4)
3855        extra("unambiguous_dset    *dset")
3856        special(SPC_NOMOD)
3857        prompt("DSET address")
3858    }
3859    field(DPVT, DBF_NOACCESS) {
3860        interest(4)
3861        extra("void                *dpvt")
3862        special(SPC_NOMOD)
3863        prompt("Device Private")
3864    }
```

```
    field(RDES, DBF_NOACCESS) {
        interest(4)
        extra("struct dbRecordType *rdes")
        special(SPC_NOMOD)
        prompt("Address of dbRecordType")
    }
    field(LSET, DBF_NOACCESS) {
        interest(4)
        extra("struct lockRecord   *lset")
        special(SPC_NOMOD)
        prompt("Lock Set")
    }
    field(PRIO, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_DOUBLE) {
        promptgroup("50 - Output")
        asl(ASL0)
        pp(TRUE)
        prompt("Desired Output")
    }
    field(OVAL, DBF_DOUBLE) {
        prompt("Output Value")
    }
    field(OUT, DBF_OUTLINK) {
```

```
3933        promptgroup("50 - Output")
3934        interest(1)
3935        prompt("Output Specification")
3936    }
3937    field(OROC, DBF_DOUBLE) {
3938        promptgroup("50 - Output")
3939        interest(1)
3940        prompt("Output Rate of Change")
3941    }
3942    field(DOL, DBF_INLINK) {
3943        promptgroup("40 - Input")
3944        interest(1)
3945        prompt("Desired Output Link")
3946    }
3947    field(OMSL, DBF_MENU) {
3948        promptgroup("50 - Output")
3949        interest(1)
3950        menu(menuOmsl)
3951        prompt("Output Mode Select")
3952    }
3953    field(OIF, DBF_MENU) {
3954        promptgroup("50 - Output")
3955        interest(1)
3956        menu(aoOIF)
3957        prompt("Out Full/Incremental")
3958    }
3959    field(PREC, DBF_SHORT) {
3960        promptgroup("80 - Display")
3961        interest(1)
3962        prop(YES)
3963        prompt("Display Precision")
3964    }
3965    field(LINR, DBF_MENU) {
3966        promptgroup("60 - Convert")
3967        menu(menuConvert)
3968        prompt("Linearization")
3969        interest(1)
3970        pp(TRUE)
3971        special(SPC_LINCONV)
3972    }
3973    field(EGUF, DBF_DOUBLE) {
3974        promptgroup("60 - Convert")
3975        interest(1)
3976        pp(TRUE)
3977        special(SPC_LINCONV)
3978        prompt("Eng Units Full")
3979    }
3980    field(EGUL, DBF_DOUBLE) {
3981        promptgroup("60 - Convert")
3982        interest(1)
3983        pp(TRUE)
3984        special(SPC_LINCONV)
3985        prompt("Eng Units Low")
3986    }
3987    field(EGU, DBF_STRING) {
3988        size(16)
3989        promptgroup("80 - Display")
3990        interest(1)
3991        prop(YES)
3992        prompt("Engineering Units")
3993    }
3994    field(ROFF, DBF_ULONG) {
3995        interest(2)
3996        pp(TRUE)
3997        prompt("Raw Offset")
3998    }
3999    field(EOFF, DBF_DOUBLE) {
4000        promptgroup("60 - Convert")
```

```
        interest(2)
        pp(TRUE)
        prompt("EGU to Raw Offset")
    }
    field(ESLO, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(2)
        initial("1")
        pp(TRUE)
        prompt("EGU to Raw Slope")
    }
    field(DRVH, DBF_DOUBLE) {
        promptgroup("30 - Action")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Drive High Limit")
    }
    field(DRVL, DBF_DOUBLE) {
        promptgroup("30 - Action")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Drive Low Limit")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(AOFF, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(1)
        pp(TRUE)
        prompt("Adjustment Offset")
    }
    field(ASLO, DBF_DOUBLE) {
        promptgroup("60 - Convert")
        interest(1)
        pp(TRUE)
        prompt("Adjustment Slope")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
```

```
4069        prompt("High Alarm Limit")
4070    }
4071    field(LOW, DBF_DOUBLE) {
4072        promptgroup("70 - Alarm")
4073        interest(1)
4074        pp(TRUE)
4075        prop(YES)
4076        prompt("Low Alarm Limit")
4077    }
4078    field(HHSV, DBF_MENU) {
4079        promptgroup("70 - Alarm")
4080        menu(menuAlarmSevr)
4081        prompt("Hihi Severity")
4082        prop(YES)
4083        interest(1)
4084        pp(TRUE)
4085    }
4086    field(LLSV, DBF_MENU) {
4087        promptgroup("70 - Alarm")
4088        menu(menuAlarmSevr)
4089        prompt("Lolo Severity")
4090        prop(YES)
4091        interest(1)
4092        pp(TRUE)
4093    }
4094    field(HSV, DBF_MENU) {
4095        promptgroup("70 - Alarm")
4096        menu(menuAlarmSevr)
4097        prompt("High Severity")
4098        prop(YES)
4099        interest(1)
4100        pp(TRUE)
4101    }
4102    field(LSV, DBF_MENU) {
4103        promptgroup("70 - Alarm")
4104        menu(menuAlarmSevr)
4105        prompt("Low Severity")
4106        prop(YES)
4107        interest(1)
4108        pp(TRUE)
4109    }
4110    field(HYST, DBF_DOUBLE) {
4111        promptgroup("70 - Alarm")
4112        interest(1)
4113        prompt("Alarm Deadband")
4114    }
4115    field(ADEL, DBF_DOUBLE) {
4116        promptgroup("80 - Display")
4117        interest(1)
4118        prompt("Archive Deadband")
4119    }
4120    field(MDEL, DBF_DOUBLE) {
4121        promptgroup("80 - Display")
4122        interest(1)
4123        prompt("Monitor Deadband")
4124    }
4125    field(RVAL, DBF_LONG) {
4126        pp(TRUE)
4127        prompt("Current Raw Value")
4128    }
4129    field(ORAW, DBF_LONG) {
4130        interest(3)
4131        special(SPC_NOMOD)
4132        prompt("Previous Raw Value")
4133    }
4134    field(RBV, DBF_LONG) {
4135        special(SPC_NOMOD)
4136        prompt("Readback Value")
```

```
4137        }
4138        field(ORBV, DBF_LONG) {
4139            interest(3)
4140            special(SPC_NOMOD)
4141            prompt("Prev Readback Value")
4142        }
4143        field(PVAL, DBF_DOUBLE) {
4144            interest(3)
4145            special(SPC_NOMOD)
4146            prompt("Previous value")
4147        }
4148        field(LALM, DBF_DOUBLE) {
4149            interest(3)
4150            special(SPC_NOMOD)
4151            prompt("Last Value Alarmed")
4152        }
4153        field(ALST, DBF_DOUBLE) {
4154            interest(3)
4155            special(SPC_NOMOD)
4156            prompt("Last Value Archived")
4157        }
4158        field(MLST, DBF_DOUBLE) {
4159            interest(3)
4160            special(SPC_NOMOD)
4161            prompt("Last Val Monitored")
4162        }
4163        field(PBRK, DBF_NOACCESS) {
4164            interest(4)
4165            extra("void *   pbrk")
4166            special(SPC_NOMOD)
4167            prompt("Ptrto brkTable")
4168        }
4169        field(INIT, DBF_SHORT) {
4170            interest(3)
4171            special(SPC_NOMOD)
4172            prompt("Initialized?")
4173        }
4174        field(LBRK, DBF_SHORT) {
4175            interest(3)
4176            special(SPC_NOMOD)
4177            prompt("LastBreak Point")
4178        }
4179        field(SIOL, DBF_OUTLINK) {
4180            promptgroup("90 - Simulate")
4181            interest(1)
4182            prompt("Simulation Output Link")
4183        }
4184        field(SIML, DBF_INLINK) {
4185            promptgroup("90 - Simulate")
4186            interest(1)
4187            prompt("Simulation Mode Link")
4188        }
4189        field(SIMM, DBF_MENU) {
4190            interest(1)
4191            menu(menuSimm)
4192            special(SPC_MOD)
4193            prompt("Simulation Mode")
4194        }
4195        field(SIMS, DBF_MENU) {
4196            promptgroup("90 - Simulate")
4197            interest(2)
4198            menu(menuAlarmSevr)
4199            prompt("Simulation Mode Severity")
4200        }
4201        field(OLDSIMM, DBF_MENU) {
4202            interest(4)
4203            menu(menuSimm)
4204            special(SPC_NOMOD)
```

```
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID output action")
    }
    field(IVOV, DBF_DOUBLE) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
    field(OMOD, DBF_UCHAR) {
        special(SPC_NOMOD)
        prompt("Was OVAL modified?")
    }
}
device(ao, CONSTANT, devAoSoft, "Soft Channel")
device(ao, CONSTANT, devAoSoftRaw, "Raw Soft Channel")
device(ao, CONSTANT, devAoSoftCallback, "Async Soft Channel")
device(ao, INST_IO, devPSCRegAo, "PSC Reg")
device(ao, INST_IO, devPSCRegF32Ao, "PSC Reg F32")
device(ao, INST_IO, devPSCRegF64Ao, "PSC Reg F64")
device(ao, INST_IO, devPSCSingleS32Ao, "PSC Single I32")
device(ao, INST_IO, devPSCSingleF32Ao, "PSC Single F32")
device(ao, INST_IO, devPSCSingleF64Ao, "PSC Single F64")
recordtype(bi) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct biRecord;
    %typedef struct bidset {
    %    dset common;
    %    long (*read_bi)(struct biRecord *prec);
    %} bidset;
    %#define HAS_bidset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
```

```
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
```

```
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
```

```
4409        menu(menuYesNo)
4410        special(SPC_NOMOD)
4411        prompt("Alarm Ack Transient")
4412    }
4413    field(DISS, DBF_MENU) {
4414        promptgroup("70 - Alarm")
4415        interest(1)
4416        menu(menuAlarmSevr)
4417        prompt("Disable Alarm Sevrty")
4418    }
4419    field(LCNT, DBF_UCHAR) {
4420        interest(2)
4421        special(SPC_NOMOD)
4422        prompt("Lock Count")
4423    }
4424    field(PACT, DBF_UCHAR) {
4425        interest(1)
4426        special(SPC_NOMOD)
4427        prompt("Record active")
4428    }
4429    field(PUTF, DBF_UCHAR) {
4430        interest(1)
4431        special(SPC_NOMOD)
4432        prompt("dbPutField process")
4433    }
4434    field(RPRO, DBF_UCHAR) {
4435        interest(1)
4436        special(SPC_NOMOD)
4437        prompt("Reprocess ")
4438    }
4439    field(ASP, DBF_NOACCESS) {
4440        interest(4)
4441        extra("struct asgMember    *asp")
4442        special(SPC_NOMOD)
4443        prompt("Access Security Pvt")
4444    }
4445    field(PPN, DBF_NOACCESS) {
4446        interest(4)
4447        extra("struct processNotify *ppn")
4448        special(SPC_NOMOD)
4449        prompt("pprocessNotify")
4450    }
4451    field(PPNR, DBF_NOACCESS) {
4452        interest(4)
4453        extra("struct processNotifyRecord *ppnr")
4454        special(SPC_NOMOD)
4455        prompt("pprocessNotifyRecord")
4456    }
4457    field(SPVT, DBF_NOACCESS) {
4458        interest(4)
4459        extra("struct scan_element *spvt")
4460        special(SPC_NOMOD)
4461        prompt("Scan Private")
4462    }
4463    field(RSET, DBF_NOACCESS) {
4464        interest(4)
4465        extra("struct typed_rset   *rset")
4466        special(SPC_NOMOD)
4467        prompt("Address of RSET")
4468    }
4469    field(DSET, DBF_NOACCESS) {
4470        interest(4)
4471        extra("unambiguous_dset    *dset")
4472        special(SPC_NOMOD)
4473        prompt("DSET address")
4474    }
4475    field(DPVT, DBF_NOACCESS) {
4476        interest(4)
```

```
4477        extra("void                *dpvt")
4478        special(SPC_NOMOD)
4479        prompt("Device Private")
4480    }
4481    field(RDES, DBF_NOACCESS) {
4482        interest(4)
4483        extra("struct dbRecordType *rdes")
4484        special(SPC_NOMOD)
4485        prompt("Address of dbRecordType")
4486    }
4487    field(LSET, DBF_NOACCESS) {
4488        interest(4)
4489        extra("struct lockRecord   *lset")
4490        special(SPC_NOMOD)
4491        prompt("Lock Set")
4492    }
4493    field(PRIO, DBF_MENU) {
4494        promptgroup("20 - Scan")
4495        interest(1)
4496        menu(menuPriority)
4497        special(SPC_SCAN)
4498        prompt("Scheduling Priority")
4499    }
4500    field(TPRO, DBF_UCHAR) {
4501        prompt("Trace Processing")
4502    }
4503    field(BKPT, DBF_NOACCESS) {
4504        interest(1)
4505        extra("epicsUInt8          bkpt")
4506        special(SPC_NOMOD)
4507        prompt("Break Point")
4508    }
4509    field(UDF, DBF_UCHAR) {
4510        promptgroup("10 - Common")
4511        interest(1)
4512        initial("1")
4513        pp(TRUE)
4514        prompt("Undefined")
4515    }
4516    field(UDFS, DBF_MENU) {
4517        promptgroup("70 - Alarm")
4518        interest(1)
4519        initial("INVALID")
4520        menu(menuAlarmSevr)
4521        prompt("Undefined Alarm Sevrty")
4522    }
4523    field(TIME, DBF_NOACCESS) {
4524        interest(2)
4525        extra("epicsTimeStamp      time")
4526        special(SPC_NOMOD)
4527        prompt("Time")
4528    }
4529    field(UTAG, DBF_UINT64) {
4530        interest(3)
4531        special(SPC_NOMOD)
4532        prompt("Time Tag")
4533    }
4534    field(FLNK, DBF_FWDLINK) {
4535        promptgroup("20 - Scan")
4536        interest(1)
4537        prompt("Forward Process Link")
4538    }
4539    field(INP, DBF_INLINK) {
4540        promptgroup("40 - Input")
4541        interest(1)
4542        prompt("Input Specification")
4543    }
4544    field(VAL, DBF_ENUM) {
```

```
4545        promptgroup("40 - Input")
4546        asl(ASL0)
4547        pp(TRUE)
4548        prompt("Current Value")
4549    }
4550    field(ZSV, DBF_MENU) {
4551        promptgroup("70 - Alarm")
4552        interest(1)
4553        menu(menuAlarmSevr)
4554        pp(TRUE)
4555        prompt("Zero Error Severity")
4556    }
4557    field(OSV, DBF_MENU) {
4558        promptgroup("70 - Alarm")
4559        interest(1)
4560        menu(menuAlarmSevr)
4561        pp(TRUE)
4562        prompt("One Error Severity")
4563    }
4564    field(COSV, DBF_MENU) {
4565        promptgroup("70 - Alarm")
4566        interest(1)
4567        menu(menuAlarmSevr)
4568        pp(TRUE)
4569        prompt("Change of State Svr")
4570    }
4571    field(ZNAM, DBF_STRING) {
4572        promptgroup("80 - Display")
4573        prompt("Zero Name")
4574        prop(YES)
4575        size(26)
4576        interest(1)
4577        pp(TRUE)
4578    }
4579    field(ONAM, DBF_STRING) {
4580        promptgroup("80 - Display")
4581        prompt("One Name")
4582        prop(YES)
4583        size(26)
4584        interest(1)
4585        pp(TRUE)
4586    }
4587    field(RVAL, DBF_ULONG) {
4588        pp(TRUE)
4589        prompt("Raw Value")
4590    }
4591    field(ORAW, DBF_ULONG) {
4592        interest(3)
4593        special(SPC_NOMOD)
4594        prompt("prev Raw Value")
4595    }
4596    field(MASK, DBF_ULONG) {
4597        interest(1)
4598        special(SPC_NOMOD)
4599        prompt("Hardware Mask")
4600    }
4601    field(LALM, DBF_USHORT) {
4602        interest(3)
4603        special(SPC_NOMOD)
4604        prompt("Last Value Alarmed")
4605    }
4606    field(MLST, DBF_USHORT) {
4607        interest(3)
4608        special(SPC_NOMOD)
4609        prompt("Last Value Monitored")
4610    }
4611    field(AFTC, DBF_DOUBLE) {
4612        promptgroup("70 - Alarm")
```

```
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(AFVL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Alarm Filter Value")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Input Link")
    }
    field(SVAL, DBF_ULONG) {
        prompt("Simulation Value")
    }
    field(SIML, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Mode Link")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuSimm)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(bi, CONSTANT, devBiSoft, "Soft Channel")
device(bi, CONSTANT, devBiSoftRaw, "Raw Soft Channel")
device(bi, CONSTANT, devBiSoftCallback, "Async Soft Channel")
device(bi, INST_IO, devBiDbState, "Db State")
device(bi, INST_IO, devPSCConnectedBi, "PSC Ctrl Connected")
device(bi, INST_IO, devPSCOnConnectBi, "PSC On Connect")
device(bi, INST_IO, devPSCRegBi, "PSC Reg")
device(bi, INST_IO, devPSCRegRBBi, "PSC RB Reg")
recordtype(bo) {
```

```
%#include "epicsTypes.h"
%#include "link.h"
%#include "epicsMutex.h"
%#include "ellLib.h"
%#include "devSup.h"
%#include "epicsTime.h"
%
%/* Declare Device Support Entry Table */
%struct boRecord;
%typedef struct bodset {
%    dset common; /*init_record returns:(0,2)=>(success,success no convert*/
%    long (*write_bo)(struct boRecord *prec); /*returns: (-1,0)=>(failure,success)*/
%} bodset;
%#define HAS_bodset
%
%#include "callback.h"
field(NAME, DBF_STRING) {
    size(61)
    special(SPC_NOMOD)
    prompt("Record Name")
}
field(DESC, DBF_STRING) {
    size(41)
    promptgroup("10 - Common")
    prompt("Descriptor")
}
field(ASG, DBF_STRING) {
    size(29)
    promptgroup("10 - Common")
    special(SPC_AS)
    prompt("Access Security Group")
}
field(SCAN, DBF_MENU) {
    promptgroup("20 - Scan")
    interest(1)
    menu(menuScan)
    special(SPC_SCAN)
    prompt("Scan Mechanism")
}
field(PINI, DBF_MENU) {
    promptgroup("20 - Scan")
    interest(1)
    menu(menuPini)
    prompt("Process at iocInit")
}
field(PHAS, DBF_SHORT) {
    promptgroup("20 - Scan")
    interest(1)
    special(SPC_SCAN)
    prompt("Scan Phase")
}
field(EVNT, DBF_STRING) {
    size(40)
    promptgroup("20 - Scan")
    interest(1)
    special(SPC_SCAN)
    prompt("Event Name")
}
field(TSE, DBF_SHORT) {
    promptgroup("20 - Scan")
    interest(1)
    prompt("Time Stamp Event")
}
field(TSEL, DBF_INLINK) {
    promptgroup("20 - Scan")
    interest(1)
    prompt("Time Stamp Link")
}
```

```
4749    field(DTYP, DBF_DEVICE) {
4750        promptgroup("10 - Common")
4751        interest(1)
4752        prompt("Device Type")
4753    }
4754    field(DISV, DBF_SHORT) {
4755        promptgroup("20 - Scan")
4756        initial("1")
4757        prompt("Disable Value")
4758    }
4759    field(DISA, DBF_SHORT) {
4760        prompt("Disable")
4761    }
4762    field(SDIS, DBF_INLINK) {
4763        promptgroup("20 - Scan")
4764        interest(1)
4765        prompt("Scanning Disable")
4766    }
4767    field(MLOK, DBF_NOACCESS) {
4768        interest(4)
4769        extra("epicsMutexId        mlok")
4770        special(SPC_NOMOD)
4771        prompt("Monitor lock")
4772    }
4773    field(MLIS, DBF_NOACCESS) {
4774        interest(4)
4775        extra("ELLLIST             mlis")
4776        special(SPC_NOMOD)
4777        prompt("Monitor List")
4778    }
4779    field(BKLNK, DBF_NOACCESS) {
4780        interest(4)
4781        extra("ELLLIST             bklnk")
4782        special(SPC_NOMOD)
4783        prompt("Backwards link tracking")
4784    }
4785    field(DISP, DBF_UCHAR) {
4786        promptgroup("10 - Common")
4787        interest(1)
4788        prompt("Disable putField")
4789    }
4790    field(PROC, DBF_UCHAR) {
4791        interest(3)
4792        pp(TRUE)
4793        prompt("Force Processing")
4794    }
4795    field(STAT, DBF_MENU) {
4796        initial("UDF")
4797        menu(menuAlarmStat)
4798        special(SPC_NOMOD)
4799        prompt("Alarm Status")
4800    }
4801    field(SEVR, DBF_MENU) {
4802        menu(menuAlarmSevr)
4803        special(SPC_NOMOD)
4804        prompt("Alarm Severity")
4805    }
4806    field(AMSG, DBF_STRING) {
4807        size(40)
4808        special(SPC_NOMOD)
4809        prompt("Alarm Message")
4810    }
4811    field(NSTA, DBF_MENU) {
4812        interest(2)
4813        menu(menuAlarmStat)
4814        special(SPC_NOMOD)
4815        prompt("New Alarm Status")
4816    }
```

```
4817    field(NSEV, DBF_MENU) {
4818        interest(2)
4819        menu(menuAlarmSevr)
4820        special(SPC_NOMOD)
4821        prompt("New Alarm Severity")
4822    }
4823    field(NAMSG, DBF_STRING) {
4824        size(40)
4825        special(SPC_NOMOD)
4826        prompt("New Alarm Message")
4827    }
4828    field(ACKS, DBF_MENU) {
4829        interest(2)
4830        menu(menuAlarmSevr)
4831        special(SPC_NOMOD)
4832        prompt("Alarm Ack Severity")
4833    }
4834    field(ACKT, DBF_MENU) {
4835        promptgroup("70 - Alarm")
4836        interest(2)
4837        initial("YES")
4838        menu(menuYesNo)
4839        special(SPC_NOMOD)
4840        prompt("Alarm Ack Transient")
4841    }
4842    field(DISS, DBF_MENU) {
4843        promptgroup("70 - Alarm")
4844        interest(1)
4845        menu(menuAlarmSevr)
4846        prompt("Disable Alarm Sevrty")
4847    }
4848    field(LCNT, DBF_UCHAR) {
4849        interest(2)
4850        special(SPC_NOMOD)
4851        prompt("Lock Count")
4852    }
4853    field(PACT, DBF_UCHAR) {
4854        interest(1)
4855        special(SPC_NOMOD)
4856        prompt("Record active")
4857    }
4858    field(PUTF, DBF_UCHAR) {
4859        interest(1)
4860        special(SPC_NOMOD)
4861        prompt("dbPutField process")
4862    }
4863    field(RPRO, DBF_UCHAR) {
4864        interest(1)
4865        special(SPC_NOMOD)
4866        prompt("Reprocess ")
4867    }
4868    field(ASP, DBF_NOACCESS) {
4869        interest(4)
4870        extra("struct asgMember    *asp")
4871        special(SPC_NOMOD)
4872        prompt("Access Security Pvt")
4873    }
4874    field(PPN, DBF_NOACCESS) {
4875        interest(4)
4876        extra("struct processNotify *ppn")
4877        special(SPC_NOMOD)
4878        prompt("pprocessNotify")
4879    }
4880    field(PPNR, DBF_NOACCESS) {
4881        interest(4)
4882        extra("struct processNotifyRecord *ppnr")
4883        special(SPC_NOMOD)
4884        prompt("pprocessNotifyRecord")
```

```
4885    }
4886    field(SPVT, DBF_NOACCESS) {
4887        interest(4)
4888        extra("struct scan_element *spvt")
4889        special(SPC_NOMOD)
4890        prompt("Scan Private")
4891    }
4892    field(RSET, DBF_NOACCESS) {
4893        interest(4)
4894        extra("struct typed_rset   *rset")
4895        special(SPC_NOMOD)
4896        prompt("Address of RSET")
4897    }
4898    field(DSET, DBF_NOACCESS) {
4899        interest(4)
4900        extra("unambiguous_dset    *dset")
4901        special(SPC_NOMOD)
4902        prompt("DSET address")
4903    }
4904    field(DPVT, DBF_NOACCESS) {
4905        interest(4)
4906        extra("void                *dpvt")
4907        special(SPC_NOMOD)
4908        prompt("Device Private")
4909    }
4910    field(RDES, DBF_NOACCESS) {
4911        interest(4)
4912        extra("struct dbRecordType *rdes")
4913        special(SPC_NOMOD)
4914        prompt("Address of dbRecordType")
4915    }
4916    field(LSET, DBF_NOACCESS) {
4917        interest(4)
4918        extra("struct lockRecord   *lset")
4919        special(SPC_NOMOD)
4920        prompt("Lock Set")
4921    }
4922    field(PRIO, DBF_MENU) {
4923        promptgroup("20 - Scan")
4924        interest(1)
4925        menu(menuPriority)
4926        special(SPC_SCAN)
4927        prompt("Scheduling Priority")
4928    }
4929    field(TPRO, DBF_UCHAR) {
4930        prompt("Trace Processing")
4931    }
4932    field(BKPT, DBF_NOACCESS) {
4933        interest(1)
4934        extra("epicsUInt8          bkpt")
4935        special(SPC_NOMOD)
4936        prompt("Break Point")
4937    }
4938    field(UDF, DBF_UCHAR) {
4939        promptgroup("10 - Common")
4940        interest(1)
4941        initial("1")
4942        pp(TRUE)
4943        prompt("Undefined")
4944    }
4945    field(UDFS, DBF_MENU) {
4946        promptgroup("70 - Alarm")
4947        interest(1)
4948        initial("INVALID")
4949        menu(menuAlarmSevr)
4950        prompt("Undefined Alarm Sevrty")
4951    }
4952    field(TIME, DBF_NOACCESS) {
```

```
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_ENUM) {
        promptgroup("50 - Output")
        asl(ASL0)
        pp(TRUE)
        prompt("Current Value")
    }
    field(OMSL, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        menu(menuOmsl)
        prompt("Output Mode Select")
    }
    field(DOL, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Desired Output Link")
    }
    field(OUT, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Specification")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("30 - Action")
        interest(1)
        prompt("Seconds to Hold High")
    }
    field(ZNAM, DBF_STRING) {
        promptgroup("80 - Display")
        prompt("Zero Name")
        prop(YES)
        size(26)
        interest(1)
        pp(TRUE)
    }
    field(ONAM, DBF_STRING) {
        promptgroup("80 - Display")
        prompt("One Name")
        prop(YES)
        size(26)
        interest(1)
        pp(TRUE)
    }
    field(RVAL, DBF_ULONG) {
        pp(TRUE)
        prompt("Raw Value")
    }
    field(ORAW, DBF_ULONG) {
        interest(3)
        special(SPC_NOMOD)
        prompt("prev Raw Value")
    }
    field(MASK, DBF_ULONG) {
```

```
5021        interest(1)
5022        special(SPC_NOMOD)
5023        prompt("Hardware Mask")
5024    }
5025    field(RPVT, DBF_NOACCESS) {
5026        interest(4)
5027        extra("void *  rpvt")
5028        special(SPC_NOMOD)
5029        prompt("Record Private")
5030    }
5031    field(WDPT, DBF_NOACCESS) {
5032        interest(4)
5033        extra("void *   wdpt")
5034        special(SPC_NOMOD)
5035        prompt("Watch Dog Timer ID")
5036    }
5037    field(ZSV, DBF_MENU) {
5038        promptgroup("70 - Alarm")
5039        interest(1)
5040        menu(menuAlarmSevr)
5041        pp(TRUE)
5042        prompt("Zero Error Severity")
5043    }
5044    field(OSV, DBF_MENU) {
5045        promptgroup("70 - Alarm")
5046        interest(1)
5047        menu(menuAlarmSevr)
5048        pp(TRUE)
5049        prompt("One Error Severity")
5050    }
5051    field(COSV, DBF_MENU) {
5052        promptgroup("70 - Alarm")
5053        interest(1)
5054        menu(menuAlarmSevr)
5055        pp(TRUE)
5056        prompt("Change of State Sevr")
5057    }
5058    field(RBV, DBF_ULONG) {
5059        special(SPC_NOMOD)
5060        prompt("Readback Value")
5061    }
5062    field(ORBV, DBF_ULONG) {
5063        interest(3)
5064        special(SPC_NOMOD)
5065        prompt("Prev Readback Value")
5066    }
5067    field(MLST, DBF_USHORT) {
5068        interest(3)
5069        special(SPC_NOMOD)
5070        prompt("Last Value Monitored")
5071    }
5072    field(LALM, DBF_USHORT) {
5073        interest(3)
5074        special(SPC_NOMOD)
5075        prompt("Last Value Alarmed")
5076    }
5077    field(SIOL, DBF_OUTLINK) {
5078        promptgroup("90 - Simulate")
5079        interest(1)
5080        prompt("Simulation Output Link")
5081    }
5082    field(SIML, DBF_INLINK) {
5083        promptgroup("90 - Simulate")
5084        interest(1)
5085        prompt("Simulation Mode Link")
5086    }
5087    field(SIMM, DBF_MENU) {
5088        interest(1)
```

```
        menu(menuSimm)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID outpt action")
    }
    field(IVOV, DBF_USHORT) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
}
device(bo, CONSTANT, devBoSoft, "Soft Channel")
device(bo, CONSTANT, devBoSoftRaw, "Raw Soft Channel")
device(bo, CONSTANT, devBoSoftCallback, "Async Soft Channel")
device(bo, INST_IO, devBoGeneralTime, "General Time")
device(bo, INST_IO, devBoDbState, "Db State")
device(bo, INST_IO, devPSCSendAllBo, "PSC Ctrl Send All")
device(bo, INST_IO, devPSCForceReConn, "PSC Ctrl Reconnect")
device(bo, INST_IO, devPSCRegBo, "PSC Reg")
device(bo, INST_IO, devPSCSingleU32Bo, "PSC Single U32")
recordtype(calc) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %#include "postfix.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
```

```
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
```

```
5225        extra("epicsMutexId        mlok")
5226        special(SPC_NOMOD)
5227        prompt("Monitor lock")
5228    }
5229    field(MLIS, DBF_NOACCESS) {
5230        interest(4)
5231        extra("ELLLIST             mlis")
5232        special(SPC_NOMOD)
5233        prompt("Monitor List")
5234    }
5235    field(BKLNK, DBF_NOACCESS) {
5236        interest(4)
5237        extra("ELLLIST             bklnk")
5238        special(SPC_NOMOD)
5239        prompt("Backwards link tracking")
5240    }
5241    field(DISP, DBF_UCHAR) {
5242        promptgroup("10 - Common")
5243        interest(1)
5244        prompt("Disable putField")
5245    }
5246    field(PROC, DBF_UCHAR) {
5247        interest(3)
5248        pp(TRUE)
5249        prompt("Force Processing")
5250    }
5251    field(STAT, DBF_MENU) {
5252        initial("UDF")
5253        menu(menuAlarmStat)
5254        special(SPC_NOMOD)
5255        prompt("Alarm Status")
5256    }
5257    field(SEVR, DBF_MENU) {
5258        menu(menuAlarmSevr)
5259        special(SPC_NOMOD)
5260        prompt("Alarm Severity")
5261    }
5262    field(AMSG, DBF_STRING) {
5263        size(40)
5264        special(SPC_NOMOD)
5265        prompt("Alarm Message")
5266    }
5267    field(NSTA, DBF_MENU) {
5268        interest(2)
5269        menu(menuAlarmStat)
5270        special(SPC_NOMOD)
5271        prompt("New Alarm Status")
5272    }
5273    field(NSEV, DBF_MENU) {
5274        interest(2)
5275        menu(menuAlarmSevr)
5276        special(SPC_NOMOD)
5277        prompt("New Alarm Severity")
5278    }
5279    field(NAMSG, DBF_STRING) {
5280        size(40)
5281        special(SPC_NOMOD)
5282        prompt("New Alarm Message")
5283    }
5284    field(ACKS, DBF_MENU) {
5285        interest(2)
5286        menu(menuAlarmSevr)
5287        special(SPC_NOMOD)
5288        prompt("Alarm Ack Severity")
5289    }
5290    field(ACKT, DBF_MENU) {
5291        promptgroup("70 - Alarm")
5292        interest(2)
```

```
5293        initial("YES")
5294        menu(menuYesNo)
5295        special(SPC_NOMOD)
5296        prompt("Alarm Ack Transient")
5297    }
5298    field(DISS, DBF_MENU) {
5299        promptgroup("70 - Alarm")
5300        interest(1)
5301        menu(menuAlarmSevr)
5302        prompt("Disable Alarm Sevrty")
5303    }
5304    field(LCNT, DBF_UCHAR) {
5305        interest(2)
5306        special(SPC_NOMOD)
5307        prompt("Lock Count")
5308    }
5309    field(PACT, DBF_UCHAR) {
5310        interest(1)
5311        special(SPC_NOMOD)
5312        prompt("Record active")
5313    }
5314    field(PUTF, DBF_UCHAR) {
5315        interest(1)
5316        special(SPC_NOMOD)
5317        prompt("dbPutField process")
5318    }
5319    field(RPRO, DBF_UCHAR) {
5320        interest(1)
5321        special(SPC_NOMOD)
5322        prompt("Reprocess ")
5323    }
5324    field(ASP, DBF_NOACCESS) {
5325        interest(4)
5326        extra("struct asgMember    *asp")
5327        special(SPC_NOMOD)
5328        prompt("Access Security Pvt")
5329    }
5330    field(PPN, DBF_NOACCESS) {
5331        interest(4)
5332        extra("struct processNotify *ppn")
5333        special(SPC_NOMOD)
5334        prompt("pprocessNotify")
5335    }
5336    field(PPNR, DBF_NOACCESS) {
5337        interest(4)
5338        extra("struct processNotifyRecord *ppnr")
5339        special(SPC_NOMOD)
5340        prompt("pprocessNotifyRecord")
5341    }
5342    field(SPVT, DBF_NOACCESS) {
5343        interest(4)
5344        extra("struct scan_element *spvt")
5345        special(SPC_NOMOD)
5346        prompt("Scan Private")
5347    }
5348    field(RSET, DBF_NOACCESS) {
5349        interest(4)
5350        extra("struct typed_rset   *rset")
5351        special(SPC_NOMOD)
5352        prompt("Address of RSET")
5353    }
5354    field(DSET, DBF_NOACCESS) {
5355        interest(4)
5356        extra("unambiguous_dset    *dset")
5357        special(SPC_NOMOD)
5358        prompt("DSET address")
5359    }
5360    field(DPVT, DBF_NOACCESS) {
```

```
5361        interest(4)
5362        extra("void                *dpvt")
5363        special(SPC_NOMOD)
5364        prompt("Device Private")
5365    }
5366    field(RDES, DBF_NOACCESS) {
5367        interest(4)
5368        extra("struct dbRecordType *rdes")
5369        special(SPC_NOMOD)
5370        prompt("Address of dbRecordType")
5371    }
5372    field(LSET, DBF_NOACCESS) {
5373        interest(4)
5374        extra("struct lockRecord   *lset")
5375        special(SPC_NOMOD)
5376        prompt("Lock Set")
5377    }
5378    field(PRIO, DBF_MENU) {
5379        promptgroup("20 - Scan")
5380        interest(1)
5381        menu(menuPriority)
5382        special(SPC_SCAN)
5383        prompt("Scheduling Priority")
5384    }
5385    field(TPRO, DBF_UCHAR) {
5386        prompt("Trace Processing")
5387    }
5388    field(BKPT, DBF_NOACCESS) {
5389        interest(1)
5390        extra("epicsUInt8          bkpt")
5391        special(SPC_NOMOD)
5392        prompt("Break Point")
5393    }
5394    field(UDF, DBF_UCHAR) {
5395        promptgroup("10 - Common")
5396        interest(1)
5397        initial("1")
5398        pp(TRUE)
5399        prompt("Undefined")
5400    }
5401    field(UDFS, DBF_MENU) {
5402        promptgroup("70 - Alarm")
5403        interest(1)
5404        initial("INVALID")
5405        menu(menuAlarmSevr)
5406        prompt("Undefined Alarm Sevrty")
5407    }
5408    field(TIME, DBF_NOACCESS) {
5409        interest(2)
5410        extra("epicsTimeStamp      time")
5411        special(SPC_NOMOD)
5412        prompt("Time")
5413    }
5414    field(UTAG, DBF_UINT64) {
5415        interest(3)
5416        special(SPC_NOMOD)
5417        prompt("Time Tag")
5418    }
5419    field(FLNK, DBF_FWDLINK) {
5420        promptgroup("20 - Scan")
5421        interest(1)
5422        prompt("Forward Process Link")
5423    }
5424    field(VAL, DBF_DOUBLE) {
5425        promptgroup("50 - Output")
5426        asl(ASL0)
5427        prompt("Result")
5428    }
```

```
    field(CALC, DBF_STRING) {
        size(160)
        promptgroup("30 - Action")
        initial("0")
        pp(TRUE)
        special(SPC_CALC)
        prompt("Calculation")
    }
    field(INPA, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input A")
    }
    field(INPB, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input B")
    }
    field(INPC, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input C")
    }
    field(INPD, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input D")
    }
    field(INPE, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input E")
    }
    field(INPF, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input F")
    }
    field(INPG, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input G")
    }
    field(INPH, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input H")
    }
    field(INPI, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input I")
    }
    field(INPJ, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input J")
    }
    field(INPK, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input K")
    }
    field(INPL, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input L")
    }
```

```
    field(INPM, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input M")
    }
    field(INPN, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input N")
    }
    field(INPO, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input O")
    }
    field(INPP, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input P")
    }
    field(INPQ, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input Q")
    }
    field(INPR, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input R")
    }
    field(INPS, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input S")
    }
    field(INPT, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input T")
    }
    field(INPU, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input U")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Rng")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
```

```
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(AFTC, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(AFVL, DBF_DOUBLE) {
```

```
        interest(3)
        special(SPC_NOMOD)
        prompt("Alarm Filter Value")
    }
    field(HYST, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(ADEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(A, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input A")
    }
    field(B, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input B")
    }
    field(C, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input C")
    }
    field(D, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input D")
    }
    field(E, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input E")
    }
    field(F, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input F")
    }
    field(G, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input G")
    }
    field(H, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input H")
    }
    field(I, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input I")
    }
    field(J, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input J")
    }
    field(K, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input K")
    }
    field(L, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input L")
    }
    field(M, DBF_DOUBLE) {
```

```
        pp(TRUE)
        prompt("Value of Input M")
    }
    field(N, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input N")
    }
    field(O, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input O")
    }
    field(P, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input P")
    }
    field(Q, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input Q")
    }
    field(R, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input R")
    }
    field(S, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input S")
    }
    field(T, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input T")
    }
    field(U, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input U")
    }
    field(LA, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of A")
    }
    field(LB, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of B")
    }
    field(LC, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of C")
    }
    field(LD, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of D")
    }
    field(LE, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of E")
    }
    field(LF, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of F")
    }
    field(LG, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
```

```
        prompt("Prev Value of G")
    }
    field(LH, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of H")
    }
    field(LI, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of I")
    }
    field(LJ, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of J")
    }
    field(LK, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of K")
    }
    field(LL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of L")
    }
    field(LM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of M")
    }
    field(LN, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of N")
    }
    field(LO, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of O")
    }
    field(LP, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of P")
    }
    field(LQ, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of Q")
    }
    field(LR, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of R")
    }
    field(LS, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of S")
    }
    field(LT, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of T")
    }
    field(LU, DBF_DOUBLE) {
```

```
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of U")
    }
    field(LALM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(ALST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Val Monitored")
    }
    field(RPCL, DBF_NOACCESS) {
        interest(4)
        extra("char rpcl[INFIX_TO_POSTFIX_SIZE(160)]")
        special(SPC_NOMOD)
        prompt("Reverse Polish Calc")
    }
}
recordtype(calcout) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %/* Declare Device Support Entry Table */
    %struct calcoutRecord;
    %typedef struct calcoutdset {
    %    dset common;
    %    long (*write)(struct calcoutRecord *prec);
    %} calcoutdset;
    %#define HAS_calcoutdset
    %
    %#include "dbScan.h"
    %#include "postfix.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
```

```
5905        interest(1)
5906        menu(menuPini)
5907        prompt("Process at iocInit")
5908    }
5909    field(PHAS, DBF_SHORT) {
5910        promptgroup("20 - Scan")
5911        interest(1)
5912        special(SPC_SCAN)
5913        prompt("Scan Phase")
5914    }
5915    field(EVNT, DBF_STRING) {
5916        size(40)
5917        promptgroup("20 - Scan")
5918        interest(1)
5919        special(SPC_SCAN)
5920        prompt("Event Name")
5921    }
5922    field(TSE, DBF_SHORT) {
5923        promptgroup("20 - Scan")
5924        interest(1)
5925        prompt("Time Stamp Event")
5926    }
5927    field(TSEL, DBF_INLINK) {
5928        promptgroup("20 - Scan")
5929        interest(1)
5930        prompt("Time Stamp Link")
5931    }
5932    field(DTYP, DBF_DEVICE) {
5933        promptgroup("10 - Common")
5934        interest(1)
5935        prompt("Device Type")
5936    }
5937    field(DISV, DBF_SHORT) {
5938        promptgroup("20 - Scan")
5939        initial("1")
5940        prompt("Disable Value")
5941    }
5942    field(DISA, DBF_SHORT) {
5943        prompt("Disable")
5944    }
5945    field(SDIS, DBF_INLINK) {
5946        promptgroup("20 - Scan")
5947        interest(1)
5948        prompt("Scanning Disable")
5949    }
5950    field(MLOK, DBF_NOACCESS) {
5951        interest(4)
5952        extra("epicsMutexId        mlok")
5953        special(SPC_NOMOD)
5954        prompt("Monitor lock")
5955    }
5956    field(MLIS, DBF_NOACCESS) {
5957        interest(4)
5958        extra("ELLLIST             mlis")
5959        special(SPC_NOMOD)
5960        prompt("Monitor List")
5961    }
5962    field(BKLNK, DBF_NOACCESS) {
5963        interest(4)
5964        extra("ELLLIST             bklnk")
5965        special(SPC_NOMOD)
5966        prompt("Backwards link tracking")
5967    }
5968    field(DISP, DBF_UCHAR) {
5969        promptgroup("10 - Common")
5970        interest(1)
5971        prompt("Disable putField")
5972    }
```

```
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
```

```
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Reprocess ")
    }
    field(ASP, DBF_NOACCESS) {
        interest(4)
        extra("struct asgMember    *asp")
        special(SPC_NOMOD)
        prompt("Access Security Pvt")
    }
    field(PPN, DBF_NOACCESS) {
        interest(4)
        extra("struct processNotify *ppn")
        special(SPC_NOMOD)
        prompt("pprocessNotify")
    }
    field(PPNR, DBF_NOACCESS) {
        interest(4)
        extra("struct processNotifyRecord *ppnr")
        special(SPC_NOMOD)
        prompt("pprocessNotifyRecord")
    }
    field(SPVT, DBF_NOACCESS) {
        interest(4)
        extra("struct scan_element *spvt")
        special(SPC_NOMOD)
        prompt("Scan Private")
    }
    field(RSET, DBF_NOACCESS) {
        interest(4)
        extra("struct typed_rset   *rset")
        special(SPC_NOMOD)
        prompt("Address of RSET")
    }
    field(DSET, DBF_NOACCESS) {
        interest(4)
        extra("unambiguous_dset    *dset")
        special(SPC_NOMOD)
        prompt("DSET address")
    }
    field(DPVT, DBF_NOACCESS) {
        interest(4)
        extra("void                *dpvt")
        special(SPC_NOMOD)
        prompt("Device Private")
    }
    field(RDES, DBF_NOACCESS) {
        interest(4)
        extra("struct dbRecordType *rdes")
        special(SPC_NOMOD)
        prompt("Address of dbRecordType")
    }
    field(LSET, DBF_NOACCESS) {
        interest(4)
        extra("struct lockRecord   *lset")
        special(SPC_NOMOD)
        prompt("Lock Set")
    }
    field(PRIO, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
```

```
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(RPVT, DBF_NOACCESS) {
        interest(4)
        extra("struct rpvtStruct *rpvt")
        special(SPC_NOMOD)
        prompt("Record Private")
    }
    field(VAL, DBF_DOUBLE) {
        promptgroup("50 - Output")
        asl(ASL0)
        prompt("Result")
    }
    field(PVAL, DBF_DOUBLE) {
        prompt("Previous Value")
    }
    field(CALC, DBF_STRING) {
        size(160)
        promptgroup("30 - Action")
        initial("0")
        pp(TRUE)
        special(SPC_CALC)
        prompt("Calculation")
    }
    field(CLCV, DBF_LONG) {
        interest(1)
        prompt("CALC Valid")
    }
```

```
    field(INPA, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input A")
    }
    field(INPB, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input B")
    }
    field(INPC, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input C")
    }
    field(INPD, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input D")
    }
    field(INPE, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input E")
    }
    field(INPF, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        special(SPC_MOD)
        prompt("Input F")
    }
    field(INPG, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        special(SPC_MOD)
        prompt("Input G")
    }
    field(INPH, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        special(SPC_MOD)
        prompt("Input H")
    }
    field(INPI, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        special(SPC_MOD)
        prompt("Input I")
    }
    field(INPJ, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        special(SPC_MOD)
        prompt("Input J")
    }
    field(INPK, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        special(SPC_MOD)
        prompt("Input K")
    }
    field(INPL, DBF_INLINK) {
        promptgroup("42 - Input G-L")
```

```
6245        interest(1)
6246        special(SPC_MOD)
6247        prompt("Input L")
6248    }
6249    field(INPM, DBF_INLINK) {
6250        promptgroup("43 - Input M-R")
6251        interest(1)
6252        special(SPC_MOD)
6253        prompt("Input M")
6254    }
6255    field(INPN, DBF_INLINK) {
6256        promptgroup("43 - Input M-R")
6257        interest(1)
6258        special(SPC_MOD)
6259        prompt("Input N")
6260    }
6261    field(INPO, DBF_INLINK) {
6262        promptgroup("43 - Input M-R")
6263        interest(1)
6264        special(SPC_MOD)
6265        prompt("Input O")
6266    }
6267    field(INPP, DBF_INLINK) {
6268        promptgroup("43 - Input M-R")
6269        interest(1)
6270        special(SPC_MOD)
6271        prompt("Input P")
6272    }
6273    field(INPQ, DBF_INLINK) {
6274        promptgroup("43 - Input M-R")
6275        interest(1)
6276        special(SPC_MOD)
6277        prompt("Input Q")
6278    }
6279    field(INPR, DBF_INLINK) {
6280        promptgroup("43 - Input M-R")
6281        interest(1)
6282        special(SPC_MOD)
6283        prompt("Input R")
6284    }
6285    field(INPS, DBF_INLINK) {
6286        promptgroup("44 - Input S-U")
6287        interest(1)
6288        special(SPC_MOD)
6289        prompt("Input S")
6290    }
6291    field(INPT, DBF_INLINK) {
6292        promptgroup("44 - Input S-U")
6293        interest(1)
6294        special(SPC_MOD)
6295        prompt("Input T")
6296    }
6297    field(INPU, DBF_INLINK) {
6298        promptgroup("44 - Input S-U")
6299        interest(1)
6300        special(SPC_MOD)
6301        prompt("Input U")
6302    }
6303    field(OUT, DBF_OUTLINK) {
6304        promptgroup("50 - Output")
6305        interest(1)
6306        special(SPC_MOD)
6307        prompt("Output Specification")
6308    }
6309    field(INAV, DBF_MENU) {
6310        interest(1)
6311        initial("1")
6312        menu(calcoutINAV)
```

```
6313        special(SPC_NOMOD)
6314        prompt("INPA PV Status")
6315    }
6316    field(INBV, DBF_MENU) {
6317        interest(1)
6318        initial("1")
6319        menu(calcoutINAV)
6320        special(SPC_NOMOD)
6321        prompt("INPB PV Status")
6322    }
6323    field(INCV, DBF_MENU) {
6324        interest(1)
6325        initial("1")
6326        menu(calcoutINAV)
6327        special(SPC_NOMOD)
6328        prompt("INPC PV Status")
6329    }
6330    field(INDV, DBF_MENU) {
6331        interest(1)
6332        initial("1")
6333        menu(calcoutINAV)
6334        special(SPC_NOMOD)
6335        prompt("INPD PV Status")
6336    }
6337    field(INEV, DBF_MENU) {
6338        interest(1)
6339        initial("1")
6340        menu(calcoutINAV)
6341        special(SPC_NOMOD)
6342        prompt("INPE PV Status")
6343    }
6344    field(INFV, DBF_MENU) {
6345        interest(1)
6346        initial("1")
6347        menu(calcoutINAV)
6348        special(SPC_NOMOD)
6349        prompt("INPF PV Status")
6350    }
6351    field(INGV, DBF_MENU) {
6352        interest(1)
6353        initial("1")
6354        menu(calcoutINAV)
6355        special(SPC_NOMOD)
6356        prompt("INPG PV Status")
6357    }
6358    field(INHV, DBF_MENU) {
6359        interest(1)
6360        initial("1")
6361        menu(calcoutINAV)
6362        special(SPC_NOMOD)
6363        prompt("INPH PV Status")
6364    }
6365    field(INIV, DBF_MENU) {
6366        interest(1)
6367        initial("1")
6368        menu(calcoutINAV)
6369        special(SPC_NOMOD)
6370        prompt("INPI PV Status")
6371    }
6372    field(INJV, DBF_MENU) {
6373        interest(1)
6374        initial("1")
6375        menu(calcoutINAV)
6376        special(SPC_NOMOD)
6377        prompt("INPJ PV Status")
6378    }
6379    field(INKV, DBF_MENU) {
6380        interest(1)
```

```
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPK PV Status")
    }
    field(INLV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPL PV Status")
    }
    field(INMV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPM PV Status")
    }
    field(INNV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPN PV Status")
    }
    field(INOV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPO PV Status")
    }
    field(INPV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPP PV Status")
    }
    field(INQV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPQ PV Status")
    }
    field(INRV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPR PV Status")
    }
    field(INSV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPS PV Status")
    }
    field(INTV, DBF_MENU) {
        interest(1)
        initial("1")
        menu(calcoutINAV)
        special(SPC_NOMOD)
        prompt("INPT PV Status")
    }
```

```
6449    field(INUV, DBF_MENU) {
6450        interest(1)
6451        initial("1")
6452        menu(calcoutINAV)
6453        special(SPC_NOMOD)
6454        prompt("INPU PV Status")
6455    }
6456    field(OUTV, DBF_MENU) {
6457        interest(1)
6458        menu(calcoutINAV)
6459        special(SPC_NOMOD)
6460        prompt("OUT PV Status")
6461    }
6462    field(OOPT, DBF_MENU) {
6463        promptgroup("50 - Output")
6464        interest(1)
6465        menu(calcoutOOPT)
6466        prompt("Output Execute Opt")
6467    }
6468    field(ODLY, DBF_DOUBLE) {
6469        promptgroup("50 - Output")
6470        interest(1)
6471        asl(ASL0)
6472        prompt("Output Execute Delay")
6473    }
6474    field(DLYA, DBF_USHORT) {
6475        asl(ASL0)
6476        special(SPC_NOMOD)
6477        prompt("Output Delay Active")
6478    }
6479    field(DOPT, DBF_MENU) {
6480        promptgroup("30 - Action")
6481        interest(1)
6482        menu(calcoutDOPT)
6483        prompt("Output Data Opt")
6484    }
6485    field(OCAL, DBF_STRING) {
6486        size(160)
6487        promptgroup("30 - Action")
6488        initial("0")
6489        pp(TRUE)
6490        special(SPC_CALC)
6491        prompt("Output Calculation")
6492    }
6493    field(OCLV, DBF_LONG) {
6494        interest(1)
6495        prompt("OCAL Valid")
6496    }
6497    field(OEVT, DBF_STRING) {
6498        size(40)
6499        promptgroup("30 - Action")
6500        asl(ASL0)
6501        special(SPC_MOD)
6502        prompt("Event To Issue")
6503    }
6504    field(EPVT, DBF_NOACCESS) {
6505        interest(4)
6506        extra("EVENTPVT epvt")
6507        special(SPC_NOMOD)
6508        prompt("Event private")
6509    }
6510    field(IVOA, DBF_MENU) {
6511        promptgroup("50 - Output")
6512        interest(2)
6513        menu(menuIvoa)
6514        prompt("INVALID output action")
6515    }
6516    field(IVOV, DBF_DOUBLE) {
```

```
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Rng")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
```

```
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HYST, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(ADEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(A, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input A")
    }
    field(B, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input B")
    }
    field(C, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input C")
    }
    field(D, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input D")
    }
    field(E, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input E")
    }
    field(F, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input F")
    }
    field(G, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input G")
    }
    field(H, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input H")
    }
```

```
    field(I, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input I")
    }
    field(J, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input J")
    }
    field(K, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input K")
    }
    field(L, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input L")
    }
    field(M, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input M")
    }
    field(N, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input N")
    }
    field(O, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input O")
    }
    field(P, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input P")
    }
    field(Q, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input Q")
    }
    field(R, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input R")
    }
    field(S, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input S")
    }
    field(T, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input T")
    }
    field(U, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input U")
    }
    field(OVAL, DBF_DOUBLE) {
        asl(ASL0)
        prompt("Output Value")
    }
    field(LA, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of A")
    }
    field(LB, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of B")
    }
    field(LC, DBF_DOUBLE) {
        interest(3)
```

```
6721        special(SPC_NOMOD)
6722        prompt("Prev Value of C")
6723    }
6724    field(LD, DBF_DOUBLE) {
6725        interest(3)
6726        special(SPC_NOMOD)
6727        prompt("Prev Value of D")
6728    }
6729    field(LE, DBF_DOUBLE) {
6730        interest(3)
6731        special(SPC_NOMOD)
6732        prompt("Prev Value of E")
6733    }
6734    field(LF, DBF_DOUBLE) {
6735        interest(3)
6736        special(SPC_NOMOD)
6737        prompt("Prev Value of F")
6738    }
6739    field(LG, DBF_DOUBLE) {
6740        interest(3)
6741        special(SPC_NOMOD)
6742        prompt("Prev Value of G")
6743    }
6744    field(LH, DBF_DOUBLE) {
6745        interest(3)
6746        special(SPC_NOMOD)
6747        prompt("Prev Value of H")
6748    }
6749    field(LI, DBF_DOUBLE) {
6750        interest(3)
6751        special(SPC_NOMOD)
6752        prompt("Prev Value of I")
6753    }
6754    field(LJ, DBF_DOUBLE) {
6755        interest(3)
6756        special(SPC_NOMOD)
6757        prompt("Prev Value of J")
6758    }
6759    field(LK, DBF_DOUBLE) {
6760        interest(3)
6761        special(SPC_NOMOD)
6762        prompt("Prev Value of K")
6763    }
6764    field(LL, DBF_DOUBLE) {
6765        interest(3)
6766        special(SPC_NOMOD)
6767        prompt("Prev Value of L")
6768    }
6769    field(LM, DBF_DOUBLE) {
6770        interest(3)
6771        special(SPC_NOMOD)
6772        prompt("Prev Value of M")
6773    }
6774    field(LN, DBF_DOUBLE) {
6775        interest(3)
6776        special(SPC_NOMOD)
6777        prompt("Prev Value of N")
6778    }
6779    field(LO, DBF_DOUBLE) {
6780        interest(3)
6781        special(SPC_NOMOD)
6782        prompt("Prev Value of O")
6783    }
6784    field(LP, DBF_DOUBLE) {
6785        interest(3)
6786        special(SPC_NOMOD)
6787        prompt("Prev Value of P")
6788    }
```

```
    field(LQ, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of Q")
    }
    field(LR, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of R")
    }
    field(LS, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of S")
    }
    field(LT, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of T")
    }
    field(LU, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of U")
    }
    field(POVL, DBF_DOUBLE) {
        asl(ASL0)
        prompt("Prev Value of OVAL")
    }
    field(LALM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(ALST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Val Monitored")
    }
    field(RPCL, DBF_NOACCESS) {
        interest(4)
        extra("char rpcl[INFIX_TO_POSTFIX_SIZE(160)]")
        special(SPC_NOMOD)
        prompt("Reverse Polish Calc")
    }
    field(ORPC, DBF_NOACCESS) {
        interest(4)
        extra("char orpc[INFIX_TO_POSTFIX_SIZE(160)]")
        special(SPC_NOMOD)
        prompt("Reverse Polish OCalc")
    }
}
device(calcout, CONSTANT, devCalcoutSoft, "Soft Channel")
device(calcout, CONSTANT, devCalcoutSoftCallback, "Async Soft Channel")
recordtype(compress) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
```

```
6857        special(SPC_NOMOD)
6858        prompt("Record Name")
6859    }
6860    field(DESC, DBF_STRING) {
6861        size(41)
6862        promptgroup("10 - Common")
6863        prompt("Descriptor")
6864    }
6865    field(ASG, DBF_STRING) {
6866        size(29)
6867        promptgroup("10 - Common")
6868        special(SPC_AS)
6869        prompt("Access Security Group")
6870    }
6871    field(SCAN, DBF_MENU) {
6872        promptgroup("20 - Scan")
6873        interest(1)
6874        menu(menuScan)
6875        special(SPC_SCAN)
6876        prompt("Scan Mechanism")
6877    }
6878    field(PINI, DBF_MENU) {
6879        promptgroup("20 - Scan")
6880        interest(1)
6881        menu(menuPini)
6882        prompt("Process at iocInit")
6883    }
6884    field(PHAS, DBF_SHORT) {
6885        promptgroup("20 - Scan")
6886        interest(1)
6887        special(SPC_SCAN)
6888        prompt("Scan Phase")
6889    }
6890    field(EVNT, DBF_STRING) {
6891        size(40)
6892        promptgroup("20 - Scan")
6893        interest(1)
6894        special(SPC_SCAN)
6895        prompt("Event Name")
6896    }
6897    field(TSE, DBF_SHORT) {
6898        promptgroup("20 - Scan")
6899        interest(1)
6900        prompt("Time Stamp Event")
6901    }
6902    field(TSEL, DBF_INLINK) {
6903        promptgroup("20 - Scan")
6904        interest(1)
6905        prompt("Time Stamp Link")
6906    }
6907    field(DTYP, DBF_DEVICE) {
6908        promptgroup("10 - Common")
6909        interest(1)
6910        prompt("Device Type")
6911    }
6912    field(DISV, DBF_SHORT) {
6913        promptgroup("20 - Scan")
6914        initial("1")
6915        prompt("Disable Value")
6916    }
6917    field(DISA, DBF_SHORT) {
6918        prompt("Disable")
6919    }
6920    field(SDIS, DBF_INLINK) {
6921        promptgroup("20 - Scan")
6922        interest(1)
6923        prompt("Scanning Disable")
6924    }
```

```
6925    field(MLOK, DBF_NOACCESS) {
6926        interest(4)
6927        extra("epicsMutexId        mlok")
6928        special(SPC_NOMOD)
6929        prompt("Monitor lock")
6930    }
6931    field(MLIS, DBF_NOACCESS) {
6932        interest(4)
6933        extra("ELLLIST             mlis")
6934        special(SPC_NOMOD)
6935        prompt("Monitor List")
6936    }
6937    field(BKLNK, DBF_NOACCESS) {
6938        interest(4)
6939        extra("ELLLIST             bklnk")
6940        special(SPC_NOMOD)
6941        prompt("Backwards link tracking")
6942    }
6943    field(DISP, DBF_UCHAR) {
6944        promptgroup("10 - Common")
6945        interest(1)
6946        prompt("Disable putField")
6947    }
6948    field(PROC, DBF_UCHAR) {
6949        interest(3)
6950        pp(TRUE)
6951        prompt("Force Processing")
6952    }
6953    field(STAT, DBF_MENU) {
6954        initial("UDF")
6955        menu(menuAlarmStat)
6956        special(SPC_NOMOD)
6957        prompt("Alarm Status")
6958    }
6959    field(SEVR, DBF_MENU) {
6960        menu(menuAlarmSevr)
6961        special(SPC_NOMOD)
6962        prompt("Alarm Severity")
6963    }
6964    field(AMSG, DBF_STRING) {
6965        size(40)
6966        special(SPC_NOMOD)
6967        prompt("Alarm Message")
6968    }
6969    field(NSTA, DBF_MENU) {
6970        interest(2)
6971        menu(menuAlarmStat)
6972        special(SPC_NOMOD)
6973        prompt("New Alarm Status")
6974    }
6975    field(NSEV, DBF_MENU) {
6976        interest(2)
6977        menu(menuAlarmSevr)
6978        special(SPC_NOMOD)
6979        prompt("New Alarm Severity")
6980    }
6981    field(NAMSG, DBF_STRING) {
6982        size(40)
6983        special(SPC_NOMOD)
6984        prompt("New Alarm Message")
6985    }
6986    field(ACKS, DBF_MENU) {
6987        interest(2)
6988        menu(menuAlarmSevr)
6989        special(SPC_NOMOD)
6990        prompt("Alarm Ack Severity")
6991    }
6992    field(ACKT, DBF_MENU) {
```

```
6993        promptgroup("70 - Alarm")
6994        interest(2)
6995        initial("YES")
6996        menu(menuYesNo)
6997        special(SPC_NOMOD)
6998        prompt("Alarm Ack Transient")
6999    }
7000    field(DISS, DBF_MENU) {
7001        promptgroup("70 - Alarm")
7002        interest(1)
7003        menu(menuAlarmSevr)
7004        prompt("Disable Alarm Sevrty")
7005    }
7006    field(LCNT, DBF_UCHAR) {
7007        interest(2)
7008        special(SPC_NOMOD)
7009        prompt("Lock Count")
7010    }
7011    field(PACT, DBF_UCHAR) {
7012        interest(1)
7013        special(SPC_NOMOD)
7014        prompt("Record active")
7015    }
7016    field(PUTF, DBF_UCHAR) {
7017        interest(1)
7018        special(SPC_NOMOD)
7019        prompt("dbPutField process")
7020    }
7021    field(RPRO, DBF_UCHAR) {
7022        interest(1)
7023        special(SPC_NOMOD)
7024        prompt("Reprocess ")
7025    }
7026    field(ASP, DBF_NOACCESS) {
7027        interest(4)
7028        extra("struct asgMember    *asp")
7029        special(SPC_NOMOD)
7030        prompt("Access Security Pvt")
7031    }
7032    field(PPN, DBF_NOACCESS) {
7033        interest(4)
7034        extra("struct processNotify *ppn")
7035        special(SPC_NOMOD)
7036        prompt("pprocessNotify")
7037    }
7038    field(PPNR, DBF_NOACCESS) {
7039        interest(4)
7040        extra("struct processNotifyRecord *ppnr")
7041        special(SPC_NOMOD)
7042        prompt("pprocessNotifyRecord")
7043    }
7044    field(SPVT, DBF_NOACCESS) {
7045        interest(4)
7046        extra("struct scan_element *spvt")
7047        special(SPC_NOMOD)
7048        prompt("Scan Private")
7049    }
7050    field(RSET, DBF_NOACCESS) {
7051        interest(4)
7052        extra("struct typed_rset   *rset")
7053        special(SPC_NOMOD)
7054        prompt("Address of RSET")
7055    }
7056    field(DSET, DBF_NOACCESS) {
7057        interest(4)
7058        extra("unambiguous_dset    *dset")
7059        special(SPC_NOMOD)
7060        prompt("DSET address")
```

```
7061    }
7062    field(DPVT, DBF_NOACCESS) {
7063        interest(4)
7064        extra("void                *dpvt")
7065        special(SPC_NOMOD)
7066        prompt("Device Private")
7067    }
7068    field(RDES, DBF_NOACCESS) {
7069        interest(4)
7070        extra("struct dbRecordType *rdes")
7071        special(SPC_NOMOD)
7072        prompt("Address of dbRecordType")
7073    }
7074    field(LSET, DBF_NOACCESS) {
7075        interest(4)
7076        extra("struct lockRecord   *lset")
7077        special(SPC_NOMOD)
7078        prompt("Lock Set")
7079    }
7080    field(PRIO, DBF_MENU) {
7081        promptgroup("20 - Scan")
7082        interest(1)
7083        menu(menuPriority)
7084        special(SPC_SCAN)
7085        prompt("Scheduling Priority")
7086    }
7087    field(TPRO, DBF_UCHAR) {
7088        prompt("Trace Processing")
7089    }
7090    field(BKPT, DBF_NOACCESS) {
7091        interest(1)
7092        extra("epicsUInt8          bkpt")
7093        special(SPC_NOMOD)
7094        prompt("Break Point")
7095    }
7096    field(UDF, DBF_UCHAR) {
7097        promptgroup("10 - Common")
7098        interest(1)
7099        initial("1")
7100        pp(TRUE)
7101        prompt("Undefined")
7102    }
7103    field(UDFS, DBF_MENU) {
7104        promptgroup("70 - Alarm")
7105        interest(1)
7106        initial("INVALID")
7107        menu(menuAlarmSevr)
7108        prompt("Undefined Alarm Sevrty")
7109    }
7110    field(TIME, DBF_NOACCESS) {
7111        interest(2)
7112        extra("epicsTimeStamp      time")
7113        special(SPC_NOMOD)
7114        prompt("Time")
7115    }
7116    field(UTAG, DBF_UINT64) {
7117        interest(3)
7118        special(SPC_NOMOD)
7119        prompt("Time Tag")
7120    }
7121    field(FLNK, DBF_FWDLINK) {
7122        promptgroup("20 - Scan")
7123        interest(1)
7124        prompt("Forward Process Link")
7125    }
7126    field(VAL, DBF_NOACCESS) {
7127        extra("void *       val")
7128        asl(ASL0)
```

```
7129        pp(TRUE)
7130        special(SPC_DBADDR)
7131        prompt("Value")
7132    }
7133    field(INP, DBF_INLINK) {
7134        promptgroup("40 - Input")
7135        interest(1)
7136        prompt("Input Specification")
7137    }
7138    field(RES, DBF_SHORT) {
7139        interest(3)
7140        asl(ASL0)
7141        special(SPC_RESET)
7142        prompt("Reset")
7143    }
7144    field(ALG, DBF_MENU) {
7145        promptgroup("30 - Action")
7146        interest(1)
7147        menu(compressALG)
7148        special(SPC_RESET)
7149        prompt("Compression Algorithm")
7150    }
7151    field(PBUF, DBF_MENU) {
7152        promptgroup("30 - Action")
7153        interest(1)
7154        initial("NO")
7155        menu(menuYesNo)
7156        special(SPC_RESET)
7157        prompt("Use Partial buffers")
7158    }
7159    field(BALG, DBF_MENU) {
7160        promptgroup("30 - Action")
7161        interest(1)
7162        menu(bufferingALG)
7163        special(SPC_RESET)
7164        prompt("Buffering Algorithm")
7165    }
7166    field(NSAM, DBF_ULONG) {
7167        promptgroup("30 - Action")
7168        interest(1)
7169        initial("1")
7170        special(SPC_NOMOD)
7171        prompt("Number of Values")
7172    }
7173    field(N, DBF_ULONG) {
7174        promptgroup("30 - Action")
7175        interest(1)
7176        initial("1")
7177        special(SPC_RESET)
7178        prompt("N to 1 Compression")
7179    }
7180    field(IHIL, DBF_DOUBLE) {
7181        promptgroup("30 - Action")
7182        interest(1)
7183        prompt("Init High Interest Lim")
7184    }
7185    field(ILIL, DBF_DOUBLE) {
7186        promptgroup("30 - Action")
7187        interest(1)
7188        prompt("Init Low Interest Lim")
7189    }
7190    field(HOPR, DBF_DOUBLE) {
7191        promptgroup("80 - Display")
7192        interest(1)
7193        prop(YES)
7194        prompt("High Operating Range")
7195    }
7196    field(LOPR, DBF_DOUBLE) {
```

```
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(OFF, DBF_ULONG) {
        special(SPC_NOMOD)
        prompt("Offset")
    }
    field(NUSE, DBF_ULONG) {
        special(SPC_NOMOD)
        prompt("Number Used")
    }
    field(OUSE, DBF_ULONG) {
        special(SPC_NOMOD)
        prompt("Old Number Used")
    }
    field(BPTR, DBF_NOACCESS) {
        interest(4)
        extra("double       *bptr")
        special(SPC_NOMOD)
        prompt("Buffer Pointer")
    }
    field(SPTR, DBF_NOACCESS) {
        interest(4)
        extra("double       *sptr")
        special(SPC_NOMOD)
        prompt("Summing Buffer Ptr")
    }
    field(WPTR, DBF_NOACCESS) {
        interest(4)
        extra("double       *wptr")
        special(SPC_NOMOD)
        prompt("Working Buffer Ptr")
    }
    field(INPN, DBF_LONG) {
        interest(4)
        special(SPC_NOMOD)
        prompt("Number of elements in Working Buffer")
    }
    field(CVB, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Compress Value Buffer")
    }
    field(INX, DBF_ULONG) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Current number of readings")
    }
}
recordtype(dfanout) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
```

```
7265    %#include "ellLib.h"
7266    %#include "devSup.h"
7267    %#include "epicsTime.h"
7268    field(NAME, DBF_STRING) {
7269        size(61)
7270        special(SPC_NOMOD)
7271        prompt("Record Name")
7272    }
7273    field(DESC, DBF_STRING) {
7274        size(41)
7275        promptgroup("10 - Common")
7276        prompt("Descriptor")
7277    }
7278    field(ASG, DBF_STRING) {
7279        size(29)
7280        promptgroup("10 - Common")
7281        special(SPC_AS)
7282        prompt("Access Security Group")
7283    }
7284    field(SCAN, DBF_MENU) {
7285        promptgroup("20 - Scan")
7286        interest(1)
7287        menu(menuScan)
7288        special(SPC_SCAN)
7289        prompt("Scan Mechanism")
7290    }
7291    field(PINI, DBF_MENU) {
7292        promptgroup("20 - Scan")
7293        interest(1)
7294        menu(menuPini)
7295        prompt("Process at iocInit")
7296    }
7297    field(PHAS, DBF_SHORT) {
7298        promptgroup("20 - Scan")
7299        interest(1)
7300        special(SPC_SCAN)
7301        prompt("Scan Phase")
7302    }
7303    field(EVNT, DBF_STRING) {
7304        size(40)
7305        promptgroup("20 - Scan")
7306        interest(1)
7307        special(SPC_SCAN)
7308        prompt("Event Name")
7309    }
7310    field(TSE, DBF_SHORT) {
7311        promptgroup("20 - Scan")
7312        interest(1)
7313        prompt("Time Stamp Event")
7314    }
7315    field(TSEL, DBF_INLINK) {
7316        promptgroup("20 - Scan")
7317        interest(1)
7318        prompt("Time Stamp Link")
7319    }
7320    field(DTYP, DBF_DEVICE) {
7321        promptgroup("10 - Common")
7322        interest(1)
7323        prompt("Device Type")
7324    }
7325    field(DISV, DBF_SHORT) {
7326        promptgroup("20 - Scan")
7327        initial("1")
7328        prompt("Disable Value")
7329    }
7330    field(DISA, DBF_SHORT) {
7331        prompt("Disable")
7332    }
```

```
7333    field(SDIS, DBF_INLINK) {
7334        promptgroup("20 - Scan")
7335        interest(1)
7336        prompt("Scanning Disable")
7337    }
7338    field(MLOK, DBF_NOACCESS) {
7339        interest(4)
7340        extra("epicsMutexId        mlok")
7341        special(SPC_NOMOD)
7342        prompt("Monitor lock")
7343    }
7344    field(MLIS, DBF_NOACCESS) {
7345        interest(4)
7346        extra("ELLLIST             mlis")
7347        special(SPC_NOMOD)
7348        prompt("Monitor List")
7349    }
7350    field(BKLNK, DBF_NOACCESS) {
7351        interest(4)
7352        extra("ELLLIST             bklnk")
7353        special(SPC_NOMOD)
7354        prompt("Backwards link tracking")
7355    }
7356    field(DISP, DBF_UCHAR) {
7357        promptgroup("10 - Common")
7358        interest(1)
7359        prompt("Disable putField")
7360    }
7361    field(PROC, DBF_UCHAR) {
7362        interest(3)
7363        pp(TRUE)
7364        prompt("Force Processing")
7365    }
7366    field(STAT, DBF_MENU) {
7367        initial("UDF")
7368        menu(menuAlarmStat)
7369        special(SPC_NOMOD)
7370        prompt("Alarm Status")
7371    }
7372    field(SEVR, DBF_MENU) {
7373        menu(menuAlarmSevr)
7374        special(SPC_NOMOD)
7375        prompt("Alarm Severity")
7376    }
7377    field(AMSG, DBF_STRING) {
7378        size(40)
7379        special(SPC_NOMOD)
7380        prompt("Alarm Message")
7381    }
7382    field(NSTA, DBF_MENU) {
7383        interest(2)
7384        menu(menuAlarmStat)
7385        special(SPC_NOMOD)
7386        prompt("New Alarm Status")
7387    }
7388    field(NSEV, DBF_MENU) {
7389        interest(2)
7390        menu(menuAlarmSevr)
7391        special(SPC_NOMOD)
7392        prompt("New Alarm Severity")
7393    }
7394    field(NAMSG, DBF_STRING) {
7395        size(40)
7396        special(SPC_NOMOD)
7397        prompt("New Alarm Message")
7398    }
7399    field(ACKS, DBF_MENU) {
7400        interest(2)
```

```
7401        menu(menuAlarmSevr)
7402        special(SPC_NOMOD)
7403        prompt("Alarm Ack Severity")
7404    }
7405    field(ACKT, DBF_MENU) {
7406        promptgroup("70 - Alarm")
7407        interest(2)
7408        initial("YES")
7409        menu(menuYesNo)
7410        special(SPC_NOMOD)
7411        prompt("Alarm Ack Transient")
7412    }
7413    field(DISS, DBF_MENU) {
7414        promptgroup("70 - Alarm")
7415        interest(1)
7416        menu(menuAlarmSevr)
7417        prompt("Disable Alarm Sevrty")
7418    }
7419    field(LCNT, DBF_UCHAR) {
7420        interest(2)
7421        special(SPC_NOMOD)
7422        prompt("Lock Count")
7423    }
7424    field(PACT, DBF_UCHAR) {
7425        interest(1)
7426        special(SPC_NOMOD)
7427        prompt("Record active")
7428    }
7429    field(PUTF, DBF_UCHAR) {
7430        interest(1)
7431        special(SPC_NOMOD)
7432        prompt("dbPutField process")
7433    }
7434    field(RPRO, DBF_UCHAR) {
7435        interest(1)
7436        special(SPC_NOMOD)
7437        prompt("Reprocess ")
7438    }
7439    field(ASP, DBF_NOACCESS) {
7440        interest(4)
7441        extra("struct asgMember    *asp")
7442        special(SPC_NOMOD)
7443        prompt("Access Security Pvt")
7444    }
7445    field(PPN, DBF_NOACCESS) {
7446        interest(4)
7447        extra("struct processNotify *ppn")
7448        special(SPC_NOMOD)
7449        prompt("pprocessNotify")
7450    }
7451    field(PPNR, DBF_NOACCESS) {
7452        interest(4)
7453        extra("struct processNotifyRecord *ppnr")
7454        special(SPC_NOMOD)
7455        prompt("pprocessNotifyRecord")
7456    }
7457    field(SPVT, DBF_NOACCESS) {
7458        interest(4)
7459        extra("struct scan_element *spvt")
7460        special(SPC_NOMOD)
7461        prompt("Scan Private")
7462    }
7463    field(RSET, DBF_NOACCESS) {
7464        interest(4)
7465        extra("struct typed_rset   *rset")
7466        special(SPC_NOMOD)
7467        prompt("Address of RSET")
7468    }
```

```
7469    field(DSET, DBF_NOACCESS) {
7470        interest(4)
7471        extra("unambiguous_dset    *dset")
7472        special(SPC_NOMOD)
7473        prompt("DSET address")
7474    }
7475    field(DPVT, DBF_NOACCESS) {
7476        interest(4)
7477        extra("void                *dpvt")
7478        special(SPC_NOMOD)
7479        prompt("Device Private")
7480    }
7481    field(RDES, DBF_NOACCESS) {
7482        interest(4)
7483        extra("struct dbRecordType *rdes")
7484        special(SPC_NOMOD)
7485        prompt("Address of dbRecordType")
7486    }
7487    field(LSET, DBF_NOACCESS) {
7488        interest(4)
7489        extra("struct lockRecord   *lset")
7490        special(SPC_NOMOD)
7491        prompt("Lock Set")
7492    }
7493    field(PRIO, DBF_MENU) {
7494        promptgroup("20 - Scan")
7495        interest(1)
7496        menu(menuPriority)
7497        special(SPC_SCAN)
7498        prompt("Scheduling Priority")
7499    }
7500    field(TPRO, DBF_UCHAR) {
7501        prompt("Trace Processing")
7502    }
7503    field(BKPT, DBF_NOACCESS) {
7504        interest(1)
7505        extra("epicsUInt8          bkpt")
7506        special(SPC_NOMOD)
7507        prompt("Break Point")
7508    }
7509    field(UDF, DBF_UCHAR) {
7510        promptgroup("10 - Common")
7511        interest(1)
7512        initial("1")
7513        pp(TRUE)
7514        prompt("Undefined")
7515    }
7516    field(UDFS, DBF_MENU) {
7517        promptgroup("70 - Alarm")
7518        interest(1)
7519        initial("INVALID")
7520        menu(menuAlarmSevr)
7521        prompt("Undefined Alarm Sevrty")
7522    }
7523    field(TIME, DBF_NOACCESS) {
7524        interest(2)
7525        extra("epicsTimeStamp      time")
7526        special(SPC_NOMOD)
7527        prompt("Time")
7528    }
7529    field(UTAG, DBF_UINT64) {
7530        interest(3)
7531        special(SPC_NOMOD)
7532        prompt("Time Tag")
7533    }
7534    field(FLNK, DBF_FWDLINK) {
7535        promptgroup("20 - Scan")
7536        interest(1)
```

```
7537        prompt("Forward Process Link")
7538    }
7539    field(VAL, DBF_DOUBLE) {
7540        promptgroup("40 - Input")
7541        asl(ASL0)
7542        pp(TRUE)
7543        prompt("Desired Output")
7544    }
7545    field(SELM, DBF_MENU) {
7546        promptgroup("30 - Action")
7547        interest(1)
7548        menu(dfanoutSELM)
7549        prompt("Select Mechanism")
7550    }
7551    field(SELN, DBF_USHORT) {
7552        interest(1)
7553        initial("1")
7554        prompt("Link Selection")
7555    }
7556    field(SELL, DBF_INLINK) {
7557        promptgroup("30 - Action")
7558        interest(1)
7559        prompt("Link Selection Loc")
7560    }
7561    field(OUTA, DBF_OUTLINK) {
7562        promptgroup("50 - Output")
7563        interest(1)
7564        prompt("Output Spec A")
7565    }
7566    field(OUTB, DBF_OUTLINK) {
7567        promptgroup("50 - Output")
7568        interest(1)
7569        prompt("Output Spec B")
7570    }
7571    field(OUTC, DBF_OUTLINK) {
7572        promptgroup("50 - Output")
7573        interest(1)
7574        prompt("Output Spec C")
7575    }
7576    field(OUTD, DBF_OUTLINK) {
7577        promptgroup("50 - Output")
7578        interest(1)
7579        prompt("Output Spec D")
7580    }
7581    field(OUTE, DBF_OUTLINK) {
7582        promptgroup("50 - Output")
7583        interest(1)
7584        prompt("Output Spec E")
7585    }
7586    field(OUTF, DBF_OUTLINK) {
7587        promptgroup("50 - Output")
7588        interest(1)
7589        prompt("Output Spec F")
7590    }
7591    field(OUTG, DBF_OUTLINK) {
7592        promptgroup("50 - Output")
7593        interest(1)
7594        prompt("Output Spec G")
7595    }
7596    field(OUTH, DBF_OUTLINK) {
7597        promptgroup("50 - Output")
7598        interest(1)
7599        prompt("Output Spec H")
7600    }
7601    field(OUTI, DBF_OUTLINK) {
7602        promptgroup("50 - Output")
7603        interest(1)
7604        prompt("Output Spec I")
```

```
    }
    field(OUTJ, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec J")
    }
    field(OUTK, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec K")
    }
    field(OUTL, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec L")
    }
    field(OUTM, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec M")
    }
    field(OUTN, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec N")
    }
    field(OUTO, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec O")
    }
    field(OUTP, DBF_OUTLINK) {
        promptgroup("50 - Output")
        interest(1)
        prompt("Output Spec P")
    }
    field(DOL, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Desired Output Link")
    }
    field(OMSL, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        menu(menuOmsl)
        prompt("Output Mode Select")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
```

```
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HYST, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
```

```
    }
    field(ADEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(LALM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(ALST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Val Monitored")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID output action")
    }
    field(IVOV, DBF_DOUBLE) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
}
recordtype(event) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct eventRecord;
    %typedef struct eventdset {
    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
    %    long (*read_event)(struct eventRecord *prec); /*(0)=> success */
    %} eventdset;
    %#define HAS_eventdset
    %
    %#include "dbScan.h"
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
```

```
7809        promptgroup("10 - Common")
7810        special(SPC_AS)
7811        prompt("Access Security Group")
7812    }
7813    field(SCAN, DBF_MENU) {
7814        promptgroup("20 - Scan")
7815        interest(1)
7816        menu(menuScan)
7817        special(SPC_SCAN)
7818        prompt("Scan Mechanism")
7819    }
7820    field(PINI, DBF_MENU) {
7821        promptgroup("20 - Scan")
7822        interest(1)
7823        menu(menuPini)
7824        prompt("Process at iocInit")
7825    }
7826    field(PHAS, DBF_SHORT) {
7827        promptgroup("20 - Scan")
7828        interest(1)
7829        special(SPC_SCAN)
7830        prompt("Scan Phase")
7831    }
7832    field(EVNT, DBF_STRING) {
7833        size(40)
7834        promptgroup("20 - Scan")
7835        interest(1)
7836        special(SPC_SCAN)
7837        prompt("Event Name")
7838    }
7839    field(TSE, DBF_SHORT) {
7840        promptgroup("20 - Scan")
7841        interest(1)
7842        prompt("Time Stamp Event")
7843    }
7844    field(TSEL, DBF_INLINK) {
7845        promptgroup("20 - Scan")
7846        interest(1)
7847        prompt("Time Stamp Link")
7848    }
7849    field(DTYP, DBF_DEVICE) {
7850        promptgroup("10 - Common")
7851        interest(1)
7852        prompt("Device Type")
7853    }
7854    field(DISV, DBF_SHORT) {
7855        promptgroup("20 - Scan")
7856        initial("1")
7857        prompt("Disable Value")
7858    }
7859    field(DISA, DBF_SHORT) {
7860        prompt("Disable")
7861    }
7862    field(SDIS, DBF_INLINK) {
7863        promptgroup("20 - Scan")
7864        interest(1)
7865        prompt("Scanning Disable")
7866    }
7867    field(MLOK, DBF_NOACCESS) {
7868        interest(4)
7869        extra("epicsMutexId        mlok")
7870        special(SPC_NOMOD)
7871        prompt("Monitor lock")
7872    }
7873    field(MLIS, DBF_NOACCESS) {
7874        interest(4)
7875        extra("ELLLIST             mlis")
7876        special(SPC_NOMOD)
```

```
7877        prompt("Monitor List")
7878    }
7879    field(BKLNK, DBF_NOACCESS) {
7880        interest(4)
7881        extra("ELLLIST             bklnk")
7882        special(SPC_NOMOD)
7883        prompt("Backwards link tracking")
7884    }
7885    field(DISP, DBF_UCHAR) {
7886        promptgroup("10 - Common")
7887        interest(1)
7888        prompt("Disable putField")
7889    }
7890    field(PROC, DBF_UCHAR) {
7891        interest(3)
7892        pp(TRUE)
7893        prompt("Force Processing")
7894    }
7895    field(STAT, DBF_MENU) {
7896        initial("UDF")
7897        menu(menuAlarmStat)
7898        special(SPC_NOMOD)
7899        prompt("Alarm Status")
7900    }
7901    field(SEVR, DBF_MENU) {
7902        menu(menuAlarmSevr)
7903        special(SPC_NOMOD)
7904        prompt("Alarm Severity")
7905    }
7906    field(AMSG, DBF_STRING) {
7907        size(40)
7908        special(SPC_NOMOD)
7909        prompt("Alarm Message")
7910    }
7911    field(NSTA, DBF_MENU) {
7912        interest(2)
7913        menu(menuAlarmStat)
7914        special(SPC_NOMOD)
7915        prompt("New Alarm Status")
7916    }
7917    field(NSEV, DBF_MENU) {
7918        interest(2)
7919        menu(menuAlarmSevr)
7920        special(SPC_NOMOD)
7921        prompt("New Alarm Severity")
7922    }
7923    field(NAMSG, DBF_STRING) {
7924        size(40)
7925        special(SPC_NOMOD)
7926        prompt("New Alarm Message")
7927    }
7928    field(ACKS, DBF_MENU) {
7929        interest(2)
7930        menu(menuAlarmSevr)
7931        special(SPC_NOMOD)
7932        prompt("Alarm Ack Severity")
7933    }
7934    field(ACKT, DBF_MENU) {
7935        promptgroup("70 - Alarm")
7936        interest(2)
7937        initial("YES")
7938        menu(menuYesNo)
7939        special(SPC_NOMOD)
7940        prompt("Alarm Ack Transient")
7941    }
7942    field(DISS, DBF_MENU) {
7943        promptgroup("70 - Alarm")
7944        interest(1)
```

```
7945        menu(menuAlarmSevr)
7946        prompt("Disable Alarm Sevrty")
7947    }
7948    field(LCNT, DBF_UCHAR) {
7949        interest(2)
7950        special(SPC_NOMOD)
7951        prompt("Lock Count")
7952    }
7953    field(PACT, DBF_UCHAR) {
7954        interest(1)
7955        special(SPC_NOMOD)
7956        prompt("Record active")
7957    }
7958    field(PUTF, DBF_UCHAR) {
7959        interest(1)
7960        special(SPC_NOMOD)
7961        prompt("dbPutField process")
7962    }
7963    field(RPRO, DBF_UCHAR) {
7964        interest(1)
7965        special(SPC_NOMOD)
7966        prompt("Reprocess ")
7967    }
7968    field(ASP, DBF_NOACCESS) {
7969        interest(4)
7970        extra("struct asgMember    *asp")
7971        special(SPC_NOMOD)
7972        prompt("Access Security Pvt")
7973    }
7974    field(PPN, DBF_NOACCESS) {
7975        interest(4)
7976        extra("struct processNotify *ppn")
7977        special(SPC_NOMOD)
7978        prompt("pprocessNotify")
7979    }
7980    field(PPNR, DBF_NOACCESS) {
7981        interest(4)
7982        extra("struct processNotifyRecord *ppnr")
7983        special(SPC_NOMOD)
7984        prompt("pprocessNotifyRecord")
7985    }
7986    field(SPVT, DBF_NOACCESS) {
7987        interest(4)
7988        extra("struct scan_element *spvt")
7989        special(SPC_NOMOD)
7990        prompt("Scan Private")
7991    }
7992    field(RSET, DBF_NOACCESS) {
7993        interest(4)
7994        extra("struct typed_rset   *rset")
7995        special(SPC_NOMOD)
7996        prompt("Address of RSET")
7997    }
7998    field(DSET, DBF_NOACCESS) {
7999        interest(4)
8000        extra("unambiguous_dset    *dset")
8001        special(SPC_NOMOD)
8002        prompt("DSET address")
8003    }
8004    field(DPVT, DBF_NOACCESS) {
8005        interest(4)
8006        extra("void                *dpvt")
8007        special(SPC_NOMOD)
8008        prompt("Device Private")
8009    }
8010    field(RDES, DBF_NOACCESS) {
8011        interest(4)
8012        extra("struct dbRecordType *rdes")
```

```
8013        special(SPC_NOMOD)
8014        prompt("Address of dbRecordType")
8015    }
8016    field(LSET, DBF_NOACCESS) {
8017        interest(4)
8018        extra("struct lockRecord   *lset")
8019        special(SPC_NOMOD)
8020        prompt("Lock Set")
8021    }
8022    field(PRIO, DBF_MENU) {
8023        promptgroup("20 - Scan")
8024        interest(1)
8025        menu(menuPriority)
8026        special(SPC_SCAN)
8027        prompt("Scheduling Priority")
8028    }
8029    field(TPRO, DBF_UCHAR) {
8030        prompt("Trace Processing")
8031    }
8032    field(BKPT, DBF_NOACCESS) {
8033        interest(1)
8034        extra("epicsUInt8          bkpt")
8035        special(SPC_NOMOD)
8036        prompt("Break Point")
8037    }
8038    field(UDF, DBF_UCHAR) {
8039        promptgroup("10 - Common")
8040        interest(1)
8041        initial("1")
8042        pp(TRUE)
8043        prompt("Undefined")
8044    }
8045    field(UDFS, DBF_MENU) {
8046        promptgroup("70 - Alarm")
8047        interest(1)
8048        initial("INVALID")
8049        menu(menuAlarmSevr)
8050        prompt("Undefined Alarm Sevrty")
8051    }
8052    field(TIME, DBF_NOACCESS) {
8053        interest(2)
8054        extra("epicsTimeStamp      time")
8055        special(SPC_NOMOD)
8056        prompt("Time")
8057    }
8058    field(UTAG, DBF_UINT64) {
8059        interest(3)
8060        special(SPC_NOMOD)
8061        prompt("Time Tag")
8062    }
8063    field(FLNK, DBF_FWDLINK) {
8064        promptgroup("20 - Scan")
8065        interest(1)
8066        prompt("Forward Process Link")
8067    }
8068    field(VAL, DBF_STRING) {
8069        size(40)
8070        promptgroup("40 - Input")
8071        asl(ASL0)
8072        special(SPC_MOD)
8073        prompt("Event Name To Post")
8074    }
8075    field(EPVT, DBF_NOACCESS) {
8076        interest(4)
8077        extra("EVENTPVT epvt")
8078        special(SPC_NOMOD)
8079        prompt("Event private")
8080    }
```

```
    field(INP, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input Specification")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Sim Input Specifctn")
    }
    field(SVAL, DBF_STRING) {
        size(40)
        prompt("Simulation Value")
    }
    field(SIML, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Sim Mode Location")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuYesNo)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Sim mode Alarm Svrty")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(event, CONSTANT, devEventSoft, "Soft Channel")
recordtype(fanout) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
```

```
8149        prompt("Record Name")
8150    }
8151    field(DESC, DBF_STRING) {
8152        size(41)
8153        promptgroup("10 - Common")
8154        prompt("Descriptor")
8155    }
8156    field(ASG, DBF_STRING) {
8157        size(29)
8158        promptgroup("10 - Common")
8159        special(SPC_AS)
8160        prompt("Access Security Group")
8161    }
8162    field(SCAN, DBF_MENU) {
8163        promptgroup("20 - Scan")
8164        interest(1)
8165        menu(menuScan)
8166        special(SPC_SCAN)
8167        prompt("Scan Mechanism")
8168    }
8169    field(PINI, DBF_MENU) {
8170        promptgroup("20 - Scan")
8171        interest(1)
8172        menu(menuPini)
8173        prompt("Process at iocInit")
8174    }
8175    field(PHAS, DBF_SHORT) {
8176        promptgroup("20 - Scan")
8177        interest(1)
8178        special(SPC_SCAN)
8179        prompt("Scan Phase")
8180    }
8181    field(EVNT, DBF_STRING) {
8182        size(40)
8183        promptgroup("20 - Scan")
8184        interest(1)
8185        special(SPC_SCAN)
8186        prompt("Event Name")
8187    }
8188    field(TSE, DBF_SHORT) {
8189        promptgroup("20 - Scan")
8190        interest(1)
8191        prompt("Time Stamp Event")
8192    }
8193    field(TSEL, DBF_INLINK) {
8194        promptgroup("20 - Scan")
8195        interest(1)
8196        prompt("Time Stamp Link")
8197    }
8198    field(DTYP, DBF_DEVICE) {
8199        promptgroup("10 - Common")
8200        interest(1)
8201        prompt("Device Type")
8202    }
8203    field(DISV, DBF_SHORT) {
8204        promptgroup("20 - Scan")
8205        initial("1")
8206        prompt("Disable Value")
8207    }
8208    field(DISA, DBF_SHORT) {
8209        prompt("Disable")
8210    }
8211    field(SDIS, DBF_INLINK) {
8212        promptgroup("20 - Scan")
8213        interest(1)
8214        prompt("Scanning Disable")
8215    }
8216    field(MLOK, DBF_NOACCESS) {
```

```
8217        interest(4)
8218        extra("epicsMutexId        mlok")
8219        special(SPC_NOMOD)
8220        prompt("Monitor lock")
8221    }
8222    field(MLIS, DBF_NOACCESS) {
8223        interest(4)
8224        extra("ELLLIST             mlis")
8225        special(SPC_NOMOD)
8226        prompt("Monitor List")
8227    }
8228    field(BKLNK, DBF_NOACCESS) {
8229        interest(4)
8230        extra("ELLLIST             bklnk")
8231        special(SPC_NOMOD)
8232        prompt("Backwards link tracking")
8233    }
8234    field(DISP, DBF_UCHAR) {
8235        promptgroup("10 - Common")
8236        interest(1)
8237        prompt("Disable putField")
8238    }
8239    field(PROC, DBF_UCHAR) {
8240        interest(3)
8241        pp(TRUE)
8242        prompt("Force Processing")
8243    }
8244    field(STAT, DBF_MENU) {
8245        initial("UDF")
8246        menu(menuAlarmStat)
8247        special(SPC_NOMOD)
8248        prompt("Alarm Status")
8249    }
8250    field(SEVR, DBF_MENU) {
8251        menu(menuAlarmSevr)
8252        special(SPC_NOMOD)
8253        prompt("Alarm Severity")
8254    }
8255    field(AMSG, DBF_STRING) {
8256        size(40)
8257        special(SPC_NOMOD)
8258        prompt("Alarm Message")
8259    }
8260    field(NSTA, DBF_MENU) {
8261        interest(2)
8262        menu(menuAlarmStat)
8263        special(SPC_NOMOD)
8264        prompt("New Alarm Status")
8265    }
8266    field(NSEV, DBF_MENU) {
8267        interest(2)
8268        menu(menuAlarmSevr)
8269        special(SPC_NOMOD)
8270        prompt("New Alarm Severity")
8271    }
8272    field(NAMSG, DBF_STRING) {
8273        size(40)
8274        special(SPC_NOMOD)
8275        prompt("New Alarm Message")
8276    }
8277    field(ACKS, DBF_MENU) {
8278        interest(2)
8279        menu(menuAlarmSevr)
8280        special(SPC_NOMOD)
8281        prompt("Alarm Ack Severity")
8282    }
8283    field(ACKT, DBF_MENU) {
8284        promptgroup("70 - Alarm")
```

```
8285        interest(2)
8286        initial("YES")
8287        menu(menuYesNo)
8288        special(SPC_NOMOD)
8289        prompt("Alarm Ack Transient")
8290    }
8291    field(DISS, DBF_MENU) {
8292        promptgroup("70 - Alarm")
8293        interest(1)
8294        menu(menuAlarmSevr)
8295        prompt("Disable Alarm Sevrty")
8296    }
8297    field(LCNT, DBF_UCHAR) {
8298        interest(2)
8299        special(SPC_NOMOD)
8300        prompt("Lock Count")
8301    }
8302    field(PACT, DBF_UCHAR) {
8303        interest(1)
8304        special(SPC_NOMOD)
8305        prompt("Record active")
8306    }
8307    field(PUTF, DBF_UCHAR) {
8308        interest(1)
8309        special(SPC_NOMOD)
8310        prompt("dbPutField process")
8311    }
8312    field(RPRO, DBF_UCHAR) {
8313        interest(1)
8314        special(SPC_NOMOD)
8315        prompt("Reprocess ")
8316    }
8317    field(ASP, DBF_NOACCESS) {
8318        interest(4)
8319        extra("struct asgMember    *asp")
8320        special(SPC_NOMOD)
8321        prompt("Access Security Pvt")
8322    }
8323    field(PPN, DBF_NOACCESS) {
8324        interest(4)
8325        extra("struct processNotify *ppn")
8326        special(SPC_NOMOD)
8327        prompt("pprocessNotify")
8328    }
8329    field(PPNR, DBF_NOACCESS) {
8330        interest(4)
8331        extra("struct processNotifyRecord *ppnr")
8332        special(SPC_NOMOD)
8333        prompt("pprocessNotifyRecord")
8334    }
8335    field(SPVT, DBF_NOACCESS) {
8336        interest(4)
8337        extra("struct scan_element *spvt")
8338        special(SPC_NOMOD)
8339        prompt("Scan Private")
8340    }
8341    field(RSET, DBF_NOACCESS) {
8342        interest(4)
8343        extra("struct typed_rset   *rset")
8344        special(SPC_NOMOD)
8345        prompt("Address of RSET")
8346    }
8347    field(DSET, DBF_NOACCESS) {
8348        interest(4)
8349        extra("unambiguous_dset    *dset")
8350        special(SPC_NOMOD)
8351        prompt("DSET address")
8352    }
```

```
8353    field(DPVT, DBF_NOACCESS) {
8354        interest(4)
8355        extra("void                *dpvt")
8356        special(SPC_NOMOD)
8357        prompt("Device Private")
8358    }
8359    field(RDES, DBF_NOACCESS) {
8360        interest(4)
8361        extra("struct dbRecordType *rdes")
8362        special(SPC_NOMOD)
8363        prompt("Address of dbRecordType")
8364    }
8365    field(LSET, DBF_NOACCESS) {
8366        interest(4)
8367        extra("struct lockRecord   *lset")
8368        special(SPC_NOMOD)
8369        prompt("Lock Set")
8370    }
8371    field(PRIO, DBF_MENU) {
8372        promptgroup("20 - Scan")
8373        interest(1)
8374        menu(menuPriority)
8375        special(SPC_SCAN)
8376        prompt("Scheduling Priority")
8377    }
8378    field(TPRO, DBF_UCHAR) {
8379        prompt("Trace Processing")
8380    }
8381    field(BKPT, DBF_NOACCESS) {
8382        interest(1)
8383        extra("epicsUInt8          bkpt")
8384        special(SPC_NOMOD)
8385        prompt("Break Point")
8386    }
8387    field(UDF, DBF_UCHAR) {
8388        promptgroup("10 - Common")
8389        interest(1)
8390        initial("1")
8391        pp(TRUE)
8392        prompt("Undefined")
8393    }
8394    field(UDFS, DBF_MENU) {
8395        promptgroup("70 - Alarm")
8396        interest(1)
8397        initial("INVALID")
8398        menu(menuAlarmSevr)
8399        prompt("Undefined Alarm Sevrty")
8400    }
8401    field(TIME, DBF_NOACCESS) {
8402        interest(2)
8403        extra("epicsTimeStamp      time")
8404        special(SPC_NOMOD)
8405        prompt("Time")
8406    }
8407    field(UTAG, DBF_UINT64) {
8408        interest(3)
8409        special(SPC_NOMOD)
8410        prompt("Time Tag")
8411    }
8412    field(FLNK, DBF_FWDLINK) {
8413        promptgroup("20 - Scan")
8414        interest(1)
8415        prompt("Forward Process Link")
8416    }
8417    field(VAL, DBF_LONG) {
8418        asl(ASL0)
8419        pp(TRUE)
8420        prompt("Used to trigger")
```

```
8421        }
8422        field(SELM, DBF_MENU) {
8423            promptgroup("30 - Action")
8424            interest(1)
8425            menu(fanoutSELM)
8426            prompt("Select Mechanism")
8427        }
8428        field(SELN, DBF_USHORT) {
8429            interest(1)
8430            initial("1")
8431            prompt("Link Selection")
8432        }
8433        field(SELL, DBF_INLINK) {
8434            promptgroup("30 - Action")
8435            interest(1)
8436            prompt("Link Selection Loc")
8437        }
8438        field(OFFS, DBF_SHORT) {
8439            promptgroup("30 - Action")
8440            interest(1)
8441            initial("0")
8442            prompt("Offset for Specified")
8443        }
8444        field(SHFT, DBF_SHORT) {
8445            promptgroup("30 - Action")
8446            interest(1)
8447            initial("-1")
8448            prompt("Shift for Mask mode")
8449        }
8450        field(LNK0, DBF_FWDLINK) {
8451            promptgroup("51 - Output 0-7")
8452            interest(1)
8453            prompt("Forward Link 0")
8454        }
8455        field(LNK1, DBF_FWDLINK) {
8456            promptgroup("51 - Output 0-7")
8457            interest(1)
8458            prompt("Forward Link 1")
8459        }
8460        field(LNK2, DBF_FWDLINK) {
8461            promptgroup("51 - Output 0-7")
8462            interest(1)
8463            prompt("Forward Link 2")
8464        }
8465        field(LNK3, DBF_FWDLINK) {
8466            promptgroup("51 - Output 0-7")
8467            interest(1)
8468            prompt("Forward Link 3")
8469        }
8470        field(LNK4, DBF_FWDLINK) {
8471            promptgroup("51 - Output 0-7")
8472            interest(1)
8473            prompt("Forward Link 4")
8474        }
8475        field(LNK5, DBF_FWDLINK) {
8476            promptgroup("51 - Output 0-7")
8477            interest(1)
8478            prompt("Forward Link 5")
8479        }
8480        field(LNK6, DBF_FWDLINK) {
8481            promptgroup("51 - Output 0-7")
8482            interest(1)
8483            prompt("Forward Link 6")
8484        }
8485        field(LNK7, DBF_FWDLINK) {
8486            promptgroup("51 - Output 0-7")
8487            interest(1)
8488            prompt("Forward Link 7")
```

```
8489        }
8490        field(LNK8, DBF_FWDLINK) {
8491            promptgroup("52 - Output 8-F")
8492            interest(1)
8493            prompt("Forward Link 8")
8494        }
8495        field(LNK9, DBF_FWDLINK) {
8496            promptgroup("52 - Output 8-F")
8497            interest(1)
8498            prompt("Forward Link 9")
8499        }
8500        field(LNKA, DBF_FWDLINK) {
8501            promptgroup("52 - Output 8-F")
8502            interest(1)
8503            prompt("Forward Link 10")
8504        }
8505        field(LNKB, DBF_FWDLINK) {
8506            promptgroup("52 - Output 8-F")
8507            interest(1)
8508            prompt("Forward Link 11")
8509        }
8510        field(LNKC, DBF_FWDLINK) {
8511            promptgroup("52 - Output 8-F")
8512            interest(1)
8513            prompt("Forward Link 12")
8514        }
8515        field(LNKD, DBF_FWDLINK) {
8516            promptgroup("52 - Output 8-F")
8517            interest(1)
8518            prompt("Forward Link 13")
8519        }
8520        field(LNKE, DBF_FWDLINK) {
8521            promptgroup("52 - Output 8-F")
8522            interest(1)
8523            prompt("Forward Link 14")
8524        }
8525        field(LNKF, DBF_FWDLINK) {
8526            promptgroup("52 - Output 8-F")
8527            interest(1)
8528            prompt("Forward Link 15")
8529        }
8530    }
8531    recordtype(histogram) {
8532        %#include "epicsTypes.h"
8533        %#include "link.h"
8534        %#include "epicsMutex.h"
8535        %#include "ellLib.h"
8536        %#include "devSup.h"
8537        %#include "epicsTime.h"
8538        %
8539        %/* Declare Device Support Entry Table */
8540        %struct histogramRecord;
8541        %typedef struct histogramdset {
8542        %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
8543        %    long (*read_histogram)(struct histogramRecord *prec); /*(0,2)=> success and add_count, don't
             add_count); if add_count then sgnl added to array*/
8544        %    long (*special_linconv)(struct histogramRecord *prec, int after);
8545        %} histogramdset;
8546        %#define HAS_histogramdset
8547        %
8548        %#include "callback.h"
8549        field(NAME, DBF_STRING) {
8550            size(61)
8551            special(SPC_NOMOD)
8552            prompt("Record Name")
8553        }
8554        field(DESC, DBF_STRING) {
8555            size(41)
```

```
8556        promptgroup("10 - Common")
8557        prompt("Descriptor")
8558    }
8559    field(ASG, DBF_STRING) {
8560        size(29)
8561        promptgroup("10 - Common")
8562        special(SPC_AS)
8563        prompt("Access Security Group")
8564    }
8565    field(SCAN, DBF_MENU) {
8566        promptgroup("20 - Scan")
8567        interest(1)
8568        menu(menuScan)
8569        special(SPC_SCAN)
8570        prompt("Scan Mechanism")
8571    }
8572    field(PINI, DBF_MENU) {
8573        promptgroup("20 - Scan")
8574        interest(1)
8575        menu(menuPini)
8576        prompt("Process at iocInit")
8577    }
8578    field(PHAS, DBF_SHORT) {
8579        promptgroup("20 - Scan")
8580        interest(1)
8581        special(SPC_SCAN)
8582        prompt("Scan Phase")
8583    }
8584    field(EVNT, DBF_STRING) {
8585        size(40)
8586        promptgroup("20 - Scan")
8587        interest(1)
8588        special(SPC_SCAN)
8589        prompt("Event Name")
8590    }
8591    field(TSE, DBF_SHORT) {
8592        promptgroup("20 - Scan")
8593        interest(1)
8594        prompt("Time Stamp Event")
8595    }
8596    field(TSEL, DBF_INLINK) {
8597        promptgroup("20 - Scan")
8598        interest(1)
8599        prompt("Time Stamp Link")
8600    }
8601    field(DTYP, DBF_DEVICE) {
8602        promptgroup("10 - Common")
8603        interest(1)
8604        prompt("Device Type")
8605    }
8606    field(DISV, DBF_SHORT) {
8607        promptgroup("20 - Scan")
8608        initial("1")
8609        prompt("Disable Value")
8610    }
8611    field(DISA, DBF_SHORT) {
8612        prompt("Disable")
8613    }
8614    field(SDIS, DBF_INLINK) {
8615        promptgroup("20 - Scan")
8616        interest(1)
8617        prompt("Scanning Disable")
8618    }
8619    field(MLOK, DBF_NOACCESS) {
8620        interest(4)
8621        extra("epicsMutexId        mlok")
8622        special(SPC_NOMOD)
8623        prompt("Monitor lock")
```

```
8624        }
8625        field(MLIS, DBF_NOACCESS) {
8626            interest(4)
8627            extra("ELLLIST             mlis")
8628            special(SPC_NOMOD)
8629            prompt("Monitor List")
8630        }
8631        field(BKLNK, DBF_NOACCESS) {
8632            interest(4)
8633            extra("ELLLIST             bklnk")
8634            special(SPC_NOMOD)
8635            prompt("Backwards link tracking")
8636        }
8637        field(DISP, DBF_UCHAR) {
8638            promptgroup("10 - Common")
8639            interest(1)
8640            prompt("Disable putField")
8641        }
8642        field(PROC, DBF_UCHAR) {
8643            interest(3)
8644            pp(TRUE)
8645            prompt("Force Processing")
8646        }
8647        field(STAT, DBF_MENU) {
8648            initial("UDF")
8649            menu(menuAlarmStat)
8650            special(SPC_NOMOD)
8651            prompt("Alarm Status")
8652        }
8653        field(SEVR, DBF_MENU) {
8654            menu(menuAlarmSevr)
8655            special(SPC_NOMOD)
8656            prompt("Alarm Severity")
8657        }
8658        field(AMSG, DBF_STRING) {
8659            size(40)
8660            special(SPC_NOMOD)
8661            prompt("Alarm Message")
8662        }
8663        field(NSTA, DBF_MENU) {
8664            interest(2)
8665            menu(menuAlarmStat)
8666            special(SPC_NOMOD)
8667            prompt("New Alarm Status")
8668        }
8669        field(NSEV, DBF_MENU) {
8670            interest(2)
8671            menu(menuAlarmSevr)
8672            special(SPC_NOMOD)
8673            prompt("New Alarm Severity")
8674        }
8675        field(NAMSG, DBF_STRING) {
8676            size(40)
8677            special(SPC_NOMOD)
8678            prompt("New Alarm Message")
8679        }
8680        field(ACKS, DBF_MENU) {
8681            interest(2)
8682            menu(menuAlarmSevr)
8683            special(SPC_NOMOD)
8684            prompt("Alarm Ack Severity")
8685        }
8686        field(ACKT, DBF_MENU) {
8687            promptgroup("70 - Alarm")
8688            interest(2)
8689            initial("YES")
8690            menu(menuYesNo)
8691            special(SPC_NOMOD)
```

```
8692        prompt("Alarm Ack Transient")
8693    }
8694    field(DISS, DBF_MENU) {
8695        promptgroup("70 - Alarm")
8696        interest(1)
8697        menu(menuAlarmSevr)
8698        prompt("Disable Alarm Sevrty")
8699    }
8700    field(LCNT, DBF_UCHAR) {
8701        interest(2)
8702        special(SPC_NOMOD)
8703        prompt("Lock Count")
8704    }
8705    field(PACT, DBF_UCHAR) {
8706        interest(1)
8707        special(SPC_NOMOD)
8708        prompt("Record active")
8709    }
8710    field(PUTF, DBF_UCHAR) {
8711        interest(1)
8712        special(SPC_NOMOD)
8713        prompt("dbPutField process")
8714    }
8715    field(RPRO, DBF_UCHAR) {
8716        interest(1)
8717        special(SPC_NOMOD)
8718        prompt("Reprocess ")
8719    }
8720    field(ASP, DBF_NOACCESS) {
8721        interest(4)
8722        extra("struct asgMember    *asp")
8723        special(SPC_NOMOD)
8724        prompt("Access Security Pvt")
8725    }
8726    field(PPN, DBF_NOACCESS) {
8727        interest(4)
8728        extra("struct processNotify *ppn")
8729        special(SPC_NOMOD)
8730        prompt("pprocessNotify")
8731    }
8732    field(PPNR, DBF_NOACCESS) {
8733        interest(4)
8734        extra("struct processNotifyRecord *ppnr")
8735        special(SPC_NOMOD)
8736        prompt("pprocessNotifyRecord")
8737    }
8738    field(SPVT, DBF_NOACCESS) {
8739        interest(4)
8740        extra("struct scan_element *spvt")
8741        special(SPC_NOMOD)
8742        prompt("Scan Private")
8743    }
8744    field(RSET, DBF_NOACCESS) {
8745        interest(4)
8746        extra("struct typed_rset   *rset")
8747        special(SPC_NOMOD)
8748        prompt("Address of RSET")
8749    }
8750    field(DSET, DBF_NOACCESS) {
8751        interest(4)
8752        extra("unambiguous_dset    *dset")
8753        special(SPC_NOMOD)
8754        prompt("DSET address")
8755    }
8756    field(DPVT, DBF_NOACCESS) {
8757        interest(4)
8758        extra("void                *dpvt")
8759        special(SPC_NOMOD)
```

```
8760        prompt("Device Private")
8761    }
8762    field(RDES, DBF_NOACCESS) {
8763        interest(4)
8764        extra("struct dbRecordType *rdes")
8765        special(SPC_NOMOD)
8766        prompt("Address of dbRecordType")
8767    }
8768    field(LSET, DBF_NOACCESS) {
8769        interest(4)
8770        extra("struct lockRecord   *lset")
8771        special(SPC_NOMOD)
8772        prompt("Lock Set")
8773    }
8774    field(PRIO, DBF_MENU) {
8775        promptgroup("20 - Scan")
8776        interest(1)
8777        menu(menuPriority)
8778        special(SPC_SCAN)
8779        prompt("Scheduling Priority")
8780    }
8781    field(TPRO, DBF_UCHAR) {
8782        prompt("Trace Processing")
8783    }
8784    field(BKPT, DBF_NOACCESS) {
8785        interest(1)
8786        extra("epicsUInt8          bkpt")
8787        special(SPC_NOMOD)
8788        prompt("Break Point")
8789    }
8790    field(UDF, DBF_UCHAR) {
8791        promptgroup("10 - Common")
8792        interest(1)
8793        initial("1")
8794        pp(TRUE)
8795        prompt("Undefined")
8796    }
8797    field(UDFS, DBF_MENU) {
8798        promptgroup("70 - Alarm")
8799        interest(1)
8800        initial("INVALID")
8801        menu(menuAlarmSevr)
8802        prompt("Undefined Alarm Sevrty")
8803    }
8804    field(TIME, DBF_NOACCESS) {
8805        interest(2)
8806        extra("epicsTimeStamp      time")
8807        special(SPC_NOMOD)
8808        prompt("Time")
8809    }
8810    field(UTAG, DBF_UINT64) {
8811        interest(3)
8812        special(SPC_NOMOD)
8813        prompt("Time Tag")
8814    }
8815    field(FLNK, DBF_FWDLINK) {
8816        promptgroup("20 - Scan")
8817        interest(1)
8818        prompt("Forward Process Link")
8819    }
8820    field(VAL, DBF_NOACCESS) {
8821        extra("void *   val")
8822        asl(ASL0)
8823        special(SPC_DBADDR)
8824        prompt("Value")
8825    }
8826    field(NELM, DBF_USHORT) {
8827        promptgroup("30 - Action")
```

```
8828        interest(1)
8829        initial("1")
8830        special(SPC_NOMOD)
8831        prompt("Num of Array Elements")
8832    }
8833    field(CSTA, DBF_SHORT) {
8834        interest(1)
8835        initial("1")
8836        special(SPC_NOMOD)
8837        prompt("Collection Status")
8838    }
8839    field(CMD, DBF_MENU) {
8840        interest(1)
8841        asl(ASL0)
8842        menu(histogramCMD)
8843        special(SPC_CALC)
8844        prompt("Collection Control")
8845    }
8846    field(ULIM, DBF_DOUBLE) {
8847        promptgroup("30 - Action")
8848        interest(1)
8849        special(SPC_RESET)
8850        prop(YES)
8851        prompt("Upper Signal Limit")
8852    }
8853    field(LLIM, DBF_DOUBLE) {
8854        promptgroup("30 - Action")
8855        interest(1)
8856        special(SPC_RESET)
8857        prop(YES)
8858        prompt("Lower Signal Limit ")
8859    }
8860    field(WDTH, DBF_DOUBLE) {
8861        interest(3)
8862        special(SPC_NOMOD)
8863        prompt("Element Width")
8864    }
8865    field(SGNL, DBF_DOUBLE) {
8866        special(SPC_MOD)
8867        prompt("Signal Value")
8868    }
8869    field(PREC, DBF_SHORT) {
8870        promptgroup("80 - Display")
8871        interest(1)
8872        prop(YES)
8873        prompt("Display Precision")
8874    }
8875    field(SVL, DBF_INLINK) {
8876        promptgroup("40 - Input")
8877        interest(1)
8878        prompt("Signal Value Location")
8879    }
8880    field(BPTR, DBF_NOACCESS) {
8881        interest(4)
8882        extra("epicsUInt32 *bptr")
8883        special(SPC_NOMOD)
8884        prompt("Buffer Pointer")
8885    }
8886    field(WDOG, DBF_NOACCESS) {
8887        interest(4)
8888        extra("void *  wdog")
8889        special(SPC_NOMOD)
8890        prompt("Watchdog callback")
8891    }
8892    field(MDEL, DBF_SHORT) {
8893        promptgroup("80 - Display")
8894        interest(1)
8895        prompt("Monitor Count Deadband")
```

```
8896    }
8897    field(MCNT, DBF_SHORT) {
8898        interest(3)
8899        special(SPC_NOMOD)
8900        prompt("Counts Since Monitor")
8901    }
8902    field(SDEL, DBF_DOUBLE) {
8903        promptgroup("80 - Display")
8904        interest(1)
8905        special(SPC_RESET)
8906        prompt("Monitor Seconds Dband")
8907    }
8908    field(SIOL, DBF_INLINK) {
8909        promptgroup("90 - Simulate")
8910        interest(1)
8911        prompt("Simulation Input Link")
8912    }
8913    field(SVAL, DBF_DOUBLE) {
8914        prompt("Simulation Value")
8915    }
8916    field(SIML, DBF_INLINK) {
8917        promptgroup("90 - Simulate")
8918        interest(1)
8919        prompt("Simulation Mode Link")
8920    }
8921    field(SIMM, DBF_MENU) {
8922        interest(1)
8923        menu(menuYesNo)
8924        special(SPC_MOD)
8925        prompt("Simulation Mode")
8926    }
8927    field(SIMS, DBF_MENU) {
8928        promptgroup("90 - Simulate")
8929        interest(2)
8930        menu(menuAlarmSevr)
8931        prompt("Simulation Mode Severity")
8932    }
8933    field(OLDSIMM, DBF_MENU) {
8934        interest(4)
8935        menu(menuSimm)
8936        special(SPC_NOMOD)
8937        prompt("Prev. Simulation Mode")
8938    }
8939    field(SSCN, DBF_MENU) {
8940        promptgroup("90 - Simulate")
8941        interest(1)
8942        initial("65535")
8943        menu(menuScan)
8944        prompt("Sim. Mode Scan")
8945    }
8946    field(SDLY, DBF_DOUBLE) {
8947        promptgroup("90 - Simulate")
8948        interest(2)
8949        initial("-1.0")
8950        prompt("Sim. Mode Async Delay")
8951    }
8952    field(SIMPVT, DBF_NOACCESS) {
8953        interest(4)
8954        extra("epicsCallback            *simpvt")
8955        special(SPC_NOMOD)
8956        prompt("Sim. Mode Private")
8957    }
8958    field(HOPR, DBF_ULONG) {
8959        promptgroup("80 - Display")
8960        interest(1)
8961        prop(YES)
8962        prompt("High Operating Range")
8963    }
```

```
8964     field(LOPR, DBF_ULONG) {
8965         promptgroup("80 - Display")
8966         interest(1)
8967         prop(YES)
8968         prompt("Low Operating Range")
8969     }
8970 }
8971 device(histogram, CONSTANT, devHistogramSoft, "Soft Channel")
8972 recordtype(int64in) {
8973     %#include "epicsTypes.h"
8974     %#include "link.h"
8975     %#include "epicsMutex.h"
8976     %#include "ellLib.h"
8977     %#include "devSup.h"
8978     %#include "epicsTime.h"
8979     %
8980     %/* Declare Device Support Entry Table */
8981     %struct int64inRecord;
8982     %typedef struct int64indset {
8983     %    dset common;
8984     %    long (*read_int64in)(struct int64inRecord *prec);
8985     %} int64indset;
8986     %#define HAS_int64indset
8987     %
8988     %#include "callback.h"
8989     field(NAME, DBF_STRING) {
8990         size(61)
8991         special(SPC_NOMOD)
8992         prompt("Record Name")
8993     }
8994     field(DESC, DBF_STRING) {
8995         size(41)
8996         promptgroup("10 - Common")
8997         prompt("Descriptor")
8998     }
8999     field(ASG, DBF_STRING) {
9000         size(29)
9001         promptgroup("10 - Common")
9002         special(SPC_AS)
9003         prompt("Access Security Group")
9004     }
9005     field(SCAN, DBF_MENU) {
9006         promptgroup("20 - Scan")
9007         interest(1)
9008         menu(menuScan)
9009         special(SPC_SCAN)
9010         prompt("Scan Mechanism")
9011     }
9012     field(PINI, DBF_MENU) {
9013         promptgroup("20 - Scan")
9014         interest(1)
9015         menu(menuPini)
9016         prompt("Process at iocInit")
9017     }
9018     field(PHAS, DBF_SHORT) {
9019         promptgroup("20 - Scan")
9020         interest(1)
9021         special(SPC_SCAN)
9022         prompt("Scan Phase")
9023     }
9024     field(EVNT, DBF_STRING) {
9025         size(40)
9026         promptgroup("20 - Scan")
9027         interest(1)
9028         special(SPC_SCAN)
9029         prompt("Event Name")
9030     }
9031     field(TSE, DBF_SHORT) {
```

```
9032        promptgroup("20 - Scan")
9033        interest(1)
9034        prompt("Time Stamp Event")
9035    }
9036    field(TSEL, DBF_INLINK) {
9037        promptgroup("20 - Scan")
9038        interest(1)
9039        prompt("Time Stamp Link")
9040    }
9041    field(DTYP, DBF_DEVICE) {
9042        promptgroup("10 - Common")
9043        interest(1)
9044        prompt("Device Type")
9045    }
9046    field(DISV, DBF_SHORT) {
9047        promptgroup("20 - Scan")
9048        initial("1")
9049        prompt("Disable Value")
9050    }
9051    field(DISA, DBF_SHORT) {
9052        prompt("Disable")
9053    }
9054    field(SDIS, DBF_INLINK) {
9055        promptgroup("20 - Scan")
9056        interest(1)
9057        prompt("Scanning Disable")
9058    }
9059    field(MLOK, DBF_NOACCESS) {
9060        interest(4)
9061        extra("epicsMutexId        mlok")
9062        special(SPC_NOMOD)
9063        prompt("Monitor lock")
9064    }
9065    field(MLIS, DBF_NOACCESS) {
9066        interest(4)
9067        extra("ELLLIST             mlis")
9068        special(SPC_NOMOD)
9069        prompt("Monitor List")
9070    }
9071    field(BKLNK, DBF_NOACCESS) {
9072        interest(4)
9073        extra("ELLLIST             bklnk")
9074        special(SPC_NOMOD)
9075        prompt("Backwards link tracking")
9076    }
9077    field(DISP, DBF_UCHAR) {
9078        promptgroup("10 - Common")
9079        interest(1)
9080        prompt("Disable putField")
9081    }
9082    field(PROC, DBF_UCHAR) {
9083        interest(3)
9084        pp(TRUE)
9085        prompt("Force Processing")
9086    }
9087    field(STAT, DBF_MENU) {
9088        initial("UDF")
9089        menu(menuAlarmStat)
9090        special(SPC_NOMOD)
9091        prompt("Alarm Status")
9092    }
9093    field(SEVR, DBF_MENU) {
9094        menu(menuAlarmSevr)
9095        special(SPC_NOMOD)
9096        prompt("Alarm Severity")
9097    }
9098    field(AMSG, DBF_STRING) {
9099        size(40)
```

```
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Reprocess ")
    }
    field(ASP, DBF_NOACCESS) {
        interest(4)
        extra("struct asgMember    *asp")
        special(SPC_NOMOD)
        prompt("Access Security Pvt")
    }
    field(PPN, DBF_NOACCESS) {
        interest(4)
```

```
9168        extra("struct processNotify *ppn")
9169        special(SPC_NOMOD)
9170        prompt("pprocessNotify")
9171    }
9172    field(PPNR, DBF_NOACCESS) {
9173        interest(4)
9174        extra("struct processNotifyRecord *ppnr")
9175        special(SPC_NOMOD)
9176        prompt("pprocessNotifyRecord")
9177    }
9178    field(SPVT, DBF_NOACCESS) {
9179        interest(4)
9180        extra("struct scan_element *spvt")
9181        special(SPC_NOMOD)
9182        prompt("Scan Private")
9183    }
9184    field(RSET, DBF_NOACCESS) {
9185        interest(4)
9186        extra("struct typed_rset   *rset")
9187        special(SPC_NOMOD)
9188        prompt("Address of RSET")
9189    }
9190    field(DSET, DBF_NOACCESS) {
9191        interest(4)
9192        extra("unambiguous_dset    *dset")
9193        special(SPC_NOMOD)
9194        prompt("DSET address")
9195    }
9196    field(DPVT, DBF_NOACCESS) {
9197        interest(4)
9198        extra("void                *dpvt")
9199        special(SPC_NOMOD)
9200        prompt("Device Private")
9201    }
9202    field(RDES, DBF_NOACCESS) {
9203        interest(4)
9204        extra("struct dbRecordType *rdes")
9205        special(SPC_NOMOD)
9206        prompt("Address of dbRecordType")
9207    }
9208    field(LSET, DBF_NOACCESS) {
9209        interest(4)
9210        extra("struct lockRecord   *lset")
9211        special(SPC_NOMOD)
9212        prompt("Lock Set")
9213    }
9214    field(PRIO, DBF_MENU) {
9215        promptgroup("20 - Scan")
9216        interest(1)
9217        menu(menuPriority)
9218        special(SPC_SCAN)
9219        prompt("Scheduling Priority")
9220    }
9221    field(TPRO, DBF_UCHAR) {
9222        prompt("Trace Processing")
9223    }
9224    field(BKPT, DBF_NOACCESS) {
9225        interest(1)
9226        extra("epicsUInt8          bkpt")
9227        special(SPC_NOMOD)
9228        prompt("Break Point")
9229    }
9230    field(UDF, DBF_UCHAR) {
9231        promptgroup("10 - Common")
9232        interest(1)
9233        initial("1")
9234        pp(TRUE)
9235        prompt("Undefined")
```

```
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_INT64) {
        promptgroup("40 - Input")
        asl(ASL0)
        pp(TRUE)
        prompt("Current value")
    }
    field(INP, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input Specification")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Units name")
    }
    field(HOPR, DBF_INT64) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_INT64) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
```

```
    field(HIGH, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Hihi Severity")
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Lolo Severity")
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("High Severity")
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Low Severity")
    }
    field(HYST, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(AFTC, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(AFVL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Alarm Filter Value")
    }
    field(ADEL, DBF_INT64) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_INT64) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(LALM, DBF_INT64) {
```

```
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(ALST, DBF_INT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_INT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Val Monitored")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Input Link")
    }
    field(SVAL, DBF_INT64) {
        prompt("Simulation Value")
    }
    field(SIML, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Mode Link")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuYesNo)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(int64in, CONSTANT, devI64inSoft, "Soft Channel")
device(int64in, CONSTANT, devI64inSoftCallback, "Async Soft Channel")
device(int64in, INST_IO, devPSCRegI64i, "PSC Reg")
```

```
device(int64in, INST_IO, devPSCRegRBI64i, "PSC RB Reg")
recordtype(int64out) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct int64outRecord;
    %typedef struct int64outdset {
    %    dset common;
    %    long (*write_int64out)(struct int64outRecord *prec);
    %} int64outdset;
    %#define HAS_int64outdset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
```

```
9508        prompt("Time Stamp Link")
9509    }
9510    field(DTYP, DBF_DEVICE) {
9511        promptgroup("10 - Common")
9512        interest(1)
9513        prompt("Device Type")
9514    }
9515    field(DISV, DBF_SHORT) {
9516        promptgroup("20 - Scan")
9517        initial("1")
9518        prompt("Disable Value")
9519    }
9520    field(DISA, DBF_SHORT) {
9521        prompt("Disable")
9522    }
9523    field(SDIS, DBF_INLINK) {
9524        promptgroup("20 - Scan")
9525        interest(1)
9526        prompt("Scanning Disable")
9527    }
9528    field(MLOK, DBF_NOACCESS) {
9529        interest(4)
9530        extra("epicsMutexId        mlok")
9531        special(SPC_NOMOD)
9532        prompt("Monitor lock")
9533    }
9534    field(MLIS, DBF_NOACCESS) {
9535        interest(4)
9536        extra("ELLLIST             mlis")
9537        special(SPC_NOMOD)
9538        prompt("Monitor List")
9539    }
9540    field(BKLNK, DBF_NOACCESS) {
9541        interest(4)
9542        extra("ELLLIST             bklnk")
9543        special(SPC_NOMOD)
9544        prompt("Backwards link tracking")
9545    }
9546    field(DISP, DBF_UCHAR) {
9547        promptgroup("10 - Common")
9548        interest(1)
9549        prompt("Disable putField")
9550    }
9551    field(PROC, DBF_UCHAR) {
9552        interest(3)
9553        pp(TRUE)
9554        prompt("Force Processing")
9555    }
9556    field(STAT, DBF_MENU) {
9557        initial("UDF")
9558        menu(menuAlarmStat)
9559        special(SPC_NOMOD)
9560        prompt("Alarm Status")
9561    }
9562    field(SEVR, DBF_MENU) {
9563        menu(menuAlarmSevr)
9564        special(SPC_NOMOD)
9565        prompt("Alarm Severity")
9566    }
9567    field(AMSG, DBF_STRING) {
9568        size(40)
9569        special(SPC_NOMOD)
9570        prompt("Alarm Message")
9571    }
9572    field(NSTA, DBF_MENU) {
9573        interest(2)
9574        menu(menuAlarmStat)
9575        special(SPC_NOMOD)
```

```
9576        prompt("New Alarm Status")
9577    }
9578    field(NSEV, DBF_MENU) {
9579        interest(2)
9580        menu(menuAlarmSevr)
9581        special(SPC_NOMOD)
9582        prompt("New Alarm Severity")
9583    }
9584    field(NAMSG, DBF_STRING) {
9585        size(40)
9586        special(SPC_NOMOD)
9587        prompt("New Alarm Message")
9588    }
9589    field(ACKS, DBF_MENU) {
9590        interest(2)
9591        menu(menuAlarmSevr)
9592        special(SPC_NOMOD)
9593        prompt("Alarm Ack Severity")
9594    }
9595    field(ACKT, DBF_MENU) {
9596        promptgroup("70 - Alarm")
9597        interest(2)
9598        initial("YES")
9599        menu(menuYesNo)
9600        special(SPC_NOMOD)
9601        prompt("Alarm Ack Transient")
9602    }
9603    field(DISS, DBF_MENU) {
9604        promptgroup("70 - Alarm")
9605        interest(1)
9606        menu(menuAlarmSevr)
9607        prompt("Disable Alarm Sevrty")
9608    }
9609    field(LCNT, DBF_UCHAR) {
9610        interest(2)
9611        special(SPC_NOMOD)
9612        prompt("Lock Count")
9613    }
9614    field(PACT, DBF_UCHAR) {
9615        interest(1)
9616        special(SPC_NOMOD)
9617        prompt("Record active")
9618    }
9619    field(PUTF, DBF_UCHAR) {
9620        interest(1)
9621        special(SPC_NOMOD)
9622        prompt("dbPutField process")
9623    }
9624    field(RPRO, DBF_UCHAR) {
9625        interest(1)
9626        special(SPC_NOMOD)
9627        prompt("Reprocess ")
9628    }
9629    field(ASP, DBF_NOACCESS) {
9630        interest(4)
9631        extra("struct asgMember    *asp")
9632        special(SPC_NOMOD)
9633        prompt("Access Security Pvt")
9634    }
9635    field(PPN, DBF_NOACCESS) {
9636        interest(4)
9637        extra("struct processNotify *ppn")
9638        special(SPC_NOMOD)
9639        prompt("pprocessNotify")
9640    }
9641    field(PPNR, DBF_NOACCESS) {
9642        interest(4)
9643        extra("struct processNotifyRecord *ppnr")
```

```
        special(SPC_NOMOD)
        prompt("pprocessNotifyRecord")
    }
    field(SPVT, DBF_NOACCESS) {
        interest(4)
        extra("struct scan_element *spvt")
        special(SPC_NOMOD)
        prompt("Scan Private")
    }
    field(RSET, DBF_NOACCESS) {
        interest(4)
        extra("struct typed_rset   *rset")
        special(SPC_NOMOD)
        prompt("Address of RSET")
    }
    field(DSET, DBF_NOACCESS) {
        interest(4)
        extra("unambiguous_dset    *dset")
        special(SPC_NOMOD)
        prompt("DSET address")
    }
    field(DPVT, DBF_NOACCESS) {
        interest(4)
        extra("void                *dpvt")
        special(SPC_NOMOD)
        prompt("Device Private")
    }
    field(RDES, DBF_NOACCESS) {
        interest(4)
        extra("struct dbRecordType *rdes")
        special(SPC_NOMOD)
        prompt("Address of dbRecordType")
    }
    field(LSET, DBF_NOACCESS) {
        interest(4)
        extra("struct lockRecord   *lset")
        special(SPC_NOMOD)
        prompt("Lock Set")
    }
    field(PRIO, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
```

```
9712        }
9713        field(TIME, DBF_NOACCESS) {
9714            interest(2)
9715            extra("epicsTimeStamp      time")
9716            special(SPC_NOMOD)
9717            prompt("Time")
9718        }
9719        field(UTAG, DBF_UINT64) {
9720            interest(3)
9721            special(SPC_NOMOD)
9722            prompt("Time Tag")
9723        }
9724        field(FLNK, DBF_FWDLINK) {
9725            promptgroup("20 - Scan")
9726            interest(1)
9727            prompt("Forward Process Link")
9728        }
9729        field(VAL, DBF_INT64) {
9730            promptgroup("50 - Output")
9731            asl(ASL0)
9732            pp(TRUE)
9733            prompt("Desired Output")
9734        }
9735        field(OUT, DBF_OUTLINK) {
9736            promptgroup("50 - Output")
9737            interest(1)
9738            prompt("Output Specification")
9739        }
9740        field(DOL, DBF_INLINK) {
9741            promptgroup("40 - Input")
9742            interest(1)
9743            prompt("Desired Output Link")
9744        }
9745        field(OMSL, DBF_MENU) {
9746            promptgroup("50 - Output")
9747            interest(1)
9748            menu(menuOmsl)
9749            prompt("Output Mode Select")
9750        }
9751        field(EGU, DBF_STRING) {
9752            size(16)
9753            promptgroup("80 - Display")
9754            interest(1)
9755            prop(YES)
9756            prompt("Units name")
9757        }
9758        field(DRVH, DBF_INT64) {
9759            promptgroup("30 - Action")
9760            interest(1)
9761            pp(TRUE)
9762            prop(YES)
9763            prompt("Drive High Limit")
9764        }
9765        field(DRVL, DBF_INT64) {
9766            promptgroup("30 - Action")
9767            interest(1)
9768            pp(TRUE)
9769            prop(YES)
9770            prompt("Drive Low Limit")
9771        }
9772        field(HOPR, DBF_INT64) {
9773            promptgroup("80 - Display")
9774            interest(1)
9775            prop(YES)
9776            prompt("High Operating Range")
9777        }
9778        field(LOPR, DBF_INT64) {
9779            promptgroup("80 - Display")
```

```
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Hihi Severity")
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Lolo Severity")
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("High Severity")
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Low Severity")
    }
    field(HYST, DBF_INT64) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(ADEL, DBF_INT64) {
        promptgroup("80 - Display")
        interest(1)
```

```
9848        prompt("Archive Deadband")
9849    }
9850    field(MDEL, DBF_INT64) {
9851        promptgroup("80 - Display")
9852        interest(1)
9853        prompt("Monitor Deadband")
9854    }
9855    field(LALM, DBF_INT64) {
9856        interest(3)
9857        special(SPC_NOMOD)
9858        prompt("Last Value Alarmed")
9859    }
9860    field(ALST, DBF_INT64) {
9861        interest(3)
9862        special(SPC_NOMOD)
9863        prompt("Last Value Archived")
9864    }
9865    field(MLST, DBF_INT64) {
9866        interest(3)
9867        special(SPC_NOMOD)
9868        prompt("Last Val Monitored")
9869    }
9870    field(SIOL, DBF_OUTLINK) {
9871        promptgroup("90 - Simulate")
9872        interest(1)
9873        prompt("Simulation Output Link")
9874    }
9875    field(SIML, DBF_INLINK) {
9876        promptgroup("90 - Simulate")
9877        interest(1)
9878        prompt("Simulation Mode Link")
9879    }
9880    field(SIMM, DBF_MENU) {
9881        interest(1)
9882        menu(menuYesNo)
9883        special(SPC_MOD)
9884        prompt("Simulation Mode")
9885    }
9886    field(SIMS, DBF_MENU) {
9887        promptgroup("90 - Simulate")
9888        interest(2)
9889        menu(menuAlarmSevr)
9890        prompt("Simulation Mode Severity")
9891    }
9892    field(OLDSIMM, DBF_MENU) {
9893        interest(4)
9894        menu(menuSimm)
9895        special(SPC_NOMOD)
9896        prompt("Prev. Simulation Mode")
9897    }
9898    field(SSCN, DBF_MENU) {
9899        promptgroup("90 - Simulate")
9900        interest(1)
9901        initial("65535")
9902        menu(menuScan)
9903        prompt("Sim. Mode Scan")
9904    }
9905    field(SDLY, DBF_DOUBLE) {
9906        promptgroup("90 - Simulate")
9907        interest(2)
9908        initial("-1.0")
9909        prompt("Sim. Mode Async Delay")
9910    }
9911    field(SIMPVT, DBF_NOACCESS) {
9912        interest(4)
9913        extra("epicsCallback            *simpvt")
9914        special(SPC_NOMOD)
9915        prompt("Sim. Mode Private")
```

```
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID output action")
    }
    field(IVOV, DBF_INT64) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
}
device(int64out, CONSTANT, devI64outSoft, "Soft Channel")
device(int64out, CONSTANT, devI64outSoftCallback, "Async Soft Channel")
device(int64out, INST_IO, devPSCRegI64o, "PSC Reg")
device(int64out, INST_IO, devPSCSingleS64I64o, "PSC Single I64")
recordtype(longin) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct longinRecord;
    %typedef struct longindset {
    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
    %    long (*read_longin)(struct longinRecord *prec); /*returns: (-1,0)=>(failure,success)*/
    %} longindset;
    %#define HAS_longindset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
```

```
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
```

```
10052         prompt("Alarm Status")
10053     }
10054     field(SEVR, DBF_MENU) {
10055         menu(menuAlarmSevr)
10056         special(SPC_NOMOD)
10057         prompt("Alarm Severity")
10058     }
10059     field(AMSG, DBF_STRING) {
10060         size(40)
10061         special(SPC_NOMOD)
10062         prompt("Alarm Message")
10063     }
10064     field(NSTA, DBF_MENU) {
10065         interest(2)
10066         menu(menuAlarmStat)
10067         special(SPC_NOMOD)
10068         prompt("New Alarm Status")
10069     }
10070     field(NSEV, DBF_MENU) {
10071         interest(2)
10072         menu(menuAlarmSevr)
10073         special(SPC_NOMOD)
10074         prompt("New Alarm Severity")
10075     }
10076     field(NAMSG, DBF_STRING) {
10077         size(40)
10078         special(SPC_NOMOD)
10079         prompt("New Alarm Message")
10080     }
10081     field(ACKS, DBF_MENU) {
10082         interest(2)
10083         menu(menuAlarmSevr)
10084         special(SPC_NOMOD)
10085         prompt("Alarm Ack Severity")
10086     }
10087     field(ACKT, DBF_MENU) {
10088         promptgroup("70 - Alarm")
10089         interest(2)
10090         initial("YES")
10091         menu(menuYesNo)
10092         special(SPC_NOMOD)
10093         prompt("Alarm Ack Transient")
10094     }
10095     field(DISS, DBF_MENU) {
10096         promptgroup("70 - Alarm")
10097         interest(1)
10098         menu(menuAlarmSevr)
10099         prompt("Disable Alarm Sevrty")
10100     }
10101     field(LCNT, DBF_UCHAR) {
10102         interest(2)
10103         special(SPC_NOMOD)
10104         prompt("Lock Count")
10105     }
10106     field(PACT, DBF_UCHAR) {
10107         interest(1)
10108         special(SPC_NOMOD)
10109         prompt("Record active")
10110     }
10111     field(PUTF, DBF_UCHAR) {
10112         interest(1)
10113         special(SPC_NOMOD)
10114         prompt("dbPutField process")
10115     }
10116     field(RPRO, DBF_UCHAR) {
10117         interest(1)
10118         special(SPC_NOMOD)
10119         prompt("Reprocess ")
```

```
10120     }
10121     field(ASP, DBF_NOACCESS) {
10122         interest(4)
10123         extra("struct asgMember    *asp")
10124         special(SPC_NOMOD)
10125         prompt("Access Security Pvt")
10126     }
10127     field(PPN, DBF_NOACCESS) {
10128         interest(4)
10129         extra("struct processNotify *ppn")
10130         special(SPC_NOMOD)
10131         prompt("pprocessNotify")
10132     }
10133     field(PPNR, DBF_NOACCESS) {
10134         interest(4)
10135         extra("struct processNotifyRecord *ppnr")
10136         special(SPC_NOMOD)
10137         prompt("pprocessNotifyRecord")
10138     }
10139     field(SPVT, DBF_NOACCESS) {
10140         interest(4)
10141         extra("struct scan_element *spvt")
10142         special(SPC_NOMOD)
10143         prompt("Scan Private")
10144     }
10145     field(RSET, DBF_NOACCESS) {
10146         interest(4)
10147         extra("struct typed_rset   *rset")
10148         special(SPC_NOMOD)
10149         prompt("Address of RSET")
10150     }
10151     field(DSET, DBF_NOACCESS) {
10152         interest(4)
10153         extra("unambiguous_dset    *dset")
10154         special(SPC_NOMOD)
10155         prompt("DSET address")
10156     }
10157     field(DPVT, DBF_NOACCESS) {
10158         interest(4)
10159         extra("void                *dpvt")
10160         special(SPC_NOMOD)
10161         prompt("Device Private")
10162     }
10163     field(RDES, DBF_NOACCESS) {
10164         interest(4)
10165         extra("struct dbRecordType *rdes")
10166         special(SPC_NOMOD)
10167         prompt("Address of dbRecordType")
10168     }
10169     field(LSET, DBF_NOACCESS) {
10170         interest(4)
10171         extra("struct lockRecord   *lset")
10172         special(SPC_NOMOD)
10173         prompt("Lock Set")
10174     }
10175     field(PRIO, DBF_MENU) {
10176         promptgroup("20 - Scan")
10177         interest(1)
10178         menu(menuPriority)
10179         special(SPC_SCAN)
10180         prompt("Scheduling Priority")
10181     }
10182     field(TPRO, DBF_UCHAR) {
10183         prompt("Trace Processing")
10184     }
10185     field(BKPT, DBF_NOACCESS) {
10186         interest(1)
10187         extra("epicsUInt8          bkpt")
```

```
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_LONG) {
        promptgroup("40 - Input")
        asl(ASL0)
        pp(TRUE)
        prompt("Current value")
    }
    field(INP, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input Specification")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(HOPR, DBF_LONG) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_LONG) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
```

```
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HYST, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(AFTC, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(AFVL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
```

```
10324         prompt("Alarm Filter Value")
10325     }
10326     field(ADEL, DBF_LONG) {
10327         promptgroup("80 - Display")
10328         interest(1)
10329         prompt("Archive Deadband")
10330     }
10331     field(MDEL, DBF_LONG) {
10332         promptgroup("80 - Display")
10333         interest(1)
10334         prompt("Monitor Deadband")
10335     }
10336     field(LALM, DBF_LONG) {
10337         interest(3)
10338         special(SPC_NOMOD)
10339         prompt("Last Value Alarmed")
10340     }
10341     field(ALST, DBF_LONG) {
10342         interest(3)
10343         special(SPC_NOMOD)
10344         prompt("Last Value Archived")
10345     }
10346     field(MLST, DBF_LONG) {
10347         interest(3)
10348         special(SPC_NOMOD)
10349         prompt("Last Val Monitored")
10350     }
10351     field(SIOL, DBF_INLINK) {
10352         promptgroup("90 - Simulate")
10353         interest(1)
10354         prompt("Sim Input Specifctn")
10355     }
10356     field(SVAL, DBF_LONG) {
10357         prompt("Simulation Value")
10358     }
10359     field(SIML, DBF_INLINK) {
10360         promptgroup("90 - Simulate")
10361         interest(1)
10362         prompt("Sim Mode Location")
10363     }
10364     field(SIMM, DBF_MENU) {
10365         interest(1)
10366         menu(menuYesNo)
10367         special(SPC_MOD)
10368         prompt("Simulation Mode")
10369     }
10370     field(SIMS, DBF_MENU) {
10371         promptgroup("90 - Simulate")
10372         interest(2)
10373         menu(menuAlarmSevr)
10374         prompt("Sim mode Alarm Svrty")
10375     }
10376     field(OLDSIMM, DBF_MENU) {
10377         interest(4)
10378         menu(menuSimm)
10379         special(SPC_NOMOD)
10380         prompt("Prev. Simulation Mode")
10381     }
10382     field(SSCN, DBF_MENU) {
10383         promptgroup("90 - Simulate")
10384         interest(1)
10385         initial("65535")
10386         menu(menuScan)
10387         prompt("Sim. Mode Scan")
10388     }
10389     field(SDLY, DBF_DOUBLE) {
10390         promptgroup("90 - Simulate")
10391         interest(2)
```

```
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(longin, CONSTANT, devLiSoft, "Soft Channel")
device(longin, CONSTANT, devLiSoftCallback, "Async Soft Channel")
device(longin, INST_IO, devLiGeneralTime, "General Time")
device(longin, INST_IO, devPSCUknCountLi, "PSC Unknown Msg Count")
device(longin, INST_IO, devPSCConnCountLi, "PSC Conn Count")
device(longin, INST_IO, devPSCBlockCountLi, "PSC Block Msg Count")
device(longin, INST_IO, devPSCRegLi, "PSC Reg")
device(longin, INST_IO, devPSCRegRBLi, "PSC RB Reg")
recordtype(longout) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct longoutRecord;
    %typedef struct longoutdset {
    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
    %    long (*write_longout)(struct longoutRecord *prec); /*(-1,0)=>(failure,success*/
    %} longoutdset;
    %#define HAS_longoutdset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
```

```
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
```

```
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("dbPutField process")
    }
    field(RPRO, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
```

```
10596         prompt("Reprocess ")
10597     }
10598     field(ASP, DBF_NOACCESS) {
10599         interest(4)
10600         extra("struct asgMember    *asp")
10601         special(SPC_NOMOD)
10602         prompt("Access Security Pvt")
10603     }
10604     field(PPN, DBF_NOACCESS) {
10605         interest(4)
10606         extra("struct processNotify *ppn")
10607         special(SPC_NOMOD)
10608         prompt("pprocessNotify")
10609     }
10610     field(PPNR, DBF_NOACCESS) {
10611         interest(4)
10612         extra("struct processNotifyRecord *ppnr")
10613         special(SPC_NOMOD)
10614         prompt("pprocessNotifyRecord")
10615     }
10616     field(SPVT, DBF_NOACCESS) {
10617         interest(4)
10618         extra("struct scan_element *spvt")
10619         special(SPC_NOMOD)
10620         prompt("Scan Private")
10621     }
10622     field(RSET, DBF_NOACCESS) {
10623         interest(4)
10624         extra("struct typed_rset   *rset")
10625         special(SPC_NOMOD)
10626         prompt("Address of RSET")
10627     }
10628     field(DSET, DBF_NOACCESS) {
10629         interest(4)
10630         extra("unambiguous_dset    *dset")
10631         special(SPC_NOMOD)
10632         prompt("DSET address")
10633     }
10634     field(DPVT, DBF_NOACCESS) {
10635         interest(4)
10636         extra("void                *dpvt")
10637         special(SPC_NOMOD)
10638         prompt("Device Private")
10639     }
10640     field(RDES, DBF_NOACCESS) {
10641         interest(4)
10642         extra("struct dbRecordType *rdes")
10643         special(SPC_NOMOD)
10644         prompt("Address of dbRecordType")
10645     }
10646     field(LSET, DBF_NOACCESS) {
10647         interest(4)
10648         extra("struct lockRecord   *lset")
10649         special(SPC_NOMOD)
10650         prompt("Lock Set")
10651     }
10652     field(PRIO, DBF_MENU) {
10653         promptgroup("20 - Scan")
10654         interest(1)
10655         menu(menuPriority)
10656         special(SPC_SCAN)
10657         prompt("Scheduling Priority")
10658     }
10659     field(TPRO, DBF_UCHAR) {
10660         prompt("Trace Processing")
10661     }
10662     field(BKPT, DBF_NOACCESS) {
10663         interest(1)
```

```
10664         extra("epicsUInt8          bkpt")
10665         special(SPC_NOMOD)
10666         prompt("Break Point")
10667     }
10668     field(UDF, DBF_UCHAR) {
10669         promptgroup("10 - Common")
10670         interest(1)
10671         initial("1")
10672         pp(TRUE)
10673         prompt("Undefined")
10674     }
10675     field(UDFS, DBF_MENU) {
10676         promptgroup("70 - Alarm")
10677         interest(1)
10678         initial("INVALID")
10679         menu(menuAlarmSevr)
10680         prompt("Undefined Alarm Sevrty")
10681     }
10682     field(TIME, DBF_NOACCESS) {
10683         interest(2)
10684         extra("epicsTimeStamp      time")
10685         special(SPC_NOMOD)
10686         prompt("Time")
10687     }
10688     field(UTAG, DBF_UINT64) {
10689         interest(3)
10690         special(SPC_NOMOD)
10691         prompt("Time Tag")
10692     }
10693     field(FLNK, DBF_FWDLINK) {
10694         promptgroup("20 - Scan")
10695         interest(1)
10696         prompt("Forward Process Link")
10697     }
10698     field(VAL, DBF_LONG) {
10699         promptgroup("50 - Output")
10700         asl(ASL0)
10701         pp(TRUE)
10702         prompt("Desired Output")
10703     }
10704     field(OUT, DBF_OUTLINK) {
10705         promptgroup("50 - Output")
10706         interest(1)
10707         special(SPC_MOD)
10708         prompt("Output Specification")
10709     }
10710     field(DOL, DBF_INLINK) {
10711         promptgroup("40 - Input")
10712         interest(1)
10713         prompt("Desired Output Link")
10714     }
10715     field(OMSL, DBF_MENU) {
10716         promptgroup("50 - Output")
10717         interest(1)
10718         menu(menuOmsl)
10719         prompt("Output Mode Select")
10720     }
10721     field(EGU, DBF_STRING) {
10722         size(16)
10723         promptgroup("80 - Display")
10724         interest(1)
10725         prop(YES)
10726         prompt("Engineering Units")
10727     }
10728     field(DRVH, DBF_LONG) {
10729         promptgroup("30 - Action")
10730         interest(1)
10731         pp(TRUE)
```

```
        prop(YES)
        prompt("Drive High Limit")
    }
    field(DRVL, DBF_LONG) {
        promptgroup("30 - Action")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Drive Low Limit")
    }
    field(HOPR, DBF_LONG) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_LONG) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_LONG) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
```

```
10800         menu(menuAlarmSevr)
10801         prompt("High Severity")
10802         prop(YES)
10803         interest(1)
10804         pp(TRUE)
10805     }
10806     field(LSV, DBF_MENU) {
10807         promptgroup("70 - Alarm")
10808         menu(menuAlarmSevr)
10809         prompt("Low Severity")
10810         prop(YES)
10811         interest(1)
10812         pp(TRUE)
10813     }
10814     field(HYST, DBF_LONG) {
10815         promptgroup("70 - Alarm")
10816         interest(1)
10817         prompt("Alarm Deadband")
10818     }
10819     field(ADEL, DBF_LONG) {
10820         promptgroup("80 - Display")
10821         interest(1)
10822         prompt("Archive Deadband")
10823     }
10824     field(MDEL, DBF_LONG) {
10825         promptgroup("80 - Display")
10826         interest(1)
10827         prompt("Monitor Deadband")
10828     }
10829     field(LALM, DBF_LONG) {
10830         interest(3)
10831         special(SPC_NOMOD)
10832         prompt("Last Value Alarmed")
10833     }
10834     field(ALST, DBF_LONG) {
10835         interest(3)
10836         special(SPC_NOMOD)
10837         prompt("Last Value Archived")
10838     }
10839     field(MLST, DBF_LONG) {
10840         interest(3)
10841         special(SPC_NOMOD)
10842         prompt("Last Val Monitored")
10843     }
10844     field(SIOL, DBF_OUTLINK) {
10845         promptgroup("90 - Simulate")
10846         interest(1)
10847         prompt("Sim Output Specifctn")
10848     }
10849     field(SIML, DBF_INLINK) {
10850         promptgroup("90 - Simulate")
10851         interest(1)
10852         prompt("Sim Mode Location")
10853     }
10854     field(SIMM, DBF_MENU) {
10855         interest(1)
10856         menu(menuYesNo)
10857         special(SPC_MOD)
10858         prompt("Simulation Mode")
10859     }
10860     field(SIMS, DBF_MENU) {
10861         promptgroup("90 - Simulate")
10862         interest(2)
10863         menu(menuAlarmSevr)
10864         prompt("Sim mode Alarm Svrty")
10865     }
10866     field(OLDSIMM, DBF_MENU) {
10867         interest(4)
```

```
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback       *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID output action")
    }
    field(IVOV, DBF_LONG) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
    field(PVAL, DBF_LONG) {
        prompt("Previous Value")
    }
    field(OUTPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsEnum16         outpvt")
        special(SPC_NOMOD)
        prompt("Output Write Control Private")
    }
    field(OOCH, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        initial("1")
        menu(menuYesNo)
        prompt("Output Exec. On Change (Opt)")
    }
    field(OOPT, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        initial("0")
        menu(longoutOOPT)
        prompt("Output Execute Opt")
    }
}
device(longout, CONSTANT, devLoSoft, "Soft Channel")
device(longout, CONSTANT, devLoSoftCallback, "Async Soft Channel")
device(longout, INST_IO, devPSCSendLo, "PSC Ctrl Send")
device(longout, INST_IO, devPSCRegLo, "PSC Reg")
device(longout, INST_IO, devPSCSingleS32Lo, "PSC Single I32")
recordtype(lsi) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
```

```
10936     %#include "devSup.h"
10937     %#include "epicsTime.h"
10938     %
10939     %/* Declare Device Support Entry Table */
10940     %struct lsiRecord;
10941     %typedef struct lsidset {
10942     %    dset common;
10943     %    long (*read_string)(struct lsiRecord *prec);
10944     %} lsidset;
10945     %#define HAS_lsidset
10946     %
10947     %#include "callback.h"
10948     field(NAME, DBF_STRING) {
10949         size(61)
10950         special(SPC_NOMOD)
10951         prompt("Record Name")
10952     }
10953     field(DESC, DBF_STRING) {
10954         size(41)
10955         promptgroup("10 - Common")
10956         prompt("Descriptor")
10957     }
10958     field(ASG, DBF_STRING) {
10959         size(29)
10960         promptgroup("10 - Common")
10961         special(SPC_AS)
10962         prompt("Access Security Group")
10963     }
10964     field(SCAN, DBF_MENU) {
10965         promptgroup("20 - Scan")
10966         interest(1)
10967         menu(menuScan)
10968         special(SPC_SCAN)
10969         prompt("Scan Mechanism")
10970     }
10971     field(PINI, DBF_MENU) {
10972         promptgroup("20 - Scan")
10973         interest(1)
10974         menu(menuPini)
10975         prompt("Process at iocInit")
10976     }
10977     field(PHAS, DBF_SHORT) {
10978         promptgroup("20 - Scan")
10979         interest(1)
10980         special(SPC_SCAN)
10981         prompt("Scan Phase")
10982     }
10983     field(EVNT, DBF_STRING) {
10984         size(40)
10985         promptgroup("20 - Scan")
10986         interest(1)
10987         special(SPC_SCAN)
10988         prompt("Event Name")
10989     }
10990     field(TSE, DBF_SHORT) {
10991         promptgroup("20 - Scan")
10992         interest(1)
10993         prompt("Time Stamp Event")
10994     }
10995     field(TSEL, DBF_INLINK) {
10996         promptgroup("20 - Scan")
10997         interest(1)
10998         prompt("Time Stamp Link")
10999     }
11000     field(DTYP, DBF_DEVICE) {
11001         promptgroup("10 - Common")
11002         interest(1)
11003         prompt("Device Type")
```

```
11004     }
11005     field(DISV, DBF_SHORT) {
11006         promptgroup("20 - Scan")
11007         initial("1")
11008         prompt("Disable Value")
11009     }
11010     field(DISA, DBF_SHORT) {
11011         prompt("Disable")
11012     }
11013     field(SDIS, DBF_INLINK) {
11014         promptgroup("20 - Scan")
11015         interest(1)
11016         prompt("Scanning Disable")
11017     }
11018     field(MLOK, DBF_NOACCESS) {
11019         interest(4)
11020         extra("epicsMutexId        mlok")
11021         special(SPC_NOMOD)
11022         prompt("Monitor lock")
11023     }
11024     field(MLIS, DBF_NOACCESS) {
11025         interest(4)
11026         extra("ELLLIST             mlis")
11027         special(SPC_NOMOD)
11028         prompt("Monitor List")
11029     }
11030     field(BKLNK, DBF_NOACCESS) {
11031         interest(4)
11032         extra("ELLLIST             bklnk")
11033         special(SPC_NOMOD)
11034         prompt("Backwards link tracking")
11035     }
11036     field(DISP, DBF_UCHAR) {
11037         promptgroup("10 - Common")
11038         interest(1)
11039         prompt("Disable putField")
11040     }
11041     field(PROC, DBF_UCHAR) {
11042         interest(3)
11043         pp(TRUE)
11044         prompt("Force Processing")
11045     }
11046     field(STAT, DBF_MENU) {
11047         initial("UDF")
11048         menu(menuAlarmStat)
11049         special(SPC_NOMOD)
11050         prompt("Alarm Status")
11051     }
11052     field(SEVR, DBF_MENU) {
11053         menu(menuAlarmSevr)
11054         special(SPC_NOMOD)
11055         prompt("Alarm Severity")
11056     }
11057     field(AMSG, DBF_STRING) {
11058         size(40)
11059         special(SPC_NOMOD)
11060         prompt("Alarm Message")
11061     }
11062     field(NSTA, DBF_MENU) {
11063         interest(2)
11064         menu(menuAlarmStat)
11065         special(SPC_NOMOD)
11066         prompt("New Alarm Status")
11067     }
11068     field(NSEV, DBF_MENU) {
11069         interest(2)
11070         menu(menuAlarmSevr)
11071         special(SPC_NOMOD)
```

```
11072         prompt("New Alarm Severity")
11073     }
11074     field(NAMSG, DBF_STRING) {
11075         size(40)
11076         special(SPC_NOMOD)
11077         prompt("New Alarm Message")
11078     }
11079     field(ACKS, DBF_MENU) {
11080         interest(2)
11081         menu(menuAlarmSevr)
11082         special(SPC_NOMOD)
11083         prompt("Alarm Ack Severity")
11084     }
11085     field(ACKT, DBF_MENU) {
11086         promptgroup("70 - Alarm")
11087         interest(2)
11088         initial("YES")
11089         menu(menuYesNo)
11090         special(SPC_NOMOD)
11091         prompt("Alarm Ack Transient")
11092     }
11093     field(DISS, DBF_MENU) {
11094         promptgroup("70 - Alarm")
11095         interest(1)
11096         menu(menuAlarmSevr)
11097         prompt("Disable Alarm Sevrty")
11098     }
11099     field(LCNT, DBF_UCHAR) {
11100         interest(2)
11101         special(SPC_NOMOD)
11102         prompt("Lock Count")
11103     }
11104     field(PACT, DBF_UCHAR) {
11105         interest(1)
11106         special(SPC_NOMOD)
11107         prompt("Record active")
11108     }
11109     field(PUTF, DBF_UCHAR) {
11110         interest(1)
11111         special(SPC_NOMOD)
11112         prompt("dbPutField process")
11113     }
11114     field(RPRO, DBF_UCHAR) {
11115         interest(1)
11116         special(SPC_NOMOD)
11117         prompt("Reprocess ")
11118     }
11119     field(ASP, DBF_NOACCESS) {
11120         interest(4)
11121         extra("struct asgMember    *asp")
11122         special(SPC_NOMOD)
11123         prompt("Access Security Pvt")
11124     }
11125     field(PPN, DBF_NOACCESS) {
11126         interest(4)
11127         extra("struct processNotify *ppn")
11128         special(SPC_NOMOD)
11129         prompt("pprocessNotify")
11130     }
11131     field(PPNR, DBF_NOACCESS) {
11132         interest(4)
11133         extra("struct processNotifyRecord *ppnr")
11134         special(SPC_NOMOD)
11135         prompt("pprocessNotifyRecord")
11136     }
11137     field(SPVT, DBF_NOACCESS) {
11138         interest(4)
11139         extra("struct scan_element *spvt")
```

```
11140         special(SPC_NOMOD)
11141         prompt("Scan Private")
11142     }
11143     field(RSET, DBF_NOACCESS) {
11144         interest(4)
11145         extra("struct typed_rset   *rset")
11146         special(SPC_NOMOD)
11147         prompt("Address of RSET")
11148     }
11149     field(DSET, DBF_NOACCESS) {
11150         interest(4)
11151         extra("unambiguous_dset    *dset")
11152         special(SPC_NOMOD)
11153         prompt("DSET address")
11154     }
11155     field(DPVT, DBF_NOACCESS) {
11156         interest(4)
11157         extra("void                *dpvt")
11158         special(SPC_NOMOD)
11159         prompt("Device Private")
11160     }
11161     field(RDES, DBF_NOACCESS) {
11162         interest(4)
11163         extra("struct dbRecordType *rdes")
11164         special(SPC_NOMOD)
11165         prompt("Address of dbRecordType")
11166     }
11167     field(LSET, DBF_NOACCESS) {
11168         interest(4)
11169         extra("struct lockRecord   *lset")
11170         special(SPC_NOMOD)
11171         prompt("Lock Set")
11172     }
11173     field(PRIO, DBF_MENU) {
11174         promptgroup("20 - Scan")
11175         interest(1)
11176         menu(menuPriority)
11177         special(SPC_SCAN)
11178         prompt("Scheduling Priority")
11179     }
11180     field(TPRO, DBF_UCHAR) {
11181         prompt("Trace Processing")
11182     }
11183     field(BKPT, DBF_NOACCESS) {
11184         interest(1)
11185         extra("epicsUInt8          bkpt")
11186         special(SPC_NOMOD)
11187         prompt("Break Point")
11188     }
11189     field(UDF, DBF_UCHAR) {
11190         promptgroup("10 - Common")
11191         interest(1)
11192         initial("1")
11193         pp(TRUE)
11194         prompt("Undefined")
11195     }
11196     field(UDFS, DBF_MENU) {
11197         promptgroup("70 - Alarm")
11198         interest(1)
11199         initial("INVALID")
11200         menu(menuAlarmSevr)
11201         prompt("Undefined Alarm Sevrty")
11202     }
11203     field(TIME, DBF_NOACCESS) {
11204         interest(2)
11205         extra("epicsTimeStamp      time")
11206         special(SPC_NOMOD)
11207         prompt("Time")
```

```
11208     }
11209     field(UTAG, DBF_UINT64) {
11210         interest(3)
11211         special(SPC_NOMOD)
11212         prompt("Time Tag")
11213     }
11214     field(FLNK, DBF_FWDLINK) {
11215         promptgroup("20 - Scan")
11216         interest(1)
11217         prompt("Forward Process Link")
11218     }
11219     field(VAL, DBF_NOACCESS) {
11220         extra("char *val")
11221         asl(ASL0)
11222         special(SPC_DBADDR)
11223         pp(TRUE)
11224         prompt("Current Value")
11225     }
11226     field(OVAL, DBF_NOACCESS) {
11227         interest(3)
11228         extra("char *oval")
11229         special(SPC_DBADDR)
11230         prompt("Old Value")
11231     }
11232     field(SIZV, DBF_USHORT) {
11233         promptgroup("40 - Input")
11234         interest(1)
11235         initial("41")
11236         special(SPC_NOMOD)
11237         prompt("Size of buffers")
11238     }
11239     field(LEN, DBF_ULONG) {
11240         special(SPC_NOMOD)
11241         prompt("Length of VAL")
11242     }
11243     field(OLEN, DBF_ULONG) {
11244         special(SPC_NOMOD)
11245         prompt("Length of OVAL")
11246     }
11247     field(INP, DBF_INLINK) {
11248         promptgroup("40 - Input")
11249         interest(1)
11250         prompt("Input Specification")
11251     }
11252     field(MPST, DBF_MENU) {
11253         promptgroup("80 - Display")
11254         interest(1)
11255         menu(menuPost)
11256         prompt("Post Value Monitors")
11257     }
11258     field(APST, DBF_MENU) {
11259         promptgroup("80 - Display")
11260         interest(1)
11261         menu(menuPost)
11262         prompt("Post Archive Monitors")
11263     }
11264     field(SIML, DBF_INLINK) {
11265         promptgroup("90 - Simulate")
11266         interest(1)
11267         prompt("Simulation Mode Link")
11268     }
11269     field(SIMM, DBF_MENU) {
11270         interest(1)
11271         menu(menuYesNo)
11272         special(SPC_MOD)
11273         prompt("Simulation Mode")
11274     }
11275     field(SIMS, DBF_MENU) {
```

```
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Input Link")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(lsi, CONSTANT, devLsiSoft, "Soft Channel")
device(lsi, INST_IO, devLsiEnviron, "getenv")
recordtype(lso) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct lsoRecord;
    %typedef struct lsodset {
    %    dset common;
    %    long (*write_string)(struct lsoRecord *prec);
    %} lsodset;
    %#define HAS_lsodset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
```

```
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
```

```
11412     }
11413     field(BKLNK, DBF_NOACCESS) {
11414         interest(4)
11415         extra("ELLLIST             bklnk")
11416         special(SPC_NOMOD)
11417         prompt("Backwards link tracking")
11418     }
11419     field(DISP, DBF_UCHAR) {
11420         promptgroup("10 - Common")
11421         interest(1)
11422         prompt("Disable putField")
11423     }
11424     field(PROC, DBF_UCHAR) {
11425         interest(3)
11426         pp(TRUE)
11427         prompt("Force Processing")
11428     }
11429     field(STAT, DBF_MENU) {
11430         initial("UDF")
11431         menu(menuAlarmStat)
11432         special(SPC_NOMOD)
11433         prompt("Alarm Status")
11434     }
11435     field(SEVR, DBF_MENU) {
11436         menu(menuAlarmSevr)
11437         special(SPC_NOMOD)
11438         prompt("Alarm Severity")
11439     }
11440     field(AMSG, DBF_STRING) {
11441         size(40)
11442         special(SPC_NOMOD)
11443         prompt("Alarm Message")
11444     }
11445     field(NSTA, DBF_MENU) {
11446         interest(2)
11447         menu(menuAlarmStat)
11448         special(SPC_NOMOD)
11449         prompt("New Alarm Status")
11450     }
11451     field(NSEV, DBF_MENU) {
11452         interest(2)
11453         menu(menuAlarmSevr)
11454         special(SPC_NOMOD)
11455         prompt("New Alarm Severity")
11456     }
11457     field(NAMSG, DBF_STRING) {
11458         size(40)
11459         special(SPC_NOMOD)
11460         prompt("New Alarm Message")
11461     }
11462     field(ACKS, DBF_MENU) {
11463         interest(2)
11464         menu(menuAlarmSevr)
11465         special(SPC_NOMOD)
11466         prompt("Alarm Ack Severity")
11467     }
11468     field(ACKT, DBF_MENU) {
11469         promptgroup("70 - Alarm")
11470         interest(2)
11471         initial("YES")
11472         menu(menuYesNo)
11473         special(SPC_NOMOD)
11474         prompt("Alarm Ack Transient")
11475     }
11476     field(DISS, DBF_MENU) {
11477         promptgroup("70 - Alarm")
11478         interest(1)
11479         menu(menuAlarmSevr)
```

```
11480         prompt("Disable Alarm Sevrty")
11481     }
11482     field(LCNT, DBF_UCHAR) {
11483         interest(2)
11484         special(SPC_NOMOD)
11485         prompt("Lock Count")
11486     }
11487     field(PACT, DBF_UCHAR) {
11488         interest(1)
11489         special(SPC_NOMOD)
11490         prompt("Record active")
11491     }
11492     field(PUTF, DBF_UCHAR) {
11493         interest(1)
11494         special(SPC_NOMOD)
11495         prompt("dbPutField process")
11496     }
11497     field(RPRO, DBF_UCHAR) {
11498         interest(1)
11499         special(SPC_NOMOD)
11500         prompt("Reprocess ")
11501     }
11502     field(ASP, DBF_NOACCESS) {
11503         interest(4)
11504         extra("struct asgMember    *asp")
11505         special(SPC_NOMOD)
11506         prompt("Access Security Pvt")
11507     }
11508     field(PPN, DBF_NOACCESS) {
11509         interest(4)
11510         extra("struct processNotify *ppn")
11511         special(SPC_NOMOD)
11512         prompt("pprocessNotify")
11513     }
11514     field(PPNR, DBF_NOACCESS) {
11515         interest(4)
11516         extra("struct processNotifyRecord *ppnr")
11517         special(SPC_NOMOD)
11518         prompt("pprocessNotifyRecord")
11519     }
11520     field(SPVT, DBF_NOACCESS) {
11521         interest(4)
11522         extra("struct scan_element *spvt")
11523         special(SPC_NOMOD)
11524         prompt("Scan Private")
11525     }
11526     field(RSET, DBF_NOACCESS) {
11527         interest(4)
11528         extra("struct typed_rset   *rset")
11529         special(SPC_NOMOD)
11530         prompt("Address of RSET")
11531     }
11532     field(DSET, DBF_NOACCESS) {
11533         interest(4)
11534         extra("unambiguous_dset    *dset")
11535         special(SPC_NOMOD)
11536         prompt("DSET address")
11537     }
11538     field(DPVT, DBF_NOACCESS) {
11539         interest(4)
11540         extra("void                *dpvt")
11541         special(SPC_NOMOD)
11542         prompt("Device Private")
11543     }
11544     field(RDES, DBF_NOACCESS) {
11545         interest(4)
11546         extra("struct dbRecordType *rdes")
11547         special(SPC_NOMOD)
```

```
11548         prompt("Address of dbRecordType")
11549     }
11550     field(LSET, DBF_NOACCESS) {
11551         interest(4)
11552         extra("struct lockRecord   *lset")
11553         special(SPC_NOMOD)
11554         prompt("Lock Set")
11555     }
11556     field(PRIO, DBF_MENU) {
11557         promptgroup("20 - Scan")
11558         interest(1)
11559         menu(menuPriority)
11560         special(SPC_SCAN)
11561         prompt("Scheduling Priority")
11562     }
11563     field(TPRO, DBF_UCHAR) {
11564         prompt("Trace Processing")
11565     }
11566     field(BKPT, DBF_NOACCESS) {
11567         interest(1)
11568         extra("epicsUInt8          bkpt")
11569         special(SPC_NOMOD)
11570         prompt("Break Point")
11571     }
11572     field(UDF, DBF_UCHAR) {
11573         promptgroup("10 - Common")
11574         interest(1)
11575         initial("1")
11576         pp(TRUE)
11577         prompt("Undefined")
11578     }
11579     field(UDFS, DBF_MENU) {
11580         promptgroup("70 - Alarm")
11581         interest(1)
11582         initial("INVALID")
11583         menu(menuAlarmSevr)
11584         prompt("Undefined Alarm Sevrty")
11585     }
11586     field(TIME, DBF_NOACCESS) {
11587         interest(2)
11588         extra("epicsTimeStamp      time")
11589         special(SPC_NOMOD)
11590         prompt("Time")
11591     }
11592     field(UTAG, DBF_UINT64) {
11593         interest(3)
11594         special(SPC_NOMOD)
11595         prompt("Time Tag")
11596     }
11597     field(FLNK, DBF_FWDLINK) {
11598         promptgroup("20 - Scan")
11599         interest(1)
11600         prompt("Forward Process Link")
11601     }
11602     field(VAL, DBF_NOACCESS) {
11603         extra("char *val")
11604         asl(ASL0)
11605         special(SPC_DBADDR)
11606         pp(TRUE)
11607         prompt("Current Value")
11608     }
11609     field(OVAL, DBF_NOACCESS) {
11610         interest(3)
11611         extra("char *oval")
11612         special(SPC_DBADDR)
11613         prompt("Previous Value")
11614     }
11615     field(SIZV, DBF_USHORT) {
```

```
11616         promptgroup("50 - Output")
11617         interest(1)
11618         initial("41")
11619         special(SPC_NOMOD)
11620         prompt("Size of buffers")
11621     }
11622     field(LEN, DBF_ULONG) {
11623         special(SPC_NOMOD)
11624         prompt("Length of VAL")
11625     }
11626     field(OLEN, DBF_ULONG) {
11627         interest(3)
11628         special(SPC_NOMOD)
11629         prompt("Length of OVAL")
11630     }
11631     field(DOL, DBF_INLINK) {
11632         promptgroup("40 - Input")
11633         interest(1)
11634         prompt("Desired Output Link")
11635     }
11636     field(IVOA, DBF_MENU) {
11637         promptgroup("50 - Output")
11638         interest(2)
11639         menu(menuIvoa)
11640         prompt("INVALID Output Action")
11641     }
11642     field(IVOV, DBF_STRING) {
11643         size(40)
11644         promptgroup("50 - Output")
11645         interest(2)
11646         prompt("INVALID Output Value")
11647     }
11648     field(OMSL, DBF_MENU) {
11649         promptgroup("50 - Output")
11650         interest(1)
11651         menu(menuOmsl)
11652         prompt("Output Mode Select")
11653     }
11654     field(OUT, DBF_OUTLINK) {
11655         promptgroup("50 - Output")
11656         interest(1)
11657         prompt("Output Specification")
11658     }
11659     field(MPST, DBF_MENU) {
11660         promptgroup("80 - Display")
11661         interest(1)
11662         menu(menuPost)
11663         prompt("Post Value Monitors")
11664     }
11665     field(APST, DBF_MENU) {
11666         promptgroup("80 - Display")
11667         interest(1)
11668         menu(menuPost)
11669         prompt("Post Archive Monitors")
11670     }
11671     field(SIML, DBF_INLINK) {
11672         promptgroup("90 - Simulate")
11673         interest(1)
11674         prompt("Simulation Mode link")
11675     }
11676     field(SIMM, DBF_MENU) {
11677         interest(1)
11678         menu(menuYesNo)
11679         special(SPC_MOD)
11680         prompt("Simulation Mode")
11681     }
11682     field(SIMS, DBF_MENU) {
11683         promptgroup("90 - Simulate")
```

```
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(SIOL, DBF_OUTLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Output Link")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(lso, CONSTANT, devLsoSoft, "Soft Channel")
device(lso, CONSTANT, devLsoSoftCallback, "Async Soft Channel")
device(lso, INST_IO, devLsoStdio, "stdio")
recordtype(mbbi) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %/* Declare Device Support Entry Table */
    %struct mbbiRecord;
    %typedef struct mbbidset {
    %    dset common; /* init_record returns: (-1,0) => (failure, success)*/
    %    long (*read_mbbi)(struct mbbiRecord *prec); /* (0, 2) => (success, success no convert)*/
    %} mbbidset;
    %#define HAS_mbbidset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
```

```
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
```

```
11820     field(BKLNK, DBF_NOACCESS) {
11821         interest(4)
11822         extra("ELLLIST             bklnk")
11823         special(SPC_NOMOD)
11824         prompt("Backwards link tracking")
11825     }
11826     field(DISP, DBF_UCHAR) {
11827         promptgroup("10 - Common")
11828         interest(1)
11829         prompt("Disable putField")
11830     }
11831     field(PROC, DBF_UCHAR) {
11832         interest(3)
11833         pp(TRUE)
11834         prompt("Force Processing")
11835     }
11836     field(STAT, DBF_MENU) {
11837         initial("UDF")
11838         menu(menuAlarmStat)
11839         special(SPC_NOMOD)
11840         prompt("Alarm Status")
11841     }
11842     field(SEVR, DBF_MENU) {
11843         menu(menuAlarmSevr)
11844         special(SPC_NOMOD)
11845         prompt("Alarm Severity")
11846     }
11847     field(AMSG, DBF_STRING) {
11848         size(40)
11849         special(SPC_NOMOD)
11850         prompt("Alarm Message")
11851     }
11852     field(NSTA, DBF_MENU) {
11853         interest(2)
11854         menu(menuAlarmStat)
11855         special(SPC_NOMOD)
11856         prompt("New Alarm Status")
11857     }
11858     field(NSEV, DBF_MENU) {
11859         interest(2)
11860         menu(menuAlarmSevr)
11861         special(SPC_NOMOD)
11862         prompt("New Alarm Severity")
11863     }
11864     field(NAMSG, DBF_STRING) {
11865         size(40)
11866         special(SPC_NOMOD)
11867         prompt("New Alarm Message")
11868     }
11869     field(ACKS, DBF_MENU) {
11870         interest(2)
11871         menu(menuAlarmSevr)
11872         special(SPC_NOMOD)
11873         prompt("Alarm Ack Severity")
11874     }
11875     field(ACKT, DBF_MENU) {
11876         promptgroup("70 - Alarm")
11877         interest(2)
11878         initial("YES")
11879         menu(menuYesNo)
11880         special(SPC_NOMOD)
11881         prompt("Alarm Ack Transient")
11882     }
11883     field(DISS, DBF_MENU) {
11884         promptgroup("70 - Alarm")
11885         interest(1)
11886         menu(menuAlarmSevr)
11887         prompt("Disable Alarm Sevrty")
```

```
11888     }
11889     field(LCNT, DBF_UCHAR) {
11890         interest(2)
11891         special(SPC_NOMOD)
11892         prompt("Lock Count")
11893     }
11894     field(PACT, DBF_UCHAR) {
11895         interest(1)
11896         special(SPC_NOMOD)
11897         prompt("Record active")
11898     }
11899     field(PUTF, DBF_UCHAR) {
11900         interest(1)
11901         special(SPC_NOMOD)
11902         prompt("dbPutField process")
11903     }
11904     field(RPRO, DBF_UCHAR) {
11905         interest(1)
11906         special(SPC_NOMOD)
11907         prompt("Reprocess ")
11908     }
11909     field(ASP, DBF_NOACCESS) {
11910         interest(4)
11911         extra("struct asgMember    *asp")
11912         special(SPC_NOMOD)
11913         prompt("Access Security Pvt")
11914     }
11915     field(PPN, DBF_NOACCESS) {
11916         interest(4)
11917         extra("struct processNotify *ppn")
11918         special(SPC_NOMOD)
11919         prompt("pprocessNotify")
11920     }
11921     field(PPNR, DBF_NOACCESS) {
11922         interest(4)
11923         extra("struct processNotifyRecord *ppnr")
11924         special(SPC_NOMOD)
11925         prompt("pprocessNotifyRecord")
11926     }
11927     field(SPVT, DBF_NOACCESS) {
11928         interest(4)
11929         extra("struct scan_element *spvt")
11930         special(SPC_NOMOD)
11931         prompt("Scan Private")
11932     }
11933     field(RSET, DBF_NOACCESS) {
11934         interest(4)
11935         extra("struct typed_rset   *rset")
11936         special(SPC_NOMOD)
11937         prompt("Address of RSET")
11938     }
11939     field(DSET, DBF_NOACCESS) {
11940         interest(4)
11941         extra("unambiguous_dset    *dset")
11942         special(SPC_NOMOD)
11943         prompt("DSET address")
11944     }
11945     field(DPVT, DBF_NOACCESS) {
11946         interest(4)
11947         extra("void                *dpvt")
11948         special(SPC_NOMOD)
11949         prompt("Device Private")
11950     }
11951     field(RDES, DBF_NOACCESS) {
11952         interest(4)
11953         extra("struct dbRecordType *rdes")
11954         special(SPC_NOMOD)
11955         prompt("Address of dbRecordType")
```

```
    }
    field(LSET, DBF_NOACCESS) {
        interest(4)
        extra("struct lockRecord   *lset")
        special(SPC_NOMOD)
        prompt("Lock Set")
    }
    field(PRIO, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_ENUM) {
        promptgroup("40 - Input")
        asl(ASL0)
        pp(TRUE)
        prompt("Current Value")
    }
    field(NOBT, DBF_USHORT) {
        promptgroup("40 - Input")
        interest(1)
        special(SPC_NOMOD)
        prompt("Number of Bits")
    }
    field(INP, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
```

```
        prompt("Input Specification")
    }
    field(ZRVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Zero Value")
    }
    field(ONVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("One Value")
    }
    field(TWVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Two Value")
    }
    field(THVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Three Value")
    }
    field(FRVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Four Value")
    }
    field(FVVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Five Value")
    }
    field(SXVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Six Value")
    }
    field(SVVL, DBF_ULONG) {
        promptgroup("41 - Input 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Seven Value")
    }
    field(EIVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
```

```
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Eight Value")
    }
    field(NIVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Nine Value")
    }
    field(TEVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Ten Value")
    }
    field(ELVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Eleven Value")
    }
    field(TVVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Twelve Value")
    }
    field(TTVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Thirteen Value")
    }
    field(FTVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Fourteen Value")
    }
    field(FFVL, DBF_ULONG) {
        promptgroup("42 - Input 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Fifteen Value")
    }
    field(ZRST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Zero String")
        size(26)
        interest(1)
```

```
        pp(TRUE)
        special(SPC_MOD)
    }
    field(ONST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("One String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TWST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Two String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(THST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Three String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FRST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Four String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FVST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Five String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(SXST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Six String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(SVST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Seven String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(EIST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
```

```
        prop(YES)
        prompt("Eight String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(NIST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Nine String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TEST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Ten String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(ELST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Eleven String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TVST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Twelve String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TTST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Thirteen String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FTST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Fourteen String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FFST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Fifteen String")
        size(26)
        interest(1)
        pp(TRUE)
```

```
12296         special(SPC_MOD)
12297     }
12298     field(ZRSV, DBF_MENU) {
12299         promptgroup("71 - Alarm 0-7")
12300         interest(1)
12301         menu(menuAlarmSevr)
12302         pp(TRUE)
12303         prompt("State Zero Severity")
12304     }
12305     field(ONSV, DBF_MENU) {
12306         promptgroup("71 - Alarm 0-7")
12307         interest(1)
12308         menu(menuAlarmSevr)
12309         pp(TRUE)
12310         prompt("State One Severity")
12311     }
12312     field(TWSV, DBF_MENU) {
12313         promptgroup("71 - Alarm 0-7")
12314         interest(1)
12315         menu(menuAlarmSevr)
12316         pp(TRUE)
12317         prompt("State Two Severity")
12318     }
12319     field(THSV, DBF_MENU) {
12320         promptgroup("71 - Alarm 0-7")
12321         interest(1)
12322         menu(menuAlarmSevr)
12323         pp(TRUE)
12324         prompt("State Three Severity")
12325     }
12326     field(FRSV, DBF_MENU) {
12327         promptgroup("71 - Alarm 0-7")
12328         interest(1)
12329         menu(menuAlarmSevr)
12330         pp(TRUE)
12331         prompt("State Four Severity")
12332     }
12333     field(FVSV, DBF_MENU) {
12334         promptgroup("71 - Alarm 0-7")
12335         interest(1)
12336         menu(menuAlarmSevr)
12337         pp(TRUE)
12338         prompt("State Five Severity")
12339     }
12340     field(SXSV, DBF_MENU) {
12341         promptgroup("71 - Alarm 0-7")
12342         interest(1)
12343         menu(menuAlarmSevr)
12344         pp(TRUE)
12345         prompt("State Six Severity")
12346     }
12347     field(SVSV, DBF_MENU) {
12348         promptgroup("71 - Alarm 0-7")
12349         interest(1)
12350         menu(menuAlarmSevr)
12351         pp(TRUE)
12352         prompt("State Seven Severity")
12353     }
12354     field(EISV, DBF_MENU) {
12355         promptgroup("72 - Alarm 8-15")
12356         interest(1)
12357         menu(menuAlarmSevr)
12358         pp(TRUE)
12359         prompt("State Eight Severity")
12360     }
12361     field(NISV, DBF_MENU) {
12362         promptgroup("72 - Alarm 8-15")
12363         interest(1)
```

```
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Nine Severity")
    }
    field(TESV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Ten Severity")
    }
    field(ELSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Eleven Severity")
    }
    field(TVSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Twelve Severity")
    }
    field(TTSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Thirteen Sevr")
    }
    field(FTSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Fourteen Sevr")
    }
    field(FFSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Fifteen Severity")
    }
    field(AFTC, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Filter Time Constant")
    }
    field(AFVL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Alarm Filter Value")
    }
    field(UNSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Unknown State Severity")
    }
    field(COSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
```

```
12432         prompt("Change of State Svr")
12433     }
12434     field(RVAL, DBF_ULONG) {
12435         pp(TRUE)
12436         prompt("Raw Value")
12437     }
12438     field(ORAW, DBF_ULONG) {
12439         interest(3)
12440         special(SPC_NOMOD)
12441         prompt("Prev Raw Value")
12442     }
12443     field(MASK, DBF_ULONG) {
12444         interest(1)
12445         special(SPC_NOMOD)
12446         prompt("Hardware Mask")
12447     }
12448     field(MLST, DBF_USHORT) {
12449         interest(3)
12450         special(SPC_NOMOD)
12451         prompt("Last Value Monitored")
12452     }
12453     field(LALM, DBF_USHORT) {
12454         interest(3)
12455         special(SPC_NOMOD)
12456         prompt("Last Value Alarmed")
12457     }
12458     field(SDEF, DBF_SHORT) {
12459         interest(3)
12460         special(SPC_NOMOD)
12461         prompt("States Defined")
12462     }
12463     field(SHFT, DBF_USHORT) {
12464         promptgroup("40 - Input")
12465         interest(1)
12466         prompt("Shift")
12467     }
12468     field(SIOL, DBF_INLINK) {
12469         promptgroup("90 - Simulate")
12470         interest(1)
12471         prompt("Simulation Input Link")
12472     }
12473     field(SVAL, DBF_ULONG) {
12474         prompt("Simulation Value")
12475     }
12476     field(SIML, DBF_INLINK) {
12477         promptgroup("90 - Simulate")
12478         interest(1)
12479         prompt("Simulation Mode Link")
12480     }
12481     field(SIMM, DBF_MENU) {
12482         interest(1)
12483         menu(menuSimm)
12484         special(SPC_MOD)
12485         prompt("Simulation Mode")
12486     }
12487     field(SIMS, DBF_MENU) {
12488         promptgroup("90 - Simulate")
12489         interest(2)
12490         menu(menuAlarmSevr)
12491         prompt("Simulation Mode Severity")
12492     }
12493     field(OLDSIMM, DBF_MENU) {
12494         interest(4)
12495         menu(menuSimm)
12496         special(SPC_NOMOD)
12497         prompt("Prev. Simulation Mode")
12498     }
12499     field(SSCN, DBF_MENU) {
```

```
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
device(mbbi, CONSTANT, devMbbiSoft, "Soft Channel")
device(mbbi, CONSTANT, devMbbiSoftRaw, "Raw Soft Channel")
device(mbbi, CONSTANT, devMbbiSoftCallback, "Async Soft Channel")
device(mbbi, INST_IO, devPSCRegMbbi, "PSC Reg")
device(mbbi, INST_IO, devPSCRegRBMbbi, "PSC RB Reg")
recordtype(mbbiDirect) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %/* Declare Device Support Entry Table */
    %struct mbbiDirectRecord;
    %typedef struct mbbidirectdset {
    %    dset common; /* init_record returns: (-1,0) => (failure, success)*/
    %    long (*read_mbbi)(struct mbbiDirectRecord *prec); /* (0, 2) => (success, success no convert)*/
    %} mbbidirectdset;
    %#define HAS_mbbidirectdset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
```

```
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
        promptgroup("20 - Scan")
        initial("1")
        prompt("Disable Value")
    }
    field(DISA, DBF_SHORT) {
        prompt("Disable")
    }
    field(SDIS, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
```

```
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
        prompt("Record active")
    }
    field(PUTF, DBF_UCHAR) {
        interest(1)
        special(SPC_NOMOD)
```

```
12704         prompt("dbPutField process")
12705     }
12706     field(RPRO, DBF_UCHAR) {
12707         interest(1)
12708         special(SPC_NOMOD)
12709         prompt("Reprocess ")
12710     }
12711     field(ASP, DBF_NOACCESS) {
12712         interest(4)
12713         extra("struct asgMember    *asp")
12714         special(SPC_NOMOD)
12715         prompt("Access Security Pvt")
12716     }
12717     field(PPN, DBF_NOACCESS) {
12718         interest(4)
12719         extra("struct processNotify *ppn")
12720         special(SPC_NOMOD)
12721         prompt("pprocessNotify")
12722     }
12723     field(PPNR, DBF_NOACCESS) {
12724         interest(4)
12725         extra("struct processNotifyRecord *ppnr")
12726         special(SPC_NOMOD)
12727         prompt("pprocessNotifyRecord")
12728     }
12729     field(SPVT, DBF_NOACCESS) {
12730         interest(4)
12731         extra("struct scan_element *spvt")
12732         special(SPC_NOMOD)
12733         prompt("Scan Private")
12734     }
12735     field(RSET, DBF_NOACCESS) {
12736         interest(4)
12737         extra("struct typed_rset   *rset")
12738         special(SPC_NOMOD)
12739         prompt("Address of RSET")
12740     }
12741     field(DSET, DBF_NOACCESS) {
12742         interest(4)
12743         extra("unambiguous_dset    *dset")
12744         special(SPC_NOMOD)
12745         prompt("DSET address")
12746     }
12747     field(DPVT, DBF_NOACCESS) {
12748         interest(4)
12749         extra("void                *dpvt")
12750         special(SPC_NOMOD)
12751         prompt("Device Private")
12752     }
12753     field(RDES, DBF_NOACCESS) {
12754         interest(4)
12755         extra("struct dbRecordType *rdes")
12756         special(SPC_NOMOD)
12757         prompt("Address of dbRecordType")
12758     }
12759     field(LSET, DBF_NOACCESS) {
12760         interest(4)
12761         extra("struct lockRecord   *lset")
12762         special(SPC_NOMOD)
12763         prompt("Lock Set")
12764     }
12765     field(PRIO, DBF_MENU) {
12766         promptgroup("20 - Scan")
12767         interest(1)
12768         menu(menuPriority)
12769         special(SPC_SCAN)
12770         prompt("Scheduling Priority")
12771     }
```

```
12772     field(TPRO, DBF_UCHAR) {
12773         prompt("Trace Processing")
12774     }
12775     field(BKPT, DBF_NOACCESS) {
12776         interest(1)
12777         extra("epicsUInt8          bkpt")
12778         special(SPC_NOMOD)
12779         prompt("Break Point")
12780     }
12781     field(UDF, DBF_UCHAR) {
12782         promptgroup("10 - Common")
12783         interest(1)
12784         initial("1")
12785         pp(TRUE)
12786         prompt("Undefined")
12787     }
12788     field(UDFS, DBF_MENU) {
12789         promptgroup("70 - Alarm")
12790         interest(1)
12791         initial("INVALID")
12792         menu(menuAlarmSevr)
12793         prompt("Undefined Alarm Sevrty")
12794     }
12795     field(TIME, DBF_NOACCESS) {
12796         interest(2)
12797         extra("epicsTimeStamp      time")
12798         special(SPC_NOMOD)
12799         prompt("Time")
12800     }
12801     field(UTAG, DBF_UINT64) {
12802         interest(3)
12803         special(SPC_NOMOD)
12804         prompt("Time Tag")
12805     }
12806     field(FLNK, DBF_FWDLINK) {
12807         promptgroup("20 - Scan")
12808         interest(1)
12809         prompt("Forward Process Link")
12810     }
12811     field(VAL, DBF_LONG) {
12812         promptgroup("40 - Input")
12813         asl(ASL0)
12814         pp(TRUE)
12815         prompt("Current Value")
12816     }
12817     field(NOBT, DBF_SHORT) {
12818         promptgroup("40 - Input")
12819         interest(1)
12820         special(SPC_NOMOD)
12821         prop(YES)
12822         prompt("Number of Bits")
12823     }
12824     field(INP, DBF_INLINK) {
12825         promptgroup("40 - Input")
12826         interest(1)
12827         prompt("Input Specification")
12828     }
12829     field(RVAL, DBF_ULONG) {
12830         pp(TRUE)
12831         prompt("Raw Value")
12832     }
12833     field(ORAW, DBF_ULONG) {
12834         interest(3)
12835         special(SPC_NOMOD)
12836         prompt("Prev Raw Value")
12837     }
12838     field(MASK, DBF_ULONG) {
12839         interest(1)
```

```
12840         special(SPC_NOMOD)
12841         prompt("Hardware Mask")
12842     }
12843     field(MLST, DBF_LONG) {
12844         interest(3)
12845         special(SPC_NOMOD)
12846         prompt("Last Value Monitored")
12847     }
12848     field(SHFT, DBF_USHORT) {
12849         promptgroup("40 - Input")
12850         interest(1)
12851         prompt("Shift")
12852     }
12853     field(SIOL, DBF_INLINK) {
12854         promptgroup("90 - Simulate")
12855         interest(1)
12856         prompt("Simulation Input Link")
12857     }
12858     field(SVAL, DBF_LONG) {
12859         prompt("Simulation Value")
12860     }
12861     field(SIML, DBF_INLINK) {
12862         promptgroup("90 - Simulate")
12863         interest(1)
12864         prompt("Simulation Mode Link")
12865     }
12866     field(SIMM, DBF_MENU) {
12867         interest(1)
12868         menu(menuSimm)
12869         special(SPC_MOD)
12870         prompt("Simulation Mode")
12871     }
12872     field(SIMS, DBF_MENU) {
12873         promptgroup("90 - Simulate")
12874         interest(2)
12875         menu(menuAlarmSevr)
12876         prompt("Simulation Mode Severity")
12877     }
12878     field(OLDSIMM, DBF_MENU) {
12879         interest(4)
12880         menu(menuSimm)
12881         special(SPC_NOMOD)
12882         prompt("Prev. Simulation Mode")
12883     }
12884     field(SSCN, DBF_MENU) {
12885         promptgroup("90 - Simulate")
12886         interest(1)
12887         initial("65535")
12888         menu(menuScan)
12889         prompt("Sim. Mode Scan")
12890     }
12891     field(SDLY, DBF_DOUBLE) {
12892         promptgroup("90 - Simulate")
12893         interest(2)
12894         initial("-1.0")
12895         prompt("Sim. Mode Async Delay")
12896     }
12897     field(SIMPVT, DBF_NOACCESS) {
12898         interest(4)
12899         extra("epicsCallback            *simpvt")
12900         special(SPC_NOMOD)
12901         prompt("Sim. Mode Private")
12902     }
12903     field(B0, DBF_UCHAR) {
12904         interest(1)
12905         pp(TRUE)
12906         prompt("Bit 0")
12907     }
```

```
    field(B1, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 1")
    }
    field(B2, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 2")
    }
    field(B3, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 3")
    }
    field(B4, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 4")
    }
    field(B5, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 5")
    }
    field(B6, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 6")
    }
    field(B7, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 7")
    }
    field(B8, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 8")
    }
    field(B9, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 9")
    }
    field(BA, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 10")
    }
    field(BB, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 11")
    }
    field(BC, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 12")
    }
    field(BD, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 13")
    }
    field(BE, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
```

```
12976         prompt("Bit 14")
12977     }
12978     field(BF, DBF_UCHAR) {
12979         interest(1)
12980         pp(TRUE)
12981         prompt("Bit 15")
12982     }
12983     field(B10, DBF_UCHAR) {
12984         interest(1)
12985         pp(TRUE)
12986         prompt("Bit 16")
12987     }
12988     field(B11, DBF_UCHAR) {
12989         interest(1)
12990         pp(TRUE)
12991         prompt("Bit 17")
12992     }
12993     field(B12, DBF_UCHAR) {
12994         interest(1)
12995         pp(TRUE)
12996         prompt("Bit 18")
12997     }
12998     field(B13, DBF_UCHAR) {
12999         interest(1)
13000         pp(TRUE)
13001         prompt("Bit 19")
13002     }
13003     field(B14, DBF_UCHAR) {
13004         interest(1)
13005         pp(TRUE)
13006         prompt("Bit 20")
13007     }
13008     field(B15, DBF_UCHAR) {
13009         interest(1)
13010         pp(TRUE)
13011         prompt("Bit 21")
13012     }
13013     field(B16, DBF_UCHAR) {
13014         interest(1)
13015         pp(TRUE)
13016         prompt("Bit 22")
13017     }
13018     field(B17, DBF_UCHAR) {
13019         interest(1)
13020         pp(TRUE)
13021         prompt("Bit 23")
13022     }
13023     field(B18, DBF_UCHAR) {
13024         interest(1)
13025         pp(TRUE)
13026         prompt("Bit 24")
13027     }
13028     field(B19, DBF_UCHAR) {
13029         interest(1)
13030         pp(TRUE)
13031         prompt("Bit 25")
13032     }
13033     field(B1A, DBF_UCHAR) {
13034         interest(1)
13035         pp(TRUE)
13036         prompt("Bit 26")
13037     }
13038     field(B1B, DBF_UCHAR) {
13039         interest(1)
13040         pp(TRUE)
13041         prompt("Bit 27")
13042     }
13043     field(B1C, DBF_UCHAR) {
```

```
        interest(1)
        pp(TRUE)
        prompt("Bit 28")
    }
    field(B1D, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 29")
    }
    field(B1E, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 30")
    }
    field(B1F, DBF_UCHAR) {
        interest(1)
        pp(TRUE)
        prompt("Bit 31")
    }
}
device(mbbiDirect, CONSTANT, devMbbiDirectSoft, "Soft Channel")
device(mbbiDirect, CONSTANT, devMbbiDirectSoftRaw, "Raw Soft Channel")
device(mbbiDirect, CONSTANT, devMbbiDirectSoftCallback, "Async Soft Channel")
device(mbbiDirect, INST_IO, devPSCRegMbbiDirect, "PSC Reg")
device(mbbiDirect, INST_IO, devPSCRegRBMbbiDirect, "PSC RB Reg")
recordtype(mbbo) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %/* Declare Device Support Entry Table */
    %struct mbboRecord;
    %typedef struct mbbodset {
    %    dset common; /*init_record returns: (0, 2) => (success, success no convert)*/
    %    long (*write_mbbo)(struct mbboRecord *prec); /*returns: (0, 2) => (success, success no convert)*/
    %} mbbodset;
    %#define HAS_mbbodset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
```

```
13112         prompt("Process at iocInit")
13113     }
13114     field(PHAS, DBF_SHORT) {
13115         promptgroup("20 - Scan")
13116         interest(1)
13117         special(SPC_SCAN)
13118         prompt("Scan Phase")
13119     }
13120     field(EVNT, DBF_STRING) {
13121         size(40)
13122         promptgroup("20 - Scan")
13123         interest(1)
13124         special(SPC_SCAN)
13125         prompt("Event Name")
13126     }
13127     field(TSE, DBF_SHORT) {
13128         promptgroup("20 - Scan")
13129         interest(1)
13130         prompt("Time Stamp Event")
13131     }
13132     field(TSEL, DBF_INLINK) {
13133         promptgroup("20 - Scan")
13134         interest(1)
13135         prompt("Time Stamp Link")
13136     }
13137     field(DTYP, DBF_DEVICE) {
13138         promptgroup("10 - Common")
13139         interest(1)
13140         prompt("Device Type")
13141     }
13142     field(DISV, DBF_SHORT) {
13143         promptgroup("20 - Scan")
13144         initial("1")
13145         prompt("Disable Value")
13146     }
13147     field(DISA, DBF_SHORT) {
13148         prompt("Disable")
13149     }
13150     field(SDIS, DBF_INLINK) {
13151         promptgroup("20 - Scan")
13152         interest(1)
13153         prompt("Scanning Disable")
13154     }
13155     field(MLOK, DBF_NOACCESS) {
13156         interest(4)
13157         extra("epicsMutexId        mlok")
13158         special(SPC_NOMOD)
13159         prompt("Monitor lock")
13160     }
13161     field(MLIS, DBF_NOACCESS) {
13162         interest(4)
13163         extra("ELLLIST             mlis")
13164         special(SPC_NOMOD)
13165         prompt("Monitor List")
13166     }
13167     field(BKLNK, DBF_NOACCESS) {
13168         interest(4)
13169         extra("ELLLIST             bklnk")
13170         special(SPC_NOMOD)
13171         prompt("Backwards link tracking")
13172     }
13173     field(DISP, DBF_UCHAR) {
13174         promptgroup("10 - Common")
13175         interest(1)
13176         prompt("Disable putField")
13177     }
13178     field(PROC, DBF_UCHAR) {
13179         interest(3)
```

```
13180         pp(TRUE)
13181         prompt("Force Processing")
13182     }
13183     field(STAT, DBF_MENU) {
13184         initial("UDF")
13185         menu(menuAlarmStat)
13186         special(SPC_NOMOD)
13187         prompt("Alarm Status")
13188     }
13189     field(SEVR, DBF_MENU) {
13190         menu(menuAlarmSevr)
13191         special(SPC_NOMOD)
13192         prompt("Alarm Severity")
13193     }
13194     field(AMSG, DBF_STRING) {
13195         size(40)
13196         special(SPC_NOMOD)
13197         prompt("Alarm Message")
13198     }
13199     field(NSTA, DBF_MENU) {
13200         interest(2)
13201         menu(menuAlarmStat)
13202         special(SPC_NOMOD)
13203         prompt("New Alarm Status")
13204     }
13205     field(NSEV, DBF_MENU) {
13206         interest(2)
13207         menu(menuAlarmSevr)
13208         special(SPC_NOMOD)
13209         prompt("New Alarm Severity")
13210     }
13211     field(NAMSG, DBF_STRING) {
13212         size(40)
13213         special(SPC_NOMOD)
13214         prompt("New Alarm Message")
13215     }
13216     field(ACKS, DBF_MENU) {
13217         interest(2)
13218         menu(menuAlarmSevr)
13219         special(SPC_NOMOD)
13220         prompt("Alarm Ack Severity")
13221     }
13222     field(ACKT, DBF_MENU) {
13223         promptgroup("70 - Alarm")
13224         interest(2)
13225         initial("YES")
13226         menu(menuYesNo)
13227         special(SPC_NOMOD)
13228         prompt("Alarm Ack Transient")
13229     }
13230     field(DISS, DBF_MENU) {
13231         promptgroup("70 - Alarm")
13232         interest(1)
13233         menu(menuAlarmSevr)
13234         prompt("Disable Alarm Sevrty")
13235     }
13236     field(LCNT, DBF_UCHAR) {
13237         interest(2)
13238         special(SPC_NOMOD)
13239         prompt("Lock Count")
13240     }
13241     field(PACT, DBF_UCHAR) {
13242         interest(1)
13243         special(SPC_NOMOD)
13244         prompt("Record active")
13245     }
13246     field(PUTF, DBF_UCHAR) {
13247         interest(1)
```

```
13248         special(SPC_NOMOD)
13249         prompt("dbPutField process")
13250     }
13251     field(RPRO, DBF_UCHAR) {
13252         interest(1)
13253         special(SPC_NOMOD)
13254         prompt("Reprocess ")
13255     }
13256     field(ASP, DBF_NOACCESS) {
13257         interest(4)
13258         extra("struct asgMember    *asp")
13259         special(SPC_NOMOD)
13260         prompt("Access Security Pvt")
13261     }
13262     field(PPN, DBF_NOACCESS) {
13263         interest(4)
13264         extra("struct processNotify *ppn")
13265         special(SPC_NOMOD)
13266         prompt("pprocessNotify")
13267     }
13268     field(PPNR, DBF_NOACCESS) {
13269         interest(4)
13270         extra("struct processNotifyRecord *ppnr")
13271         special(SPC_NOMOD)
13272         prompt("pprocessNotifyRecord")
13273     }
13274     field(SPVT, DBF_NOACCESS) {
13275         interest(4)
13276         extra("struct scan_element *spvt")
13277         special(SPC_NOMOD)
13278         prompt("Scan Private")
13279     }
13280     field(RSET, DBF_NOACCESS) {
13281         interest(4)
13282         extra("struct typed_rset   *rset")
13283         special(SPC_NOMOD)
13284         prompt("Address of RSET")
13285     }
13286     field(DSET, DBF_NOACCESS) {
13287         interest(4)
13288         extra("unambiguous_dset    *dset")
13289         special(SPC_NOMOD)
13290         prompt("DSET address")
13291     }
13292     field(DPVT, DBF_NOACCESS) {
13293         interest(4)
13294         extra("void                *dpvt")
13295         special(SPC_NOMOD)
13296         prompt("Device Private")
13297     }
13298     field(RDES, DBF_NOACCESS) {
13299         interest(4)
13300         extra("struct dbRecordType *rdes")
13301         special(SPC_NOMOD)
13302         prompt("Address of dbRecordType")
13303     }
13304     field(LSET, DBF_NOACCESS) {
13305         interest(4)
13306         extra("struct lockRecord   *lset")
13307         special(SPC_NOMOD)
13308         prompt("Lock Set")
13309     }
13310     field(PRIO, DBF_MENU) {
13311         promptgroup("20 - Scan")
13312         interest(1)
13313         menu(menuPriority)
13314         special(SPC_SCAN)
13315         prompt("Scheduling Priority")
```

```
13316     }
13317     field(TPRO, DBF_UCHAR) {
13318         prompt("Trace Processing")
13319     }
13320     field(BKPT, DBF_NOACCESS) {
13321         interest(1)
13322         extra("epicsUInt8          bkpt")
13323         special(SPC_NOMOD)
13324         prompt("Break Point")
13325     }
13326     field(UDF, DBF_UCHAR) {
13327         promptgroup("10 - Common")
13328         interest(1)
13329         initial("1")
13330         pp(TRUE)
13331         prompt("Undefined")
13332     }
13333     field(UDFS, DBF_MENU) {
13334         promptgroup("70 - Alarm")
13335         interest(1)
13336         initial("INVALID")
13337         menu(menuAlarmSevr)
13338         prompt("Undefined Alarm Sevrty")
13339     }
13340     field(TIME, DBF_NOACCESS) {
13341         interest(2)
13342         extra("epicsTimeStamp      time")
13343         special(SPC_NOMOD)
13344         prompt("Time")
13345     }
13346     field(UTAG, DBF_UINT64) {
13347         interest(3)
13348         special(SPC_NOMOD)
13349         prompt("Time Tag")
13350     }
13351     field(FLNK, DBF_FWDLINK) {
13352         promptgroup("20 - Scan")
13353         interest(1)
13354         prompt("Forward Process Link")
13355     }
13356     field(VAL, DBF_ENUM) {
13357         promptgroup("50 - Output")
13358         asl(ASL0)
13359         pp(TRUE)
13360         special(SPC_DBADDR)
13361         prompt("Desired Value")
13362     }
13363     field(DOL, DBF_INLINK) {
13364         promptgroup("40 - Input")
13365         interest(1)
13366         prompt("Desired Output Link")
13367     }
13368     field(OMSL, DBF_MENU) {
13369         promptgroup("50 - Output")
13370         interest(1)
13371         menu(menuOmsl)
13372         prompt("Output Mode Select")
13373     }
13374     field(NOBT, DBF_USHORT) {
13375         promptgroup("50 - Output")
13376         interest(1)
13377         special(SPC_NOMOD)
13378         prompt("Number of Bits")
13379     }
13380     field(OUT, DBF_OUTLINK) {
13381         promptgroup("50 - Output")
13382         interest(1)
13383         prompt("Output Specification")
```

```
    }
    field(ZRVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Zero Value")
    }
    field(ONVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("One Value")
    }
    field(TWVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Two Value")
    }
    field(THVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Three Value")
    }
    field(FRVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Four Value")
    }
    field(FVVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Five Value")
    }
    field(SXVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Six Value")
    }
    field(SVVL, DBF_ULONG) {
        promptgroup("51 - Output 0-7")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Seven Value")
    }
    field(EIVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
```

```
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Eight Value")
    }
    field(NIVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Nine Value")
    }
    field(TEVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Ten Value")
    }
    field(ELVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Eleven Value")
    }
    field(TVVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Twelve Value")
    }
    field(TTVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Thirteen Value")
    }
    field(FTVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Fourteen Value")
    }
    field(FFVL, DBF_ULONG) {
        promptgroup("52 - Output 8-15")
        interest(1)
        base(HEX)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Fifteen Value")
    }
    field(ZRST, DBF_STRING) {
        promptgroup("81 - Display 0-7")
        prop(YES)
        prompt("Zero String")
        size(26)
        interest(1)
        pp(TRUE)
```

```
13520         special(SPC_MOD)
13521     }
13522     field(ONST, DBF_STRING) {
13523         promptgroup("81 - Display 0-7")
13524         prop(YES)
13525         prompt("One String")
13526         size(26)
13527         interest(1)
13528         pp(TRUE)
13529         special(SPC_MOD)
13530     }
13531     field(TWST, DBF_STRING) {
13532         promptgroup("81 - Display 0-7")
13533         prop(YES)
13534         prompt("Two String")
13535         size(26)
13536         interest(1)
13537         pp(TRUE)
13538         special(SPC_MOD)
13539     }
13540     field(THST, DBF_STRING) {
13541         promptgroup("81 - Display 0-7")
13542         prop(YES)
13543         prompt("Three String")
13544         size(26)
13545         interest(1)
13546         pp(TRUE)
13547         special(SPC_MOD)
13548     }
13549     field(FRST, DBF_STRING) {
13550         promptgroup("81 - Display 0-7")
13551         prop(YES)
13552         prompt("Four String")
13553         size(26)
13554         interest(1)
13555         pp(TRUE)
13556         special(SPC_MOD)
13557     }
13558     field(FVST, DBF_STRING) {
13559         promptgroup("81 - Display 0-7")
13560         prop(YES)
13561         prompt("Five String")
13562         size(26)
13563         interest(1)
13564         pp(TRUE)
13565         special(SPC_MOD)
13566     }
13567     field(SXST, DBF_STRING) {
13568         promptgroup("81 - Display 0-7")
13569         prop(YES)
13570         prompt("Six String")
13571         size(26)
13572         interest(1)
13573         pp(TRUE)
13574         special(SPC_MOD)
13575     }
13576     field(SVST, DBF_STRING) {
13577         promptgroup("81 - Display 0-7")
13578         prop(YES)
13579         prompt("Seven String")
13580         size(26)
13581         interest(1)
13582         pp(TRUE)
13583         special(SPC_MOD)
13584     }
13585     field(EIST, DBF_STRING) {
13586         promptgroup("82 - Display 8-15")
13587         prop(YES)
```

```
        prompt("Eight String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(NIST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Nine String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TEST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Ten String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(ELST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Eleven String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TVST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Twelve String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(TTST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Thirteen String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FTST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Fourteen String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
    }
    field(FFST, DBF_STRING) {
        promptgroup("82 - Display 8-15")
        prop(YES)
        prompt("Fifteen String")
        size(26)
        interest(1)
        pp(TRUE)
        special(SPC_MOD)
```

```
13656     }
13657     field(ZRSV, DBF_MENU) {
13658         promptgroup("71 - Alarm 0-7")
13659         interest(1)
13660         menu(menuAlarmSevr)
13661         pp(TRUE)
13662         prompt("State Zero Severity")
13663     }
13664     field(ONSV, DBF_MENU) {
13665         promptgroup("71 - Alarm 0-7")
13666         interest(1)
13667         menu(menuAlarmSevr)
13668         pp(TRUE)
13669         prompt("State One Severity")
13670     }
13671     field(TWSV, DBF_MENU) {
13672         promptgroup("71 - Alarm 0-7")
13673         interest(1)
13674         menu(menuAlarmSevr)
13675         pp(TRUE)
13676         prompt("State Two Severity")
13677     }
13678     field(THSV, DBF_MENU) {
13679         promptgroup("71 - Alarm 0-7")
13680         interest(1)
13681         menu(menuAlarmSevr)
13682         pp(TRUE)
13683         prompt("State Three Severity")
13684     }
13685     field(FRSV, DBF_MENU) {
13686         promptgroup("71 - Alarm 0-7")
13687         interest(1)
13688         menu(menuAlarmSevr)
13689         pp(TRUE)
13690         prompt("State Four Severity")
13691     }
13692     field(FVSV, DBF_MENU) {
13693         promptgroup("71 - Alarm 0-7")
13694         interest(1)
13695         menu(menuAlarmSevr)
13696         pp(TRUE)
13697         prompt("State Five Severity")
13698     }
13699     field(SXSV, DBF_MENU) {
13700         promptgroup("71 - Alarm 0-7")
13701         interest(1)
13702         menu(menuAlarmSevr)
13703         pp(TRUE)
13704         prompt("State Six Severity")
13705     }
13706     field(SVSV, DBF_MENU) {
13707         promptgroup("71 - Alarm 0-7")
13708         interest(1)
13709         menu(menuAlarmSevr)
13710         pp(TRUE)
13711         prompt("State Seven Severity")
13712     }
13713     field(EISV, DBF_MENU) {
13714         promptgroup("72 - Alarm 8-15")
13715         interest(1)
13716         menu(menuAlarmSevr)
13717         pp(TRUE)
13718         prompt("State Eight Severity")
13719     }
13720     field(NISV, DBF_MENU) {
13721         promptgroup("72 - Alarm 8-15")
13722         interest(1)
13723         menu(menuAlarmSevr)
```

```
        pp(TRUE)
        prompt("State Nine Severity")
    }
    field(TESV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Ten Severity")
    }
    field(ELSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Eleven Severity")
    }
    field(TVSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Twelve Severity")
    }
    field(TTSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Thirteen Sevr")
    }
    field(FTSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Fourteen Sevr")
    }
    field(FFSV, DBF_MENU) {
        promptgroup("72 - Alarm 8-15")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("State Fifteen Sevr")
    }
    field(UNSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Unknown State Sevr")
    }
    field(COSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Change of State Sevr")
    }
    field(RVAL, DBF_ULONG) {
        pp(TRUE)
        prompt("Raw Value")
    }
    field(ORAW, DBF_ULONG) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Raw Value")
    }
```

```
13792     field(RBV, DBF_ULONG) {
13793         special(SPC_NOMOD)
13794         prompt("Readback Value")
13795     }
13796     field(ORBV, DBF_ULONG) {
13797         interest(3)
13798         special(SPC_NOMOD)
13799         prompt("Prev Readback Value")
13800     }
13801     field(MASK, DBF_ULONG) {
13802         interest(1)
13803         special(SPC_NOMOD)
13804         prompt("Hardware Mask")
13805     }
13806     field(MLST, DBF_USHORT) {
13807         interest(3)
13808         special(SPC_NOMOD)
13809         prompt("Last Value Monitored")
13810     }
13811     field(LALM, DBF_USHORT) {
13812         interest(3)
13813         special(SPC_NOMOD)
13814         prompt("Last Value Alarmed")
13815     }
13816     field(SDEF, DBF_SHORT) {
13817         interest(3)
13818         special(SPC_NOMOD)
13819         prompt("States Defined")
13820     }
13821     field(SHFT, DBF_USHORT) {
13822         promptgroup("50 - Output")
13823         interest(1)
13824         prompt("Shift")
13825     }
13826     field(SIOL, DBF_OUTLINK) {
13827         promptgroup("90 - Simulate")
13828         interest(1)
13829         prompt("Simulation Output Link")
13830     }
13831     field(SIML, DBF_INLINK) {
13832         promptgroup("90 - Simulate")
13833         interest(1)
13834         prompt("Simulation Mode Link")
13835     }
13836     field(SIMM, DBF_MENU) {
13837         interest(1)
13838         menu(menuSimm)
13839         special(SPC_MOD)
13840         prompt("Simulation Mode")
13841     }
13842     field(SIMS, DBF_MENU) {
13843         promptgroup("90 - Simulate")
13844         interest(2)
13845         menu(menuAlarmSevr)
13846         prompt("Simulation Mode Severity")
13847     }
13848     field(OLDSIMM, DBF_MENU) {
13849         interest(4)
13850         menu(menuSimm)
13851         special(SPC_NOMOD)
13852         prompt("Prev. Simulation Mode")
13853     }
13854     field(SSCN, DBF_MENU) {
13855         promptgroup("90 - Simulate")
13856         interest(1)
13857         initial("65535")
13858         menu(menuScan)
13859         prompt("Sim. Mode Scan")
```

```
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID outpt action")
    }
    field(IVOV, DBF_USHORT) {
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
}
device(mbbo, CONSTANT, devMbboSoft, "Soft Channel")
device(mbbo, CONSTANT, devMbboSoftRaw, "Raw Soft Channel")
device(mbbo, CONSTANT, devMbboSoftCallback, "Async Soft Channel")
device(mbbo, INST_IO, devPSCRegMbbo, "PSC Reg")
device(mbbo, INST_IO, devPSCSingleU32Mbbo, "PSC Single U32")
recordtype(mbboDirect) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %/* Declare Device Support Entry Table */
    %struct mbboDirectRecord;
    %typedef struct mbbodirectdset {
    %    dset common; /*init_record returns: (0, 2)=>(success, success no convert)*/
    %    long (*write_mbbo)(struct mbboDirectRecord *prec); /*returns: (0, 2)=>(success, success no convert)*/
    %} mbbodirectdset;
    %#define HAS_mbbodirectdset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
```

```
13928     }
13929     field(PINI, DBF_MENU) {
13930         promptgroup("20 - Scan")
13931         interest(1)
13932         menu(menuPini)
13933         prompt("Process at iocInit")
13934     }
13935     field(PHAS, DBF_SHORT) {
13936         promptgroup("20 - Scan")
13937         interest(1)
13938         special(SPC_SCAN)
13939         prompt("Scan Phase")
13940     }
13941     field(EVNT, DBF_STRING) {
13942         size(40)
13943         promptgroup("20 - Scan")
13944         interest(1)
13945         special(SPC_SCAN)
13946         prompt("Event Name")
13947     }
13948     field(TSE, DBF_SHORT) {
13949         promptgroup("20 - Scan")
13950         interest(1)
13951         prompt("Time Stamp Event")
13952     }
13953     field(TSEL, DBF_INLINK) {
13954         promptgroup("20 - Scan")
13955         interest(1)
13956         prompt("Time Stamp Link")
13957     }
13958     field(DTYP, DBF_DEVICE) {
13959         promptgroup("10 - Common")
13960         interest(1)
13961         prompt("Device Type")
13962     }
13963     field(DISV, DBF_SHORT) {
13964         promptgroup("20 - Scan")
13965         initial("1")
13966         prompt("Disable Value")
13967     }
13968     field(DISA, DBF_SHORT) {
13969         prompt("Disable")
13970     }
13971     field(SDIS, DBF_INLINK) {
13972         promptgroup("20 - Scan")
13973         interest(1)
13974         prompt("Scanning Disable")
13975     }
13976     field(MLOK, DBF_NOACCESS) {
13977         interest(4)
13978         extra("epicsMutexId        mlok")
13979         special(SPC_NOMOD)
13980         prompt("Monitor lock")
13981     }
13982     field(MLIS, DBF_NOACCESS) {
13983         interest(4)
13984         extra("ELLLIST             mlis")
13985         special(SPC_NOMOD)
13986         prompt("Monitor List")
13987     }
13988     field(BKLNK, DBF_NOACCESS) {
13989         interest(4)
13990         extra("ELLLIST             bklnk")
13991         special(SPC_NOMOD)
13992         prompt("Backwards link tracking")
13993     }
13994     field(DISP, DBF_UCHAR) {
13995         promptgroup("10 - Common")
```

```
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Ack Severity")
    }
    field(ACKT, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(2)
        initial("YES")
        menu(menuYesNo)
        special(SPC_NOMOD)
        prompt("Alarm Ack Transient")
    }
    field(DISS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        prompt("Disable Alarm Sevrty")
    }
    field(LCNT, DBF_UCHAR) {
        interest(2)
        special(SPC_NOMOD)
        prompt("Lock Count")
    }
    field(PACT, DBF_UCHAR) {
        interest(1)
```

```
14064         special(SPC_NOMOD)
14065         prompt("Record active")
14066     }
14067     field(PUTF, DBF_UCHAR) {
14068         interest(1)
14069         special(SPC_NOMOD)
14070         prompt("dbPutField process")
14071     }
14072     field(RPRO, DBF_UCHAR) {
14073         interest(1)
14074         special(SPC_NOMOD)
14075         prompt("Reprocess ")
14076     }
14077     field(ASP, DBF_NOACCESS) {
14078         interest(4)
14079         extra("struct asgMember    *asp")
14080         special(SPC_NOMOD)
14081         prompt("Access Security Pvt")
14082     }
14083     field(PPN, DBF_NOACCESS) {
14084         interest(4)
14085         extra("struct processNotify *ppn")
14086         special(SPC_NOMOD)
14087         prompt("pprocessNotify")
14088     }
14089     field(PPNR, DBF_NOACCESS) {
14090         interest(4)
14091         extra("struct processNotifyRecord *ppnr")
14092         special(SPC_NOMOD)
14093         prompt("pprocessNotifyRecord")
14094     }
14095     field(SPVT, DBF_NOACCESS) {
14096         interest(4)
14097         extra("struct scan_element *spvt")
14098         special(SPC_NOMOD)
14099         prompt("Scan Private")
14100     }
14101     field(RSET, DBF_NOACCESS) {
14102         interest(4)
14103         extra("struct typed_rset   *rset")
14104         special(SPC_NOMOD)
14105         prompt("Address of RSET")
14106     }
14107     field(DSET, DBF_NOACCESS) {
14108         interest(4)
14109         extra("unambiguous_dset    *dset")
14110         special(SPC_NOMOD)
14111         prompt("DSET address")
14112     }
14113     field(DPVT, DBF_NOACCESS) {
14114         interest(4)
14115         extra("void                *dpvt")
14116         special(SPC_NOMOD)
14117         prompt("Device Private")
14118     }
14119     field(RDES, DBF_NOACCESS) {
14120         interest(4)
14121         extra("struct dbRecordType *rdes")
14122         special(SPC_NOMOD)
14123         prompt("Address of dbRecordType")
14124     }
14125     field(LSET, DBF_NOACCESS) {
14126         interest(4)
14127         extra("struct lockRecord   *lset")
14128         special(SPC_NOMOD)
14129         prompt("Lock Set")
14130     }
14131     field(PRIO, DBF_MENU) {
```

```
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_LONG) {
        promptgroup("50 - Output")
        asl(ASL0)
        pp(TRUE)
        prompt("Word")
    }
    field(OMSL, DBF_MENU) {
        promptgroup("50 - Output")
        interest(1)
        menu(menuOmsl)
        pp(TRUE)
        prompt("Output Mode Select")
    }
    field(NOBT, DBF_SHORT) {
        promptgroup("50 - Output")
        interest(1)
        special(SPC_NOMOD)
        prop(YES)
        prompt("Number of Bits")
    }
    field(DOL, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
```

```
14200         prompt("Desired Output Link")
14201     }
14202     field(OUT, DBF_OUTLINK) {
14203         promptgroup("50 - Output")
14204         interest(1)
14205         prompt("Output Specification")
14206     }
14207     field(RVAL, DBF_ULONG) {
14208         pp(TRUE)
14209         special(SPC_NOMOD)
14210         prompt("Raw Value")
14211     }
14212     field(ORAW, DBF_ULONG) {
14213         interest(3)
14214         special(SPC_NOMOD)
14215         prompt("Prev Raw Value")
14216     }
14217     field(RBV, DBF_ULONG) {
14218         special(SPC_NOMOD)
14219         prompt("Readback Value")
14220     }
14221     field(ORBV, DBF_ULONG) {
14222         interest(3)
14223         special(SPC_NOMOD)
14224         prompt("Prev Readback Value")
14225     }
14226     field(MASK, DBF_ULONG) {
14227         interest(1)
14228         special(SPC_NOMOD)
14229         prompt("Hardware Mask")
14230     }
14231     field(MLST, DBF_LONG) {
14232         interest(3)
14233         special(SPC_NOMOD)
14234         prompt("Last Value Monitored")
14235     }
14236     field(OBIT, DBF_LONG) {
14237         interest(3)
14238         special(SPC_NOMOD)
14239         prompt("Last Bit mask Monitored")
14240     }
14241     field(SHFT, DBF_USHORT) {
14242         promptgroup("50 - Output")
14243         interest(1)
14244         prompt("Shift")
14245     }
14246     field(SIOL, DBF_OUTLINK) {
14247         promptgroup("90 - Simulate")
14248         interest(1)
14249         prompt("Simulation Output Link")
14250     }
14251     field(SIML, DBF_INLINK) {
14252         promptgroup("90 - Simulate")
14253         interest(1)
14254         prompt("Simulation Mode Link")
14255     }
14256     field(SIMM, DBF_MENU) {
14257         interest(1)
14258         menu(menuSimm)
14259         special(SPC_MOD)
14260         prompt("Simulation Mode")
14261     }
14262     field(SIMS, DBF_MENU) {
14263         promptgroup("90 - Simulate")
14264         interest(2)
14265         menu(menuAlarmSevr)
14266         prompt("Simulation Mode Severity")
14267     }
```

```
14268     field(OLDSIMM, DBF_MENU) {
14269         interest(4)
14270         menu(menuSimm)
14271         special(SPC_NOMOD)
14272         prompt("Prev. Simulation Mode")
14273     }
14274     field(SSCN, DBF_MENU) {
14275         promptgroup("90 - Simulate")
14276         interest(1)
14277         initial("65535")
14278         menu(menuScan)
14279         prompt("Sim. Mode Scan")
14280     }
14281     field(SDLY, DBF_DOUBLE) {
14282         promptgroup("90 - Simulate")
14283         interest(2)
14284         initial("-1.0")
14285         prompt("Sim. Mode Async Delay")
14286     }
14287     field(SIMPVT, DBF_NOACCESS) {
14288         interest(4)
14289         extra("epicsCallback            *simpvt")
14290         special(SPC_NOMOD)
14291         prompt("Sim. Mode Private")
14292     }
14293     field(IVOA, DBF_MENU) {
14294         promptgroup("50 - Output")
14295         interest(2)
14296         menu(menuIvoa)
14297         prompt("INVALID outpt action")
14298     }
14299     field(IVOV, DBF_LONG) {
14300         promptgroup("50 - Output")
14301         interest(2)
14302         prompt("INVALID output value")
14303     }
14304     field(B0, DBF_UCHAR) {
14305         promptgroup("51 - Output 0-7")
14306         interest(1)
14307         asl(ASL0)
14308         pp(TRUE)
14309         special(SPC_MOD)
14310         prompt("Bit 0")
14311     }
14312     field(B1, DBF_UCHAR) {
14313         promptgroup("51 - Output 0-7")
14314         interest(1)
14315         asl(ASL0)
14316         pp(TRUE)
14317         special(SPC_MOD)
14318         prompt("Bit 1")
14319     }
14320     field(B2, DBF_UCHAR) {
14321         promptgroup("51 - Output 0-7")
14322         interest(1)
14323         asl(ASL0)
14324         pp(TRUE)
14325         special(SPC_MOD)
14326         prompt("Bit 2")
14327     }
14328     field(B3, DBF_UCHAR) {
14329         promptgroup("51 - Output 0-7")
14330         interest(1)
14331         asl(ASL0)
14332         pp(TRUE)
14333         special(SPC_MOD)
14334         prompt("Bit 3")
14335     }
```

```
14336     field(B4, DBF_UCHAR) {
14337         promptgroup("51 - Output 0-7")
14338         interest(1)
14339         asl(ASL0)
14340         pp(TRUE)
14341         special(SPC_MOD)
14342         prompt("Bit 4")
14343     }
14344     field(B5, DBF_UCHAR) {
14345         promptgroup("51 - Output 0-7")
14346         interest(1)
14347         asl(ASL0)
14348         pp(TRUE)
14349         special(SPC_MOD)
14350         prompt("Bit 5")
14351     }
14352     field(B6, DBF_UCHAR) {
14353         promptgroup("51 - Output 0-7")
14354         interest(1)
14355         asl(ASL0)
14356         pp(TRUE)
14357         special(SPC_MOD)
14358         prompt("Bit 6")
14359     }
14360     field(B7, DBF_UCHAR) {
14361         promptgroup("51 - Output 0-7")
14362         interest(1)
14363         asl(ASL0)
14364         pp(TRUE)
14365         special(SPC_MOD)
14366         prompt("Bit 7")
14367     }
14368     field(B8, DBF_UCHAR) {
14369         promptgroup("52 - Output 8-15")
14370         interest(1)
14371         asl(ASL0)
14372         pp(TRUE)
14373         special(SPC_MOD)
14374         prompt("Bit 8")
14375     }
14376     field(B9, DBF_UCHAR) {
14377         promptgroup("52 - Output 8-15")
14378         interest(1)
14379         asl(ASL0)
14380         pp(TRUE)
14381         special(SPC_MOD)
14382         prompt("Bit 9")
14383     }
14384     field(BA, DBF_UCHAR) {
14385         promptgroup("52 - Output 8-15")
14386         interest(1)
14387         asl(ASL0)
14388         pp(TRUE)
14389         special(SPC_MOD)
14390         prompt("Bit 10")
14391     }
14392     field(BB, DBF_UCHAR) {
14393         promptgroup("52 - Output 8-15")
14394         interest(1)
14395         asl(ASL0)
14396         pp(TRUE)
14397         special(SPC_MOD)
14398         prompt("Bit 11")
14399     }
14400     field(BC, DBF_UCHAR) {
14401         promptgroup("52 - Output 8-15")
14402         interest(1)
14403         asl(ASL0)
```

```
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 12")
    }
    field(BD, DBF_UCHAR) {
        promptgroup("52 - Output 8-15")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 13")
    }
    field(BE, DBF_UCHAR) {
        promptgroup("52 - Output 8-15")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 14")
    }
    field(BF, DBF_UCHAR) {
        promptgroup("52 - Output 8-15")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 15")
    }
    field(B10, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 16")
    }
    field(B11, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 17")
    }
    field(B12, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 18")
    }
    field(B13, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 19")
    }
    field(B14, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 20")
    }
```

```
    field(B15, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 21")
    }
    field(B16, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 22")
    }
    field(B17, DBF_UCHAR) {
        promptgroup("53 - Output 16-23")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 23")
    }
    field(B18, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 24")
    }
    field(B19, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 25")
    }
    field(B1A, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 26")
    }
    field(B1B, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 27")
    }
    field(B1C, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 28")
    }
    field(B1D, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
```

```
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 29")
    }
    field(B1E, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 30")
    }
    field(B1F, DBF_UCHAR) {
        promptgroup("54 - Output 24-31")
        interest(1)
        asl(ASL0)
        pp(TRUE)
        special(SPC_MOD)
        prompt("Bit 31")
    }
}
device(mbboDirect, CONSTANT, devMbboDirectSoft, "Soft Channel")
device(mbboDirect, CONSTANT, devMbboDirectSoftRaw, "Raw Soft Channel")
device(mbboDirect, CONSTANT, devMbboDirectSoftCallback, "Async Soft Channel")
device(mbboDirect, INST_IO, devPSCRegMbboDirect, "PSC Reg")
device(mbboDirect, INST_IO, devPSCSingleU32MbboDirect, "PSC Single U32")
recordtype(permissive) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
```

```
14608     field(EVNT, DBF_STRING) {
14609         size(40)
14610         promptgroup("20 - Scan")
14611         interest(1)
14612         special(SPC_SCAN)
14613         prompt("Event Name")
14614     }
14615     field(TSE, DBF_SHORT) {
14616         promptgroup("20 - Scan")
14617         interest(1)
14618         prompt("Time Stamp Event")
14619     }
14620     field(TSEL, DBF_INLINK) {
14621         promptgroup("20 - Scan")
14622         interest(1)
14623         prompt("Time Stamp Link")
14624     }
14625     field(DTYP, DBF_DEVICE) {
14626         promptgroup("10 - Common")
14627         interest(1)
14628         prompt("Device Type")
14629     }
14630     field(DISV, DBF_SHORT) {
14631         promptgroup("20 - Scan")
14632         initial("1")
14633         prompt("Disable Value")
14634     }
14635     field(DISA, DBF_SHORT) {
14636         prompt("Disable")
14637     }
14638     field(SDIS, DBF_INLINK) {
14639         promptgroup("20 - Scan")
14640         interest(1)
14641         prompt("Scanning Disable")
14642     }
14643     field(MLOK, DBF_NOACCESS) {
14644         interest(4)
14645         extra("epicsMutexId        mlok")
14646         special(SPC_NOMOD)
14647         prompt("Monitor lock")
14648     }
14649     field(MLIS, DBF_NOACCESS) {
14650         interest(4)
14651         extra("ELLLIST             mlis")
14652         special(SPC_NOMOD)
14653         prompt("Monitor List")
14654     }
14655     field(BKLNK, DBF_NOACCESS) {
14656         interest(4)
14657         extra("ELLLIST             bklnk")
14658         special(SPC_NOMOD)
14659         prompt("Backwards link tracking")
14660     }
14661     field(DISP, DBF_UCHAR) {
14662         promptgroup("10 - Common")
14663         interest(1)
14664         prompt("Disable putField")
14665     }
14666     field(PROC, DBF_UCHAR) {
14667         interest(3)
14668         pp(TRUE)
14669         prompt("Force Processing")
14670     }
14671     field(STAT, DBF_MENU) {
14672         initial("UDF")
14673         menu(menuAlarmStat)
14674         special(SPC_NOMOD)
14675         prompt("Alarm Status")
```

```
14676     }
14677     field(SEVR, DBF_MENU) {
14678         menu(menuAlarmSevr)
14679         special(SPC_NOMOD)
14680         prompt("Alarm Severity")
14681     }
14682     field(AMSG, DBF_STRING) {
14683         size(40)
14684         special(SPC_NOMOD)
14685         prompt("Alarm Message")
14686     }
14687     field(NSTA, DBF_MENU) {
14688         interest(2)
14689         menu(menuAlarmStat)
14690         special(SPC_NOMOD)
14691         prompt("New Alarm Status")
14692     }
14693     field(NSEV, DBF_MENU) {
14694         interest(2)
14695         menu(menuAlarmSevr)
14696         special(SPC_NOMOD)
14697         prompt("New Alarm Severity")
14698     }
14699     field(NAMSG, DBF_STRING) {
14700         size(40)
14701         special(SPC_NOMOD)
14702         prompt("New Alarm Message")
14703     }
14704     field(ACKS, DBF_MENU) {
14705         interest(2)
14706         menu(menuAlarmSevr)
14707         special(SPC_NOMOD)
14708         prompt("Alarm Ack Severity")
14709     }
14710     field(ACKT, DBF_MENU) {
14711         promptgroup("70 - Alarm")
14712         interest(2)
14713         initial("YES")
14714         menu(menuYesNo)
14715         special(SPC_NOMOD)
14716         prompt("Alarm Ack Transient")
14717     }
14718     field(DISS, DBF_MENU) {
14719         promptgroup("70 - Alarm")
14720         interest(1)
14721         menu(menuAlarmSevr)
14722         prompt("Disable Alarm Sevrty")
14723     }
14724     field(LCNT, DBF_UCHAR) {
14725         interest(2)
14726         special(SPC_NOMOD)
14727         prompt("Lock Count")
14728     }
14729     field(PACT, DBF_UCHAR) {
14730         interest(1)
14731         special(SPC_NOMOD)
14732         prompt("Record active")
14733     }
14734     field(PUTF, DBF_UCHAR) {
14735         interest(1)
14736         special(SPC_NOMOD)
14737         prompt("dbPutField process")
14738     }
14739     field(RPRO, DBF_UCHAR) {
14740         interest(1)
14741         special(SPC_NOMOD)
14742         prompt("Reprocess ")
14743     }
```

```
14744     field(ASP, DBF_NOACCESS) {
14745         interest(4)
14746         extra("struct asgMember    *asp")
14747         special(SPC_NOMOD)
14748         prompt("Access Security Pvt")
14749     }
14750     field(PPN, DBF_NOACCESS) {
14751         interest(4)
14752         extra("struct processNotify *ppn")
14753         special(SPC_NOMOD)
14754         prompt("pprocessNotify")
14755     }
14756     field(PPNR, DBF_NOACCESS) {
14757         interest(4)
14758         extra("struct processNotifyRecord *ppnr")
14759         special(SPC_NOMOD)
14760         prompt("pprocessNotifyRecord")
14761     }
14762     field(SPVT, DBF_NOACCESS) {
14763         interest(4)
14764         extra("struct scan_element *spvt")
14765         special(SPC_NOMOD)
14766         prompt("Scan Private")
14767     }
14768     field(RSET, DBF_NOACCESS) {
14769         interest(4)
14770         extra("struct typed_rset   *rset")
14771         special(SPC_NOMOD)
14772         prompt("Address of RSET")
14773     }
14774     field(DSET, DBF_NOACCESS) {
14775         interest(4)
14776         extra("unambiguous_dset    *dset")
14777         special(SPC_NOMOD)
14778         prompt("DSET address")
14779     }
14780     field(DPVT, DBF_NOACCESS) {
14781         interest(4)
14782         extra("void                *dpvt")
14783         special(SPC_NOMOD)
14784         prompt("Device Private")
14785     }
14786     field(RDES, DBF_NOACCESS) {
14787         interest(4)
14788         extra("struct dbRecordType *rdes")
14789         special(SPC_NOMOD)
14790         prompt("Address of dbRecordType")
14791     }
14792     field(LSET, DBF_NOACCESS) {
14793         interest(4)
14794         extra("struct lockRecord   *lset")
14795         special(SPC_NOMOD)
14796         prompt("Lock Set")
14797     }
14798     field(PRIO, DBF_MENU) {
14799         promptgroup("20 - Scan")
14800         interest(1)
14801         menu(menuPriority)
14802         special(SPC_SCAN)
14803         prompt("Scheduling Priority")
14804     }
14805     field(TPRO, DBF_UCHAR) {
14806         prompt("Trace Processing")
14807     }
14808     field(BKPT, DBF_NOACCESS) {
14809         interest(1)
14810         extra("epicsUInt8          bkpt")
14811         special(SPC_NOMOD)
```

```
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_USHORT) {
        promptgroup("40 - Input")
        asl(ASL0)
        pp(TRUE)
        prompt("Status")
    }
    field(WFLG, DBF_USHORT) {
        pp(TRUE)
        prompt("Wait Flag")
    }
    field(LABL, DBF_STRING) {
        size(20)
        promptgroup("80 - Display")
        interest(1)
        pp(TRUE)
        prompt("Button Label")
    }
    field(OVAL, DBF_USHORT) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Old Status")
    }
    field(OFLG, DBF_USHORT) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Old Flag")
    }
}
recordtype(printf) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
```

```
    %/* Declare Device Support Entry Table */
    %struct printfRecord;
    %typedef struct printfdset {
    %    dset common;
    %    long (*write_string)(struct printfRecord *prec);
    %} printfdset;
    %#define HAS_printfdset
    %
    %/* Number of INPx fields defined */
    %#define PRINTF_NLINKS 10
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
    field(DISV, DBF_SHORT) {
```

```
14948         promptgroup("20 - Scan")
14949         initial("1")
14950         prompt("Disable Value")
14951     }
14952     field(DISA, DBF_SHORT) {
14953         prompt("Disable")
14954     }
14955     field(SDIS, DBF_INLINK) {
14956         promptgroup("20 - Scan")
14957         interest(1)
14958         prompt("Scanning Disable")
14959     }
14960     field(MLOK, DBF_NOACCESS) {
14961         interest(4)
14962         extra("epicsMutexId        mlok")
14963         special(SPC_NOMOD)
14964         prompt("Monitor lock")
14965     }
14966     field(MLIS, DBF_NOACCESS) {
14967         interest(4)
14968         extra("ELLLIST             mlis")
14969         special(SPC_NOMOD)
14970         prompt("Monitor List")
14971     }
14972     field(BKLNK, DBF_NOACCESS) {
14973         interest(4)
14974         extra("ELLLIST             bklnk")
14975         special(SPC_NOMOD)
14976         prompt("Backwards link tracking")
14977     }
14978     field(DISP, DBF_UCHAR) {
14979         promptgroup("10 - Common")
14980         interest(1)
14981         prompt("Disable putField")
14982     }
14983     field(PROC, DBF_UCHAR) {
14984         interest(3)
14985         pp(TRUE)
14986         prompt("Force Processing")
14987     }
14988     field(STAT, DBF_MENU) {
14989         initial("UDF")
14990         menu(menuAlarmStat)
14991         special(SPC_NOMOD)
14992         prompt("Alarm Status")
14993     }
14994     field(SEVR, DBF_MENU) {
14995         menu(menuAlarmSevr)
14996         special(SPC_NOMOD)
14997         prompt("Alarm Severity")
14998     }
14999     field(AMSG, DBF_STRING) {
15000         size(40)
15001         special(SPC_NOMOD)
15002         prompt("Alarm Message")
15003     }
15004     field(NSTA, DBF_MENU) {
15005         interest(2)
15006         menu(menuAlarmStat)
15007         special(SPC_NOMOD)
15008         prompt("New Alarm Status")
15009     }
15010     field(NSEV, DBF_MENU) {
15011         interest(2)
15012         menu(menuAlarmSevr)
15013         special(SPC_NOMOD)
15014         prompt("New Alarm Severity")
15015     }
```

```
15016     field(NAMSG, DBF_STRING) {
15017         size(40)
15018         special(SPC_NOMOD)
15019         prompt("New Alarm Message")
15020     }
15021     field(ACKS, DBF_MENU) {
15022         interest(2)
15023         menu(menuAlarmSevr)
15024         special(SPC_NOMOD)
15025         prompt("Alarm Ack Severity")
15026     }
15027     field(ACKT, DBF_MENU) {
15028         promptgroup("70 - Alarm")
15029         interest(2)
15030         initial("YES")
15031         menu(menuYesNo)
15032         special(SPC_NOMOD)
15033         prompt("Alarm Ack Transient")
15034     }
15035     field(DISS, DBF_MENU) {
15036         promptgroup("70 - Alarm")
15037         interest(1)
15038         menu(menuAlarmSevr)
15039         prompt("Disable Alarm Sevrty")
15040     }
15041     field(LCNT, DBF_UCHAR) {
15042         interest(2)
15043         special(SPC_NOMOD)
15044         prompt("Lock Count")
15045     }
15046     field(PACT, DBF_UCHAR) {
15047         interest(1)
15048         special(SPC_NOMOD)
15049         prompt("Record active")
15050     }
15051     field(PUTF, DBF_UCHAR) {
15052         interest(1)
15053         special(SPC_NOMOD)
15054         prompt("dbPutField process")
15055     }
15056     field(RPRO, DBF_UCHAR) {
15057         interest(1)
15058         special(SPC_NOMOD)
15059         prompt("Reprocess ")
15060     }
15061     field(ASP, DBF_NOACCESS) {
15062         interest(4)
15063         extra("struct asgMember    *asp")
15064         special(SPC_NOMOD)
15065         prompt("Access Security Pvt")
15066     }
15067     field(PPN, DBF_NOACCESS) {
15068         interest(4)
15069         extra("struct processNotify *ppn")
15070         special(SPC_NOMOD)
15071         prompt("pprocessNotify")
15072     }
15073     field(PPNR, DBF_NOACCESS) {
15074         interest(4)
15075         extra("struct processNotifyRecord *ppnr")
15076         special(SPC_NOMOD)
15077         prompt("pprocessNotifyRecord")
15078     }
15079     field(SPVT, DBF_NOACCESS) {
15080         interest(4)
15081         extra("struct scan_element *spvt")
15082         special(SPC_NOMOD)
15083         prompt("Scan Private")
```

```
15084     }
15085     field(RSET, DBF_NOACCESS) {
15086         interest(4)
15087         extra("struct typed_rset   *rset")
15088         special(SPC_NOMOD)
15089         prompt("Address of RSET")
15090     }
15091     field(DSET, DBF_NOACCESS) {
15092         interest(4)
15093         extra("unambiguous_dset    *dset")
15094         special(SPC_NOMOD)
15095         prompt("DSET address")
15096     }
15097     field(DPVT, DBF_NOACCESS) {
15098         interest(4)
15099         extra("void                *dpvt")
15100         special(SPC_NOMOD)
15101         prompt("Device Private")
15102     }
15103     field(RDES, DBF_NOACCESS) {
15104         interest(4)
15105         extra("struct dbRecordType *rdes")
15106         special(SPC_NOMOD)
15107         prompt("Address of dbRecordType")
15108     }
15109     field(LSET, DBF_NOACCESS) {
15110         interest(4)
15111         extra("struct lockRecord   *lset")
15112         special(SPC_NOMOD)
15113         prompt("Lock Set")
15114     }
15115     field(PRIO, DBF_MENU) {
15116         promptgroup("20 - Scan")
15117         interest(1)
15118         menu(menuPriority)
15119         special(SPC_SCAN)
15120         prompt("Scheduling Priority")
15121     }
15122     field(TPRO, DBF_UCHAR) {
15123         prompt("Trace Processing")
15124     }
15125     field(BKPT, DBF_NOACCESS) {
15126         interest(1)
15127         extra("epicsUInt8          bkpt")
15128         special(SPC_NOMOD)
15129         prompt("Break Point")
15130     }
15131     field(UDF, DBF_UCHAR) {
15132         promptgroup("10 - Common")
15133         interest(1)
15134         initial("1")
15135         pp(TRUE)
15136         prompt("Undefined")
15137     }
15138     field(UDFS, DBF_MENU) {
15139         promptgroup("70 - Alarm")
15140         interest(1)
15141         initial("INVALID")
15142         menu(menuAlarmSevr)
15143         prompt("Undefined Alarm Sevrty")
15144     }
15145     field(TIME, DBF_NOACCESS) {
15146         interest(2)
15147         extra("epicsTimeStamp      time")
15148         special(SPC_NOMOD)
15149         prompt("Time")
15150     }
15151     field(UTAG, DBF_UINT64) {
```

```
15152         interest(3)
15153         special(SPC_NOMOD)
15154         prompt("Time Tag")
15155     }
15156     field(FLNK, DBF_FWDLINK) {
15157         promptgroup("20 - Scan")
15158         interest(1)
15159         prompt("Forward Process Link")
15160     }
15161     field(VAL, DBF_NOACCESS) {
15162         extra("char *val")
15163         asl(ASL0)
15164         special(SPC_DBADDR)
15165         pp(TRUE)
15166         prompt("Result")
15167     }
15168     field(SIZV, DBF_USHORT) {
15169         promptgroup("30 - Action")
15170         interest(1)
15171         initial("41")
15172         special(SPC_NOMOD)
15173         prompt("Size of VAL buffer")
15174     }
15175     field(LEN, DBF_ULONG) {
15176         special(SPC_NOMOD)
15177         prompt("Length of VAL")
15178     }
15179     field(OUT, DBF_OUTLINK) {
15180         promptgroup("50 - Output")
15181         interest(1)
15182         prompt("Output Specification")
15183     }
15184     field(FMT, DBF_STRING) {
15185         size(81)
15186         promptgroup("30 - Action")
15187         pp(TRUE)
15188         prompt("Format String")
15189     }
15190     field(IVLS, DBF_STRING) {
15191         size(16)
15192         promptgroup("30 - Action")
15193         initial("LNK")
15194         prompt("Invalid Link String")
15195     }
15196     field(INP0, DBF_INLINK) {
15197         promptgroup("40 - Input")
15198         interest(1)
15199         prompt("Input 0")
15200     }
15201     field(INP1, DBF_INLINK) {
15202         promptgroup("40 - Input")
15203         interest(1)
15204         prompt("Input 1")
15205     }
15206     field(INP2, DBF_INLINK) {
15207         promptgroup("40 - Input")
15208         interest(1)
15209         prompt("Input 2")
15210     }
15211     field(INP3, DBF_INLINK) {
15212         promptgroup("40 - Input")
15213         interest(1)
15214         prompt("Input 3")
15215     }
15216     field(INP4, DBF_INLINK) {
15217         promptgroup("40 - Input")
15218         interest(1)
15219         prompt("Input 4")
```

```
    }
    field(INP5, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input 5")
    }
    field(INP6, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input 6")
    }
    field(INP7, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input 7")
    }
    field(INP8, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input 8")
    }
    field(INP9, DBF_INLINK) {
        promptgroup("40 - Input")
        interest(1)
        prompt("Input 9")
    }
}
device(printf, CONSTANT, devPrintfSoft, "Soft Channel")
device(printf, CONSTANT, devPrintfSoftCallback, "Async Soft Channel")
device(printf, INST_IO, devPrintfStdio, "stdio")
recordtype(sel) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
```

```
15288         interest(1)
15289         special(SPC_SCAN)
15290         prompt("Scan Phase")
15291     }
15292     field(EVNT, DBF_STRING) {
15293         size(40)
15294         promptgroup("20 - Scan")
15295         interest(1)
15296         special(SPC_SCAN)
15297         prompt("Event Name")
15298     }
15299     field(TSE, DBF_SHORT) {
15300         promptgroup("20 - Scan")
15301         interest(1)
15302         prompt("Time Stamp Event")
15303     }
15304     field(TSEL, DBF_INLINK) {
15305         promptgroup("20 - Scan")
15306         interest(1)
15307         prompt("Time Stamp Link")
15308     }
15309     field(DTYP, DBF_DEVICE) {
15310         promptgroup("10 - Common")
15311         interest(1)
15312         prompt("Device Type")
15313     }
15314     field(DISV, DBF_SHORT) {
15315         promptgroup("20 - Scan")
15316         initial("1")
15317         prompt("Disable Value")
15318     }
15319     field(DISA, DBF_SHORT) {
15320         prompt("Disable")
15321     }
15322     field(SDIS, DBF_INLINK) {
15323         promptgroup("20 - Scan")
15324         interest(1)
15325         prompt("Scanning Disable")
15326     }
15327     field(MLOK, DBF_NOACCESS) {
15328         interest(4)
15329         extra("epicsMutexId        mlok")
15330         special(SPC_NOMOD)
15331         prompt("Monitor lock")
15332     }
15333     field(MLIS, DBF_NOACCESS) {
15334         interest(4)
15335         extra("ELLLIST             mlis")
15336         special(SPC_NOMOD)
15337         prompt("Monitor List")
15338     }
15339     field(BKLNK, DBF_NOACCESS) {
15340         interest(4)
15341         extra("ELLLIST             bklnk")
15342         special(SPC_NOMOD)
15343         prompt("Backwards link tracking")
15344     }
15345     field(DISP, DBF_UCHAR) {
15346         promptgroup("10 - Common")
15347         interest(1)
15348         prompt("Disable putField")
15349     }
15350     field(PROC, DBF_UCHAR) {
15351         interest(3)
15352         pp(TRUE)
15353         prompt("Force Processing")
15354     }
15355     field(STAT, DBF_MENU) {
```

```
15356         initial("UDF")
15357         menu(menuAlarmStat)
15358         special(SPC_NOMOD)
15359         prompt("Alarm Status")
15360     }
15361     field(SEVR, DBF_MENU) {
15362         menu(menuAlarmSevr)
15363         special(SPC_NOMOD)
15364         prompt("Alarm Severity")
15365     }
15366     field(AMSG, DBF_STRING) {
15367         size(40)
15368         special(SPC_NOMOD)
15369         prompt("Alarm Message")
15370     }
15371     field(NSTA, DBF_MENU) {
15372         interest(2)
15373         menu(menuAlarmStat)
15374         special(SPC_NOMOD)
15375         prompt("New Alarm Status")
15376     }
15377     field(NSEV, DBF_MENU) {
15378         interest(2)
15379         menu(menuAlarmSevr)
15380         special(SPC_NOMOD)
15381         prompt("New Alarm Severity")
15382     }
15383     field(NAMSG, DBF_STRING) {
15384         size(40)
15385         special(SPC_NOMOD)
15386         prompt("New Alarm Message")
15387     }
15388     field(ACKS, DBF_MENU) {
15389         interest(2)
15390         menu(menuAlarmSevr)
15391         special(SPC_NOMOD)
15392         prompt("Alarm Ack Severity")
15393     }
15394     field(ACKT, DBF_MENU) {
15395         promptgroup("70 - Alarm")
15396         interest(2)
15397         initial("YES")
15398         menu(menuYesNo)
15399         special(SPC_NOMOD)
15400         prompt("Alarm Ack Transient")
15401     }
15402     field(DISS, DBF_MENU) {
15403         promptgroup("70 - Alarm")
15404         interest(1)
15405         menu(menuAlarmSevr)
15406         prompt("Disable Alarm Sevrty")
15407     }
15408     field(LCNT, DBF_UCHAR) {
15409         interest(2)
15410         special(SPC_NOMOD)
15411         prompt("Lock Count")
15412     }
15413     field(PACT, DBF_UCHAR) {
15414         interest(1)
15415         special(SPC_NOMOD)
15416         prompt("Record active")
15417     }
15418     field(PUTF, DBF_UCHAR) {
15419         interest(1)
15420         special(SPC_NOMOD)
15421         prompt("dbPutField process")
15422     }
15423     field(RPRO, DBF_UCHAR) {
```

```
15424         interest(1)
15425         special(SPC_NOMOD)
15426         prompt("Reprocess ")
15427     }
15428     field(ASP, DBF_NOACCESS) {
15429         interest(4)
15430         extra("struct asgMember    *asp")
15431         special(SPC_NOMOD)
15432         prompt("Access Security Pvt")
15433     }
15434     field(PPN, DBF_NOACCESS) {
15435         interest(4)
15436         extra("struct processNotify *ppn")
15437         special(SPC_NOMOD)
15438         prompt("pprocessNotify")
15439     }
15440     field(PPNR, DBF_NOACCESS) {
15441         interest(4)
15442         extra("struct processNotifyRecord *ppnr")
15443         special(SPC_NOMOD)
15444         prompt("pprocessNotifyRecord")
15445     }
15446     field(SPVT, DBF_NOACCESS) {
15447         interest(4)
15448         extra("struct scan_element *spvt")
15449         special(SPC_NOMOD)
15450         prompt("Scan Private")
15451     }
15452     field(RSET, DBF_NOACCESS) {
15453         interest(4)
15454         extra("struct typed_rset   *rset")
15455         special(SPC_NOMOD)
15456         prompt("Address of RSET")
15457     }
15458     field(DSET, DBF_NOACCESS) {
15459         interest(4)
15460         extra("unambiguous_dset    *dset")
15461         special(SPC_NOMOD)
15462         prompt("DSET address")
15463     }
15464     field(DPVT, DBF_NOACCESS) {
15465         interest(4)
15466         extra("void                *dpvt")
15467         special(SPC_NOMOD)
15468         prompt("Device Private")
15469     }
15470     field(RDES, DBF_NOACCESS) {
15471         interest(4)
15472         extra("struct dbRecordType *rdes")
15473         special(SPC_NOMOD)
15474         prompt("Address of dbRecordType")
15475     }
15476     field(LSET, DBF_NOACCESS) {
15477         interest(4)
15478         extra("struct lockRecord   *lset")
15479         special(SPC_NOMOD)
15480         prompt("Lock Set")
15481     }
15482     field(PRIO, DBF_MENU) {
15483         promptgroup("20 - Scan")
15484         interest(1)
15485         menu(menuPriority)
15486         special(SPC_SCAN)
15487         prompt("Scheduling Priority")
15488     }
15489     field(TPRO, DBF_UCHAR) {
15490         prompt("Trace Processing")
15491     }
```

```
15492     field(BKPT, DBF_NOACCESS) {
15493         interest(1)
15494         extra("epicsUInt8          bkpt")
15495         special(SPC_NOMOD)
15496         prompt("Break Point")
15497     }
15498     field(UDF, DBF_UCHAR) {
15499         promptgroup("10 - Common")
15500         interest(1)
15501         initial("1")
15502         pp(TRUE)
15503         prompt("Undefined")
15504     }
15505     field(UDFS, DBF_MENU) {
15506         promptgroup("70 - Alarm")
15507         interest(1)
15508         initial("INVALID")
15509         menu(menuAlarmSevr)
15510         prompt("Undefined Alarm Sevrty")
15511     }
15512     field(TIME, DBF_NOACCESS) {
15513         interest(2)
15514         extra("epicsTimeStamp      time")
15515         special(SPC_NOMOD)
15516         prompt("Time")
15517     }
15518     field(UTAG, DBF_UINT64) {
15519         interest(3)
15520         special(SPC_NOMOD)
15521         prompt("Time Tag")
15522     }
15523     field(FLNK, DBF_FWDLINK) {
15524         promptgroup("20 - Scan")
15525         interest(1)
15526         prompt("Forward Process Link")
15527     }
15528     field(VAL, DBF_DOUBLE) {
15529         promptgroup("40 - Input")
15530         asl(ASL0)
15531         special(SPC_NOMOD)
15532         prompt("Result")
15533     }
15534     field(SELM, DBF_MENU) {
15535         promptgroup("30 - Action")
15536         menu(selSELM)
15537         prompt("Select Mechanism")
15538     }
15539     field(SELN, DBF_USHORT) {
15540         prompt("Index value")
15541     }
15542     field(PREC, DBF_SHORT) {
15543         promptgroup("80 - Display")
15544         interest(1)
15545         prop(YES)
15546         prompt("Display Precision")
15547     }
15548     field(NVL, DBF_INLINK) {
15549         promptgroup("30 - Action")
15550         interest(1)
15551         prompt("Index Value Location")
15552     }
15553     field(INPA, DBF_INLINK) {
15554         promptgroup("41 - Input A-F")
15555         interest(1)
15556         prompt("Input A")
15557     }
15558     field(INPB, DBF_INLINK) {
15559         promptgroup("41 - Input A-F")
```

```
        interest(1)
        prompt("Input B")
    }
    field(INPC, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input C")
    }
    field(INPD, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input D")
    }
    field(INPE, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input E")
    }
    field(INPF, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input F")
    }
    field(INPG, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input G")
    }
    field(INPH, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input H")
    }
    field(INPI, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input I")
    }
    field(INPJ, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input J")
    }
    field(INPK, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input K")
    }
    field(INPL, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input L")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Rng")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
```

```
15628         interest(1)
15629         prop(YES)
15630         prompt("Low Operating Range")
15631     }
15632     field(HIHI, DBF_DOUBLE) {
15633         promptgroup("70 - Alarm")
15634         interest(1)
15635         pp(TRUE)
15636         prop(YES)
15637         prompt("Hihi Alarm Limit")
15638     }
15639     field(LOLO, DBF_DOUBLE) {
15640         promptgroup("70 - Alarm")
15641         interest(1)
15642         pp(TRUE)
15643         prop(YES)
15644         prompt("Lolo Alarm Limit")
15645     }
15646     field(HIGH, DBF_DOUBLE) {
15647         promptgroup("70 - Alarm")
15648         interest(1)
15649         pp(TRUE)
15650         prop(YES)
15651         prompt("High Alarm Limit")
15652     }
15653     field(LOW, DBF_DOUBLE) {
15654         promptgroup("70 - Alarm")
15655         interest(1)
15656         pp(TRUE)
15657         prop(YES)
15658         prompt("Low Alarm Limit")
15659     }
15660     field(HHSV, DBF_MENU) {
15661         promptgroup("70 - Alarm")
15662         menu(menuAlarmSevr)
15663         prompt("Hihi Severity")
15664         prop(YES)
15665         interest(1)
15666         pp(TRUE)
15667     }
15668     field(LLSV, DBF_MENU) {
15669         promptgroup("70 - Alarm")
15670         menu(menuAlarmSevr)
15671         prompt("Lolo Severity")
15672         prop(YES)
15673         interest(1)
15674         pp(TRUE)
15675     }
15676     field(HSV, DBF_MENU) {
15677         promptgroup("70 - Alarm")
15678         menu(menuAlarmSevr)
15679         prompt("High Severity")
15680         prop(YES)
15681         interest(1)
15682         pp(TRUE)
15683     }
15684     field(LSV, DBF_MENU) {
15685         promptgroup("70 - Alarm")
15686         menu(menuAlarmSevr)
15687         prompt("Low Severity")
15688         prop(YES)
15689         interest(1)
15690         pp(TRUE)
15691     }
15692     field(HYST, DBF_DOUBLE) {
15693         promptgroup("70 - Alarm")
15694         interest(1)
15695         prompt("Alarm Deadband")
```

```
15696     }
15697     field(ADEL, DBF_DOUBLE) {
15698         promptgroup("80 - Display")
15699         interest(1)
15700         prompt("Archive Deadband")
15701     }
15702     field(MDEL, DBF_DOUBLE) {
15703         promptgroup("80 - Display")
15704         interest(1)
15705         prompt("Monitor Deadband")
15706     }
15707     field(A, DBF_DOUBLE) {
15708         pp(TRUE)
15709         prompt("Value of Input A")
15710     }
15711     field(B, DBF_DOUBLE) {
15712         pp(TRUE)
15713         prompt("Value of Input B")
15714     }
15715     field(C, DBF_DOUBLE) {
15716         pp(TRUE)
15717         prompt("Value of Input C")
15718     }
15719     field(D, DBF_DOUBLE) {
15720         pp(TRUE)
15721         prompt("Value of Input D")
15722     }
15723     field(E, DBF_DOUBLE) {
15724         pp(TRUE)
15725         prompt("Value of Input E")
15726     }
15727     field(F, DBF_DOUBLE) {
15728         pp(TRUE)
15729         prompt("Value of Input F")
15730     }
15731     field(G, DBF_DOUBLE) {
15732         pp(TRUE)
15733         prompt("Value of Input G")
15734     }
15735     field(H, DBF_DOUBLE) {
15736         pp(TRUE)
15737         prompt("Value of Input H")
15738     }
15739     field(I, DBF_DOUBLE) {
15740         pp(TRUE)
15741         prompt("Value of Input I")
15742     }
15743     field(J, DBF_DOUBLE) {
15744         pp(TRUE)
15745         prompt("Value of Input J")
15746     }
15747     field(K, DBF_DOUBLE) {
15748         pp(TRUE)
15749         prompt("Value of Input K")
15750     }
15751     field(L, DBF_DOUBLE) {
15752         pp(TRUE)
15753         prompt("Value of Input L")
15754     }
15755     field(LA, DBF_DOUBLE) {
15756         interest(3)
15757         special(SPC_NOMOD)
15758         prompt("Prev Value of A")
15759     }
15760     field(LB, DBF_DOUBLE) {
15761         interest(3)
15762         special(SPC_NOMOD)
15763         prompt("Prev Value of B")
```

```
15764     }
15765     field(LC, DBF_DOUBLE) {
15766         interest(3)
15767         special(SPC_NOMOD)
15768         prompt("Prev Value of C")
15769     }
15770     field(LD, DBF_DOUBLE) {
15771         interest(3)
15772         special(SPC_NOMOD)
15773         prompt("Prev Value of D")
15774     }
15775     field(LE, DBF_DOUBLE) {
15776         interest(3)
15777         special(SPC_NOMOD)
15778         prompt("Prev Value of E")
15779     }
15780     field(LF, DBF_DOUBLE) {
15781         interest(3)
15782         special(SPC_NOMOD)
15783         prompt("Prev Value of F")
15784     }
15785     field(LG, DBF_DOUBLE) {
15786         interest(3)
15787         special(SPC_NOMOD)
15788         prompt("Prev Value of G")
15789     }
15790     field(LH, DBF_DOUBLE) {
15791         interest(3)
15792         special(SPC_NOMOD)
15793         prompt("Prev Value of H")
15794     }
15795     field(LI, DBF_DOUBLE) {
15796         interest(3)
15797         special(SPC_NOMOD)
15798         prompt("Prev Value of I")
15799     }
15800     field(LJ, DBF_DOUBLE) {
15801         interest(3)
15802         special(SPC_NOMOD)
15803         prompt("Prev Value of J")
15804     }
15805     field(LK, DBF_DOUBLE) {
15806         interest(3)
15807         special(SPC_NOMOD)
15808         prompt("Prev Value of K")
15809     }
15810     field(LL, DBF_DOUBLE) {
15811         interest(3)
15812         special(SPC_NOMOD)
15813         prompt("Prev Value of L")
15814     }
15815     field(LALM, DBF_DOUBLE) {
15816         interest(3)
15817         special(SPC_NOMOD)
15818         prompt("Last Value Alarmed")
15819     }
15820     field(ALST, DBF_DOUBLE) {
15821         interest(3)
15822         special(SPC_NOMOD)
15823         prompt("Last Value Archived")
15824     }
15825     field(MLST, DBF_DOUBLE) {
15826         interest(3)
15827         special(SPC_NOMOD)
15828         prompt("Last Val Monitored")
15829     }
15830     field(NLST, DBF_USHORT) {
15831         interest(3)
```

```
        special(SPC_NOMOD)
        prompt("Last Index Monitored")
    }
}
recordtype(seq) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Event")
    }
    field(TSEL, DBF_INLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Time Stamp Link")
    }
    field(DTYP, DBF_DEVICE) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Device Type")
    }
```

```
15900     field(DISV, DBF_SHORT) {
15901         promptgroup("20 - Scan")
15902         initial("1")
15903         prompt("Disable Value")
15904     }
15905     field(DISA, DBF_SHORT) {
15906         prompt("Disable")
15907     }
15908     field(SDIS, DBF_INLINK) {
15909         promptgroup("20 - Scan")
15910         interest(1)
15911         prompt("Scanning Disable")
15912     }
15913     field(MLOK, DBF_NOACCESS) {
15914         interest(4)
15915         extra("epicsMutexId        mlok")
15916         special(SPC_NOMOD)
15917         prompt("Monitor lock")
15918     }
15919     field(MLIS, DBF_NOACCESS) {
15920         interest(4)
15921         extra("ELLLIST             mlis")
15922         special(SPC_NOMOD)
15923         prompt("Monitor List")
15924     }
15925     field(BKLNK, DBF_NOACCESS) {
15926         interest(4)
15927         extra("ELLLIST             bklnk")
15928         special(SPC_NOMOD)
15929         prompt("Backwards link tracking")
15930     }
15931     field(DISP, DBF_UCHAR) {
15932         promptgroup("10 - Common")
15933         interest(1)
15934         prompt("Disable putField")
15935     }
15936     field(PROC, DBF_UCHAR) {
15937         interest(3)
15938         pp(TRUE)
15939         prompt("Force Processing")
15940     }
15941     field(STAT, DBF_MENU) {
15942         initial("UDF")
15943         menu(menuAlarmStat)
15944         special(SPC_NOMOD)
15945         prompt("Alarm Status")
15946     }
15947     field(SEVR, DBF_MENU) {
15948         menu(menuAlarmSevr)
15949         special(SPC_NOMOD)
15950         prompt("Alarm Severity")
15951     }
15952     field(AMSG, DBF_STRING) {
15953         size(40)
15954         special(SPC_NOMOD)
15955         prompt("Alarm Message")
15956     }
15957     field(NSTA, DBF_MENU) {
15958         interest(2)
15959         menu(menuAlarmStat)
15960         special(SPC_NOMOD)
15961         prompt("New Alarm Status")
15962     }
15963     field(NSEV, DBF_MENU) {
15964         interest(2)
15965         menu(menuAlarmSevr)
15966         special(SPC_NOMOD)
15967         prompt("New Alarm Severity")
```

```
15968     }
15969     field(NAMSG, DBF_STRING) {
15970         size(40)
15971         special(SPC_NOMOD)
15972         prompt("New Alarm Message")
15973     }
15974     field(ACKS, DBF_MENU) {
15975         interest(2)
15976         menu(menuAlarmSevr)
15977         special(SPC_NOMOD)
15978         prompt("Alarm Ack Severity")
15979     }
15980     field(ACKT, DBF_MENU) {
15981         promptgroup("70 - Alarm")
15982         interest(2)
15983         initial("YES")
15984         menu(menuYesNo)
15985         special(SPC_NOMOD)
15986         prompt("Alarm Ack Transient")
15987     }
15988     field(DISS, DBF_MENU) {
15989         promptgroup("70 - Alarm")
15990         interest(1)
15991         menu(menuAlarmSevr)
15992         prompt("Disable Alarm Sevrty")
15993     }
15994     field(LCNT, DBF_UCHAR) {
15995         interest(2)
15996         special(SPC_NOMOD)
15997         prompt("Lock Count")
15998     }
15999     field(PACT, DBF_UCHAR) {
16000         interest(1)
16001         special(SPC_NOMOD)
16002         prompt("Record active")
16003     }
16004     field(PUTF, DBF_UCHAR) {
16005         interest(1)
16006         special(SPC_NOMOD)
16007         prompt("dbPutField process")
16008     }
16009     field(RPRO, DBF_UCHAR) {
16010         interest(1)
16011         special(SPC_NOMOD)
16012         prompt("Reprocess ")
16013     }
16014     field(ASP, DBF_NOACCESS) {
16015         interest(4)
16016         extra("struct asgMember    *asp")
16017         special(SPC_NOMOD)
16018         prompt("Access Security Pvt")
16019     }
16020     field(PPN, DBF_NOACCESS) {
16021         interest(4)
16022         extra("struct processNotify *ppn")
16023         special(SPC_NOMOD)
16024         prompt("pprocessNotify")
16025     }
16026     field(PPNR, DBF_NOACCESS) {
16027         interest(4)
16028         extra("struct processNotifyRecord *ppnr")
16029         special(SPC_NOMOD)
16030         prompt("pprocessNotifyRecord")
16031     }
16032     field(SPVT, DBF_NOACCESS) {
16033         interest(4)
16034         extra("struct scan_element *spvt")
16035         special(SPC_NOMOD)
```

```
16036         prompt("Scan Private")
16037     }
16038     field(RSET, DBF_NOACCESS) {
16039         interest(4)
16040         extra("struct typed_rset   *rset")
16041         special(SPC_NOMOD)
16042         prompt("Address of RSET")
16043     }
16044     field(DSET, DBF_NOACCESS) {
16045         interest(4)
16046         extra("unambiguous_dset    *dset")
16047         special(SPC_NOMOD)
16048         prompt("DSET address")
16049     }
16050     field(DPVT, DBF_NOACCESS) {
16051         interest(4)
16052         extra("void                *dpvt")
16053         special(SPC_NOMOD)
16054         prompt("Device Private")
16055     }
16056     field(RDES, DBF_NOACCESS) {
16057         interest(4)
16058         extra("struct dbRecordType *rdes")
16059         special(SPC_NOMOD)
16060         prompt("Address of dbRecordType")
16061     }
16062     field(LSET, DBF_NOACCESS) {
16063         interest(4)
16064         extra("struct lockRecord   *lset")
16065         special(SPC_NOMOD)
16066         prompt("Lock Set")
16067     }
16068     field(PRIO, DBF_MENU) {
16069         promptgroup("20 - Scan")
16070         interest(1)
16071         menu(menuPriority)
16072         special(SPC_SCAN)
16073         prompt("Scheduling Priority")
16074     }
16075     field(TPRO, DBF_UCHAR) {
16076         prompt("Trace Processing")
16077     }
16078     field(BKPT, DBF_NOACCESS) {
16079         interest(1)
16080         extra("epicsUInt8          bkpt")
16081         special(SPC_NOMOD)
16082         prompt("Break Point")
16083     }
16084     field(UDF, DBF_UCHAR) {
16085         promptgroup("10 - Common")
16086         interest(1)
16087         initial("1")
16088         pp(TRUE)
16089         prompt("Undefined")
16090     }
16091     field(UDFS, DBF_MENU) {
16092         promptgroup("70 - Alarm")
16093         interest(1)
16094         initial("INVALID")
16095         menu(menuAlarmSevr)
16096         prompt("Undefined Alarm Sevrty")
16097     }
16098     field(TIME, DBF_NOACCESS) {
16099         interest(2)
16100         extra("epicsTimeStamp      time")
16101         special(SPC_NOMOD)
16102         prompt("Time")
16103     }
```

```
    field(UTAG, DBF_UINT64) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Time Tag")
    }
    field(FLNK, DBF_FWDLINK) {
        promptgroup("20 - Scan")
        interest(1)
        prompt("Forward Process Link")
    }
    field(VAL, DBF_LONG) {
        asl(ASL0)
        pp(TRUE)
        prompt("Used to trigger")
    }
    field(SELM, DBF_MENU) {
        promptgroup("30 - Action")
        interest(1)
        menu(seqSELM)
        prompt("Select Mechanism")
    }
    field(SELN, DBF_USHORT) {
        interest(1)
        initial("1")
        prompt("Link Selection")
    }
    field(SELL, DBF_INLINK) {
        promptgroup("30 - Action")
        interest(1)
        prompt("Link Selection Loc")
    }
    field(OFFS, DBF_SHORT) {
        promptgroup("30 - Action")
        interest(1)
        initial("0")
        prompt("Offset for Specified")
    }
    field(SHFT, DBF_SHORT) {
        promptgroup("30 - Action")
        interest(1)
        initial("-1")
        prompt("Shift for Mask mode")
    }
    field(OLDN, DBF_USHORT) {
        interest(4)
        prompt("Old Selection")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Display Precision")
    }
    field(DLY0, DBF_DOUBLE) {
        promptgroup("41 - Link 0-7")
        interest(1)
        prompt("Delay 0")
    }
    field(DOL0, DBF_INLINK) {
        promptgroup("41 - Link 0-7")
        interest(1)
        prompt("Input link 0")
    }
    field(DO0, DBF_DOUBLE) {
        interest(1)
        prompt("Value 0")
    }
    field(LNK0, DBF_OUTLINK) {
        promptgroup("41 - Link 0-7")
```

```
16172         interest(1)
16173         prompt("Output Link 0")
16174     }
16175     field(DLY1, DBF_DOUBLE) {
16176         promptgroup("41 - Link 0-7")
16177         interest(1)
16178         prompt("Delay 1")
16179     }
16180     field(DOL1, DBF_INLINK) {
16181         promptgroup("41 - Link 0-7")
16182         interest(1)
16183         prompt("Input link1")
16184     }
16185     field(DO1, DBF_DOUBLE) {
16186         interest(1)
16187         prompt("Value 1")
16188     }
16189     field(LNK1, DBF_OUTLINK) {
16190         promptgroup("41 - Link 0-7")
16191         interest(1)
16192         prompt("Output Link 1")
16193     }
16194     field(DLY2, DBF_DOUBLE) {
16195         promptgroup("41 - Link 0-7")
16196         interest(1)
16197         prompt("Delay 2")
16198     }
16199     field(DOL2, DBF_INLINK) {
16200         promptgroup("41 - Link 0-7")
16201         interest(1)
16202         prompt("Input link 2")
16203     }
16204     field(DO2, DBF_DOUBLE) {
16205         interest(1)
16206         prompt("Value 2")
16207     }
16208     field(LNK2, DBF_OUTLINK) {
16209         promptgroup("41 - Link 0-7")
16210         interest(1)
16211         prompt("Output Link 2")
16212     }
16213     field(DLY3, DBF_DOUBLE) {
16214         promptgroup("41 - Link 0-7")
16215         interest(1)
16216         prompt("Delay 3")
16217     }
16218     field(DOL3, DBF_INLINK) {
16219         promptgroup("41 - Link 0-7")
16220         interest(1)
16221         prompt("Input link 3")
16222     }
16223     field(DO3, DBF_DOUBLE) {
16224         interest(1)
16225         prompt("Value 3")
16226     }
16227     field(LNK3, DBF_OUTLINK) {
16228         promptgroup("41 - Link 0-7")
16229         interest(1)
16230         prompt("Output Link 3")
16231     }
16232     field(DLY4, DBF_DOUBLE) {
16233         promptgroup("41 - Link 0-7")
16234         interest(1)
16235         prompt("Delay 4")
16236     }
16237     field(DOL4, DBF_INLINK) {
16238         promptgroup("41 - Link 0-7")
16239         interest(1)
```

```
16240         prompt("Input link 4")
16241     }
16242     field(DO4, DBF_DOUBLE) {
16243         interest(1)
16244         prompt("Value 4")
16245     }
16246     field(LNK4, DBF_OUTLINK) {
16247         promptgroup("41 - Link 0-7")
16248         interest(1)
16249         prompt("Output Link 4")
16250     }
16251     field(DLY5, DBF_DOUBLE) {
16252         promptgroup("41 - Link 0-7")
16253         interest(1)
16254         prompt("Delay 5")
16255     }
16256     field(DOL5, DBF_INLINK) {
16257         promptgroup("41 - Link 0-7")
16258         interest(1)
16259         prompt("Input link 5")
16260     }
16261     field(DO5, DBF_DOUBLE) {
16262         interest(1)
16263         prompt("Value 5")
16264     }
16265     field(LNK5, DBF_OUTLINK) {
16266         promptgroup("41 - Link 0-7")
16267         interest(1)
16268         prompt("Output Link 5")
16269     }
16270     field(DLY6, DBF_DOUBLE) {
16271         promptgroup("41 - Link 0-7")
16272         interest(1)
16273         prompt("Delay 6")
16274     }
16275     field(DOL6, DBF_INLINK) {
16276         promptgroup("41 - Link 0-7")
16277         interest(1)
16278         prompt("Input link 6")
16279     }
16280     field(DO6, DBF_DOUBLE) {
16281         interest(1)
16282         prompt("Value 6")
16283     }
16284     field(LNK6, DBF_OUTLINK) {
16285         promptgroup("41 - Link 0-7")
16286         interest(1)
16287         prompt("Output Link 6")
16288     }
16289     field(DLY7, DBF_DOUBLE) {
16290         promptgroup("41 - Link 0-7")
16291         interest(1)
16292         prompt("Delay 7")
16293     }
16294     field(DOL7, DBF_INLINK) {
16295         promptgroup("41 - Link 0-7")
16296         interest(1)
16297         prompt("Input link 7")
16298     }
16299     field(DO7, DBF_DOUBLE) {
16300         interest(1)
16301         prompt("Value 7")
16302     }
16303     field(LNK7, DBF_OUTLINK) {
16304         promptgroup("41 - Link 0-7")
16305         interest(1)
16306         prompt("Output Link 7")
16307     }
```

```
    field(DLY8, DBF_DOUBLE) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Delay 8")
    }
    field(DOL8, DBF_INLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Input link 8")
    }
    field(DO8, DBF_DOUBLE) {
        interest(1)
        prompt("Value 8")
    }
    field(LNK8, DBF_OUTLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Output Link 8")
    }
    field(DLY9, DBF_DOUBLE) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Delay 9")
    }
    field(DOL9, DBF_INLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Input link 9")
    }
    field(DO9, DBF_DOUBLE) {
        interest(1)
        prompt("Value 9")
    }
    field(LNK9, DBF_OUTLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Output Link 9")
    }
    field(DLYA, DBF_DOUBLE) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Delay 10")
    }
    field(DOLA, DBF_INLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Input link 10")
    }
    field(DOA, DBF_DOUBLE) {
        interest(1)
        prompt("Value 10")
    }
    field(LNKA, DBF_OUTLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Output Link 10")
    }
    field(DLYB, DBF_DOUBLE) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Delay 11")
    }
    field(DOLB, DBF_INLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Input link 11")
    }
    field(DOB, DBF_DOUBLE) {
```

```
16376         interest(1)
16377         prompt("Value 11")
16378     }
16379     field(LNKB, DBF_OUTLINK) {
16380         promptgroup("42 - Link 8-F")
16381         interest(1)
16382         prompt("Output Link 11")
16383     }
16384     field(DLYC, DBF_DOUBLE) {
16385         promptgroup("42 - Link 8-F")
16386         interest(1)
16387         prompt("Delay 12")
16388     }
16389     field(DOLC, DBF_INLINK) {
16390         promptgroup("42 - Link 8-F")
16391         interest(1)
16392         prompt("Input link 12")
16393     }
16394     field(DOC, DBF_DOUBLE) {
16395         interest(1)
16396         prompt("Value 12")
16397     }
16398     field(LNKC, DBF_OUTLINK) {
16399         promptgroup("42 - Link 8-F")
16400         interest(1)
16401         prompt("Output Link 12")
16402     }
16403     field(DLYD, DBF_DOUBLE) {
16404         promptgroup("42 - Link 8-F")
16405         interest(1)
16406         prompt("Delay 13")
16407     }
16408     field(DOLD, DBF_INLINK) {
16409         promptgroup("42 - Link 8-F")
16410         interest(1)
16411         prompt("Input link 13")
16412     }
16413     field(DOD, DBF_DOUBLE) {
16414         interest(1)
16415         prompt("Value 13")
16416     }
16417     field(LNKD, DBF_OUTLINK) {
16418         promptgroup("42 - Link 8-F")
16419         interest(1)
16420         prompt("Output Link 13")
16421     }
16422     field(DLYE, DBF_DOUBLE) {
16423         promptgroup("42 - Link 8-F")
16424         interest(1)
16425         prompt("Delay 14")
16426     }
16427     field(DOLE, DBF_INLINK) {
16428         promptgroup("42 - Link 8-F")
16429         interest(1)
16430         prompt("Input link 14")
16431     }
16432     field(DOE, DBF_DOUBLE) {
16433         interest(1)
16434         prompt("Value 14")
16435     }
16436     field(LNKE, DBF_OUTLINK) {
16437         promptgroup("42 - Link 8-F")
16438         interest(1)
16439         prompt("Output Link 14")
16440     }
16441     field(DLYF, DBF_DOUBLE) {
16442         promptgroup("42 - Link 8-F")
16443         interest(1)
```

```
        prompt("Delay 15")
    }
    field(DOLF, DBF_INLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Input link 15")
    }
    field(DOF, DBF_DOUBLE) {
        interest(1)
        prompt("Value 15")
    }
    field(LNKF, DBF_OUTLINK) {
        promptgroup("42 - Link 8-F")
        interest(1)
        prompt("Output Link 15")
    }
}
recordtype(state) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
```

```
16512         interest(1)
16513         prompt("Time Stamp Event")
16514     }
16515     field(TSEL, DBF_INLINK) {
16516         promptgroup("20 - Scan")
16517         interest(1)
16518         prompt("Time Stamp Link")
16519     }
16520     field(DTYP, DBF_DEVICE) {
16521         promptgroup("10 - Common")
16522         interest(1)
16523         prompt("Device Type")
16524     }
16525     field(DISV, DBF_SHORT) {
16526         promptgroup("20 - Scan")
16527         initial("1")
16528         prompt("Disable Value")
16529     }
16530     field(DISA, DBF_SHORT) {
16531         prompt("Disable")
16532     }
16533     field(SDIS, DBF_INLINK) {
16534         promptgroup("20 - Scan")
16535         interest(1)
16536         prompt("Scanning Disable")
16537     }
16538     field(MLOK, DBF_NOACCESS) {
16539         interest(4)
16540         extra("epicsMutexId        mlok")
16541         special(SPC_NOMOD)
16542         prompt("Monitor lock")
16543     }
16544     field(MLIS, DBF_NOACCESS) {
16545         interest(4)
16546         extra("ELLLIST             mlis")
16547         special(SPC_NOMOD)
16548         prompt("Monitor List")
16549     }
16550     field(BKLNK, DBF_NOACCESS) {
16551         interest(4)
16552         extra("ELLLIST             bklnk")
16553         special(SPC_NOMOD)
16554         prompt("Backwards link tracking")
16555     }
16556     field(DISP, DBF_UCHAR) {
16557         promptgroup("10 - Common")
16558         interest(1)
16559         prompt("Disable putField")
16560     }
16561     field(PROC, DBF_UCHAR) {
16562         interest(3)
16563         pp(TRUE)
16564         prompt("Force Processing")
16565     }
16566     field(STAT, DBF_MENU) {
16567         initial("UDF")
16568         menu(menuAlarmStat)
16569         special(SPC_NOMOD)
16570         prompt("Alarm Status")
16571     }
16572     field(SEVR, DBF_MENU) {
16573         menu(menuAlarmSevr)
16574         special(SPC_NOMOD)
16575         prompt("Alarm Severity")
16576     }
16577     field(AMSG, DBF_STRING) {
16578         size(40)
16579         special(SPC_NOMOD)
```

```
16580         prompt("Alarm Message")
16581     }
16582     field(NSTA, DBF_MENU) {
16583         interest(2)
16584         menu(menuAlarmStat)
16585         special(SPC_NOMOD)
16586         prompt("New Alarm Status")
16587     }
16588     field(NSEV, DBF_MENU) {
16589         interest(2)
16590         menu(menuAlarmSevr)
16591         special(SPC_NOMOD)
16592         prompt("New Alarm Severity")
16593     }
16594     field(NAMSG, DBF_STRING) {
16595         size(40)
16596         special(SPC_NOMOD)
16597         prompt("New Alarm Message")
16598     }
16599     field(ACKS, DBF_MENU) {
16600         interest(2)
16601         menu(menuAlarmSevr)
16602         special(SPC_NOMOD)
16603         prompt("Alarm Ack Severity")
16604     }
16605     field(ACKT, DBF_MENU) {
16606         promptgroup("70 - Alarm")
16607         interest(2)
16608         initial("YES")
16609         menu(menuYesNo)
16610         special(SPC_NOMOD)
16611         prompt("Alarm Ack Transient")
16612     }
16613     field(DISS, DBF_MENU) {
16614         promptgroup("70 - Alarm")
16615         interest(1)
16616         menu(menuAlarmSevr)
16617         prompt("Disable Alarm Sevrty")
16618     }
16619     field(LCNT, DBF_UCHAR) {
16620         interest(2)
16621         special(SPC_NOMOD)
16622         prompt("Lock Count")
16623     }
16624     field(PACT, DBF_UCHAR) {
16625         interest(1)
16626         special(SPC_NOMOD)
16627         prompt("Record active")
16628     }
16629     field(PUTF, DBF_UCHAR) {
16630         interest(1)
16631         special(SPC_NOMOD)
16632         prompt("dbPutField process")
16633     }
16634     field(RPRO, DBF_UCHAR) {
16635         interest(1)
16636         special(SPC_NOMOD)
16637         prompt("Reprocess ")
16638     }
16639     field(ASP, DBF_NOACCESS) {
16640         interest(4)
16641         extra("struct asgMember    *asp")
16642         special(SPC_NOMOD)
16643         prompt("Access Security Pvt")
16644     }
16645     field(PPN, DBF_NOACCESS) {
16646         interest(4)
16647         extra("struct processNotify *ppn")
```

```
16648         special(SPC_NOMOD)
16649         prompt("pprocessNotify")
16650     }
16651     field(PPNR, DBF_NOACCESS) {
16652         interest(4)
16653         extra("struct processNotifyRecord *ppnr")
16654         special(SPC_NOMOD)
16655         prompt("pprocessNotifyRecord")
16656     }
16657     field(SPVT, DBF_NOACCESS) {
16658         interest(4)
16659         extra("struct scan_element *spvt")
16660         special(SPC_NOMOD)
16661         prompt("Scan Private")
16662     }
16663     field(RSET, DBF_NOACCESS) {
16664         interest(4)
16665         extra("struct typed_rset   *rset")
16666         special(SPC_NOMOD)
16667         prompt("Address of RSET")
16668     }
16669     field(DSET, DBF_NOACCESS) {
16670         interest(4)
16671         extra("unambiguous_dset    *dset")
16672         special(SPC_NOMOD)
16673         prompt("DSET address")
16674     }
16675     field(DPVT, DBF_NOACCESS) {
16676         interest(4)
16677         extra("void                *dpvt")
16678         special(SPC_NOMOD)
16679         prompt("Device Private")
16680     }
16681     field(RDES, DBF_NOACCESS) {
16682         interest(4)
16683         extra("struct dbRecordType *rdes")
16684         special(SPC_NOMOD)
16685         prompt("Address of dbRecordType")
16686     }
16687     field(LSET, DBF_NOACCESS) {
16688         interest(4)
16689         extra("struct lockRecord   *lset")
16690         special(SPC_NOMOD)
16691         prompt("Lock Set")
16692     }
16693     field(PRIO, DBF_MENU) {
16694         promptgroup("20 - Scan")
16695         interest(1)
16696         menu(menuPriority)
16697         special(SPC_SCAN)
16698         prompt("Scheduling Priority")
16699     }
16700     field(TPRO, DBF_UCHAR) {
16701         prompt("Trace Processing")
16702     }
16703     field(BKPT, DBF_NOACCESS) {
16704         interest(1)
16705         extra("epicsUInt8          bkpt")
16706         special(SPC_NOMOD)
16707         prompt("Break Point")
16708     }
16709     field(UDF, DBF_UCHAR) {
16710         promptgroup("10 - Common")
16711         interest(1)
16712         initial("1")
16713         pp(TRUE)
16714         prompt("Undefined")
16715     }
```

```
16716     field(UDFS, DBF_MENU) {
16717         promptgroup("70 - Alarm")
16718         interest(1)
16719         initial("INVALID")
16720         menu(menuAlarmSevr)
16721         prompt("Undefined Alarm Sevrty")
16722     }
16723     field(TIME, DBF_NOACCESS) {
16724         interest(2)
16725         extra("epicsTimeStamp      time")
16726         special(SPC_NOMOD)
16727         prompt("Time")
16728     }
16729     field(UTAG, DBF_UINT64) {
16730         interest(3)
16731         special(SPC_NOMOD)
16732         prompt("Time Tag")
16733     }
16734     field(FLNK, DBF_FWDLINK) {
16735         promptgroup("20 - Scan")
16736         interest(1)
16737         prompt("Forward Process Link")
16738     }
16739     field(VAL, DBF_STRING) {
16740         size(20)
16741         promptgroup("40 - Input")
16742         asl(ASL0)
16743         pp(TRUE)
16744         prompt("Value")
16745     }
16746     field(OVAL, DBF_STRING) {
16747         size(20)
16748         interest(3)
16749         special(SPC_NOMOD)
16750         prompt("Prev Value")
16751     }
16752 }
16753 recordtype(stringin) {
16754     %#include "epicsTypes.h"
16755     %#include "link.h"
16756     %#include "epicsMutex.h"
16757     %#include "ellLib.h"
16758     %#include "devSup.h"
16759     %#include "epicsTime.h"
16760     %
16761     %/* Declare Device Support Entry Table */
16762     %struct stringinRecord;
16763     %typedef struct stringindset {
16764     %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
16765     %    long (*read_stringin)(struct stringinRecord *prec); /*returns: (-1,0)=>(failure,success)*/
16766     %} stringindset;
16767     %#define HAS_stringindset
16768     %
16769     %#include "callback.h"
16770     field(NAME, DBF_STRING) {
16771         size(61)
16772         special(SPC_NOMOD)
16773         prompt("Record Name")
16774     }
16775     field(DESC, DBF_STRING) {
16776         size(41)
16777         promptgroup("10 - Common")
16778         prompt("Descriptor")
16779     }
16780     field(ASG, DBF_STRING) {
16781         size(29)
16782         promptgroup("10 - Common")
16783         special(SPC_AS)
```

```
16784         prompt("Access Security Group")
16785     }
16786     field(SCAN, DBF_MENU) {
16787         promptgroup("20 - Scan")
16788         interest(1)
16789         menu(menuScan)
16790         special(SPC_SCAN)
16791         prompt("Scan Mechanism")
16792     }
16793     field(PINI, DBF_MENU) {
16794         promptgroup("20 - Scan")
16795         interest(1)
16796         menu(menuPini)
16797         prompt("Process at iocInit")
16798     }
16799     field(PHAS, DBF_SHORT) {
16800         promptgroup("20 - Scan")
16801         interest(1)
16802         special(SPC_SCAN)
16803         prompt("Scan Phase")
16804     }
16805     field(EVNT, DBF_STRING) {
16806         size(40)
16807         promptgroup("20 - Scan")
16808         interest(1)
16809         special(SPC_SCAN)
16810         prompt("Event Name")
16811     }
16812     field(TSE, DBF_SHORT) {
16813         promptgroup("20 - Scan")
16814         interest(1)
16815         prompt("Time Stamp Event")
16816     }
16817     field(TSEL, DBF_INLINK) {
16818         promptgroup("20 - Scan")
16819         interest(1)
16820         prompt("Time Stamp Link")
16821     }
16822     field(DTYP, DBF_DEVICE) {
16823         promptgroup("10 - Common")
16824         interest(1)
16825         prompt("Device Type")
16826     }
16827     field(DISV, DBF_SHORT) {
16828         promptgroup("20 - Scan")
16829         initial("1")
16830         prompt("Disable Value")
16831     }
16832     field(DISA, DBF_SHORT) {
16833         prompt("Disable")
16834     }
16835     field(SDIS, DBF_INLINK) {
16836         promptgroup("20 - Scan")
16837         interest(1)
16838         prompt("Scanning Disable")
16839     }
16840     field(MLOK, DBF_NOACCESS) {
16841         interest(4)
16842         extra("epicsMutexId        mlok")
16843         special(SPC_NOMOD)
16844         prompt("Monitor lock")
16845     }
16846     field(MLIS, DBF_NOACCESS) {
16847         interest(4)
16848         extra("ELLLIST             mlis")
16849         special(SPC_NOMOD)
16850         prompt("Monitor List")
16851     }
```

```
16852     field(BKLNK, DBF_NOACCESS) {
16853         interest(4)
16854         extra("ELLLIST             bklnk")
16855         special(SPC_NOMOD)
16856         prompt("Backwards link tracking")
16857     }
16858     field(DISP, DBF_UCHAR) {
16859         promptgroup("10 - Common")
16860         interest(1)
16861         prompt("Disable putField")
16862     }
16863     field(PROC, DBF_UCHAR) {
16864         interest(3)
16865         pp(TRUE)
16866         prompt("Force Processing")
16867     }
16868     field(STAT, DBF_MENU) {
16869         initial("UDF")
16870         menu(menuAlarmStat)
16871         special(SPC_NOMOD)
16872         prompt("Alarm Status")
16873     }
16874     field(SEVR, DBF_MENU) {
16875         menu(menuAlarmSevr)
16876         special(SPC_NOMOD)
16877         prompt("Alarm Severity")
16878     }
16879     field(AMSG, DBF_STRING) {
16880         size(40)
16881         special(SPC_NOMOD)
16882         prompt("Alarm Message")
16883     }
16884     field(NSTA, DBF_MENU) {
16885         interest(2)
16886         menu(menuAlarmStat)
16887         special(SPC_NOMOD)
16888         prompt("New Alarm Status")
16889     }
16890     field(NSEV, DBF_MENU) {
16891         interest(2)
16892         menu(menuAlarmSevr)
16893         special(SPC_NOMOD)
16894         prompt("New Alarm Severity")
16895     }
16896     field(NAMSG, DBF_STRING) {
16897         size(40)
16898         special(SPC_NOMOD)
16899         prompt("New Alarm Message")
16900     }
16901     field(ACKS, DBF_MENU) {
16902         interest(2)
16903         menu(menuAlarmSevr)
16904         special(SPC_NOMOD)
16905         prompt("Alarm Ack Severity")
16906     }
16907     field(ACKT, DBF_MENU) {
16908         promptgroup("70 - Alarm")
16909         interest(2)
16910         initial("YES")
16911         menu(menuYesNo)
16912         special(SPC_NOMOD)
16913         prompt("Alarm Ack Transient")
16914     }
16915     field(DISS, DBF_MENU) {
16916         promptgroup("70 - Alarm")
16917         interest(1)
16918         menu(menuAlarmSevr)
16919         prompt("Disable Alarm Sevrty")
```

```
16920     }
16921     field(LCNT, DBF_UCHAR) {
16922         interest(2)
16923         special(SPC_NOMOD)
16924         prompt("Lock Count")
16925     }
16926     field(PACT, DBF_UCHAR) {
16927         interest(1)
16928         special(SPC_NOMOD)
16929         prompt("Record active")
16930     }
16931     field(PUTF, DBF_UCHAR) {
16932         interest(1)
16933         special(SPC_NOMOD)
16934         prompt("dbPutField process")
16935     }
16936     field(RPRO, DBF_UCHAR) {
16937         interest(1)
16938         special(SPC_NOMOD)
16939         prompt("Reprocess ")
16940     }
16941     field(ASP, DBF_NOACCESS) {
16942         interest(4)
16943         extra("struct asgMember    *asp")
16944         special(SPC_NOMOD)
16945         prompt("Access Security Pvt")
16946     }
16947     field(PPN, DBF_NOACCESS) {
16948         interest(4)
16949         extra("struct processNotify *ppn")
16950         special(SPC_NOMOD)
16951         prompt("pprocessNotify")
16952     }
16953     field(PPNR, DBF_NOACCESS) {
16954         interest(4)
16955         extra("struct processNotifyRecord *ppnr")
16956         special(SPC_NOMOD)
16957         prompt("pprocessNotifyRecord")
16958     }
16959     field(SPVT, DBF_NOACCESS) {
16960         interest(4)
16961         extra("struct scan_element *spvt")
16962         special(SPC_NOMOD)
16963         prompt("Scan Private")
16964     }
16965     field(RSET, DBF_NOACCESS) {
16966         interest(4)
16967         extra("struct typed_rset   *rset")
16968         special(SPC_NOMOD)
16969         prompt("Address of RSET")
16970     }
16971     field(DSET, DBF_NOACCESS) {
16972         interest(4)
16973         extra("unambiguous_dset    *dset")
16974         special(SPC_NOMOD)
16975         prompt("DSET address")
16976     }
16977     field(DPVT, DBF_NOACCESS) {
16978         interest(4)
16979         extra("void                *dpvt")
16980         special(SPC_NOMOD)
16981         prompt("Device Private")
16982     }
16983     field(RDES, DBF_NOACCESS) {
16984         interest(4)
16985         extra("struct dbRecordType *rdes")
16986         special(SPC_NOMOD)
16987         prompt("Address of dbRecordType")
```

```
16988     }
16989     field(LSET, DBF_NOACCESS) {
16990         interest(4)
16991         extra("struct lockRecord   *lset")
16992         special(SPC_NOMOD)
16993         prompt("Lock Set")
16994     }
16995     field(PRIO, DBF_MENU) {
16996         promptgroup("20 - Scan")
16997         interest(1)
16998         menu(menuPriority)
16999         special(SPC_SCAN)
17000         prompt("Scheduling Priority")
17001     }
17002     field(TPRO, DBF_UCHAR) {
17003         prompt("Trace Processing")
17004     }
17005     field(BKPT, DBF_NOACCESS) {
17006         interest(1)
17007         extra("epicsUInt8          bkpt")
17008         special(SPC_NOMOD)
17009         prompt("Break Point")
17010     }
17011     field(UDF, DBF_UCHAR) {
17012         promptgroup("10 - Common")
17013         interest(1)
17014         initial("1")
17015         pp(TRUE)
17016         prompt("Undefined")
17017     }
17018     field(UDFS, DBF_MENU) {
17019         promptgroup("70 - Alarm")
17020         interest(1)
17021         initial("INVALID")
17022         menu(menuAlarmSevr)
17023         prompt("Undefined Alarm Sevrty")
17024     }
17025     field(TIME, DBF_NOACCESS) {
17026         interest(2)
17027         extra("epicsTimeStamp      time")
17028         special(SPC_NOMOD)
17029         prompt("Time")
17030     }
17031     field(UTAG, DBF_UINT64) {
17032         interest(3)
17033         special(SPC_NOMOD)
17034         prompt("Time Tag")
17035     }
17036     field(FLNK, DBF_FWDLINK) {
17037         promptgroup("20 - Scan")
17038         interest(1)
17039         prompt("Forward Process Link")
17040     }
17041     field(VAL, DBF_STRING) {
17042         size(40)
17043         promptgroup("40 - Input")
17044         asl(ASL0)
17045         pp(TRUE)
17046         prompt("Current Value")
17047     }
17048     field(OVAL, DBF_STRING) {
17049         size(40)
17050         interest(3)
17051         special(SPC_NOMOD)
17052         prompt("Previous Value")
17053     }
17054     field(INP, DBF_INLINK) {
17055         promptgroup("40 - Input")
```

```
        interest(1)
        prompt("Input Specification")
    }
    field(MPST, DBF_MENU) {
        promptgroup("80 - Display")
        interest(1)
        menu(stringinPOST)
        prompt("Post Value Monitors")
    }
    field(APST, DBF_MENU) {
        promptgroup("80 - Display")
        interest(1)
        menu(stringinPOST)
        prompt("Post Archive Monitors")
    }
    field(SIOL, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Input Link")
    }
    field(SVAL, DBF_STRING) {
        size(40)
        pp(TRUE)
        prompt("Simulation Value")
    }
    field(SIML, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Mode Link")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuYesNo)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
}
```

```
device(stringin, CONSTANT, devSiSoft, "Soft Channel")
device(stringin, CONSTANT, devSiSoftCallback, "Async Soft Channel")
device(stringin, INST_IO, devTimestampSI, "Soft Timestamp")
device(stringin, INST_IO, devSiGeneralTime, "General Time")
device(stringin, INST_IO, devSiEnviron, "getenv")
device(stringin, INST_IO, devPSCMessageSI, "PSC Ctrl Message")
device(stringin, INST_IO, devPSCBlockSi, "PSC Block")
recordtype(stringout) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct stringoutRecord;
    %typedef struct stringoutdset {
    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
    %    long (*write_stringout)(struct stringoutRecord *prec); /*(-1,0)=>(failure,success)*/
    %} stringoutdset;
    %#define HAS_stringoutdset
    %
    %#include "callback.h"
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuScan)
        special(SPC_SCAN)
        prompt("Scan Mechanism")
    }
    field(PINI, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPini)
        prompt("Process at iocInit")
    }
    field(PHAS, DBF_SHORT) {
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Scan Phase")
    }
    field(EVNT, DBF_STRING) {
        size(40)
        promptgroup("20 - Scan")
        interest(1)
        special(SPC_SCAN)
        prompt("Event Name")
    }
    field(TSE, DBF_SHORT) {
        promptgroup("20 - Scan")
```

```
17192         interest(1)
17193         prompt("Time Stamp Event")
17194     }
17195     field(TSEL, DBF_INLINK) {
17196         promptgroup("20 - Scan")
17197         interest(1)
17198         prompt("Time Stamp Link")
17199     }
17200     field(DTYP, DBF_DEVICE) {
17201         promptgroup("10 - Common")
17202         interest(1)
17203         prompt("Device Type")
17204     }
17205     field(DISV, DBF_SHORT) {
17206         promptgroup("20 - Scan")
17207         initial("1")
17208         prompt("Disable Value")
17209     }
17210     field(DISA, DBF_SHORT) {
17211         prompt("Disable")
17212     }
17213     field(SDIS, DBF_INLINK) {
17214         promptgroup("20 - Scan")
17215         interest(1)
17216         prompt("Scanning Disable")
17217     }
17218     field(MLOK, DBF_NOACCESS) {
17219         interest(4)
17220         extra("epicsMutexId        mlok")
17221         special(SPC_NOMOD)
17222         prompt("Monitor lock")
17223     }
17224     field(MLIS, DBF_NOACCESS) {
17225         interest(4)
17226         extra("ELLLIST             mlis")
17227         special(SPC_NOMOD)
17228         prompt("Monitor List")
17229     }
17230     field(BKLNK, DBF_NOACCESS) {
17231         interest(4)
17232         extra("ELLLIST             bklnk")
17233         special(SPC_NOMOD)
17234         prompt("Backwards link tracking")
17235     }
17236     field(DISP, DBF_UCHAR) {
17237         promptgroup("10 - Common")
17238         interest(1)
17239         prompt("Disable putField")
17240     }
17241     field(PROC, DBF_UCHAR) {
17242         interest(3)
17243         pp(TRUE)
17244         prompt("Force Processing")
17245     }
17246     field(STAT, DBF_MENU) {
17247         initial("UDF")
17248         menu(menuAlarmStat)
17249         special(SPC_NOMOD)
17250         prompt("Alarm Status")
17251     }
17252     field(SEVR, DBF_MENU) {
17253         menu(menuAlarmSevr)
17254         special(SPC_NOMOD)
17255         prompt("Alarm Severity")
17256     }
17257     field(AMSG, DBF_STRING) {
17258         size(40)
17259         special(SPC_NOMOD)
```

```
17260         prompt("Alarm Message")
17261     }
17262     field(NSTA, DBF_MENU) {
17263         interest(2)
17264         menu(menuAlarmStat)
17265         special(SPC_NOMOD)
17266         prompt("New Alarm Status")
17267     }
17268     field(NSEV, DBF_MENU) {
17269         interest(2)
17270         menu(menuAlarmSevr)
17271         special(SPC_NOMOD)
17272         prompt("New Alarm Severity")
17273     }
17274     field(NAMSG, DBF_STRING) {
17275         size(40)
17276         special(SPC_NOMOD)
17277         prompt("New Alarm Message")
17278     }
17279     field(ACKS, DBF_MENU) {
17280         interest(2)
17281         menu(menuAlarmSevr)
17282         special(SPC_NOMOD)
17283         prompt("Alarm Ack Severity")
17284     }
17285     field(ACKT, DBF_MENU) {
17286         promptgroup("70 - Alarm")
17287         interest(2)
17288         initial("YES")
17289         menu(menuYesNo)
17290         special(SPC_NOMOD)
17291         prompt("Alarm Ack Transient")
17292     }
17293     field(DISS, DBF_MENU) {
17294         promptgroup("70 - Alarm")
17295         interest(1)
17296         menu(menuAlarmSevr)
17297         prompt("Disable Alarm Sevrty")
17298     }
17299     field(LCNT, DBF_UCHAR) {
17300         interest(2)
17301         special(SPC_NOMOD)
17302         prompt("Lock Count")
17303     }
17304     field(PACT, DBF_UCHAR) {
17305         interest(1)
17306         special(SPC_NOMOD)
17307         prompt("Record active")
17308     }
17309     field(PUTF, DBF_UCHAR) {
17310         interest(1)
17311         special(SPC_NOMOD)
17312         prompt("dbPutField process")
17313     }
17314     field(RPRO, DBF_UCHAR) {
17315         interest(1)
17316         special(SPC_NOMOD)
17317         prompt("Reprocess ")
17318     }
17319     field(ASP, DBF_NOACCESS) {
17320         interest(4)
17321         extra("struct asgMember    *asp")
17322         special(SPC_NOMOD)
17323         prompt("Access Security Pvt")
17324     }
17325     field(PPN, DBF_NOACCESS) {
17326         interest(4)
17327         extra("struct processNotify *ppn")
```

```
17328         special(SPC_NOMOD)
17329         prompt("pprocessNotify")
17330     }
17331     field(PPNR, DBF_NOACCESS) {
17332         interest(4)
17333         extra("struct processNotifyRecord *ppnr")
17334         special(SPC_NOMOD)
17335         prompt("pprocessNotifyRecord")
17336     }
17337     field(SPVT, DBF_NOACCESS) {
17338         interest(4)
17339         extra("struct scan_element *spvt")
17340         special(SPC_NOMOD)
17341         prompt("Scan Private")
17342     }
17343     field(RSET, DBF_NOACCESS) {
17344         interest(4)
17345         extra("struct typed_rset   *rset")
17346         special(SPC_NOMOD)
17347         prompt("Address of RSET")
17348     }
17349     field(DSET, DBF_NOACCESS) {
17350         interest(4)
17351         extra("unambiguous_dset    *dset")
17352         special(SPC_NOMOD)
17353         prompt("DSET address")
17354     }
17355     field(DPVT, DBF_NOACCESS) {
17356         interest(4)
17357         extra("void                *dpvt")
17358         special(SPC_NOMOD)
17359         prompt("Device Private")
17360     }
17361     field(RDES, DBF_NOACCESS) {
17362         interest(4)
17363         extra("struct dbRecordType *rdes")
17364         special(SPC_NOMOD)
17365         prompt("Address of dbRecordType")
17366     }
17367     field(LSET, DBF_NOACCESS) {
17368         interest(4)
17369         extra("struct lockRecord   *lset")
17370         special(SPC_NOMOD)
17371         prompt("Lock Set")
17372     }
17373     field(PRIO, DBF_MENU) {
17374         promptgroup("20 - Scan")
17375         interest(1)
17376         menu(menuPriority)
17377         special(SPC_SCAN)
17378         prompt("Scheduling Priority")
17379     }
17380     field(TPRO, DBF_UCHAR) {
17381         prompt("Trace Processing")
17382     }
17383     field(BKPT, DBF_NOACCESS) {
17384         interest(1)
17385         extra("epicsUInt8          bkpt")
17386         special(SPC_NOMOD)
17387         prompt("Break Point")
17388     }
17389     field(UDF, DBF_UCHAR) {
17390         promptgroup("10 - Common")
17391         interest(1)
17392         initial("1")
17393         pp(TRUE)
17394         prompt("Undefined")
17395     }
```

```
17396     field(UDFS, DBF_MENU) {
17397         promptgroup("70 - Alarm")
17398         interest(1)
17399         initial("INVALID")
17400         menu(menuAlarmSevr)
17401         prompt("Undefined Alarm Sevrty")
17402     }
17403     field(TIME, DBF_NOACCESS) {
17404         interest(2)
17405         extra("epicsTimeStamp      time")
17406         special(SPC_NOMOD)
17407         prompt("Time")
17408     }
17409     field(UTAG, DBF_UINT64) {
17410         interest(3)
17411         special(SPC_NOMOD)
17412         prompt("Time Tag")
17413     }
17414     field(FLNK, DBF_FWDLINK) {
17415         promptgroup("20 - Scan")
17416         interest(1)
17417         prompt("Forward Process Link")
17418     }
17419     field(VAL, DBF_STRING) {
17420         size(40)
17421         promptgroup("50 - Output")
17422         asl(ASL0)
17423         pp(TRUE)
17424         prompt("Current Value")
17425     }
17426     field(OVAL, DBF_STRING) {
17427         size(40)
17428         interest(3)
17429         special(SPC_NOMOD)
17430         prompt("Previous Value")
17431     }
17432     field(DOL, DBF_INLINK) {
17433         promptgroup("40 - Input")
17434         interest(1)
17435         prompt("Desired Output Link")
17436     }
17437     field(OMSL, DBF_MENU) {
17438         promptgroup("50 - Output")
17439         interest(1)
17440         menu(menuOmsl)
17441         prompt("Output Mode Select")
17442     }
17443     field(OUT, DBF_OUTLINK) {
17444         promptgroup("50 - Output")
17445         interest(1)
17446         prompt("Output Specification")
17447     }
17448     field(MPST, DBF_MENU) {
17449         promptgroup("80 - Display")
17450         interest(1)
17451         menu(stringoutPOST)
17452         prompt("Post Value Monitors")
17453     }
17454     field(APST, DBF_MENU) {
17455         promptgroup("80 - Display")
17456         interest(1)
17457         menu(stringoutPOST)
17458         prompt("Post Archive Monitors")
17459     }
17460     field(SIOL, DBF_OUTLINK) {
17461         promptgroup("90 - Simulate")
17462         interest(1)
17463         prompt("Simulation Output Link")
```

```
    }
    field(SIML, DBF_INLINK) {
        promptgroup("90 - Simulate")
        interest(1)
        prompt("Simulation Mode Link")
    }
    field(SIMM, DBF_MENU) {
        interest(1)
        menu(menuYesNo)
        special(SPC_MOD)
        prompt("Simulation Mode")
    }
    field(SIMS, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(2)
        menu(menuAlarmSevr)
        prompt("Simulation Mode Severity")
    }
    field(OLDSIMM, DBF_MENU) {
        interest(4)
        menu(menuSimm)
        special(SPC_NOMOD)
        prompt("Prev. Simulation Mode")
    }
    field(SSCN, DBF_MENU) {
        promptgroup("90 - Simulate")
        interest(1)
        initial("65535")
        menu(menuScan)
        prompt("Sim. Mode Scan")
    }
    field(SDLY, DBF_DOUBLE) {
        promptgroup("90 - Simulate")
        interest(2)
        initial("-1.0")
        prompt("Sim. Mode Async Delay")
    }
    field(SIMPVT, DBF_NOACCESS) {
        interest(4)
        extra("epicsCallback            *simpvt")
        special(SPC_NOMOD)
        prompt("Sim. Mode Private")
    }
    field(IVOA, DBF_MENU) {
        promptgroup("50 - Output")
        interest(2)
        menu(menuIvoa)
        prompt("INVALID output action")
    }
    field(IVOV, DBF_STRING) {
        size(40)
        promptgroup("50 - Output")
        interest(2)
        prompt("INVALID output value")
    }
}
device(stringout, CONSTANT, devSoSoft, "Soft Channel")
device(stringout, CONSTANT, devSoSoftCallback, "Async Soft Channel")
device(stringout, INST_IO, devSoStdio, "stdio")
device(stringout, INST_IO, devPSCBlockSo, "PSC Block")
recordtype(sub) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %struct subRecord;
```

```
17532     %typedef long (*SUBFUNCPTR)(struct subRecord *);
17533     field(NAME, DBF_STRING) {
17534         size(61)
17535         special(SPC_NOMOD)
17536         prompt("Record Name")
17537     }
17538     field(DESC, DBF_STRING) {
17539         size(41)
17540         promptgroup("10 - Common")
17541         prompt("Descriptor")
17542     }
17543     field(ASG, DBF_STRING) {
17544         size(29)
17545         promptgroup("10 - Common")
17546         special(SPC_AS)
17547         prompt("Access Security Group")
17548     }
17549     field(SCAN, DBF_MENU) {
17550         promptgroup("20 - Scan")
17551         interest(1)
17552         menu(menuScan)
17553         special(SPC_SCAN)
17554         prompt("Scan Mechanism")
17555     }
17556     field(PINI, DBF_MENU) {
17557         promptgroup("20 - Scan")
17558         interest(1)
17559         menu(menuPini)
17560         prompt("Process at iocInit")
17561     }
17562     field(PHAS, DBF_SHORT) {
17563         promptgroup("20 - Scan")
17564         interest(1)
17565         special(SPC_SCAN)
17566         prompt("Scan Phase")
17567     }
17568     field(EVNT, DBF_STRING) {
17569         size(40)
17570         promptgroup("20 - Scan")
17571         interest(1)
17572         special(SPC_SCAN)
17573         prompt("Event Name")
17574     }
17575     field(TSE, DBF_SHORT) {
17576         promptgroup("20 - Scan")
17577         interest(1)
17578         prompt("Time Stamp Event")
17579     }
17580     field(TSEL, DBF_INLINK) {
17581         promptgroup("20 - Scan")
17582         interest(1)
17583         prompt("Time Stamp Link")
17584     }
17585     field(DTYP, DBF_DEVICE) {
17586         promptgroup("10 - Common")
17587         interest(1)
17588         prompt("Device Type")
17589     }
17590     field(DISV, DBF_SHORT) {
17591         promptgroup("20 - Scan")
17592         initial("1")
17593         prompt("Disable Value")
17594     }
17595     field(DISA, DBF_SHORT) {
17596         prompt("Disable")
17597     }
17598     field(SDIS, DBF_INLINK) {
17599         promptgroup("20 - Scan")
```

```
        interest(1)
        prompt("Scanning Disable")
    }
    field(MLOK, DBF_NOACCESS) {
        interest(4)
        extra("epicsMutexId        mlok")
        special(SPC_NOMOD)
        prompt("Monitor lock")
    }
    field(MLIS, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             mlis")
        special(SPC_NOMOD)
        prompt("Monitor List")
    }
    field(BKLNK, DBF_NOACCESS) {
        interest(4)
        extra("ELLLIST             bklnk")
        special(SPC_NOMOD)
        prompt("Backwards link tracking")
    }
    field(DISP, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        prompt("Disable putField")
    }
    field(PROC, DBF_UCHAR) {
        interest(3)
        pp(TRUE)
        prompt("Force Processing")
    }
    field(STAT, DBF_MENU) {
        initial("UDF")
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("Alarm Status")
    }
    field(SEVR, DBF_MENU) {
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("Alarm Severity")
    }
    field(AMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("Alarm Message")
    }
    field(NSTA, DBF_MENU) {
        interest(2)
        menu(menuAlarmStat)
        special(SPC_NOMOD)
        prompt("New Alarm Status")
    }
    field(NSEV, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
        prompt("New Alarm Severity")
    }
    field(NAMSG, DBF_STRING) {
        size(40)
        special(SPC_NOMOD)
        prompt("New Alarm Message")
    }
    field(ACKS, DBF_MENU) {
        interest(2)
        menu(menuAlarmSevr)
        special(SPC_NOMOD)
```

```
17668         prompt("Alarm Ack Severity")
17669     }
17670     field(ACKT, DBF_MENU) {
17671         promptgroup("70 - Alarm")
17672         interest(2)
17673         initial("YES")
17674         menu(menuYesNo)
17675         special(SPC_NOMOD)
17676         prompt("Alarm Ack Transient")
17677     }
17678     field(DISS, DBF_MENU) {
17679         promptgroup("70 - Alarm")
17680         interest(1)
17681         menu(menuAlarmSevr)
17682         prompt("Disable Alarm Sevrty")
17683     }
17684     field(LCNT, DBF_UCHAR) {
17685         interest(2)
17686         special(SPC_NOMOD)
17687         prompt("Lock Count")
17688     }
17689     field(PACT, DBF_UCHAR) {
17690         interest(1)
17691         special(SPC_NOMOD)
17692         prompt("Record active")
17693     }
17694     field(PUTF, DBF_UCHAR) {
17695         interest(1)
17696         special(SPC_NOMOD)
17697         prompt("dbPutField process")
17698     }
17699     field(RPRO, DBF_UCHAR) {
17700         interest(1)
17701         special(SPC_NOMOD)
17702         prompt("Reprocess ")
17703     }
17704     field(ASP, DBF_NOACCESS) {
17705         interest(4)
17706         extra("struct asgMember    *asp")
17707         special(SPC_NOMOD)
17708         prompt("Access Security Pvt")
17709     }
17710     field(PPN, DBF_NOACCESS) {
17711         interest(4)
17712         extra("struct processNotify *ppn")
17713         special(SPC_NOMOD)
17714         prompt("pprocessNotify")
17715     }
17716     field(PPNR, DBF_NOACCESS) {
17717         interest(4)
17718         extra("struct processNotifyRecord *ppnr")
17719         special(SPC_NOMOD)
17720         prompt("pprocessNotifyRecord")
17721     }
17722     field(SPVT, DBF_NOACCESS) {
17723         interest(4)
17724         extra("struct scan_element *spvt")
17725         special(SPC_NOMOD)
17726         prompt("Scan Private")
17727     }
17728     field(RSET, DBF_NOACCESS) {
17729         interest(4)
17730         extra("struct typed_rset   *rset")
17731         special(SPC_NOMOD)
17732         prompt("Address of RSET")
17733     }
17734     field(DSET, DBF_NOACCESS) {
17735         interest(4)
```

```
17736         extra("unambiguous_dset    *dset")
17737         special(SPC_NOMOD)
17738         prompt("DSET address")
17739     }
17740     field(DPVT, DBF_NOACCESS) {
17741         interest(4)
17742         extra("void                *dpvt")
17743         special(SPC_NOMOD)
17744         prompt("Device Private")
17745     }
17746     field(RDES, DBF_NOACCESS) {
17747         interest(4)
17748         extra("struct dbRecordType *rdes")
17749         special(SPC_NOMOD)
17750         prompt("Address of dbRecordType")
17751     }
17752     field(LSET, DBF_NOACCESS) {
17753         interest(4)
17754         extra("struct lockRecord   *lset")
17755         special(SPC_NOMOD)
17756         prompt("Lock Set")
17757     }
17758     field(PRIO, DBF_MENU) {
17759         promptgroup("20 - Scan")
17760         interest(1)
17761         menu(menuPriority)
17762         special(SPC_SCAN)
17763         prompt("Scheduling Priority")
17764     }
17765     field(TPRO, DBF_UCHAR) {
17766         prompt("Trace Processing")
17767     }
17768     field(BKPT, DBF_NOACCESS) {
17769         interest(1)
17770         extra("epicsUInt8          bkpt")
17771         special(SPC_NOMOD)
17772         prompt("Break Point")
17773     }
17774     field(UDF, DBF_UCHAR) {
17775         promptgroup("10 - Common")
17776         interest(1)
17777         initial("1")
17778         pp(TRUE)
17779         prompt("Undefined")
17780     }
17781     field(UDFS, DBF_MENU) {
17782         promptgroup("70 - Alarm")
17783         interest(1)
17784         initial("INVALID")
17785         menu(menuAlarmSevr)
17786         prompt("Undefined Alarm Sevrty")
17787     }
17788     field(TIME, DBF_NOACCESS) {
17789         interest(2)
17790         extra("epicsTimeStamp      time")
17791         special(SPC_NOMOD)
17792         prompt("Time")
17793     }
17794     field(UTAG, DBF_UINT64) {
17795         interest(3)
17796         special(SPC_NOMOD)
17797         prompt("Time Tag")
17798     }
17799     field(FLNK, DBF_FWDLINK) {
17800         promptgroup("20 - Scan")
17801         interest(1)
17802         prompt("Forward Process Link")
17803     }
```

```
    field(VAL, DBF_DOUBLE) {
        asl(ASL0)
        pp(TRUE)
        prompt("Result")
    }
    field(INAM, DBF_STRING) {
        size(40)
        promptgroup("30 - Action")
        interest(1)
        special(SPC_NOMOD)
        prompt("Init Routine Name")
    }
    field(SNAM, DBF_STRING) {
        size(40)
        promptgroup("30 - Action")
        interest(1)
        special(SPC_MOD)
        prompt("Subroutine Name")
    }
    field(SADR, DBF_NOACCESS) {
        interest(4)
        extra("SUBFUNCPTR sadr")
        special(SPC_NOMOD)
        prompt("Subroutine Address")
    }
    field(INPA, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input A")
    }
    field(INPB, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input B")
    }
    field(INPC, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input C")
    }
    field(INPD, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input D")
    }
    field(INPE, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input E")
    }
    field(INPF, DBF_INLINK) {
        promptgroup("41 - Input A-F")
        interest(1)
        prompt("Input F")
    }
    field(INPG, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input G")
    }
    field(INPH, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input H")
    }
    field(INPI, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
```

```
        prompt("Input I")
    }
    field(INPJ, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input J")
    }
    field(INPK, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input K")
    }
    field(INPL, DBF_INLINK) {
        promptgroup("42 - Input G-L")
        interest(1)
        prompt("Input L")
    }
    field(INPM, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input M")
    }
    field(INPN, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input N")
    }
    field(INPO, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input O")
    }
    field(INPP, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input P")
    }
    field(INPQ, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input Q")
    }
    field(INPR, DBF_INLINK) {
        promptgroup("43 - Input M-R")
        interest(1)
        prompt("Input R")
    }
    field(INPS, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input S")
    }
    field(INPT, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input T")
    }
    field(INPU, DBF_INLINK) {
        promptgroup("44 - Input S-U")
        interest(1)
        prompt("Input U")
    }
    field(EGU, DBF_STRING) {
        size(16)
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Engineering Units")
```

```
    }
    field(HOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("High Operating Range")
    }
    field(LOPR, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Low Operating Range")
    }
    field(HIHI, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Hihi Alarm Limit")
    }
    field(LOLO, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Lolo Alarm Limit")
    }
    field(HIGH, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("High Alarm Limit")
    }
    field(LOW, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        pp(TRUE)
        prop(YES)
        prompt("Low Alarm Limit")
    }
    field(PREC, DBF_SHORT) {
        promptgroup("80 - Display")
        interest(1)
        prop(YES)
        prompt("Display Precision")
    }
    field(BRSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        menu(menuAlarmSevr)
        pp(TRUE)
        prompt("Bad Return Severity")
    }
    field(HHSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Hihi Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LLSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Lolo Severity")
        prop(YES)
        interest(1)
```

```
        pp(TRUE)
    }
    field(HSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("High Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(LSV, DBF_MENU) {
        promptgroup("70 - Alarm")
        menu(menuAlarmSevr)
        prompt("Low Severity")
        prop(YES)
        interest(1)
        pp(TRUE)
    }
    field(HYST, DBF_DOUBLE) {
        promptgroup("70 - Alarm")
        interest(1)
        prompt("Alarm Deadband")
    }
    field(ADEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Archive Deadband")
    }
    field(MDEL, DBF_DOUBLE) {
        promptgroup("80 - Display")
        interest(1)
        prompt("Monitor Deadband")
    }
    field(A, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input A")
    }
    field(B, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input B")
    }
    field(C, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input C")
    }
    field(D, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input D")
    }
    field(E, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input E")
    }
    field(F, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input F")
    }
    field(G, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input G")
    }
    field(H, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input H")
    }
    field(I, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input I")
```

```
    }
    field(J, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input J")
    }
    field(K, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input K")
    }
    field(L, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input L")
    }
    field(M, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input M")
    }
    field(N, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input N")
    }
    field(O, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input O")
    }
    field(P, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input P")
    }
    field(Q, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input Q")
    }
    field(R, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input R")
    }
    field(S, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input S")
    }
    field(T, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input T")
    }
    field(U, DBF_DOUBLE) {
        pp(TRUE)
        prompt("Value of Input U")
    }
    field(LA, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of A")
    }
    field(LB, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of B")
    }
    field(LC, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of C")
    }
    field(LD, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of D")
```

```
    }
    field(LE, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of E")
    }
    field(LF, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of F")
    }
    field(LG, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of G")
    }
    field(LH, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of H")
    }
    field(LI, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of I")
    }
    field(LJ, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of J")
    }
    field(LK, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of K")
    }
    field(LL, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of L")
    }
    field(LM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of M")
    }
    field(LN, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of N")
    }
    field(LO, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of O")
    }
    field(LP, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of P")
    }
    field(LQ, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of Q")
    }
    field(LR, DBF_DOUBLE) {
        interest(3)
```

```
        special(SPC_NOMOD)
        prompt("Prev Value of R")
    }
    field(LS, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of S")
    }
    field(LT, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of T")
    }
    field(LU, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Prev Value of U")
    }
    field(LALM, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Alarmed")
    }
    field(ALST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Archived")
    }
    field(MLST, DBF_DOUBLE) {
        interest(3)
        special(SPC_NOMOD)
        prompt("Last Value Monitored")
    }
}
recordtype(subArray) {
    %#include "epicsTypes.h"
    %#include "link.h"
    %#include "epicsMutex.h"
    %#include "ellLib.h"
    %#include "devSup.h"
    %#include "epicsTime.h"
    %
    %/* Declare Device Support Entry Table */
    %struct subArrayRecord;
    %typedef struct sadset {
    %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
    %    long (*read_sa)(struct subArrayRecord *prec); /*returns: (-1,0)=>(failure,success)*/
    %} sadset;
    %#define HAS_sadset
    %#define SA_POSTS_NORD
    %
    field(NAME, DBF_STRING) {
        size(61)
        special(SPC_NOMOD)
        prompt("Record Name")
    }
    field(DESC, DBF_STRING) {
        size(41)
        promptgroup("10 - Common")
        prompt("Descriptor")
    }
    field(ASG, DBF_STRING) {
        size(29)
        promptgroup("10 - Common")
        special(SPC_AS)
        prompt("Access Security Group")
    }
    field(SCAN, DBF_MENU) {
```

```
18280         promptgroup("20 - Scan")
18281         interest(1)
18282         menu(menuScan)
18283         special(SPC_SCAN)
18284         prompt("Scan Mechanism")
18285     }
18286     field(PINI, DBF_MENU) {
18287         promptgroup("20 - Scan")
18288         interest(1)
18289         menu(menuPini)
18290         prompt("Process at iocInit")
18291     }
18292     field(PHAS, DBF_SHORT) {
18293         promptgroup("20 - Scan")
18294         interest(1)
18295         special(SPC_SCAN)
18296         prompt("Scan Phase")
18297     }
18298     field(EVNT, DBF_STRING) {
18299         size(40)
18300         promptgroup("20 - Scan")
18301         interest(1)
18302         special(SPC_SCAN)
18303         prompt("Event Name")
18304     }
18305     field(TSE, DBF_SHORT) {
18306         promptgroup("20 - Scan")
18307         interest(1)
18308         prompt("Time Stamp Event")
18309     }
18310     field(TSEL, DBF_INLINK) {
18311         promptgroup("20 - Scan")
18312         interest(1)
18313         prompt("Time Stamp Link")
18314     }
18315     field(DTYP, DBF_DEVICE) {
18316         promptgroup("10 - Common")
18317         interest(1)
18318         prompt("Device Type")
18319     }
18320     field(DISV, DBF_SHORT) {
18321         promptgroup("20 - Scan")
18322         initial("1")
18323         prompt("Disable Value")
18324     }
18325     field(DISA, DBF_SHORT) {
18326         prompt("Disable")
18327     }
18328     field(SDIS, DBF_INLINK) {
18329         promptgroup("20 - Scan")
18330         interest(1)
18331         prompt("Scanning Disable")
18332     }
18333     field(MLOK, DBF_NOACCESS) {
18334         interest(4)
18335         extra("epicsMutexId        mlok")
18336         special(SPC_NOMOD)
18337         prompt("Monitor lock")
18338     }
18339     field(MLIS, DBF_NOACCESS) {
18340         interest(4)
18341         extra("ELLLIST             mlis")
18342         special(SPC_NOMOD)
18343         prompt("Monitor List")
18344     }
18345     field(BKLNK, DBF_NOACCESS) {
18346         interest(4)
18347         extra("ELLLIST             bklnk")
```

```
18348         special(SPC_NOMOD)
18349         prompt("Backwards link tracking")
18350     }
18351     field(DISP, DBF_UCHAR) {
18352         promptgroup("10 - Common")
18353         interest(1)
18354         prompt("Disable putField")
18355     }
18356     field(PROC, DBF_UCHAR) {
18357         interest(3)
18358         pp(TRUE)
18359         prompt("Force Processing")
18360     }
18361     field(STAT, DBF_MENU) {
18362         initial("UDF")
18363         menu(menuAlarmStat)
18364         special(SPC_NOMOD)
18365         prompt("Alarm Status")
18366     }
18367     field(SEVR, DBF_MENU) {
18368         menu(menuAlarmSevr)
18369         special(SPC_NOMOD)
18370         prompt("Alarm Severity")
18371     }
18372     field(AMSG, DBF_STRING) {
18373         size(40)
18374         special(SPC_NOMOD)
18375         prompt("Alarm Message")
18376     }
18377     field(NSTA, DBF_MENU) {
18378         interest(2)
18379         menu(menuAlarmStat)
18380         special(SPC_NOMOD)
18381         prompt("New Alarm Status")
18382     }
18383     field(NSEV, DBF_MENU) {
18384         interest(2)
18385         menu(menuAlarmSevr)
18386         special(SPC_NOMOD)
18387         prompt("New Alarm Severity")
18388     }
18389     field(NAMSG, DBF_STRING) {
18390         size(40)
18391         special(SPC_NOMOD)
18392         prompt("New Alarm Message")
18393     }
18394     field(ACKS, DBF_MENU) {
18395         interest(2)
18396         menu(menuAlarmSevr)
18397         special(SPC_NOMOD)
18398         prompt("Alarm Ack Severity")
18399     }
18400     field(ACKT, DBF_MENU) {
18401         promptgroup("70 - Alarm")
18402         interest(2)
18403         initial("YES")
18404         menu(menuYesNo)
18405         special(SPC_NOMOD)
18406         prompt("Alarm Ack Transient")
18407     }
18408     field(DISS, DBF_MENU) {
18409         promptgroup("70 - Alarm")
18410         interest(1)
18411         menu(menuAlarmSevr)
18412         prompt("Disable Alarm Sevrty")
18413     }
18414     field(LCNT, DBF_UCHAR) {
18415         interest(2)
```

```
18416         special(SPC_NOMOD)
18417         prompt("Lock Count")
18418     }
18419     field(PACT, DBF_UCHAR) {
18420         interest(1)
18421         special(SPC_NOMOD)
18422         prompt("Record active")
18423     }
18424     field(PUTF, DBF_UCHAR) {
18425         interest(1)
18426         special(SPC_NOMOD)
18427         prompt("dbPutField process")
18428     }
18429     field(RPRO, DBF_UCHAR) {
18430         interest(1)
18431         special(SPC_NOMOD)
18432         prompt("Reprocess ")
18433     }
18434     field(ASP, DBF_NOACCESS) {
18435         interest(4)
18436         extra("struct asgMember    *asp")
18437         special(SPC_NOMOD)
18438         prompt("Access Security Pvt")
18439     }
18440     field(PPN, DBF_NOACCESS) {
18441         interest(4)
18442         extra("struct processNotify *ppn")
18443         special(SPC_NOMOD)
18444         prompt("pprocessNotify")
18445     }
18446     field(PPNR, DBF_NOACCESS) {
18447         interest(4)
18448         extra("struct processNotifyRecord *ppnr")
18449         special(SPC_NOMOD)
18450         prompt("pprocessNotifyRecord")
18451     }
18452     field(SPVT, DBF_NOACCESS) {
18453         interest(4)
18454         extra("struct scan_element *spvt")
18455         special(SPC_NOMOD)
18456         prompt("Scan Private")
18457     }
18458     field(RSET, DBF_NOACCESS) {
18459         interest(4)
18460         extra("struct typed_rset   *rset")
18461         special(SPC_NOMOD)
18462         prompt("Address of RSET")
18463     }
18464     field(DSET, DBF_NOACCESS) {
18465         interest(4)
18466         extra("unambiguous_dset    *dset")
18467         special(SPC_NOMOD)
18468         prompt("DSET address")
18469     }
18470     field(DPVT, DBF_NOACCESS) {
18471         interest(4)
18472         extra("void                *dpvt")
18473         special(SPC_NOMOD)
18474         prompt("Device Private")
18475     }
18476     field(RDES, DBF_NOACCESS) {
18477         interest(4)
18478         extra("struct dbRecordType *rdes")
18479         special(SPC_NOMOD)
18480         prompt("Address of dbRecordType")
18481     }
18482     field(LSET, DBF_NOACCESS) {
18483         interest(4)
```

```
18484         extra("struct lockRecord   *lset")
18485         special(SPC_NOMOD)
18486         prompt("Lock Set")
18487     }
18488     field(PRIO, DBF_MENU) {
18489         promptgroup("20 - Scan")
18490         interest(1)
18491         menu(menuPriority)
18492         special(SPC_SCAN)
18493         prompt("Scheduling Priority")
18494     }
18495     field(TPRO, DBF_UCHAR) {
18496         prompt("Trace Processing")
18497     }
18498     field(BKPT, DBF_NOACCESS) {
18499         interest(1)
18500         extra("epicsUInt8          bkpt")
18501         special(SPC_NOMOD)
18502         prompt("Break Point")
18503     }
18504     field(UDF, DBF_UCHAR) {
18505         promptgroup("10 - Common")
18506         interest(1)
18507         initial("1")
18508         pp(TRUE)
18509         prompt("Undefined")
18510     }
18511     field(UDFS, DBF_MENU) {
18512         promptgroup("70 - Alarm")
18513         interest(1)
18514         initial("INVALID")
18515         menu(menuAlarmSevr)
18516         prompt("Undefined Alarm Sevrty")
18517     }
18518     field(TIME, DBF_NOACCESS) {
18519         interest(2)
18520         extra("epicsTimeStamp      time")
18521         special(SPC_NOMOD)
18522         prompt("Time")
18523     }
18524     field(UTAG, DBF_UINT64) {
18525         interest(3)
18526         special(SPC_NOMOD)
18527         prompt("Time Tag")
18528     }
18529     field(FLNK, DBF_FWDLINK) {
18530         promptgroup("20 - Scan")
18531         interest(1)
18532         prompt("Forward Process Link")
18533     }
18534     field(VAL, DBF_NOACCESS) {
18535         extra("void *       val")
18536         asl(ASL0)
18537         pp(TRUE)
18538         special(SPC_DBADDR)
18539         prompt("Value")
18540     }
18541     field(PREC, DBF_SHORT) {
18542         promptgroup("80 - Display")
18543         interest(1)
18544         prop(YES)
18545         prompt("Display Precision")
18546     }
18547     field(FTVL, DBF_MENU) {
18548         promptgroup("30 - Action")
18549         interest(1)
18550         menu(menuFtype)
18551         special(SPC_NOMOD)
```

```
18552         prompt("Field Type of Value")
18553     }
18554     field(INP, DBF_INLINK) {
18555         promptgroup("40 - Input")
18556         interest(1)
18557         prompt("Input Specification")
18558     }
18559     field(EGU, DBF_STRING) {
18560         size(16)
18561         promptgroup("80 - Display")
18562         interest(1)
18563         prop(YES)
18564         prompt("Engineering Units")
18565     }
18566     field(HOPR, DBF_DOUBLE) {
18567         promptgroup("80 - Display")
18568         interest(1)
18569         prop(YES)
18570         prompt("High Operating Range")
18571     }
18572     field(LOPR, DBF_DOUBLE) {
18573         promptgroup("80 - Display")
18574         interest(1)
18575         prop(YES)
18576         prompt("Low Operating Range")
18577     }
18578     field(MALM, DBF_ULONG) {
18579         promptgroup("30 - Action")
18580         prompt("Maximum Elements")
18581         prop(YES)
18582         interest(1)
18583         initial("1")
18584         special(SPC_NOMOD)
18585     }
18586     field(NELM, DBF_ULONG) {
18587         promptgroup("30 - Action")
18588         initial("1")
18589         pp(TRUE)
18590         prompt("Number of Elements")
18591     }
18592     field(INDX, DBF_ULONG) {
18593         promptgroup("30 - Action")
18594         pp(TRUE)
18595         prompt("Substring Index")
18596     }
18597     field(BUSY, DBF_SHORT) {
18598         special(SPC_NOMOD)
18599         prompt("Busy Indicator")
18600     }
18601     field(NORD, DBF_LONG) {
18602         special(SPC_NOMOD)
18603         prompt("Number elements read")
18604     }
18605     field(BPTR, DBF_NOACCESS) {
18606         interest(4)
18607         extra("void *       bptr")
18608         special(SPC_NOMOD)
18609         prompt("Buffer Pointer")
18610     }
18611 }
18612 device(subArray, CONSTANT, devSASoft, "Soft Channel")
18613 recordtype(waveform) {
18614     %#include "epicsTypes.h"
18615     %#include "link.h"
18616     %#include "epicsMutex.h"
18617     %#include "ellLib.h"
18618     %#include "devSup.h"
18619     %#include "epicsTime.h"
```

```
18620     %
18621     %/* Declare Device Support Entry Table */
18622     %struct waveformRecord;
18623     %typedef struct wfdset {
18624     %    dset common; /*init_record returns: (-1,0)=>(failure,success)*/
18625     %    long (*read_wf)(struct waveformRecord *prec); /*returns: (-1,0)=>(failure,success)*/
18626     %} wfdset;
18627     %#define HAS_wfdset
18628     %#define WF_POSTS_NORD
18629     %
18630     %#include "callback.h"
18631     field(NAME, DBF_STRING) {
18632         size(61)
18633         special(SPC_NOMOD)
18634         prompt("Record Name")
18635     }
18636     field(DESC, DBF_STRING) {
18637         size(41)
18638         promptgroup("10 - Common")
18639         prompt("Descriptor")
18640     }
18641     field(ASG, DBF_STRING) {
18642         size(29)
18643         promptgroup("10 - Common")
18644         special(SPC_AS)
18645         prompt("Access Security Group")
18646     }
18647     field(SCAN, DBF_MENU) {
18648         promptgroup("20 - Scan")
18649         interest(1)
18650         menu(menuScan)
18651         special(SPC_SCAN)
18652         prompt("Scan Mechanism")
18653     }
18654     field(PINI, DBF_MENU) {
18655         promptgroup("20 - Scan")
18656         interest(1)
18657         menu(menuPini)
18658         prompt("Process at iocInit")
18659     }
18660     field(PHAS, DBF_SHORT) {
18661         promptgroup("20 - Scan")
18662         interest(1)
18663         special(SPC_SCAN)
18664         prompt("Scan Phase")
18665     }
18666     field(EVNT, DBF_STRING) {
18667         size(40)
18668         promptgroup("20 - Scan")
18669         interest(1)
18670         special(SPC_SCAN)
18671         prompt("Event Name")
18672     }
18673     field(TSE, DBF_SHORT) {
18674         promptgroup("20 - Scan")
18675         interest(1)
18676         prompt("Time Stamp Event")
18677     }
18678     field(TSEL, DBF_INLINK) {
18679         promptgroup("20 - Scan")
18680         interest(1)
18681         prompt("Time Stamp Link")
18682     }
18683     field(DTYP, DBF_DEVICE) {
18684         promptgroup("10 - Common")
18685         interest(1)
18686         prompt("Device Type")
18687     }
```

```
18688     field(DISV, DBF_SHORT) {
18689         promptgroup("20 - Scan")
18690         initial("1")
18691         prompt("Disable Value")
18692     }
18693     field(DISA, DBF_SHORT) {
18694         prompt("Disable")
18695     }
18696     field(SDIS, DBF_INLINK) {
18697         promptgroup("20 - Scan")
18698         interest(1)
18699         prompt("Scanning Disable")
18700     }
18701     field(MLOK, DBF_NOACCESS) {
18702         interest(4)
18703         extra("epicsMutexId        mlok")
18704         special(SPC_NOMOD)
18705         prompt("Monitor lock")
18706     }
18707     field(MLIS, DBF_NOACCESS) {
18708         interest(4)
18709         extra("ELLLIST             mlis")
18710         special(SPC_NOMOD)
18711         prompt("Monitor List")
18712     }
18713     field(BKLNK, DBF_NOACCESS) {
18714         interest(4)
18715         extra("ELLLIST             bklnk")
18716         special(SPC_NOMOD)
18717         prompt("Backwards link tracking")
18718     }
18719     field(DISP, DBF_UCHAR) {
18720         promptgroup("10 - Common")
18721         interest(1)
18722         prompt("Disable putField")
18723     }
18724     field(PROC, DBF_UCHAR) {
18725         interest(3)
18726         pp(TRUE)
18727         prompt("Force Processing")
18728     }
18729     field(STAT, DBF_MENU) {
18730         initial("UDF")
18731         menu(menuAlarmStat)
18732         special(SPC_NOMOD)
18733         prompt("Alarm Status")
18734     }
18735     field(SEVR, DBF_MENU) {
18736         menu(menuAlarmSevr)
18737         special(SPC_NOMOD)
18738         prompt("Alarm Severity")
18739     }
18740     field(AMSG, DBF_STRING) {
18741         size(40)
18742         special(SPC_NOMOD)
18743         prompt("Alarm Message")
18744     }
18745     field(NSTA, DBF_MENU) {
18746         interest(2)
18747         menu(menuAlarmStat)
18748         special(SPC_NOMOD)
18749         prompt("New Alarm Status")
18750     }
18751     field(NSEV, DBF_MENU) {
18752         interest(2)
18753         menu(menuAlarmSevr)
18754         special(SPC_NOMOD)
18755         prompt("New Alarm Severity")
```

```
18756     }
18757     field(NAMSG, DBF_STRING) {
18758         size(40)
18759         special(SPC_NOMOD)
18760         prompt("New Alarm Message")
18761     }
18762     field(ACKS, DBF_MENU) {
18763         interest(2)
18764         menu(menuAlarmSevr)
18765         special(SPC_NOMOD)
18766         prompt("Alarm Ack Severity")
18767     }
18768     field(ACKT, DBF_MENU) {
18769         promptgroup("70 - Alarm")
18770         interest(2)
18771         initial("YES")
18772         menu(menuYesNo)
18773         special(SPC_NOMOD)
18774         prompt("Alarm Ack Transient")
18775     }
18776     field(DISS, DBF_MENU) {
18777         promptgroup("70 - Alarm")
18778         interest(1)
18779         menu(menuAlarmSevr)
18780         prompt("Disable Alarm Sevrty")
18781     }
18782     field(LCNT, DBF_UCHAR) {
18783         interest(2)
18784         special(SPC_NOMOD)
18785         prompt("Lock Count")
18786     }
18787     field(PACT, DBF_UCHAR) {
18788         interest(1)
18789         special(SPC_NOMOD)
18790         prompt("Record active")
18791     }
18792     field(PUTF, DBF_UCHAR) {
18793         interest(1)
18794         special(SPC_NOMOD)
18795         prompt("dbPutField process")
18796     }
18797     field(RPRO, DBF_UCHAR) {
18798         interest(1)
18799         special(SPC_NOMOD)
18800         prompt("Reprocess ")
18801     }
18802     field(ASP, DBF_NOACCESS) {
18803         interest(4)
18804         extra("struct asgMember    *asp")
18805         special(SPC_NOMOD)
18806         prompt("Access Security Pvt")
18807     }
18808     field(PPN, DBF_NOACCESS) {
18809         interest(4)
18810         extra("struct processNotify *ppn")
18811         special(SPC_NOMOD)
18812         prompt("pprocessNotify")
18813     }
18814     field(PPNR, DBF_NOACCESS) {
18815         interest(4)
18816         extra("struct processNotifyRecord *ppnr")
18817         special(SPC_NOMOD)
18818         prompt("pprocessNotifyRecord")
18819     }
18820     field(SPVT, DBF_NOACCESS) {
18821         interest(4)
18822         extra("struct scan_element *spvt")
18823         special(SPC_NOMOD)
```

```
        prompt("Scan Private")
    }
    field(RSET, DBF_NOACCESS) {
        interest(4)
        extra("struct typed_rset   *rset")
        special(SPC_NOMOD)
        prompt("Address of RSET")
    }
    field(DSET, DBF_NOACCESS) {
        interest(4)
        extra("unambiguous_dset    *dset")
        special(SPC_NOMOD)
        prompt("DSET address")
    }
    field(DPVT, DBF_NOACCESS) {
        interest(4)
        extra("void                *dpvt")
        special(SPC_NOMOD)
        prompt("Device Private")
    }
    field(RDES, DBF_NOACCESS) {
        interest(4)
        extra("struct dbRecordType *rdes")
        special(SPC_NOMOD)
        prompt("Address of dbRecordType")
    }
    field(LSET, DBF_NOACCESS) {
        interest(4)
        extra("struct lockRecord   *lset")
        special(SPC_NOMOD)
        prompt("Lock Set")
    }
    field(PRIO, DBF_MENU) {
        promptgroup("20 - Scan")
        interest(1)
        menu(menuPriority)
        special(SPC_SCAN)
        prompt("Scheduling Priority")
    }
    field(TPRO, DBF_UCHAR) {
        prompt("Trace Processing")
    }
    field(BKPT, DBF_NOACCESS) {
        interest(1)
        extra("epicsUInt8          bkpt")
        special(SPC_NOMOD)
        prompt("Break Point")
    }
    field(UDF, DBF_UCHAR) {
        promptgroup("10 - Common")
        interest(1)
        initial("1")
        pp(TRUE)
        prompt("Undefined")
    }
    field(UDFS, DBF_MENU) {
        promptgroup("70 - Alarm")
        interest(1)
        initial("INVALID")
        menu(menuAlarmSevr)
        prompt("Undefined Alarm Sevrty")
    }
    field(TIME, DBF_NOACCESS) {
        interest(2)
        extra("epicsTimeStamp      time")
        special(SPC_NOMOD)
        prompt("Time")
    }
```

```
18892     field(UTAG, DBF_UINT64) {
18893         interest(3)
18894         special(SPC_NOMOD)
18895         prompt("Time Tag")
18896     }
18897     field(FLNK, DBF_FWDLINK) {
18898         promptgroup("20 - Scan")
18899         interest(1)
18900         prompt("Forward Process Link")
18901     }
18902     field(VAL, DBF_NOACCESS) {
18903         extra("void *       val")
18904         asl(ASL0)
18905         pp(TRUE)
18906         special(SPC_DBADDR)
18907         prompt("Value")
18908     }
18909     field(RARM, DBF_SHORT) {
18910         promptgroup("30 - Action")
18911         interest(1)
18912         pp(TRUE)
18913         prompt("Rearm the waveform")
18914     }
18915     field(PREC, DBF_SHORT) {
18916         promptgroup("80 - Display")
18917         interest(1)
18918         prop(YES)
18919         prompt("Display Precision")
18920     }
18921     field(INP, DBF_INLINK) {
18922         promptgroup("40 - Input")
18923         interest(1)
18924         prompt("Input Specification")
18925     }
18926     field(EGU, DBF_STRING) {
18927         size(16)
18928         promptgroup("80 - Display")
18929         interest(1)
18930         prop(YES)
18931         prompt("Engineering Units")
18932     }
18933     field(HOPR, DBF_DOUBLE) {
18934         promptgroup("80 - Display")
18935         interest(1)
18936         prop(YES)
18937         prompt("High Operating Range")
18938     }
18939     field(LOPR, DBF_DOUBLE) {
18940         promptgroup("80 - Display")
18941         interest(1)
18942         prop(YES)
18943         prompt("Low Operating Range")
18944     }
18945     field(NELM, DBF_ULONG) {
18946         promptgroup("30 - Action")
18947         interest(1)
18948         initial("1")
18949         special(SPC_NOMOD)
18950         prompt("Number of Elements")
18951     }
18952     field(FTVL, DBF_MENU) {
18953         promptgroup("30 - Action")
18954         interest(1)
18955         menu(menuFtype)
18956         special(SPC_NOMOD)
18957         prompt("Field Type of Value")
18958     }
18959     field(BUSY, DBF_SHORT) {
```

```
18960         special(SPC_NOMOD)
18961         prompt("Busy Indicator")
18962     }
18963     field(NORD, DBF_ULONG) {
18964         special(SPC_NOMOD)
18965         prompt("Number elements read")
18966     }
18967     field(BPTR, DBF_NOACCESS) {
18968         interest(4)
18969         extra("void *       bptr")
18970         special(SPC_NOMOD)
18971         prompt("Buffer Pointer")
18972     }
18973     field(SIOL, DBF_INLINK) {
18974         promptgroup("90 - Simulate")
18975         interest(1)
18976         prompt("Simulation Input Link")
18977     }
18978     field(SIML, DBF_INLINK) {
18979         promptgroup("90 - Simulate")
18980         interest(1)
18981         prompt("Simulation Mode Link")
18982     }
18983     field(SIMM, DBF_MENU) {
18984         interest(1)
18985         menu(menuYesNo)
18986         special(SPC_MOD)
18987         prompt("Simulation Mode")
18988     }
18989     field(SIMS, DBF_MENU) {
18990         promptgroup("90 - Simulate")
18991         interest(2)
18992         menu(menuAlarmSevr)
18993         prompt("Simulation Mode Severity")
18994     }
18995     field(OLDSIMM, DBF_MENU) {
18996         interest(4)
18997         menu(menuSimm)
18998         special(SPC_NOMOD)
18999         prompt("Prev. Simulation Mode")
19000     }
19001     field(SSCN, DBF_MENU) {
19002         promptgroup("90 - Simulate")
19003         interest(1)
19004         initial("65535")
19005         menu(menuScan)
19006         prompt("Sim. Mode Scan")
19007     }
19008     field(SDLY, DBF_DOUBLE) {
19009         promptgroup("90 - Simulate")
19010         interest(2)
19011         initial("-1.0")
19012         prompt("Sim. Mode Async Delay")
19013     }
19014     field(SIMPVT, DBF_NOACCESS) {
19015         interest(4)
19016         extra("epicsCallback            *simpvt")
19017         special(SPC_NOMOD)
19018         prompt("Sim. Mode Private")
19019     }
19020     field(MPST, DBF_MENU) {
19021         promptgroup("80 - Display")
19022         interest(1)
19023         menu(waveformPOST)
19024         prompt("Post Value Monitors")
19025     }
19026     field(APST, DBF_MENU) {
19027         promptgroup("80 - Display")
```

```
        interest(1)
        menu(waveformPOST)
        prompt("Post Archive Monitors")
    }
    field(HASH, DBF_ULONG) {
        interest(3)
        prompt("Hash of OnChange data.")
    }
}
device(waveform, CONSTANT, devWfSoft, "Soft Channel")
device(waveform, INST_IO, devPSCBlockInWf8, "PSC Block I8 In")
device(waveform, INST_IO, devPSCBlockOutWf8, "PSC Block I8 Out")
device(waveform, INST_IO, devPSCBlockInWf16, "PSC Block I16 In")
device(waveform, INST_IO, devPSCBlockOutWf16, "PSC Block I16 Out")
device(waveform, INST_IO, devPSCBlockInWf24, "PSC Block I24 In")
device(waveform, INST_IO, devPSCBlockInWf32, "PSC Block I32 In")
device(waveform, INST_IO, devPSCBlockOutWf32, "PSC Block I32 Out")
device(waveform, INST_IO, devPSCBlockInWfF32, "PSC Block F32 In")
device(waveform, INST_IO, devPSCBlockOutWfF32, "PSC Block F32 Out")
device(waveform, INST_IO, devPSCBlockInWfF64, "PSC Block F64 In")
device(waveform, INST_IO, devPSCBlockOutWfF64, "PSC Block F64 Out")
driver(drvPSC)
link(calc, lnkCalcIf)
link(const, lnkConstIf)
link(debug, lnkDebugIf)
link(state, lnkStateIf)
link(trace, lnkTraceIf)
registrar(PSCRegister)
registrar(arrInitialize)
registrar(asSub)
registrar(dbndInitialize)
registrar(decInitialize)
registrar(rsrvRegistrar)
registrar(syncInitialize)
registrar(tsInitialize)
registrar(utagInitialize)
variable(CASDEBUG, int)
variable(PSCDebug, int)
variable(PSCInactivityTime, int)
variable(PSCMaxSendBuffer, int)
variable(asCaDebug, int)
variable(atExitDebug, int)
variable(boHIGHlimit, double)
variable(boHIGHprecision, int)
variable(calcoutODLYlimit, double)
variable(calcoutODLYprecision, int)
variable(callbackParallelThreadsDefault, int)
variable(dbAccessDebugPUTF, int)
variable(dbBptNotMonotonic, int)
variable(dbConvertStrict, int)
variable(dbJLinkDebug, int)
variable(dbQuietMacroWarnings, int)
variable(dbRecordsAbcSorted, int)
variable(dbRecordsOnceOnly, int)
variable(dbTemplateMaxVars, int)
variable(dbThreadRealtimeLock, int)
variable(histogramSDELprecision, int)
variable(lnkDebug_debug, int)
variable(logClientDebug, int)
variable(seqDLYlimit, double)
variable(seqDLYprecision, int)
```

Listing C.25: EPICS IOC: dbd/psuMonitor.dbd

```
#!../../bin/linux-x86_64/psuMonitor

< envPaths
```

```
4
5  cd "${TOP}"
6
7  dbLoadDatabase("dbd/psuMonitor.dbd")
8  psuMonitor_registerRecordDeviceDriver(pdbbase)
9
10 epicsEnvSet("P",      "TEST:")
11 epicsEnvSet("R",      "GIGA")
12 epicsEnvSet("DEVICE", "GIGA_PSC")
13 epicsEnvSet("HOST",   "192.168.1.177")
14 epicsEnvSet("PORT",   "8765")
15
16 # Final argument enables PSCDriver's receive-inactivity timeout.
17 var PSCDebug 5
18 createPSC("$(DEVICE)", "$(HOST)", $(PORT), 1)
19
20 dbLoadRecords("db/gigaPscBase.db", "P=$(P),R=$(R),DEVICE=$(DEVICE)")
21 dbLoadTemplate("db/gigaPsc.substitutions", "P=$(P),R=$(R),DEVICE=$(DEVICE)")
22
23 cd "${TOP}/iocBoot/${IOC}"
24 iocInit()
```

Listing C.26: EPICS IOC: iocBoot/iocpsuMonitor/st.cmd

```
1  file "db/gigaPscChannel.template" {
2  pattern {
3    CHAN, DESC, VOFF, IOFF, VLOWOFF, VHIGHOFF, ILOWOFF, IHIGHOFF, SOFF,
4    VLOWADDR, VHIGHADDR, ILOWADDR, IHIGHADDR, VLO, VHI, ILO_MA, IHI_MA
5  }
6  {
7    "6VA",  "+6 V A",  0,  4,  0,  4,  8, 12,  0,  0,  1,  2,  3,   5.40,   6.60, 0.00, 1200
8  }
9  {
10   "6VB",  "+6 V B",  8, 12, 16, 20, 24, 28,  4,  4,  5,  6,  7,   5.40,   6.60, 0.00, 1200
11 }
12 {
13   "P5V",  "+5 V",   16, 20, 32, 36, 40, 44,  8,  8,  9, 10, 11,   4.50,   5.50, 0.00, 1000
14 }
15 {
16   "M5V",  "-5 V",   24, 28, 48, 52, 56, 60, 12, 12, 13, 14, 15,  -5.50,  -4.50, 0.00, 1000
17 }
18 {
19   "P15V", "+15 V",  32, 36, 64, 68, 72, 76, 16, 16, 17, 18, 19,  13.50,  16.50, 0.00, 750
20 }
21 {
22   "M15V", "-15 V",  40, 44, 80, 84, 88, 92, 20, 20, 21, 22, 23, -16.50, -13.50, 0.00, 750
23 }
24 }
```

Listing C.27: EPICS IOC: psuMonitorApp/Db/gigaPsc.substitutions

```
1  record(bi, "$(P)$(R):CONNECTED") {
2      field(DESC, "GIGA PSC TCP connection")
3      field(DTYP, "PSC Ctrl Connected")
4      field(INP,  "@$(DEVICE)")
5      field(SCAN, "I/O Intr")
6      field(ZNAM, "Disconnected")
7      field(ONAM, "Connected")
8      field(ZSV,  "MAJOR")
9  }
10
11 record(stringin, "$(P)$(R):DRIVER_MESSAGE") {
12     field(DESC, "Last PSCDriver message")
13     field(DTYP, "PSC Ctrl Message")
14     field(INP,  "@$(DEVICE)")
15     field(SCAN, "I/O Intr")
```

```
}

record(longin, "$(P)$(R):CONNECTION_COUNT") {
    field(DESC, "PSC connection count")
    field(DTYP, "PSC Conn Count")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "1 second")
}

record(longin, "$(P)$(R):UNKNOWN_MESSAGE_COUNT") {
    field(DESC, "Unknown PSC message count")
    field(DTYP, "PSC Unknown Msg Count")
    field(INP,  "@$(DEVICE)")
    field(SCAN, "1 second")
}

# PSC Single F32 records queue one message. This flushes queued messages to TCP.
record(bo, "$(P)$(R):SEND_ALL") {
    field(DESC, "Flush PSC transmit queue")
    field(DTYP, "PSC Ctrl Send All")
    field(OUT,  "@$(DEVICE)")
    field(ZNAM, "Send")
    field(ONAM, "Send")
}
```

Listing C.28: EPICS IOC: psuMonitorApp/Db/gigaPscBase.db

```
# Direct PSC receive records. These are intentionally RAW/internal PVs.

record(ai, "$(P)$(R):$(CHAN):VOLT_RAW") {
field(DESC, "$(DESC) voltage from PSC")
field(DTYP, "PSC Reg F32")
field(INP,  "@$(DEVICE) 1 $(VOFF)")
field(SCAN, "I/O Intr")
field(TSE,  "-2")
field(PREC, "4")
field(EGU,  "V")
}

record(ai, "$(P)$(R):$(CHAN):CURR_RAW") {
field(DESC, "$(DESC) current from PSC")
field(DTYP, "PSC Reg F32")
field(INP,  "@$(DEVICE) 1 $(IOFF)")
field(SCAN, "I/O Intr")
field(TSE,  "-2")
field(PREC, "4")
field(EGU,  "A")
}

# Public calc records copy and scale the direct values.

# The CONNECTED CP/MS link causes immediate processing and propagates

# a MAJOR alarm when the TCP connection drops.

record(calc, "$(P)$(R):$(CHAN):VOLT") {
field(DESC, "$(DESC) voltage")
field(INPA, "$(P)$(R):$(CHAN):VOLT_RAW CP NMS")
field(INPB, "$(P)$(R):CONNECTED CP MS")
field(CALC, "A")
field(PREC, "4")
field(EGU,  "V")
field(LOW,  "$(VLO)")
field(LSV,  "MAJOR")
field(HIGH, "$(VHI)")
field(HSV,  "MAJOR")
}
```

```
41
42 record(calc, "$(P)$(R):$(CHAN):CURR") {
43 field(DESC, "$(DESC) current")
44 field(INPA, "$(P)$(R):$(CHAN):CURR_RAW CP NMS")
45 field(INPB, "$(P)$(R):CONNECTED CP MS")
46
47
48 # Convert amperes received from the Arduino to milliamperes.
49 field(CALC, "A*1000")
50
51 field(PREC, "3")
52 field(EGU,  "mA")
53
54 # The public current alarm limits are expressed in milliamperes.
55 field(LOW,  "$(ILO_MA)")
56 field(LSV,  "MAJOR")
57 field(HIGH, "$(IHI_MA)")
58 field(HSV,  "MAJOR")
59
60
61 }
62
63 # Device-confirmed alarm-limit readbacks from PSC message 2.
64
65 # Current readbacks remain in amperes because they come directly
66
67 # from the Arduino/PSC protocol.
68
69 record(ai, "$(P)$(R):$(CHAN):VLOW_RBV") {
70 field(DTYP, "PSC Reg F32")
71 field(INP,  "@$(DEVICE) 2 $(VLOWOFF)")
72 field(SCAN, "I/O Intr")
73 field(PREC, "4")
74 field(EGU,  "V")
75 field(FLNK, "$(P)$(R):$(CHAN):SYNC_VLOW")
76 }
77
78 record(ai, "$(P)$(R):$(CHAN):VHIGH_RBV") {
79 field(DTYP, "PSC Reg F32")
80 field(INP,  "@$(DEVICE) 2 $(VHIGHOFF)")
81 field(SCAN, "I/O Intr")
82 field(PREC, "4")
83 field(EGU,  "V")
84 field(FLNK, "$(P)$(R):$(CHAN):SYNC_VHIGH")
85 }
86
87 record(ai, "$(P)$(R):$(CHAN):ILOW_RBV") {
88 field(DTYP, "PSC Reg F32")
89 field(INP,  "@$(DEVICE) 2 $(ILOWOFF)")
90 field(SCAN, "I/O Intr")
91 field(PREC, "4")
92 field(EGU,  "A")
93 field(FLNK, "$(P)$(R):$(CHAN):SYNC_ILOW")
94 }
95
96 record(ai, "$(P)$(R):$(CHAN):IHIGH_RBV") {
97 field(DTYP, "PSC Reg F32")
98 field(INP,  "@$(DEVICE) 2 $(IHIGHOFF)")
99 field(SCAN, "I/O Intr")
100 field(PREC, "4")
101 field(EGU,  "A")
102 field(FLNK, "$(P)$(R):$(CHAN):SYNC_IHIGH")
103 }
104
105 # Keep the public measurement alarm fields synchronized
106
107 # with the device-confirmed alarm limits.
108
```

```
record(calcout, "$(P)$(R):$(CHAN):SYNC_VLOW") {
field(INPA, "$(P)$(R):$(CHAN):VLOW_RBV NPP NMS")
field(CALC, "A")
field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):VOLT.LOW NPP")
}

record(calcout, "$(P)$(R):$(CHAN):SYNC_VHIGH") {
field(INPA, "$(P)$(R):$(CHAN):VHIGH_RBV NPP NMS")
field(CALC, "A")
field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):VOLT.HIGH NPP")
}

record(calcout, "$(P)$(R):$(CHAN):SYNC_ILOW") {
field(INPA, "$(P)$(R):$(CHAN):ILOW_RBV NPP NMS")


# Convert the device limit from amperes to milliamperes before
# writing it into the CURR record's LOW alarm field.
field(CALC, "A*1000")

field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):CURR.LOW NPP")


}

record(calcout, "$(P)$(R):$(CHAN):SYNC_IHIGH") {
field(INPA, "$(P)$(R):$(CHAN):IHIGH_RBV NPP NMS")


# Convert the device limit from amperes to milliamperes before
# writing it into the CURR record's HIGH alarm field.
field(CALC, "A*1000")

field(OOPT, "Every Time")
field(DOPT, "Use CALC")
field(OUT,  "$(P)$(R):$(CHAN):CURR.HIGH NPP")


}

# Device-side alarm mask:

# Bit 0: VLOW

# Bit 1: VHIGH

# Bit 2: ILOW

# Bit 3: IHIGH

record(longin, "$(P)$(R):$(CHAN):ALARM_MASK") {
field(DESC, "$(DESC) device alarm mask")
field(DTYP, "PSC Reg")
field(INP,  "@$(DEVICE) 3 $(SOFF)")
field(SCAN, "I/O Intr")
}

record(calc, "$(P)$(R):$(CHAN):ANY_ALARM") {
field(DESC, "$(DESC) any device alarm")
field(INPA, "$(P)$(R):$(CHAN):ALARM_MASK CP NMS")
field(INPB, "$(P)$(R):CONNECTED CP MS")
field(CALC, "A!=0")
```

```
field(HIGH, "0.5")
field(HSV,  "MAJOR")
}

# Immediate persistent writes through PSC Single F32 message ID 10.

# info(SYNC, "SAME") lets the Arduino echo the actual accepted value back.

record(ao, "$(P)$(R):$(CHAN):VLOW_SP") {
field(DESC, "$(DESC) voltage low setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(VLOWADDR)")
field(PREC, "4")
field(EGU,  "V")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):VHIGH_SP") {
field(DESC, "$(DESC) voltage high setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(VHIGHADDR)")
field(PREC, "4")
field(EGU,  "V")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):ILOW_SP") {
field(DESC, "$(DESC) current low setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(ILOWADDR)")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}

record(ao, "$(P)$(R):$(CHAN):IHIGH_SP") {
field(DESC, "$(DESC) current high setpoint")
field(DTYP, "PSC Single F32")
field(OUT,  "@$(DEVICE) 10 $(IHIGHADDR)")
field(PREC, "4")
field(EGU,  "A")
field(FLNK, "$(P)$(R):SEND_ALL")
info(SYNC, "SAME")
}
```

Listing C.29: EPICS IOC: psuMonitorApp/Db/gigaPscChannel.template

```
#!/bin/bash

cd "/epics/iocs/psuMonitorIOC/iocBoot/iocpsuMonitor"
./st.cmd
```

Listing C.30: EPICS IOC: st.cmd

Appendix D

System Test Plan

D.1 Introduction

Once the detailed design and prototype construction have been done, then it is time to test the entire power supply monitoring system to ensure that the hardware, embedded firmware, user interface, and EPICS interface are working as expected. This chapter describes the test plan that will be used in the evaluation of the prototype.

The purpose of testing is not just checking individual components of the design, but to demonstrate that the entire system is capable of monitoring all six power supply rails, displaying measurements locally, sending measurements through the network and operating within limits. Some parts of the system test will also be videotaped for use in the final project video.

D.1.1 System Under Test

The system under test is the completed six-channel power supply monitoring system developed for this capstone project. The system monitors the following power rails:

- $+5\,\mathrm{V}A$
- $+5\,\mathrm{V}B$
- $+5\,\mathrm{V}$
- $-5\,\mathrm{V}$
- $+15\,\mathrm{V}$
- $-15\,\mathrm{V}$

Each rail has its own separate voltage and current monitoring. The measurement circuits are isolated from the communication channel on the Arduino side and are connected to Arduino

GIGA R1 WiFi. The Arduino has a local graphical display by using GIGA Display Shield and Ethernet connection by a custom interface shield I've built. The Ethernet connection works with EPICS Input/Output Controller (IOC), making it possible to monitor the rails measurement and status remotely.

The testing of the system checks out the entire system but not each circuit block separately. This test involves the sensor circuits, isolation circuits, Arduino firmware, display, Ethernet connection, EPICS IOC, and related software as one whole system.

D.1.1.1 Test Objectives

The primary objectives of the system test are to verify that:

1. the complete system powers on and initializes without errors;
2. all six power rails are identified and displayed correctly;
3. the voltage measurement for each rail agrees with an external reference measurement within the specified system tolerance;
4. the current measurement for each rail agrees with the applied load within the specified system tolerance;
5. positive and negative supply rails are reported with the correct voltage polarity and current convention;
6. measurements shown on the local display correspond with the values available through EPICS;
7. voltage and current operating limits produce the appropriate status indication;
8. user-configurable settings remain stored following a power cycle;
9. Ethernet or IOC communication can be interrupted and restored without requiring a complete reprogramming of the monitor;
10. measurement data can be recorded and used to generate a permanent test report; and
11. the completed system can successfully perform the functions required for the final capstone demonstration.

D.1.2 Scope of Testing

D.1.2.1 Features to be Tested

System testing will include the following functions:

- system startup and initialization;
- detection and monitoring of all six power rails;
- voltage measurement;
- current measurement;
- measurement response to changes in load;
- correct handling of positive and negative rails;
- local GIGA Display operation;
- Ethernet communication;
- EPICS process-variable updates;
- voltage and current limit/status indications;
- storage and restoration of persistent settings;
- communication-loss recovery;
- repeated measurements to identify intermittent operation; and
- automatic recording and reporting of system test results.

D.1.2.2 Features Not Included in This Test

The system test is intended to verify the functional requirements of the prototype only. The following tests are therefore outside the scope of this system-level test plan:

- EMC testing;
- qualification tests for environmental temperature/humidity;
- qualification tests for vibrations or mechanical shock;
- qualification tests for life cycle/accelerated aging;
- destructive overvoltage tests; and
- destructive overcurrent tests.

D.1.3 Test Equipment and Software

The following equipment and software will be used during system testing:

Equipment/Software	Purpose
Power supply monitor prototype	Complete hardware and firmware system under test.
DC bench supplies	Provide the six input rails required by the monitor.
Rigol DL3021 electronic load	Provide programmable and repeatable loading of individual power rails.
Computer running EPICS IOC	Receive voltage, current, and status information from the Arduino over Ethernet.
Phoebus/EPICS display	Provide operator-level observation of process variables and system status.
Python scripts	Control the electronic load, acquire EPICS values using PyEPICS, save raw test data, and automate portions of the test sequence.
Ethernet network	Provide communication between the Arduino and EPICS IOC.

D.1.4 Test Configuration

During testing, the assembled power supply monitor will be operated in its normal system configuration. All six monitored supplies will be connected to the appropriate monitoring inputs. The Arduino, GIGA Display Shield, custom monitoring hardware, and Ethernet interface will be powered and operating simultaneously.

For measurements involving the programmable electronic load, only the rail under test will be connected to the electronic load. The remaining rails will remain energized so that the monitor continues operating as a complete system, and may or may not be connected across resistive loads.

The electronic load measurement will provide the reference for both load voltage and current.

D.1.5 System Acceptance Criteria

The test is intended to establish whether the assembled prototype satisfies its functional requirements. The following system-level acceptance criteria will be used.

Parameter	Acceptance Criterion
System startup	System initialization should not involve a firmware error or user action, and all six rails should be shown.
Voltage measurement	The monitor reading should match the external reference voltage within $\pm 2\%$ across the full tested operating range.

Parameter	Acceptance Criterion
Current measurement	Monitor reading should match the reference current within ±5% across the full intended operating range. In addition, the uncertainty and resolution of the reference device should be considered at low currents.
Positive/negative rail determination	Each measurement should use the correct polarity. The current in the negative channel should follow the sign convention used in the firmware.
Display functionality	The display should show the proper identification of the rail, along with the rail voltage, current, and limits.
EPICS operation	All process variables should communicate and report their status without errors in normal operations.
Display and EPICS agreement	Local and remote readings should be identical to the same underlying measurement. Differences from rounding to integer in display are allowable.
Limit status	Any programmed limit should change status as the corresponding measurement crosses the limit and return to the normal status upon the measurement returning to the allowable range.
Persistent configuration	Any stored settings should be preserved after powering down and up the system.
Network recovery	After the restoration of Ethernet communication, EPICS should continue to update without any firmware reprogramming.
Test data	Each automatic test should yield a recorded raw-data file allowing for test results reconstruction.

The voltage and current accuracy requirements above are system-level requirements rather than component-level accuracy specifications. They include the combined effects of the sensor, shunt resistor, analog-to-digital conversion, calibration, wiring, firmware calculations, and reference-instrument uncertainty.

D.1.6 System Test Matrix

The system tests that are planned are listed in Table D.3 below.

Table D.3: System Level Test Matrix

ID	Test	Method	Pass Criterion	Result
ST-01	Power On and Initialization	Apply power to the entire system and check the operation of Arduino, the local display, the sensor channels and the network interface.	The system initializes properly, all six channels are operational.	☐ Pass ☐ Fail ☐ Retest
ST-02	Rail Identification	Apply power to the six rails and correlate the displayed channel with the corresponding input physical connection.	The correct rail identification with the right polarity occurs.	☐ Pass ☐ Fail ☐ Retest
ST-03	Voltage Accuracy	Measure each rail using a DMM or the E-Load and compare the reference value with the local and EPICS values at different operating points.	The error is within the specified voltage tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-04	Current Accuracy	Apply different loads to each rail using the programmable electronic load and compare the reference current with the monitor measurement.	The error is within the specified current tolerance.	☐ Pass ☐ Fail ☐ Retest
ST-05	Dynamic Load Response	Change the current of the electronic load while monitoring the local display and EPICS values.	The current and voltage readings change accordingly to the load change.	☐ Pass ☐ Fail ☐ Retest
ST-06	Local Display	Monitor voltage, current, rail identification and status of all channels.	All necessary information is visible and correlates with the connected rail.	☐ Pass ☐ Fail ☐ Retest

ID	Test	Method	Pass Criterion	Result
ST-07	EPICS Communications	Run the IOC and check all voltage, current and status process variables.	All necessary process variables are connected and updated properly.	☐ Pass ☐ Fail ☐ Retest
ST-08	Limit / Alarm Operation	Set a limit and change the current of the electronic load or supply until the measured value reaches this limit.	The proper limit indication is created and then is cleared when the value is inside the limits.	☐ Pass ☐ Fail ☐ Retest
ST-09	Persistent Settings	Change some settings, cycle system power and check the value after the power-up.	The stored value is restored properly.	☐ Pass ☐ Fail ☐ Retest
ST-10	Communication Recovery	Disconnect Ethernet while the system is running and reconnect the network and monitor the IOC.	Monitoring on the local machine continues and communication recovers.	☐ Pass ☐ Fail ☐ Retest
ST-11	Repeatability	Make repeated measurements at selected points, including after power cycle.	Measurement remains within the specified acceptance limits with no intermittent failures of the channels.	☐ Pass ☐ Fail ☐ Retest
ST-12	End-To-End Demonstration	Run all parts of the system and demonstrate the operation of local and remote monitoring by changing the load on the selected channel.	All hardware, firmware, display, network, IOC and test software run cooperatively with no system level failure.	☐ Pass ☐ Fail ☐ Retest

D.1.7 Detailed Test Procedures

D.1.7.1 ST-01: Power-On and Initialization

1. Inspect all power, sensor, display, and Ethernet connections.
2. Verify that the electronic load is disabled.
3. Energize the required monitored power supplies.
4. Apply power to the Arduino and monitoring electronics.
5. Observe the Arduino startup sequence.
6. Confirm that the GIGA Display initializes.
7. Confirm that all six expected rail channels are displayed.
8. Start or verify operation of the EPICS IOC.
9. Verify that the IOC establishes communication with the monitor.
10. Record any startup errors or unexpected behavior.

The test passes when the complete system reaches its normal operating state without manual recovery, all six rail channels are present, and network communication can be established.

D.1.7.2 ST-03 and ST-04: Voltage and Current Measurement

Voltage and current accuracy testing will be performed for each of the six rails. Testing will be automated where practical to improve repeatability and preserve the raw test data.

For each selected rail:

1. Select the rail to be tested in the Python test program.
2. Verify the physical connection of the rail and electronic load.
3. Connect the DMM directly across the rail.
4. Set the electronic load to its initial test condition with the input disabled.
5. Enable the load.
6. Allow the readings to stabilize.
7. Record the DMM or E-Load reference voltage.
8. Record the electronic-load reference current.

9. Acquire the corresponding EPICS voltage and current values using PyEPICS.
10. Calculate the voltage and current measurement errors.
11. Repeat the measurement at several loads distributed throughout the intended operating range of the rail.
12. Disable the load before changing connections.
13. Save all raw measurements to the test-data file.

Voltage error will be calculated as

$$\text{Voltage Error (\%)} = \frac{V_{\text{monitor}} - V_{\text{reference}}}{|V_{\text{reference}}|} \times 100. \tag{D.1}$$

Current error will be calculated as

$$\text{Current Error (\%)} = \frac{I_{\text{monitor}} - I_{\text{reference}}}{|I_{\text{reference}}|} \times 100. \tag{D.2}$$

Absolute error will also be retained in the raw data. This is particularly important for low-current measurements where percentage error alone can become misleading.

A rail passes when each required measurement point remains within the specified acceptance criteria. If a point fails, the rail may be retested after the cause of the failure is investigated. Original failed data will not be deleted when a retest is performed.

D.1.7.3 ST-07: EPICS Communication Test

1. Power the complete monitor and connect it to the Ethernet network.
2. Start the EPICS IOC.
3. Verify that the IOC connects to the Arduino TCP server.
4. Observe the voltage and current process variables for all six rails.
5. Compare the EPICS values with the local display.
6. Change the load on a selected rail.
7. Verify that the corresponding EPICS voltage and current values update.
8. Repeat the load change on at least one positive and one negative rail.

The test passes when the required process variables remain connected, values change in response to the physical system, and EPICS and local-display values agree within their displayed resolution.

D.1.7.4 ST-08: Limit and Status Test

1. Select a rail for testing.
2. Record the existing voltage and current limit settings.
3. Configure a test limit that can be crossed safely without exceeding the electrical rating of the hardware.
4. Operate the rail below the limit and verify that normal status is displayed.
5. Increase or decrease the applied load until the selected threshold is crossed.
6. Verify the local status indication.
7. Verify the corresponding EPICS status.
8. Return the measurement to the normal operating range.
9. Verify that the status returns to normal.
10. Restore the original limit setting.

No test limit shall require the monitored supply, sensor, wiring, or electronic load to be operated outside its rated limits.

D.1.7.5 ST-09: Persistent Settings Test

1. Change one or more user-adjustable system settings to known values.
2. Verify that the settings are accepted by the firmware.
3. Remove power from the Arduino and monitor electronics.
4. Wait several seconds.
5. Restore power.
6. Allow the firmware to complete initialization.
7. Read the previously modified settings.
8. Compare the restored values with the values stored before power removal.

The test passes when all settings intended to be persistent are restored without requiring Ethernet communication or manual re-entry.

D.1.7.6 ST-10: Communication-Loss and Recovery Test

1. Establish normal communication between the monitor and IOC.
2. Verify that EPICS values are updating.
3. Disconnect the Ethernet cable.
4. Verify that local monitoring and the GIGA Display continue operating.
5. Observe the IOC response to the communication failure.
6. Reconnect the Ethernet cable.
7. Verify that communication is re-established.
8. Confirm that the process variables resume updating with valid values.

The system passes if the loss of Ethernet communication does not prevent local operation and normal EPICS communications resume after the network connection is restored.

D.1.8 Automated Test Data Collection

A Python test application will be used for measurement testing. The application interfaces with the Rigol DL3021 programmable electronic load and the EPICS IOC. EPICS Channel Access is performed using the PyEPICS library.

The test operator may select an individual power rail instead of being required to test all six rails in a single session. This permits channels to be tested or retested at different times while retaining all results as part of the same system test campaign.

For each test point, the following information will be retained when applicable:

- test date and time;
- rail identification;
- test iteration;
- electronic-load operating mode;
- programmed load value;
- reference voltage;
- reference current;
- EPICS voltage measurement;
- EPICS current measurement;

- absolute voltage error;
- percentage voltage error;
- absolute current error;
- percentage current error;
- applicable limits;
- test result; and
- operator notes.

Raw data will be preserved independently from the final report so that the report can be regenerated without repeating the physical test. After the required tests have been completed, the saved data will be processed using ReportLab to produce a final test report containing the recorded values and pass/fail results.

D.1.9 Test Execution and Result Classification

Each test will be assigned one of the following results:

PASS: The test was completed and all applicable acceptance criteria were met.

FAIL: The test was completed but one or more acceptance criteria were not met.

INCOMPLETE: The test could not be completed because of an unrelated equipment, software, or setup problem.

RETEST: A previously failed or incomplete test was repeated following a corrective action.

A failed test will not be deleted or overwritten following corrective action. The original result and subsequent retest will both be retained to provide traceability of the development process.

D.1.10 Suspension and Resumption Criteria

Testing will be immediately suspended if any of the following conditions occur:

- unexpected overheating of a component or conductor;
- evidence of a short circuit or incorrect wiring;
- operation outside the rated voltage or current of the test equipment;

- unstable or uncontrolled power-supply output;
- a firmware condition that could apply an unintended load;
- loss of isolation where isolation is required; or
- any condition that could damage the prototype or test equipment.

Testing may resume after the cause has been identified, corrective action has been completed, and the affected portion of the system has been inspected or verified before power is reapplied.

D.1.11 Test Responsibilities

As the designer and developer of the capstone project, Mike Capotosto will serve as the primary test operator. Responsibilities include:

- preparing and inspecting the test setup;
- configuring the test equipment;
- executing each test procedure;
- recording results;
- identifying and documenting failures;
- making required corrections;
- performing retests;
- maintaining the raw test data; and
- preparing the final test report and demonstration.

Where practical, automated data acquisition will be used to reduce transcription errors and improve repeatability.

D.1.12 Testing Risks and Contingencies

Several risks may affect system testing. Improper electronic-load connections, particularly when testing the negative supply rails, could damage the electronic load or power supply. Connections will therefore be verified before the load input is enabled.

Network problems could also produce an apparent measurement failure when the actual sensor hardware is operating correctly. Local display values and direct instrument measurements will be used to distinguish a network or IOC problem from a hardware measurement problem.

A failed sensor channel could prevent the complete six-channel system test from being performed. Because the test software allows individual rail selection, completed channel data can be retained while the affected channel is repaired and subsequently retested.

Finally, automatic test software itself could introduce an error in the recorded results. Critical results will therefore be compared with the front-panel readings of the electronic load, local display, and EPICS interface during initial testing to confirm correct operation of the automation.

D.1.13 Test Deliverables

The completed system test will produce the following deliverables:

- this system test plan;
- saved raw test data;
- automatically generated test report;
- documentation of failed tests and corrective actions;
- results of any required retests;
- photographs of the final test configuration; and
- video footage demonstrating selected system-level tests.

D.1.14 Test Completion Criteria

System-level testing will be considered complete when:

1. every test identified in Table D.3 has been executed;
2. each of the six rail channels has completed voltage and current verification;
3. all critical functional tests have received a PASS result;
4. any failed tests have been documented and either corrected and retested or identified as known limitations;
5. raw test data has been preserved;
6. the final test report has been generated

Successful completion of this test plan will demonstrate that the individual hardware and software portions developed throughout the project operate together as a complete power supply monitoring system.

D.1.15 Test Plan and Results

The test plan developed in in the previous chapter was performed after completion of hardware bring-up, firmware development, local display integration, Ethernet communications, and EPICS IOC integration. Testing was done for the whole assembled system instead of the individual circuit blocks in order to assess the interactions between the sensor hardware, Arduino controller, firmware, local display, Ethernet communications, EPICS IOC, and test software.

The final test produced the automated PDF system test report containing acceptance criteria, overall test matrix, rail-by-rail measurement results, error plots, operator observations, detailed system test checklist, completion criteria, and engineering justification for the selected tolerances. The complete generated report is attached at the end of Part A.

D.1.15.1 Test Execution

Tests were conducted on each of the six monitored power supply rails:

- +5VA,
- +5VB,
- +5 V,
- −5 V,
- +15 V, and
- −15 V.

A Rigol programmable electronic load was used to set multiple load cases on each rail. During each test point, the external reference voltage and current were compared with the corresponding measurements reported by the EPICS IOC. The local GIGA display was also observed to check rail identity, polarity, measurements updates, limits state and communication state.

The +5VA and +5VB channels were tested at approximately 0 A, 1 A, and 2 A. The other +5 V, −5 V, +15 V, and −15 V channels were tested at approximately 0 A, 0.5 A, and 1 A. The result was multiple operating points for every monitored rail instead of checking the system under a single load.

The Python test software automated the acquisition of the EPICS values, measurement error calculation, application of acceptance limits, data saving and generation of the final PDF report. When operator observation was required, such as the verification of the information displayed on the local display, the result was recorded in the system test checklist.

D.1.15.2 Acceptance Criteria

The acceptance criteria formulated in the previous chapter were kept for the final system test. The main electrical acceptance criteria were:

- The IOC voltage measurements must agree with the external reference within ±2% across the whole test range.
- The IOC current measurements must agree with the external reference within ±5% across the whole intended range.
- For low current values, the bigger of the percentage tolerance and absolute error floor is used. The absolute floor is 25 mA for the 39 mΩ channels and 20 mA for the 50 mΩ channels.
- Positive and negative rail voltages must be represented correctly in terms of polarity.
- The local-display and EPICS values must represent the same underlying measurement within the display resolution and rounding.
- Programmable voltage and current limits must work correctly in terms of assertion and de-assertion.
- Persistent settings must survive the removal and restoration of the controller power.
- Local monitoring must operate during network interruption and the EPICS communication must be restored after Ethernet recovery.

These criteria provided the measurable limits by which the completed prototype could be evaluated instead of the subjective judgment that the system appeared to be working.

D.1.15.3 System Test Results

The system test of the completed system produced the overall result of **PASS**. All six rail measurement tests passed, and each of the twelve system level tests formulated in Chapter 6 also passed.

Table D.4 summarizes the final system test disposition.

Also, the latest accepted attempt for each individual rail passed, as summarized in Table D.5.

All required voltage samples satisfied the system-level voltage accuracy criterion, and all required current samples satisfied the current accuracy criterion, including the low current absolute tolerance when applicable. Both negative voltage channels reported the correct negative voltage polarity while preserving the positive current magnitude convention used by the firmware.

Table D.4: Final system test results.

Test ID	Test	Result
ST-01	Power On and Initialization	PASS
ST-02	Rail Identification	PASS
ST-03	Voltage Accuracy	PASS
ST-04	Current Accuracy	PASS
ST-05	Dynamic Load Response	PASS
ST-06	Local Display	PASS
ST-07	EPICS Communications	PASS
ST-08	Limit / Alarm Operation	PASS
ST-09	Persistent Settings	PASS
ST-10	Communication Recovery	PASS
ST-11	Repeatability	PASS
ST-12	End-To-End Demonstration	PASS

Table D.5: Final rail measurement results.

Rail	IOC Channel	Result
+5VA	5VA	PASS
+5VB	5VB	PASS
+5 V	P5V	PASS
−5 V	M5V	PASS
+15 V	P15V	PASS
−15 V	M15V	PASS

Dynamic load testing also passed on all six channels. The changes in electronic load current were reflected on the local display and on the corresponding EPICS process variables, and none of the channels became unresponsive during the load sweeps.

Local display testing confirmed that all six channel identifications, voltage values, current values, programmed limits, status indication, and communication information were visible and corresponded to the appropriate physical rail. All local display observations recorded during the test passed for every monitored channel.

EPICS communication testing confirmed that the IOC connected to the Arduino TCP server and that the required voltage and current process variables for all six rails were connected and updated. The load changes were observed by EPICS for both positive and negative channels, confirming the proper operation of the complete measurement and communication chain.

Limit testing confirmed that the system reacted correctly when the applied measurement crossed the programmed threshold. Both the local display and the EPICS status changed when the threshold was exceeded and returned to normal when the measurement came back inside the configured limits.

Persistent settings testing was performed by changing known user-adjustable settings, removing power from the monitoring electronics, restoring power, and comparing the recovered values with the values stored prior to the power cycling. The stored values were recovered correctly.

Communication recovery testing was performed by interrupting Ethernet communication while the monitor was running. The local GIGA display continued to operate during the interruption. After Ethernet was restored, the communication with the IOC was recovered without firmware reprogramming and the EPICS process variables started to update again with the valid values.

Repeatability testing was also passed. The measurements repeated at selected operating points, including after the power cycle, were within the established acceptance limits, and no intermittent sensor or communication problems were observed.

Finally, the end-to-end demonstration confirmed simultaneous operation of the complete hardware, firmware, local display, network interface, EPICS IOC, Phoebus interface and automated test software. The change of the load on the monitored channel produced the expected result locally and remotely without system-level problems.

D.1.15.4 Conclusions and Results

The final test results confirm that the completed prototype satisfied the main functional and measurement requirements established for the project. All twelve system level tests passed and all six monitoring channels passed their final voltage and current verification.

The voltage and current tests are especially important since they give the quantitative proof that the monitor implements its intended measurement function. The testing was performed at multiple operating points on each channel instead of the single nominal load case. This

gives the higher confidence that the measurement system works properly across the intended range of loads.

The test results also confirmed the functions that cannot be confirmed by measurement accuracy alone. Correct rail identification and polarity were confirmed, including the two negative voltage channels. The local graphical interface and EPICS interface displayed the same underlying information, programmable limits worked correctly, persistent settings were recovered from the storage, and the network communication was restored after the intentional interruption.

The conclusion regarding the operation of the system in the broad range of temperature, electromagnetic compatibility, vibration, mechanical shock, accelerated aging, and destructive fault conditions cannot be made since these qualification tests were intentionally excluded from the system test plan. The additional measurements are needed to make claims regarding performance under those conditions.

Some of the problems discovered during the board bring-up were fixed before the final system test. They included the SDA polarity problem in the isolated I^2C interface, pull-up and signal state problems, and startup sequencing problems. The successful final acceptance test confirms that the corrective modifications were effective and that these earlier development problems did not prevent the completed prototype from passing the final system test.

D.1.15.5 Recommendations and Improvement Opportunities

Despite the fact that the completed system satisfied the acceptance criteria, some improvements are recommended for the future PCB revision.

The most important hardware improvement is the incorporation of the SDA polarity fix directly in the PCB design. The prototype required the SN74LVC2G04 inverter to correct the SDA signal polarity after the originally installed LTC2945 device variant turned out to be incompatible with the intended isolation interface. The future revision should use the correct LTC2945 variant throughout the design or incorporate the required inversion directly in the schematic and PCB.

All required I^2C pull-up resistors and defined logic states should also be incorporated directly in the next PCB revision. It will eliminate the prototype modification and improve the repeatability of assembly and startup.

The firmware initialization can also be made more tolerant of arbitrary power-up sequencing. During the development, the communication could be failed when the controller tried to initialize the sensor before the corresponding isolated rail circuitry was powered. The future firmware revision could periodically retry the unavailable sensors so that the communication is established automatically when the monitored rail is powered.

The per-channel measurement calibration is another potential improvement. The present system calculates the current using the configured current sense resistance. The measurement of the actual resistance or the determination of the calibration factor against the external reference will allow the individual gain correction to be stored for each channel. It will help

to reduce the systematic errors caused by the shunt tolerance and allow tighter acceptance limits if the improved measurement accuracy is required.

The additional environmental characterization is also useful if the design is going to be developed beyond the capstone prototype. The testing at multiple ambient temperatures can be used to determine whether the temperature affects the measurement accuracy. EMC testing can determine the susceptibility to and generation of the electromagnetic interference. The long duration testing can be used to detect the intermittent communication, thermal, network or firmware problems that might not occur during the shorter acceptance test.

The automated test system should be retained for the future revisions. Since the raw measurements are saved separately from the report generation, the future hardware or firmware revisions can be tested using the same acceptance procedure and direct comparison between the revisions is possible.

These recommendations represent the refinements in reliability, manufacturability, calibration and qualification and not the changes to the basic architecture. The successful final test proves that the overall system concept is suitable for the further development.

D.1.15.6 Generated System Test Report

The automated System Test Report generated from the final accepted test data is attached below as the permanent part of the project test record. The report contains the detailed rail-by-rail measurements, measurement error plots, system test checklist, acceptance results, and engineering justification for the selected tolerances.

Multi-Rail Isolated Power Supply Monitor System Test Report

Overall system status	PASS	**Checklist file**	capstone_system_test_checklist.csv
Rail measurement status	PASS	**Raw measurement data**	capstone_rail_test_raw.csv
DUT	1	**Operator(s)**	MC
Latest-attempt date range	2026-10-18T18:34:42 to 2026-10-18T18:48:59	**IOC prefix**	TEST:GIGA
Electronic load	RIGOL TECHNOLOGIES, DL3021, DL3A284M00251	**Report generated**	2026-10-18T18:53:22

Acceptance criteria used

- **Voltage accuracy:** IOC voltage must agree with the external reference within ±2% across the tested operating range.
- **Current accuracy:** IOC current must agree with the external reference within ±5%. At low current, the larger of the percentage tolerance or an absolute floor is used: 25 mA for 39 mOhm channels and 20 mA for 50 mOhm channels.
- **Polarity:** Positive and negative rail voltages must have the correct sign. Current follows the firmware convention for all rails.
- **Display / EPICS agreement:** Local-display and EPICS values must represent the same underlying measurement within displayed resolution or rounding.
- **Limits and persistence:** Programmed limits must assert and clear correctly, and persistent settings must survive a power cycle.
- **Network recovery:** EPICS communication must resume normally after Ethernet communication is interrupted and restored.
- **Automated verification:** Rail accuracy, Channel Access connectivity, PV updates, and other electronically observable checks are derived automatically from the latest rail-test results.
- **Operator verification:** During each rail test, the operator is prompted only for local-display observations that cannot be measured automatically. These observations are reused for the applicable system-test checklist items.

System test matrix — latest result

ID	Test	Result	Attempt	Source / notes
ST-01	Power On and Initialization	PASS	1	4 auto + 6 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-02	Rail Identification	PASS	—	5 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-03	Voltage Accuracy	PASS	—	2 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-04	Current Accuracy	PASS	—	2 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-05	Dynamic Load Response	PASS	—	5 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-06	Local Display	PASS	—	5 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-07	EPICS Communications	PASS	—	8 auto: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-08	Limit / Alarm Operation	PASS	1	10 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-09	Persistent Settings	PASS	1	8 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-10	Communication Recovery	PASS	1	2 auto + 6 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-11	Repeatability	PASS	1	3 auto + 2 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.
ST-12	End-To-End Demonstration	PASS	1	5 auto + 2 checklist: All checklist steps are satisfied by automatic evidence and/or operator confirmation.

Disposition by rail (latest attempt only)

Rail	Attempt	IOC channel	Status	Start	End
+5VA	1	6VA	PASS	2026-10-18 18:34:42	2026-10-18 18:35:10
+5VB	2	6VB	PASS	2026-10-18 18:37:29	2026-10-18 18:37:57
+5V	1	P5V	PASS	2026-10-18 18:39:27	2026-10-18 18:39:55
-5V	1	M5V	PASS	2026-10-18 18:40:59	2026-10-18 18:41:28
+15V	1	P15V	PASS	2026-10-18 18:43:04	2026-10-18 18:43:32
-15V	3	M15V	PASS	2026-10-18 18:46:38	2026-10-18 18:47:06

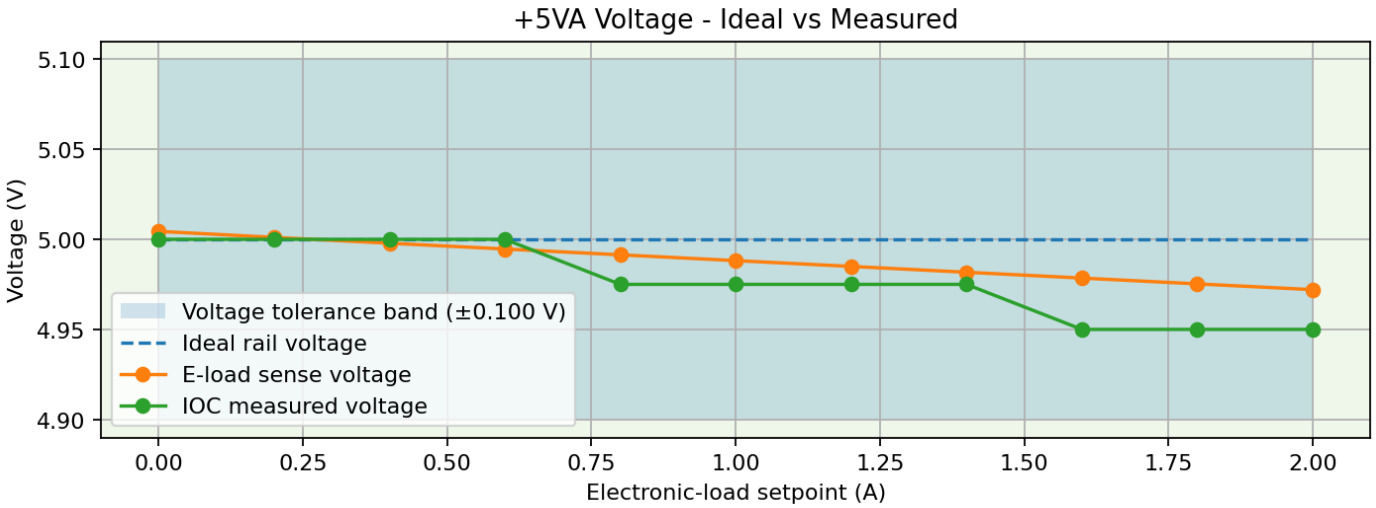
Local-display observations captured during rail testing

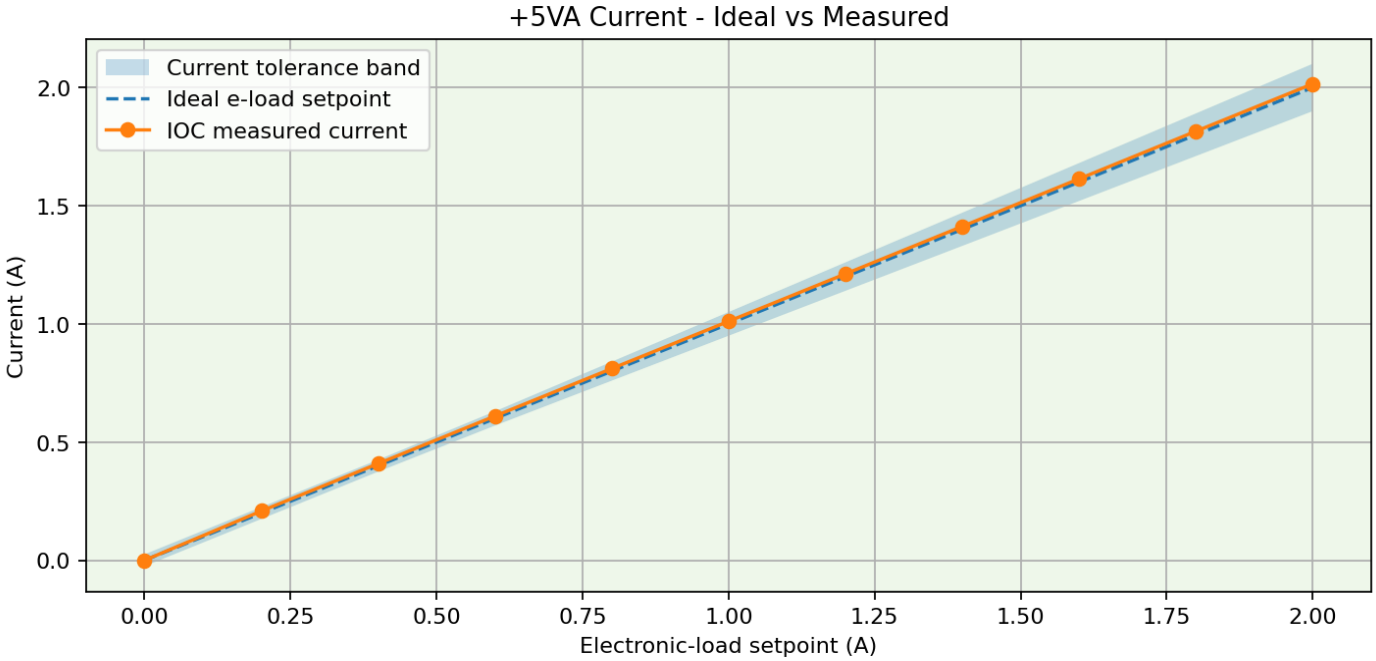
Rail	Identity / polarity	Updates with load	Agrees with EPICS	Limits / status	Network state
+5VA	PASS	PASS	PASS	PASS	PASS
+5VB	PASS	PASS	PASS	PASS	PASS
+5V	PASS	PASS	PASS	PASS	PASS
-5V	PASS	PASS	PASS	PASS	PASS
+15V	PASS	PASS	PASS	PASS	PASS
-15V	PASS	PASS	PASS	PASS	PASS

+5VA detailed results - latest attempt

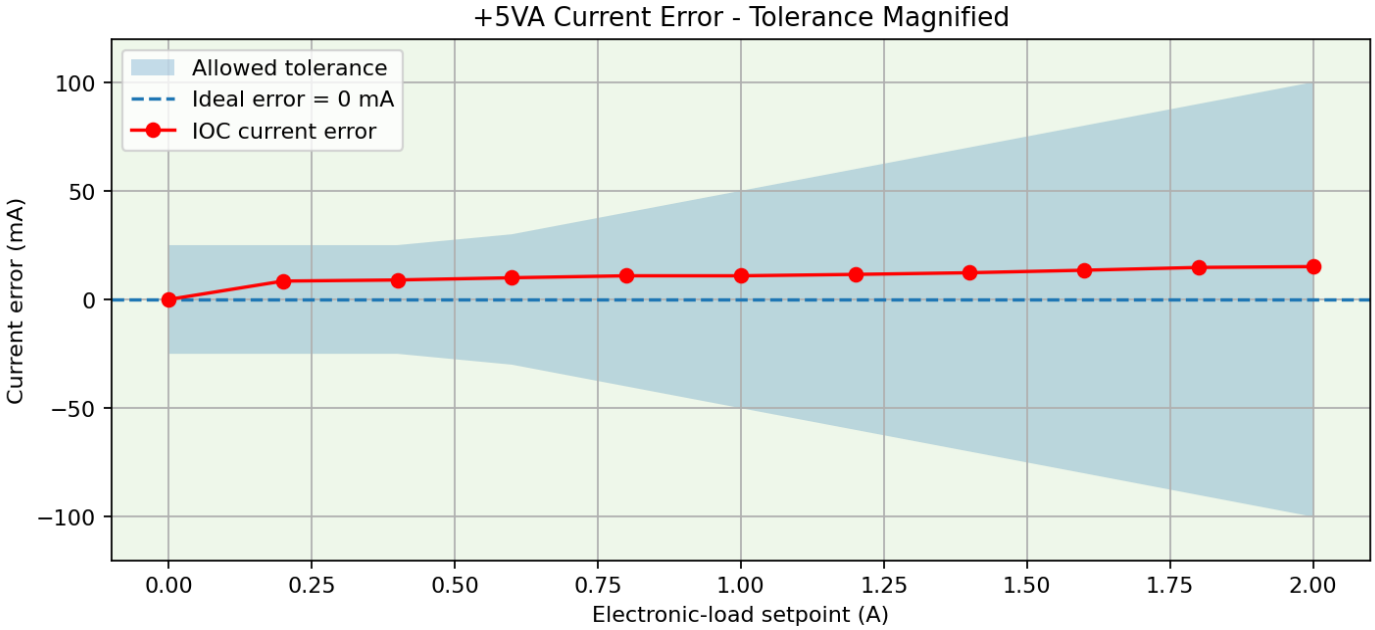
Attempt 1 — PASS IOC channel: 6VA Start: 2026-10-18T18:34:42 End: 2026-10-18T18:35:10

Ideal versus measured response





Current error detail

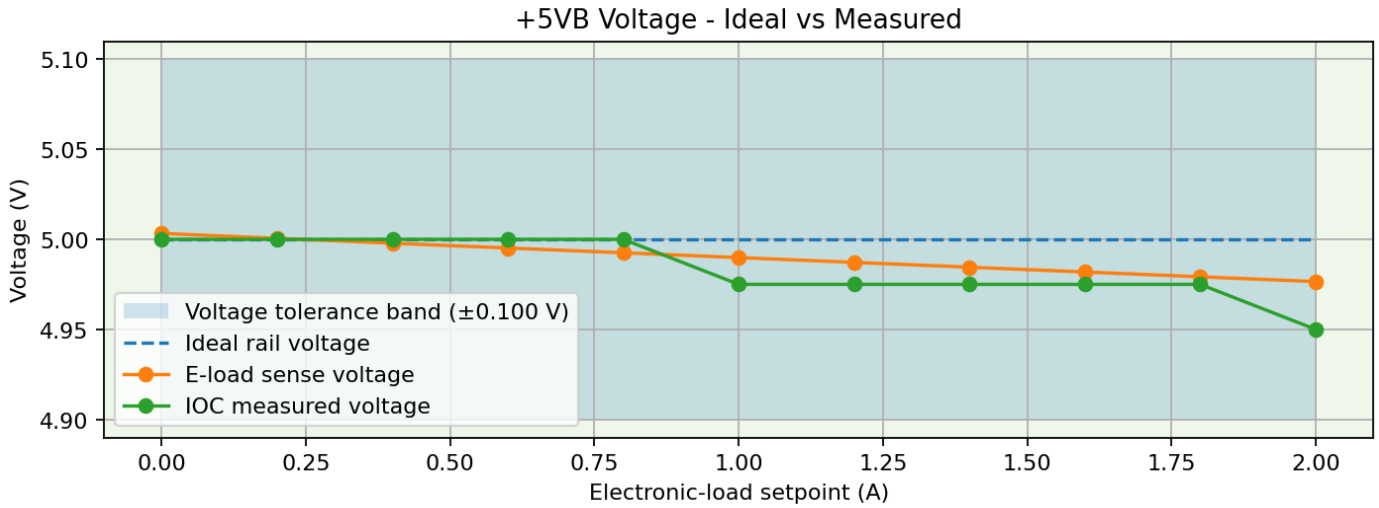


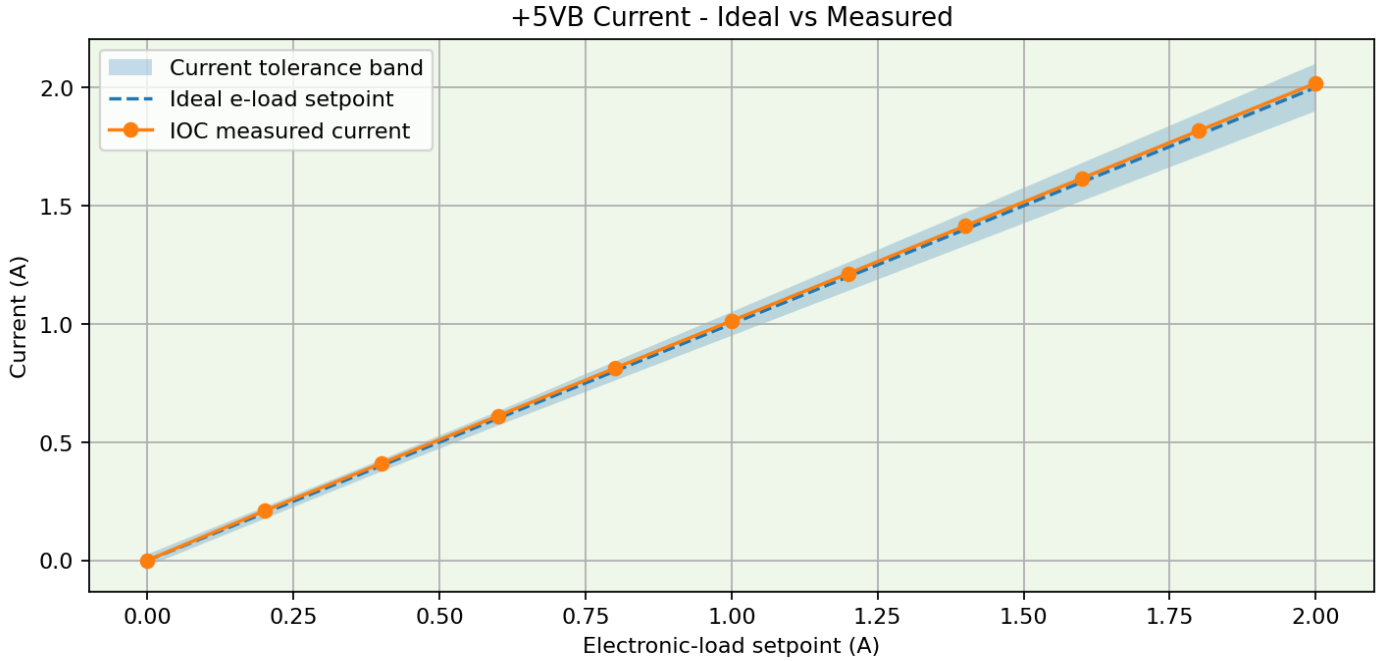
Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	5.0044	+5.0044	+5.0000	-0.0044	0.1001	0.0000	0.0000	+0.0000	0.0250	PASS
0.2000	5.0011	+5.0011	+5.0000	-0.0011	0.1000	0.1995	0.2085	+0.0090	0.0250	PASS
0.4000	4.9978	+4.9978	+5.0000	+0.0022	0.1000	0.3991	0.4090	+0.0099	0.0250	PASS
0.6000	4.9946	+4.9946	+5.0000	+0.0054	0.1000	0.5999	0.6100	+0.0101	0.0300	PASS
0.8000	4.9914	+4.9914	+4.9750	-0.0164	0.1000	0.8003	0.8109	+0.0106	0.0400	PASS
1.0000	4.9882	+4.9882	+4.9750	-0.0132	0.1000	0.9997	1.0109	+0.0112	0.0500	PASS
1.2000	4.9850	+4.9850	+4.9750	-0.0100	0.1000	1.1993	1.2115	+0.0123	0.0600	PASS
1.4000	4.9817	+4.9817	+4.9750	-0.0067	0.1000	1.3995	1.4123	+0.0128	0.0700	PASS
1.6000	4.9785	+4.9785	+4.9500	-0.0285	0.1000	1.5999	1.6135	+0.0135	0.0800	PASS
1.8000	4.9753	+4.9753	+4.9500	-0.0253	0.1000	1.8002	1.8147	+0.0145	0.0900	PASS
2.0000	4.9721	+4.9721	+4.9500	-0.0221	0.1000	1.9997	2.0151	+0.0154	0.1000	PASS

+5VB detailed results - latest attempt

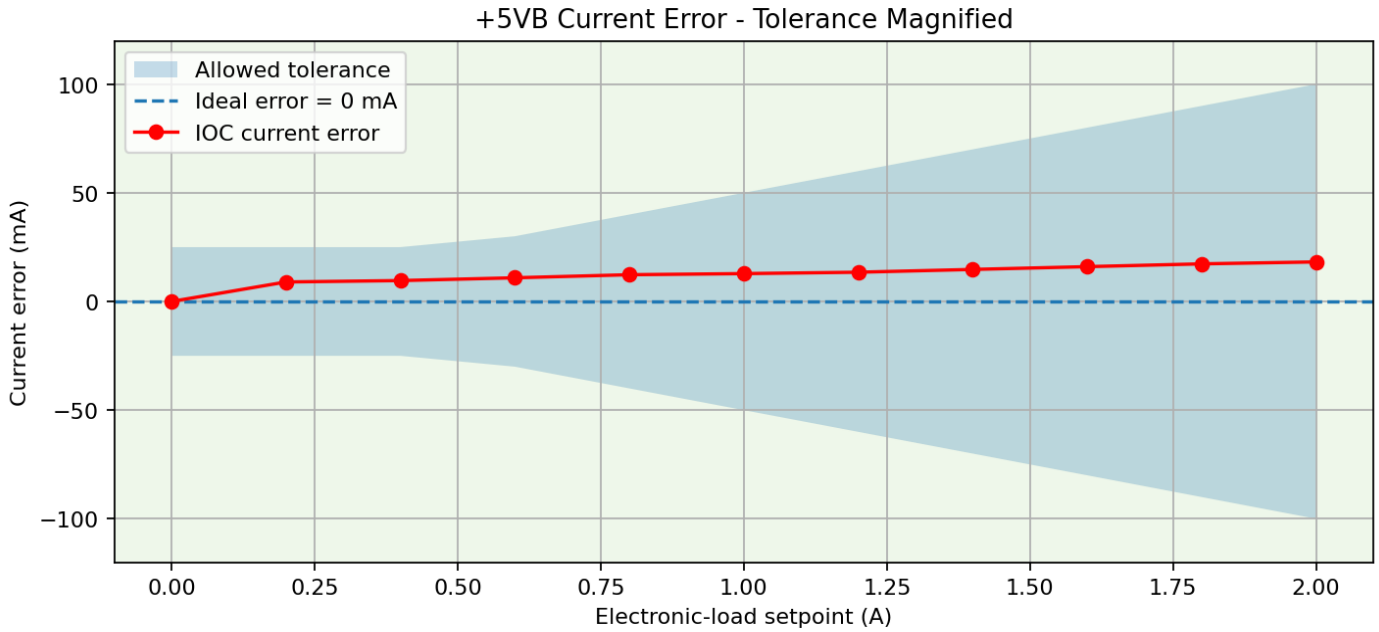
Attempt 2 — PASS IOC channel: 6VB Start: 2026-10-18T18:37:29 End: 2026-10-18T18:37:57

Ideal versus measured response





Current error detail

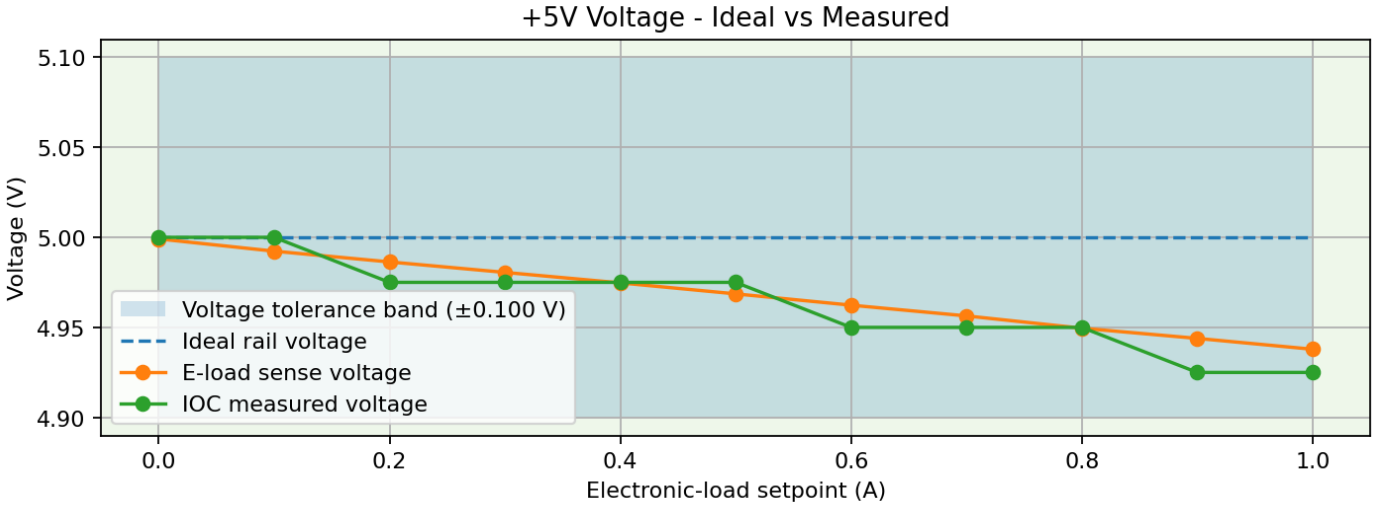


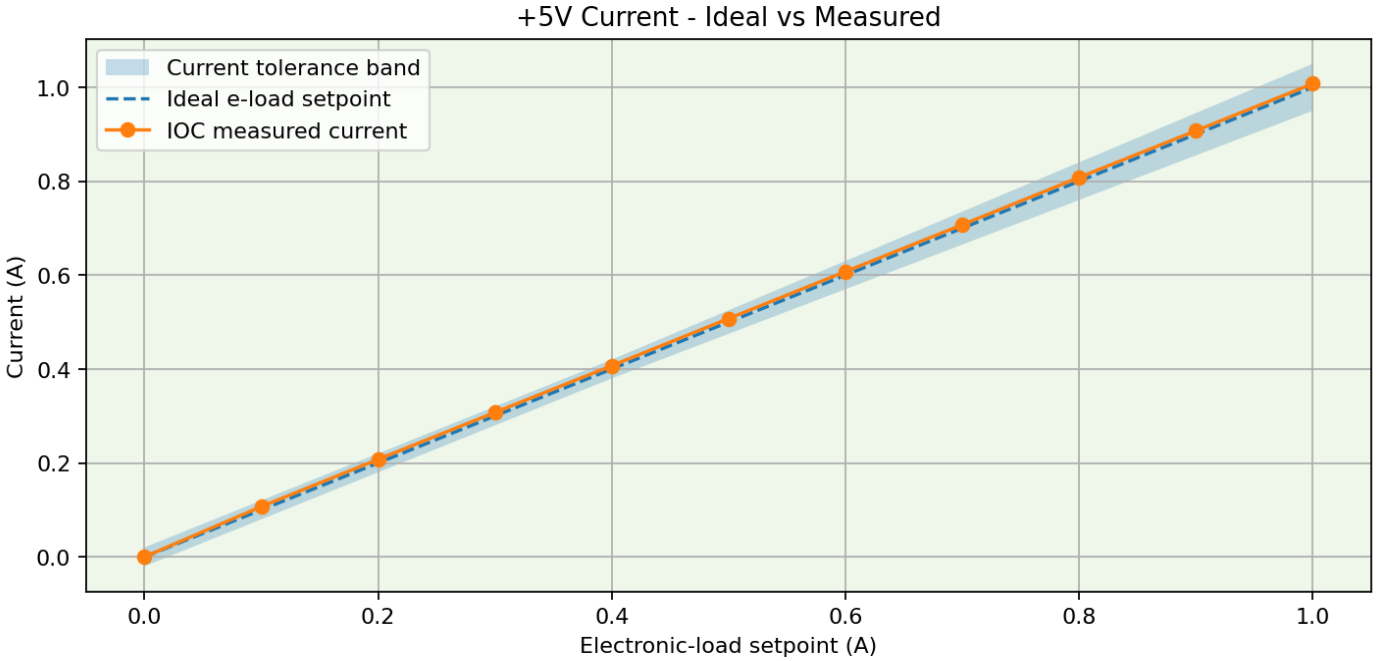
Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	5.0033	+5.0033	+5.0000	-0.0033	0.1001	0.0000	0.0000	+0.0000	0.0250	PASS
0.2000	5.0005	+5.0005	+5.0000	-0.0005	0.1000	0.1993	0.2090	+0.0096	0.0250	PASS
0.4000	4.9979	+4.9979	+5.0000	+0.0021	0.1000	0.3990	0.4096	+0.0106	0.0250	PASS
0.6000	4.9952	+4.9952	+5.0000	+0.0048	0.1000	0.5997	0.6109	+0.0112	0.0300	PASS
0.8000	4.9925	+4.9925	+5.0000	+0.0075	0.1000	0.8002	0.8123	+0.0121	0.0400	PASS
1.0000	4.9899	+4.9899	+4.9750	-0.0149	0.1000	0.9995	1.0128	+0.0133	0.0500	PASS
1.2000	4.9872	+4.9872	+4.9750	-0.0122	0.1000	1.1991	1.2135	+0.0143	0.0600	PASS
1.4000	4.9845	+4.9845	+4.9750	-0.0095	0.1000	1.3994	1.4147	+0.0153	0.0700	PASS
1.6000	4.9819	+4.9819	+4.9750	-0.0069	0.1000	1.5998	1.6160	+0.0162	0.0800	PASS
1.8000	4.9792	+4.9792	+4.9750	-0.0042	0.1000	1.8002	1.8173	+0.0171	0.0900	PASS
2.0000	4.9765	+4.9765	+4.9500	-0.0265	0.1000	1.9997	2.0182	+0.0185	0.1000	PASS

+5V detailed results - latest attempt

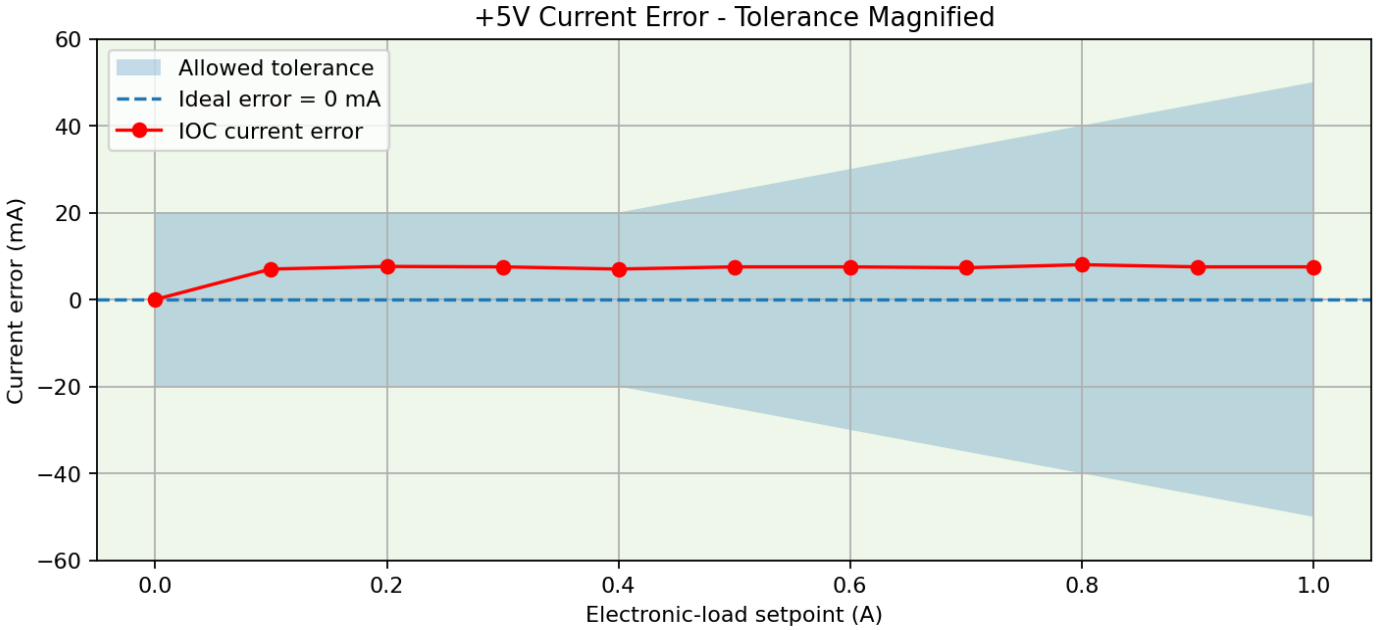
Attempt 1 — PASS IOC channel: P5V Start: 2026-10-18T18:39:27 End: 2026-10-18T18:39:55

Ideal versus measured response





Current error detail

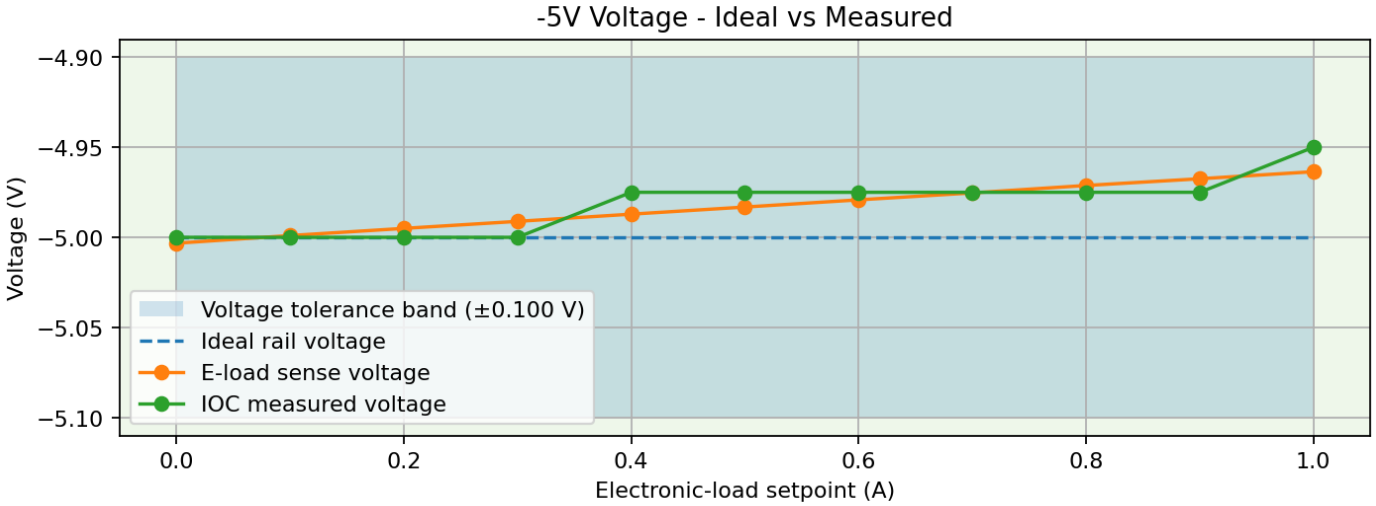


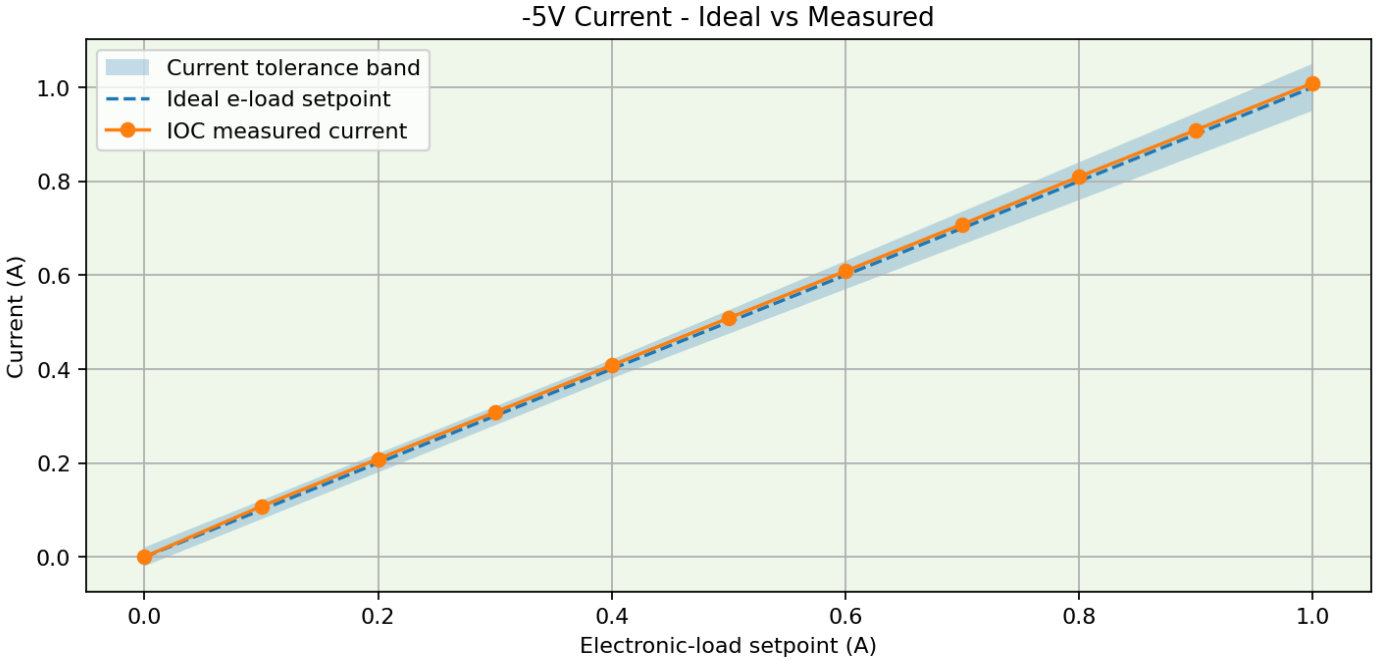
Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	4.9991	+4.9991	+5.0000	+0.0009	0.1000	0.0000	0.0000	+0.0000	0.0200	PASS
0.1000	4.9923	+4.9923	+5.0000	+0.0077	0.1000	0.0986	0.1070	+0.0084	0.0200	PASS
0.2000	4.9863	+4.9863	+4.9750	-0.0113	0.1000	0.1995	0.2076	+0.0081	0.0200	PASS
0.3000	4.9805	+4.9805	+4.9750	-0.0055	0.1000	0.2992	0.3075	+0.0083	0.0200	PASS
0.4000	4.9747	+4.9747	+4.9750	+0.0003	0.1000	0.3991	0.4070	+0.0079	0.0200	PASS
0.5000	4.9686	+4.9686	+4.9750	+0.0064	0.1000	0.4996	0.5075	+0.0079	0.0250	PASS
0.6000	4.9623	+4.9623	+4.9500	-0.0123	0.1000	0.5999	0.6075	+0.0076	0.0300	PASS
0.7000	4.9564	+4.9564	+4.9500	-0.0064	0.1000	0.6996	0.7073	+0.0077	0.0350	PASS
0.8000	4.9496	+4.9496	+4.9500	+0.0004	0.1000	0.8003	0.8080	+0.0077	0.0400	PASS
0.9000	4.9439	+4.9439	+4.9250	-0.0189	0.1000	0.8998	0.9075	+0.0077	0.0450	PASS
1.0000	4.9379	+4.9379	+4.9250	-0.0129	0.1000	0.9996	1.0075	+0.0079	0.0500	PASS

-5V detailed results - latest attempt

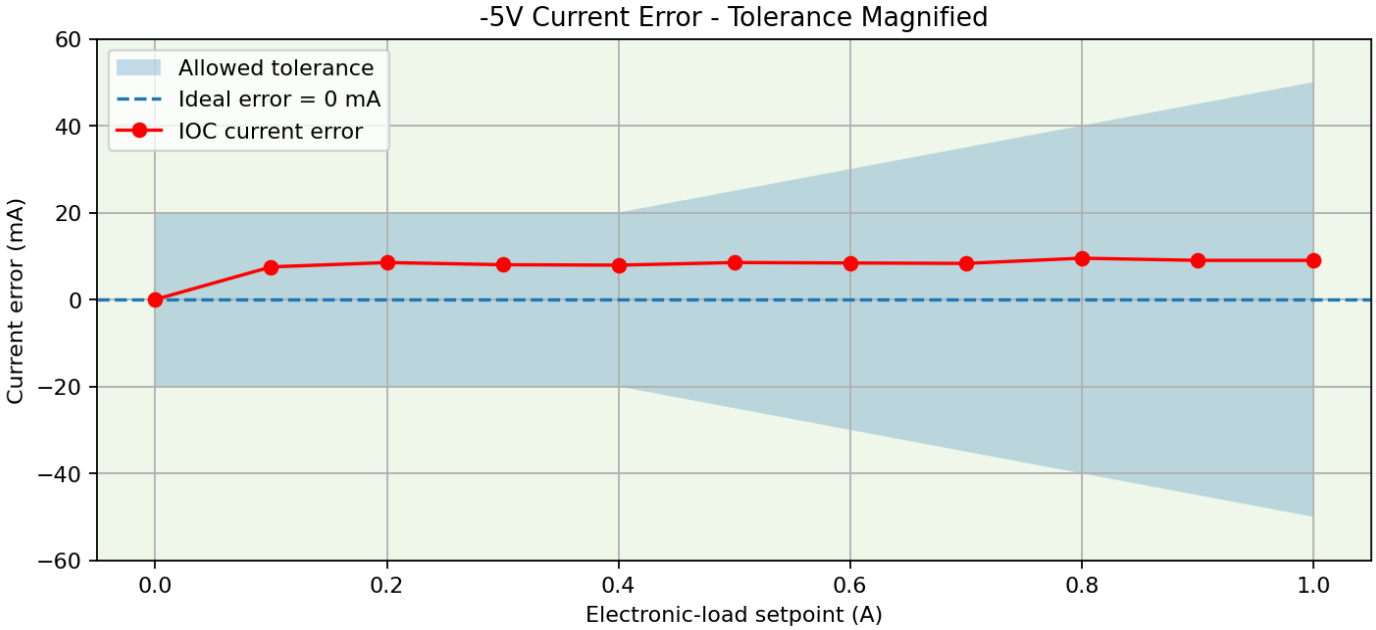
Attempt 1 — PASS IOC channel: M5V Start: 2026-10-18T18:40:59 End: 2026-10-18T18:41:28

Ideal versus measured response





Current error detail

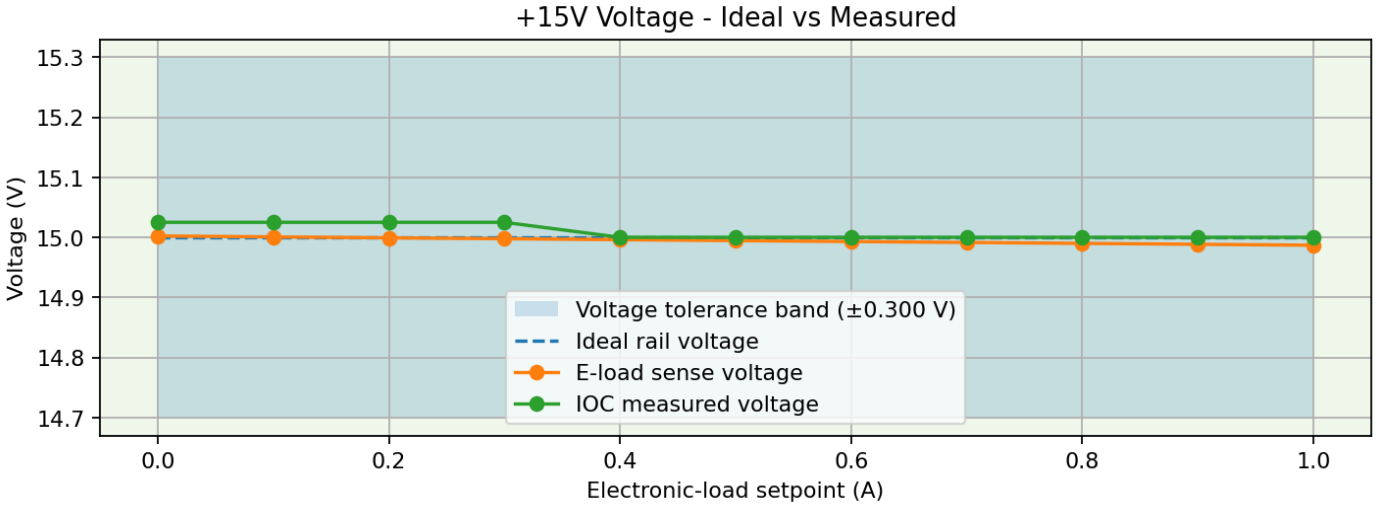


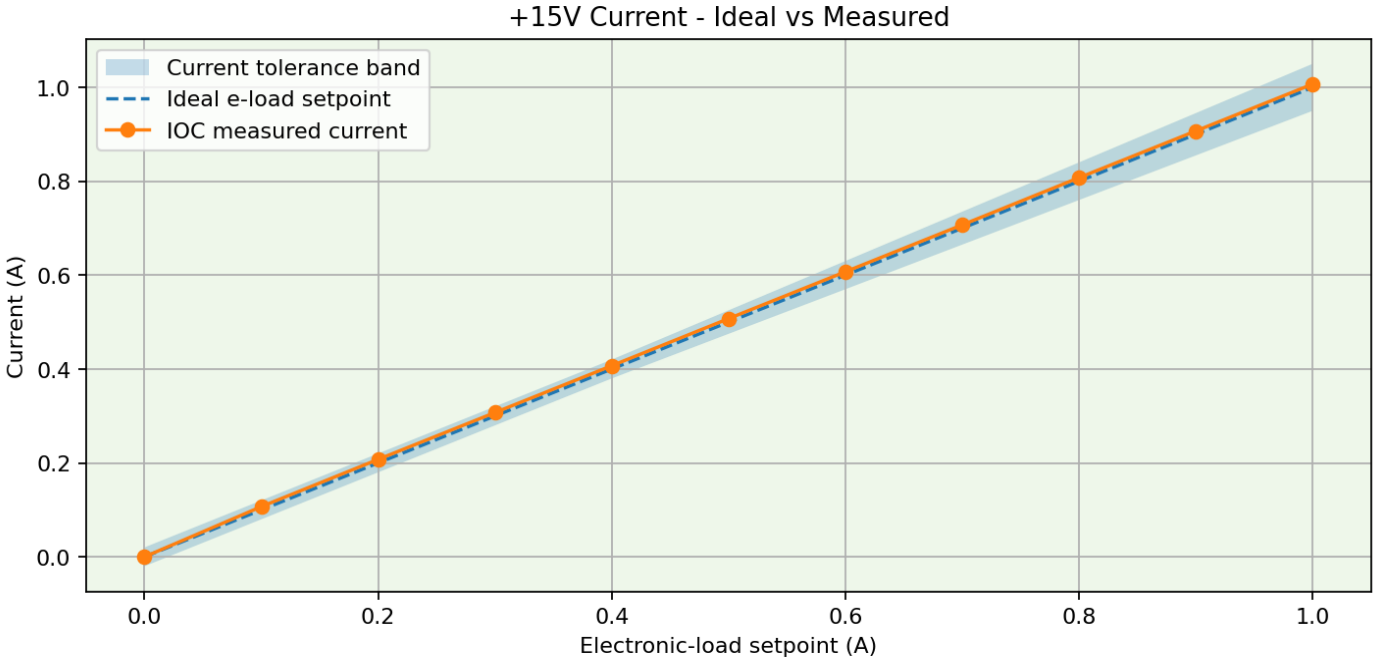
Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	5.0033	-5.0033	-5.0000	+0.0033	0.1001	0.0000	0.0000	+0.0000	0.0200	PASS
0.1000	4.9990	-4.9990	-5.0000	-0.0010	0.1000	0.0986	0.1075	+0.0089	0.0200	PASS
0.2000	4.9950	-4.9950	-5.0000	-0.0050	0.1000	0.1995	0.2085	+0.0090	0.0200	PASS
0.3000	4.9911	-4.9911	-5.0000	-0.0089	0.1000	0.2993	0.3080	+0.0087	0.0200	PASS
0.4000	4.9871	-4.9871	-4.9750	+0.0121	0.1000	0.3991	0.4079	+0.0088	0.0200	PASS
0.5000	4.9832	-4.9832	-4.9750	+0.0082	0.1000	0.4997	0.5085	+0.0088	0.0250	PASS
0.6000	4.9792	-4.9792	-4.9750	+0.0042	0.1000	0.5998	0.6084	+0.0086	0.0300	PASS
0.7000	4.9753	-4.9753	-4.9750	+0.0003	0.1000	0.6996	0.7083	+0.0087	0.0350	PASS
0.8000	4.9712	-4.9712	-4.9750	-0.0038	0.1000	0.8003	0.8095	+0.0092	0.0400	PASS
0.9000	4.9675	-4.9675	-4.9750	-0.0075	0.1000	0.8998	0.9090	+0.0092	0.0450	PASS
1.0000	4.9636	-4.9636	-4.9500	+0.0136	0.1000	0.9997	1.0090	+0.0093	0.0500	PASS

+15V detailed results - latest attempt

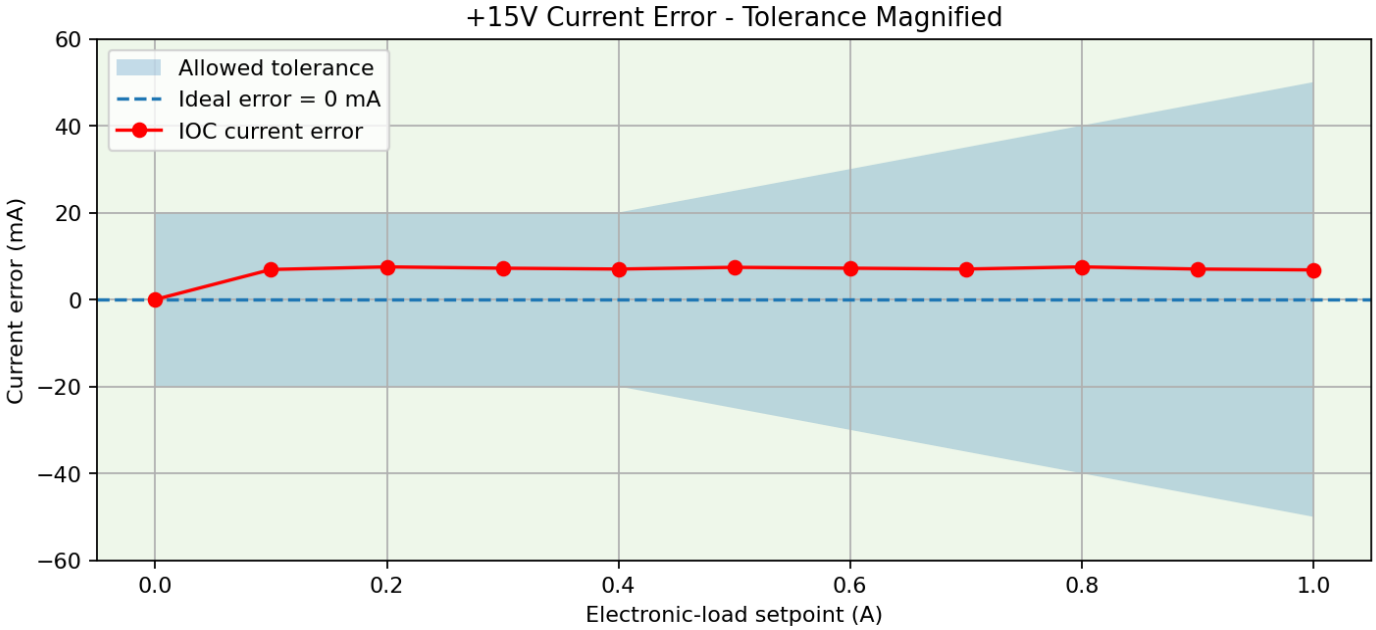
Attempt 1 — PASS IOC channel: P15V Start: 2026-10-18T18:43:04 End: 2026-10-18T18:43:32

Ideal versus measured response





Current error detail

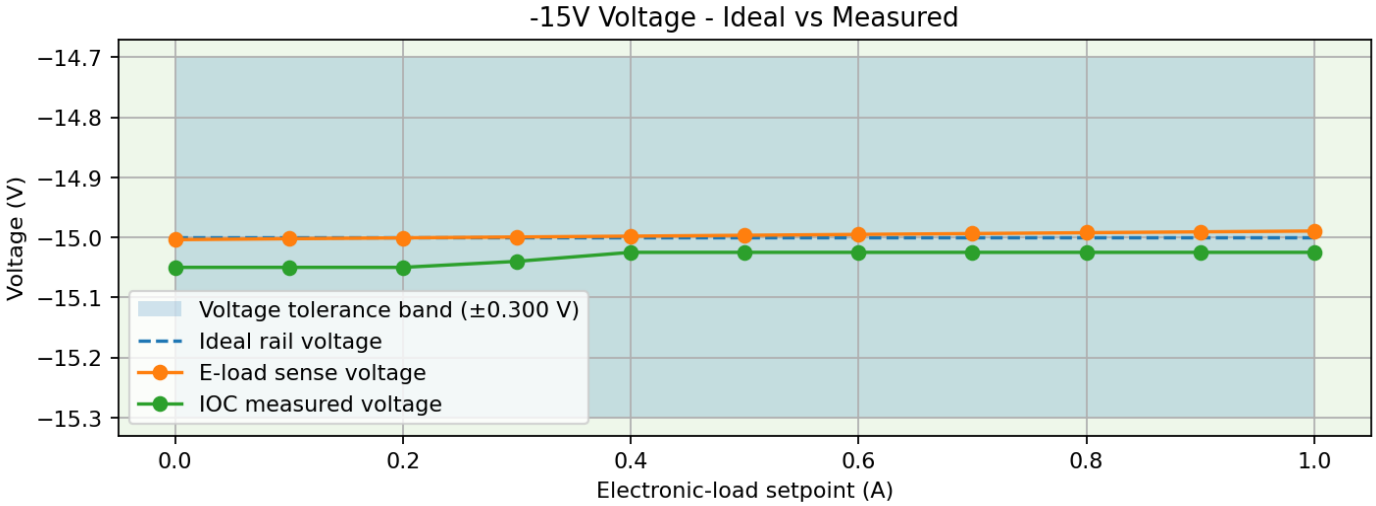


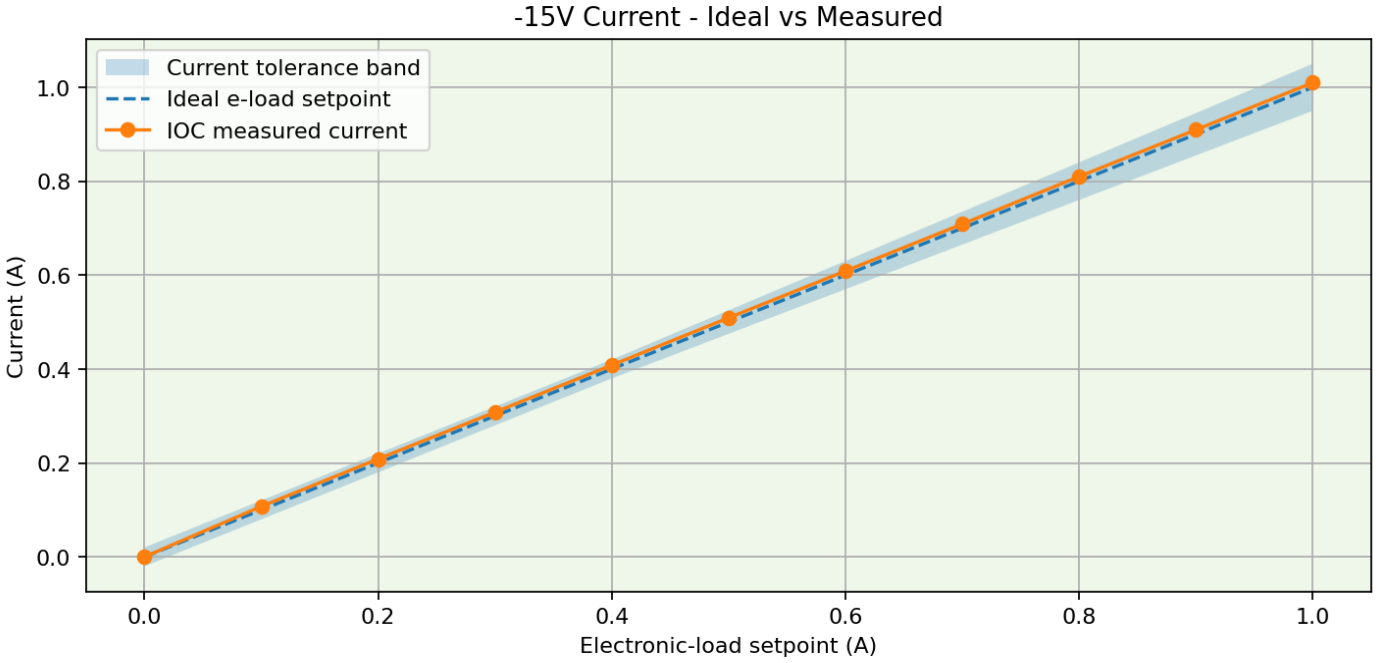
Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	15.0023	+15.0023	+15.0250	+0.0227	0.3000	0.0000	0.0000	+0.0000	0.0200	PASS
0.1000	15.0007	+15.0007	+15.0250	+0.0243	0.3000	0.0985	0.1069	+0.0084	0.0200	PASS
0.2000	14.9992	+14.9992	+15.0250	+0.0258	0.3000	0.1995	0.2075	+0.0080	0.0200	PASS
0.3000	14.9977	+14.9977	+15.0250	+0.0273	0.3000	0.2993	0.3072	+0.0079	0.0200	PASS
0.4000	14.9961	+14.9961	+15.0000	+0.0039	0.2999	0.3991	0.4070	+0.0079	0.0200	PASS
0.5000	14.9945	+14.9945	+15.0000	+0.0055	0.2999	0.4997	0.5074	+0.0077	0.0250	PASS
0.6000	14.9930	+14.9930	+15.0000	+0.0070	0.2999	0.5999	0.6072	+0.0073	0.0300	PASS
0.7000	14.9915	+14.9915	+15.0000	+0.0085	0.2998	0.6996	0.7070	+0.0074	0.0350	PASS
0.8000	14.9900	+14.9900	+15.0000	+0.0100	0.2998	0.8003	0.8075	+0.0072	0.0400	PASS
0.9000	14.9884	+14.9884	+15.0000	+0.0116	0.2998	0.8999	0.9070	+0.0071	0.0450	PASS
1.0000	14.9868	+14.9868	+15.0000	+0.0132	0.2997	0.9997	1.0068	+0.0071	0.0500	PASS

-15V detailed results - latest attempt

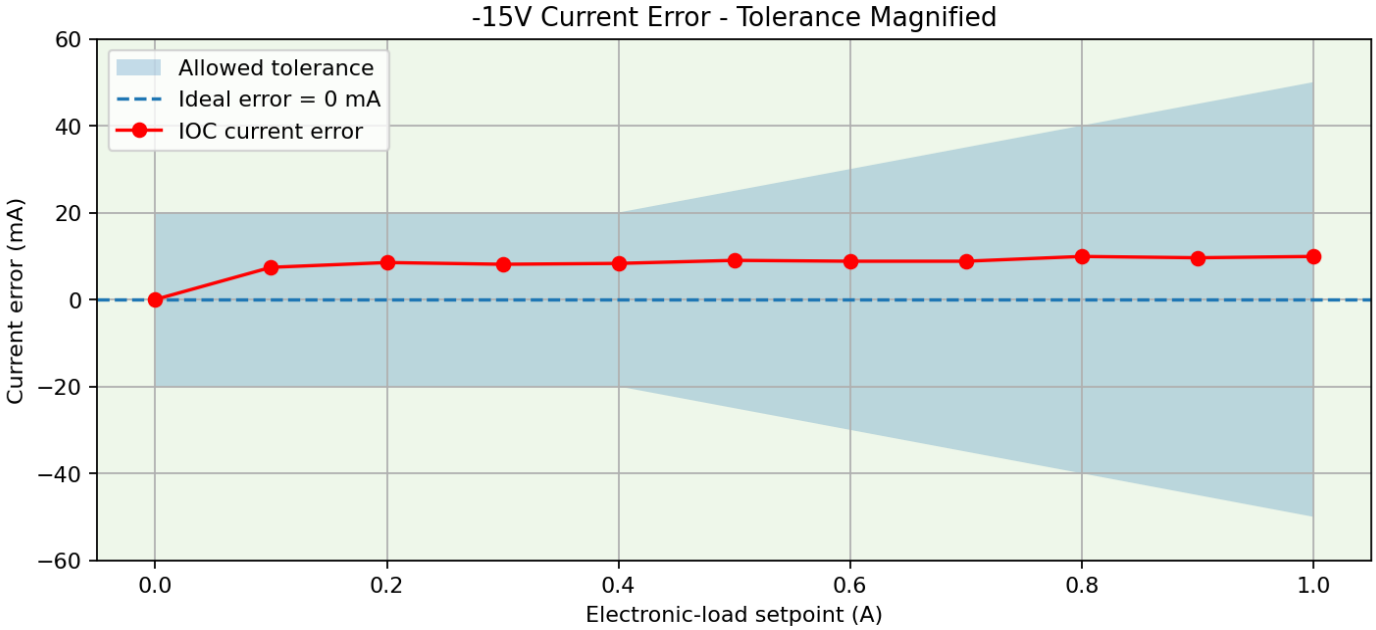
Attempt 3 — PASS IOC channel: M15V Start: 2026-10-18T18:46:38 End: 2026-10-18T18:47:06

Ideal versus measured response





Current error detail



Set I (A)	Rigol V (V)	V ref (V)	IOC V (V)	V err (V)	V tol (V)	Rigol I (A)	IOC I (A)	I err (A)	I tol (A)	Result
0.0000	15.0039	-15.0039	-15.0500	-0.0461	0.3001	0.0000	0.0000	+0.0000	0.0200	PASS
0.1000	15.0023	-15.0023	-15.0500	-0.0477	0.3000	0.0985	0.1074	+0.0089	0.0200	PASS
0.2000	15.0009	-15.0009	-15.0500	-0.0491	0.3000	0.1995	0.2085	+0.0090	0.0200	PASS
0.3000	14.9994	-14.9994	-15.0400	-0.0406	0.3000	0.2992	0.3081	+0.0089	0.0200	PASS
0.4000	14.9981	-14.9981	-15.0250	-0.0269	0.3000	0.3992	0.4083	+0.0091	0.0200	PASS
0.5000	14.9966	-14.9966	-15.0250	-0.0284	0.2999	0.4997	0.5090	+0.0093	0.0250	PASS
0.6000	14.9952	-14.9952	-15.0250	-0.0298	0.2999	0.5998	0.6088	+0.0090	0.0300	PASS
0.7000	14.9938	-14.9938	-15.0250	-0.0312	0.2999	0.6996	0.7088	+0.0092	0.0350	PASS
0.8000	14.9923	-14.9923	-15.0250	-0.0327	0.2998	0.8003	0.8099	+0.0096	0.0400	PASS
0.9000	14.9909	-14.9909	-15.0250	-0.0341	0.2998	0.8999	0.9096	+0.0097	0.0450	PASS
1.0000	14.9895	-14.9895	-15.0250	-0.0355	0.2998	0.9997	1.0099	+0.0102	0.0500	PASS

System test checklist — merged automatic/manual evidence

Each checklist step is filled from automatic rail-test evidence whenever the software can prove it. Only items that require visual/operator judgment remain manual. The source column identifies whether the current result came from automatic evidence, the latest checklist entry, or is still pending.

ST-01: Power On and Initialization — PASS

Method: Apply power to the complete system and verify Arduino, display, sensor channels, and network operation.
Pass criterion: The system initializes properly and all six channels are operational.

#	Checklist item	Result	Source / evidence
1	All power, sensor, display, and Ethernet connections have been inspected.	PASS	CHECKLIST
2	The electronic load is disabled before power-up.	PASS	CHECKLIST
3	The required monitored power supplies are energized.	PASS	CHECKLIST
4	Power has been applied to the Arduino and monitoring electronics.	PASS	CHECKLIST
5	The Arduino startup sequence completed normally.	PASS	CHECKLIST
6	The GIGA Display initialized.	PASS	AUTO: The operator interacted with the initialized GIGA Display during rail testing.
7	All six expected rail channels are displayed.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
8	The EPICS IOC is running.	PASS	AUTO: A completed rail test required a running EPICS IOC.
9	The IOC established communication with the monitor.	PASS	AUTO: A completed rail test required successful IOC/monitor communication.
10	No startup error or unexpected behavior requiring manual recovery was observed.	PASS	CHECKLIST

ST-02: Rail Identification — PASS

Method: Correlate each displayed channel with its physical rail connection and polarity.
Pass criterion: The correct rail identification and polarity are shown for each connected rail.

#	Checklist item	Result	Source / evidence
1	All six monitored rails are energized and connected to their intended monitor inputs.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
2	Each physical rail connection was correlated with the corresponding displayed channel.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
3	The +5VA, +5VB, +5V, -5V, +15V, and -15V rail identifications are correct.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
4	Positive and negative rail voltage polarity is reported correctly.	PASS	AUTO: Recorded IOC voltage data has the correct polarity on all six latest rail tests.
5	Negative-rail current follows the sign convention used by the firmware.	PASS	AUTO: Both negative-rail IOC currents follow the positive load-current magnitude convention.

ST-03: Voltage Accuracy — PASS

Method: Measure each rail with the reference instrument and compare it with the EPICS monitor value at multiple operating points.
Pass criterion: Voltage error is within ±2% across the full tested operating range.

#	Checklist item	Result	Source / evidence
1	All six rails have a completed latest voltage-measurement attempt.	PASS	AUTO: All six latest rail attempts satisfy the configured voltage acceptance criterion.
2	Every required voltage sample in the latest attempts meets the ±2% system-level criterion.	PASS	AUTO: All six latest rail attempts satisfy the configured voltage acceptance criterion.

ST-04: Current Accuracy — PASS

Method: Apply multiple electronic-load currents to each rail and compare the reference current with the monitor measurement.
Pass criterion: Current error is within ±5% across the full intended operating range; reference uncertainty/resolution is considered at low current.

#	Checklist item	Result	Source / evidence
1	All six rails have a completed latest current-measurement attempt.	PASS	AUTO: All six latest rail attempts satisfy the configured current acceptance criterion.
2	Every required current sample in the latest attempts meets the ±5% system-level criterion, with the configured low-current floor.	PASS	AUTO: All six latest rail attempts satisfy the configured current acceptance criterion.

ST-05: Dynamic Load Response — PASS

Method: Change electronic-load current while monitoring the local display and EPICS values.
Pass criterion: Current and voltage readings change according to the load change.

#	Checklist item	Result	Source / evidence
1	The electronic-load current was changed over a safe range while the system was operating.	PASS	AUTO: Automated current sweeps were recorded on: +5VA, +5VB, +5V, -5V, +15V, -15V.
2	The local display current reading changed in response to the load change.	PASS	AUTO: Confirmed during rail test(s): +5VA, +5VB, +5V, -5V, +15V, -15V
3	The EPICS current reading changed in response to the load change.	PASS	AUTO: EPICS current followed the programmed load sweep on: +5VA, +5VB, +5V, -5V, +15V, -15V.
4	The local and EPICS voltage readings responded appropriately to any rail-voltage change under load.	PASS	AUTO: Confirmed during rail test(s): +5VA, +5VB, +5V, -5V, +15V, -15V
5	No channel became unresponsive or produced an intermittent communication failure during the load change.	PASS	AUTO: All six latest rail tests completed without a Channel Access read failure.

ST-06: Local Display — PASS

Method: Observe voltage, current, rail identification, limits, and status for all channels.
Pass criterion: All necessary information is visible and correlates with the connected rail.

#	Checklist item	Result	Source / evidence
1	All six rail identifications are visible on the GIGA Display.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
2	Voltage is visible for each rail and corresponds with the connected rail.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
3	Current is visible for each rail and corresponds with the connected rail.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
4	Configured limits/status indications are visible for each rail.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
5	Communication/network state is visible and reasonable for the current operating condition.	PASS	AUTO: Confirmed during the latest automated test of all six rails.

ST-07: EPICS Communications — PASS

Method: Run the IOC and verify all required voltage, current, and status process variables.
Pass criterion: Required process variables are connected and update properly; local display and EPICS agree within displayed resolution.

#	Checklist item	Result	Source / evidence
1	The complete monitor is powered and connected to Ethernet.	PASS	AUTO: All six latest rail tests completed with a live Ethernet/IOC connection.
2	The EPICS IOC is running.	PASS	AUTO: A completed rail test required the EPICS IOC to be running.
3	The IOC connects to the Arduino TCP server.	PASS	AUTO: The IOC connection check and rail PV resolution succeeded during testing.
4	Voltage and current process variables for all six rails are connected and updating.	PASS	AUTO: Voltage/current PVs for all six rails were resolved and sampled successfully.
5	EPICS values have been compared with the local display and agree within displayed resolution/rounding.	PASS	AUTO: Confirmed during the latest automated test of all six rails.
6	A load change on a selected rail caused the corresponding EPICS voltage/current values to update.	PASS	AUTO: Recorded EPICS current values followed at least one programmed load sweep.
7	The load-change/update check was completed on at least one positive rail.	PASS	AUTO: A positive rail completed a multi-point load sweep with valid EPICS updates.
8	The load-change/update check was completed on at least one negative rail.	PASS	AUTO: A negative rail completed a multi-point load sweep with valid EPICS updates.

ST-08: Limit / Alarm Operation — PASS

Method: Set a safe test limit and change load or supply until the threshold is crossed, then return inside limits.
Pass criterion: The proper limit indication is created when the threshold is crossed and clears after the value returns inside the limits.

#	Checklist item	Result	Source / evidence
1	A rail was selected for the limit/status test.	PASS	CHECKLIST
2	The existing voltage/current limit settings were recorded before the test.	PASS	CHECKLIST
3	A safe test limit was configured without exceeding the rating of the monitored supply, sensor, wiring, or electronic load.	PASS	CHECKLIST
4	Normal status was verified while the measurement remained inside the limit.	PASS	CHECKLIST
5	The applied load or supply was changed until the selected threshold was crossed.	PASS	CHECKLIST

#	Checklist item	Result	Source / evidence
6	The local status indication changed appropriately when the threshold was crossed.	PASS	CHECKLIST
7	The corresponding EPICS status changed appropriately when the threshold was crossed.	PASS	CHECKLIST
8	The measurement was returned to the normal operating range.	PASS	CHECKLIST
9	Local and EPICS status returned to normal.	PASS	CHECKLIST
10	The original limit setting was restored.	PASS	CHECKLIST

ST-09: Persistent Settings — PASS

Method: Change user-adjustable settings, power-cycle the controller/monitor electronics, and verify the settings after restart.
Pass criterion: Settings intended to be persistent are restored without Ethernet communication or manual re-entry.

#	Checklist item	Result	Source / evidence
1	One or more user-adjustable settings were changed to known values.	PASS	CHECKLIST
2	The firmware accepted the changed settings.	PASS	CHECKLIST
3	Power was removed from the Arduino and monitor electronics.	PASS	CHECKLIST
4	The system remained unpowered for several seconds.	PASS	CHECKLIST
5	Power was restored.	PASS	CHECKLIST
6	Firmware initialization completed normally after power restoration.	PASS	CHECKLIST
7	The previously modified settings were read after restart.	PASS	CHECKLIST
8	The restored settings match the values stored before power removal.	PASS	CHECKLIST

ST-10: Communication Recovery — PASS

Method: Disconnect Ethernet while the system is running, reconnect it, and monitor local operation and the IOC.
Pass criterion: Local monitoring continues during the network interruption and normal EPICS communication resumes after reconnection without firmware reprogramming.

#	Checklist item	Result	Source / evidence
1	Normal communication between the monitor and IOC was established before the interruption.	PASS	AUTO: Normal monitor/IOC communication was established during rail testing.
2	EPICS values were verified to be updating before the interruption.	PASS	AUTO: EPICS values were sampled successfully before any communication-recovery test.
3	The Ethernet cable/network connection was disconnected while the system was running.	PASS	CHECKLIST
4	Local monitoring and the GIGA Display continued operating during the Ethernet interruption.	PASS	CHECKLIST
5	The IOC response to communication loss was observed and was reasonable.	PASS	CHECKLIST
6	Ethernet was reconnected.	PASS	CHECKLIST
7	Communication was re-established without firmware reprogramming.	PASS	CHECKLIST
8	Process variables resumed updating with valid values.	PASS	CHECKLIST

ST-11: Repeatability — PASS

Method: Make repeated measurements at selected points, including after a power cycle.
Pass criterion: Measurements remain within the specified acceptance limits with no intermittent channel failures.

#	Checklist item	Result	Source / evidence
1	Repeated measurements were made at selected operating points.	PASS	AUTO: Multiple completed measurement attempts exist for: -15V, +5VB.
2	Repeated measurements were made after at least one power cycle.	N/A	CHECKLIST
3	Repeated voltage measurements remained within the specified acceptance limits.	PASS	AUTO: Repeated voltage samples remain within the configured acceptance limit.
4	Repeated current measurements remained within the specified acceptance limits.	PASS	CHECKLIST
5	No intermittent sensor or communication failures were observed during repeatability testing.	PASS	AUTO: Repeated attempts completed without an acquisition/communication abort.

ST-12 End-To-End Demonstration — PASS

Method: Operate the complete system and demonstrate local and remote monitoring while changing the load on a selected channel.
Pass criterion: Hardware, firmware, display, network, IOC, and test software operate cooperatively with no system-level failure.

#	Checklist item	Result	Source / evidence
1	The complete hardware and firmware system is operating simultaneously.	PASS	AUTO: The complete measurement chain has been exercised successfully on all six rails.
2	The local GIGA Display is operating and showing valid rail information.	PASS	AUTO: Local-display identity and update behavior were confirmed on all six latest rail tests.
3	Ethernet and the EPICS IOC are connected and updating.	PASS	AUTO: Ethernet/IOC communication was active while all six latest rail tests were acquired.
4	Remote monitoring through EPICS/Phoebus is operating.	PASS	CHECKLIST
5	The test software can acquire/record data from the system.	PASS	AUTO: The Python test software successfully acquired and saved measurement data.
6	Changing the load on a selected channel is reflected in both local and remote monitoring.	PASS	AUTO: A load sweep was reflected in recorded EPICS data and confirmed on the local display.
7	The end-to-end demonstration completed without a system-level failure.	PASS	CHECKLIST

Test completion criteria

Completion criterion	Status
Every test identified in the system test matrix has been executed.	YES
All six rail channels have completed voltage and current verification.	YES
All system-test items have an effective PASS result.	YES
Failed/incomplete tests are retained in the append-only checklist history for correction/retest or documentation as limitations.	RECORDED
Raw measurement data has been preserved.	YES
Final test report has been generated.	YES

Engineering basis for tolerances

The LTC2945 uses a 12-bit VIN converter with 25 mV/LSB and a 102.4 V full-scale. Its guaranteed total unadjusted error is ±0.75% of full-scale, which corresponds to approximately ±0.768 V for VIN. The report uses a tighter characterization/acceptance limit of 2% with a 100 mV floor, as this is a more reasonable limit at the used range.

Rail	Shunt	LTC2945 ADC TUE equivalent	1% shunt at test max	Recommended acceptance
+5VA	39.0 mΩ	19.7 mA	20.0 mA	±max(5% reading, 25 mA)
+5VB	39.0 mΩ	19.7 mA	20.0 mA	±max(5% reading, 25 mA)
+5V	50.0 mΩ	15.4 mA	10.0 mA	±max(5% reading, 20 mA)
-5V	50.0 mΩ	15.4 mA	10.0 mA	±max(5% reading, 20 mA)
+15V	50.0 mΩ	15.4 mA	10.0 mA	±max(5% reading, 20 mA)
-15V	50.0 mΩ	15.4 mA	10.0 mA	±max(5% reading, 20 mA)

The current system-test criterion is ±5% of the reference value. At low current, where a percentage-only limit approaches zero and instrument/ADC resolution becomes significant, the script uses the larger of the 5% window and the configured absolute floor. The floor also covers the LTC2945 delta-sense full-scale TUE, shunt tolerance, and a small reference/rounding margin.

Appendix E

Week 3 Assignment, Part B in re: Child Labor

Scenario

Consider the following scenario. Your design is a high-profile product for your company in the U.S.A. As such, the media makes every effort to monitor the project's progress. Several vendors have been selected for sourcing the parts, all overseas. Things are looking good until your manager comes into your office and tells you that the media has just released a story that the manufacturer of your highest-cost part uses child labor, that is children as young as 14. The legal working age in the manufacturer's country is 14. To complicate matters, to change vendors would result in an additional cost of 15item. You have been invited to a meeting to discuss how to proceed. Using your text from your ethics course (and the internet as well if you wish), prepare a recommendation for the meeting. This recommendation should not be based on opinion but rather on problem-solving approaches when dealing with cross-cultural ethics problems. Would your recommendation be different, and if so, why, if the manufacturer used children as young as 12?

Recommendation

In such a scenario, I would direct the company to investigate the supplier before they jump to any rash decisions. Cross-cultural ethics issues shouldn't be viewed only from the lens of US customs, or only by the legality in the foreign country. The company needs to take all conditions into account – laws, working conditions, safety and education, and the effects that the decision can have on all stakeholders. Workers who are 14 may be ethically defensible if the work the 14 year olds are doing is legal, safe, voluntary, and pay is commensurate with the work, while not interfering with their education. Further, the International Labour Organization does allow workers as young as 14 for some countries, so long as the work is deemed non-hazardous (International Labour Organization, 1973). The company must

verify the allegations and apply reasonable labor standards, rather than changing suppliers at the whim of social or media pressures. The additional 15% cost in changing suppliers is worth considering – but it alone shouldn't be the deciding factor if workers are indeed being exploited or put at risk. If the investigation finds issues such as hazardous working conditions, excessive hours that interfere with education requirements, coercion of workers, or the like, the company should require corrective action and move to another supplier if problems are not resolved. Workers that are as young as 12 would shift my recommendation. Since the scenario states that the working age is 14 in this hypothetical, a child at 12 would be violating the labor laws in the country. If this were discovered, I would demand suspension of new purchases pending investigation and remediation of the issue. Ethically, this would force the use of a new supplier despite the cost difference, if it were found to be true and it weren't resolved.

Appendix F

Design Standards

F.1 Standards and Specifications Used in the Design

Engineering standards were applied in the design of the Multi-Rail Isolated Power Supply Monitoring System. All the standards discussed in this chapter were used as design guidelines and component-selection criteria, and there were no intentions to obtain product certification for any standards mentioned.

F.1.1 IPC-2152 Printed-Circuit Conductor Sizing

IPC-2152, *Standard for Determining Current Carrying Capacity in Printed Board Design*, was used to calculate the ampacity of the PCB tracks and copper pours.[1] The standard provides equations to solve for conductor dimensions, copper thickness, current, and temperature rise. This information can then be used to determine the trace and pour sizing for the load.

This standard is directly applicable to the Sensor Board design since the monitored rails are routed via the PCB and can carry up to 2A on the +5VA and +5VB digital rails. As shown in Appendix A, the estimates provided by the standard were approximately 3.27 mil at 0.5 A, 10.9 mil at 1 A, and 36.4 mil at 2 A for the assumed temperature-rise in 1 oz external copper. Sizes calculated were taken as absolute minimums. 2oz copper was used for both internal and external layers to decrease conductor impedance despite a slight increase in cost.

F.1.2 I^2C Bus Specification

Communication between the LTC2945 PMICs and the Arduino uses an I^2C bus. The bus specification used in the design is the NXP UM10204, *I^2C-bus Specification and User Man-*

[1] IPC, *IPC-2152: Standard for Determining Current Carrying Capacity in Printed Board Design*, 2009.

ual.[2]

The design follows the standard bus topology and timing as described in the standards document. Since the six measurement channels are electrically isolated from the microcontroller, the I^2C signals need to be isolated carefully, so additional capacitance isn't added to the bus. I^2C buffering and isolation components were chosen considering with logic levels, bus capacitance, and direction control.

The firmware also follows the standard I^2C protocol. Using a standardized bus allows commercially available monitors, buffers, isolators, and the Arduino Wire interface to work together using well established components.

F.1.3 IEEE 802.3 Ethernet

The remote-monitoring capability of the system is based on Ethernet connectivity. The IEEE 802.3 standard defines the Ethernet physical and data-link-layer architecture used in commercial Ethernet products.[3]

Using a standardized Ethernet interface allows the monitor to connect to standard networks instead of requiring any proprietary or custom solutions. The Ethernet interface is used for both the PSC connection used by the EPICS IOC and the embedded HTTP configuration interface. The use of standard Ethernet interface drove the choice of the network controller, RJ45 interface, magnetics, and supporting circuits.

F.1.4 Isolation Component Safety Ratings

Electrical isolation was one of the design requirements, since the six monitored power supplies cannot be considered to share a common electrical reference. The ACPL-064L digital optocouplers used for the isolated communication paths are UL 1577 recognized for an isolation voltage of 3750 V_{RMS} for one minute according to the data sheet.[4]

This component's isolation rating exceeds the required isolation (<50V) for the low-voltage monitoring system. System-level isolation also depends on PCB creepage/clearance, connectors, wiring, assembly practices, and the final enclosure.

F.2 Societal Impact

The principal societal impact of the monitoring system is the improved data availability. The six isolated power supplies that would otherwise have to be monitored individually can now be measured from one local display and from the EPICS control system. Abnormal voltage,

[2]NXP Semiconductors, *UM10204: I^2C-bus Specification and User Manual*, Rev. 7.0, 2021.

[3]IEEE, *IEEE Std 802.3-2022: IEEE Standard for Ethernet*, 2022.

[4]Broadcom, *ACPL-064L, ACPL-M61L, ACPL-W61L, ACPL-K64L Low-Power 10-MBd Digital CMOS Optocouplers Data Sheet*.

current, or communication conditions can be detected faster. In a laboratory or industrial setting, this can reduce troubleshooting time and help to avoid operation of the equipment with a power supply failure.

There is also a obligation associated with presenting measurements to an operator. Erroneous data can be more problematic than unavailable data if an operator believes that a failed measurement is a good one. For this reason, communication failure detection was included in the design. The firmware distinguishes an failed or offline sensor from a legitimate zero measurement, and presents the communication state together with the measured values. This was an important design decision since the monitor is meant to present information which an operator can rely on for making maintenance and operational decisions.

F.3 Global Impact

The design is dependent on a global electronics supply chain. ICs, passive components, PCB fabrication, and commercial Arduino modules can be manufactured in several different countries even when they are supplied through United States distributors. This gives access to a wide range of components and lower manufacturing costs, but at the same time creates risks associated with component availability, obsolescence, shipping disruptions, and supplier dependence.

The design mitigates some of these risks through the use of standard component packages, conventional PCB assembly, repetitive circuit blocks, and standard interfaces such as I^2C and Ethernet. makes qualification of replacement components or alternate manufacturing sources much easier than it would be with highly proprietary hardware.

Global sourcing creates an ethical obligation that is not completely covered by component price and technical performance. For a production version of the monitor, the selection of a supplier should also include consideration of responsible labor practices, material sourcing, regulatory compliance, and traceability of the supplier. These considerations considerations would become increasingly important for large scale production of the design.

F.4 Legal and Regulatory Impact

This project is an engineering prototype and can't be considered to have any system-level regulatory approval solely because individual components carry approvals or compliance markings. If the monitor were converted into a commercial product, the required regulatory evaluation would depend on the installation, enclosure, power source, network interfaces, and markets in which it would be sold.

For example, no attention was paid to RoHS compliance, as the boards are intentionally assembled with leaded solder, so leaded HASL or ENIG were both acceptable ordering options. Use of the monitor with leaded solder however, would be an issue for many markets with RoHS mandates.

EMC, product-safety, and radio-frequency requirements would also have to be identified for the final installation before the production product could be offered for sale. EMC lab compliance testing can be extremely expensive.

The distinction between component compliance and system compliance is therefore important. The design can take advantage of the recognized components and accepted engineering practice without implying any approval not obtained for the finished assembly. This can most easily be related to the idea of "UL Listed" and "UL Registered" (UL vs UR labels) - UL listed components are certified on their own standing, but UR components need to be certified in the installation, and are only a recognized component of a system.

F.5 Ethical Impact

The first ethical obligation associated with the monitor is the accurate and unambiguous presentation of the measured information. The system should not hide sensor failures or substitute erroneous data with apparently valid measurements. The implemented communication failure detection, alarm indication, and separate sensor failure reporting were included partly for this reason.

The next consideration is network security. The current HTTP and PSC interfaces are designed for use in a controlled local network. They do not have security features necessary to be connected to an untrusted network. If the design were deployed in an unprotected network, additional requirements for authentication, access control, encryption, network segmentation, and configuration management should be dfined. Any possibility of unauthorized modification of alarm limits or the network configuration would undermine the reliability of the monitoring system, even if the measurement hardware would continue operating properly.

Ethical considerations also also relevant to procurement. Cost and availability are important engineering considerations, but they should not be the only criteria for selecting a production supplier. Labor practices, material sourcing, counterfeit parts risk, and the ability to provide traceable parts are also appropriate considerations for a product moving from prototype quantities into production.

F.6 Environmental Impact

The main environmental impacts of the monitor come from PCB fabrication, semiconductor manufacturing, raw-materials, transportation, electricity consumption, and eventual electronic waste. It's impossible to make a design with zero environmental impact, but some aspects of this design reduce the quantity of hardware that would need to be disposed of during the useful service life of the product.

The Sensor Board, Arduino Shield, Arduino GIGA, display, and other major assemblies are separable. Therefore, a failed subsystem can therefore be repaired or replaced independent of the complete monitoring system. Standard component packages and commercially available

modules were preferred where possible, and this increases the probability of replacement of an individual component. Alarm limits and network configuration are stored in software which allows changing the configuration without producing a new PCB.

For production quantities, RoHS-compliant components should be used where possible. The end-of-life plan should include recovery or recycling of the electronic assemblies. These considerations align with RoHS and WEEE requirements to encourage collection, reuse, recovery, and recycling of electronic equipment.

F.7 Relationship to the Original Requirements

Several of the societal and ethical considerations described above were indirectly represented in the original requirements definition. Electrical isolation was explicitly required to allow the monitoring of one rail would not defeat the isolation of the other. The communication fault detection was required to ensure erroneous data would not be interpreted as a valid electrical measurement. User-configurable alarm limits, local status indication, EPICS monitoring, and persistent configuration storage were also included as functional requirements. These requirements increase the reliability, maintainability, and the quality of the information presented to the operator.

Environmental considerations were introduced later in the design through serviceability, component reuse, and the use of common parts. Regulatory certification, ethical supplier selection, and cybersecurity requirements were not among major items in the original requirements definition. If the prototype were developed into a production product, these aspects should be transformed into explicit requirements with acceptance criteria.

Appendix G

Role of Physics and Other Sciences

To measure the currents of the various supplies, the load current is passed through a shunt resistor for each supply. Ohm's law, $V = IR$, converts the sampled current into a voltage that can be measured by the LTC2945 PMIC. For example, the digital rail's 2 A current flowing through a 39 mΩ shunt produces a 78 mV differential voltage. The resistor also converts part of the electrical energy into heat according to Joule heating, $P = I^2R$. That same 2 A operating point dissipates 0.156 W in the shunt. These two equations create the main current-sensor design tradeoff: a larger resistance produces more measurable voltage and better current resolution, but also produces greater voltage drop and heat. The selected shunt must therefore provide adequate ADC signal while remaining comfortably below its power rating and while causing acceptably small voltage/IR drop to the monitored rail.

Voltage measurement relies on the physics of electric potential difference and on analog-to-digital (ADC) conversion. Each LTC2945 is referenced to its own local power-supply domain via optocouplers, so it can measure the potential of that rail without requiring all six rails to share a common ground. The converter represents an analog input with a finite number of digital states. With the LTC2945's 25 mV-per-count resolution, quantization sets the bar for the lower measurable limit. The current channel similarly resolves 25 μV per count across the shunt. Mathematics is required not only to convert the ADC code into volts and amps but also to determine whether the selected shunt and expected signal range make use of the full ADC span. The calculations in Appendix B explicitly convert between current, shunt voltage, ADC counts, and measurement resolution.

Galvanic isolation is another important physical concept in the project. Two circuits can exchange information without sharing a conductive DC path by using an isolator. In the ACPL-064L, an electrical input drives an LED, which is then detected by a photodiode on the other side, forming an isolated barrier between the two sides of the circuit. The signal crosses the isolation barrier while the two circuit grounds remain electrically independent. This is important because directly connecting the returns of floating supplies can create ground loops or force current through a path that the original power system did not intend. The mathematical treatment of the isolation interface includes logic voltage limits, LED/interface current, and the allowable voltage across the isolation barrier. Although the monitored rails are only in the tens of volts, the isolation components have ratings in excess of 100V,

providing ample headroom.

The digital communication buses also depend on basic circuit behavior. An I^2C bus uses open-drain I/O, meaning that devices pull a line low while a pull-up element or active current source returns it high. Bus capacitance and pull-up strength determine the line's electrical rise time. Pull ups that are too weak, or too much bus capacitance can prevent a logic-high level from being reached quickly enough at the selected clock rate. The TCA9803 buffer used in the design includes an active current source on one side, which eliminates the need to be concerned with setting RC time constants so long as bus capacitance remains below the max for the driver.

Power conversion on the controller hardware also uses electromagnetic energy storage. The TPS62912 is a switching buck regulator: it alternately connects and disconnects the input through switching devices while an inductor and capacitors store and release energy to produce a lower average DC output. The basic relationships among voltage, current, inductance, capacitance, switching frequency, ripple, and load determine output quality. For this project the regulator is required to produce a 3.3 V logic rail from a higher input while supplying the display, Ethernet, and interface electronics. The preliminary load estimate is compared with the regulator's output-current rating so the design has sufficient current and thermal margin rather than selecting the regulator only from its nominal output voltage.

Finally, the project relies on mathematics to turn these physical principles into engineering decisions. Kirchhoff's and Ohm's laws are used to solve the prototype test network; $P = VI$, $P = I^2R$, and related equations are used for component heating and power budgeting; ADC scale factors are used to convert between raw counts and engineering units; inequalities define acceptable alarm regions; and tolerance analysis estimates how resistor error and converter error propagate into the reported current. The firmware then implements these same relationships numerically. In this sense the software is not separate from the physics of the hardware: it is the mathematical model that converts measured electrical quantities into the voltage, current, alarm, and status information presented to the user and to EPICS.

Appendix H

Employer Letter

Employer Acknowledgement Letter

Signed letter from supervisor
To be inserted once he signs...

Appendix I

Updated Component List

The Week 2 component list is intentionally presented in the three-column format requested by the assignment. Quantities represent the present design quantity for one complete monitoring-system hardware set and exclude optional/DNP parts and purchasing spares. Detailed manufacturer and distributor information remains in the KiCad bills of materials.

Table I.1: Updated preliminary component list.

Item	Part Description	Qty.
1	Custom isolated multi-rail sensor PCB	1
2	Custom Arduino GIGA interface/shield PCB	1
3	Arduino GIGA R1 WiFi controller, ABX00063	1
4	Arduino GIGA Display Shield, ASX00039	1
5	LTC2945IUD-1 voltage/current monitor	6
6	ACPL-064L dual-channel digital optocoupler	12
7	TCA9803 I^2C bus buffer/repeater	2
8	P82B715 I^2C bus extender	2
9	TPS62912 low-noise 2 A buck regulator	1
10	W6100-Q Ethernet controller	1
11	SI-46014-F RJ45 Ethernet connector/magnetics assembly	1
12	NTA7002NT1G N-channel MOSFET	6
13	39 mΩ, 1% four-terminal current-sense resistor	2
14	50 mΩ, 1% four-terminal current-sense resistor	4
15	2 kΩ resistor	18
16	4.7 kΩ resistor, populated positions only	8
17	14.7 kΩ resistor	1

Item	Part Description	Qty.
18	12.3 kΩ resistor	1
19	10 kΩ resistor	5
20	49.9 Ω resistor	4
21	3.3 Ω resistor	4
22	470 Ω resistor	2
23	1 MΩ resistor	1
24	0 Ω jumper resistor / populated zero-ohm link	4
25	100 nF ceramic capacitor	37
26	1 μF ceramic capacitor	12
27	10 μF ceramic capacitor	2
28	22 μF ceramic capacitor	5
29	470 nF ceramic capacitor	1
30	22 nF ceramic capacitor	1
31	3.3 μF capacitor	1
32	1 nF capacitor	1
33	18 pF capacitor	2
34	2.2 μH power inductor	1
35	120 Ω ferrite bead	3
36	Sensor-board EMI/filter bead (BLE18PS080SN1D)	1
37	25 MHz crystal, FL2500169	1
38	Molex 0039299049 connector/header positions	5
39	Molex 0039289088 connector/header	2
40	Molex 0039289108 connector/header	2
41	Arduino shield connector set	1
42	Keystone 5000 test point	3
43	Keystone 5001 test point	2
44	Keystone 5015 test point	9

This list is expected to change slightly after first-article assembly and final verification. In particular, optional pull-up resistors and other DNP footprints may be populated temporarily for troubleshooting, and component values may be adjusted if testing shows that additional margin or filtering is required.

I.1 Bill of Materials

The current bills of materials for the Sensor Board and Arduino Shield Board are maintained with their respective KiCad hardware-design files in the project `hardware` directory. They are included below for reference.

The BOMs below were updated udring the DDR to add extended BOM costs for quantity ranges 1, 100, and 1000 boards.
Price breaks were set for using the required quantity of components at production quantities, going to re-spooled or full reels, so some components may have the $7 reeling fees added.

I.1.1 Sensor Board Bill of Materials

I.1.1.1 Preliminary

BOM: Power Sensor Board | Production | 1 Tape and Reel/Reeling Fees Allc:No | Add 10% spares for components excluding modules

Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price
C1,C3,C21,C23,C26,C28,C31,1u			CL10A105KB8NNNC	Samsung Electro-Mechanics	1276-1860-6-ND	Digikey	12	14	100	$3.77
C2,C4,C5,C11,C12,C14,C15,C100n			GRM155R61H104KE14D	Murata Electronics	490-10697-6-ND	Digikey	23	26	100	$1.41
C6	22n		GRM155R71H222KA01D	Murata Electronics	490-1305-6-ND	Digikey	1	2	100	$1.01
C7,C8	10u		C2012X7S1E106K125AC	TDK	445-181600-6-ND	Digikey	2	3	10	$3.58
C9	470n		C1005X5R1E474K050BB	TDK	445-8020-6-ND	Digikey	1	2	100	$3.17
C10,C13,C16,C19,C20	22u		GMC21X7S226M10NT	Cal-Chip Electronics, Inc.	4713-GMC21X7S226M10NTDKR-ND	Digikey	5	6	15	$11.30
C18	~	DNP					1	2		
FB1	10n		BLE18PS080SN1D	Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digikey	1	2	10	$2.30
H1,H2,H3,H4	MountingHole_Pad						4	5		
J101	+6VA		0039299049	Molex	WM14518-ND	Digikey	1	2	1	$2.70
J102	+6VB		0039299049	Molex	WM14518-ND	Digikey	1	2	1	$2.70
J103	0039289088		0039289088	Molex	WM3835-ND	Digikey	1	2	1	$4.58
J104	+/-15V		0039299049	Molex	WM14518-ND	Digikey	1	2	1	$2.70
J105	+/-5V		0039299049	Molex	WM14518-ND	Digikey	1	2	1	$2.70
J106,J107		39289108	0039289108	Molex	23-0039289108-ND	Digikey	2	3	1	$5.02
J108	Conn_01x04		0039299049	Molex	WM14518-ND	Digikey	1	2	1	$2.70
L1	2u2		74438356022	Wurth Elektronik	732-8232-6-ND	Digikey	1	2	3	$5.82
Q1,Q2,Q3,Q4,Q5,Q6	BS170F		NTA7002NT1G	onsemi	NTA7002NT1GOSDKR-ND	Digikey	6	7	25	$4.43
R2,R3,R4,R5,R14,R15,R16,R170R		DNP		5108 Keystone Electronics	36-5108DKR-ND	Digikey	8	9		
R6,R12,R13,R33,R34,R35,R36,2k			ERJ-2RKF2001X	Panasonic Industry	P2.00KLDKR-ND	Digikey	18	20	100	$1.64
R7,R18	39mR		LVK20R039FER	Ohmite	273-LVK20R039FERDKR-ND	Ohmite	2	3	5	$9.45
R8,R11,R19,R20	50mR		LVK20R050FER	Ohmite	273-LVK20R050FERDKR-ND	Ohmite	4	5	5	$9.45
R9,R10	0R			5108 Keystone Electronics	36-5108DKR-ND	Digikey	2	3	10	$2.01
R21	0R		CRCW04020000Z0ED	Vishay Dale	541-0.0JDKR-ND	Digikey	1	2	100	$0.99
R22	~	DNP					1	2		
R23,R24	4k7		CRCW04024K70FKED	Panasonic Industry	541-4.70KLCT-ND	Digikey	2	3	100	$1.57
R25	14k7		ERA-2APB1472X	Panasonic Industry	P17731CT-ND	Digikey	1	2	10	$4.52
R26	4k7		ERA-2ARB472X	Panasonic Industry	P4.7KQPCT-ND	Digikey	1	2	10	$4.99
R27,R28,R29,R30,R31,R32	4k7	DNP	CRCW04024K70FKED	Vishay Dale	541-4.70KLCT-ND	Digikey	6	7		
TP1,TP3		5000		5000 Keystone	36-5000-ND	Digikey	2	3	10	$2.04
TP2		5001		5001 Keystone	36-5001-ND	Digikey	1	2	10	$2.03
TP4,TP5,TP6,TP7,TP8,TP9		5015		5015 Keystone	36-5015CT-ND	Digikey	6	7	25	$8.29
U1,U8,U11,U14,U17,U20	LTC2945IUD-1		LTC2945IUD#TRPBF	Analog Devices Inc.	LTC2945IUD#TRPBFDKR-ND	Digikey	6	7	10	$110.97
U2,U3,U9,U10,U12,U13,U15,U	ACPL-064L-500E		ACPL-064L-500E	Broadcom Limited	516-2614-6-ND	Digikey	12	14	15	$56.27
U4	TPS62912		TPS62912RPUR	Texas Instruments	595-TPS62912RPUR	Mouser	1	2	3	$7.77
U5,U6	TCA9803		TCA9803DGKR	Texas Instruments	296-47829-6-ND	Digikey	2	3	10	$6.84
U7	P82B715DR		P82B715DR	Texas Instruments	296-24729-6-ND	Digikey	1	2	5	$13.55
									TOTAL COST:	$302.26

I.1.1.2 Final

Table I.2: Final Sensor Board Bill of Materials and Production Cost Analysis

Reference	Qty./System	Specification	Manufacturer	Mfr. P/N	Pricing Source	1 System	100 Systems	1000 Systems
C1,C3,C21,C23,C26,C28,C31, C33,C36,C38,C41,C43	12	1 µF ±10%, 50 V, X5R, 0603	Samsung Electro-Mechanics	CL10A105KB8NNNC	DigiKey	$1.12	$54.73	$374.72
C2,C4,C5,C11,C12,C14,C15, C17,C22,C24,C25,C27,C29, C30,C32,C34,C35,C37,C39, C40,C42,C44,C45	23	100 nF ±10%, 50 V, X5R, 0402	Murata Electronics	GRM155R61H104KE14D	Mouser	$0.44	$34.83	$180.00
C6	1	2.2 nF ±10%, 50 V, X7R, 0402	Murata Electronics	GRM155R71H222KA01D	DigiKey	$0.22	$8.20	$15.25
C7,C8	2	10 µF ±10%, 25 V, X7S, 0805	TDK	C2012X7S1E106K125AC	DigiKey	$1.77	$61.21	$446.16
C9	1	470 nF ±10%, 25 V, X5R, 0402	TDK	C1005X5R1E474K050BB	DigiKey	$0.20	$10.40	$28.95
C10,C13,C16,C19,C20	5	22 µF ±20%, 10 V, X7S, 0805	Cal-Chip Electronics	GMC21X7S226M10NT	DigiKey	$4.52	$421.15	$4,148.50
FB1	1	Ferrite bead, 8.5 Ω @ 100 MHz, 8 A, 4 mΩ max DCR, 0603	Murata Electronics	BLE18PS080SN1D	Mouser	$0.62	$23.50	$89.50
J101,J102,J104,J105,J108	5	4-position Mini-Fit Jr. vertical header, 4.2 mm	Molex	0039299049	DigiKey	$15.78	$931.41	$7,941.18
J103	1	8-position Mini-Fit Jr. vertical header, 4.2 mm	Molex	0039289088	DigiKey	$9.16	$364.00	$3,249.31
J106,J107	2	10-position Mini-Fit Jr. vertical header, 4.2 mm	Molex	0039289108	DigiKey	$14.94	$790.75	$6,722.03
L1	1	2.2 µH ±20%, 4.7 A rated, 6.2 A Isat	Würth Elektronik	74438356022	DigiKey	$3.88	$190.92	$1,702.10
Q1,Q2,Q3,Q4,Q5,Q6	6	N-MOSFET, 30 V, 154 mA, 300 mW, SC-75	onsemi	NTA7002NT1G	Mouser	$2.03	$60.46	$432.00
R6,R12,R13,R33,R34,R35, R36,R37,R38,R39,R40,R41, R42,R43,R44,R45,R46,R47	18	2.00 kΩ ±1%, 0.10 W, 0402	Panasonic Industry	ERJ-2RKF2001X	DigiKey	$0.62	$24.19	$103.80
R7,R18	2	39 mΩ ±1%, 0.75 W, Kelvin 2010	Ohmite	LVK20R039FER	DigiKey	$4.56	$176.40	$1,503.00
R8,R11,R19,R20	4	50 mΩ ±1%, 0.75 W, Kelvin 2010	Ohmite	LVK20R050FER	Newark	$9.85	$376.64	$2,140.00
R9,R10	2	0 Ω jumper, 1206	Keystone Electronics	5108	DigiKey	$0.81	$51.00	$447.00
R21	1	0 Ω jumper, 1/16 W, 0402	Vishay Dale	CRCW04020000Z0ED	DigiKey	$0.20	$8.09	$14.18
R23,R24	2	4.70 kΩ ±1%, 1/16 W, 0402	Vishay Dale	CRCW04024K70FKED	DigiKey	$0.30	$10.45	$26.40
R25	1	14.7 kΩ ±0.1%, 1/16 W, 0402	Panasonic Industry	ERA-2APB1472X	DigiKey	$0.96	$43.31	$312.74
R26	1	4.70 kΩ ±0.1%, 1/16 W, 0402	Panasonic Industry	ERA-2ARB472X	DigiKey	$1.20	$52.45	$409.25
TP1,TP3	2	Red through-hole test point	Keystone Electronics	5000	DigiKey	$0.81	$41.80	$418.00
TP2	1	Black through-hole test point	Keystone Electronics	5001	DigiKey	$0.54	$20.90	$209.00
TP4,TP5,TP6,TP7,TP8,TP9	6	SMT miniature test point	Keystone Electronics	5015	DigiKey	$3.43	$160.94	$1,335.67
U1,U8,U11,U14,U17,U20	6	Power/current monitor, 4–80 V input, 12-QFN	Analog Devices	LTC2945IUD#TRPBF	DigiKey	$98.56	$6,287.96	$62,816.56
U2,U3,U9,U10,U12,U13,U15, U16,U18,U19,U21,U22	12	Dual digital optoisolator, 3.75 kVrms, 10 MBd, 8-SO	Broadcom	ACPL-064L-500E	DigiKey	$52.51	$3,506.85	$34,532.19
U4	1	2 A adjustable synchronous buck, 3–17 V input	Texas Instruments	TPS62912RPUR	DigiKey	$5.06	$176.32	$1,565.46
U5,U6	2	Dual-channel I^2C level translator/buffer, 8-VSSOP	Texas Instruments	TCA9803DGKR	DigiKey	$2.85	$126.53	$1,077.39
U7	1	I^2C bus buffer/re-driver, 4.5–12 V, 8-SOIC	Texas Instruments	P82B715DR	DigiKey	$5.42	$188.32	$1,582.29
					Component subtotal	**$242.36**	**$14,203.71**	**$133,822.63**
PCB	1	Custom fabricated bare Sensor Board PCB; 5-board MOQ for prototype quantity	—	Custom		$473.57	$1,288.53	$6,825.82
					Sensor Board total including PCB	**$715.93**	**$15,492.24**	**$140,648.45**

Pricing note: Costs are extended purchase costs for each production scale. A 10% spare allowance is included for non-module components; Arduino modules are purchased at exact system quantity. For 100- and 1000-system production, custom reels/re-reels and full manufacturer reels were compared lower line purchase cost was used. Bare PCB fabrication is included from the provided quote. Shipping, tax, tariffs, PCB assembly, enclosure, cables, and other non-PCB system hardware are excluded.

I.1.2 Arduino Shield Board Bill of Materials

I.1.2.1 Preliminary

BOM: Arduino Shield Board — Production — 1 Tape and Reel/Reeling Fees Alic No — Add 10% spares for components excluding modules

Reference	Value	DNP	MPN	Manufacturer	DPN	Distributor	Qty	Qty+10%	Order Qty	Extended Price
A201	Arduino_Giga_R1_WiFi		ABX00063	Arduino	1050-ABX00063-ND	Digikey	1	2	1	$69.60
C6	22n	DNP	GRM155R71H222KA01D	Murata Electronics	490-1305-6-ND	Digikey	1	2	0	
C7,C8	10u	DNP	C2012X7S1E106K125AC	TDK	445-181600-6-ND	Digikey	2	3	0	
C9	470n	DNP	C1005X5R1E474K050BB		445-8020-6-ND	Digikey	1	2	0	
C10,C13,C16,C19,C20	22u	DNP	GMC21X7S226M10NT	CalChip Electronics, Inc.	4713-GMC21X7S226M10NTDKR-ND	Digikey	5	6	0	
C17,C201,C202,C203,C211, C215,C216,C217,C220,C221 ,C223,C224,C225,C227	100n		GRM155R61H104KE14D	Murata Electronics	490-10697-6-ND	Digikey	14	16	100	$1.41
C218	3u3					Digikey	1	2		
C219	1n		1206GC102KAT1A	KYOCERA AVX	478-2956-6-ND	Digikey	1	2	10	$1.43
C222,C226	18p		GRM1555C1H180JA01D	Murata Electronics	490-5858-6-ND	Digikey	2	3	100	$1.43
FB1	10n	DNP	BLE18PS080SN1D	Murata Electronics	490-BLE18PS080SN1DDKR-ND	Digikey	1	2		
FB201,FB202,FB203	120R		BBUP0160805121Y00	Pulse Electronics	553-BBUP0160805121Y00CT-ND	Digikey	3	4	25	$1.97
J201	SI-46014-F		SI-46014-F	Bel Fuse Inc.	5923-SI-46014-F-ND	Digikey	1	2	1	$5.81
J309	0039289088		0039289088	Molex	WM3835-ND	Digikey	1	2	1	$4.58
L1	2u2	DNP	744383562022	Wurth Elektronik	732-8232-6-ND	Digikey	1	2	3	$5.82
N101	Giga_Display		ASX00039	Arduino	1050-ASX00039-ND	Digikey	1	2	1	$63.60
N201	Connector_Set			85 Adafruit Industries	1528-1074-ND	Digikey	1	2	2	$3.90
R21	0R	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digikey	1	2		
R23,R24,R205,R206,R207	4k7		CRCW04024K70FKED	Vishay Dale	541-4.70KLCT-ND	Digikey	5	6	100	$1.57
R25	15k2	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digikey	1	2		
R26	4k87	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digikey	1	2		
R31,R32	4k7	DNP	ERJ-2RKF4701X	Panasonic Industry	P4.70KLDKR-ND	Digikey	2	3		
R101,R102,R103,R104,R105	10k					Digikey	5	6		
R208	12k3		RN73R1ETTP1232B25	KOA Speer Electronics, Inc.	2019-RN73R1ETTP1232B25DKR-ND	Digikey	1	2	50	$3.02
R209,R210,R215,R216	49R9		CRCW040249R9FKED	Vishay Dale	541-49.9LDKR-ND	Digikey	4	5	100	$1.57
R211,R212,R213,R214	3R3		CRCW04023R30FKED	Vishay Dale	541-3.30LDKR-ND	Digikey	4	5	100	$2.03
R217	1M		CRCW04021M00FKED	Vishay Dale	541-1.00MLDKR-ND	Digikey	1	2	100	$1.57
R218,R219	470R		ERJ-2RKF4700X	Panasonic Industry	P470LDKR-ND	Digikey	2	3	100	$1.64
R220	0R		CRCW04020000Z0ED	Vishay Dale	541-0.0JDKR-ND	Digikey	1	2	100	$0.99
TP1,TP3		2 DNP		5000 Keystone	36-5000-ND	Digikey	2	3		
TP2		1 DNP		5001 Keystone	36-5001-ND	Digikey	1	2		
TP101		1		5000 Keystone	36-5000-ND	Digikey	1	2	10	$2.04
TP102		1		5001 Keystone	36-5001-ND	Digikey	1	2	10	$2.03
TP201,TP202,TP203		3		5015 Keystone	36-5015CT-ND	Digikey	3	4	10	$3.70
U4	TPS62912	DNP					1	2		
U7	P82715DR		P82B715DR		296-24729-6-ND		1	2	2	$5.42
U202	W6100-Q		W6100Q	WIZnet	950-W6100Q	WIZnet	1	2	2	$5.04
Y201	25MHz		FL2500169	Diodes Incorporated	31-FL2500169DKR-ND	Digikey	1	2	2	$1.38
									TOTAL COST:	$191.55

I.1.2.2 Final

Table I.3: Final Arduino Shield Board Bill of Materials and Production Cost Analysis

Reference	Qty./System	Specification	Manufacturer	Mfr. P/N	Pricing Source	1 System	100 Systems	1000 Systems
A201	1	Arduino GIGA R1 WiFi module	Arduino	ABX00063	DigiKey	$69.60	$6,229.02	$62,290.20
C17,C201,C202,C203,C211, C215,C216,C217,C220,C221, C223,C224,C225,C227	14	100 nF ±10%, 50 V, X5R, 0402	Murata Electronics	GRM155R61H104KE14D	Mouser	$0.27	$25.48	$120.00
C218	1	3.3 µF ±10%, 25 V, X5R, 0603	Cal-Chip Electronics, Inc.	GMC10X5R335K25NT	DigiKey	$0.92	$26.95	$166.17
C219	1	1 nF ±10%, 2 kV, X7R, 1206	KYOCERA AVX	1206GC102KAT1A	DigiKey	$0.60	$19.19	$101.12
C222,C226	2	18 pF ±5%, 50 V, C0G/NP0, 0402	Murata Electronics	GRM1555C1H180JA01D	DigiKey	$0.30	$10.21	$29.53
FB201,FB202,FB203	3	Ferrite bead, 120 Ω @ 100 MHz, 3 A, 0603	Pulse Electronics	BBUP00160805121Y00	DigiKey	$0.52	$24.98	$147.91
J201	1	RJ45 MagJack, 10/100Base-T, through-hole	Bel Fuse	SI-46014-F	DigiKey	$11.62	$469.26	$4,251.93
J309	1	8-position Mini-Fit Jr. vertical header, 4.2 mm	Molex	0039289088	DigiKey	$9.16	$364.00	$3,249.31
N101	1	Arduino GIGA Display Shield module	Arduino	ASX00039	DigiKey	$63.60	$5,692.01	$56,920.10
N201	1	Arduino stacking header set	Adafruit Industries	85	DigiKey	$3.90	$214.50	$2,145.00
R23,R24,R205,R206,R207	5	4.70 kΩ ±1%, 1/16 W, 0402	Vishay Dale	CRCW04024K70FKED	DigiKey	$0.60	$12.69	$42.31
R101,R102,R103,R104,R105	5	10 kΩ ±1%, 0.10 W, 50 V, 0402	Panasonic Industry	ERJ-2RKF1002X	DigiKey / Newark	$0.31	$16.90	$41.71
R208	1	12.3 kΩ ±0.1%, 1/16 W, 0402	KOA Speer	RN73R1ETTP1232B25	DigiKey	$0.20	$13.28	$59.25
R209,R210,R215,R216	4	49.9 Ω ±1%, 1/16 W, 0402	Vishay Dale	CRCW040249R9FKED	DigiKey	$0.50	$12.40	$45.81
R211,R212,R213,R214	4	3.30 Ω ±1%, 1/16 W, 0402	Vishay Dale	CRCW04023R30FKED	DigiKey	$0.50	$14.08	$58.96
R217	1	1.00 MΩ ±1%, 1/16 W, 0402	Vishay Dale	CRCW04021M00FKED	DigiKey	$0.20	$8.73	$16.70
R218,R219	2	470 Ω ±1%, 0.10 W, 0402	Panasonic Industry	ERJ-2RKF4700X	DigiKey	$0.30	$10.41	$26.10
R220	1	0 Ω jumper, 1/16 W, 0402	Vishay Dale	CRCW04020000Z0ED	DigiKey	$0.20	$8.09	$14.18
TP101	1	Red through-hole test point	Keystone Electronics	5000	DigiKey	$0.54	$20.90	$209.00
TP102	1	Black through-hole test point	Keystone Electronics	5001	DigiKey	$0.54	$20.90	$209.00
TP201,TP202,TP203	3	SMT miniature test point	Keystone Electronics	5015	DigiKey	$1.96	$99.70	$765.24
U7	1	I^2C bus buffer/re-driver, 4.5–12 V, 8-SOIC	Texas Instruments	P82B715DR	DigiKey	$5.42	$188.32	$1,582.29
U202	1	Hardwired TCP/IP Ethernet controller, QFN	WIZnet	W6100Q	Mouser	$7.96	$280.90	$2,416.00
Y201	1	25 MHz crystal, ±10 ppm, 18 pF load, 3.2×2.5 mm	Diodes Incorporated	FL2500169	DigiKey	$1.18	$55.81	$431.12
					Component subtotal	**$180.90**	**$13,838.70**	**$135,338.93**
PCB	1	Custom fabricated bare Arduino Shield PCB; 5-board MOQ for prototype quantity	—	Custom	Vendor quote	$122.19	$250.77	$1,402.23
					Arduino Shield total including PCB	**$303.09**	**$14,089.47**	**$136,741.16**

PCB pricing note: Arduino Shield PCB fabrication is quoted at $122.19 for the 5-board minimum order, $250.77 for 100 boards, and $1,402.23 for 1000 boards. For the single-system scenario, the full 5-board minimum-order cost is included even though only one PCB is required.

Component pricing note: Costs are extended purchase costs for each production scale. A 10% spare allowance is included for non-module components, while the Arduino GIGA R1 WiFi and GIGA Display modules are purchased at the exact required system quantity. For the 100- and 1000-system production cases, cut-tape pricing, distributor re-reel services, and full manufacturer reels were compared where applicable, and the lowest practical purchase cost was used. Shipping, tax, tariffs, PCB assembly, enclosure, cables, and other non-PCB system hardware are excluded.

I.1.3 System Costs

Table I.4: System Cost versus Production Quantity

Qty.	Sensor Components	Sensor PCB	Shield Components	Shield PCB	Total Purchase Cost	Cost/System
1	$242.36	$473.57	$180.90	$122.19	$1,019.03	$1,019.03
100	$14,203.71	$1,288.53	$13,838.70	$250.77	$29,581.71	$295.82
1,000	$133,822.63	$6,825.82	$135,338.93	$1,402.23	$277,389.61	$277.39

PCB pricing note: The Sensor Board is quoted at $473.57 for the 5-board minimum order, $1,288.53 for 100 boards, and $6,825.82 for 1,000 boards. The Arduino Shield Board is quoted at $122.19 for the 5-board minimum order, $250.77 for 100 boards, and $1,402.23 for 1,000 boards. The 1-system scenario therefore includes the full 5-board MOQ purchase cost for each PCB type.
Component pricing note: Shipping, tax, tariffs, PCB assembly, enclosure, cables, and other non-PCB system hardware are not included.

Appendix J

Installing EPICS Base and IOCs

J.1 Introduction

A simple IOC was written to interface the Arduino Giga which monitors the six supplies. The following section details how to install EPICS Base and required modules, and compile the IOC. It is written assuming a Linux environment; this particular build was done on Ubuntu 22.04LTS.

J.2 EPICS Installation Overview

EPICS installations performed by M. Capotosto loosely follow the below directory structure:

- `EPICS_BASE=/epics/base` or `/epics/base_[version]` or similar
- `SUPPORT=/epics/support` – or sometimes MODULES instead of SUPPORT is used.

  Modules required for the PSU monitor are:
  – pscdrv: Portable Streaming Controller Driver
- `epics/iocs` - each IOC is cloned into this repository
- `epics/systemd-softioc` - This is a utility for managing IOCs as a system service

J.3 Installing EPICS Base and Required Modules

1. Install Dependencies for Build Install git, build-essential, and perl:
   `sudo apt -y install git build-essential perl libtirpc-dev re2c libevent-dev autoconf automake libtool pkg-config`

2. Create the EPICS user group, and the empty EPICS directory:
```
sudo groupadd epics
sudo usermod -aG epics $USER
newgrp epics
sudo mkdir -p /epics
sudo chown $USER:epics /epics
sudo chmod 2775 /epics
```

3. EPICS Base is cloned into `/epics/base` via:
```
git clone -recursive -b 7.0
   https://github.com/epics-base/epics-base.git /epics/base
cd /epics/base
```

4. Build EPICS:
```
make clean
make
```

5. Create the remaining directory structure:
```
mkdir -p /epics/iocs /epics/modules
```

6. Clone PSC Driver:
   (a) `cd /epics/modules`
   (b) `git clone https://github.com/mdavidsaver/pscdrv.git pscdrv`
   (c) `cd /epics/modules/pscdrv`
   (d) Set up required files in `configure/RELEASE, configure/RELEASE.LOCAL`
   (e) Build with `make clean`, and `make`

7. Clone Manage IOCs Utility:
   (a) `cd /epics`
   (b) `git clone https://github.com/NSLS2/systemd-softioc.git`
   (c) Navigate to the repo directory `cd systemd-softioc`
   (d) Make the install script executable and run:
```
sudo chmod +x install.sh
sudo ./install.sh
sudo usermod -aG epics softioc
newgrp
sudo chown softioc:epics /epics/iocs
sudo chmod 2775 /epics/iocs
```

J.4 Installing the PSU Monitor IOC

1. Change to IOC directory `cd /epics/iocs`
2. Clone the repo:
   `git clone https://github.com/capotosto/PS_Monitor psuMonitorIOC`
3. `cd psuMonitorIOC`
4. Create the `RELEASE.local` file, `touch configure/RELEASE.local`
5. ```
   echo -e "EPICS_BASE = /epics/base\n
       PSCDRV = /epics/modules/pscdrv\n
   ```
6. Build the IOC: `make clean`, `make`
7. Update the st.cmd file as needed: `cd iocBoot/iocpsuMonitor`
8. Create the st.cmd file used by systemd-softioc:

   ```
   echo -e '#!/bin/bash\n\n
     cd "/epics/iocs/psuMonitorIOC/iocBoot/iocpsuMonitor"\n./st.cmd' > st.cmd
   ```
9. Make the new st.cmd executable: `chmod +x st.cmd`
10. Find the next available port: `manage-iocs nextport`
11. Configure this in the config file: `nano config`
12. Set the `NAME=`, `PORT=` your next port, `USER=softioc`, and `HOST=$HOSTNAME`
13. Update the directory path in the root-level st.cmd to the `iocBoot/iocpsuMonitor/st.cmd`
14. Enroll in manage-iocs as a service: `sudo manage-iocs install psuMonitorIOC`
15. Enable auto-start on boot: `sudo manage-iocs enable psuMonitorIOC`
16. Check the status, it should be running! `manage-iocs status`
17. And if you want other information on the ioc: `manage-iocs report`
   ```

Appendix K

CS-Studio - Phoebus

K.1 Introduction

CS-Studio Phoebus is installed using precompiled binaries. This application requires a Java runtime environment be installed.

The 'BOB' files - the user interface panels, are located in the IOC's directory, `/epics/iocs/PS_Monitor/opi`

Behavior of some features of Phoebus must be modified by the shell script used to launch Phoebus, an example is included with the below installation instructions to modify the behavior of the 'Home' button, used to bring up the custom 'psuMonitor.bob' panel using the Home button, instead of having to manually navigate to the file when needed.

K.2 Installing Phoebus

1. If not installed, install Java, e.g.:
   ```
   sudo apt update && sudo apt install openjdk-17-jdk
   ```
2. Create the directory: `sudo mkdir /opt/phoebus`
   ```
   sudo chown $USER:epics /opt/phoebus
   chmod 2775 /opt/phoebus
   ```
3. Download Phoebus and extract to the `/opt/phoebus` directory
   https://github.com/ControlSystemStudio/phoebus/releases/download/v4.7.4/phoebus-4.7.4-linux.tar.gz

   ```
   cd /opt/phoebus
   ```

```
wget https://github.com/ControlSystemStudio/phoebus/releases/download/
v4.7.4/phoebus-4.7.4-linux.tar.gz

tar -xvzf phoebus-4.7.4-linux.tar.gz -C . -strip-components=1
```

4. Make the shell script executable: `chmod +x phoebus.sh`

5. Alias Phoebus to run via command `run-phoebus`:
   ```
   echo 'alias run-phoebus="/opt/phoebus/phoebus.sh"' » ~/.bashrc
   source ~/.bashrc
   ```

6. Now test! Try run-phoebus from a terminal.